UNIVERSIDAD AUTÓNOMA GABRIEL RENÉ MORENO

FACULTAD DE INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN Y TELECOMUNICACIONE

INGENIERÍA INFORMÁTICA - INGENIERÍA EN SISTEMAS

SISTEMA DE INFORMACION WEB Y MOVIL OMNICANAL PARA TIENDA VIRTUAL DE MODA FEMENINA

Grupo: 1

Integrantes:

Sánchez Sandy Andres Étienne	220154759
Saavedra Alberto Diego Joaquin	221180702

23 de septiembre de 2026

Índice

1. Perfil del proyecto 1
1.1 Introducción 1
1.2 Objetivo general 2
1.3 Objetivos específicos 2
1.4 Descripción del problema 3
1.5 Alcance del incremento 4
1.6 Características del sistema 5
1.7 Stack tecnológico 5
1.8 Situaciones de uso del incremento 6
1.9 Consideraciones 7
1.10 Evaluación de la arquitectura heredada y estrategia de evolución 7
1.10.1 Alternativas consideradas 7
1.10.2 Evaluación por capa 8
1.10.3 Camino elegido 8
2. Marco teórico del primer ciclo 10
2.1 Marco teórico referencial 10
2.1.1 Antecedentes internacionales 10
2.1.2 Antecedentes nacionales 10
2.1.3 Justificación del proyecto 11
2.1.4 Arquitectura cliente-servidor 11
2.1.5 Patrón de arquitectura 12
2.1.6 Metodología de desarrollo de software 12
2.1.7 Tecnologías utilizadas 13
2.1.8 Base de datos relacional 13
2.1.9 Lenguaje TypeScript y framework NestJS 13
2.2 Datos 13
2.3 Procesos 14
3. Desarrollo de software bajo el modelo SaaS 15
3.1 Definición y características esenciales 15
3.2 SaaS frente a otros modelos de entrega 16
3.3 Variantes del modelo SaaS 18
3.3.1 SaaS horizontal y vertical 18
3.3.2 Single-tenant y multi-tenant 18

3.3.3 Marca blanca (white-label) 19
3.3.4 Plano de control y plano de aplicación 19
3.4 Modelos de cobro 20
3.5 Ciclo de vida del cliente 22
3.6 Métricas de negocio 25
3.7 Identidad cuando un usuario tiene varias tiendas 25
3.8 Ventajas, riesgos y responsabilidades del proveedor 26
3.9 Aplicación en EnsayAR 27
3.9.1 Posicionamiento y estructura 27
3.9.2 Planes y precios 28
3.9.3 Recorrido de autoservicio 29
3.9.4 Pago, idempotencia y compensación 30
3.9.5 Aprovisionamiento automático y límites 31
3.9.6 Renovación, cambio de plan y vencimiento 31
3.9.7 Identidad: una cuenta, varias tiendas 32
3.9.8 Métricas de la plataforma 33
3.9.9 Verificación automatizada 34
3.9.10 Síntesis de decisiones 34
3.9.11 Limitaciones reconocidas 35
4. Organización de inquilinos (multi-tenant) 36
4.1 Qué es un inquilino 36
4.2 El aislamiento como espectro 37
4.3 Modelos de aislamiento de datos 37
4.3.1 Base de datos por inquilino 37
4.3.2 Esquema por inquilino 38
4.3.3 Base compartida con identificador de inquilino 38
4.3.4 Terminología de AWS y Azure 39
4.3.5 Comparación 40
4.4 Resolución del inquilino en cada petición 40
4.5 Propagación del contexto en Node.js 41
4.6 Capas de defensa 43
4.6.1 Filtro centralizado en la aplicación 43
4.6.2 Valores por defecto y restricciones en la base 43
4.6.3 Row-Level Security de PostgreSQL 44

4.7 Resolución paso a paso en NestJS con Prisma 45
4.8 SQL crudo con set_config 48
4.9 Pruebas de aislamiento 49
4.10 Evolución de la estrategia 50
4.11 Estrategia elegida en EnsayAR y justificación 51
4.11.1 Decisión 51
4.11.2 Las capas en EnsayAR 52
4.11.3 Justificación 52
4.11.4 Síntesis de la solución en NestJS 53
4.11.5 Limitaciones reconocidas 54
4.12 Evolución implementada: un esquema PostgreSQL por tienda 54
4.12.1 Motivación 54
4.12.2 Clasificación de los datos 55
4.12.3 Componentes 55
4.12.4 Verificación 56
4.12.5 Alcance y límites 56
5. Pagos para países de habla hispana 57
5.1 Introducción y alcance 57
5.2 Conceptos fundamentales 57
5.2.1 Participantes de un pago electrónico 57
5.2.2 Ciclo de vida de una transacción 58
5.2.3 Otros medios de pago 58
5.3 Medios de pago en los países de habla hispana 59
5.4 Modelos de integración con una pasarela 60
5.5 Estudio de costos 61
5.5.1 Componentes del costo 61
5.5.2 Tarifas publicadas 61
5.5.3 Comparación aplicada a EnsayAR 62
5.5.4 Recomendación por país 63
5.6 Restricciones locales: Bolivia y PayPal 63
5.7 Requisitos para operar con pagos reales 64
5.8 Seguridad y cumplimiento 64
5.9 Diseño en EnsayAR 65
5.9.1 Catálogo de métodos por país 65

5.9.2 Puerto de pasarela y adaptador simulado ... 66
5.9.3 Atomicidad del alta ... 66
5.9.4 Cobro de ventas ... 67
5.10 Estado de implementación ... 67
6. Operación sin conexión, PWA y sincronización ... 68
6.1 Offline-first y modelos de consistencia ... 68
6.1.1 Concepto de offline-first ... 68
6.1.2 Consistencia fuerte y consistencia eventual ... 69
6.1.3 Teorema CAP aplicado ... 69
6.1.4 PACELC ... 70
6.2 Criterios para decidir qué opera sin conexión ... 71
6.2.1 Hechos consumados frente a operaciones que requieren confirmación ... 71
6.2.2 Matriz de EnsayAR ... 72
6.2.3 Ventana offline configurable ... 73
6.3 Persistencia local en el cliente ... 74
6.3.1 IndexedDB en el navegador ... 74
6.3.2 SQLite en el móvil ... 74
6.3.3 AsyncStorage y Web Storage ... 74
6.3.4 Cuotas, límites y desalojo ... 75
6.4 Aplicaciones web progresivas ... 76
6.4.1 Definición y origen ... 76
6.4.2 Manifiesto e instalabilidad ... 76
6.4.3 Service worker y su ciclo de vida ... 77
6.4.4 Estrategias de caché ... 78
6.4.5 Background Sync ... 79
6.4.6 Push API, Notifications API y VAPID ... 79
6.4.7 Ventajas frente a una aplicación nativa ... 80
6.4.8 Limitaciones en iOS ... 80
6.5 Sincronización ... 81
6.5.1 Cola de operaciones y orden ... 81
6.5.2 Reintentos y backoff exponencial ... 82
6.5.3 Estados de una operación ... 82
6.5.4 Sincronización por lotes ... 84
6.6 Identificadores generados en el dispositivo ... 84

6.7 Conflictos y control de concurrencia 85
6.7.1 Estrategias de resolución 85
6.7.2 Concurrencia optimista y pesimista 86
6.7.3 El caso de la última unidad paso a paso 87
6.8 Idempotencia 89
6.8.1 Definición formal 89
6.8.2 Métodos HTTP idempotentes 90
6.8.3 La cabecera Idempotency-Key 90
6.8.4 Qué se almacena y por cuánto tiempo 91
6.8.5 Doble y triple clic con red inestable 92
6.9 Transacciones y rollback 93
6.9.1 Propiedades ACID 93
6.9.2 Niveles de aislamiento 93
6.9.3 SELECT ... FOR UPDATE frente a UPDATE condicional 94
6.9.4 Una transacción por operación sincronizada 94
6.10 Monolito modular frente a microservicios 95
6.10.1 Rollback local en el monolito 95
6.10.2 Transacciones distribuidas y 2PC 95
6.10.3 Patrón Saga 96
6.10.4 Ejemplo: saga de venta si EnsayAR se dividiera 97
6.10.5 Patrón Transactional Outbox 98
6.11 Notificaciones push y otras capacidades aprovechadas 99
6.12 Aplicación en EnsayAR 99
6.12.1 Servidor 100
6.12.2 Web 100
6.12.3 Móvil 101
6.12.4 Verificación automatizada 101
6.12.5 Limitaciones conocidas 101
6.12.6 Procedimiento de verificación del conflicto de stock 102
7. Inteligencia artificial para reportes dinámicos 104
7.1 Introducción y alcance del capítulo 104
7.2 Del reporte estático al reporte dinámico 105
7.2.1 Reportes estáticos, parametrizados y ad hoc 105
7.2.2 Anatomía de un reporte 105

7.2.3 El reporte como consulta declarativa 106
7.3 Interfaces en lenguaje natural a bases de datos 106
7.3.1 Primera generación: sistemas sintácticos, semánticos y de representación intermedia 106
7.3.2 Text-to-SQL neuronal y los conjuntos de evaluación 107
7.3.3 El cambio que introdujeron los modelos de lenguaje de gran escala 108
7.4 Riesgos de dejar que el modelo escriba SQL libre 109
7.5 Enfoque adoptado: capa semántica y representación intermedia 110
7.5.1 Las capas semánticas en la industria 110
7.5.2 La especificación como contrato entre la IA y el compilador 111
7.5.3 Comparación de enfoques 112
7.6 Arquitectura en tubería 112
7.6.1 Etapa 1: captura 113
7.6.2 Etapa 2: normalización 114
7.6.3 Etapa 3: interpretación 114
7.6.4 Etapa 4: validación 114
7.6.5 Etapa 5: compilación 115
7.6.6 Etapa 6: ejecución aislada por tienda 116
7.6.7 Etapa 7: presentación, narración y exportación 116
7.6.8 Etapa 8: auditoría 116
7.7 La capa semántica de EnsayAR en detalle 117
7.7.1 Fuentes 117
7.7.2 Anatomía de un campo 117
7.7.3 Agregaciones 118
7.7.4 Joins opcionales y cálculos encapsulados 118
7.7.5 Tiempo y zona horaria 119
7.7.6 Datos agregados para el ejemplo del docente 119
7.8 Ingeniería de prompts aplicada 119
7.8.1 Contexto de esquema 120
7.8.2 Valores reales del dominio 120
7.8.3 Aprendizaje con pocos ejemplos 120
7.8.4 Salida JSON estructurada 121
7.8.5 Temperatura y determinismo 121
7.8.6 Datos inexistentes: avisar, no inventar 122
7.9 El intérprete por reglas 122

7.10 Reconocimiento de voz ... 124
7.10.1 Fundamentos ... 124
7.10.2 Alternativas tecnológicas ... 124
7.10.3 Implementación en EnsayAR ... 125
7.10.4 Normalización posterior al reconocimiento ... 125
7.11 Ambigüedad y su resolución ... 126
7.12 Seguridad ... 127
7.13 Generación del Excel, del PDF y de la narración automática ... 129
7.13.1 Excel ... 129
7.13.2 PDF ... 129
7.13.3 Narración automática ... 129
7.14 Disponibilidad: cortacircuito y degradación controlada ... 130
7.15 Evaluación ... 131
7.15.1 Verificación automatizada del módulo ... 131
7.15.2 Comparación con SQL independiente ... 131
7.15.3 Cómo medir la precisión del intérprete con IA ... 132
7.16 Ejemplo completo paso a paso ... 133
7.16.1 Pedido ... 133
7.16.2 Normalización e interpretación ... 133
7.16.3 Especificación validada ... 134
7.16.4 SQL generado y parámetros ... 134
7.16.5 Resultado ... 135
7.16.6 Narración y avisos ... 135
7.17 Ejemplos complementarios: agrupación y criterio sobre totales ... 135
7.17.1 Agrupación por dos niveles ... 135
7.17.2 Criterio sobre el total del grupo (HAVING) ... 137
7.18 Limitaciones y trabajo futuro ... 138
7.19 Conclusiones del capítulo ... 139
8. Analítica con aprendizaje automático ... 141
8.1 Introducción y alcance ... 141
8.2 Fundamentos del aprendizaje automático ... 141
8.2.1 Aprendizaje supervisado ... 141
8.2.2 Aprendizaje no supervisado ... 142
8.2.3 Entrenamiento, validación y prueba ... 142

8.2.4 Métricas de evaluación 142
8.3 Datos sintéticos 143
8.3.1 Justificación 143
8.3.2 Proceso de generación 143
8.3.3 Separación de los datos reales 144
8.4 Modelo supervisado: pronóstico de demanda 144
8.4.1 Formulación 144
8.4.2 Algoritmo 145
8.4.3 Entrenamiento y evaluación 145
8.4.4 Uso del resultado 146
8.5 Modelo no supervisado: segmentación de clientas 146
8.5.1 Variables RFM 146
8.5.2 Algoritmo 146
8.5.3 Número de grupos y evaluación 147
8.5.4 Interpretación 147
8.6 Tablero 147
8.7 Arquitectura del módulo 148
8.8 Estado de implementación y verificación 149
9. Proceso de desarrollo con ICONIX 150
9.0 La metodología ICONIX 150
9.0.1 Origen y autores 150
9.0.2 Principios del proceso 152
9.0.3 Las cuatro etapas y sus hitos 153
9.0.4 Reglas para escribir casos de uso en ICONIX 155
9.0.5 El análisis de robustez 156
9.0.6 Diagramas de secuencia y asignación de responsabilidades 159
9.0.7 Trazabilidad 161
9.0.8 ICONIX frente a otros procesos 161
9.0.9 Herramientas: Enterprise Architect 163
9.0.10 Ventajas y críticas 165
9.0.11 Por qué ICONIX en el segundo ciclo de EnsayAR 166
9.0.12 Organización iterativa por lotes 166
9.0.13 Reutilización de los artefactos del primer ciclo 167
9.0.14 Hitos y calendario del ciclo 168

9.0.15 Los scripts de verificación como pruebas de integración 169
9.0.16 Organización del resto del capítulo 170
9.1 Requisitos 170
9.1.1 Requisitos del segundo ciclo 171
9.1.2 Modelo de dominio 179
9.1.3 Prototipos de interfaz 185
9.1.4 Actores 189
9.1.5 Diagramas de casos de uso 190
9.1.6 Texto de los casos de uso 205
9.1.7 Revisión de requisitos 230
9.2 Análisis y diseño preliminar 231
Lote 0: Casos heredados del primer ciclo, revisados con robustez y secuencia 232
Lote 1: SaaS multi-tienda 258
Lote 2: Operación sin conexión y sincronización 263
Lote 3: Reportes generativos y ventas al crédito 268
Lote 4: Analítica con aprendizaje automático 272
Lote 5: Pagos para países hispanohablantes 273
9.2.1 Revisión de diseño preliminar (PDR) 273
9.3 Diseño detallado 274
Lote 0: Casos heredados del primer ciclo, revisados con robustez y secuencia 275
Lote 1: SaaS multi-tienda 301
Lote 2: Operación sin conexión y sincronización 305
Lote 3: Reportes generativos y ventas al crédito 310
Lote 4: Analítica con aprendizaje automático 313
Lote 5: Pagos para países hispanohablantes 315
9.3.1 Modelo estático de diseño 315
9.3.2 Arquitectura 331
9.3.3 Máquinas de estado de la sincronización 339
9.3.4 Revisión crítica de diseño (CDR) 343
9.4 Implementación y pruebas 343
9.4.1 Plan de pruebas 344
9.4.2 Casos de prueba 344
9.4.3 Pruebas de integración 345
9.4.4 Pruebas unitarias 346

9.4.5 Matriz de trazabilidad 346
9.4.6 Revisión de código y sincronización del modelo 350
10. Adopción y capacitación 351
10.1 Entorno de demostración 351
10.2 Alta de una tienda nueva (autoservicio) 352
10.3 Una cuenta, varias tiendas 352
10.4 Guía del dueño de la plataforma 353
10.5 Guía de caja (con y sin conexión) 353
10.6 Guía del encargado 354
10.7 Guía de administración de la tienda 354
10.8 Guías de clienta y proveedor 355
10.9 Simulacros 355
10.10 Guion de capacitación 355
10.11 Mensajes clave y alcance comunicado a los usuarios 356
11. Conclusiones 357
12. Recomendaciones 359
13. Referencias 361
14. Anexos 370
14.1 Repositorio 370
14.2 URL de despliegue y código QR 370
14.3 Puesta en marcha local 370
14.4 Verificaciones automáticas 371
14.5 Cuentas de demostración 371
14.6 Regenerar el modelo de Enterprise Architect 372
14.7 Generar este informe 372

1. Perfil del proyecto

1.1 Introducción

EnsayAR nació en el primer ciclo como una plataforma omnicanal para una cadena de moda femenina: catálogo con variantes de talla y color, punto de venta, inventario por almacén, reservas, probador virtual 2D y 3D, bot de la tienda y reportes en lenguaje natural, todo sobre un único stock compartido por la web, la app móvil y la caja. El nombre une *ensayar* (probarse la ropa) y *AR* (realidad aumentada).

El segundo ciclo cambia la escala del producto. EnsayAR deja de ser el sistema de una cadena y pasa a ser un **software como servicio**: cualquier tienda de ropa de un país de habla hispana entra a la página comercial, elige un plan, verifica su correo, paga y recibe su propia tienda lista para operar, sin hablar con nadie. Todas las tiendas comparten la misma aplicación y la misma base de datos, pero cada una ve solo sus datos. Una misma persona puede administrar varias tiendas con un solo inicio de sesión.

El cambio de escala trae exigencias que una sola cadena no tenía:

- **Conectividad irregular.** Muchas tiendas operan con internet inestable, así que la caja debe poder vender, registrar clientas y dar de alta productos sin red, y sincronizar después sin duplicar nada ni vender dos veces la última unidad.
- **Preguntas que nadie previó.** Las dueñas hacen preguntas que no estaban en ningún menú: el reporte se pide por texto o por voz, puede cruzar varias tablas y se entrega en pantalla, en PDF o en Excel.
- **Analítica y cobros regionales.** La plataforma ofrece analítica con aprendizaje automático y cobra en monedas y medios de pago de toda la región.

El segundo ciclo es un **incremento**: todo lo construido en el primero sigue funcionando y se presenta. La arquitectura de monolito modular en NestJS, los clientes React y Expo y PostgreSQL se conservan; lo que cambia es que cada módulo opera dentro de una tienda, y que el proceso se documenta con ICONIX en lugar del Proceso Unificado.

1.2 Objetivo general

Convertir EnsayAR en una plataforma SaaS multi-tienda para el comercio de moda de los países de habla hispana, que permita a cualquier tienda suscribirse y operar de forma aislada, vender con o sin conexión en web y móvil, pedir reportes complejos en lenguaje natural, obtener analítica predictiva y cobrar con medios de pago regionales, documentando el incremento con la metodología ICONIX.

1.3 Objetivos específicos

Del primer ciclo, que se mantienen:

1. Modelar el inventario matricial (producto, variante de talla y color, stock por almacén) con disponible igual a cantidad menos reservado, compartido por web, móvil y caja.
2. Ofrecer catálogo, ficha, favoritos, opiniones, carrito, compra, reservas y probador virtual 2D y 3D en la web (PWA) y en la app Expo.
3. Operar el punto de venta con apertura y cierre de caja, cupones, precio mayorista, transferencias, despacho, devoluciones y comprobante simulado.
4. Gestionar los roles de clienta, cajero, encargado, administrador y proveedor.

Del segundo ciclo:

5. Aislar los datos de cada tienda en una base compartida, de modo que ninguna consulta pueda leer ni modificar datos de otra tienda aunque el código olvide filtrar.

6. Automatizar el alta de una tienda: página de planes, registro, verificación del correo, pago de la suscripción y creación de la tienda con su sucursal, almacén y administrador.
7. Permitir que una persona con varias tiendas use un solo inicio de sesión y cambie de tienda, y dar al dueño de la plataforma métricas de suscripciones con proyección.
8. Vender, registrar clientas y dar de alta productos sin conexión en la PWA y en la app móvil, con identificadores generados en el dispositivo, sincronización idempotente, conflicto de stock resuelto en la base y una ventana offline configurable por tienda.
9. Generar reportes desde lenguaje natural, por texto o voz, con filtros, cálculos, agrupaciones y orden sobre varias tablas, sin que el modelo de IA escriba SQL, y exportarlos a PDF y Excel.
10. Incorporar analítica con un modelo supervisado y uno no supervisado entrenados con datos sintéticos, y una pasarela de pago real en modo de pruebas para la región.

1.4 Descripción del problema

Pregunta	Respuesta
¿Quién lo padece?	Las tiendas de ropa pequeñas y medianas de la región, que no pueden pagar un sistema propio, y las cadenas que operan con conectividad irregular.
¿Cuál es el problema?	Un sistema instalado por tienda es caro de mantener; un sistema en la nube que se detiene sin internet no sirve en una caja; y los reportes fijos no responden las preguntas reales de la dueña.
¿Cuál es el impacto?	Ventas perdidas cuando cae la red, stock vendido dos veces entre cajas, decisiones sin datos y costos de cobro altos por usar pasarelas pensadas para otros mercados.
¿Qué se propone?	EnsayAR como SaaS: alta automática de tiendas, aislamiento por tienda, operación sin conexión con sincronización segura, reportes generativos, analítica y pagos regionales.

El problema del primer ciclo sigue vigente dentro de cada tienda: si cada canal lleva su propio saldo, la última unidad de una talla y color puede venderse dos veces. El segundo ciclo lo agrava, porque ahora dos cajas pueden vender esa unidad **sin conexión**, y el conflicto recién

aparece al sincronizar. La solución es la misma en ambos casos: el descuento de stock es una actualización condicional en la base, que solo se aplica si queda disponible.

1.5 Alcance del incremento

Punto de la consigna	Qué se construyó	Estado
1. SaaS multi-tienda	Página de planes, registro con verificación de correo, suscripción con pago idempotente, alta automática de la tienda, selector de tiendas con un solo login, límites por plan y panel de métricas (MRR, ARR, ARPU, churn, proyección a 6 meses). Aislamiento con `tienda_id` en 36 modelos y una extensión de Prisma que falla cerrada; modalidad piloto con un esquema PostgreSQL por tienda (sección 4.12).	Hecho y verificado (`verificar:tenancy`, `verificar:saas`); esquemas validados en laboratorio
2. Operación sin conexión	PWA con service worker, caché de datos y cola IndexedDB; app móvil con SQLite; UUID en el cliente; sincronización por lotes con estado por operación; idempotencia en 11 rutas; conflicto de la última unidad visible y resoluble; ventana offline de 1 a 30 días; Web Push.	Hecho y verificado (`verificar:offline`)
3. Reportes generativos	Capa semántica de 6 fuentes, intérprete con Azure OpenAI y respaldo por reglas, especificación JSON validada, SQL parametrizado aislado por tienda, voz en web y móvil, PDF y Excel; ventas al crédito y abonos.	Hecho y verificado (`verificar:reportes`)
4. Analítica con aprendizaje automático	Pronóstico de demanda con regresión lineal, segmentación RFM con k-medias, datos sintéticos reproducibles y tablero web `/analitica` (capítulo 8; UC52 y UC53).	Hecho y verificado con datos sintéticos (`verificar-analitica-http.ts`)
5. Pagos para países hispanohablantes	Estudio de costos por país, catálogo de métodos de 22 países con validación en el servidor, puerto de pasarela con adaptador simulado, idempotencia y compensación por reembolso (capítulo 5).	Implementado en modo simulado; falta el adaptador al entorno de pruebas de una pasarela real

El alcance funcional del primer ciclo (catálogo, caja, inventario, reservas, probadores, bot, mayorista, transferencias, despacho, devoluciones y reportes) se mantiene completo y ahora

opera dentro de cada tienda. Sus casos de uso UC01 a UC35 se revisaron con robustez y secuencia en el capítulo 9.

1.6 Características del sistema

- **Multi-tienda:** una aplicación, una base y muchas tiendas aisladas; la tienda se resuelve por el token de sesión y, antes del login, por la cabecera `X-Tienda` con el slug de la tienda.
- **Autoservicio:** la tienda se crea sola al aprobarse el pago, sin intervención humana.
- **Una cuenta, varias tiendas:** la identidad (Cuenta) es global y la membresía (Usuario) es por tienda.
- **Offline-first en caja:** la venta en efectivo, el registro de clientas y el alta de productos funcionan sin red; los pagos con QR, tarjeta y el crédito exigen conexión.
- **Sincronización segura:** UUID del dispositivo, clave de idempotencia, una transacción por operación y actualización condicional del stock.
- **Reportes generativos:** texto o voz, la IA propone y el servidor valida y compila; salida en pantalla, PDF o Excel.
- **Omnicanal:** un saldo por variante y almacén; web, móvil y caja compiten por la misma unidad.
- **Probador 2D y 3D, bot de tienda, precio mayorista, despacho y devoluciones**, heredados del primer ciclo.

1.7 Stack tecnológico

Capa	Tecnología	Uso en el segundo ciclo
API	NestJS 11 + TypeScript	Monolito modular; contexto de tienda con AsyncLocalStorage
Acceso a datos	Prisma 6	Extensión que filtra e inyecta `tiendaId`; transacciones con `set_config` para SQL crudo
Base de datos	PostgreSQL	Base compartida con columna de

Capa	Tecnología	Uso en el segundo ciclo
		tienda y valor por defecto desde la sesión
Web	React 19 + Vite, PWA	Service worker, IndexedDB, Web Speech API, Web Push
Móvil	Expo SDK 53 / React Native	`expo-sqlite`, NetInfo, `expo-file-system` y `expo-sharing`
IA	Azure OpenAI	Intérprete de reportes, bot y ficha; respaldo por reglas cuando no responde
Exportación	exceljs, pdfkit	Excel con hoja "Cómo se generó"; PDF A4 apaisado paginado
Modelado	Enterprise Architect 15	Modelo ICONIX generado por automatización desde `casos.py`

1.8 Situaciones de uso del incremento

Situación 1: una tienda nueva. La dueña de una boutique de Quito entra a la página de planes, elige el plan inicial por tres meses, registra su cuenta y confirma el correo. Paga con la tarjeta de prueba; si pulsa pagar dos veces, el cobro se hace una sola vez. Al aprobarse el pago la plataforma crea su tienda con sucursal, almacén y su usuario administrador, y la deja dentro sin pedir otro login.

Situación 2: dos cajas sin internet. En una tienda se corta la red. Dos cajeros venden en efectivo la última blusa talla M. Ambos tickets quedan pendientes en sus dispositivos. Al volver la red, el primer lote se aplica; el segundo encuentra el stock en cero y queda en conflicto, visible en la pantalla de sincronización, donde el cajero decide vender lo disponible o descartar. El stock nunca queda negativo.

Situación 3: la pregunta de la dueña. La dueña dicta: "clientes con nombre, dirección y email que me deben de 500 a 3000 bolivianos por ventas al crédito, en PDF". El sistema arma la especificación, la valida, la compila a SQL con parámetros, la ejecuta solo sobre su tienda y descarga el PDF con los criterios y los totales.

Situación 4: una dueña con dos tiendas. La administradora de Vesta también administra Aurora Boutique. Desde el selector de la barra lateral cambia de tienda; el sistema emite un token nuevo para Aurora y le muestra sus productos, sin pedir la clave otra vez.

1.9 Consideraciones

Tema	Criterio
Pagos	El pago de la suscripción y de las ventas es de prueba; no hay cobro bancario real. La conexión con el entorno de pruebas de una pasarela real se describe en el capítulo 5.
IA	La clave de Azure OpenAI del entorno de demostración responde 401; mientras no se renueve, el intérprete por reglas atiende los reportes.
Móvil	La operación sin red se probó en Expo web; en Android se verificó que el paquete compila.
Datos de demo	Tiendas Vesta y nueve tiendas `demo-*` de distintos países con historia de suscripciones; 34 clientas y 320 ventas sintéticas de 180 días para reportes.
Fuera de alcance	Cobro bancario real, facturación fiscal real y aislamiento con políticas RLS en la base (documentado como evolución).

1.10 Evaluación de la arquitectura heredada y estrategia de evolución

Un segundo ciclo de desarrollo pone a prueba las decisiones del primero. Los requisitos nuevos (varias tiendas en una sola instalación, operación sin conexión, reportes generados desde lenguaje natural, analítica y pagos regionales) no estaban previstos cuando se diseñó EnsayAR, y la forma en que la arquitectura los absorbe mide su calidad.

1.10.1 Alternativas consideradas

Ante un cambio de requisitos de esta magnitud existen tres caminos. El primero es reutilizar la arquitectura sin cambios y agregar los módulos nuevos alrededor; es el más rápido, pero no sirve cuando el requisito nuevo afecta a todas las capas, como ocurre con el aislamiento por tienda. El segundo es modificar las capas afectadas y conservar el resto; exige analizar con cuidado el impacto, pero aprovecha el código probado. El tercero es descartar la versión anterior

y reconstruir el sistema con una arquitectura pensada para los requisitos nuevos; resuelve los problemas de fondo a costa de repetir todo el trabajo del primer ciclo y de volver a probar funciones que ya operaban.

1.10.2 Evaluación por capa

La tabla siguiente resume cómo respondió cada capa de la arquitectura del primer ciclo a los requisitos nuevos.

Capa	Arquitectura del primer ciclo	Cambio requerido en el segundo ciclo	Valoración
Organización del backend	Monolito modular NestJS con un módulo por área	Se agregaron los módulos de plataforma, tenencia, sincronización, reportes inteligentes y créditos sin modificar los existentes	La modularidad absorbió el crecimiento
Acceso a datos	Prisma sobre PostgreSQL, una sola tienda	Columna `tienda_id` en 36 modelos y una extensión de Prisma que la aplica en todas las consultas	Cambio transversal, pero concentrado en un punto gracias al ORM
Transacciones	Venta con bloqueo de stock y UUID único	Se reutilizaron para la sincronización y se agregaron la idempotencia y la actualización condicional	Base sólida; se extendió sin reescribir
Cliente web	React con una cola de ventas en IndexedDB	Service worker, caché de datos, cola para clientas y productos, Web Push	Se amplió el mecanismo existente
Cliente móvil	React Native con Expo	Base local SQLite y sincronización por lotes	Se agregó una capa de persistencia
Reportes	Reportes predefinidos y consulta por voz simple	Capa semántica, compilador de SQL parametrizado y exportación	Se reemplazó el intérprete; la interfaz se conservó
Infraestructura	Una instancia EC2 de AWS con la API, la web y PostgreSQL	Sin cambios de topología	Adecuada para la escala actual

1.10.3 Camino elegido

EnsayAR siguió el segundo camino: modificar capas. La arquitectura del primer ciclo resistió el cambio en lo esencial. El monolito modular permitió agregar áreas completas sin tocar las existentes, y las decisiones de integridad del primer ciclo (identificadores únicos generados en el cliente y bloqueo del stock dentro de una transacción) resultaron ser exactamente la base que necesitaba la sincronización sin conexión. El único cambio verdaderamente transversal fue

el aislamiento por tienda, que afectó a todos los modelos de datos; aun así, se resolvió en un único punto del acceso a datos y no en cada servicio, lo que confirma el valor de haber centralizado la persistencia en un ORM.

Reconstruir el sistema no estaba justificado: los 35 casos de uso del primer ciclo siguen operando dentro de cada tienda y se verificaron de nuevo con robustez y secuencia en el capítulo 9. El costo de rehacerlos habría consumido el tiempo necesario para los requisitos nuevos sin aportar una mejora proporcional.

La evaluación también identifica los límites de la arquitectura actual. El aislamiento depende de la aplicación y no de la base de datos, por lo que la incorporación de políticas de seguridad por fila de PostgreSQL es la evolución natural (sección 4.10). Para las tiendas que requieran aislamiento físico, el modelo de base compartida ya cuenta con una alternativa implementada y validada en laboratorio: un esquema PostgreSQL por tienda, que se activa por configuración (sección 4.12). Por último, si el módulo de analítica adopta modelos más costosos de entrenar, será el primer candidato a separarse del monolito como un servicio independiente (sección 8.7).

2. Marco teórico del primer ciclo

2.1 Marco teórico referencial

2.1.1 Antecedentes internacionales

El comercio electrónico de moda creció al permitir consultar catálogos, comparar precios y comprar sin desplazarse al local. Plataformas de gran escala y tiendas en línea generalizan un flujo conocido: búsqueda, ficha del producto, variantes, carrito, cobro y seguimiento del pedido. Ese modelo resuelve distancia y horario, pero en ropa deja un hueco: la clienta no puede probarse la prenda como en el camerino.

En la industria han aparecido soluciones de **probador virtual** y realidad aumentada que superponen la prenda sobre una foto o sobre el cuerpo estimado por la cámara. El nivel industrial suele incluir mallas tridimensionales y simulación de caída de tela; el nivel académico y de producto mínimo viable suele limitarse a un overlay 2D anclado a puntos del torso. EnsayAR se sitúa en ese segundo rango: demostrar el ensayo asistido sobre el propio cuerpo, sin pretender un motor de moda industrial.

2.1.2 Antecedentes nacionales

En Bolivia, y en particular en Santa Cruz, las cadenas de moda femenina concentran la venta en sucursales físicas. El prestigio y los lanzamientos saturan el local: la clienta debe desplazarse, esperar camerino y comprar solo en horario de tienda. Abrir más puntos de venta no escala de forma sostenible: cada sucursal suma espacio, personal y costo operativo, y las ciudades sin tienda quedan mal atendidas.

La compra de ropa por catálogo digital o redes sociales existe, pero muchas veces se hace **sin ensayar**. El color en pantalla puede diferir, la talla no siempre coincide y las devoluciones o

el abandono de compra son consecuencias frecuentes. El problema local no es la falta de Internet; es la falta de un sistema que una el inventario de la tienda con un canal en línea y con una forma de ensayar la prenda antes de decidir.

2.1.3 Justificación del proyecto

EnsayAR responde a ese escenario con **un inventario** (producto × talla × color × almacén) y **dos canales** que descuentan el mismo saldo: la web/móvil y el punto de venta. Si la clienta compra la última unidad en línea y el cajero intenta cobrar la misma variante, la segunda operación debe fallar. Dos aplicaciones con inventarios copiados no son omnicanalidad.

El nombre del sistema junta *ensayar* (probarse ropa) y *AR* (realidad aumentada). El MVP del parcial (UAGRM, Sistemas de Información II, gestión 2026-2) incluye catálogo, venta en caja y en línea, reservas, pagos simulados, reportes y un probador virtual acotado, documentado con el Proceso Unificado de Desarrollo de Software.

2.1.4 Arquitectura cliente-servidor

EnsayAR es un sistema **cliente-servidor**. Los clientes de presentación son:

- la aplicación web en **React** (tienda, POS y administración);
- la aplicación móvil en **React Native / Expo** (clienta y cámara del probador).

Ambos consumen una API REST implementada en **NestJS**. Los clientes no almacenan el saldo de inventario ni deciden el precio final: envían peticiones autenticadas y reciben la respuesta del servidor.

El **servidor de datos** es **PostgreSQL**. La API escribe y lee el dominio (usuarios, productos, stock, ventas, reservas, etc.) a través de Prisma. La coherencia entre canales depende de que el stock viva en un solo lugar y se actualice bajo transacciones.

2.1.5 Patrón de arquitectura

El patrón elegido es el **monolito modular**: un solo despliegue de la API, organizado en módulos (autenticación, catálogo, inventario, ventas, caja, reservas, reportes, administración). Es adecuado para el MVP del parcial: un equipo, una base de datos y transacciones locales entre venta y stock.

No se adoptan **microservicios**. Según la indicación del curso, **FastAPI** queda reservado a escenarios de microservicios y **no** es el backend de este parcial. Tampoco forman parte del stack Angular, Flutter ni Spring. Partir el sistema en varios servicios añadiría red y consistencia distribuida sin beneficio para la defensa del examen.

2.1.6 Metodología de desarrollo de software

La metodología del informe es el **Proceso Unificado de Desarrollo de Software** (Booch, Rumbaugh y Jacobson). No es Scrum: Scrum organiza el trabajo en sprints, pero no prescribe por sí mismo el modelado de requisitos, análisis, diseño, implementación y pruebas que exige el parcial.

El Proceso Unificado se representa con dos ejes:

1. **Fases:** Inicio, Elaboración, Construcción y Transición.
2. **Disciplinas** que se repiten en cada ciclo: captura de requisitos, análisis, diseño, implementación y pruebas.

Un **ciclo** produce una versión operativa (un incremento). Es análogo a un sprint en cuanto a ritmo, pero el índice del informe no se organiza por sprints: la **Parte III** se estructura por esos flujos de trabajo (fases y disciplinas), como el documento de referencia del curso. El proceso está dirigido por casos de uso y centrado en la arquitectura.

2.1.7 Tecnologías utilizadas

Capa	Tecnología
API	NestJS 11 y TypeScript
Acceso a datos	Prisma 6
Base de datos	PostgreSQL
Web	React 19 y Vite
Móvil	Expo 53 / React Native 0.79
Nube	Amazon Web Services: una instancia EC2 con Linux que aloja el proxy inverso con TLS, la API y PostgreSQL; dominio examensi2.duckdns.org

La autenticación usa JWT y el almacenamiento de contraseñas con hash. Los pagos del parcial son **simulados** (efectivo, QR de prueba, contra entrega): no se integra una pasarela productiva.

2.1.8 Base de datos relacional

PostgreSQL aporta integridad referencial, transacciones y bloqueo a nivel de fila para el stock. El inventario matricial (producto, variante, stock por almacén) y las ventas concurrentes desde web y POS exigen que dos cobros sobre la misma unidad no dejen la cantidad en negativo. Prisma versiona el esquema con migraciones y tipa el acceso desde NestJS.

2.1.9 Lenguaje TypeScript y framework NestJS

TypeScript tipa el dominio compartido entre API y clientes, reduce errores de contrato y facilita el mantenimiento del monolito. **NestJS** organiza el backend en módulos con inyección de dependencias, controladores, guards de rol y servicios de dominio, en un estilo cercano a un backend empresarial. El docente lo prefirió frente a Express para este examen y lo distinguió de Next.js. NestJS es el único backend del parcial.

2.2 Datos

El modelo de datos de EnsayAR gira en torno a la moda omnicanal. Un **Usuario** tiene uno o más **Roles** (clienta, cajero, encargado, administrador, proveedor). La operación física se

organiza en **Sucursales**, cada una con uno o más **Almacenes**. El **Producto** se concreta en **Variantes** (talla y color); el **Stock** registra, por variante y almacén, la cantidad y lo reservado. La regla de negocio es: **disponible = cantidad − reservado**.

Las operaciones de negocio enlazan ese inventario con la venta y la logística: **MovimientoInventario** deja rastro en kardex; **Venta** y **LineaVenta** documentan el cobro; **PagoSimulado** y **ComprobanteSimulado** cubren el entorno académico; **Reserva** bloquea unidades sin venderlas aún; **Caja/Turno** abre y cierra el POS; **Cupón** aplica descuentos; **Favorito**, **Opinión** y **Pedido** atienden a la clienta; **Auditoría** registra acciones sensibles.

Entidad	Descripción breve
Usuario	Cuenta de acceso al sistema
Rol	Permisos (clienta, cajero, encargado, administrador, proveedor)
Sucursal	Punto de operación de la cadena
Almacén	Depósito de stock ligado a una sucursal
Producto	Prenda o artículo del catálogo
Variante	Combinación talla × color del producto
Stock	Cantidad y reservado por variante y almacén
MovimientoInventario	Asiento de kardex (venta, reserva, ajuste, ingreso, transferencia)
Venta	Cabecera de cobro en línea o en POS
LineaVenta	Ítem vendido (variante, cantidad, precio)
PagoSimulado	Medio de pago de prueba asociado a la venta
Reserva	Bloqueo temporal de unidades para la clienta
Caja / Turno	Apertura, cobros y cierre del POS
Cupón	Descuento aplicable al total
Favorito	Variantes marcadas por la clienta
Opinión	Valoración o comentario sobre un producto
Pedido	Orden de compra en línea (despacho / seguimiento)
ComprobanteSimulado	Ticket o comprobante de entorno académico
Auditoría	Bitácora de acciones administrativas

2.3 Procesos

Los procesos del sistema EnsayAR se organizan por módulos funcionales. Cada módulo agrupa los casos de uso que la plataforma ejecuta en la web, en el móvil y en la API; su detalle se presenta a continuación.

3. Desarrollo de software bajo el modelo SaaS

En el primer ciclo del proyecto, EnsayAR se concibió como el sistema de una sola tienda de moda femenina: catálogo en línea, punto de venta, inventario por sucursal, reportes generativos y probador virtual. El segundo ciclo cambia la naturaleza del producto. Ya no se entrega un sistema a una tienda, sino un servicio que muchas tiendas contratan, pagan periódicamente y usan sin instalar nada. El cambio no es solo comercial: obliga a revisar la identidad de los usuarios, el modelo de datos, el cobro, el aprovisionamiento y la operación. Este capítulo fundamenta el modelo Software como Servicio (SaaS), describe sus variantes y su lógica económica, y cierra con lo que efectivamente se implementa en EnsayAR y por qué.

3.1 Definición y características esenciales

La referencia más citada para definir la computación en la nube es la publicación especial 800-145 del NIST (Mell y Grance, 2011). Allí la nube se describe como un modelo que permite acceder por red, de forma ubicua y bajo demanda, a un conjunto compartido de recursos configurables que se aprovisionan y liberan rápidamente con un esfuerzo mínimo de gestión o de interacción con el proveedor. El modelo se compone de cinco características esenciales, tres modelos de servicio y cuatro modelos de despliegue.

SaaS es el modelo de servicio en el que el consumidor usa las aplicaciones del proveedor, que se ejecutan sobre una infraestructura en la nube, y accede a ellas desde distintos dispositivos mediante un navegador o una aplicación cliente. El consumidor no administra la red, los servidores, el sistema operativo, el almacenamiento ni las capacidades de la aplicación, salvo algunos parámetros de configuración propios de su cuenta. La frontera de responsabilidad queda desplazada casi por completo hacia el proveedor: el cliente gestiona sus datos y su configuración.

Las cinco características esenciales del NIST adquieren un significado concreto cuando el servicio es una aplicación de negocio:

- **Autoservicio bajo demanda:** el cliente obtiene el servicio sin intervención humana del proveedor; en SaaS, registrarse, elegir un plan, pagar y empezar a usar el sistema sin hablar con nadie.
- **Amplio acceso por red:** el servicio se consume desde navegador, teléfono o tableta mediante mecanismos estándar.
- **Agrupación de recursos:** los recursos del proveedor se comparten entre muchos consumidores bajo un modelo multi-inquilino y se asignan dinámicamente. Esta característica conecta directamente con el capítulo 4.
- **Elasticidad rápida:** la capacidad crece o se reduce con la demanda.
- **Servicio medido:** el uso se mide y se controla, lo que habilita cobrar por consumo, imponer límites y dar transparencia a ambas partes.

Los modelos de despliegue del NIST son nube privada, comunitaria, pública e híbrida. Un SaaS dirigido a pequeños comercios se ofrece normalmente sobre nube pública, porque su economía depende de compartir infraestructura entre muchos clientes pequeños. Para el cliente, la definición se resume en tres rasgos: no compra licencias ni infraestructura sino una suscripción; el proveedor opera, actualiza y respalda la aplicación; y el cliente puede cambiar de nivel a medida que su negocio cambia.

3.2 SaaS frente a otros modelos de entrega

On-premise. En el modelo tradicional, el cliente compra una licencia, normalmente perpetua, y ejecuta el software en sus equipos. Asume hardware, instalación, respaldos, seguridad y actualizaciones, y cada cliente puede quedarse en una versión distinta, lo que

fragmenta el soporte. Ofrece control total y no depende de la conectividad, pero su costo de entrada es prohibitivo para una tienda pequeña.

ASP (proveedor de servicios de aplicación). En los años noventa, terceros comenzaron a alojar la aplicación del cliente en su centro de datos y a ofrecerla por red. Aunque el cliente no administra servidores, cada uno tiene su propia instancia, a menudo personalizada, de un software que no fue diseñado para compartirse. Chong y Carraro (2006) propusieron un modelo de madurez SaaS de cuatro niveles y ubicaron al ASP en el primero: instancias individuales y personalizadas. Los niveles siguientes agregan configurabilidad con un mismo código para todos, una sola instancia eficiente para muchos inquilinos y, por último, escalabilidad mediante una granja de instancias idénticas balanceadas.

IaaS y PaaS. Infraestructura como Servicio entrega procesamiento, almacenamiento y red, y el consumidor instala y controla el sistema operativo y sus aplicaciones. Plataforma como Servicio entrega un entorno de ejecución gestionado donde el consumidor despliega su propio código sin administrar servidores. SaaS está un nivel más arriba: el consumidor no despliega código, usa una aplicación terminada. Los modelos se apilan: un proveedor SaaS es a su vez cliente de PaaS o IaaS. EnsayAR es SaaS para las tiendas y cliente IaaS de AWS, donde una instancia EC2 aloja la API, la web y la base PostgreSQL.

Capa	On-premise	ASP	IaaS	PaaS	SaaS
Hardware e instalaciones	Cliente	Proveedor	Proveedor	Proveedor	Proveedor
Sistema operativo y red	Cliente	Proveedor	Cliente	Proveedor	Proveedor
Entorno de ejecución y base de datos	Cliente	Proveedor	Cliente	Proveedor	Proveedor
Aplicación	Cliente (licenciada)	Proveedor, una instancia por cliente	Cliente	Cliente	Proveedor, instancia compartida
Datos y configuración	Cliente	Cliente	Cliente	Cliente	Cliente
Pago típico	Licencia y mantenimiento	Cuota de alojamiento	Por recurso	Por recurso	Suscripción o uso

3.3 Variantes del modelo SaaS

3.3.1 SaaS horizontal y vertical

Un SaaS horizontal resuelve una necesidad transversal a muchas industrias, como correo, ofimática o contabilidad genérica: su mercado es enorme, pero el producto tiende a ser genérico y la competencia es intensa. Un SaaS vertical se especializa en un sector y modela su vocabulario y sus procesos; su mercado es menor, pero el valor percibido es mayor. EnsayAR es un SaaS vertical para tiendas de moda: maneja variantes por talla y color, temporadas, colecciones, reservas para probarse en tienda, probador virtual y venta en caja que tolera cortes de internet. Ese ajuste al rubro justifica que una tienda pague por él en lugar de usar una herramienta genérica.

3.3.2 Single-tenant y multi-tenant

Un inquilino (tenant) es el cliente que contrata el servicio, con sus usuarios y sus datos. En un despliegue single-tenant cada cliente tiene su propia copia de la aplicación y de la base, aunque la opere el proveedor. En uno multi-tenant, una misma instancia atiende a todos y los datos se separan lógicamente. Entre ambos extremos hay combinaciones, como aplicación compartida con bases separadas o clientes grandes en infraestructura dedicada. El Azure Architecture Center subraya que la elección es también comercial: si se espera escalar a muchos clientes, compartir infraestructura es casi obligatorio porque mantener recursos por cliente se vuelve insostenible; si se esperan pocos clientes con exigencias fuertes de aislamiento, lo dedicado puede justificarse aunque cueste más. El capítulo 4 desarrolla este tema.

3.3.3 Marca blanca (white-label)

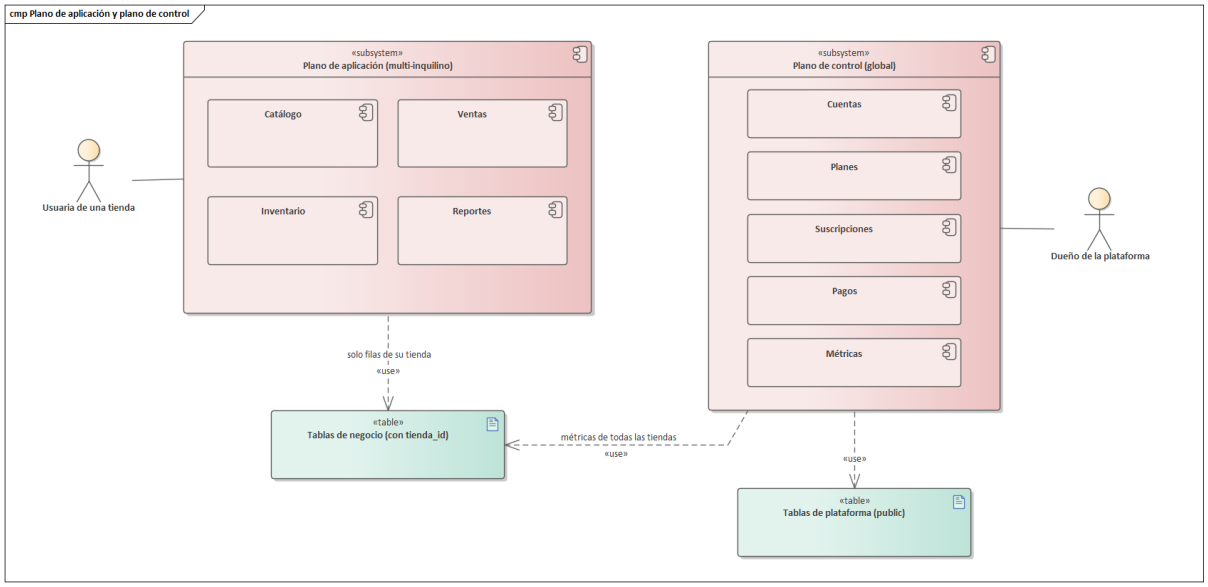
En la variante white-label, el cliente presenta el servicio con su propia identidad (logotipo, colores, dominio, a veces nombre comercial) sin que el usuario final perciba al proveedor. Es habitual en comercio electrónico, donde la tienda quiere que sus compradoras vean su marca. Exige dominios personalizados con certificados, plantillas configurables y correos enviados en nombre del cliente.

3.3.4 Plano de control y plano de aplicación

La guía SaaS Architecture Fundamentals de AWS divide toda solución SaaS en dos planos. El plano de control reúne los servicios para incorporar, autenticar, administrar, operar y analizar inquilinos: alta, identidad, registro de inquilinos, facturación, métricas y consola del proveedor. El plano de aplicación contiene la funcionalidad de negocio que usan los inquilinos. El documento aclara que los servicios del plano de control no son multi-inquilino: son globales y ven a todos los clientes, mientras que el plano de aplicación sí debe aplicar aislamiento. Esta separación ayuda a razonar qué operaciones pueden cruzar inquilinos y cuáles nunca deben hacerlo.

Figura 3.1

Plano de aplicación y plano de control en EnsayAR



Nota. Diagrama de componentes UML 2.5: el plano de aplicación solo accede a las filas de su tienda; el plano de control es global y lee las métricas de todas las tiendas.

3.4 Modelos de cobro

La guía de modelos de precios del Azure Architecture Center parte de una idea simple: el precio debe cubrir el costo de servir a cada inquilino y dejar margen, y el modelo elegido condiciona lo que el sistema debe medir.

Suscripción por período. Es el modelo dominante en SaaS para pequeñas empresas: un monto fijo por un mes, tres meses o un año. Los períodos largos llevan descuento y benefician a ambas partes: el cliente paga menos por mes y el proveedor recibe caja por adelantado y reduce el abandono durante el período. El período mensual, en cambio, baja la barrera de entrada. La suscripción genera ingresos recurrentes y predecibles, base de las métricas de 3.6.

Por usuario y por niveles. En el cobro por usuario el precio crece con la cantidad de usuarios. Azure lo considera fácil de implementar, pero advierte que incentiva a compartir

credenciales y puede ser poco rentable si unos usuarios consumen mucho más que otros; una variante cobra solo usuarios activos. El cobro por niveles ofrece paquetes con distinto alcance funcional, combinados con límites de uso (usuarios, sucursales, productos) y a veces con distintos acuerdos de servicio. Es claro comercialmente y ofrece un camino natural de crecimiento, pero obliga al sistema a saber qué corresponde a cada inquilino y a hacerlo cumplir en el servidor.

Por uso. El cliente paga lo que consume: transacciones, reportes, almacenamiento o llamadas a una API. La barrera de entrada es mínima y los ingresos crecen con los costos, pero exige medir con precisión el consumo de cada inquilino, produce ingresos menos predecibles y puede motivar al cliente a usar menos el producto.

Freemium y prueba gratuita. El freemium mantiene un nivel gratuito permanente con capacidades limitadas; la prueba gratuita da acceso temporal a un plan pago. Azure trata ambos como extensiones del cobro por niveles y advierte sus costos: operar inquilinos que no pagan, darlos de baja y definir políticas de retención. Skok señala que, si se usa prueba gratuita, la tasa de conversión es una de las primeras cifras a vigilar.

Cumplimiento de límites. Cualquiera sea el modelo, los límites solo existen si el servidor los hace cumplir. Ocultar un botón no basta, porque la API puede invocarse directamente; la verificación debe ocurrir donde se crea el recurso limitado, con una respuesta explícita que permita a la interfaz ofrecer el cambio de plan.

Mercado hispanohablante. La plataforma apunta a Bolivia y al resto de países de habla hispana. El medio de pago popular en un país (el QR interbancario en Bolivia) puede ser irrelevante en otro, y la pasarela elegida determina la comisión que termina pagando el cliente. Ese análisis corresponde al capítulo de formas de pago; desde la perspectiva SaaS interesa una

propiedad transversal: el cobro debe ser **idempotente**, de modo que un doble clic o un reintento por red inestable no genere dos cobros ni dos tiendas.

3.5 Ciclo de vida del cliente

En un SaaS de autoservicio, cada etapa del ciclo de vida del inquilino debe completarse sin intervención del personal del proveedor.

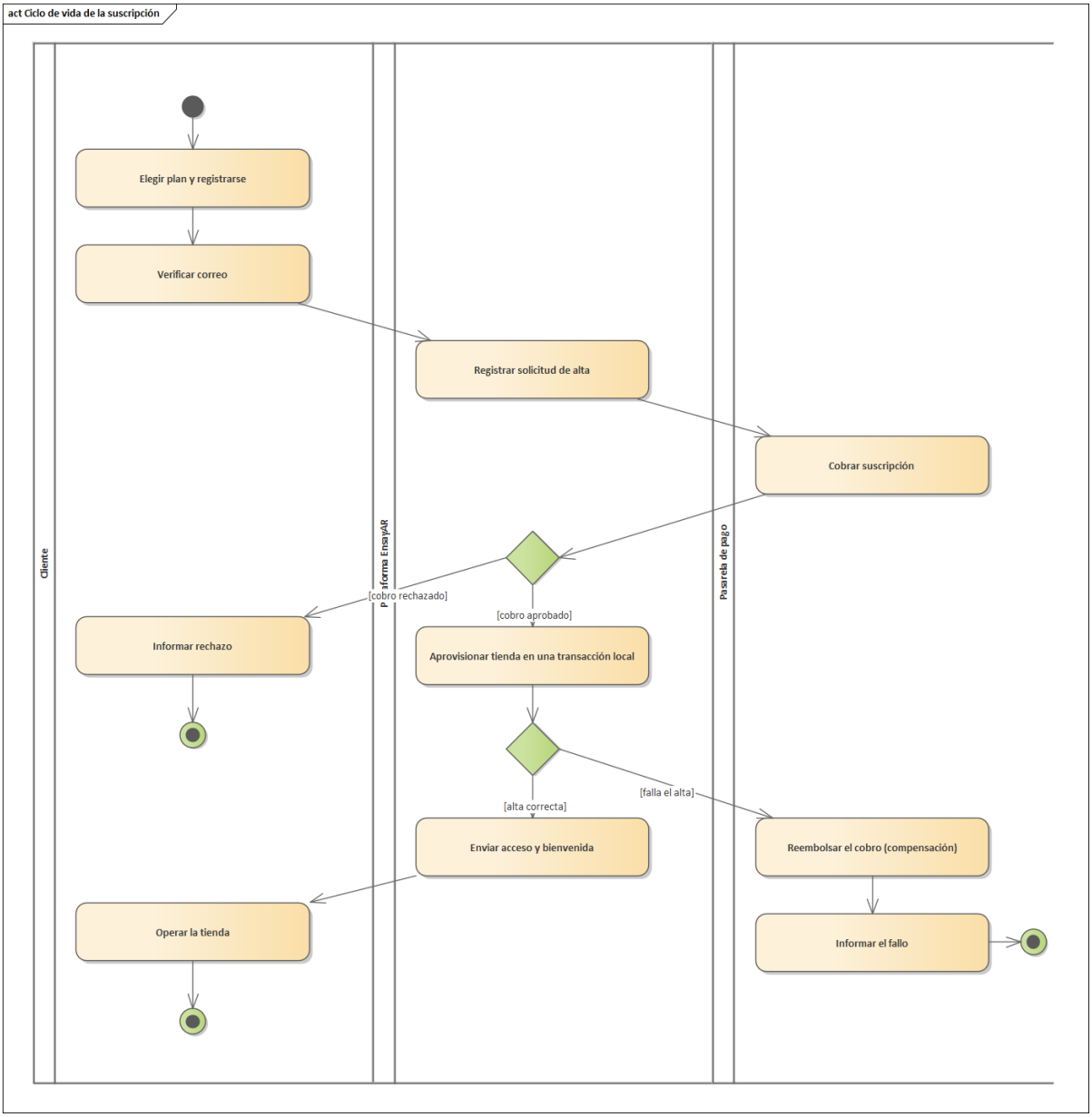
1. **Descubrimiento:** una página comercial pública presenta el producto, los módulos, los planes, los precios y las preguntas frecuentes.
2. **Registro autoservicio:** el interesado crea una cuenta con su correo y una clave, sin formularios de contacto ni llamadas.
3. **Verificación del correo:** el sistema envía un enlace con un token aleatorio y de vida corta; hasta confirmarlo, la cuenta no opera. Así se prueba que el correo existe y pertenece a quien se registra, se frenan altas falsas y se asegura un canal para avisos de cobro y recuperación.
4. **Elección de plan y pago:** el cliente elige nivel y período, paga, y el sistema informa con claridad cualquier rechazo.
5. **Aprovisionamiento automático:** tras el pago, el sistema crea el espacio de trabajo con una configuración mínima utilizable y deja al cliente dentro. AWS describe este servicio de onboarding como el que orquesta la creación del inquilino, su administrador, sus políticas y sus recursos.
6. **Uso y expansión:** el cliente invita a su equipo y opera; al alcanzar un límite, se le ofrece subir de plan.
7. **Renovación o cambio de plan:** un cambio a un plan menor debe validar que el uso actual cabe en los nuevos límites.

8. **Vencimiento, baja y retención:** si no renueva, el servicio se suspende; conviene conservar los datos un plazo razonable para permitir la reactivación y luego eliminarlos o entregarlos según la política declarada.

Entre las etapas 4 y 5 hay un punto técnico delicado. El cobro ocurre en un sistema externo y el aprovisionamiento en la base propia, y no existe una transacción que abarque ambos. Si el cobro se aprueba y la creación del inquilino falla, el cliente pagó por nada. La solución habitual es tratar el proceso como una saga: se cobra, se aprovisiona en una transacción local y, si esta falla, se ejecuta una acción compensatoria que reembolsa el pago.

Figura 3.2

Ciclo de vida de la suscripción con compensación



Nota. Diagrama de actividad UML 2.5 con particiones: si el aprovisionamiento falla después del cobro, la pasarela reembolsa el pago y la plataforma informa el fallo.

3.6 Métricas de negocio

Un negocio de suscripción no se gestiona con las métricas de una venta única. Skok, en SaaS Metrics 2.0, resume el éxito de un SaaS en tres palancas (adquirir, retener y monetizar clientes) y propone medir la economía unitaria comparando cuánto deja un cliente durante su vida con cuánto cuesta conseguirlo.

Métrica	Definición	Cálculo habitual
MRR	Ingreso recurrente mensual	Suma de suscripciones vigentes normalizadas a un mes; un plan anual aporta su monto entre 12
ARR	Ingreso recurrente anual	MRR por 12
ARPU o ARPA	Ingreso medio por cliente	MRR entre clientes vigentes
Churn de clientes	Proporción de clientes perdidos	Clientes perdidos en el período entre clientes al inicio
Churn de ingresos	Proporción del MRR perdido	MRR perdido por bajas y reducciones entre MRR al inicio
LTV	Valor de vida del cliente	Aproximación: ARPU por margen bruto, entre churn mensual
CAC	Costo de adquirir un cliente	Gasto de ventas y marketing entre clientes nuevos del período
LTV/CAC y recuperación	Rentabilidad de la adquisición	Cociente LTV/CAC; meses de ingreso para cubrir el CAC

Skok reporta como referencia que los SaaS sanos superan un LTV/CAC de 3 y recuperan el CAC en menos de 12 meses, y advierte que el churn debe corregirse primero: si los clientes se van, invertir en adquisición equivale a llenar un balde agujereado. Las proyecciones combinan estas magnitudes; un modelo simple estima el MRR del mes siguiente como el actual por la fracción retenida (uno menos el churn) más los clientes nuevos esperados por el ARPU. Como toda proyección depende de sus supuestos, estos deben mostrarse junto al resultado.

3.7 Identidad cuando un usuario tiene varias tiendas

Es frecuente que una persona participe en varios inquilinos: una dueña con dos tiendas o una cajera que trabaja en dos locales. Hay dos enfoques.

El **login por tienda** da a cada inquilino su propia tabla de usuarios con correo y clave. Es simple y aísla por completo, pero multiplica las credenciales, duplica la autenticación, impide una vista de "mis tiendas" y complica la recuperación de cuenta.

La **cuenta global con membresías** mantiene una sola identidad por persona y una membresía por cada inquilino donde participa, con el rol que tiene allí. La guía de identidad multi-inquilino del Azure Architecture Center recomienda, cuando es posible, una sola identidad por persona, porque simplifica la administración y porque muchas protecciones de los proveedores de identidad se apoyan en esa unicidad. La misma guía describe el patrón: tras autenticarse, el usuario selecciona el inquilino activo y el sistema emite un token con el identificador de ese inquilino y el rol correspondiente; cambiar de inquilino exige un token nuevo.

Criterio	Login por tienda	Cuenta global con membresías
Credenciales por persona	Una por tienda	Una sola
Rol distinto por tienda	Natural	Se guarda en la membresía
Vista de todas mis tiendas	No es posible sin cruzar inquilinos	Directa
Cambio de tienda	Cerrar sesión y volver a entrar	Nuevo token sin pedir la clave
Verificación de correo	Repetida en cada tienda	Una sola vez
Riesgo principal	Claves débiles y repetidas	El token debe quedar atado a una sola tienda activa

El riesgo del segundo enfoque se controla con una regla: el token de trabajo lleva una sola tienda activa, el servidor confía en esa tienda y no en lo que el cliente declare en otra parte de la petición, y cada cambio pasa por una verificación de membresía.

3.8 Ventajas, riesgos y responsabilidades del proveedor

Para el cliente, SaaS significa costo inicial bajo, pago proporcional a su tamaño, acceso desde cualquier lugar y actualizaciones sin esfuerzo. Para el proveedor, ingresos recurrentes, una sola versión que mantener y visibilidad sobre el uso real del producto. Los riesgos también son claros: el cliente depende de la conectividad y de la continuidad del proveedor, y puede quedar

atado si no puede exportar sus datos; el proveedor concentra el riesgo, porque un despliegue defectuoso afecta a todos, una fuga entre inquilinos destruye la confianza y un inquilino que consume demasiado degrada a los demás (vecino ruidoso). De ello se derivan sus responsabilidades:

- **Disponibilidad:** objetivos de servicio realistas, monitoreo y diseño para fallos; en un punto de venta, la tolerancia a cortes de red del lado del cliente es parte de la disponibilidad percibida.
- **Seguridad y aislamiento:** ningún inquilino ve ni modifica datos de otro, las credenciales se protegen, el tráfico se cifra y el aislamiento se prueba de forma automatizada.
- **Actualizaciones:** cambios desplegados a todos sin interrupción, con migraciones de esquema controladas.
- **Respaldo y recuperación:** procedimientos de restauración, incluso cuando el incidente afecta a un solo inquilino.
- **Transparencia comercial:** precios claros, comprobantes, avisos de vencimiento y una política explícita sobre los datos de quien no renueva.

3.9 Aplicación en EnsayAR

Esta sección describe lo implementado en el repositorio y justifica cada decisión. Los nombres de modelos, rutas y archivos son los del código.

3.9.1 Posicionamiento y estructura

EnsayAR se define como un **SaaS vertical, multi-inquilino y de autoservicio** para tiendas de moda femenina de Bolivia y de países hispanohablantes. Es un monolito modular con NestJS 11, Prisma 6 y PostgreSQL, una web React 19 instalable como PWA y una app móvil

Expo. Todas las tiendas comparten una instancia de la API y una base de datos, separadas por identificador de tienda (capítulo 4). En la escala de Chong y Carraro se ubica en el tercer nivel y queda preparado para el cuarto, porque la autenticación usa tokens JWT en lugar de sesiones en memoria y la tienda se resuelve en cada petición, de modo que la API puede replicarse detrás de un balanceador. La marca blanca es parcial: cada tienda tiene su vitrina con su nombre en la ruta /t/:slug, pero no hay dominios, logotipos ni colores propios.

La separación de planos de AWS se refleja en el backend. El módulo backend/src/plataforma es el plano de control: planes, registro y verificación de cuentas, suscripción, renovación y métricas, bajo el prefijo /plataforma, al que el middleware de inquilinos no asigna tienda. Los demás módulos (catálogo, ventas, caja, inventario, reservas, reportes, sincronización) son el plano de aplicación y operan siempre dentro de una tienda. En el esquema Prisma, el plano de control son los modelos Cuenta, Tienda, Plan, Suscripcion y PagoSuscripcion; los modelos de negocio llevan la columna tienda_id.

3.9.2 Planes y precios

La migración 20261108120000_saas_multitenant carga tres planes que combinan niveles funcionales con límites de uso, facturados en dólares estadounidenses como moneda neutral para un mercado regional; cada tienda conserva su moneda local para operar.

Plan	Precio mensual	Usuarios de equipo	Sucursales	Productos	Módulos principales
Inicial	19 USD	3	1	200	Catálogo, tienda en línea, punto de venta, inventario, reservas
Crecimiento	49 USD	10	3	2000	Lo anterior más probador virtual, reportes con IA y asistente IA
Cadena	129 USD	50	20	Sin límite	Lo anterior más analítica avanzada y precios mayoristas

El enumerado PeriodoPlan define MENSUAL, TRIMESTRAL y ANUAL. El archivo backend/src/plataforma/precios.ts fija 1, 3 y 12 meses con descuentos de 0 %, 10 % y 20 %, y calcula el precio del período como precio mensual por meses por uno menos el descuento. El plan Inicial cuesta así 19 USD por mes, 51,30 por trimestre o 182,40 por año.

Se descartó el cobro por usuario puro porque incentivaría compartir la clave de caja, y el cobro por consumo porque la tienda pequeña valora un costo previsible; la cantidad de usuarios funciona como límite de cada nivel. No se implementa freemium ni prueba gratuita: la landing ofrece la tienda de demostración Vesta para explorar el producto sin registrarse, sin el costo de operar inquilinos que no pagan.

3.9.3 Recorrido de autoservicio

El recorrido se reparte entre frontend/src/plataforma y backend/src/plataforma/plataforma.controller.ts:

1. **Landing (/planes, Planes.tsx):** producto, selector de período con descuentos, planes con límites y módulos, pasos del alta y preguntas frecuentes. Consume GET /plataforma/planes.
2. **Registro (/empezar):** POST /plataforma/registro. Si el correo ya está verificado responde EMAIL_EN_USO; si existe sin verificar, reenvía el enlace.
3. **Verificación (/verificar):** POST /plataforma/verificar consume un token de 24 bytes aleatorios que vence a las 48 horas y devuelve una sesión de cuenta. Sin verificar, POST /plataforma/login responde EMAIL_NO_VERIFICADO y la pantalla ofrece reenviar.
4. **Asistente (/plataforma/nueva, NuevaTienda.tsx):** plan, datos de la tienda y pago. El slug se deriva del nombre y se consulta en GET /plataforma/slug-disponible; el país se elige de una lista hispanohablante con su moneda.

5. **Pago y alta:** POST /plataforma/suscribir cobra, crea la tienda y devuelve una sesión dentro de ella; la dueña entra a su panel sin un segundo login.
6. **Panel de la dueña (/plataforma, MisTiendas.tsx):** lista sus tiendas, permite entrar, ver la vitrina, revisar el uso frente a los límites, ver el historial de pagos y renovar o cambiar de plan.

El correo se envía por SMTP con nodemailer, compatible con Azure Communication Services. Sin credenciales SMTP, el servicio opera en modo simulado y la respuesta incluye el enlace para completar la demostración; la regla de negocio no cambia: sin verificación no hay ingreso ni suscripción.

3.9.4 Pago, idempotencia y compensación

El cobro se abstrae en la clase PasarelaSuscripcion (backend/src/plataforma/pasarela-suscripcion.ts), un puerto con las operaciones cobrar y reembolsar. Hoy se inyecta PasarelaSimulada, que valida el número con el algoritmo de Luhn y el formato del vencimiento y reproduce las tarjetas de prueba habituales: 4242 4242 4242 4242 se aprueba; las terminadas en 0002, 9995 y 0069 se rechazan por tarjeta rechazada, fondos insuficientes y tarjeta vencida. Pasar a una pasarela real en sandbox consiste en escribir otra implementación del puerto.

La idempotencia opera en dos niveles. El frontend genera una clave con crypto.randomUUID y la reutiliza en los reintentos del mismo intento, renovándola solo tras un rechazo. En el backend, idempotency_key es única en pagos_suscripcion; antes de cobrar, PlataformaService.suscribir busca esa clave y, si ya hay un pago aprobado, no cobra y devuelve la tienda creada marcada como repetida. El alta sigue además el patrón de saga: si el cobro se rechaza, se responde 402 PAGO_RECHAZADO y no se crea nada; si se aprueba, tienda, suscripción, pago y espacio inicial se crean en una sola transacción y, si esta falla, se reembolsa.

```
const cobro = await this.pasarela.cobrar(monto, plan.moneda, dto.pago, dto.idempotencyKey);
if (!cobro.aprobado) throw new HttpException({ codigo: 'PAGO_RECHAZADO' }, 402);
try {
  await TenantContext.enTienda(tiendaId, () =>
    this.prisma.$transaction(async (tx) => { /* tienda, suscripción, pago, aprovisionar */ }),
  );
} catch (error) {
  await this.pasarela.reembolsar(cobro.referencia).catch(() => undefined);
  throw error;
}
```

3.9.5 Aprovisionamiento automático y límites

El método aprovisionar crea, en la misma transacción, una sucursal "Casa matriz", un "Almacén principal", el perfil ADMIN de la cuenta que paga y seis categorías base. Todo corre dentro de TenantContext.enTienda con el identificador recién generado, de modo que la extensión de Prisma asigna la nueva tienda a cada registro sin que el código mencione tienda_id. El alta reutiliza el mismo mecanismo de aislamiento que el resto del sistema y no crea bases ni esquemas ni ejecuta migraciones: es instantánea. El slug se valida con una expresión regular y se rechaza si es una palabra reservada (www, api, admin, plataforma, planes, entre otras), previendo una futura resolución por subdominio.

El servicio PlanLimitesService (backend/src/tenancy/plan-limites.service.ts) calcula el uso de la tienda (usuarios de equipo activos, sucursales y productos) y lo compara con el plan vigente. Se invoca en el servidor antes de crear o reactivar usuarios de equipo, crear sucursales y crear productos, incluidos los que llegan por la sincronización offline y por el portal de proveedores. Al superar el límite responde 403 LIMITE_PLAN con el recurso, el máximo y el plan. Las clientas compradoras no cuentan, porque limitarlas contradiría el objetivo de la tienda.

3.9.6 Renovación, cambio de plan y vencimiento

POST /plataforma/tiendas/:id/renovar solo lo invoca la cuenta dueña (campo duenoCuentaId), con la misma idempotencia y compensación. Renovar el mismo plan extiende desde el vencimiento vigente, así que pagar antes no hace perder días. Cambiar de plan cancela

la suscripción vigente y la nueva rige de inmediato con sus límites; antes de cobrar, el servidor verifica que el uso actual quepa en el plan destino o responde PLAN_INSUFICIENTE.

TiendasService considera vigente una tienda activa con una suscripción activa no vencida, y guarda el resultado en una caché de 30 segundos que se invalida al suscribir o renovar. Si una tienda no está vigente, toda petición de su espacio recibe 402 con SUSCRIPCION_VENCIDA, o TIENDA_SUSPENDIDA si su estado es suspendida. Los datos se conservan y la tienda vuelve a funcionar al renovar.

3.9.7 Identidad: una cuenta, varias tiendas

Se adopta la cuenta global con membresías. El modelo Cuenta guarda la identidad: correo único, hash bcrypt de la clave, correo verificado, token de verificación y el indicador esPlataforma del operador del SaaS. El modelo Usuario es el perfil de una cuenta dentro de una tienda (rol, sucursal, estado activo), con unicidad en (cuenta_id, tienda_id) y (tienda_id, email). La migración convierte a cada usuario existente en una cuenta con el mismo identificador y traslada la clave a cuentas, de modo que nadie pierde el acceso.

Hay dos tipos de token JWT (backend/src/auth/jwt-payload.ts): el de cuenta identifica a la persona y sirve al plano de control; el de tienda identifica el perfil, su rol, la tienda activa y la cuenta. CuentaAuthGuard acepta ambos, lo que permite consultar GET /auth/mis-tiendas e invocar POST /auth/cambiar-tienda desde dentro de una tienda; este verifica que la tienda destino esté vigente y que la cuenta tenga un perfil activo allí, y emite un nuevo token de tienda sin pedir la clave. JwtAuthGuard no confía solo en la firma: comprueba que el token no esté revocado, que la tienda siga vigente y que el perfil siga activo con el mismo rol y la misma cuenta.

En la interfaz, SelectorTienda (frontend/src/shared/SelectorTienda.tsx) muestra en la barra lateral las tiendas de la cuenta con el rol en cada una y las vencidas deshabilitadas. Al cambiar, la aplicación recarga porque los datos en memoria pertenecen a la tienda anterior, y fijarTienda (frontend/src/tienda.ts) descarta los datos locales de la tienda, como carrito, favoritos y catálogos offline. Si una cuenta intenta ingresar a una tienda donde no tiene perfil, el login responde SIN_ACCESO_TIENDA; si una clienta se registra en otra tienda con un correo existente, se reutiliza la cuenta cuando la clave coincide; si una administradora da de alta a un empleado ya registrado, se agrega la membresía sin cambiar su clave.

3.9.8 Métricas de la plataforma

GET /plataforma/admin/metricas está protegido por CuentaAuthGuard y PlataformaGuard, que exige esPlataforma; la página /plataforma/admin (Metricas.tsx) presenta los resultados. PlataformaService.metricas calcula:

- **MRR:** suma, sobre la suscripción vigente de cada tienda, del monto pagado entre los meses del período; **ARR** es MRR por 12 y **ARPU** es MRR entre tiendas vigentes.
- **Churn mensual:** tiendas cuya última suscripción venció en los últimos 30 días sin renovarse, entre esas bajas más las tiendas vigentes. Es churn de clientes, no de ingresos.
- **Series:** ingresos y altas de los últimos 12 meses, distribución por plan y período, y renovaciones de los próximos 30 días.
- **Proyección a 6 meses:** MRR del mes siguiente igual al actual por uno menos el churn, más las altas promedio de los últimos 3 meses por el ARPU; los ingresos esperados suman las renovaciones de cada mes, ponderadas por la probabilidad de seguir activas, más las altas estimadas. La respuesta incluye estos supuestos en texto.

LTV y CAC no se calculan: el LTV podría derivarse de los datos existentes, pero el CAC exige registrar el gasto de captación, que hoy no se registra. Se documentan como métricas conceptuales.

3.9.9 Verificación automatizada

El script backend/scripts/verificar-saas.ts (npm run verificar:saas) recorre el ciclo completo contra la API en ejecución con quince comprobaciones: planes y precios por período, bloqueo antes de verificar el correo, slug ocupado, tarjeta rechazada sin tienda creada, alta con sucursal, almacén y administradora, reintento idempotente sin doble cobro, aislamiento de catálogos, productos y reportes, acceso denegado a una tienda ajena, límites del plan Inicial, una cuenta en dos tiendas con cambio sin reingreso, vista de la dueña, elevación del límite al subir de plan, respuestas 402 y 404 para tiendas vencidas o inexistentes y acceso restringido a las métricas. Al terminar borra los datos de prueba.

3.9.10 Síntesis de decisiones

Decisión	Alternativa descartada	Justificación
SaaS vertical para moda	Punto de venta genérico	El valor está en las reglas del rubro (variantes, reservas, probador)
Niveles con límites	Por usuario o por consumo	Precio previsible; no incentiva compartir claves
Períodos de 1, 3 y 12 meses con descuento	Solo mensual	Pedido del docente; mejora caja y retención
Tienda demo	Freemium	Evita operar inquilinos que no pagan
Pasarela detrás de un puerto	Llamada directa a la pasarela	Cambiar de simulada a real sin tocar el alta
Idempotencia por clave única	Confiar en que no se repita	Doble clic y red inestable son casos reales
Saga con reembolso	Transacción distribuida	No hay transacción entre pasarela y base
Cuenta global con membresías	Login por tienda	Un login, rol por tienda, cambio sin reingreso
Plano de control separado	Mezclar plataforma y negocio	Las operaciones globales quedan acotadas

3.9.11 Limitaciones reconocidas

- La pasarela es simulada; la integración real corresponde al capítulo de formas de pago.
- Los módulos de cada plan se publican en la landing y en GET /tienda, pero el backend no bloquea un módulo no contratado; el cumplimiento se basa en los límites cuantitativos.
- No hay baja voluntaria ni borrado automático tras el vencimiento: la baja ocurre por no renovar y los datos se conservan. El script backend/scripts/lib/borrar-tienda.ts elimina una tienda completa, pero solo se usa en pruebas.
- El cambio de plan no prorratea los días no consumidos, y no existe una pantalla del operador para suspender tiendas aunque el estado suspendida se respete.
- Varias pantallas del frontend aún muestran montos con el símbolo Bs fijo, aunque la tienda tenga otra moneda.

4. Organización de inquilinos (multi-tenant)

El capítulo anterior estableció que EnsayAR se ofrece como un servicio que muchas tiendas contratan. Este capítulo responde la pregunta técnica central de ese modelo: cómo una sola aplicación atiende a muchas tiendas sin que ninguna vea ni altere los datos de otra. Se define qué es un inquilino, se comparan los modelos de aislamiento, se explica cómo se identifica al inquilino en cada petición y cómo ese dato viaja por el código de un servidor Node.js, se describen las capas de defensa disponibles en NestJS, Prisma y PostgreSQL, y se cierra con la estrategia adoptada en EnsayAR y su justificación.

4.1 Qué es un inquilino

Un inquilino es la unidad de aislamiento de una solución SaaS: el cliente que contrata el servicio, junto con sus usuarios, su configuración y sus datos. La palabra viene de la analogía inmobiliaria: muchos inquilinos ocupan un mismo edificio, comparten estructura, agua y ascensores, pero cada uno tiene su departamento cerrado con llave.

La guía de modelos de inquilinato del Azure Architecture Center recomienda definir explícitamente qué es un inquilino antes de diseñar, porque no siempre coincide con un usuario. En soluciones de empresa a empresa, el inquilino suele ser una organización con muchos usuarios; en soluciones de empresa a consumidor, puede ser una persona o una familia. La misma guía distingue al inquilino lógico del despliegue físico donde residen sus datos, porque una solución puede crecer hasta tener varios despliegues y necesitar saber en cuál vive cada inquilino.

En EnsayAR, el inquilino es la **tienda**, representada por el modelo Tienda. Una tienda tiene sucursales, almacenes, productos, ventas, clientas y empleados. La persona no es el inquilino: una misma persona, representada por una Cuenta, puede pertenecer a varias tiendas

con un perfil Usuario distinto en cada una. Separar persona e inquilino es lo que permite el login único descrito en el capítulo 3.

4.2 El aislamiento como espectro

El aislamiento no es una propiedad binaria. Azure lo describe como un espectro que va desde "no compartir nada" (cómputo, red, base de datos y dominio separados por inquilino) hasta "compartir todo", con posiciones intermedias donde, por ejemplo, se comparte la aplicación pero cada inquilino tiene su base. Además, el aislamiento tiene varias dimensiones que conviene separar:

- **Datos:** que un inquilino no pueda leer ni modificar registros de otro.
- **Rendimiento:** que la carga de un inquilino no degrade a los demás (vecino ruidoso).
- **Identidad:** que un usuario autenticado en un inquilino no pueda actuar en otro sin autorización.
- **Operación:** que un fallo, un despliegue o una restauración afecte al menor número posible de inquilinos.

Cada dimensión puede resolverse con un grado de aislamiento distinto. Un SaaS puede compartir el cómputo, aislar lógicamente los datos y aplicar límites de uso por inquilino para el rendimiento. Lo importante es que la elección sea consciente y esté justificada.

4.3 Modelos de aislamiento de datos

En la capa de datos se reconocen tres modelos clásicos.

4.3.1 Base de datos por inquilino

Cada inquilino tiene su propia base de datos y la aplicación elige la conexión según el inquilino de la petición. Es el aislamiento lógico más fuerte que se puede lograr sin duplicar la

aplicación: un error de filtrado no puede exponer datos ajenos porque la conexión ni siquiera alcanza la otra base. Permite respaldar, restaurar, mover o borrar un inquilino de forma independiente, y aislar el rendimiento de los clientes más pesados. Sus costos son altos: cada base consume recursos aunque el inquilino esté inactivo, las migraciones de esquema deben aplicarse a cada base y coordinar sus versiones, el alta requiere crear y migrar una base, y los reportes globales del proveedor deben consultar muchas bases.

4.3.2 Esquema por inquilino

Todos los inquilinos comparten un servidor y una base, pero cada uno tiene su propio esquema con sus propias tablas. En PostgreSQL esto se apoya en el search_path de la sesión. El aislamiento es intermedio: las tablas están separadas, pero comparten servidor, memoria y conexiones. Es más barato que una base por inquilino, pero conserva el problema de las migraciones repetidas por esquema y el catálogo de la base crece con cada inquilino. Además, muchos ORM, entre ellos Prisma, están pensados para trabajar con un esquema conocido por conexión, lo que complica cambiar de esquema en cada petición.

4.3.3 Base compartida con identificador de inquilino

Todos los inquilinos comparten las mismas tablas y cada fila lleva una columna con el identificador del inquilino, típicamente llamada tenant_id. Es el modelo más eficiente: una sola base, una sola migración, alta instantánea porque solo se insertan filas, y reportes globales con una consulta. Su riesgo también es evidente: la separación depende de que **todas** las consultas filtren por esa columna. Un solo olvido produce una fuga de datos entre clientes. Restaurar a un solo inquilino es más difícil: Azure señala que, en una base compartida, suele requerir restaurar la base completa en otro recurso y recuperar selectivamente los datos de ese inquilino. Y el

vecino ruidoso es más probable, porque todos comparten los mismos índices y la misma capacidad.

4.3.4 Terminología de AWS y Azure

El AWS Well-Architected SaaS Lens nombra tres modelos de aislamiento aplicables a cada capa o servicio:

- **Silo:** recursos dedicados por inquilino. AWS lo considera una variante válida de SaaS siempre que identidad, alta, métricas, despliegue y operación sigan siendo compartidos; intercambia economía de escala por cumplimiento normativo o por requisitos del dominio.
- **Pool:** recursos compartidos por todos los inquilinos, con almacenamiento indexado por identificador de inquilino. Aporta agilidad, eficiencia de costo y una operación unificada, pero AWS advierte que la dificultad de aislar no autoriza a relajar los requisitos de aislamiento, y menciona como desventajas el vecino ruidoso y la dificultad de atribuir costos por inquilino.
- **Bridge:** combinación de ambos según la capa. Por ejemplo, una capa web compartida y una capa de negocio y almacenamiento por inquilino.

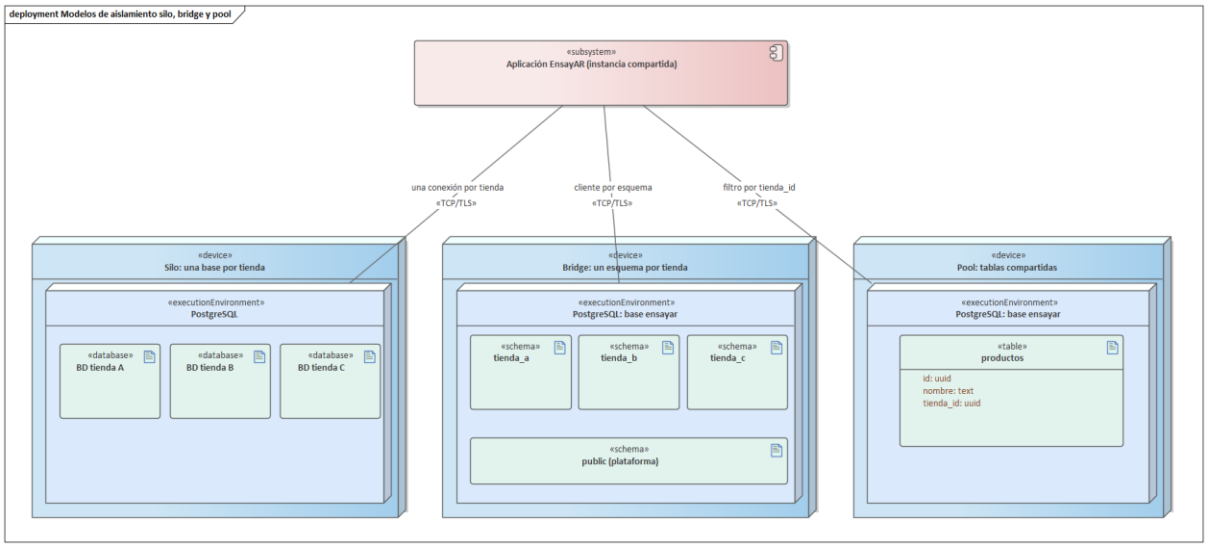
Azure usa nombres equivalentes: despliegues automatizados de un solo inquilino, despliegues completamente multi-inquilino, despliegues particionados verticalmente (la mayoría de los inquilinos en infraestructura compartida y algunos en dedicada) y particionados horizontalmente (algunas capas compartidas y otras dedicadas, por ejemplo una aplicación común con bases por inquilino). Con esta terminología, la base por inquilino es un silo en la capa de datos, el esquema por inquilino es una forma de bridge dentro de un mismo servidor, y la base compartida con tenant_id es pool.

4.3.5 Comparación

Criterio	Base por inquilino (silo)	Esquema por inquilino (bridge)	Base compartida con tenant_id (pool)
Aislamiento de datos	Fuerte: conexión distinta	Medio: tablas distintas, mismo servidor	Lógico: depende del filtrado y de RLS
Costo por inquilino	Alto, incluso inactivo	Medio	Muy bajo
Alta de un inquilino	Crear y migrar una base	Crear y migrar un esquema	Insertar filas, instantánea
Escalabilidad en cantidad de inquilinos	Limitada por la gestión de muchas bases	Limitada por el catálogo y las migraciones	Alta; escala con índices y particionado
Migraciones de esquema	Una por base, con riesgo de versiones mezcladas	Una por esquema	Una sola
Respaldo y restauración por inquilino	Directa	Posible por esquema	Compleja: restauración selectiva
Vecino ruidoso	Bien aislado en datos	Parcial	Mayor riesgo; requiere límites y monitoreo
Cumplimiento y residencia de datos	Más sencillo de demostrar	Intermedio	Requiere controles y auditoría adicionales
Reportes globales del proveedor	Consultas a muchas bases	Consultas a muchos esquemas	Una consulta

Figura 4.1

Modelos de aislamiento silo, bridge y pool



Nota. Diagrama de despliegue UML 2.5: los tres comparten la aplicación; cambia la base, el esquema o la tabla que guarda los datos de cada tienda.

4.4 Resolución del inquilino en cada petición

Antes de filtrar datos hay que saber a qué inquilino pertenece cada petición. La guía de mapeo de peticiones del Azure Architecture Center describe varios mecanismos:

- **Subdominio o dominio:** luna.ensayar.app identifica a la tienda Luna a partir de la cabecera Host. Es natural para el usuario y facilita la marca blanca, pero exige DNS comodín, certificados y una forma de elegir tienda desde una página de entrada común.
- **Ruta:** ensayar.app/t/luna. No requiere DNS y es fácil de compartir, pero todas las rutas de la aplicación deben llevar el prefijo.
- **Cabecera HTTP:** una cabecera propia, como X-Tienda. Azure advierte que no sirve para peticiones GET que el navegador emite por sí solo ni cuando un proxy puede eliminar cabeceras; es apropiada cuando el proveedor controla el cliente, como una SPA o una app móvil que llaman a una API.
- **Claims del token:** el token de acceso incluye el identificador del inquilino. Es el mecanismo más fuerte, porque el dato viene firmado por el servidor y no puede alterarse sin invalidar la firma.

La misma guía agrega dos advertencias. La primera: aunque la petición traiga un identificador de inquilino, la aplicación debe validar que el usuario esté autorizado para ese inquilino; el identificador no concede acceso por sí solo. La segunda: la resolución ocurre en cada petición, por lo que debe ser barata; si requiere consultar la base, conviene usar caché.

En la práctica se combinan mecanismos con un orden de precedencia: el token manda cuando hay sesión, y el subdominio, la ruta o la cabecera solo eligen tienda para las pantallas públicas sin sesión, como la vitrina o el login.

4.5 Propagación del contexto en Node.js

Una vez resuelto el inquilino, el dato debe llegar a todas las consultas que la petición ejecute, incluidas las que ocurren en servicios profundos que no reciben el objeto de la petición. En lenguajes con un hilo por petición se usa almacenamiento local de hilo. Node.js atiende miles

de peticiones concurrentes en un solo hilo con un bucle de eventos: mientras una petición espera a la base, otra avanza. Una variable global no sirve, porque la tienda de una petición sería sobrescrita por la siguiente.

Hay tres alternativas. La primera es pasar el identificador como parámetro a cada función, lo que es explícito pero contamina todas las firmas y es fácil de olvidar. La segunda, propia de NestJS, son los proveedores con alcance de petición: el contenedor crea una instancia por petición con acceso al objeto Request, pero ese alcance se propaga a todos los proveedores que dependen de él y multiplica las instancias creadas. La tercera es **AsyncLocalStorage**, del módulo async_hooks de Node.js, marcado como estable en la documentación oficial. Su método run(store, callback) ejecuta el callback con un almacén asociado, y getStore() devuelve ese almacén desde cualquier operación asíncrona iniciada dentro del callback, sin importar cuántas llamadas o await haya en medio. Cada petición obtiene así su propio contexto, aislado del de las demás.

```
const als = new AsyncLocalStorage<TenantStore>();

export const TenantContext = {
  run<T>(store: TenantStore, fn: () => T) { return als.run(store, fn); },
  tiendaId() { return als.getStore()?.tiendaId; },
  enTienda<T>(tiendaId: string, fn: () => T | PromiseLike<T>) {
    return als.run({ tiendaId }, async () => await fn());
  },
  sinFiltro<T>(fn: () => T | PromiseLike<T>) {
    return als.run({ sinFiltro: true }, async () => await fn());
  },
};
```

El fragmento resume backend/src/tenancy/tenant-context.ts. Los métodos enTienda y sinFiltro abren contextos explícitos para el código que no nace de una petición HTTP, como el alta de una tienda, la semilla de datos o las métricas del proveedor. Ambos esperan el resultado dentro del contexto, porque las consultas de Prisma son perezosas y se ejecutan al ser esperadas.

4.6 Capas de defensa

En el modelo pool, una omisión de filtro es una fuga. Por eso se aplica defensa en profundidad: varias capas independientes, de modo que el fallo de una no exponga datos.

4.6.1 Filtro centralizado en la aplicación

La primera capa es que ningún desarrollador tenga que recordar el filtro. AWS, en su documento sobre estrategias de aislamiento de inquilinos, recomienda ocultar los detalles del aislamiento en bibliotecas compartidas, porque el modelo pool depende de que los desarrolladores sigan la convención y una convención manual termina fallando. En Prisma esto se logra con una extensión de cliente del tipo query: la documentación de Prisma muestra que $allModels y $allOperations permiten interceptar todas las operaciones de todos los modelos y modificar sus argumentos antes de ejecutarlas. La extensión agrega el filtro por inquilino a lecturas, actualizaciones y borrados, y asigna el inquilino a las altas. El principio clave es **fallar cerrado**: si no hay inquilino en el contexto, la consulta se rechaza en lugar de ejecutarse sin filtro.

4.6.2 Valores por defecto y restricciones en la base

La segunda capa traslada parte de la regla a PostgreSQL. La columna del inquilino puede declararse con un valor por defecto tomado de una variable de configuración de la sesión, de modo que una inserción que omita la columna reciba el inquilino fijado en la transacción. La función current_setting(nombre) lanza un error si la variable no existe, salvo que se le pase missing_ok en verdadero, en cuyo caso devuelve NULL. Usada sin missing_ok en un valor por defecto, convierte la ausencia de contexto en un error de inserción en vez de una fila huérfana. Las restricciones de unicidad deben, además, incluir la columna del inquilino: un SKU o el nombre de una categoría deben ser únicos dentro de cada tienda, no en toda la plataforma.

4.6.3 Row-Level Security de PostgreSQL

La tercera capa es la seguridad a nivel de fila. Según la documentación de PostgreSQL, al habilitar RLS en una tabla todo acceso normal debe estar permitido por una política; si no hay ninguna, se aplica una denegación por defecto. Cada política define una expresión booleana con USING para las filas visibles y WITH CHECK para las filas que se insertan o modifican.

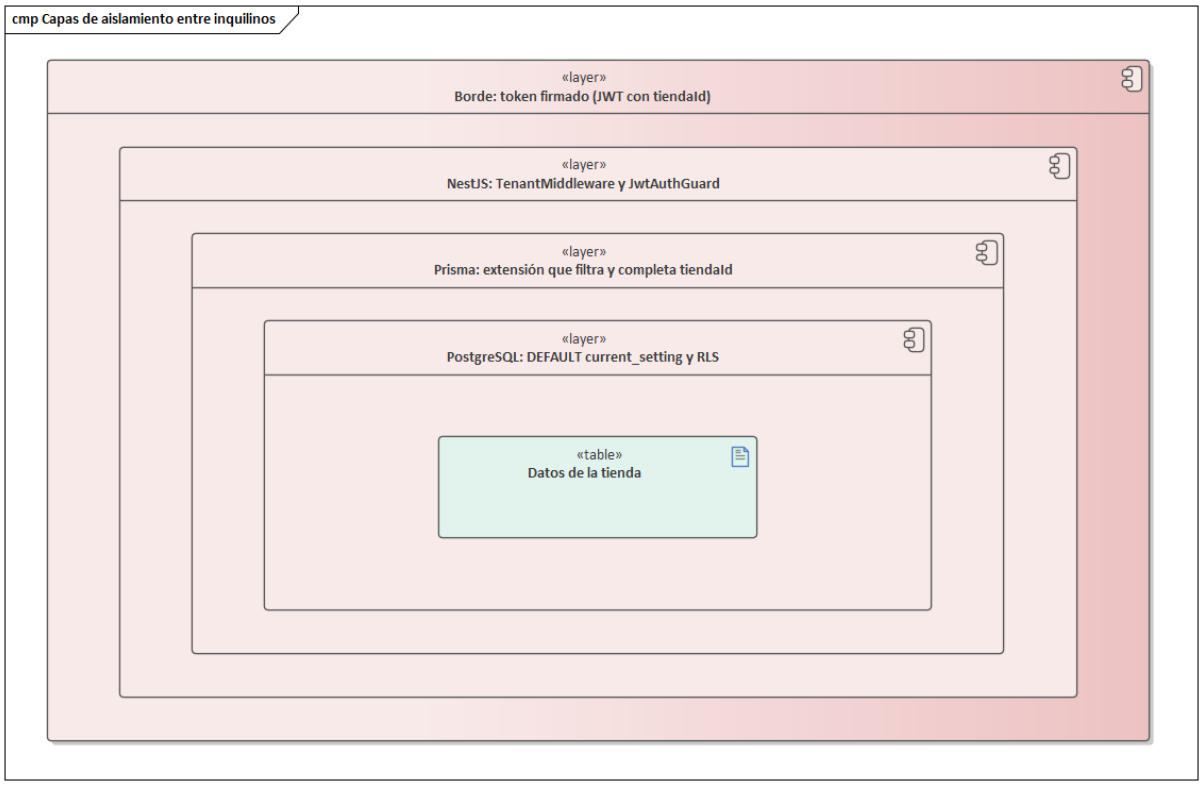
```
ALTER TABLE productos ENABLE ROW LEVEL SECURITY;
ALTER TABLE productos FORCE ROW LEVEL SECURITY;
CREATE POLICY aislamiento_tienda ON productos
  USING (tienda_id = current_setting('app.tienda_id', true))
  WITH CHECK (tienda_id = current_setting('app.tienda_id', true));
```

La documentación advierte detalles que deben tenerse en cuenta. Los superusuarios y los roles con atributo BYPASSRLS siempre omiten las políticas, y el dueño de la tabla también, salvo que se use FORCE ROW LEVEL SECURITY. Las verificaciones de integridad referencial, como claves únicas y foráneas, no pasan por RLS y pueden filtrar información por canales encubiertos. Por eso la aplicación debe conectarse con un rol sin BYPASSRLS, y las migraciones y las tareas de plataforma con otro rol o con una política de excepción explícita.

El punto delicado con un ORM es el pool de conexiones: cada consulta puede usar una conexión distinta. El ejemplo oficial de Prisma para RLS resuelve esto fijando la variable con set_config en modo local y ejecutando la consulta en la misma transacción por lotes, de modo que comparten conexión y la variable desaparece al terminar. El propio ejemplo aclara que no está pensado para producción y que las transacciones explícitas pueden no comportarse como se espera. Azure, por su parte, señala que muchas soluciones multi-inquilino no usan RLS por la complejidad de diseñarla, probarla y mantenerla. RLS es la defensa más fuerte del modelo pool, pero su costo de adopción es real.

Figura 4.2

Capas de aislamiento entre inquilinos en EnsayAR



Nota. Diagrama de componentes UML 2.5: cada capa repite la restricción por tienda, de modo que la falla de una no expone datos de otra tienda.

4.7 Resolución paso a paso en NestJS con Prisma

La pregunta "cómo se resuelve multi-tenant en una aplicación con Nest" tiene una respuesta en cinco pasos. Se ilustra con el código real de EnsayAR.

Paso 1. Abrir el contexto en un middleware global. NestJS permite registrar middleware de clase con inyección de dependencias en el método configure del módulo raíz; con forRoutes('*') se aplica a todas las rutas. El middleware abre el contexto de AsyncLocalStorage y ejecuta todo el resto de la petición dentro de él. Según la documentación de Nest, un error pasado a next llega a la capa de excepciones, que responde el código HTTP correspondiente.

```
use(req: Request, _res: Response, next: NextFunction) {
  const store: TenantStore = {};
  TenantContext.run(store, () => {
    this.resolver(req, store).then(() => next(), next);
  });
}
```

Paso 2. Resolver la tienda. El método resolver de backend/src/tenancy/tenant.middleware.ts aplica la precedencia de 4.4. Si la ruta empieza con /plataforma, no asigna tienda, porque es el plano de control. Si hay un token de tienda válido, manda la tienda del token, y la cabecera no puede cambiarla. Si no hay token, usa la cabecera X-Tienda con el slug y, en su defecto, la tienda configurada en TIENDA_POR_DEFECTO. Una cabecera con un slug inexistente produce 404 TIENDA_NO_EXISTE; una tienda sin suscripción vigente produce 402. La búsqueda pasa por TiendasService, con una caché de 30 segundos para no consultar la base en cada petición.

Paso 3. Confirmar la tienda en el guard de autenticación. JwtAuthGuard verifica la firma y la revocación del token, exige que sea de tipo tienda, comprueba que la tienda siga vigente y fija la tienda en el contexto con TenantContext.fijar. Luego consulta el perfil Usuario, ya filtrado por esa tienda, y exige que siga activo con el mismo rol y la misma cuenta que dice el token. Así, aunque alguien conserve un token viejo, un cambio de rol o una baja surten efecto de inmediato.

Paso 4. Aplicar la tienda en todas las consultas. El proveedor PrismaService no es un PrismaClient directo, sino el cliente extendido por crearPrismaTenant (backend/src/prisma/prisma.module.ts). La extensión de backend/src/tenancy/prisma-tenant.extension.ts calcula, a partir del metamodelo de Prisma, el conjunto de modelos con campo tiendaId que no son de plataforma, y en cada operación hace lo siguiente:

- Si el contexto es sinFiltro, ejecuta la consulta tal cual; solo lo usa el código de plataforma.

- Si el modelo debe filtrarse y no hay tienda en el contexto, lanza TENANT_REQUERIDO: falla cerrada.
- En lecturas, conteos, agregaciones y borrados, agrega tiendaId al where, incluso en findUnique, que en Prisma 6 admite campos no únicos adicionales en el filtro.
- En altas, inyecta tiendaId en los datos y lo propaga a las altas anidadas, como las variantes creadas junto con un producto.
- En actualizaciones y upsert, agrega el filtro y elimina tiendaId de los datos, para que ningún registro pueda "mudarse" a otra tienda.

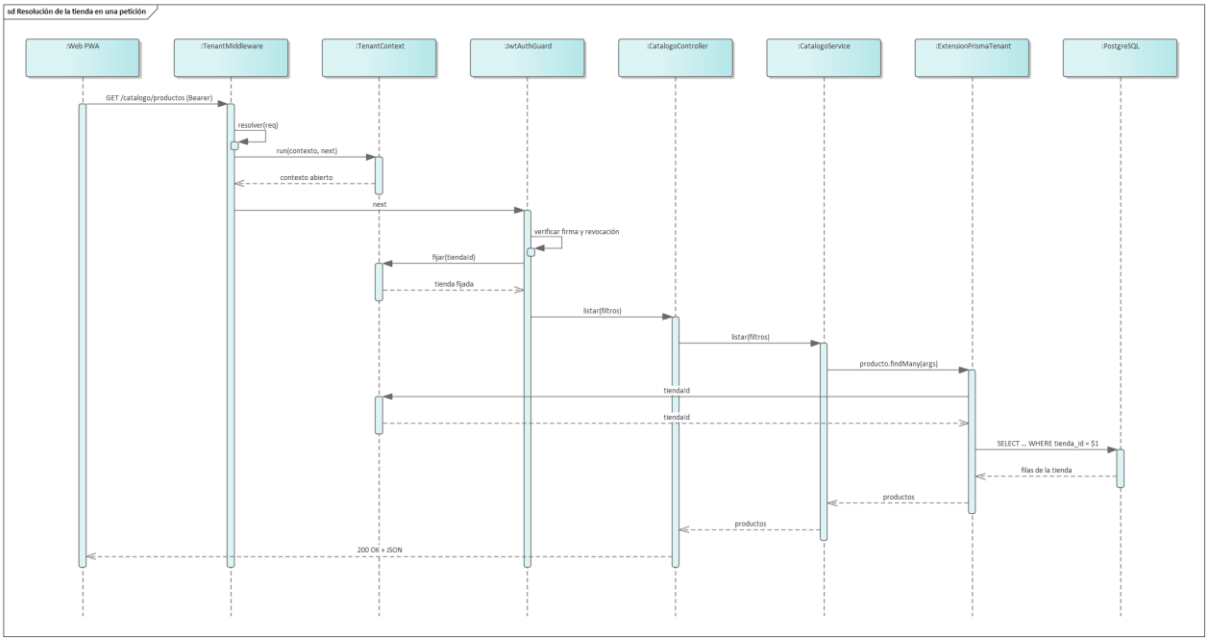
```
const filtra = MODELOS_CON_TIENDA.has(model);
const tiendaId = store?.tiendaId;
if (!tiendaId) {
  if (filtra) throw new Error(`TENANT_REQUERIDO: ${model}.${operation}`);
  return query(args);
}
if (LECTURAS_Y_BORRADOS.has(operation) && filtra) a.where = { ...a.where, tiendaId };
else if (operation === 'create') a.data = conCreate(model, a.data, tiendaId);
// createMany, update, updateMany y upsert siguen la misma idea
```

Paso 5. Validar identificadores que llegan del cliente. Las claves foráneas no conocen de tiendas: un identificador válido de una variante de otra tienda pasaría la restricción referencial. Por eso, antes de escribir con identificadores recibidos, la función exigirDeLaTienda (backend/src/tenancy/pertenencia.ts) cuenta el registro a través del cliente extendido; como la consulta ya está filtrada, un identificador ajeno da cero y se responde 404 REFERENCIA_NO_EXISTE.

Con estos cinco pasos, los servicios de negocio escriben consultas como si la aplicación tuviera una sola tienda; la tienda se resuelve una vez en el borde y se aplica de forma transversal.

Figura 4.3

Resolución de la tienda en una petición GET /catalogo/productos



Nota. Diagrama de secuencia UML 2.5: el guard fija la tienda en el contexto y la extensión de Prisma la aplica en la consulta.

4.8 SQL crudo con set_config

Las extensiones de consulta de Prisma interceptan las operaciones de los modelos, pero el SQL crudo (queryRaw y executeRaw) no tiene un modelo al que agregar el filtro. Para estos casos se usan dos técnicas.

La primera es filtrar explícitamente: las actualizaciones de stock en ventas y de saldo en créditos incluyen la condición tienda_id igual a la tienda del contexto dentro del propio UPDATE, que además verifica atómicamente la cantidad disponible.

La segunda es fijar la tienda en la sesión de PostgreSQL. Según la documentación oficial, set_config(nombre, valor, is_local) con is_local en verdadero aplica el valor solo durante la

transacción actual. Los reportes ejecutan, en una misma transacción por lotes de Prisma, el set_config y luego la consulta:

```
const tiendaId = TenantContext.tiendaIdOrThrow();
const [, filas] = await this.prisma.$transaction([
  this.prisma.$queryRaw`SELECT set_config('app.tienda_id', ${tiendaId}, true)`,
  this.prisma.$queryRawUnsafe<T[]>(sql),
]);
```

Del lado de la base, la vista v_reporte_ventas, creada en la migración 20261108120000_saas_multitenant, filtra con tienda_id igual a current_setting('app.tienda_id', true). Sin tienda fijada, current_setting devuelve NULL, la comparación nunca es verdadera y la vista no devuelve filas: también falla cerrada. El reporte generativo no ejecuta SQL libre: traduce una especificación validada a SQL parametrizado, y cada una de sus fuentes (backend/src/reportes/inteligente/catalogo.ts) incorpora la misma condición sobre current_setting como filtro base. En el módulo de reportes donde una consulta puede llegar como texto SQL, el validador backend/src/reportes/sql-seguro.ts exige una sola sentencia SELECT, prohíbe palabras como set_config, current_setting, set o reset y cualquier referencia a tablas reales, y solo admite la vista ya filtrada. Sin esa validación, una instrucción maliciosa podría fijar otra tienda antes de consultar.

4.9 Pruebas de aislamiento

Azure recomienda probar explícitamente que los datos de un inquilino no se filtran a otro, sea cual sea el modelo elegido. Las pruebas de aislamiento tienen dos niveles.

En el nivel de unidad de persistencia se ejercita la extensión directamente contra la base. El script backend/scripts/verificar-tenancy.ts (npm run verificar:tenancy) comprueba que al menos 30 modelos llevan tienda_id, que una tienda vacía ve cero productos, que sin tienda en el contexto la consulta falla con TENANT_REQUERIDO, que las altas anidadas heredan la tienda, que un producto de otra tienda es invisible aunque se conozca su identificador, que updateMany

desde otra tienda afecta cero filas, que no se puede "mudar" un registro cambiando su tiendaId y que la vista de reportes no ve filas de otra tienda.

En el nivel de API, backend/scripts/verificar-saas.ts crea dos tiendas reales por el flujo de suscripción y verifica por HTTP que los catálogos y reportes de una no muestran datos de la otra, que con sesión de una tienda la cabecera X-Tienda de otra no cambia nada, que un producto ajeno pedido por identificador responde 404, que la administradora de una tienda no puede iniciar sesión en otra y que una tienda vencida responde 402. Estas pruebas reproducen el escenario que el docente anunció para la defensa: crear varias tiendas y verificar que cada una mantiene su espacio.

Una buena práctica adicional es incorporar estas pruebas a la integración continua, de modo que cualquier modelo nuevo sin tienda_id o cualquier consulta cruda sin filtro rompa la construcción antes de llegar a producción.

4.10 Evolución de la estrategia

El modelo pool no es un destino final sino un punto de partida que puede evolucionar con el negocio.

RLS completa. El siguiente paso de endurecimiento es habilitar y forzar RLS con una política por cada tabla de negocio, conectar la aplicación con un rol sin BYPASSRLS y ejecutar cada operación con la tienda fijada mediante set_config local. Con ello, incluso una consulta cruda sin filtro o un error futuro en la extensión quedarían contenidos por la base. El costo es envolver las operaciones en transacciones, prever un rol o una política de excepción para el plano de control y las migraciones, y revisar las transacciones interactivas existentes.

Mover un inquilino grande a su propia base. Si una cadena crece hasta convertirse en vecino ruidoso o exige aislamiento contractual, puede pasar a un despliegue particionado

verticalmente: su propia base, con la misma aplicación. Se necesita un catálogo de inquilinos que guarde la ubicación de cada uno y que la resolución de 4.4 elija la conexión. Como cada fila ya lleva tienda_id, extraer los datos de un inquilino es una exportación filtrada por esa columna. Esta evolución puede venderse como un plan dedicado de mayor precio, tal como sugiere Azure.

Sharding. Con muchos miles de tiendas, una sola base deja de alcanzar. El patrón de sharding distribuye los inquilinos entre varias bases, cada una con uno o más inquilinos, mediante un mapa de shards. Azure distingue este patrón de los deployment stamps: los shards dividen solo la capa de datos sin duplicar el resto de la infraestructura. La columna tienda_id es la clave natural de distribución, y existen extensiones de PostgreSQL que distribuyen tablas por una columna de este tipo.

Vecino ruidoso. En paralelo, conviene limitar la tasa de peticiones por tienda, medir el consumo por inquilino y aplicar los patrones de limitación que Azure recomienda para bases compartidas.

4.11 Estrategia elegida en EnsayAR y justificación

4.11.1 Decisión

EnsayAR adopta el modelo **pool: base PostgreSQL compartida con columna tienda_id**, protegido por defensa en profundidad. Las tablas de plataforma (cuentas, tiendas, planes, suscripciones, pagos_suscripcion y tokens_revocados) no se filtran por tienda; suscripciones guarda tienda_id solo como clave foránea hacia la tienda que paga, porque pertenece al plano de control. Las 39 tablas de negocio del esquema actual sí la llevan: las 36 que convierte la migración 20261108120000_saas_multitenant y tres agregadas en migraciones posteriores (cuentas por cobrar, abonos de crédito y suscripciones de notificaciones push). Esa migración asigna los datos existentes a la tienda Vesta, crea índices por tienda_id y reemplaza las

unicidades globales por unicidades por tienda, por ejemplo (tienda_id, sku) en variantes, (tienda_id, codigo) en cupones y (tienda_id, email) en usuarios. La migración 20261108130000_tienda_desde_sesion fija el valor por defecto de tienda_id en current_setting('app.tienda_id') para todas esas tablas.

4.11.2 Las capas en EnsayAR

Capa	Mecanismo	Archivo o recurso
Resolución	Token de tienda, cabecera X-Tienda o tienda por defecto; 402 y 404	backend/src/tenancy/tenant.middleware.ts
Contexto	AsyncLocalStorage con enTienda y sinFiltro	backend/src/tenancy/tenant-context.ts
Autenticación	Guard que fija la tienda del token y revalida el perfil	backend/src/auth/jwt-auth.guard.ts
Filtro de aplicación	Extensión de Prisma que filtra, inyecta y falla cerrada	backend/src/tenancy/prisma-tenant.extension.ts
Pertenencia	Validación de identificadores recibidos	backend/src/tenancy/pertenencia.ts
Base de datos	DEFAULT desde current_setting y unicidades por tienda	migraciones 20261108120000 y 20261108130000
SQL crudo	set_config local, vista filtrada y validador de SQL	reportes.service.ts, v_reporte_ventas, sql-seguro.ts
Pruebas	Scripts de aislamiento y de punta a punta	verificar-tenancy.ts y verificar-saas.ts

En los clientes, la web guarda el slug de la tienda activa (frontend/src/tienda.ts) y lo envía en la cabecera X-Tienda; la ruta /t/:slug cambia de tienda para la vitrina. La app móvil envía la misma cabecera desde su almacenamiento local. Con sesión, en ambos casos manda el token.

4.11.3 Justificación

Alternativa	Motivo para no adoptarla hoy
Base por tienda	El alta debe ser instantánea tras el pago; crear y migrar una base por cada suscripción contradice el autoservicio y multiplica el costo en la nube para tiendas pequeñas
Esquema por tienda como modo único	Prisma trabaja con un esquema por conexión y un historial de migraciones; exige versionar migraciones por esquema y mantener un cliente por tienda. Se implementó como modalidad piloto (sección 4.12), sin reemplazar todavía al pool en producción
Pool sin capas adicionales	Un olvido de filtro sería una fuga; por eso el filtro se centraliza y la base aporta su propia barrera

El perfil del cliente refuerza la decisión: tiendas pequeñas y medianas, con volúmenes modestos y sensibles al precio, para quienes la eficiencia del modelo pool se traduce en planes baratos. El costo de una base por tienda se trasladaría al precio y dejaría a EnsayAR en desventaja frente a competidores. La extensión de Prisma convierte el riesgo principal del pool en una regla automática, y el diseño deja abierta la evolución hacia RLS completa, inquilinos dedicados y sharding sin reescribir los servicios, porque todos los datos ya están marcados con su tienda.

4.11.4 Síntesis de la solución en NestJS

Una arquitectura multi-tenant permite que una sola instancia de la aplicación atienda a varios clientes, denominados inquilinos, con sus datos separados. Los tres modelos de datos habituales son la base por inquilino, el esquema por inquilino y la base compartida con identificador. EnsayAR adopta el tercero porque el alta de una tienda debe ser automática e inmediata y el costo por tienda debe mantenerse bajo.

En NestJS la tienda se resuelve en el borde de la petición y se aplica de forma transversal. Un middleware global abre un contexto con AsyncLocalStorage y obtiene la tienda desde el token o la cabecera X-Tienda; el guard JWT comprueba que coincida con la del token; una extensión de Prisma agrega tienda_id a todas las lecturas y escrituras y rechaza la operación si no hay tienda; la base de datos asigna tienda_id por defecto desde la variable de sesión, y el SQL directo fija esa variable con set_config dentro de una transacción. El aislamiento se comprueba con scripts que crean tiendas reales y verifican que ninguna accede a los datos de otra.

4.11.5 Limitaciones reconocidas

- No hay políticas RLS por tabla: la base aporta el valor por defecto y la vista filtrada, pero no rechaza por sí misma una consulta cruda sin filtro. Es la evolución prioritaria.
- Algunas consultas crudas con SELECT FOR UPDATE de inventario, reservas y pagos filtran solo por identificadores ya validados dentro de la tienda, sin repetir tienda_id en el SQL; con RLS quedarían cubiertas por la base.
- Las tablas de negocio no tienen clave foránea hacia tiendas ni una restricción que impida un tienda_id vacío. Se recomienda agregarlas, porque en una conexión reutilizada donde ya se ejecutó un set_config local, la variable de sesión puede quedar definida con un valor vacío y el valor por defecto dejaría de fallar.
- La resolución por subdominio no está implementada; la tienda se elige por token, cabecera o ruta del frontend.
- La caché de tiendas de 30 segundos es local a cada instancia de la API; con varias réplicas, una suspensión puede tardar hasta ese plazo en aplicarse en todas.

4.12 Evolución implementada: un esquema PostgreSQL por tienda

4.12.1 Motivación

El modelo pool concentra el aislamiento en la aplicación. Para reforzarlo físicamente y permitir el respaldo y la restauración de cada tienda por separado, se implementó una segunda modalidad de aislamiento basada en el modelo bridge de la sección 4.3: una sola base PostgreSQL, con los datos de plataforma en el esquema `public` y los datos operativos de cada tienda en un esquema propio. La modalidad se activa mediante variables de configuración y convive con el modelo pool, que sigue siendo el modo de operación en producción.

4.12.2 Clasificación de los datos

El contrato de `backend/src/tenancy/dominio-ambitos.ts` clasifica los 45 modelos de Prisma en dos ámbitos, y sus pruebas lo contrastan con el esquema generado. Seis modelos son de plataforma y permanecen en `public`: `Cuenta`, `Tienda`, `Plan`, `Suscripcion`, `PagoSuscripcion` y `TokenRevocado`. Los 39 restantes pertenecen a una sola tienda y se ubican en su esquema, desde perfiles, catálogo e inventario hasta ventas, créditos, reportes y sincronización. La relación entre ámbitos se limita a la referencia del perfil local `Usuario` hacia la `Cuenta` global; las demás claves foráneas de negocio apuntan siempre a tablas del mismo esquema.

4.12.3 Componentes

Componente	Responsabilidad
Nombre del esquema	Se deriva del UUID de la tienda con el formato `tienda_<uuid>`, nunca de un texto enviado por el cliente; se rechazan identificadores no válidos
`PrismaEsquemasService`	Mantiene un cliente de Prisma por esquema con un límite de clientes y desalojo del menos usado; nunca desaloja un cliente con operaciones en curso
`AprovisionadorEsquemaService`	Durante el alta crea, en la misma transacción que la tienda, la suscripción y el pago simulado, el esquema con sus 39 tablas, enums, claves, vistas y datos iniciales; ante un error se revierte todo
`TransaccionesMixtasService`	Ejecuta en una sola transacción operaciones de plataforma y de tienda, como un pago global junto con un registro local
Registro de versiones	La tabla `_ensayar_migraciones_tienda` registra qué migraciones se aplicaron a cada esquema
Scripts de migración y auditoría	`migrar-tiendas-aisladas.ts` copia los datos de cada tienda a su esquema sin borrar el origen y `auditar-migracion-esquemas.ts` compara el contenido fila por fila

Los módulos de catálogo, inventario, ventas, caja, pagos simulados, sincronización offline, reservas, pedidos, devoluciones, reportes, créditos, usuarios y analítica se enrutan al esquema de la tienda autenticada cuando la modalidad está activa. La tienda se sigue resolviendo en el middleware y el guard descritos en la sección 4.7; lo único que cambia es el cliente de base de datos que recibe cada servicio.

4.12.4 Verificación

La modalidad se validó en una base de laboratorio independiente de la de producción. Se respaldó la base con `pg_dump` y se restauró en una base aislada, con 28 esquemas de tienda íntegros. Las pruebas de punta a punta comprobaron que las escrituras operativas no tocan las tablas compartidas de `public` y que una tienda no puede leer productos ni variantes de otra. Con dos ventas simultáneas de la última unidad de stock, una se aprobó (201) y la otra se rechazó con 409 `STOCK_INSUFICIENTE`, con el stock final exacto.

4.12.5 Alcance y límites

La modalidad permanece desactivada por defecto. Antes de adoptarla en producción falta ejecutar la migración sobre la base desplegada, conectar la aplicación con un rol sin privilegios de administración y aplicar las migraciones versionadas a todos los esquemas existentes en cada despliegue. Con muchos miles de tiendas, la cantidad de esquemas y de clientes de conexión también exige medir el consumo de memoria del servidor. Por eso EnsayAR mantiene el modelo pool como modo principal y conserva el esquema por tienda como la evolución preparada para las tiendas que requieran aislamiento físico.

5. Pagos para países de habla hispana

5.1 Introducción y alcance

En el primer ciclo, EnsayAR operaba para una sola tienda boliviana y aceptaba pago contra entrega, efectivo en caja y un QR simulado. El segundo ciclo cambia el problema en dos sentidos. Primero, la plataforma pasa a ser un servicio que cobra suscripciones a tiendas de distintos países (capítulo 3). Segundo, cada una de esas tiendas necesita cobrar a sus propias clientas con los medios de pago habituales en su país. El sistema maneja por lo tanto dos flujos de cobro independientes: el de la plataforma hacia la tienda, por la suscripción, y el de la tienda hacia la clienta, por la venta.

La consigna del segundo parcial exige que los pagos funcionen al menos en todos los países de habla hispana, que la elección de la pasarela se base en un estudio de costos orientado a cobrar la menor comisión posible al cliente, que la simulación utilice una pasarela real en su entorno de pruebas y que se expliquen las restricciones locales, como el predominio del QR en Bolivia o la dificultad de recibir fondos con PayPal desde ese país. Este capítulo desarrolla esos cuatro puntos y describe cómo se reflejan en el diseño de EnsayAR.

5.2 Conceptos fundamentales

5.2.1 Participantes de un pago electrónico

Un pago con tarjeta involucra a varios participantes. El titular de la tarjeta compra; el comercio vende; el banco emisor otorgó la tarjeta al titular y autoriza o rechaza el cargo; el banco adquirente mantiene la cuenta del comercio y recibe los fondos; y la red o marca (Visa, Mastercard) enruta los mensajes entre emisor y adquirente. Entre el comercio y el adquirente suele intervenir un proveedor de servicios de pago, que ofrece la pasarela técnica (la interfaz con

la que se integra el sistema) y, con frecuencia, actúa como agregador: el comercio no firma un contrato directo con un banco adquirente, sino que opera bajo la cuenta maestra del proveedor. Mercado Pago, Stripe y Libélula funcionan de esta manera para comercios pequeños.

5.2.2 Ciclo de vida de una transacción

El cobro con tarjeta atraviesa tres etapas. En la autorización, el emisor verifica fondos y riesgo y reserva el importe. En la captura, el comercio confirma que el cobro debe efectuarse; puede ser inmediata o diferida, por ejemplo hasta despachar el pedido. En la liquidación, los fondos llegan a la cuenta del comercio, descontadas las comisiones, en un plazo que va desde el mismo día hasta varias semanas. A estas etapas se suman dos eventos posteriores posibles: el reembolso, que inicia el comercio, y el contracargo, que inicia el titular ante su banco y que el comercio debe disputar con evidencias.

5.2.3 Otros medios de pago

Las transferencias inmediatas de cuenta a cuenta mueven el dinero en segundos y sin intervención de las marcas de tarjeta; Pix en Brasil, SPEI en México, PSE en Colombia y Bizum en España son ejemplos de este tipo (Banco Central do Brasil, s.f.; dLocal, s.f.). El pago por QR es una forma de iniciar ese tipo de transferencia: el comercio muestra un código con el importe y la referencia, y la clienta lo escanea desde la aplicación de su banco. Las billeteras digitales (Yape y Plin en Perú, Nequi en Colombia, Tigo Money en Bolivia y Paraguay) guardan saldo propio o cuentas asociadas. Por último, los pagos en efectivo con referencia permiten pagar en una red de agentes (OXXO en México, Pago Efectivo en Ecuador y Perú, Redpagos y Abitab en Uruguay) un cupón emitido por el comercio, que se confirma de forma asíncrona horas después.

La diferencia práctica para el diseño es el momento de la confirmación. Una tarjeta se autoriza en segundos y la respuesta es síncrona; una transferencia o un QR pueden tardar, y un

pago en efectivo puede confirmarse al día siguiente o no confirmarse nunca. Por ello un sistema de cobro debe modelar estados intermedios (pendiente, aprobado, rechazado, vencido) y no puede suponer que mostrar un QR equivale a haber recibido el dinero.

5.3 Medios de pago en los países de habla hispana

La Tabla siguiente resume los medios de pago identificados para cada país hispanohablante a partir de la documentación de cobertura de dLocal (s.f.), un procesador especializado en mercados emergentes, complementada con fuentes oficiales de cada medio. La presencia de un medio en la documentación de un proveedor indica que existe y que puede integrarse, no su participación de mercado ni su disponibilidad para un comercio concreto.

País	Moneda	Medios identificados además de tarjeta
Argentina	ARS	Mercado Pago, transferencia
Bolivia	BOB	QR interoperable, transferencia, Tigo Money
Chile	CLP	Webpay, Khipu, Mercado Pago, transferencia
Colombia	COP	PSE, Nequi, transferencia
Costa Rica	CRC	SINPE Móvil, transferencia
Cuba	CUP	Transfermóvil, EnZona
Ecuador	USD	Deuna, Pago Efectivo, transferencia
El Salvador	USD	PuntoXpress, Akipago, transferencia
España	EUR	Bizum, transferencia
Guatemala	GTQ	PayCash, SoyFri, transferencia
Honduras	HNL	Transferencia
México	MXN	SPEI, Mercado Pago, OXXO, transferencia
Nicaragua	NIO	PuntoXpress, transferencia
Panamá	USD	Yappy, Punto Pago, PayCash, transferencia
Paraguay	PYG	Tigo Money, transferencia
Perú	PEN	Yape, Plin, transferencia
Puerto Rico	USD	Transferencia
República Dominicana	DOP	Transferencia Banco Popular
Uruguay	UYU	Redpagos, Abitab, transferencia
Venezuela	VES	Pago Móvil, transferencia

Brasil no es un país hispanohablante, pero se incluye en el catálogo del sistema por su peso regional y porque Pix es el caso de referencia de un sistema de pagos instantáneos operado por un banco central (Banco Central do Brasil, s.f.).

Dos observaciones se desprenden de la tabla. La primera es que la tarjeta es el único medio presente en casi todos los países, pero no siempre es el preferido: en Bolivia, Perú y Brasil el pago móvil por QR o billetera ocupa ese lugar en el comercio minorista. La segunda es que ningún proveedor cubre por sí solo todos los medios locales de todos los países, de modo que un sistema regional debe poder trabajar con más de una pasarela.

5.4 Modelos de integración con una pasarela

Existen tres formas de integrar una pasarela, que se diferencian por quién captura los datos de pago y, por lo tanto, por la carga de cumplimiento normativo que asume el comercio.

En el modelo de redirección, el sistema crea una orden de pago en la pasarela y envía a la clienta a una página alojada por ella (por ejemplo, Checkout Pro de Mercado Pago). La pasarela captura la tarjeta, cobra y devuelve a la clienta al comercio, mientras notifica el resultado al servidor por un webhook. El comercio nunca ve los datos de la tarjeta.

En el modelo embebido, la pasarela provee componentes que se cargan dentro de la página del comercio (campos alojados o un iframe) y devuelven un token que representa la tarjeta. El servidor del comercio cobra usando ese token. La experiencia es más integrada y los datos sensibles siguen sin pasar por el comercio.

En el modelo de API directa, el comercio recibe los datos de la tarjeta en su servidor y los envía a la pasarela. Ofrece control total, pero obliga a cumplir íntegramente la norma PCI DSS (PCI Security Standards Council, 2022), lo que no es razonable para una plataforma como EnsayAR.

Los dos primeros modelos son los adecuados. EnsayAR adopta la redirección para las ventas en línea de las tiendas y el modelo embebido para la suscripción, en ambos casos con confirmación del resultado por webhook.

5.5 Estudio de costos

5.5.1 Componentes del costo

El costo de aceptar un pago no se reduce a la comisión porcentual publicada. Intervienen también un cargo fijo por transacción, que pesa más en ventas de importe bajo; un recargo cuando la tarjeta es extranjera; una comisión por conversión de moneda cuando la tarjeta y la cuenta del comercio operan en divisas distintas; el impuesto al valor agregado sobre la comisión, que en México, Chile y Argentina se suma a la tarifa publicada; el plazo de acreditación, ya que varios proveedores cobran menos cuanto más tarda el comercio en recibir el dinero; y los costos de retiro a la cuenta bancaria local.

5.5.2 Tarifas publicadas

La siguiente tabla reúne las tarifas estándar publicadas por los proveedores con presencia en la región, consultadas en noviembre de 2026. Son tarifas de lista para comercios sin negociación; los volúmenes altos acceden a condiciones especiales.

Proveedor	País del comercio	Tarifa estándar con tarjeta	Observaciones	Fuente
Stripe	España	1,5 % + 0,25 € (tarjetas del EEE)	3,15 % + 0,25 € con tarjetas internacionales; +2 % si hay conversión	Stripe (s.f.-e)
Stripe	México	3,6 % + MXN 3	+0,5 % con tarjeta internacional; +2 % si hay conversión; sin IVA	Stripe (s.f.-b)
Stripe	Brasil	3,99 % + BRL 0,39	+2 % con tarjeta internacional; Pix al 2 %	Stripe (s.f.-c)
Mercado Pago	México	3,49 % + MXN 4 al instante	2,95 % + MXN 4 con acreditación a 30 días	Mercado Pago (2026)
Mercado Pago	Chile	3,19 % + IVA al	2,89 % + IVA a 10	Mercado Pago (s.f.-b)

Proveedor	País del comercio	Tarifa estándar con tarjeta	Observaciones	Fuente
		instante	días	
Mercado Pago	Argentina	Variable por provincia y plazo	El plazo más corto es el más caro	Mercado Pago (s.f.-a)
PayPal	Bolivia	3,40 % + cargo fijo (nacional)	5,40 % + cargo fijo para destinatarios en Chile, Perú, Uruguay y otros mercados de la región	PayPal (2026)
Libélula	Bolivia	No publicada	Comisión según contrato; integra QR, tarjetas, Tigo Money y Pix	Libélula (s.f.)

Stripe no permite abrir cuentas de comercio en la mayoría de los países hispanohablantes de América; en la región opera para comercios constituidos en México y Brasil, además de España. Mercado Pago cubre Argentina, Brasil, Chile, Colombia, México, Perú y Uruguay. Para el resto de Centroamérica, el Caribe, Bolivia, Paraguay y Venezuela, la cobertura se obtiene mediante procesadores locales o agregadores transfronterizos como dLocal, cuyas tarifas no son públicas y se negocian por volumen.

5.5.3 Comparación aplicada a EnsayAR

Para comparar proveedores se toma como referencia el cobro mensual del plan Crecimiento, de 49 USD (sección 3.9.2). La tabla muestra solo la parte porcentual de la comisión; el cargo fijo y el impuesto sobre la comisión se suman en cada caso.

Escenario	Tasa	Comisión sobre 49 USD
Stripe, tienda en España, tarjeta europea	1,5 %	0,74 USD
Mercado Pago Chile, acreditación a 10 días	2,89 %	1,42 USD
Mercado Pago México, al instante	3,49 %	1,71 USD
Stripe México, tarjeta nacional	3,6 %	1,76 USD
Stripe México, tarjeta extranjera con conversión	6,1 %	2,99 USD
PayPal, destinatario en un mercado con tarifa de 5,40 %	5,40 %	2,65 USD

El resultado muestra que el factor que más encarece un cobro no es el proveedor, sino cobrar en una moneda distinta a la de la tarjeta o con una tarjeta de otro país. Un cobro nacional

en moneda local cuesta entre la mitad y un tercio de uno transfronterizo con conversión. De ahí se deriva el criterio de selección: cobrar a cada tienda y a cada clienta en su moneda local, a través del proveedor con adquirencia local en ese país, y reservar las opciones internacionales para los casos en que no exista alternativa local.

5.5.4 Recomendación por país

Con ese criterio, la estrategia recomendada para EnsayAR es la siguiente. En México, Chile, Colombia, Perú, Argentina y Uruguay se utiliza Mercado Pago, que ofrece tarjetas y los medios locales más usados (SPEI, OXXO, PSE, Yape) con una sola integración; en México, Stripe es una alternativa de costo similar. En España, Stripe ofrece la tarifa más baja de la comparación y acepta Bizum. En Bolivia, el cobro se realiza con QR interoperable a través de una pasarela local como Libélula, que además emite la factura electrónica exigida en el país. Para Centroamérica, el Caribe, Paraguay y Venezuela, se recurre a un agregador regional con medios locales. PayPal se mantiene como opción complementaria para clientas del exterior, no como medio principal.

5.6 Restricciones locales: Bolivia y PayPal

Bolivia combina dos características que condicionan el diseño. Por un lado, el QR interoperable se convirtió en el medio de pago electrónico dominante del comercio minorista: cualquier aplicación bancaria puede pagar un QR generado por otro banco, sin costo para la persona que paga. Por otro lado, ninguno de los grandes procesadores internacionales (Stripe, Mercado Pago) permite abrir una cuenta de comercio en el país, por lo que una tienda boliviana depende de pasarelas locales o del QR de su propio banco.

PayPal ilustra la asimetría entre pagar y cobrar. Desde Bolivia es sencillo pagar con PayPal usando una tarjeta internacional. Cobrar, en cambio, presenta dificultades. El acuerdo de

disponibilidad de PayPal ubica a Bolivia entre los mercados con capacidad de solo envío, mientras otras páginas del mismo proveedor publican tarifas para recibir pagos; la información oficial no es concluyente y debe confirmarse para cada cuenta. Aun cuando la recepción esté habilitada, la tarifa es de 3,40 % a 5,40 % más un cargo fijo (PayPal, 2026), a lo que se suman la conversión de moneda y los costos de retiro. Por estas razones, PayPal no es una opción adecuada como medio principal para las tiendas bolivianas.

Fuera de Bolivia, el QR no tiene la misma aceptación. Brasil (Pix) y Perú (Yape y Plin) usan esquemas propios, y en México o España el pago con tarjeta y la transferencia inmediata siguen siendo los medios esperados. Un sistema regional no puede, por lo tanto, imponer el QR como medio universal: debe ofrecer en cada país los medios que allí se usan.

5.7 Requisitos para operar con pagos reales

La integración técnica con una pasarela es la parte más sencilla de aceptar pagos reales. Antes, la tienda o la plataforma debe constituirse legalmente, contar con un número de identificación tributaria y una cuenta bancaria a su nombre, y superar el proceso de verificación de identidad del proveedor de pagos, que revisa el giro del negocio, los representantes legales y el origen de los fondos. Si el comercio trabaja con un banco adquirente directo, necesita además un contrato de afiliación. En Bolivia, cada venta debe acompañarse de factura electrónica, lo que obliga a que la pasarela o el sistema se integren con la facturación. Ninguno de estos trámites se realizó para EnsayAR, que opera con fines académicos; por ese motivo todos los cobros se ejecutan en modo de prueba.

5.8 Seguridad y cumplimiento

La norma PCI DSS establece los requisitos de seguridad para cualquier organización que almacene, procese o transmita datos de tarjetas (PCI Security Standards Council, 2022). El

alcance de su cumplimiento depende del modelo de integración: con redirección o campos alojados por la pasarela, el comercio solo debe completar el cuestionario de autoevaluación más reducido, porque los datos nunca pasan por sus servidores. EnsayAR se ajusta a ese caso y no guarda números de tarjeta: solo conserva los últimos cuatro dígitos y la referencia del cobro.

La confirmación por webhook introduce dos riesgos. El primero es la suplantación: un tercero podría enviar una notificación falsa de pago aprobado. Se mitiga verificando la firma que la pasarela adjunta a cada notificación con un secreto compartido (Stripe, s.f.-d). El segundo es la duplicación: las pasarelas reintentan la notificación hasta recibir confirmación, de modo que el mismo evento puede llegar varias veces. Se mitiga procesando cada evento de forma idempotente, con su identificador como clave única. El mismo principio se aplica al iniciar el cobro: la petición lleva una clave de idempotencia para que un reintento por falla de red no genere un segundo cargo (Stripe, s.f.-a).

5.9 Diseño en EnsayAR

5.9.1 Catálogo de métodos por país

El módulo `backend/src/pagos/metodos-pais.ts` define el catálogo de métodos de pago de 22 países: los 20 hispanohablantes de la tabla anterior, Brasil y Haití. Cada país tiene una moneda y una lista de métodos; la tarjeta y la transferencia aparecen en casi todos, PayPal solo en los países donde la documentación consultada confirma la recepción de pagos y Binance Pay como opción simulada con fines académicos. Todos los métodos están marcados como simulados, de modo que la interfaz nunca presenta como real una integración que no existe.

El catálogo se expone en `GET /plataforma/paises/:pais/metodos-pago` y alimenta tanto el asistente de alta de tienda como la renovación del plan. La tienda guarda su país y su moneda, y el servidor valida que el método elegido pertenezca al país de la tienda aunque la

petición no provenga de la interfaz, por ejemplo desde un cliente HTTP manual. El método usado queda registrado junto al pago en el historial de la suscripción.

5.9.2 Puerto de pasarela y adaptador simulado

El cobro de suscripciones depende de una abstracción, la clase `PasarelaSuscripcion`, con dos operaciones: cobrar un importe con una clave de idempotencia y reembolsar un cobro. El servicio de plataforma solo conoce esa abstracción. La implementación actual, `PasarelaSimulada`, valida el número de tarjeta con el algoritmo de Luhn, rechaza las tarjetas de prueba estándar de fondos insuficientes, tarjeta rechazada y tarjeta vencida, y devuelve el mismo resultado si recibe dos veces la misma clave. Este diseño sigue el patrón de puertos y adaptadores: reemplazar la simulación por el entorno de pruebas de Stripe o de Mercado Pago consiste en escribir un nuevo adaptador con la misma interfaz, sin modificar la lógica de suscripción.

5.9.3 Atomicidad del alta

El alta de una tienda combina un cobro externo con varias escrituras en la base de datos. Si el cobro se rechaza, la respuesta es `402 PAGO_RECHAZADO` y no se crea ningún registro. Si se aprueba, la tienda, la suscripción, el pago y el espacio inicial se crean en una sola transacción; si esa transacción falla, el servicio solicita el reembolso del cobro. Es una saga de dos pasos con una acción compensatoria, como se describe en la sección 6.10.3. La clave de idempotencia del pago es única en la tabla `pagos_suscripcion`, de modo que repetir la solicitud no cobra dos veces y devuelve la tienda ya creada.

5.9.4 Cobro de ventas

En las ventas de cada tienda, el caso de uso UC27 cubre el pago simulado con efectivo, QR, tarjeta o transferencia, y UC54 define el pago mediante una pasarela internacional. En UC54, el sistema crea una `TransaccionPasarela` con una clave de idempotencia, la clienta completa el pago en la página de la pasarela y la venta solo se confirma cuando llega la notificación de pago aprobado; una segunda notificación del mismo evento se ignora. Los pagos con QR o tarjeta no están disponibles sin conexión, porque requieren la confirmación de un tercero; sin red la caja solo acepta efectivo (sección 6.2.2).

5.10 Estado de implementación

Al cierre de este informe están implementados el catálogo de métodos por país, la validación del método según el país de la tienda, el registro del método en el historial de pagos, el puerto de pasarela con su adaptador simulado, la idempotencia del cobro y la compensación por reembolso en el alta. Estas funciones cuentan con pruebas automatizadas (`metodos-pais.spec.ts` y `renovacion-permisos.spec.ts`) y con verificaciones de extremo a extremo sobre una base PostgreSQL aislada.

Queda pendiente reemplazar el adaptador simulado por el entorno de pruebas de una pasarela real y aplicar el catálogo de métodos al checkout de la tienda en línea y al punto de venta, que hoy siguen usando el pago simulado del primer ciclo. El diseño de esa integración está completo en el caso de uso UC54 y en su diagrama de secuencia (sección 9.3).

6. Operación sin conexión, PWA y sincronización

El segundo parcial incorpora un requisito que modifica la manera de concebir EnsayAR: la plataforma debe seguir operando cuando el dispositivo pierde internet, tanto en la web como en el móvil, y reconciliar después lo ocurrido con el servidor de forma completa y predecible. El planteamiento docente tiene dos aristas. La frase "se cayó el sistema" deja de ser una excusa y pasa a ser síntoma de un desarrollo deficiente. Además, no basta con que la aplicación funcione sin red: se exige demostrar que los problemas clásicos de la sincronización (identificadores, venta simultánea de la última unidad, reenvíos por red inestable y deshacer transacciones incompletas) se resuelven con mecanismos formales.

Este capítulo desarrolla el fundamento teórico de esas exigencias y lo vincula con la implementación concreta de EnsayAR: modelos de consistencia, criterios para decidir qué operaciones pueden diferirse, persistencia local, aplicaciones web progresivas, sincronización, identificadores, conflictos, idempotencia, transacciones y las consecuencias de un monolito modular frente a microservicios. La sección final recorre la implementación real y propone un guion para demostrar el conflicto en la defensa.

6.1 Offline-first y modelos de consistencia

6.1.1 Concepto de offline-first

Se denomina **offline-first** al enfoque de diseño que trata la ausencia de red como un estado normal y no como una excepción. En una aplicación tradicional cada acción depende del servidor: si la petición falla, la acción se pierde. En una aplicación offline-first la interfaz trabaja primero contra un almacenamiento local, registra allí las acciones del usuario y delega en un proceso independiente su envío al servidor cuando la conectividad lo permita.

En consecuencia, el dispositivo pasa a ser una réplica parcial de los datos y sus escrituras se convierten en **intenciones** pendientes de confirmación: afirma "se vendió esta blusa a las 10:42" y el servidor decide si puede aplicarla, con reglas explícitas para cuando ya no es aplicable porque el mundo cambió mientras el dispositivo estaba aislado. Mostrar la última página guardada no es offline-first si no permite registrar acciones nuevas, y el parcial exige registrar ventas, clientes y productos sin red.

6.1.2 Consistencia fuerte y consistencia eventual

La **consistencia fuerte**, en su forma estricta llamada linealizabilidad, garantiza que el sistema se comporta como si existiera una única copia de los datos: toda lectura posterior a una escritura confirmada devuelve ese valor o uno más reciente (Kleppmann, 2017). Si una caja vende la última unidad de un vestido, ninguna otra puede verlo disponible después de confirmada la venta. La **consistencia eventual** es más débil: si cesan las escrituras, todas las réplicas convergerán al mismo valor, pero entretanto pueden diferir y no se garantiza cuándo convergen (Kleppmann, 2017).

Un sistema offline-first es eventualmente consistente respecto del cliente: el dispositivo desconectado muestra un stock que puede no coincidir con el del servidor. Lo que el diseño debe asegurar es que el punto de convergencia mantenga invariantes fuertes. EnsayAR adopta por ello una arquitectura híbrida: el cliente opera con consistencia eventual y el servidor PostgreSQL aplica cada operación con consistencia fuerte, de modo que el stock nunca queda negativo aunque dos dispositivos lo hayan creído disponible.

6.1.3 Teorema CAP aplicado

El teorema CAP, conjeturado por Eric Brewer y demostrado por Gilbert y Lynch en 2002, establece que un sistema distribuido sometido a una partición de red no puede garantizar a

la vez consistencia (linealizabilidad) y disponibilidad (toda petición a un nodo activo recibe respuesta) (Abadi, 2012; Kleppmann, 2017). La lectura de "elegir dos de tres" es engañosa: las particiones no se eligen, ocurren, y la decisión real es qué sacrificar cuando ocurren.

En EnsayAR la partición es literal: el teléfono de la vendedora queda aislado cuando se corta el wifi de la tienda. **Priorizar consistencia (CP)** es rechazar la operación hasta recuperar la red, lo correcto para un pago con tarjeta, que sin la pasarela podría resultar inexistente o duplicado. **Priorizar disponibilidad (AP)** es aceptarla localmente y reconciliarla después, lo adecuado para una venta en efectivo, porque el dinero ya está en la caja. La conclusión aplicada es que CAP se decide **por operación**: EnsayAR es AP para la venta en efectivo y las altas de clientas y productos, y CP para pagos electrónicos, créditos, devoluciones, inventario y cierre de caja.

6.1.4 PACELC

Abadi (2012) observó que CAP describe solo el comportamiento durante una partición y propuso PACELC: si hay partición (P), el sistema elige entre disponibilidad (A) y consistencia (C); en caso contrario (E), entre latencia (L) y consistencia (C). La segunda mitad captura un compromiso cotidiano: aun con la red funcionando, la consistencia exige coordinación síncrona, y coordinar cuesta tiempo.

En EnsayAR, durante la partición el POS elige disponibilidad para el efectivo (PA). Sin partición, la venta espera la confirmación de la transacción antes de emitir el ticket, aceptando latencia a cambio de consistencia (EC); en cambio, para leer el catálogo el service worker espera la red como máximo cuatro segundos y luego sirve la última copia, sacrificando consistencia de lectura por latencia acotada (EL).

Situación	Operación	Prioriza
Con partición (P)	Venta en efectivo, alta de clientas y	Disponibilidad (A)

Situación	Operación	Prioriza
	productos	
Con partición (P)	Pago QR, créditos, devoluciones y cierre de caja	Consistencia (C)
Sin partición (E)	Lectura del catálogo	Latencia (L)
Sin partición (E)	Confirmación de la venta	Consistencia (C)

6.2 Criterios para decidir qué opera sin conexión

6.2.1 Hechos consumados frente a operaciones que requieren confirmación

El docente lo planteó con una pregunta: "un pago, ¿se podrá hacer offline? No". De ahí se desprende un criterio general. Una operación puede registrarse sin conexión cuando describe un **hecho que ya ocurrió en el mundo físico** y cuyo registro tardío no cambia su naturaleza. No puede registrarse sin conexión cuando su validez **depende de la confirmación de un tercero** o de un estado global que el dispositivo no conoce. El criterio se traduce en cinco preguntas por caso de uso:

1. ¿El hecho ya ocurrió de forma irreversible (dinero en mano, prenda entregada)?
2. ¿Interviene un tercero que debe autorizar (banco, pasarela QR, procesador de tarjetas)?
3. ¿La decisión depende de un saldo global que otros dispositivos modifican (deuda de una clienta, arqueo)?
4. ¿Existe una regla determinista para resolver el caso si el servidor no puede aplicar la operación?
5. ¿Reintentar arriesga un efecto doble que la idempotencia no puede neutralizar?

La venta en efectivo responde sí a la primera y no a las demás, salvo por el stock, que es un contador global; ese riesgo se acepta porque existe una regla determinista para resolverlo (sección 6.7). El pago con QR falla la segunda: sin la pasarela no se sabe si el dinero se transfirió.

6.2.2 Matriz de EnsayAR

La política se codifica en `backend/src/sync/politica.ts` como la constante `POLITICA_OFFLINE`, con dos listas, `disponibles` y `soloEnLinea`, donde cada acción lleva su justificación. Se expone en `GET /sync/politica` y la interfaz la muestra en la pantalla de sincronización, de modo que una misma fuente rige el comportamiento y la documentación visible.

Operación	Sin red	Justificación
Venta en efectivo en caja	Sí	El dinero se recibe en mano; el stock se valida al sincronizar con un UPDATE condicional
Registro de clientes	Sí	El dispositivo genera el UUID; si el correo ya existe, se fusiona
Registro de productos y variantes	Sí	Llevan UUID del dispositivo; un SKU repetido queda como conflicto
Consulta de catálogo, precios y stock local	Sí	Se lee la última copia; el stock es orientativo hasta sincronizar
Pagos con QR o tarjeta	No	La pasarela debe confirmar en el momento; reintentar arriesga cobrar dos veces
Ventas al crédito y abonos	No	Exigen ver la deuda real; dos cajas con datos viejos podrían cobrar el mismo abono
Compras de la tienda online	No	La clienta paga por pasarela y necesita stock reservado al instante
Devoluciones	No	Mueven dinero y stock sobre una venta que puede no estar sincronizada
Cierre de caja	No	El arqueo debe incluir todas las ventas; se bloquea con operaciones pendientes
Ajustes, ingresos y transferencias de inventario	No	Corrigen el stock real; aplicarlos sobre una copia vieja produce errores
Usuarios, roles y suscripción	No	Afectan permisos y facturación; requieren validación central
Reportes y analítica	No	Se calculan sobre los datos consolidados del servidor

La matriz no es solo declarativa. El lote de sincronización admite únicamente los tipos `venta`, `cliente` y `producto`, y rechaza con `PAGO_REQUIERE_RED` toda venta cuyo medio de pago no sea efectivo. El cierre de caja se deshabilita en la web y en el móvil mientras haya

operaciones del dispositivo pendientes o en conflicto, porque un arqueo que ignore ventas no sincronizadas produciría un faltante ficticio.

6.2.3 Ventana offline configurable

Aceptar operaciones diferidas plantea otra pregunta: ¿cuánto puede esperar una operación antes de que aplicarla deje de tener sentido? Una venta de hace tres horas se integra sin problemas; una de hace dos meses afectaría reportes cerrados, cajas arqueadas y stock contado, y probablemente delata un dispositivo olvidado o un reloj mal configurado.

EnsayAR resuelve esto con una **ventana offline** por tienda, guardada en `tiendas.ventana_offline_dias`, con valor por defecto de 7 días y rango de 1 a 30 (constantes `VENTANA_MIN_DIAS` y `VENTANA_MAX_DIAS`). Solo un administrador la modifica, mediante `PATCH /sync/politica`. Al procesar cada operación, el servidor compara su fecha de creación en el dispositivo con la hora actual: si la antigüedad supera la ventana, la operación queda `vencida` con el código `FUERA_DE_VENTANA` para revisión de un administrador; si la fecha está más de cinco minutos en el futuro, se rechaza con `FECHA_INVALIDA`, porque un reloj adelantado falsearía la cronología.

Siete días cubren una sucursal sin conexión durante un fin de semana largo o un corte prolongado del proveedor y coinciden con el ciclo semanal de reposición y conteo de una tienda pequeña; el tope de 30 evita que una operación atraviese un cierre mensual. La configurabilidad responde al modelo SaaS, y el cambio rige de inmediato: una operación vencida puede reenviarse tras ampliar la ventana y se aplica con su fecha real.

6.3 Persistencia local en el cliente

6.3.1 IndexedDB en el navegador

IndexedDB es la base de datos transaccional del navegador. Almacena objetos estructurados en almacenes identificados por clave, admite índices secundarios y ejecuta toda operación dentro de transacciones. Su interfaz es asíncrona, por lo que no bloquea la interfaz, y está disponible en páginas y service workers. Es la tecnología adecuada para una cola de operaciones porque lo guardado sobrevive al cierre de la pestaña y al reinicio del dispositivo. En la web de EnsayAR, `frontend/src/offline/cola.ts` abre la base `ensayar-offline` con un almacén `operaciones` cuya clave es el `id` de la operación y un índice por `tienda`, que mantiene colas separadas si un mismo navegador se usa en dos tiendas del SaaS.

6.3.2 SQLite en el móvil

SQLite es un motor relacional embebido: no requiere servidor, guarda la base en un archivo y ofrece SQL con transacciones ACID. En el móvil es la opción natural cuando los datos locales necesitan consultas, orden o integridad, y fue la alternativa que sugirió el docente. La aplicación React Native usa `expo-sqlite` para abrir `ensayar-offline.db` en modo de registro anticipado (`PRAGMA journal_mode = WAL`), que permite leer mientras se escribe, y crea dos tablas: `operaciones` (`id`, `tienda`, `tipo`, `creada_en`, `datos`, `resumen`, `estado`, `error`), con un índice por `(tienda, creada_en)` para recorrer la cola en orden; y `cache` (`clave`, `valor`, `guardado_en`) para las copias de catálogo, turno y clientas.

6.3.3 AsyncStorage y Web Storage

AsyncStorage es el almacenamiento clave-valor asíncrono de React Native: guarda cadenas sin estructura ni índices, de modo que cualquier búsqueda exige leer y deserializar todo

el contenido. Sirve para preferencias y valores pequeños, no como cola voluminosa. Su análogo en el navegador es **Web Storage** (`localStorage`), síncrono y limitado a cadenas, con un máximo de 5 MiB por origen (MDN Web Docs, s.f.-b). En EnsayAR, AsyncStorage conserva el identificador del dispositivo y sirve de respaldo de la cola en Expo web, donde no hay SQLite nativo: `movil/src/offline/almacen.ts` expone la misma interfaz que `almacen.native.ts` y el empaquetador elige según la plataforma. En la web, `localStorage` guarda el identificador del dispositivo y las copias de datos por tienda (`frontend/src/offline/cache.ts`), mientras la cola vive en IndexedDB.

6.3.4 Cuotas, límites y desalojo

El almacenamiento del navegador no es ilimitado ni, por defecto, permanente. MDN Web Docs (s.f.-b) distingue el modo de **mejor esfuerzo**, predeterminado, en el que el navegador puede borrar los datos de un origen ante falta de espacio siguiendo una política de uso menos reciente, y el modo **persistente**, que se solicita con `navigator.storage.persist()` y solo el usuario puede borrar. Las cuotas varían: Chromium permite a un origen hasta el 60 % del disco; Firefox, en mejor esfuerzo, el menor valor entre el 10 % del disco y 10 GiB por sitio; Safari, desde iOS 17 y macOS 14, alrededor del 60 %. Safari además borra de forma proactiva los datos creados por script en orígenes sin interacción durante siete días de uso cuando la prevención de rastreo está activa. Exceder la cuota produce `QuotaExceededError`.

Para EnsayAR la cuota no es una restricción práctica, pues una venta ocupa del orden de un kilobyte; el riesgo real es el desalojo, porque una cola borrada equivale a ventas perdidas. Por eso la interfaz muestra siempre las operaciones pendientes; solicitar almacenamiento persistente queda como mejora.

6.4 Aplicaciones web progresivas

6.4.1 Definición y origen

Una **aplicación web progresiva** (PWA) es un sitio web que, mediante tecnologías estándar, adquiere capacidades de aplicación instalada: funcionamiento sin conexión, instalación en la pantalla de inicio, ventana propia y notificaciones. El término lo acuñaron Alex Russell y Frances Berriman en junio de 2015, al enumerar como rasgos de esta clase de aplicaciones ser adaptables, independientes de la conectividad gracias a los service workers, siempre actualizadas, servidas por TLS, capaces de reenganchar con notificaciones push, instalables sin tienda y enlazables por URL (Russell, 2015). "Progresiva" alude a la mejora progresiva: funciona como sitio común en cualquier navegador y se vuelve "app" gradualmente donde el navegador lo permite.

6.4.2 Manifiesto e instalabilidad

El **manifiesto de aplicación web** es un archivo JSON enlazado desde la página que declara nombre, iconos, URL de inicio, alcance, modo de presentación, colores y atajos. Con él, el navegador puede ofrecer la instalación y abrir la aplicación sin barra de direcciones cuando la presentación es `standalone`. El manifiesto de EnsayAR (`frontend/public/manifest.webmanifest`) declara `id` igual a `/`, nombre "EnsayAR Vesta", nombre corto "Vesta", `start_url` en `/inicio`, presentación `standalone`, idioma `es`, iconos de 192 y 512 píxeles (este último `maskable`, adaptable a las máscaras de cada sistema) y tres atajos: Punto de venta (`/pos`), Catálogo y Sincronización. El `id` da una identidad estable aunque cambie la URL de inicio; WebKit lo admite desde iOS 16.4 (Eidson y Simmons, 2023).

En los navegadores que lo emiten, el evento `beforeinstallprompt` permite diferir el diálogo nativo y lanzarlo desde un botón propio. `frontend/src/offline/pwa.ts` captura ese

evento, expone el botón "Instalar app" con el hook `useInstalar` y detecta si ya corre instalada consultando `display-mode: standalone`.

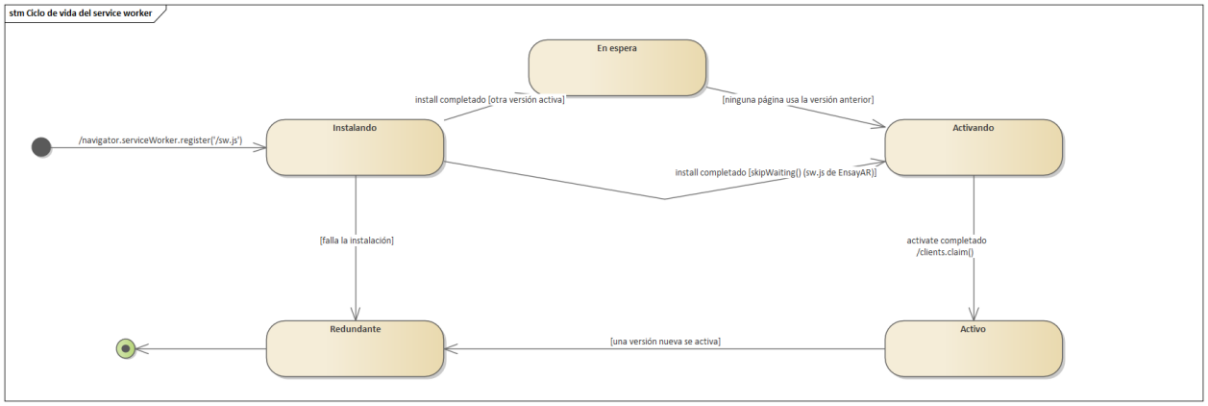
6.4.3 Service worker y su ciclo de vida

Un **service worker** es un script que el navegador ejecuta en segundo plano, separado de la página, y que actúa como proxy programable entre la aplicación y la red: intercepta las peticiones de las páginas que controla, decide si responder desde caché o desde red y recibe eventos como `push` aun con la página cerrada. Exige HTTPS o localhost. Su ciclo de vida busca hacer posible el offline-first y que solo una versión del sitio esté activa a la vez (Archibald, s.f.-b). Tras el **registro** con `navigator.serviceWorker.register('/sw.js')`, el evento `install` ocurre una vez por versión y es el momento de precargar lo indispensable; si otra versión controla páginas, la nueva queda **en espera** hasta que ninguna la use, salvo que llame a `self.skipWaiting()`; en `activate` se borran las cachés anteriores y `clients.claim()` toma el control de las páginas abiertas; y cualquier cambio byte a byte en el script genera una nueva versión que repite el ciclo.

El `sw.js` de EnsayAR declara `VERSION = 'v2'` y deriva tres cachés: `ensayar-shell-v2`, `ensayar-estaticos-v2` y `ensayar-api-v2`. En `install` precarga la raíz `/`, el manifiesto y el favicon y llama a `skipWaiting()`; en `activate` elimina toda caché ajena a la lista vigente y ejecuta `clients.claim()`. Cambiar la versión basta para invalidar ordenadamente lo guardado por la anterior.

Figura 6.1

Ciclo de vida del service worker de EnsayAR



Nota. Máquina de estados UML 2.5: la llamada a skipWaiting() omite el estado en espera y clients.claim() toma el control de las páginas abiertas.

6.4.4 Estrategias de caché

El service worker no impone una política: se elige por tipo de recurso. Archibald (s.f.-a) cataloga las estrategias habituales, de las que tres son centrales:

- **Cache first:** se responde desde la caché y solo se recurre a la red si no hay copia. Conviene a recursos versionados que no cambian.
- **Network first:** se intenta la red y, si falla, se sirve la última copia. Prioriza la frescura en datos cambiantes cuya copia vieja es mejor que nada.
- **Stale-while-revalidate:** se responde con la copia y, en paralelo, se pide la versión nueva para actualizar la caché. Combina la latencia de la caché con una frescura de un paso de atraso.

El `sw.js` de EnsayAR combina las tres. Las navegaciones siguen red primero y, sin red, devuelven el `index.html` guardado, lo que permite recargar el POS sin conexión. Los estáticos (`/assets/`, `/uploads/`, `/catalogo/`, `/icons/`, imágenes y fuentes) pasan por `estaticoCachePrimero`, que en rigor implementa stale-while-revalidate. Las lecturas de la API

de una lista blanca (tienda, catálogo, sucursales, temporadas, colecciones, turno abierto y política offline) siguen red primero con un tope de cuatro segundos y caen a la copia ante demora o error 5xx. La disponibilidad por sucursal (`/catalogo/disponibilidad`) queda fuera a propósito: el POS la descuenta localmente con cada venta sin red y una copia del service worker la sobrescribiría con valores previos. Por seguridad SaaS, la clave de cada respuesta guardada incluye la cabecera `X-Tienda` y una huella SHA-256 truncada del token, para no servir nunca a una tienda o usuario la copia de otro. Solo se interceptan `GET` del mismo origen; las escrituras sin red corresponden a la cola.

6.4.5 Background Sync

La **API de sincronización en segundo plano** permite que una página registre una etiqueta y que el navegador despierte al service worker con un evento `sync` al detectar conectividad, incluso sin ninguna pestaña del sitio abierta (Archibald, s.f.-a). Es la herramienta idónea para vaciar una cola de salida, pero no está implementada en todos los motores de uso masivo. EnsayAR no la usa y adopta directamente el mecanismo alternativo: sincronizar al abrir la aplicación, al recibir el evento `online` y cada 30 segundos mientras haya pendientes. El costo es que la cola solo se vacía con la aplicación abierta; el beneficio, un comportamiento idéntico en todos los navegadores.

6.4.6 Push API, Notifications API y VAPID

La **Push API** permite recibir mensajes del servidor aunque la aplicación no esté en primer plano ni cargada: un service worker activo se suscribe con `PushManager.subscribe()` y obtiene una `PushSubscription` con un *endpoint* y claves de cifrado (MDN Web Docs, s.f.-a). El *endpoint* es una URL de capacidad que basta para enviar mensajes, por lo que debe mantenerse en reserva. La **Notifications API** muestra el aviso mediante `showNotification()`, previo

permiso del usuario, y entre el servidor de aplicación y el dispositivo media el **servicio push** del fabricante del navegador.

VAPID, definido en la RFC 8292, identifica al servidor de aplicación ante el servicio push. El servidor genera un par de claves ECDSA sobre P-256; al suscribirse, el navegador entrega la pública como `applicationServerKey`, y cada envío incluye un JWT firmado con la privada, con audiencia igual al origen del *endpoint* y expiración no mayor a 24 horas (Thomson y Beverloo, 2017). Así, el servicio push rechaza mensajes de quien no posea la clave privada y una URL filtrada no basta para enviar notificaciones. EnsayAR carga las claves desde `VAPID_PUBLIC_KEY`, `VAPID_PRIVATE_KEY` y `VAPID_SUBJECT` y firma los envíos con la biblioteca `web-push`, con 24 horas de vida por mensaje.

6.4.7 Ventajas frente a una aplicación nativa

Frente a la aplicación nativa, la PWA se instala desde el navegador en segundos y ocupa poco espacio, algo importante para vendedoras con teléfonos de gama baja, caso que mencionó el docente; se actualiza en cuanto se publica una versión, sin revisión de tienda; usa un único código para escritorio, Android e iOS; y es enlazable, con permisos que se piden cuando hacen falta. Sus desventajas son un acceso más limitado al hardware, menor control de la ejecución en segundo plano y dependencia de cada navegador. Por eso EnsayAR mantiene ambas vías: la PWA para quien no quiere instalar nada y la aplicación React Native con SQLite para el uso intensivo en caja.

6.4.8 Limitaciones en iOS

En iOS los navegadores han estado históricamente obligados a usar WebKit, de modo que las capacidades de una PWA dependen de Apple. El push web llegó recién con iOS y iPadOS 16.4 y solo para aplicaciones **añadidas a la pantalla de inicio**, con un permiso que debe

pedirse tras una interacción directa del usuario (Eidson y Simmons, 2023): una vendedora que use EnsayAR desde Safari sin instalarla no recibirá avisos de conflictos. Además, no existe `beforeinstallprompt` (la instalación es manual desde el menú Compartir), no hay Background Sync y el desalojo proactivo de Safari (6.3.4) puede borrar datos sin interacción reciente. En iOS conviene usar la aplicación instalada y sincronizar con frecuencia; para la caja, la aplicación nativa ofrece mayores garantías.

6.5 Sincronización

6.5.1 Cola de operaciones y orden

El núcleo de un cliente offline-first es una **cola de operaciones**, también llamada outbox del cliente por analogía con la bandeja de salida del correo. Cada acción que modifica datos se materializa como un registro inmutable con identificador único, tipo, fecha de ocurrencia, datos para reproducirla y estado. La interfaz escribe en la cola y actualiza su vista local; un proceso independiente envía los registros pendientes. Así, "sincronizar el estado" se reduce a "transmitir una secuencia de intenciones", más simple y auditable.

El modelo exige respetar el **orden causal**: si se registra una clienta nueva y luego se le vende una blusa, la venta referencia a la clienta y no puede aplicarse antes. Por eso la cola se recorre por fecha de creación y el servidor procesa cada lote en el orden recibido. En EnsayAR, `OperacionOffline` (web) y `OperacionLocal` (móvil) comparten forma: `id` (UUID que actúa como clave de idempotencia), `tipo` (`venta`, `cliente` o `producto`), `creadaEn` en ISO 8601, `datos`, `tienda`, un `resumen` legible y `estado`; las funciones `listarOps` ordenan por `creadaEn`.

6.5.2 Reintentos y backoff exponencial

Toda comunicación puede fallar de forma transitoria y el cliente debe reintentar, pero reintentar sin pausa contra un servidor caído agrava la caída, sobre todo si muchos clientes se reconectan a la vez. La técnica estándar es el **backoff exponencial**: tras cada fallo, la espera se multiplica por un factor (1, 2, 4, 8 segundos) hasta un máximo, con una componente aleatoria (*jitter*) para que los clientes no se sincronicen entre sí. Igual de importante es distinguir los fallos transitorios (sin red, 502, 503, 504) de los definitivos (400, 422), que repetidos producen el mismo resultado.

EnsayAR aplica esa distinción pero no implementa backoff exponencial. La función `esFalloRed` (`frontend/src/offline/red.ts`) clasifica como fallo de red la falta de conexión, los `TypeError` de `fetch`, las respuestas 502, 503 y 504 y las 5xx sin cuerpo de NestJS; en esos casos la operación sigue `pendiente`. La cadencia es fija: al recuperar la red, al abrir la aplicación y cada 30 segundos mientras haya pendientes. Con pocos dispositivos por sucursal la carga es despreciable; a gran escala convendría añadir backoff con *jitter*. Además, `sincronizar` comparte una única promesa en vuelo, de modo que el evento `online`, el temporizador y un clic manual no disparan envíos paralelos de la misma cola.

6.5.3 Estados de una operación

Sin estados explícitos, una operación fallida desaparece en silencio o se reintenta para siempre. Los estados de EnsayAR y su correspondencia con los conceptos teóricos son los siguientes.

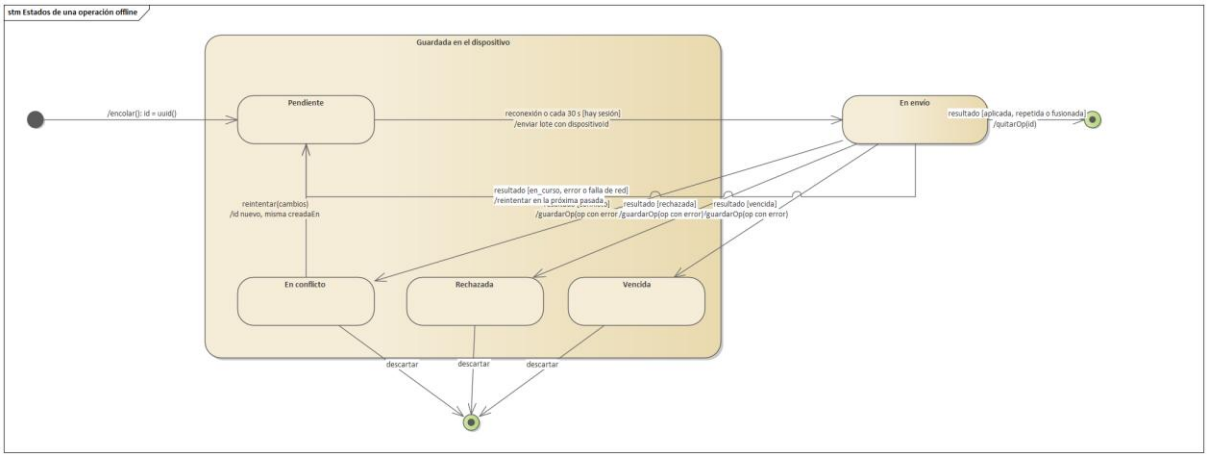
Estado	Dónde	Significado	Acción del cliente
pendiente	Cliente	Guardada localmente, sin respuesta definitiva	Reintentar en la próxima pasada
aplicada	Servidor	Ejecutada y confirmada por primera vez	Quitar de la cola
repetida (duplicada)	Servidor	Mismo id y contenido ya aplicados; se devuelve el	Quitar de la cola

Estado	Dónde	Significado	Acción del cliente
		resultado guardado	
fusionada	Servidor	Cliente con correo existente asociado al registro previo	Quitar y reasignar ventas pendientes
conflicto	Ambos	Válida pero no aplicable por el estado actual (stock, turno, SKU)	Mostrar y ofrecer resolución
vencida	Ambos	Fuera de la ventana offline	Revisión de un administrador
rechazada	Ambos	Inválida o no permitida (datos, permisos, pago sin red, fecha futura)	Mostrar; normalmente descartar
en_curso	Servidor	Otra petición procesa el mismo id	Mantener pendiente
error	Servidor	Fallo interno; el registro se libera	Mantener pendiente
descartada	Servidor	Conflicto dado por resuelto por un responsable	Ninguna

El estado teórico "duplicada" se llama `repetida` en el código. Un conflicto es una operación correcta que choca con el estado del mundo y admite resolución de negocio; un rechazo es una operación que nunca debió emitirse.

Figura 6.2

Estados de una operación offline



Nota. Máquina de estados UML 2.5: un conflicto puede reintentarse con un UUID nuevo o darse por descartado mediante una resolución administrativa.

6.5.4 Sincronización por lotes

La **sincronización por lotes** agrupa varias operaciones en una petición, evitando multiplicar latencia y autenticación en redes móviles, y devuelve un resultado por cada una. Para ser robusta, el fallo de una operación no debe arrastrar a las demás y reenviar el lote no debe duplicar nada. `POST /sync/lote` recibe `dispositivoId`, un `nombreDispositivo` opcional y de 1 a 200 `operaciones` (los clientes envían tramos de 50); registra el dispositivo en `dispositivos`, procesa las operaciones en orden, cada una en su propia transacción, y responde con un resumen por estado y el resultado de cada una. La primera condición la asegura la transacción por operación; la segunda, el UUID de cada operación (6.8).

6.6 Identificadores generados en el dispositivo

El docente señaló los identificadores como uno de los dos problemas clásicos. Con claves **autoincrementales**, el número lo asigna la base central al insertar: un dispositivo desconectado no puede conocerlo, y si elige uno propio, dos dispositivos elegirán el mismo. Además, sin identificador definitivo no puede crear registros que referencien al nuevo: la venta a una clienta registrada sin red quedaría sin clave foránea válida. Los rangos reservados por dispositivo o los identificadores temporales que luego se reemplazan añaden complejidad y puntos de fallo.

La alternativa es generar en el cliente identificadores con probabilidad de colisión despreciable. La RFC 9562, que en 2024 reemplazó a la RFC 4122, define varias versiones de **UUID** de 128 bits (Davis et al., 2024):

- **UUID v4:** 122 bits aleatorios; la colisión puede tratarse como nula en la práctica. No guarda orden, por lo que las inserciones caen en posiciones aleatorias del índice de árbol B y lo fragmentan en tablas muy grandes.

- **UUID v7:** los primeros 48 bits son la marca de tiempo Unix en milisegundos y el resto es aleatorio, lo que añade orden temporal y mejora la localidad de los índices (Davis et al., 2024).
- **ULID:** especificación comunitaria ajena a la RFC, con 48 bits de tiempo y 80 aleatorios, representada en 26 caracteres Base32 ordenables lexicográficamente.

Generar el identificador en el dispositivo permite además construir grafos de registros sin conexión (la clienta y su venta llevan UUID definitivos desde el inicio) y convierte al identificador en la **clave de idempotencia** natural de la operación. EnsayAR usa UUID v4: `crypto.randomUUID()` en la web, con una implementación sobre `crypto.getRandomValues()` cuando la PWA se abre por IP local sin HTTPS, y `Crypto.randomUUID()` de `expo-crypto` en el móvil. Clientes y productos creados sin red se insertan con el UUID del dispositivo como clave primaria; las ventas lo guardan en `ventas.uuid`, con un índice único por `(tienda_id, uuid)` creado en la migración `20261108200000_offline_sync`. Migrar a v7 sería una mejora de rendimiento de índices, no de corrección.

6.7 Conflictos y control de concurrencia

6.7.1 Estrategias de resolución

Un **conflicto** surge cuando dos réplicas modifican el mismo dato de forma independiente y sus cambios no se combinan trivialmente. Kleppmann (2017) describe varias estrategias:

- **Last-write-wins (LWW):** gana la escritura con marca de tiempo más reciente. Es simple, pero pierde datos en silencio y depende de relojes que pueden diferir; aplicada al stock, una caja sobrescribiría la venta de la otra.

- **Fusión (merge):** se combinan los cambios con reglas del dominio, por ejemplo conservando campos no superpuestos.
- **CRDT:** tipos de datos replicados diseñados para que cualquier orden de aplicación converja al mismo estado, como contadores o conjuntos. No pueden imponer una restricción global como "el stock no puede ser negativo", porque esa regla exige coordinación.
- **Servidor autoritativo:** las réplicas envían intenciones y un servidor central decide con las reglas de negocio sobre el estado vigente; lo rechazado vuelve como conflicto.

EnsayAR adopta el **servidor autoritativo** para stock y ventas, porque la invariante de no vender lo inexistente solo se garantiza en un punto central. Para clientas aplica **fusión** por correo: si dos cajas registran sin red a la misma clienta, la segunda operación resulta `fusionada` y las ventas que referían al identificador local pasan a apuntar al del servidor, en el lote en curso, en lotes posteriores y en la cola del dispositivo. Para productos, un SKU ya usado produce el conflicto `SKU_EN_USO`.

6.7.2 Concurrencia optimista y pesimista

En el servidor, el control de concurrencia define cómo se protegen los datos ante transacciones simultáneas. El **pesimista** asume que el conflicto es probable y bloquea el recurso antes de leerlo, típicamente con `SELECT ... FOR UPDATE`. El **optimista** asume que es raro: no bloquea al leer, sino que verifica al escribir que el dato siga cumpliendo la condición esperada y, si no, aborta e informa el conflicto.

La sincronización offline es intrínsecamente optimista: el dispositivo actuó sin poder bloquear nada y el servidor verifica después. La clave es que esa verificación sea **atómica**. Leer el stock, comparar en el código y luego escribir deja una ventana de carrera entre lectura y escritura. EnsayAR la elimina delegando la comparación en la propia sentencia de actualización.

6.7.3 El caso de la última unidad paso a paso

El escenario de referencia para validar el diseño es el siguiente: queda una unidad de un vestido, dos vendedores sin red la venden y, al reconectarse, uno debe enterarse de que su venta no puede aplicarse. La pieza central está en `backend/src/ventas/ventas.service.ts`, dentro de la transacción de cobro:

```
UPDATE stock_almacen
SET cantidad = cantidad - :n
WHERE tienda_id = :tienda
  AND almacen_id = :almacen
  AND variante_id = :variante
  AND cantidad - reservado >= :n;
```

La columna `reservado` contiene las unidades retenidas por pagos QR o con tarjeta aún no confirmados, de modo que el disponible real es `cantidad - reservado`. La sentencia descuenta solo si el disponible alcanza; si no, no modifica ninguna fila. El servicio inspecciona las filas afectadas y, si son cero, lanza `StockInsuficienteError` con el disponible y la cantidad pedida, lo que aborta toda la transacción.

Que dos ejecuciones simultáneas no puedan ambas descontar la última unidad lo garantiza PostgreSQL: en Read Committed, su nivel predeterminado, cuando un `UPDATE` encuentra una fila que otra transacción concurrente modificó, espera a que esta confirme o deshaga; si confirma, reevalúa la condición `WHERE` sobre la versión actualizada y solo procede si se sigue cumpliendo (PostgreSQL Global Development Group, s.f.-c). El recorrido es el siguiente:

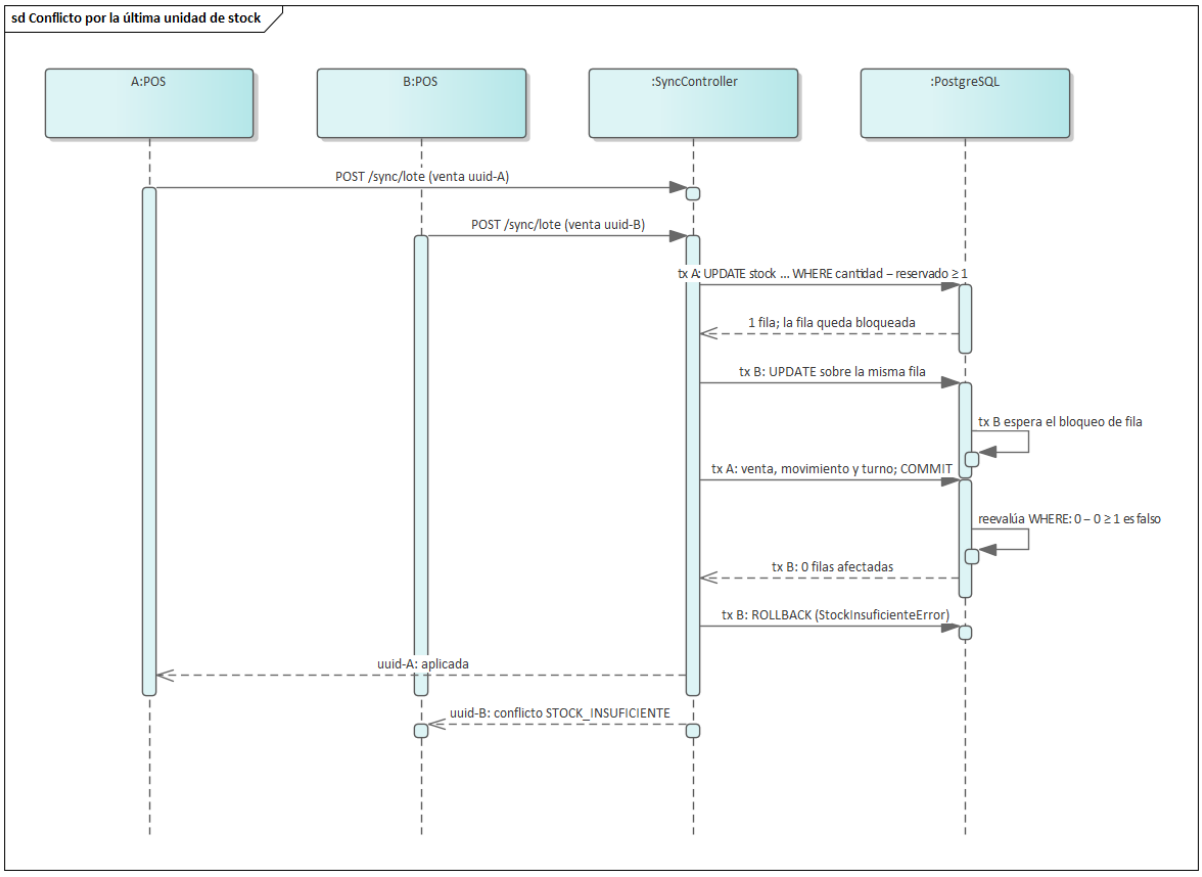
1. Estado inicial: `cantidad = 1`, `reservado = 0`. Ambos dispositivos tienen esa copia.
2. Se corta la red. El vendedor A vende la unidad en efectivo; el POS guarda la operación con su UUID, descuenta su copia local y muestra la venta como **Pendiente**.
3. El vendedor B, también sin red, hace lo mismo con otro UUID; su pantalla también muestra **Pendiente**.

4. Vuelve la red y ambos envían su lote a `POST /sync/lote`, quizá en el mismo instante.
5. La transacción de A ejecuta el `UPDATE`: `1 - 0 >= 1` se cumple; la fila queda con `cantidad = 0` y bloqueada hasta el final de la transacción.
6. La transacción de B intenta el mismo `UPDATE`, encuentra la fila bloqueada y espera.
7. A crea la venta, sus líneas, el movimiento de inventario, suma el total al turno, registra la auditoría y confirma. Su operación queda `aplicada` y sale de la cola.
8. B reanuda; PostgreSQL reevalúa la condición sobre la fila actualizada (`0 - 0 >= 1` es falso) y el `UPDATE` afecta cero filas.
9. Se lanza `StockInsuficienteError`, la transacción de B se deshace completa y la operación queda en `conflicto` con el código `STOCK_INSUFICIENTE` y el detalle `{varianteId, disponible: 0, pedido: 1}`.
10. B ve **Conflicto** con el mensaje "Stock insuficiente: había 0 y la venta pedía 1. Otra caja vendió antes"; administradores y encargados reciben una notificación interna y push que enlaza a `/sincronizacion`.

El stock nunca queda negativo y la resolución corresponde al negocio. **Vender lo disponible** limita cada línea al disponible informado y reencola la operación con un UUID nuevo conservando la fecha original (no se ofrece si el disponible es cero). **Descartar** la elimina de la cola local, lo que implica anular la venta y devolver el dinero o buscar la prenda en otra sucursal. En el servidor, la operación original sigue como conflicto hasta que un encargado la marca resuelta, lo que la deja `descartada` con la auditoría `sync_conflicto_descartado`. En ventas de varias líneas, los `UPDATE` se ejecutan en **orden fijo de identificador de variante** para que dos ventas cruzadas adquieran los bloqueos en la misma secuencia y no se interbloqueen.

Figura 6.3

Conflicto de stock entre dos dispositivos que vendieron la última unidad sin conexión



Nota. Diagrama de secuencia UML 2.5: PostgreSQL bloquea la fila, reevalúa la condición después del COMMIT de A y la transacción de B termina en conflicto.

6.8 Idempotencia

6.8.1 Definición formal

En matemática, una función f es **idempotente** si aplicarla varias veces equivale a aplicarla una vez, es decir, si f(f(x)) = f(x) para todo x de su dominio; el valor absoluto es un ejemplo clásico. En sistemas distribuidos, una operación es idempotente si ejecutarla una o n veces deja el sistema en el mismo estado. La definición se refiere al **efecto** sobre el estado, no a

la respuesta: borrar un recurso dos veces deja el mismo estado aunque la segunda respuesta diga que no existía.

La idempotencia importa porque en una red no confiable el emisor no distingue entre "la petición se perdió" y "se procesó pero se perdió la respuesta". Ante la duda, lo único seguro es reintentar, y reintentar solo es seguro si la operación es idempotente. Las redes ofrecen entrega "al menos una vez"; la idempotencia del receptor convierte esa garantía en un efecto "exactamente una vez".

6.8.2 Métodos HTTP idempotentes

En HTTP un método es idempotente si el efecto previsto de varias peticiones idénticas es el mismo que el de una. Según la RFC 9110, `OPTIONS`, `HEAD`, `GET`, `PUT` y `DELETE` lo son, mientras que `POST` y `PATCH` no (Jena et al., 2026). `PUT` lo es porque reemplaza el recurso por un valor dado; `POST` suele crear uno nuevo en cada llamada. Registrar una venta es una creación y se expresa con `POST /ventas`: sin una medida adicional, un reintento crearía otra venta y descontaría stock dos veces. Esa idempotencia no la da el protocolo; la construye la aplicación.

6.8.3 La cabecera Idempotency-Key

La práctica más difundida es que el cliente genere una clave única por operación lógica y la envíe en cada intento. Stripe la popularizó: el cliente agrega `Idempotency-Key` con un valor de suficiente entropía (sugiere UUID v4) y puede repetir la petición tras un error de conexión sin crear un segundo objeto; el servidor guarda el código de estado y el cuerpo de la primera ejecución, los devuelve en los reintentos y responde con error si los parámetros no coinciden con los originales (Stripe, s.f.-a).

El grupo HTTPAPI de la IETF estandariza esta práctica en el borrador "The Idempotency-Key HTTP Header Field", cuya versión 08 se publicó en octubre de 2026 (Jena et

al., 2026). Entre otras cosas establece que la clave la genera el cliente y que el recurso puede combinarla con una **huella** del contenido; que reutilizar una clave con otro contenido debería responderse con `422 Unprocessable Content`; que un reintento mientras la original sigue en proceso debería recibir `409 Conflict`; que si el endpoint exige la cabecera y falta corresponde `400 Bad Request`; y que el recurso debe publicar su política de expiración de claves.

6.8.4 Qué se almacena y por cuánto tiempo

Un registro de idempotencia guarda, como mínimo, la clave, una huella del contenido (para distinguir un reintento de una clave reutilizada por error), el estado de procesamiento (para detectar reintentos concurrentes) y la respuesta. En cuanto a la duración, Stripe permite purgar las claves con al menos 24 horas de antigüedad y trata como nueva una clave reutilizada tras la purga (Stripe, s.f.-a); el borrador IETF deja la política a cada recurso pero exige documentarla (Jena et al., 2026). La regla práctica es que la retención supere el tiempo máximo de reintento, que en un sistema offline no son minutos sino la ventana offline completa.

EnsayAR usa una sola tabla, `operaciones_sync`, para los dos caminos de idempotencia: las operaciones del lote y las peticiones HTTP con `Idempotency-Key`. Sus columnas relevantes son `operation_id`, `hash` (SHA-256 del contenido), `estado` (que nace `en_curso`), `resultado` (JSON con la respuesta), `tipo`, `dispositivo_id`, `usuario_id` y `hecha_en`, con índice único sobre `(tienda_id, operation_id)`. Las claves no se purgan: se conservan indefinidamente, lo que cubre con holgura la ventana máxima de 30 días y deja un historial auditable. El costo es el crecimiento de la tabla, asumible para una tienda; una depuración periódica de registros definitivos más antiguos que la ventana sería la evolución natural.

6.8.5 Doble y triple clic con red inestable

El segundo escenario del docente no es la falta de red sino su inestabilidad: el cajero pulsa "Guardar", la venta se registra, pero la respuesta se pierde, y el cajero pulsa tres o cuatro veces más. Sin idempotencia habría cuatro ventas. EnsayAR se defiende en tres capas.

La primera es el interceptor global `IdempotenciaInterceptor` (`backend/src/sync/idempotencia.interceptor.ts`), activo en los 11 endpoints marcados con `@Idempotente()`, entre ellos `POST /ventas`, apertura y cierre de caja, ingresos de inventario, abonos, devoluciones y pagos. Con la cabecera presente, construye el identificador `http:{usuario}:{clave}` y una huella SHA-256 del método, la ruta, los parámetros y el cuerpo, e intenta insertar un registro `en_curso`:

1. Si la inserción tiene éxito, la petición es la primera y se ejecuta.
2. Si choca con el índice único y la huella difiere, responde 422 con `IDEMPOTENCIA_CONFLICTO`.
3. Si la huella coincide y el registro terminó, devuelve la respuesta guardada con `Idempotent-Replayed: true`.
4. Si sigue en curso, espera hasta 25 veces 200 milisegundos y luego responde 409 con `OPERACION_EN_CURSO`; un registro en curso de más de 60 segundos se considera huérfano y se libera.
5. Al terminar, un 4xx queda guardado como definitivo y un 5xx borra el registro para permitir reintentar.

La segunda capa es el UUID de la venta, que el POS envía como `Idempotency-Key`: dentro de la transacción, el servicio devuelve la venta previa con ese UUID si el contenido coincide o lanza `UuidConflictError` si difiere. La tercera es el índice único `(tienda_id, uuid)`

en `ventas`: si dos transacciones simultáneas superan esa búsqueda, la segunda choca con el índice y el servicio devuelve la venta ganadora. El script de verificación comprueba que cuatro `POST /ventas` simultáneos con la misma clave producen una sola venta. En el lote offline la lógica es análoga con el UUID de la operación. A diferencia del borrador IETF, EnsayAR no exige la cabecera; sin ella no hay protección HTTP, aunque las ventas siguen protegidas por su UUID.

6.9 Transacciones y rollback

6.9.1 Propiedades ACID

Una **transacción** agrupa lecturas y escrituras en una unidad que se confirma o se deshace entera. Sus propiedades se resumen en ACID (Kleppmann, 2017): **atomicidad**, todas las escrituras se aplican o ninguna, lo que hace posible el rollback; **consistencia**, la base pasa de un estado válido a otro según las invariantes de la aplicación; **aislamiento**, las transacciones concurrentes no producen resultados imposibles en una ejecución secuencial, en el grado que fije el nivel elegido; y **durabilidad**, lo confirmado sobrevive a caídas.

6.9.2 Niveles de aislamiento

El estándar SQL define cuatro niveles según las anomalías que permiten: Read Uncommitted, Read Committed, Repeatable Read y Serializable. PostgreSQL implementa Read Uncommitted como Read Committed, que es su nivel por defecto (PostgreSQL Global Development Group, s.f.-c). En Read Committed cada sentencia ve los datos confirmados al comenzar, de modo que dos lecturas en la misma transacción pueden diferir; Repeatable Read fija una instantánea para toda la transacción; Serializable garantiza equivalencia con alguna ejecución en serie a costa de abortar transacciones que deben reintentarse.

Las transacciones de EnsayAR no fijan un nivel explícito, por lo que rige Read Committed. Es suficiente porque las invariantes críticas no dependen de lecturas previas sino que se verifican en la propia sentencia de escritura, y la reevaluación del `WHERE` propia de ese nivel es justamente lo que vuelve correcta la actualización condicional.

6.9.3 SELECT ... FOR UPDATE frente a UPDATE condicional

Hay dos formas atómicas de proteger un descuento: la **pesimista**, que lee con `SELECT ... FOR UPDATE` (bloqueando la fila), compara en la aplicación y escribe; y la **actualización condicional**, una sola sentencia `UPDATE ... WHERE condición` cuyas filas afectadas indican si la condición se cumplió.

Criterio	SELECT ... FOR UPDATE	UPDATE condicional
Viajes a la base	Dos (leer y escribir)	Uno
Duración del bloqueo	Desde la lectura hasta el final	Desde la escritura hasta el final
Lógica de validación	En la aplicación, flexible	En SQL, limitada a la condición
Información del fallo	Se conoce el valor leído	Requiere leer después para informar el disponible
Riesgo de error	Olvidar el FOR UPDATE reintroduce la carrera	Bajo, la atomicidad es intrínseca

EnsayAR usa ambas donde corresponde. Para el stock, la actualización condicional, porque la regla es una desigualdad simple y conviene acortar el bloqueo de una fila muy disputada. Para el turno de caja, `SELECT ... FOR UPDATE` sobre `caja_turnos` en `validarTurnoEnTx`, porque la validación es más rica (turno abierto, propio del cajero, almacén de la sucursal del turno) y debe impedir que el turno se cierre mientras se registra la venta. Si falla, la operación offline queda en conflicto con códigos como `TURNO_CERRADO` y el móvil ofrece reasignarla con el botón "Pasar a mi turno".

6.9.4 Una transacción por operación sincronizada

Una sola transacción para todo el lote haría que un conflicto en la operación 37 deshiciera las 36 válidas anteriores, obligaría a reintentar todo y mantendría bloqueos durante el lote

completo. EnsayAR usa **una transacción por operación**: cada venta, con sus líneas, su movimiento de inventario, la actualización del turno y su auditoría, se confirma o se deshace como unidad, con independencia de las demás.

Así se materializa el rollback que pide el docente. El script de verificación lo prueba con una venta de dos líneas donde la segunda variante no tiene stock: el primer `UPDATE` se ejecuta, el segundo afecta cero filas, la excepción aborta la transacción y el stock de la primera variante vuelve a su valor previo. El registro en `operaciones_sync` se escribe fuera de la transacción de negocio: si esta falla por un error interno se elimina para permitir el reintento, y si falla por una regla de negocio se conserva para que el reintento reciba el mismo veredicto.

6.10 Monolito modular frente a microservicios

6.10.1 Rollback local en el monolito

EnsayAR es un **monolito modular**: una aplicación NestJS desplegada como un solo proceso, organizada en módulos con fronteras claras (ventas, inventario, caja, sincronización, notificaciones, push, tenencia, plataforma) que comparten una base PostgreSQL y se comunican por inyección de dependencias, no por red. Cuando una venta modifica tablas que pertenecen a tres módulos (`ventas`, `stock_almacen`, `caja_turnos`), todas las escrituras ocurren en la misma conexión y transacción, y un fallo en cualquier paso se revierte con una única instrucción. No hay estados intermedios visibles ni compensaciones que programar. Ese es el sentido de la observación del docente: con una sola base, el rollback "uno a uno" es simple.

6.10.2 Transacciones distribuidas y 2PC

Si venta, inventario y caja fueran microservicios con base propia, no habría transacción común. La solución clásica es el **protocolo de confirmación en dos fases** (2PC): un coordinador

pide a cada participante que se prepare dejando sus cambios provisionales y bloqueados; si todos votan sí, ordena confirmar, y si alguno vota no, deshacer. Kleppmann (2017) expone sus problemas: si el coordinador cae, un participante que votó sí queda en duda reteniendo bloqueos; basta un participante caído para que nada avance; y los bloqueos duran al menos dos viajes de red. Además, muchas bases e intermediarios de mensajería no soportan 2PC (Richardson, s.f.-b), y Richardson (s.f.-a) enumera "2PC no es una opción" entre las fuerzas que conducen al patrón Saga.

6.10.3 Patrón Saga

Una **saga** es una secuencia coordinada de transacciones locales, cada una confinada a un servicio. Si todas completan, la operación de negocio queda hecha; si una falla por violar una regla, se ejecutan **transacciones compensatorias** que deshacen semánticamente los pasos ya confirmados (Richardson, s.f.-a). Se coordina por **coreografía**, donde cada servicio publica un evento de dominio que dispara la transacción del siguiente, sin componente central, o por **orquestación**, donde un orquestador indica a cada participante qué ejecutar y decide el siguiente paso o las compensaciones. La primera acopla menos pero dificulta seguir el flujo; la segunda concentra la lógica en un punto comprensible. Richardson (s.f.-a) advierte dos desventajas inherentes: no hay rollback automático, porque cada compensación se diseña explícitamente, y no hay aislamiento, porque otras transacciones ven los estados intermedios, lo que obliga a contramedidas de diseño.

Una compensación no es un rollback técnico: no borra lo ocurrido, sino que registra una acción inversa con significado de negocio. Compensar un descuento de stock es registrar un reingreso; compensar un cobro es emitir un reembolso. EnsayAR ya aplica este razonamiento en el alta SaaS: si la creación de la tienda falla en la base después de cobrar la suscripción, el

servicio de plataforma invoca el reembolso en la pasarela, porque un cobro externo no se deshace con un rollback de PostgreSQL.

6.10.4 Ejemplo: saga de venta si EnsayAR se dividiera

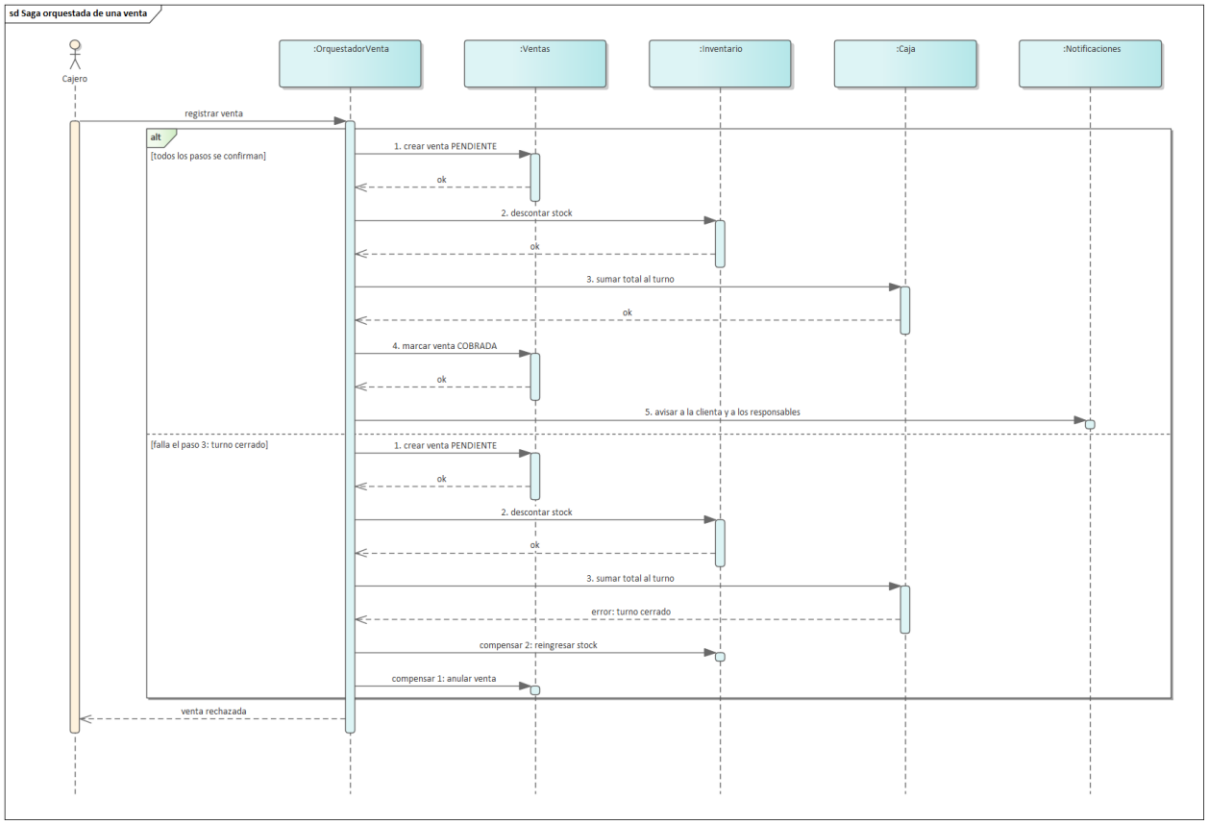
Supóngase EnsayAR dividido en cuatro servicios con base propia: Ventas, Inventario, Caja y Notificaciones. La venta en efectivo sincronizada se implementaría como una saga orquestada por Ventas:

Paso	Servicio	Transacción local	Compensación
1	Ventas	Crear la venta PENDIENTE con el UUID del dispositivo	Marcar la venta RECHAZADA
2	Inventario	Descontar stock con el UPDATE condicional	Reingresar las unidades
3	Caja	Sumar el total al turno abierto	Restar el total del turno
4	Ventas	Marcar la venta COBRADA	No requiere (paso final)
5	Notificaciones	Avisar a la clienta y a los responsables	No requiere (reintentable)

Si el paso 2 falla por stock insuficiente, se compensa el paso 1 y la operación offline se informa como conflicto, igual que hoy. Si el paso 3 falla porque el turno se cerró, se compensa el 2 (reingreso) y luego el 1. El estado PENDIENTE actúa como contramedida de aislamiento: reportes y arqueo deben ignorar ventas pendientes. La notificación va después del último paso compensable porque un aviso enviado no puede retirarse. En la variante coreografiada, los mismos pasos se encadenarían con eventos como `VentaCreada`, `StockDescontado` o `TurnoInvalido`, y cada servicio debería ser idempotente frente a eventos repetidos.

Figura 6.4

Saga orquestada de una venta



Nota. Diagrama de secuencia UML 2.5 con fragmento alt: arriba, el flujo exitoso; abajo, el fallo en Caja con las compensaciones de Inventario y Ventas en orden inverso.

6.10.5 Patrón Transactional Outbox

Una saga requiere que cada servicio actualice su base y publique un evento de forma atómica. Publicar dentro de la transacción es inseguro, porque si esta se deshace el evento anuncia algo que no ocurrió; publicar después de confirmar también, porque si el proceso cae entre ambos pasos el evento se pierde (Richardson, s.f.-b). El patrón **Transactional Outbox** escribe el mensaje en una tabla de salida de la misma base, dentro de la transacción que modifica las entidades, y un proceso separado, el relé, lo publica después. Así, los mensajes se envían si y solo si la transacción confirma, y en orden. Como el relé puede publicar un mensaje dos veces si

cae tras enviarlo y antes de marcarlo, los consumidores deben ser idempotentes, típicamente registrando los identificadores ya procesados (Richardson, s.f.-b).

La analogía con EnsayAR es directa: la cola del dispositivo es un outbox del lado del cliente, y `operaciones_sync` cumple el papel del registro de mensajes procesados que exige el consumidor idempotente. Si EnsayAR se dividiera en servicios, el patrón se aplicaría entre ellos con una tabla de salida por base.

6.11 Notificaciones push y otras capacidades aprovechadas

El docente insistió en que la PWA no se agota en el offline. En EnsayAR, cuando una sincronización produce conflictos, `SyncService` crea para cada administrador y encargado activo la notificación "Conflicto al sincronizar", con enlace a `/sincronizacion`, y el servicio de notificaciones reenvía toda notificación interna como push mediante `PushService`.

La suscripción comienza con el botón "Activar notificaciones": la aplicación obtiene la clave pública con `GET /push/clave`, pide permiso, se suscribe con `pushManager.subscribe()` y registra la suscripción con `POST /push/suscribir`. El servidor la guarda en `push_suscripciones` con su *endpoint* y las claves `p256dh` y `auth`, y la elimina cuando el servicio push responde 404 o 410. En el service worker, el evento `push` muestra la notificación con una etiqueta (`tag`) que agrupa avisos del mismo tipo, y `notificationclick` enfoca la ventana abierta y navega al enlace. Se aprovechan además la instalación con atajos, la presentación `standalone`, la caché del armazón y los eventos `online` y `offline`, reflejados en un indicador permanente de conexión y pendientes.

6.12 Aplicación en EnsayAR

Esta sección recorre la implementación real, cerrada el 8 de noviembre de 2026 con la migración `20261108200000_offline_sync`.

6.12.1 Servidor

El módulo `backend/src/sync/` expone cinco rutas protegidas por JWT y roles: `GET` y `PATCH /sync/politica`, `POST /sync/lote`, `GET /sync/operaciones` y `POST /sync/operaciones/:id/descartar`. Cada operación del lote pasa por la idempotencia (6.8) y la ventana (6.2.3) y luego se despacha por tipo: la venta delega en `VentasService.cobrar` con su hora real (`hechaEn`); el cliente se crea con el UUID del dispositivo o se fusiona por correo; el producto se crea con variantes, stock inicial y movimiento de ajuste en una transacción. Las referencias a datos de otra tienda se rechazan con `REFERENCIA_NO_EXISTE`, apoyadas en la columna `tienda_id`, cuyo valor por defecto proviene de la variable de sesión `app.tienda_id`. La migración `20261108200000_offline_sync` añadió a `operaciones_sync` las columnas `hash`, `hecha_en`, `resultado`, `tipo` y `usuario_id` con su índice único por tienda; agregó `ventana_offline_dias` a `tiendas`; creó `push_suscripciones`; y reemplazó el índice único global de `ventas.uuid` por uno por `(tienda_id, uuid)`.

6.12.2 Web

La web registra el service worker desde `frontend/src/main.tsx` y concentra la lógica offline en `frontend/src/offline/`: `cola.ts`, `sync.ts`, `red.ts`, `cache.ts` y `pwa.ts`. Con red, el POS cobra con `POST /ventas` usando el UUID de la venta como `Idempotency-Key`; si la petición falla por red, encola la venta con ese mismo UUID, de modo que si el servidor alcanzó a registrarla la sincronización devuelve `repetida`. Sin red, encola directamente y descuenta el stock local; el alta de clientas y de productos funciona igual. La página `frontend/src/pages/Sincronizacion.tsx` tiene tres paneles: la cola del dispositivo con "Vender lo disponible" y "Descartar"; los conflictos de toda la tienda, para encargados y

administradores, con fecha real, dispositivo y botón "Marcar resuelto"; y la política offline con el formulario de la ventana de 1 a 30 días.

6.12.3 Móvil

La aplicación React Native con Expo SDK 53 replica el modelo: SQLite en `almacen.native.ts`, respaldo en AsyncStorage para Expo web, UUID con `expo-crypto` y NetInfo, que considera al dispositivo en línea solo si está conectado e internet es alcanzable. El hook `useAutoSync` sincroniza al montar, al reconectar y cada 30 segundos, en tramos de 50. La función `cambiosReintento` decide la resolución según el código: ajusta las líneas al disponible ante `STOCK_INSUFICIENTE`, reasigna al turno abierto ante conflictos de turno y no ofrece reintento para `PAGO_REQUIERE_RED`, `PERMISO_DENEGADO` ni `DATOS_INVALIDOS`. La sesión guardada permite arrancar sin conexión; el servidor valida el token al sincronizar.

6.12.4 Verificación automatizada

El script `backend/scripts/verificar-offline.ts` (`npm run verificar:offline`) crea una tienda real con dos cajeros y realiza catorce comprobaciones contra la API en ejecución, entre ellas la última unidad vendida por dos cajas a la vez, el rollback de una venta de dos líneas, el reenvío de un lote como `repetida`, la fusión de clientas, cuatro `POST /ventas` simultáneos con la misma clave, el rechazo de una venta QR sin red y el comportamiento de la ventana al ampliarla de 7 a 10 días.

6.12.5 Limitaciones conocidas

El JWT dura 12 horas y la ventana 7 días: si la sesión vence sin red, la cola se conserva y la aplicación pide iniciar sesión antes de enviarla. La cadencia de reintento es fija, sin backoff exponencial; no se usa Background Sync; las claves de idempotencia no se depuran; y no se pide

almacenamiento persistente. La versión móvil se probó en Expo web, con el respaldo de AsyncStorage, y se verificó que el paquete de Android compila, pero no se ejecutó con SQLite en un dispositivo Android físico.

6.12.6 Procedimiento de verificación del conflicto de stock

El siguiente procedimiento reproduce de forma controlada el caso de la última unidad y permite comprobar el comportamiento descrito en este capítulo.

1. Se preparan dos cajas con turno abierto en la misma sucursal (por ejemplo, la PWA en un portátil y la aplicación móvil en un teléfono) y se elige una variante con stock 2.
2. En la caja A se corta la red (modo avión o herramientas de desarrollo) y se venden las 2 unidades en efectivo. La venta queda **Pendiente** y el contador de operaciones sin sincronizar aumenta.
3. En la caja B, con red, se vende 1 unidad de la misma variante; el servidor la aplica y el stock central queda en 1.
4. En ese momento la caja A considera que tiene 2 unidades disponibles, porque su copia es eventualmente consistente, mientras el servidor mantiene la invariante con consistencia fuerte.
5. Al restablecer la red en A, la venta pasa a **Conflicto** con el mensaje "había 1 y la venta pedía 2"; el stock no queda negativo y el administrador recibe el aviso.
6. En `/sincronizacion`, el administrador ve el conflicto con su fecha real y el dispositivo de origen.
7. En A, la opción **Vender lo disponible** reencola la operación con un UUID nuevo y 1 unidad; el servidor la aplica, el stock queda en 0 y el conflicto original se marca como resuelto.

8. Por último, varias pulsaciones de "Cobrar" con red inestable producen una sola venta, gracias a las cabeceras `Idempotency-Key` e `Idempotent-Replayed`.

El procedimiento confirma tres propiedades del diseño: el dispositivo registra hechos y el servidor decide si son aplicables; el descuento de stock es una única sentencia condicional que PostgreSQL reevalúa bajo bloqueo; y cada operación lleva un UUID generado en el dispositivo que la vuelve idempotente ante cualquier número de reintentos.

7. Inteligencia artificial para reportes dinámicos

7.1 Introducción y alcance del capítulo

En el segundo parcial, el docente pidió reforzar los reportes de EnsayAR con un mecanismo en el que el usuario expresa en lenguaje natural, por texto o por voz, el reporte que necesita, y el sistema lo construye en el momento. Un mismo pedido puede combinar varios criterios de selección, cálculos, agrupaciones y ordenamientos, y el resultado debe poder verse en pantalla, en PDF o en Excel. El ejemplo dado en clase resume la exigencia: "sácame una lista de los clientes que me muestre su nombre, su dirección, su email, de todos aquellos clientes que me deben, de tal monto a tal monto, por ventas al crédito". La misma consigna fija el vocabulario del problema: "un reporte tiene filas, columnas, criterios, agrupaciones, ordenación".

Este capítulo fundamenta cómo se estructura un sistema de inteligencia artificial capaz de generar reportes totalmente dinámicos, desde la teoría y desde la implementación. Primero se caracteriza el reporte y se revisa la evolución de las interfaces en lenguaje natural a bases de datos. Luego se analizan los riesgos de que un modelo escriba SQL sin restricciones y se justifica el enfoque adoptado: una capa semántica con una representación intermedia, en la que la IA traduce el pedido a una especificación JSON y nunca a SQL. Después se describe la arquitectura en tubería, la ingeniería de prompts, la voz, la ambigüedad, la seguridad, la exportación, la disponibilidad y la evaluación, y se cierra con un ejemplo completo con datos reales.

Las afirmaciones sobre la implementación corresponden al código del módulo de reportes inteligentes del backend (NestJS 11, Prisma 6 y PostgreSQL), a la pantalla web en React 19 y a la pantalla móvil en React Native con Expo. Los ejemplos se obtuvieron ejecutando el servicio sobre la tienda de demostración "vesta", con datos sintéticos.

7.2 Del reporte estático al reporte dinámico

7.2.1 Reportes estáticos, parametrizados y ad hoc

En los sistemas de información tradicionales, un reporte es un artefacto diseñado de antemano: un desarrollador define la consulta, maqueta el formato y lo publica en un menú. Herramientas como Crystal Reports o JasperReports popularizaron este **reporte estático**. Su ventaja es el control, porque la consulta se revisa una sola vez y su rendimiento se conoce. Su desventaja es la rigidez: cada necesidad nueva exige un ciclo de desarrollo.

El **reporte parametrizado** agrega flexibilidad sin cambiar la estructura: el usuario completa algunos huecos, como un rango de fechas o una sucursal. Cubre bien las preguntas recurrentes, pero no las imprevistas.

El **reporte ad hoc o dinámico** deja que el usuario decida en el momento qué datos quiere, con qué criterios, cómo agruparlos y en qué orden. Históricamente se ofreció mediante constructores visuales de consultas, que exigen conocer la estructura de los datos. La IA generativa propone reemplazar ese constructor por una conversación. Los reportes estáticos no desaparecen: en EnsayAR siguen existiendo los reportes fijos del primer parcial, como el reporte del día, y la analítica con aprendizaje automático se documenta en otro capítulo. El reporte dinámico cubre las preguntas que nadie previó.

7.2.2 Anatomía de un reporte

Para que una máquina construya reportes, primero hay que descomponer el concepto en piezas formales. A partir de la consigna del docente y de la estructura de una consulta SQL de lectura, se identifican los siguientes componentes:

Componente	Pregunta que responde	Equivalente SQL
Fuente	¿De qué entidad de negocio se habla?	FROM y JOIN

Componente	Pregunta que responde	Equivalente SQL
Filas	¿Qué representa cada renglón?	Granularidad de la fuente
Columnas	¿Qué datos se muestran?	Lista del SELECT
Criterios	¿Qué registros entran?	WHERE
Agrupaciones	¿Cómo se resumen los registros?	GROUP BY
Agregaciones	¿Qué cálculo se hace por grupo?	SUM, AVG, COUNT, MIN, MAX
Criterios sobre totales	¿Qué grupos entran?	HAVING
Ordenación	¿En qué secuencia se presentan?	ORDER BY
Límite	¿Cuántos renglones como máximo?	LIMIT
Totales	¿Cuánto suman las columnas numéricas?	Fila de pie
Formato	¿Dónde se entrega?	Pantalla, PDF o Excel

La **granularidad** depende de la fuente: en un reporte de ventas cada fila puede ser una prenda vendida, y en uno de clientes, una clienta. Al agrupar, cada fila pasa a representar un grupo. La distinción entre **criterios sobre registros** (WHERE) y **criterios sobre totales** (HAVING) es sutil para el usuario, pero decisiva para el motor: "ventas de más de 2000 Bs" puede referirse a ventas individuales o a meses cuyo total supera ese monto, según haya o no una agrupación.

7.2.3 El reporte como consulta declarativa

Todos estos componentes son **declarativos**: describen qué se quiere, no cómo obtenerlo. Si un reporte puede describirse como un conjunto finito de decisiones, entender el pedido se reduce a llenar una estructura de datos con ellas. Construir el SQL deja de ser tarea de la IA y pasa a ser tarea de un compilador determinista. Esta es la idea central del diseño de EnsayAR.

7.3 Interfaces en lenguaje natural a bases de datos

7.3.1 Primera generación: sistemas sintácticos, semánticos y de representación intermedia

Androutsopoulos, Ritchie y Thanisch (1995), en una revisión clásica sobre las interfaces en lenguaje natural a bases de datos (NLIDB, por sus siglas en inglés), sitúan los primeros prototipos a fines de los sesenta y principios de los setenta. El más conocido es LUNAR, que consultaba análisis químicos de rocas lunares y, como sus contemporáneos, estaba atado a una

base de datos concreta. En los ochenta destacó CHAT-80, escrito en Prolog, que traducía preguntas en inglés a expresiones Prolog.

La revisión distingue tres familias de arquitecturas. En los **sistemas sintácticos**, el árbol sintáctico de la pregunta se traduce directamente a una consulta. En los de **gramática semántica**, las categorías de la gramática ya son conceptos del dominio. En los de **lenguaje de representación intermedia**, la pregunta se transforma primero en una expresión lógica independiente de la base, y un módulo separado la traduce al lenguaje de consulta. Esta última familia anticipa la idea central de este capítulo: separar la comprensión del lenguaje de la generación de la consulta.

Los problemas que identificaron esos trabajos siguen vigentes: la portabilidad entre bases de datos, la ambigüedad, la dificultad del usuario para saber si una pregunta rechazada excede la cobertura lingüística del sistema o su cobertura conceptual, y la necesidad de que el sistema explique qué entendió.

7.3.2 Text-to-SQL neuronal y los conjuntos de evaluación

Con el aprendizaje profundo, el problema se reformuló como traducción: dada una pregunta y un esquema, producir la consulta SQL. Este planteamiento se conoce como text-to-SQL y su avance dependió de grandes conjuntos anotados. WikiSQL (Zhong et al., 2017, citado en Hong et al., 2024) ofreció muchos pares pregunta-consulta, pero sobre tablas únicas.

Spider (Yu et al., 2018) elevó la exigencia: 10 181 preguntas y 5693 consultas SQL complejas sobre 200 bases de datos con múltiples tablas, de 138 dominios. Separó las bases de entrenamiento de las de prueba, de modo que el modelo debe generalizar a esquemas nuevos, e incluyó muchos más GROUP BY, HAVING, ORDER BY y consultas anidadas que los

conjuntos previos. En su publicación original, el mejor modelo lograba un 12,4 % de coincidencia exacta.

BIRD (Li et al., 2023) acercó la evaluación al mundo real: 12 751 pares pregunta-SQL sobre 95 bases que suman 33,4 GB, de 37 dominios profesionales. Su énfasis está en los **valores** almacenados: datos sucios, la necesidad de conocimiento externo para relacionar la pregunta con esos valores y la eficiencia de las consultas. En el artículo original, GPT-4 alcanzaba un 54,89 % de exactitud de ejecución, frente a un 92,96 % humano.

Esta literatura usa dos métricas principales. La **coincidencia exacta** compara componente por componente la consulta generada con la de referencia. La **exactitud de ejecución** compara los resultados de ejecutar ambas. La sección 7.15 retoma estas ideas.

7.3.3 El cambio que introdujeron los modelos de lenguaje de gran escala

Brown et al. (2020) mostraron que un modelo suficientemente grande, GPT-3 con 175 000 millones de parámetros, resuelve tareas nuevas sin reentrenamiento, a partir de instrucciones y unos pocos ejemplos incluidos en el texto de entrada. Este aprendizaje en contexto transformó el text-to-SQL: basta con describir el esquema en el prompt y agregar ejemplos resueltos. La revisión de Hong et al. (2024) organiza las técnicas surgidas desde entonces: enlace de esquema (schema linking), descomposición en subtareas, selección de ejemplos, autocorrección a partir de errores de ejecución y ajuste fino. También registra conjuntos más exigentes, como Spider 2.0, orientado a flujos empresariales.

Los LLM resolvieron gran parte del problema lingüístico: entienden paráfrasis, sinónimos, errores de tipeo y el español coloquial de Bolivia. Pero no resolvieron por sí solos los problemas de **confiabilidad, seguridad y gobierno de los datos** que aparecen al ejecutar una consulta generada sobre la base de producción de un negocio.

7.4 Riesgos de dejar que el modelo escriba SQL libre

La arquitectura más directa consiste en enviar al modelo el esquema y la pregunta, recibir SQL y ejecutarlo. En un sistema SaaS multi-inquilino que maneja ventas, deudas y datos personales, esta arquitectura presenta cinco riesgos.

Inyección SQL. OWASP la define como la situación en la que datos no confiables pasan a formar parte de la sintaxis de una consulta y alteran su lógica. Recomienda parametrizar los valores y restringir las partes estructurales (tablas, columnas, dirección de orden) a una lista blanca definida por la aplicación. Si el modelo escribe toda la consulta, toda la estructura es dinámica, y un usuario puede redactar su pedido para que el modelo incluya sentencias no deseadas (inyección de instrucciones o prompt injection).

Alucinación de tablas y columnas. El modelo produce el texto más probable, no necesariamente el correcto. Ante "el teléfono de la clienta", puede inventar una columna inexistente o elegir otra parecida con distinto significado. BIRD documenta precisamente la dificultad de relacionar la pregunta con los valores reales.

Fuga entre inquilinos. En EnsayAR, todas las tiendas comparten la base y cada fila lleva su identificador de tienda. Si el modelo escribe la consulta, el aislamiento depende de que recuerde el filtro en cada tabla y subconsulta. Un solo olvido expondría los datos de una tienda ante otra.

Consultas costosas. Una consulta libre puede generar productos cartesianos o recorrer tablas completas sin límite. En un servicio compartido, la consulta pesada de un inquilino degrada el servicio de todos.

No determinismo. La misma pregunta puede producir consultas distintas en ejecuciones distintas. En un reporte de gestión, dos personas que piden lo mismo deben ver lo mismo, y el auditor debe poder reproducir lo ejecutado.

A estos riesgos se suma la **disponibilidad**: si el único camino pasa por un servicio externo de IA, una credencial vencida deja al negocio sin reportes. Este riesgo se materializó durante el desarrollo: la clave de Azure OpenAI responde con el código HTTP 401, y aun así el módulo sigue funcionando (sección 7.14).

En el primer parcial, el modelo proponía una consulta SQL sobre una vista de ventas y un validador comprobaba que fuera un SELECT sobre esa vista. Era seguro, pero limitado a una sola vista. Para el segundo parcial se lo reemplazó por el enfoque siguiente.

7.5 Enfoque adoptado: capa semántica y representación intermedia

7.5.1 Las capas semánticas en la industria

La inteligencia de negocios llegó a una respuesta parecida por razones de gobierno de datos. Una **capa semántica** se interpone entre los usuarios y las tablas físicas y describe los datos en términos del negocio: dimensiones, medidas, relaciones y reglas de cálculo. Los usuarios, y hoy también los agentes de IA, consultan esa capa, y ella genera el SQL.

Según su documentación, **LookML**, el lenguaje de modelado de Looker, describe dimensiones, agregados, cálculos y relaciones, y Looker usa ese modelo para construir las consultas. Las expresiones SQL se escriben una sola vez y se reutilizan para generar consultas ad hoc, mientras el usuario de negocio elige dimensiones, medidas y filtros sin escribir SQL. LookML separa la estructura de la consulta (cómo se unen las tablas) de su contenido (columnas, cálculos y filtros).

La **dbt Semantic Layer**, basada en MetricFlow, define métricas sobre los modelos existentes y resuelve automáticamente las uniones. Su documentación destaca la consistencia: si una métrica cambia, el cambio llega a todas las herramientas, y los asistentes de IA pueden consultar métricas gobernadas en lugar de adivinar sobre las tablas crudas.

Cube organiza el modelo en cubos con medidas (datos cuantitativos) y dimensiones (datos categóricos). Al pedir una medida junto con una dimensión, genera automáticamente una consulta con GROUP BY, y ofrece vistas para controlar qué parte del modelo se expone.

Las tres herramientas comparten una tesis: el usuario elige entre piezas previamente revisadas y un generador determinista arma el SQL. EnsayAR la adopta a escala de un módulo, con un catálogo propio en TypeScript.

7.5.2 La especificación como contrato entre la IA y el compilador

El propio código resume el enfoque: el modelo "no ve la base ni escribe SQL: recibe el catálogo semántico y devuelve una especificación JSON que después valida y compila el backend". La IA queda confinada a lo que hace bien, comprender el lenguaje, y se la excluye de lo riesgoso: la sintaxis SQL, los joins y los filtros de seguridad.

La especificación es una representación intermedia en el sentido de Androutsopoulos et al. (1995): no depende de cómo se almacenan los datos, sino de cómo los nombra el negocio. Contiene los componentes de 7.2.2: título, fuente, columnas con agregación opcional, filtros (campo, operador, valor, segundo valor y agregación opcional), agrupaciones, orden, límite y formato. Tres productores pueden generarla: el modelo de IA, el intérprete por reglas y el usuario, al editar los criterios en pantalla. Todos pasan por el mismo validador y el mismo compilador.

7.5.3 Comparación de enfoques

Aspecto	SQL libre generado por el modelo	Especificación sobre capa semántica
Superficie de inyección	Toda la consulta	Solo valores, siempre parametrizados
Campos inventados	Posibles	Rechazados por el validador
Aislamiento por tienda	Depende del modelo	Fijo en cada fuente
Costo de la consulta	Impredecible	Joins revisados y límite máximo
Reproducibilidad	Baja	La especificación se guarda y se recompila
Funcionamiento sin IA	Imposible	Intérprete por reglas
Expresividad	Máxima	Acotada al catálogo

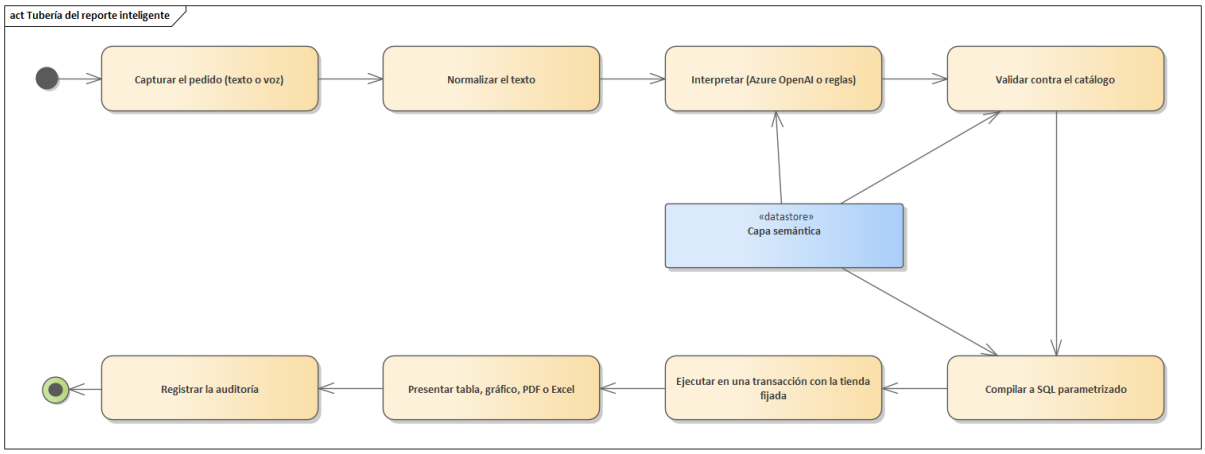
El costo es la expresividad: no se puede pedir lo que el catálogo no describe. Es una decisión consciente, porque en un sistema de gestión es preferible un reporte que dice "ese dato no existe" a uno que inventa una respuesta plausible.

7.6 Arquitectura en tubería

El módulo se organiza como una tubería (pipeline) de ocho etapas. Cada una recibe una representación del pedido y entrega otra más precisa: audio, texto, texto normalizado, especificación propuesta, especificación validada, SQL con parámetros, filas, y finalmente presentación y registro de auditoría.

Figura 7.1

Tubería del reporte inteligente



Nota. Diagrama de actividad UML 2.5: la capa semántica, como almacén de datos, alimenta la interpretación, la validación y la compilación a SQL.

El servicio expone cinco rutas, protegidas por autenticación JWT y por un guardia que solo admite los roles ADMIN, ENCARGADO y SUPERADMIN:

Método	Ruta	Función
GET	/reportes/inteligente/catalogo	Fuentes, campos, operadores, ejemplos y estado del modelo
POST	/reportes/inteligente	Pregunta en lenguaje natural, devuelve el reporte
POST	/reportes/inteligente/interpretar	Pregunta, devuelve solo la especificación
POST	/reportes/inteligente/ejecutar	Especificación editada, devuelve el reporte
POST	/reportes/inteligente/exportar	Especificación o pregunta, devuelve PDF o Excel

7.6.1 Etapa 1: captura

La entrada es un texto de entre 3 y 600 caracteres, validado en el controlador. Puede escribirse o dictarse: en la web, con la Web Speech API del navegador; en el móvil, con el micrófono del teclado del sistema. En ambos casos, el backend recibe texto (sección 7.10).

7.6.2 Etapa 2: normalización

Antes de interpretar conviene reducir la variabilidad superficial. La función normalizarTexto, común a todo el módulo, descompone los caracteres con la forma Unicode NFD, elimina los diacríticos y pasa a minúsculas, de modo que "Sácame" y "SACAME" son idénticos. El intérprete por reglas agrega una segunda capa: quita la puntuación y las palabras "Bs" o "bolivianos" junto a las cifras, elimina el punto de miles ("3.000" pasa a "3000"), convierte la coma decimal y transforma los números dichos en palabras en cifras ("mil quinientos" pasa a "1500").

7.6.3 Etapa 3: interpretación

EnsayAR tiene dos intérpretes con la misma salida. El **intérprete por modelo** envía a Azure OpenAI (deployment gpt-4.1-mini) un prompt con el catálogo y los valores de la tienda, junto con la pregunta recortada a 600 caracteres, y pide JSON (sección 7.8). El **intérprete por reglas** aplica expresiones regulares y heurísticas en español (sección 7.9). Se intenta primero el modelo. Si no está configurado, no responde, devuelve algo que no es JSON o propone una especificación inválida, interviene el intérprete por reglas.

7.6.4 Etapa 4: validación

Toda especificación, venga de donde venga, pasa por validarEspecificacion, que aplica una lista blanca: fuente existente, campos de esa fuente, operadores acordes al tipo, agregaciones con sentido y valores bien formados. También normaliza: completa columnas por defecto, ordena los extremos de un rango y acota el límite (sección 7.12).

7.6.5 Etapa 5: compilación

El compilador traduce la especificación validada a un único SELECT parametrizado. Es una función pura y determinista: la misma especificación produce siempre el mismo SQL y los mismos parámetros. Solo concatena expresiones del catálogo, revisadas a mano, y todo valor del usuario va como parámetro numerado ($1, $2, etc.) con conversión de tipo explícita. Sus reglas principales son estas:

- El reporte se considera agrupado si hay agrupaciones, si alguna columna o filtro lleva agregación o si alguna columna ya es un agregado, como el número de tickets.
- En un reporte agrupado, cada medida recibe la agregación indicada o la agregación por defecto del campo; si no tiene, la suma.
- Todo dato descriptivo que aparezca como columna en un reporte agrupado se convierte en grupo implícito, para que la consulta siempre sea válida.
- El GROUP BY se escribe por posición de columna (GROUP BY 1, 2).
- Los filtros sobre registros van al WHERE y los filtros sobre agregados, al HAVING.
- El ORDER BY usa la posición de la columna, o su expresión si no está en la salida, siempre con NULLS LAST.
- Solo se agregan los joins opcionales que necesitan los campos usados.
- La consulta cierra con un LIMIT entero, ya acotado por el validador.

Sin orden explícito, un reporte agrupado se ordena de mayor a menor por la última medida agregada, y uno sin agrupar, por el orden por defecto de la fuente. Ordenar un reporte agrupado por un dato no agrupado produce el error ORDEN_INVALIDO.

7.6.6 Etapa 6: ejecución aislada por tienda

La consulta se ejecuta en una transacción de Prisma con dos sentencias. La primera es SELECT set_config('app.tienda_id', identificador, true). Según la documentación de PostgreSQL, si el tercer argumento (is_local) es verdadero, el valor solo rige durante la transacción actual. La segunda es la consulta compilada. Cada fuente incluye en su condición base current_setting('app.tienda_id', true), que devuelve el valor fijado o NULL si no existe. Como el filtro pertenece a la definición de la fuente, ningún intérprete puede omitirlo. El identificador proviene del contexto de inquilino de la petición, nunca del texto del usuario.

Luego los resultados se serializan: los decimales y enteros grandes pasan a números y las fechas a cadenas AAAA-MM-DD. Por último, se calculan los totales de las columnas totalizables.

7.6.7 Etapa 7: presentación, narración y exportación

La respuesta incluye el título, el motor que interpretó (modelo, reglas o edición manual), la especificación, una descripción en lenguaje natural de lo entendido, columnas, filas, totales, narración, avisos, SQL, parámetros y tiempo en milisegundos. Las secciones 7.11 y 7.13 describen cómo se usan.

7.6.8 Etapa 8: auditoría

Cada ejecución se guarda en la tabla reporte_ejecuciones, que corresponde al modelo ReporteEjecucion: pregunta (hasta 500 caracteres), SQL ejecutado, cantidad de filas, especificación completa en JSON, usuario y fecha. La tienda se asigna por defecto con current_setting('app.tienda_id'). Guardar la especificación, y no solo el SQL, permite reproducir el reporte: el compilador es determinista y los valores de los filtros están dentro de la

especificación. Las exportaciones vuelven a ejecutar la especificación, así que también quedan registradas.

7.7 La capa semántica de EnsayAR en detalle

7.7.1 Fuentes

El catálogo define seis fuentes. Cada una declara su FROM con los joins obligatorios, una condición base con el aislamiento por tienda y las reglas de negocio, joins opcionales, campos, columnas por defecto, medidas al agrupar, orden por defecto, campo de fecha y medida principal.

Fuente	Cada fila es	Campos	Condición base, además de la tienda
Ventas	Una línea de venta	28	Venta cobrada o devuelta, con unidades no devueltas
Clientes	Una clienta registrada	15	Usuario con rol CLIENTA
Créditos	Una cuenta por cobrar	15	Ninguna
Inventario	Una variante en un almacén	16	Ninguna
Productos	Un producto del catálogo	12	Ninguna
Movimientos	Un movimiento de kardex	13	Ninguna

En total hay 99 campos. Lo que no está en el catálogo no se puede pedir: no existen, por ejemplo, el hash de la contraseña, el teléfono ni el costo de compra.

7.7.2 Anatomía de un campo

Cada campo tiene una **clave** estable (por ejemplo, deuda), una **etiqueta** visible ("Deuda (saldo crédito)"), un **tipo** (texto, entero, numero, dinero, fecha, fechahora o logico) y una **expresión SQL** fija, como COALESCE(cr.deuda, 0). Además declara sus **sinónimos**, sin tildes y en minúsculas; para la deuda son, entre otros, "debe", "deben", "adeuda", "saldo" y "por cobrar". Puede indicar también los **joins** opcionales que necesita y si ya es un **agregado** (como COUNT(DISTINCT v.id)), lo que obliga a agrupar. Otros atributos marcan las **dimensiones** numéricas que no se suman, como el año o la hora; la lista cerrada de **valores** de una

enumeración, como los medios de pago, y la **agregación por defecto**: el precio unitario se promedia, la deuda se suma y los días de mora toman el máximo.

Un campo es **medida** si es entero, numero o dinero y no está marcado como dimensión. Es la misma distinción entre medidas y dimensiones que usan Cube y LookML, y gobierna buena parte de las decisiones del validador y del compilador.

7.7.3 Agregaciones

Agregación	SQL generado	Etiqueta
suma	SUM(e)	Total ...
promedio	ROUND(AVG(e)::numeric, 2)	Promedio ...
conteo	COUNT(e)	N.º de ...
conteo_distinto	COUNT(DISTINCT e)	... distintos
minimo	MIN(e)	... mínimo
maximo	MAX(e)	... máximo

7.7.4 Joins opcionales y cálculos encapsulados

Las expresiones complejas se encapsulan en joins laterales que solo se agregan cuando algún campo los usa:

- **Dirección de la clienta**: un LEFT JOIN LATERAL toma una sola dirección y prefiere la etiquetada "Casa", para no repetir clientas con varias direcciones.
- **Crédito de la clienta**: otro join lateral resume sus cuentas por cobrar (deuda, total vendido a crédito, créditos abiertos, días de mora y próximo vencimiento).
- **Importe neto**: el importe de una línea prorratea el descuento de cupón de la venta y descuenta lo devuelto; para eso, un join lateral calcula el total bruto de la venta.

La unidad de inclusión es el join, no el campo: si el reporte usa solo la deuda, se agrega el join de crédito completo. El catálogo prevé dependencias entre joins (joinsRequieren), aunque ninguna fuente las usa todavía.

7.7.5 Tiempo y zona horaria

Las fechas se guardan en UTC, pero el negocio piensa en la hora de Bolivia. Toda expresión de fecha convierte la columna con AT TIME ZONE 'UTC' AT TIME ZONE 'America/La_Paz', y "hoy" se calcula en esa zona, de modo que "ventas de ayer" significa ayer en La Paz. El mes se representa como texto AAAA-MM, que se ordena cronológicamente de forma alfabética, y el día de la semana se obtiene con EXTRACT(ISODOW) sobre un arreglo de nombres en español.

7.7.6 Datos agregados para el ejemplo del docente

El ejemplo del docente supone ventas al crédito y deudas, que el sistema del primer parcial no tenía. Se incorporaron así:

- El medio de pago "credito" del punto de venta exige una clienta (de lo contrario, responde CREDITO_REQUIERE_CLIENTA), fija un plazo de 30 días y no suma dinero a la caja.
- El modelo CuentaCobrar guarda monto, saldo y vencimiento de cada venta al crédito. Una restricción CHECK exige que el saldo esté entre cero y el monto.
- El modelo AbonoCredito guarda los pagos parciales, con una restricción CHECK de monto positivo. El abono descuenta el saldo con un UPDATE condicional, para que dos abonos simultáneos nunca lo dejen negativo, y es idempotente mediante la cabecera Idempotency-Key.
- Una devolución de una venta al crédito ajusta el saldo de la cuenta.

7.8 Ingeniería de prompts aplicada

La ingeniería de prompts consiste en diseñar las instrucciones y el contexto que recibe un modelo de lenguaje para que resuelva una tarea con precisión. En EnsayAR, la función

promptInterprete arma el prompt, que se envía como mensaje de sistema. Su diseño responde a seis decisiones.

7.8.1 Contexto de esquema

El prompt describe la base en términos de la capa semántica, no de las tablas físicas. Para cada fuente incluye su clave, su descripción y sus campos con tipo, indicando si el campo "ya es total" y, en las enumeraciones, sus valores. También enumera las agregaciones y los operadores por tipo de dato, el formato de fechas (AAAA-MM-DD), la fecha de hoy en la hora de Bolivia, para resolver expresiones como "el mes pasado", y la moneda (Bs). Esto equivale a un enlace de esquema resuelto de antemano: en lugar de buscar entre decenas de tablas, el modelo recibe solo los 99 campos con significado de negocio.

7.8.2 Valores reales del dominio

BIRD mostró que muchos errores provienen de desconocer los valores almacenados. Si el usuario dice "Sopocachi", el modelo no sabe si es una sucursal, una ciudad o una clienta, ni cómo está escrito. EnsayAR arma un **diccionario de la tienda** que guarda en caché 60 segundos por tienda: sucursales, ciudades, categorías y ocasiones (de hasta 500 productos), colores (hasta 100) y proveedores (hasta 100). El prompt incluye hasta 40 valores de cada lista, con la instrucción de usarlos "tal cual". El diccionario también reúne nombres de productos y de clientas, pero solo los usa el intérprete por reglas; el prompt no los envía, lo que reduce su tamaño y evita exponer nombres de personas al servicio externo.

7.8.3 Aprendizaje con pocos ejemplos

Siguiendo a Brown et al. (2020), el prompt incluye cuatro ejemplos resueltos (few-shot), cada uno con un patrón distinto:

1. El caso del docente, con columnas explícitas y un rango de deuda.
2. Un "top 5" de productos más vendidos del mes, con agrupación, orden por suma y formato Excel.
3. Una agrupación por dos niveles con un criterio sobre el total del grupo (HAVING), en PDF.
4. Un pedido que menciona el teléfono, un dato inexistente, resuelto con un aviso.

Los ejemplos también documentan el formato: el modelo aprende la forma exacta de los filtros y del orden con más fidelidad que con una descripción abstracta.

7.8.4 Salida JSON estructurada

La petición a Azure OpenAI incluye response_format con el tipo json_object (JSON mode). Según la documentación de Microsoft, este modo garantiza un JSON válido pero no la adhesión a un esquema, y exige que la palabra "JSON" figure en los mensajes. El prompt la incluye ("RESPONDE SOLO este JSON") junto con la plantilla del objeto esperado. La documentación recomienda, cuando es posible, structured outputs: response_format de tipo json_schema con strict activado, que obliga al modelo a seguir un esquema.

En EnsayAR, la garantía de esquema la da el validador propio, porque hay reglas que dependen del catálogo y que un esquema JSON genérico expresa con dificultad, como que un campo pertenezca a la fuente elegida. Antes de interpretar la respuesta, leerJson elimina posibles delimitadores de bloque de código y rechaza cualquier respuesta que no sea un objeto.

7.8.5 Temperatura y determinismo

La temperatura controla la aleatoriedad del muestreo. EnsayAR usa 0,2, un máximo de 700 tokens y un tiempo límite de 25 segundos. La temperatura baja reduce la variabilidad, pero no la elimina. El determinismo real lo aportan el compilador, que sí es determinista, y la especificación guardada en la auditoría.

7.8.6 Datos inexistentes: avisar, no inventar

La regla más importante del prompt dice: "Si piden un dato que no existe (teléfono, CI, costo…), no lo inventes: omítelo y explícalo en avisos". El servicio acepta hasta cinco avisos del modelo y los muestra al usuario. El intérprete por reglas aplica la misma política con patrones para teléfono, documento de identidad o NIT, edad o cumpleaños, y costo o margen. En esos casos responde, por ejemplo: "El dato teléfono no se registra en el sistema; se omitió del reporte". En un reporte de cobranzas, una columna de teléfonos inventados sería peor que una columna ausente.

Otras reglas completan la guía. Si el usuario no indica columnas, quedan vacías y el sistema pone las habituales. Si se agrupa, las medidas llevan agregación, y un criterio sobre el total de un grupo la lleva en el filtro. "Más vendidos" ordena por suma de unidades o de importe, y "top N" fija el límite. Para nombres parciales se usa el operador contiene, y el formato solo es PDF o Excel si se pide.

7.9 El intérprete por reglas

El intérprete por reglas es un analizador de dominio restringido que produce la misma especificación que el modelo. No compite con un LLM en cobertura lingüística: garantiza que el módulo funcione sin IA y sirve de referencia determinista. Procesa el texto en este orden:

1. **Avisos** de datos inexistentes.
2. **Formato**: "en Excel", "planilla" u "hoja de cálculo" fijan Excel; "en PDF" o "para imprimir", PDF.
3. **Fechas absolutas**, antes que los números, para que "2025" no se lea como monto: rangos ("del 3 al 15 de marzo"), días con mes y año, fechas ISO o con barras, meses sueltos y años. Si el mes nombrado es posterior al actual y no se indica el año, se asume el anterior.

4. **Fechas relativas**: hoy, ayer, esta semana, este mes, el mes pasado, este año, el año pasado, el último trimestre (90 días), el último semestre (180) y "los últimos N días, semanas, meses o años", con un tope de 3650 días.
5. **Límite**: "top 10" o "los 5", si el número no va seguido de una unidad de tiempo o de cantidad.
6. **Fuente**: "sin ventas" lleva a productos; en otro caso, gana la fuente cuyo sinónimo aparece primero en la frase, con una prioridad fija para los empates.
7. **Menciones de campos**, buscando los sinónimos del más largo al más corto y sin solapamientos.
8. **Orden explícito**: "ordenado por deuda de mayor a menor", "alfabético".
9. **Agrupaciones**: "por sucursal", "de cada categoría", "agrupado por mes", encadenadas con comas, "y" o "e" ("por sucursal y medio de pago"), además de rankings y superlativos.
10. **Criterios numéricos**: "de 500 a 3000", "más de", "al menos", "menos de", "hasta". Cada número se asigna a la medida mencionada más cerca o a la que sugiere el contexto; en clientes, "deben" apunta a la deuda. Si hay agrupación, el criterio recibe la agregación por defecto y va al HAVING.
11. **Palabras clave por fuente**: medios de pago y canal en ventas; mayoristas y morosas en clientes; vencidos o pagados en créditos; agotados en inventario; tipos de movimiento en el kardex.
12. **Valores del diccionario**: tallas, sucursales, ciudades, categorías, colores, productos, proveedores y clientas. Gana la coincidencia de mayor puntaje según su longitud, si es completa y si va precedida por el nombre del campo.
13. **Fechas** sobre el campo de fecha de la fuente; si no lo tiene, se avisa.

14. **Orden implícito**: "los que más deben" ordena por deuda descendente; "más vendidos", por unidades o importe; un período agrupado, cronológicamente; en otro caso, por la medida filtrada.
15. **Columnas**: si el usuario nombró dos o más datos, se muestran esos; si no, las habituales. Se agregan la medida filtrada y la de orden, para que se vea el dato que justifica la selección.

Los números dictados se convierten con una pequeña gramática de numerales que trata unidades, decenas, centenas, "mil" y la conjunción "y": "treinta y cinco" pasa a 35 y "mil quinientos", a 1500.

7.10 Reconocimiento de voz

7.10.1 Fundamentos

El reconocimiento automático del habla (ASR) convierte una señal de audio en texto mediante redes neuronales entrenadas con grandes volúmenes de audio transcrito. Whisper (Radford et al., 2022) se entrenó con 680 000 horas de supervisión multilingüe y multitarea y generaliza sin ajuste fino. Para un sistema de reportes, el ASR suma errores propios: palabras mal reconocidas, números en letras, falta de puntuación y nombres del dominio que el modelo acústico desconoce.

7.10.2 Alternativas tecnológicas

- **Web Speech API**: según MDN, tiene una parte de síntesis y otra de reconocimiento asíncrono (SpeechRecognition), que puede usar un servicio de la plataforma del usuario, la opción por defecto, o ejecutarse localmente. No requiere claves, pero depende del navegador.

- **Azure Speech**: ofrece transcripción en tiempo real con resultados intermedios, transcripción rápida, por lotes y modelos personalizados, además de listas de frases para mejorar el reconocimiento de términos del dominio.
- **Whisper**: modelo abierto que puede ejecutarse en un servidor propio o consumirse como servicio, sobre archivos de audio.

7.10.3 Implementación en EnsayAR

La web usa la Web Speech API, configurada con el idioma es-BO, resultados intermedios activados y modo no continuo. El texto parcial aparece en vivo en el cuadro de la pregunta y, al terminar de hablar, si tiene al menos tres caracteres, el reporte se genera sin otro clic. Si el micrófono no tiene permiso, se muestra un mensaje específico, y si el navegador no implementa la API, se sugiere usar Chrome, Edge o el dictado del teclado del celular. En el móvil se usa el micrófono del teclado del sistema, lo que evita dependencias nativas adicionales. Azure Speech y Whisper quedan como alternativas futuras, sobre todo para sesgar el reconocimiento con el vocabulario de la tienda.

7.10.4 Normalización posterior al reconocimiento

Como el backend siempre recibe texto, la robustez frente al ASR depende de la normalización y de la interpretación. El verificador incluye este caso: "dame los clientes que me deben de cien a mil quinientos bolivianos con su correo" debe producir un filtro de deuda entre 100 y 1500. Las fechas relativas se resuelven contra la fecha de hoy en La Paz, que el servicio calcula y entrega tanto al prompt como al intérprete por reglas.

7.11 Ambigüedad y su resolución

En los reportes, la ambigüedad puede ser léxica (una palabra con varios significados), de referencia (a qué entidad alude un nombre), de alcance (si un criterio aplica a registros o a grupos) o de omisión (lo que el usuario da por supuesto). EnsayAR combina cuatro mecanismos para resolverla.

Sinónimos por campo y por fuente. "Clienta", "cliente" y "compradora" designan el mismo campo. La misma palabra puede cambiar de significado según la fuente: "monto" es la deuda en clientes, el monto original en créditos y el importe en ventas. Por eso los sinónimos se resuelven después de elegir la fuente.

Valores del diccionario. Si "Sopocachi" coincide con parte del nombre de la sucursal "La Paz Sopocachi", se genera un filtro de sucursal que contiene ese texto. Las comparaciones ignoran tildes y mayúsculas en ambos lados: en el texto se aplica normalizarTexto, y en SQL, translate(lower(...)) con la misma tabla de equivalencias. Así, "Fernandez" encuentra "Fernández".

Valores por defecto explícitos. Lo omitido se completa con valores declarados en el catálogo: columnas habituales, medidas al agrupar, orden por defecto, límite de 200 filas y formato pantalla. Al estar escritos, son predecibles y documentables.

Transparencia y criterios editables. La pantalla muestra lo entendido como etiquetas: la fuente, cada criterio ("Deuda (saldo crédito) entre Bs 500,00 y Bs 3.000,00"), cada agrupación y el orden. El usuario puede quitar un criterio o una agrupación con la marca × e invertir el orden con un clic. Cada acción reenvía la especificación modificada a la ruta de ejecución con el motor "manual", sin volver a interpretar el texto. Androutsopoulos et al. (1995) describen una solución análoga en NLIDB clásicas como TQA, cuyo módulo de paráfrasis convertía la consulta

interpretada de vuelta a lenguaje natural para que el usuario detectara malentendidos. Los mensajes de error, por su parte, enumeran los datos disponibles en la fuente, lo que ayuda a distinguir un problema de redacción de un dato que no existe. Los avisos completan el mecanismo: datos inexistentes, períodos ignorados, caída del modelo o resultados truncados por el límite.

7.12 Seguridad

La seguridad se diseñó en capas, para que la falla de una no comprometa el sistema.

Valores parametrizados. El compilador nunca inserta valores del usuario en el texto SQL. Cada valor se referencia como $n con conversión explícita (::numeric, ::date, ::text, ::text[] o ::int), y los comodines de LIKE (%, _ y la barra invertida) se escapan. Es la defensa primaria que recomienda OWASP.

Lista blanca de estructura. Fuente, columnas, joins, agregaciones, operadores y dirección de orden solo pueden tomar valores definidos en el código. Cada violación tiene un código propio:

Código	Causa
ESPECIFICACION_VACIA	No llegó un objeto
FUENTE_DESCONOCIDA	La fuente no está en el catálogo
CAMPO_NO_PERMITIDO	El campo no pertenece a la fuente
AGREGACION_DESCONOCIDA	La agregación no existe
AGREGACION_DOBLE	Se agrega algo que ya es un total
AGREGACION_INVALIDA	Suma o promedio de un dato no numérico
OPERADOR_INVALIDO	El operador no corresponde al tipo del campo
VALOR_INVALIDO	Número, fecha, texto o lista mal formados
AGRUPACION_INVALIDA	Agrupar por una medida o más de 4 niveles
DEMASIADAS_COLUMNAS	Más de 20 columnas
DEMASIADOS_FILTROS	Más de 15 criterios

El compilador agrega ORDEN_INVALIDO, y el controlador, FALTA_REPORTE cuando una exportación no trae especificación ni pregunta. El servicio los devuelve como HTTP 400 con el código y un mensaje legible.

Los operadores dependen del tipo. Los textos admiten igual, distinto, contiene, no_contiene, en, vacio y no_vacio. Los números admiten igual, distinto, mayor, mayor_igual, menor, menor_igual y entre. Las fechas admiten, además de igual, desde, hasta y entre, los períodos hoy, ayer, ultimos_dias, esta_semana, este_mes, mes_anterior, este_anio y anio_anterior. Los valores también se acotan: los textos a 120 caracteres, las listas a entre 1 y 30 elementos y los días a entre 1 y 3650. Un valor de enumeración debe coincidir con uno admitido.

Límite de filas. Por defecto se devuelven 200 filas, con un máximo de 2000, y se avisa si el resultado alcanza el límite. Junto con los joins revisados, esto acota el costo de cualquier consulta.

Aislamiento multi-inquilino. La tienda se fija con set_config en la transacción y cada fuente la exige con current_setting. Además, los modelos de Prisma asignan por defecto el identificador de tienda con current_setting('app.tienda_id').

Roles. Solo ADMIN, ENCARGADO y SUPERADMIN usan el módulo; un cajero recibe 403.

El cliente nunca envía SQL ni filas. La exportación recibe una especificación o una pregunta, y el servidor vuelve a validar, compilar y ejecutar. Un cliente alterado no puede pedir un PDF con datos arbitrarios.

Mínima exposición al proveedor de IA. El modelo recibe el catálogo, algunos valores de catálogo de la tienda y la pregunta, pero no filas, nombres de clientas ni montos. Tampoco redacta la narración del resultado.

7.13 Generación del Excel, del PDF y de la narración automática

7.13.1 Excel

El Excel se genera en el servidor con exceljs. La hoja "Reporte" tiene un encabezado con la tienda, el título, los criterios, la fecha de generación, la cantidad de filas, el intérprete y los avisos. Los datos se escriben con su tipo real, así que el usuario puede seguir calculando, y cada columna recibe un formato según su tipo (#,##0.00 para dinero, dd/mm/yyyy para fechas). Al final va una fila TOTAL. La hoja tiene autofiltro, encabezado inmovilizado e impresión apaisada ajustada al ancho. Una segunda hoja, "Cómo se generó", documenta los criterios, el intérprete, los parámetros y el SQL, lo que convierte cada archivo en un artefacto auditable.

7.13.2 PDF

El PDF se genera con pdfkit en A4 apaisado, con el mismo encabezado. El ancho de cada columna depende del largo típico de su contenido (percentil 90 de las primeras 200 filas). Las filas alternan el color de fondo, los números se alinean a la derecha y los textos largos se truncan con puntos suspensivos. Cuando la tabla no cabe, se agrega una página y se repite la fila de títulos. Al final va la fila TOTAL y cada página lleva un pie con el título, la tienda y "Página i de n".

7.13.3 Narración automática

La generación de lenguaje natural (NLG) puede basarse en plantillas, que se completan con valores, o en modelos neuronales que redactan libremente y pueden alucinar cifras. Un número inventado en el resumen de un reporte de gestión es inaceptable, por eso EnsayAR usa plantillas: el código lo define como un "resumen escrito a partir de los números reales del resultado (no lo inventa un modelo)".

La función narrar enuncia la cantidad de filas, la fuente y el total de la primera columna de dinero sumable. Si hay una dimensión, una medida y más de una fila, nombra la fila que encabeza el ranking y su porcentaje del total listado. Por último, enumera los criterios. Si no hay resultados, lo dice y repite los criterios. La fila de totales sigue una regla explícita: se suman las agregaciones de suma y conteo, las medidas cuya agregación por defecto es la suma y los conteos de filas, pero nunca promedios, máximos ni números de tickets distintos, porque su suma no tendría sentido.

7.14 Disponibilidad: cortacircuito y degradación controlada

En los sistemas distribuidos, el patrón cortacircuito (circuit breaker) evita seguir llamando a una dependencia que falla: tras una falla, el circuito "se abre" durante un tiempo y las peticiones toman un camino alternativo sin esperar el error.

EnsayAR implementa una versión simple. El cliente del modelo devuelve null ante cualquier problema: falta de configuración, respuesta HTTP no exitosa (como el 401 actual), error de red o los 25 segundos de espera agotados. Al recibir null, el servicio marca el modelo como caído durante cinco minutos, agrega el aviso "El modelo de IA no respondió; se usó el intérprete local por reglas" y sigue con las reglas. Durante esos cinco minutos, las peticiones van directo a las reglas. La ruta de catálogo informa si el modelo está configurado y disponible, y la pantalla web ajusta su encabezado ("IA generativa" o "Intérprete local"). Si el modelo responde con una especificación inválida, el circuito no se abre, porque el servicio funciona: se avisa del rechazo y se usan las reglas.

Esto es una **degradación controlada**: ante la falla de la IA, el sistema pasa a un modo de menor cobertura lingüística, con la misma seguridad, el mismo formato de salida y la misma auditoría. Hoy la clave de Azure OpenAI responde 401 y el módulo atiende todos los ejemplos

del capítulo con reglas locales. Cuando se renueve la clave, el modelo volverá a intervenir sin cambios de código.

7.15 Evaluación

7.15.1 Verificación automatizada del módulo

El proyecto incluye un verificador de extremo a extremo (npm run verificar:reportes), que requiere el backend levantado y los datos sintéticos de npm run datos:reportes. Según la documentación del proyecto, esos datos son 34 clientas, 320 ventas en 180 días y créditos con abonos. El verificador tiene 24 chequeos obligatorios y uno condicional:

Grupo	Qué se comprueba
Intérprete por reglas (9, uno condicional)	Ejemplo del docente; números dictados; top 10 con período y Excel; agrupación por dos niveles; créditos vencidos en PDF; HAVING con meses en orden; "2000 a 5000" como monto y no como año; valores reales de la tienda (si existe la sucursal Sopocachi); aviso por teléfono
Compilador y ejecución (5)	Siete especificaciones corren en PostgreSQL; coincidencia con un cálculo independiente; montos como parámetros; aislamiento por tienda; rechazo de campos, fuentes y operadores inválidos
API, exportación y permisos (5)	Ruta principal; Excel válido con hoja técnica; PDF de una página y PDF paginado sin hojas en blanco; inyección tratada como texto; 400 por campo prohibido y 403 para cajero
Crédito y abonos (6)	Crédito sin clienta rechazado; cuenta creada sin sumar a caja; abono idempotente; abonos simultáneos sin saldo negativo; listado de créditos; devolución que cancela la deuda

El chequeo de inyección envía como valor de un filtro el texto "x'; DROP TABLE ventas; --". El reporte devuelve cero filas y la tabla de ventas sigue existiendo, porque el valor viaja como parámetro y se trata como texto.

7.15.2 Comparación con SQL independiente

El chequeo más importante se inspira en la exactitud de ejecución. Para el ejemplo del docente, el verificador calcula con una consulta escrita a mano cuántas clientas tienen un saldo

total en el rango: agrupa cuentas_cobrar por clienta y filtra con HAVING SUM(saldo) BETWEEN 100 AND 500. Luego comprueba que el reporte devuelve esa misma cantidad de filas y que todas las deudas listadas están dentro del rango. Así, el motor se contrasta con una fuente de verdad que no comparte código con el compilador.

7.15.3 Cómo medir la precisión del intérprete con IA

Los chequeos anteriores validan las reglas y el resto de la tubería. Para medir el intérprete con modelo cuando la clave esté vigente, se propone este protocolo, inspirado en Spider y BIRD:

1. Reunir unas cien preguntas redactadas por personas ajenas al desarrollo, que cubran las seis fuentes e incluyan agrupaciones, HAVING, fechas relativas, valores del diccionario, datos inexistentes y dictado por voz.
2. Escribir a mano la especificación de referencia de cada pregunta.
3. Medir por componente, al estilo de la coincidencia exacta de Spider: fuente, precisión y exhaustividad de columnas, filtros y agrupaciones, orden, límite y formato.
4. Medir la exactitud de ejecución comparando las filas de la especificación de referencia con las de la generada.
5. Calcular la tasa de validez, es decir, cuántas respuestas del modelo superan el validador sin recurrir a las reglas.
6. Correr el mismo conjunto con el modelo y con las reglas, para cuantificar la ganancia de la IA.
7. Repetir cada pregunta varias veces para medir la estabilidad con temperatura 0,2.

Comparar especificaciones componente por componente es mucho más simple que comparar SQL, otra ventaja de la representación intermedia.

7.16 Ejemplo completo paso a paso

Esta sección recorre la tubería con el pedido del docente, usando una respuesta real del servicio en la tienda de demostración. Los datos de las clientas son sintéticos.

7.16.1 Pedido

El administrador envía a POST /reportes/inteligente el texto: "Sácame una lista de los clientes que me muestre su nombre, su dirección, su email, de todos aquellos clientes que me deben de 500 a 3000 bolivianos por ventas al crédito".

7.16.2 Normalización e interpretación

El modelo no responde (la clave da 401), así que el servicio abre el cortacircuito, agrega el aviso y usa las reglas. La normalización elimina tildes y mayúsculas y quita "bolivianos" junto a la cifra, de modo que queda "de 500 a 3000". Luego el intérprete:

1. No encuentra formato, por lo que deja pantalla.
2. No encuentra fechas: el patrón de años solo reconoce cifras 20XX precedidas de "en", "del" o "año".
3. Elige la fuente clientes, cuyo sinónimo aparece primero; "crédito" y "ventas" vienen después.
4. Detecta nombre, dirección, email y "deben", sinónimo de la deuda.
5. Reconoce "de 500 a 3000" como rango y lo asigna a la medida más cercana, la deuda.
6. No agrega "deuda mayor a cero", que la palabra "deben" dispararía, porque ya hay un filtro sobre la deuda.
7. Ordena por la medida filtrada, de mayor a menor.
8. Muestra exactamente los datos nombrados.

7.16.3 Especificación validada

```
{
  "titulo": "Clientes · Deuda (saldo crédito) entre Bs 500,00 y Bs 3.000,00",
  "fuente": "clientes",
  "columnas": [
    { "campo": "nombre" },
    { "campo": "direccion" },
    { "campo": "email" },
    { "campo": "deuda" }
  ],
  "filtros": [
    { "campo": "deuda", "op": "entre", "valor": 500, "valor2": 3000 }
  ],
  "agrupar": [],
  "orden": [{ "campo": "deuda", "dir": "desc" }],
  "limite": 200,
  "formato": "pantalla"
}
```

El validador aceptó la fuente, los cuatro campos y el operador entre, admitido para el tipo dinero. Generó el título automáticamente, porque las reglas no lo proponen, y fijó el límite por defecto.

7.16.4 SQL generado y parámetros

```
SELECT COALESCE(NULLIF(u.nombre, ''), u.email) AS "nombre",
       COALESCE(dir.detalle, 'Sin dirección') AS "direccion",
       u.email AS "email",
       COALESCE(cr.deuda, 0) AS "deuda"
FROM usuarios u JOIN cuentas cu ON cu.id = u.cuenta_id
LEFT JOIN LATERAL (
    SELECT d.detalle, d.ciudad FROM direcciones_cliente d
    WHERE d.usuario_id = u.id
    ORDER BY (d.etiqueta = 'Casa') DESC, d.id LIMIT 1) dir ON true
LEFT JOIN LATERAL (
    SELECT COALESCE(SUM(c.saldo), 0) AS deuda, COALESCE(SUM(c.monto), 0) AS fiado,
           COUNT(*) FILTER (WHERE c.saldo > 0)::int AS abiertos,
           COALESCE(MAX(GREATEST(0, (now() AT TIME ZONE 'America/La_Paz')::date - (c.vence_en AT
TIME ZONE 'UTC' AT TIME ZONE 'America/La_Paz')::date)) FILTER (WHERE c.saldo > 0), 0)::int AS
mora,
           MIN(c.vence_en) FILTER (WHERE c.saldo > 0) AS proximo
    FROM cuentas_cobrar c WHERE c.clienta_id = u.id) cr ON true
WHERE u.tienda_id = current_setting('app.tienda_id', true) AND u.rol = 'CLIENTA'
  AND COALESCE(cr.deuda, 0) BETWEEN $1::numeric AND $2::numeric
ORDER BY 4 DESC NULLS LAST
LIMIT 200
```

Los parámetros son [500, 3000]. Solo se agregaron los joins de dirección y crédito; el de compras quedó fuera porque ningún campo pedido lo usa. El aislamiento por tienda y el rol CLIENTA vienen de la condición base de la fuente. Los montos no aparecen en el texto, sino

como $1 y $2. Antes, en la misma transacción, se ejecutó set_config('app.tienda_id', identificador de la tienda, true).

7.16.5 Resultado

La consulta tardó 30 milisegundos y devolvió 8 filas, con una deuda total de Bs 7.709,19. Las tres primeras:

Nombre	Dirección	Email	Deuda (Bs)
Elena Gutiérrez	Av. América #1956, zona Barrio Hamacas	elena.gutierrez@sint.ensayar.bo	1.925,05
Andrea Soliz	Av. San Martín #2774, zona Equipetrol	andrea.soliz@sint.ensayar.bo	1.153,15
Wendy Salazar	Av. San Martín #2236, zona Obrajes	wendy.salazar@sint.ensayar.bo	1.145,00

7.16.6 Narración y avisos

La narración fue: "8 filas de clientes, con deuda (saldo crédito) de Bs 7.709,19. Encabeza Elena Gutiérrez con Bs 1.925,05 (25% del total listado). Criterios: Deuda (saldo crédito) entre Bs 500,00 y Bs 3.000,00." El único aviso fue: "El modelo de IA no respondió; se usó el intérprete local por reglas." En pantalla, el usuario ve las etiquetas de la fuente, el criterio y el orden, y puede quitar el criterio, invertir el orden o descargar el reporte en Excel o PDF. Si el pedido hubiera terminado con "en Excel", el archivo se habría solicitado automáticamente.

7.17 Ejemplos complementarios: agrupación y criterio sobre totales

7.17.1 Agrupación por dos niveles

Pedido: "ventas por sucursal y medio de pago de los últimos 180 días". Llegó dentro de los cinco minutos posteriores a la caída detectada en el ejemplo anterior, así que el cortacircuito lo envió directo a las reglas, sin intentar el modelo; por eso no trae avisos. El intérprete eligió ventas, encadenó las dos dimensiones, tradujo el período a ultimos_dias con valor 180 y usó las medidas por defecto de la fuente al agrupar.

```
{
  "fuente": "ventas",
  "columnas": [{ "campo": "tickets" }, { "campo": "importe" }],
  "filtros": [{ "campo": "fecha", "op": "ultimos_dias", "valor": 180 }],
  "agrupar": ["sucursal", "medio_pago"],
  "orden": [{ "campo": "importe", "agregacion": "suma", "dir": "desc" }],
  "limite": 200,
  "formato": "pantalla"
}
```

```
SELECT s.nombre AS "sucursal",
       v.medio_pago::text AS "medio_pago",
       COUNT(DISTINCT v.id) AS "tickets",
       SUM(ROUND((lv.cantidad - lv.cantidad_devuelta) * lv.precio_unitario * v.total /
NULLIF(br.total, 0), 2)) AS "suma_importe"
FROM lineas_venta lv
  JOIN ventas v ON v.id = lv.venta_id
  JOIN variantes va ON va.id = lv.variante_id
  JOIN productos p ON p.id = va.producto_id
  JOIN almacenes a ON a.id = v.almacen_id
  JOIN sucursales s ON s.id = a.sucursal_id
JOIN LATERAL (SELECT SUM(x.cantidad * x.precio_unitario) AS total FROM lineas_venta x WHERE
x.venta_id = v.id) br ON true
WHERE lv.tienda_id = current_setting('app.tienda_id', true) AND v.estado IN ('cobrada',
'devuelta_parcial', 'devuelta') AND lv.cantidad > lv.cantidad_devuelta
  AND ((v.creada_en AT TIME ZONE 'UTC' AT TIME ZONE 'America/La_Paz')::date)::date BETWEEN (now()
AT TIME ZONE 'America/La_Paz')::date - ($1::int - 1) AND (now() AT TIME ZONE
'America/La_Paz')::date
GROUP BY 1, 2
ORDER BY 4 DESC NULLS LAST
LIMIT 200
```

El parámetro es [180]. El importe, sin agregación en la especificación, recibió en el compilador su agregación por defecto (suma) y pasó a llamarse suma_importe. Los tickets ya son un agregado y se usan tal cual. El join del total bruto se agregó porque lo necesita la expresión del importe.

Resultaron 12 grupos, con un importe total de Bs 227.518,58, en 73 milisegundos. Los tres primeros:

Sucursal	Medio de pago	Tickets	Total importe (Bs)
Santa Cruz Centro	efectivo	81	56.884,80
Santa Cruz Centro	qr	51	34.584,93
La Paz Sopocachi	efectivo	51	31.829,17

La narración fue: "12 filas de ventas, con total importe de Bs 227.518,58. Encabeza Santa Cruz Centro · efectivo con Bs 56.884,80 (25% del total listado). Criterios: Fecha en los últimos 180 días." Al estar agrupado y tener entre 2 y 15 filas, la web muestra además un gráfico de

barras. Entre los grupos aparece el medio "credito" (19 tickets en Sopocachi y 15 en Santa Cruz Centro), lo que confirma que las ventas al crédito ya forman parte del análisis.

7.17.2 Criterio sobre el total del grupo (HAVING)

Pedido: "ventas por mes del último semestre con más de 2000 Bs". Como el reporte está agrupado por mes, "más de 2000" se refiere al total de cada mes. El intérprete asignó el número a la medida principal de ventas (importe) con agregación suma.

```
{
  "fuente": "ventas",
  "columnas": [{ "campo": "tickets" }, { "campo": "importe" }],
  "filtros": [
    { "campo": "importe", "op": "mayor", "agregacion": "suma", "valor": 2000 },
    { "campo": "fecha", "op": "ultimos_dias", "valor": 180 }
  ],
  "agrupar": ["mes"],
  "orden": [{ "campo": "mes", "dir": "asc" }],
  "limite": 200,
  "formato": "pantalla"
}
```

El SQL se muestra abreviado. Los puntos suspensivos reemplazan el FROM, los joins y la condición base, idénticos a los del ejemplo anterior:

```
SELECT to_char((v.creada_en AT TIME ZONE 'UTC' AT TIME ZONE 'America/La_Paz'), 'YYYY-MM') AS
"mes",
       COUNT(DISTINCT v.id) AS "tickets",
       SUM(ROUND((lv.cantidad - lv.cantidad_devuelta) * lv.precio_unitario * v.total /
NULLIF(br.total, 0), 2)) AS "suma_importe"
...
  AND ((v.creada_en AT TIME ZONE 'UTC' AT TIME ZONE 'America/La_Paz')::date)::date BETWEEN (now()
AT TIME ZONE 'America/La_Paz')::date - ($2::int - 1) AND (now() AT TIME ZONE
'America/La_Paz')::date
GROUP BY 1
HAVING SUM(ROUND((lv.cantidad - lv.cantidad_devuelta) * lv.precio_unitario * v.total /
NULLIF(br.total, 0), 2)) > $1::numeric
ORDER BY 1 ASC NULLS LAST
LIMIT 200
```

Los parámetros son [2000, 180]. El criterio con agregación fue al HAVING y el de fecha al WHERE. Los parámetros se numeran según el orden de los filtros en la especificación, no según las cláusulas, por eso el monto es $1. El orden es cronológico porque se agrupa por un período y no se pidió límite.

Resultaron siete meses, de 2026-05 a 2026-11, en 42 milisegundos:

Mes	Tickets	Total importe (Bs)
2026-05	20	16.199,45
2026-06	56	33.404,30
2026-07	38	30.842,61
2026-08	61	42.803,75
2026-09	57	38.473,46
2026-10	65	43.306,92
2026-11	38	22.488,09

Con los datos de demostración, todos los meses superan los Bs 2.000, así que el HAVING no excluye ninguno; con un umbral de 30 000 Bs, en cambio, se filtrarían varios. Mayo aparece incompleto porque los 180 días empiezan a mitad de ese mes, y noviembre está en curso.

7.18 Limitaciones y trabajo futuro

Cobertura del catálogo. Solo se puede pedir lo que describen las seis fuentes y sus 99 campos. Quedan fuera, por ejemplo, las reservas para probador, las suscripciones, los turnos de caja, las devoluciones como fuente propia y el costo de compra, que no se registra. Agregar una fuente no modifica el compilador ni el validador.

Una fuente por reporte. Las preguntas que cruzan granularidades distintas, como "stock y venta del mes por producto y sucursal", solo se resuelven en parte con joins laterales. Una extensión posible son las fuentes compuestas, al estilo de los explores de LookML.

Gráficos y edición limitados. La web solo muestra barras para reportes agrupados de 2 a 15 filas, y el móvil no muestra gráficos. La especificación podría incluir un tipo de visualización (líneas para series de tiempo, barras apiladas para dos dimensiones), también presente en el PDF. Además, la web permite quitar criterios e invertir el orden, pero no cambiar el valor de un criterio ni agregar uno nuevo, y en el móvil los criterios no se editan.

Sinónimos estáticos. Los sinónimos están en el código. La tabla de ejecuciones, que ya guarda la pregunta y la especificación, permitiría aprender sinónimos: detectar los reportes que el

usuario corrige, proponer sinónimos nuevos a un administrador e incorporarlos al catálogo de la tienda.

Aislamiento a nivel de aplicación. El filtro por tienda está en cada fuente, pero la base no lo impone. Según la documentación de PostgreSQL, con Row-Level Security habilitado, todo acceso debe estar permitido por una política, y sin políticas rige una denegación por defecto. Políticas basadas en current_setting('app.tienda_id'), con un rol de solo lectura para el módulo, sumarían una barrera independiente del código. La misma documentación advierte que los superusuarios, los roles con BYPASSRLS y normalmente los dueños de las tablas no quedan sujetos a las políticas, así que el rol de la aplicación debe ser distinto del propietario.

Caché. Hoy solo el diccionario se guarda 60 segundos. Podrían guardarse también las interpretaciones de preguntas idénticas y, por poco tiempo, los resultados de especificaciones idénticas. La dbt Semantic Layer documenta una caché de resultados similar.

Salida estricta y voz del dominio. Pasar de JSON mode a structured outputs, con un esquema derivado del catálogo, reduciría las respuestas inválidas. Integrar Azure Speech con listas de frases tomadas del diccionario mejoraría el reconocimiento de sucursales, productos y proveedores.

Evaluación pendiente del modelo. Con la clave actual no fue posible medir el intérprete con IA. El protocolo de 7.15.3 queda listo para aplicarse cuando se renueve.

7.19 Conclusiones del capítulo

Generar reportes dinámicos con IA no se reduce a pedirle SQL a un modelo. Para que un sistema de gestión pueda confiar en esos reportes, la IA debe ocupar un lugar preciso dentro de una arquitectura mayor. En EnsayAR, ese lugar es la traducción del lenguaje natural a una

especificación que expresa, en términos del negocio, fuente, columnas, criterios, agrupaciones, agregaciones, criterios sobre totales, orden, límite y formato.

Alrededor de esa traducción, los componentes deterministas aportan lo que el modelo no garantiza: un catálogo que define qué se puede pedir, un validador por lista blanca, SQL parametrizado y reproducible, aislamiento por tienda, una narración que no inventa cifras, exportaciones auditables y un registro de cada ejecución. La degradación al intérprete por reglas asegura que el negocio no se quede sin reportes cuando la IA falla, como demuestra el estado actual del proyecto.

El diseño combina la representación intermedia de las primeras NLIDB, la capa semántica de la inteligencia de negocios y la capacidad lingüística de los modelos actuales. Así, un usuario puede pedir con su voz la lista de clientas que le deben entre 500 y 3000 bolivianos, con nombre, dirección y email, y recibirla en segundos en pantalla, en PDF o en Excel.

8. Analítica con aprendizaje automático

8.1 Introducción y alcance

Los reportes del capítulo 7 responden preguntas sobre lo que ya ocurrió: cuánto se vendió, en qué sucursal, con qué medio de pago. La analítica con aprendizaje automático responde dos preguntas de otra naturaleza. La primera es predictiva: cuánto se venderá de cada producto en las próximas semanas, para decidir la reposición. La segunda es descriptiva pero no evidente a simple vista: qué grupos de clientas con comportamientos parecidos existen, para dirigir a cada grupo una acción comercial distinta.

La consigna del segundo parcial exige aplicar un modelo supervisado y uno no supervisado, entrenarlos con datos sintéticos generados para el caso y presentar los resultados en tableros. Este capítulo expone los fundamentos de cada técnica, justifica los modelos elegidos para una tienda de moda y describe el diseño del módulo de analítica de EnsayAR, modelado en los casos de uso UC52 (ver tablero predictivo) y UC53 (ver segmentación de clientas).

8.2 Fundamentos del aprendizaje automático

8.2.1 Aprendizaje supervisado

En el aprendizaje supervisado se dispone de ejemplos formados por un conjunto de variables de entrada y el valor que se desea predecir, llamado etiqueta. El algoritmo ajusta una función que relaciona entradas y etiqueta de modo que el error sobre los ejemplos conocidos sea pequeño y, sobre todo, que el modelo generalice a casos nuevos (Hastie et al., 2009). Si la etiqueta es numérica, como las unidades vendidas, el problema es de regresión; si es una categoría, como "volverá a comprar" o "no volverá", es de clasificación.

8.2.2 Aprendizaje no supervisado

En el aprendizaje no supervisado los ejemplos no tienen etiqueta. El objetivo es descubrir estructura en los datos, por ejemplo agrupar observaciones similares (agrupamiento o clustering), reducir el número de variables conservando la información o detectar observaciones atípicas. Como no existe una respuesta correcta con la cual comparar, la evaluación se basa en medidas internas de calidad de los grupos y en la utilidad de la interpretación para el negocio.

8.2.3 Entrenamiento, validación y prueba

Un modelo que memoriza los ejemplos de entrenamiento obtiene un error muy bajo sobre ellos y uno alto sobre datos nuevos; este fenómeno se denomina sobreajuste. Para detectarlo, los datos se dividen en un conjunto de entrenamiento, con el que se ajusta el modelo; uno de validación, con el que se eligen sus hiperparámetros; y uno de prueba, que solo se usa al final para estimar el desempeño real. Cuando los datos son series de tiempo, la división debe respetar el orden cronológico: se entrena con el pasado y se evalúa con el futuro, porque mezclar semanas posteriores en el entrenamiento produce estimaciones optimistas (Hyndman y Athanasopoulos, 2021).

8.2.4 Métricas de evaluación

Para la regresión se emplean el error absoluto medio (MAE), que expresa el error promedio en las mismas unidades de la variable; la raíz del error cuadrático medio (RMSE), que penaliza más los errores grandes; y el error porcentual absoluto medio (MAPE), que facilita la comparación entre productos de distinto volumen, aunque se vuelve inestable cuando las ventas reales son cercanas a cero. Para el agrupamiento se utiliza el coeficiente de silueta, que compara la distancia media de cada punto a su propio grupo con la distancia al grupo vecino más cercano

y toma valores entre −1 y 1; valores cercanos a 1 indican grupos compactos y bien separados (Rousseeuw, 1987).

8.3 Datos sintéticos

8.3.1 Justificación

Una tienda que se suscribe a EnsayAR no tiene historia en la plataforma, y las tiendas de demostración tienen pocas semanas de ventas. Entrenar un modelo de demanda requiere al menos uno o dos ciclos estacionales completos. Los datos sintéticos resuelven este problema: se generan con un proceso estadístico controlado que reproduce los patrones que el modelo debe aprender. Además, como los patrones se conocen de antemano, permiten verificar que el modelo los recupera, algo imposible con datos reales cuya estructura se desconoce (Jordon et al., 2022).

8.3.2 Proceso de generación

Los datos sintéticos se generan en memoria, dentro del propio backend, con un generador congruencial lineal de semilla fija (`backend/src/analitica/modelos.ts`). Cada ejecución produce exactamente los mismos ejemplos, de modo que las métricas informadas en este capítulo son reproducibles.

Para el modelo de demanda se generan 600 ejemplos. Cada ejemplo representa una variante con sus unidades vendidas en los últimos 7 días y en los últimos 28 días; la etiqueta es la demanda de los 7 días siguientes. La relación de fondo es lineal, con mayor peso para la venta reciente, y se le suma un ruido uniforme que imita la variabilidad de las ventas de pocas unidades. El valor se acota en cero, porque no existen ventas negativas.

Para la segmentación se generan 180 clientas repartidas en partes iguales entre tres perfiles ocultos, descritos con las variables de recencia, frecuencia y monto de la sección 8.5.

Perfil oculto	Días desde la última compra	Número de compras	Monto acumulado
Frecuente de alto gasto	3 a 20	6 a 13	700 a 1700
Ocasional	20 a 50	2 a 6	150 a 650
Inactiva	50 a 90	1 a 3	20 a 180

El monto se expresa en unidades sintéticas, no en la moneda de una tienda. El perfil no forma parte de las variables de entrada: solo sirve para comprobar después si el agrupamiento reencuentra la estructura conocida.

8.3.3 Separación de los datos reales

Los ejemplos sintéticos solo existen en memoria durante el entrenamiento; no se insertan en las tablas de ventas ni de clientas de ninguna tienda, por lo que no pueden mezclarse con la operación real. Los modelos ya entrenados se aplican a los datos reales de la tienda autenticada, siempre a través del contexto multi-tenant descrito en el capítulo 4, y el tablero muestra un aviso permanente que indica que los modelos se entrenaron y evaluaron con datos sintéticos. Para las pruebas de integración se crearon tiendas, productos y ventas sintéticos en la base de laboratorio, separada de la base de producción.

8.4 Modelo supervisado: pronóstico de demanda

8.4.1 Formulación

El problema se formula como una regresión: estimar las unidades que se venderán de cada variante en los próximos 7 días a partir de dos variables, las unidades vendidas en los últimos 7 días y en los últimos 28 días. Las ventas se toman netas de devoluciones. Se eligieron pocas variables porque son las únicas que cualquier tienda tiene desde sus primeras semanas en la plataforma; variables como la estacionalidad anual o el efecto de las fechas comerciales exigen al menos un año de historia.

8.4.2 Algoritmo

Se utiliza una regresión lineal ajustada por descenso de gradiente sobre variables estandarizadas. La media y la desviación estándar de cada variable se calculan solo con el conjunto de entrenamiento y se aplican después al de prueba, para que ninguna información de la prueba influya en el ajuste. El modelo parte de la media de la etiqueta y ajusta sus pesos durante 600 épocas con una tasa de aprendizaje de 0,05; las predicciones negativas se llevan a cero.

La regresión lineal se eligió por tres razones: es interpretable, porque cada peso indica cuánto aporta cada variable a la estimación; no requiere bibliotecas externas, lo que permite ejecutarla dentro del mismo backend NestJS; y su costo de entrenamiento y de predicción es despreciable. Como referencia se usa una línea base que predice siempre la media del entrenamiento. Un modelo solo aporta valor si mejora claramente esa referencia (Hastie et al., 2009).

8.4.3 Entrenamiento y evaluación

Los 600 ejemplos se dividen antes de estimar cualquier parámetro: los primeros 480 se usan para entrenar y los 120 restantes para la prueba. Los resultados sobre el conjunto de prueba son los siguientes.

Modelo	**MAE en prueba (unidades)**
Línea base: media del entrenamiento	6,21
Regresión lineal	1,23

La regresión reduce el error absoluto medio en un 80 % respecto de la línea base. Estas cifras miden el ajuste del modelo a la distribución sintética generada; no estiman la precisión con ventas reales, que deberá medirse con una validación cronológica cuando las tiendas acumulen historia suficiente (sección 8.2.3). Las pruebas unitarias de `modelos.spec.ts` verifican que el entrenamiento sea reproducible y que el modelo supere a la línea base.

8.4.4 Uso del resultado

El tablero aplica el modelo a las ventas cobradas de los últimos 28 días de la tienda y muestra hasta 20 variantes, ordenadas por demanda estimada, junto con sus ventas de 7 y 28 días y el stock disponible, calculado como la cantidad menos lo reservado y sumado en todas las sucursales. La comparación entre la demanda estimada y el disponible orienta la reposición, pero el sistema no genera compras automáticamente: la decisión sigue siendo del encargado.

8.5 Modelo no supervisado: segmentación de clientas

8.5.1 Variables RFM

La segmentación se basa en el análisis RFM, una técnica clásica del marketing de bases de datos que describe a cada cliente con tres variables: recencia, frecuencia y monto (Fader et al., 2005). En EnsayAR se calculan sobre las ventas cobradas de los últimos 90 días: días desde la última compra, número de compras y monto acumulado. Como el monto tiene una escala mucho mayor que las otras dos variables y dominaría el cálculo de distancias, las tres se estandarizan con la media y la desviación del conjunto de entrenamiento.

8.5.2 Algoritmo

Se utiliza el algoritmo k-medias (Lloyd, 1982), que divide las observaciones en k grupos de forma que cada una pertenezca al grupo con el centro más cercano. Parte de k centros iniciales, asigna cada clienta al centro más próximo, recalcula cada centro como el promedio de sus clientas y repite hasta que los centros dejan de moverse. La implementación toma como centros iniciales los tres primeros ejemplos y se detiene cuando el desplazamiento máximo de los centros es menor que 10^{-8} o tras 100 iteraciones. Es un algoritmo eficiente y fácil de explicar, y sus centros pueden leerse directamente como el perfil promedio de cada segmento.

8.5.3 Número de grupos y evaluación

Se fijó k = 3, el número de perfiles con que se generaron los datos. Las 180 clientas sintéticas se dividen en 135 para entrenar y 45 para evaluar, y sobre estas últimas se calcula el coeficiente de silueta medio, que resulta de 0,55. Según Rousseeuw (1987), un valor superior a 0,5 indica una estructura de grupos razonable, de modo que el algoritmo recupera los tres perfiles generados. Cuando existan datos reales, el número de grupos deberá elegirse comparando la silueta para valores de k entre 2 y 8, porque la cantidad de perfiles reales no se conoce de antemano.

8.5.4 Interpretación

El tablero clasifica en uno de los tres grupos a cada clienta con compras identificadas en los últimos 90 días y muestra, para cada grupo, la cantidad de clientas y su centro expresado en las tres variables. Los grupos se presentan numerados y nunca se exponen las identidades de las clientas. La lectura comercial es orientativa: el grupo de compras frecuentes y gasto alto es el destinatario natural de un aviso anticipado de nueva colección, el de compras ocasionales responde mejor a promociones y el de clientas inactivas a una campaña de reactivación.

8.6 Tablero

El tablero se encuentra en la página `/analitica` de la aplicación web y corresponde a los casos de uso UC52 y UC53. Solo pueden abrirlo el administrador y el superadministrador de una tienda cuyo plan vigente incluya el módulo de analítica. Presenta un resumen de los datos usados (ventas cobradas en los 90 días, clientas con compras y variantes con ventas en los últimos 28 días), la tabla de demanda estimada con el stock disponible, la tabla de segmentos y las métricas de evaluación de ambos modelos, junto con el aviso de que se entrenaron con datos sintéticos.

El diseño sigue el principio de que un tablero debe permitir comprender la situación de un vistazo, con pocos indicadores bien elegidos y sin decoración superflua (Few, 2006). Cuando la tienda no tiene ventas en el período, el tablero muestra un estado vacío explicativo. Si el período supera las 5000 ventas, el servicio responde con el error 422 `DEMASIADAS_VENTAS` en lugar de calcular sobre una muestra truncada que daría resultados engañosos.

8.7 Arquitectura del módulo

La analítica es un módulo más del monolito modular NestJS (`backend/src/analitica/`), compuesto por un controlador, un servicio y el archivo de modelos. El endpoint `GET /analitica/panel` está protegido por `JwtAuthGuard` y `RolesGuard`, que exigen una sesión válida con rol `ADMIN` o `SUPERADMIN`; además, el servicio verifica que la suscripción esté vigente y que el plan incluya el módulo, y en caso contrario responde 403 `MODULO_NO_INCLUIDO`. Ambos modelos se entrenan una sola vez al cargar el módulo, en milisegundos, y quedan en memoria; cada consulta solo aplica los modelos a los datos de la tienda.

La lectura de datos funciona en los dos modos de aislamiento descritos en el capítulo 4. En el modo habitual usa `PrismaService` sobre las tablas compartidas, filtrando por `tiendaId`; cuando se activa el modo de esquema por tienda, usa el cliente del esquema de la tienda autenticada que entrega `PrismaEsquemasService` (sección 4.12). La tienda se obtiene siempre del contexto de la petición, nunca de un parámetro del cliente.

En esta versión, las predicciones y los segmentos se calculan en cada consulta; las entidades `ModeloPredictivo` y `SegmentoCliente` del modelo de dominio quedan reservadas para persistir versiones de los modelos cuando se entrenen con datos reales. Si en el futuro se adoptan modelos más complejos, como un bosque aleatorio de regresión (Breiman, 2001)

implementado con scikit-learn (Pedregosa et al., 2011), el entrenamiento será el primer candidato a separarse del monolito como un servicio independiente, tal como anticipa la sección 6.10.

8.8 Estado de implementación y verificación

El módulo está implementado en el backend y en la web. Las pruebas unitarias de `modelos.spec.ts` y `analitica.service.spec.ts` cubren la reproducibilidad del entrenamiento, la comparación con la línea base y el comportamiento del servicio. El script `backend/scripts/verificar-analitica-http.ts`, ejecutado sobre la base de laboratorio, comprobó por HTTP los siguientes casos:

- Una petición sin sesión recibe 401.
- Un rol no autorizado, como el cajero, recibe 403.
- Una tienda cuyo plan no incluye el módulo de analítica recibe 403.
- Una tienda con el plan Cadena recibe el tablero completo (200), tanto en tablas compartidas como en esquemas por tienda.
- La cabecera `X-Tienda` no permite consultar los datos de otra tienda.
- Una tienda sin ventas en el período recibe el estado vacío.

Los resultados son exploratorios. Falta la validación cronológica con ventas reales, el tratamiento de varias monedas, la estacionalidad anual y la medición de la capacidad con volúmenes de producción; por ello el tablero no debe usarse por sí solo para decidir compras ni para evaluar personas.

9. Proceso de desarrollo con ICONIX

El primer ciclo de EnsayAR se documentó con el Proceso Unificado, cuyo objetivo en ese momento era estabilizar una arquitectura que todavía no existía. El segundo ciclo parte de otra situación: la plataforma funciona, está desplegada y su estructura de monolito modular está probada. Lo que ahora se exige es incorporar capacidades nuevas (modelo SaaS multi-tienda, operación sin conexión con sincronización, reportes generativos con ventas al crédito, analítica con aprendizaje automático y pagos internacionales) sin perder el control sobre qué requisito se resuelve en qué parte del código. Por indicación de la cátedra, el proceso de este ciclo es ICONIX. Este capítulo expone su fundamento teórico y la forma en que se aplica al proyecto.

9.0 La metodología ICONIX

ICONIX es un proceso de análisis y diseño orientado a objetos que conduce el desarrollo desde los casos de uso hasta el código mediante una secuencia corta de modelos UML. Usa pocos diagramas, exige que cada uno derive del anterior e introduce un paso intermedio, el análisis de robustez, que obliga a convertir el texto de cada caso de uso en una primera imagen de los objetos que lo realizan.

9.0.1 Origen y autores

El proceso surge en ICONIX Software Engineering, empresa fundada por Doug Rosenberg en 1984 y dedicada desde comienzos de los años noventa a la capacitación en análisis y diseño orientado a objetos. En ese período competían tres métodos: el de Grady Booch, la Object Modeling Technique (OMT) de James Rumbaugh y Objectory de Ivar Jacobson. La empresa desarrolló una herramienta que admitía las tres notaciones y, para enseñar a usarla, sintetizó un enfoque que combinaba lo más útil de cada una. Esa síntesis, presentada inicialmente

como una aproximación unificada al modelado de objetos, antecedió en algunos años a la unificación que dio origen a UML.

De Jacobson provienen los casos de uso como motor del desarrollo, el análisis de robustez y el uso de diagramas de interacción para repartir comportamiento; de Rumbaugh, el modelo estático del dominio construido con el vocabulario del problema; de Booch, el cuidado en el diseño detallado de clases y operaciones. Los autores reconocen que la parte de Jacobson resultó ser el núcleo: un proceso dirigido por casos de uso no alcanza para llegar al código si no existe un paso que vincule el texto del requisito con los objetos del diseño.

La metodología quedó documentada en una serie de libros de Rosenberg con distintos coautores.

Año	Título	Autores	Editorial
1999	Use Case Driven Object Modeling with UML: A Practical Approach	Doug Rosenberg, Kendall Scott	Addison-Wesley
2001	Applying Use Case Driven Object Modeling with UML: An Annotated e-Commerce Example	Doug Rosenberg, Kendall Scott	Addison-Wesley
2005	Agile Development with ICONIX Process: People, Process, and Pragmatism	Doug Rosenberg, Matt Stephens, Mark Collins-Cope	Apress
2007	Use Case Driven Object Modeling with UML: Theory and Practice	Doug Rosenberg, Matt Stephens	Apress
2010	Design Driven Testing: Test Smarter, Not Harder	Matt Stephens, Doug Rosenberg	Apress

Kendall Scott acompañó la etapa inicial, centrada en el modelado; el libro de 2001 es un cuaderno de ejercicios sobre una librería en línea, con un capítulo para cada una de las tres revisiones del proceso. Matt Stephens aportó la mirada del desarrollador: el libro de 2007, segunda edición del de 1999 alineada con UML 2.0, incluye ese mismo ejemplo implementado en Java hasta las pruebas unitarias. Esa trayectoria explica la importancia creciente que las obras posteriores dan a la relación entre modelo, código y pruebas.

9.0.2 Principios del proceso

Dirigido por casos de uso. El caso de uso es la unidad de trabajo: todo modelo posterior se construye para realizar un caso concreto, y todo elemento del diseño debe justificarse por algún paso de algún caso. ICONIX prefiere casos de uso concretos, escritos en términos de lo que el usuario hace sobre las pantallas, antes que casos esenciales e independientes de la tecnología.

Mínimo: lo justo de UML. El núcleo se limita a cinco diagramas: modelo de dominio, casos de uso, robustez, secuencia y clases de diseño. Los autores recuerdan que la propia literatura fundacional de UML admite que la mayor parte del modelado se resuelve con una fracción pequeña del lenguaje. Otros diagramas, como los de estados, pueden sumarse cuando el problema lo pida.

Iterativo e incremental. Se toma un lote pequeño de casos de uso, se lo lleva por todas las etapas hasta obtener código probado y se repite con el lote siguiente. La secuencia es estricta entre etapas y flexible dentro de ellas.

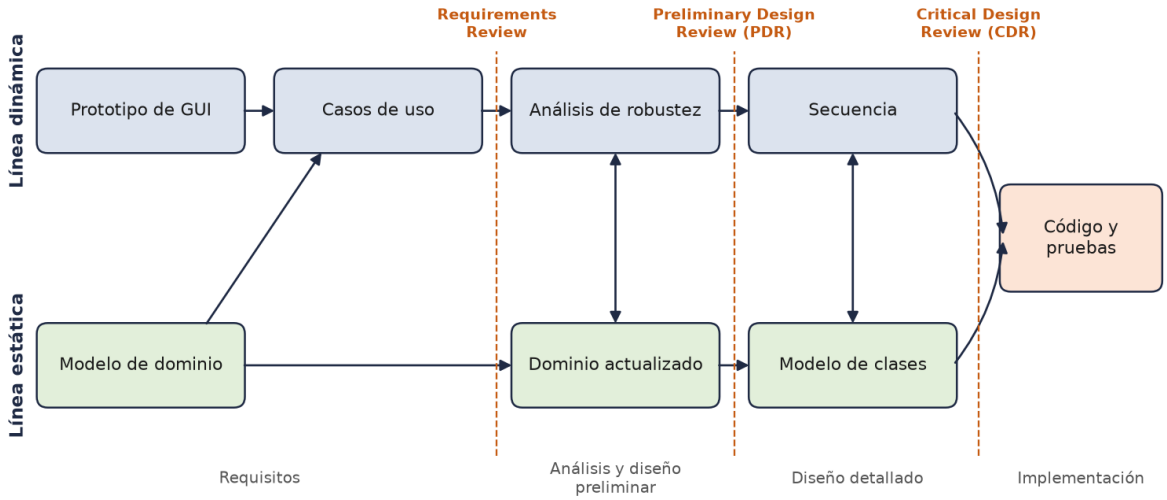
Trazable de punta a punta. Cada frase del caso de uso se reconoce en la robustez, cada controlador reaparece como mensaje en la secuencia, cada mensaje termina como operación de una clase y cada caso tiene pruebas para su curso básico y sus alternos.

Contra la parálisis por análisis. Los autores identifican dos fallas opuestas: modelar indefinidamente sin programar, y programar sin haber pensado el diseño. ICONIX ataca la primera con pocos diagramas y criterios claros de terminación, y la segunda exigiendo que el diseño exista antes que el código. Lo describen como algo cercano a un recetario: pasos repetibles y adaptables que indican siempre qué hacer a continuación.

El proceso avanza en dos líneas paralelas. La dinámica parte de los prototipos de interfaz y los casos de uso, pasa por la robustez y llega a la secuencia; la estática parte del modelo de dominio, lo actualiza con lo que descubre la robustez y lo convierte en el modelo de clases que usa el código.

Figura 9.0.1

Vista general de ICONIX con la línea dinámica (prototipo de GUI, casos de uso, robustez, secuencia) y la línea estática (dominio, dominio actualizado, modelo de clases), convergiendo en código y pruebas



9.0.3 Las cuatro etapas y sus hitos

Las tres primeras etapas terminan con una revisión formal que funciona como puerta: el lote no avanza sin superarla. La cuarta termina con la entrega del incremento.

Etapa	Artefactos	Hito de cierre
Requisitos	Requisitos funcionales, modelo de dominio, prototipos de GUI, casos de uso	Revisión de requisitos (Requirements Review)
Análisis y diseño preliminar	Robustez por caso de uso, dominio actualizado, casos de uso reescritos	Revisión de diseño preliminar (PDR)
Diseño detallado	Secuencia por caso de uso, modelo estático con operaciones	Revisión crítica de diseño (CDR)
Implementación	Código, pruebas unitarias, pruebas de integración, modelo sincronizado	Entrega

Requisitos. Los requisitos funcionales son un insumo que puede venir del cliente o del equipo, y se mantienen como lista separada del texto de los casos de uso. En paralelo se construye el modelo de dominio, un diagrama de clases sin operaciones que fija un vocabulario común. Para iniciarlo se sugiere la inspección gramatical: los sustantivos de los documentos disponibles son candidatos a clases o atributos. También se elaboran prototipos o guiones gráficos de la interfaz, porque los casos de uso deben describir el diálogo con pantallas concretas. Luego se identifican los casos de uso, se agrupan en paquetes, se les asignan los requisitos y se escribe un primer borrador de cada uno. La revisión de requisitos comprueba con el cliente que el texto refleje lo esperado, que el dominio cubra las abstracciones principales y que cada caso tenga curso básico y alternos. Los autores admiten revisar por lotes, justo antes de diseñar cada uno.

Análisis y diseño preliminar. Para cada caso del lote se dibuja un diagrama de robustez. Al hacerlo aparecen huecos (pasos sin respuesta del sistema, objetos que no están en el dominio, pantallas sin nombre, alternos olvidados) que se corrigen en su origen: se reescribe el texto y se actualiza el dominio. También se nombran todos los controladores. La revisión de diseño preliminar aplica la llamada prueba del resaltador: se recorre cada frase del caso y se marca el elemento del diagrama que le corresponde, hasta que no quede texto sin objeto ni objeto sin texto. Se verifica además que las entidades figuren en el dominio, que se respeten las reglas de conexión y que no se adelante detalle de diseño.

Diseño detallado. Se dibuja un diagrama de secuencia por caso de uso, con básico y alternos juntos, cuya función principal es asignar comportamiento a las clases. Con cada operación agregada, el dominio se transforma en modelo estático de diseño, al que se suman las clases técnicas que exige la arquitectura. La revisión crítica de diseño verifica que la secuencia

coincida con el texto, que las operaciones estén en las clases adecuadas, que atributos y firmas estén completos, que los programadores consideren construible el diseño y que exista un plan de pruebas.

Implementación. El código se escribe a partir del diseño. Las pruebas unitarias se organizan en torno a los controladores de robustez, con al menos una prueba por función lógica. Las pruebas de integración y de escenario se derivan de los casos de uso y recorren el curso básico y cada alterno. La etapa cierra con una revisión de código que es también una actualización del modelo, para que código y diagramas no diverjan antes del siguiente lote.

9.0.4 Reglas para escribir casos de uso en ICONIX

Como todo el diseño se deriva del texto, un caso de uso ambiguo produce una robustez ambigua y esa ambigüedad llega al código. Por eso ICONIX fija reglas de redacción.

- **Voz activa, con estructura sustantivo, verbo, sustantivo.** Cada oración indica quién actúa, qué hace y sobre qué objeto. "Se registra la venta" no sirve porque oculta quién actúa.
- **Evento y respuesta.** Se describen los dos lados del diálogo; un paso del usuario sin respuesta del sistema indica comportamiento faltante.
- **Curso básico y cursos alternos.** El básico es el camino en que todo sale bien; los alternos cubren errores y variantes, y concentran buena parte del trabajo real del software.
- **Regla de los dos párrafos.** El curso básico ocupa uno o dos párrafos y cada alterno una o dos oraciones, de modo que el caso completo quepa en un solo diagrama de secuencia.
- **Referencias al dominio y a las pantallas.** Los sustantivos son nombres de clases del dominio, escritos igual que en el modelo, y las fronteras se nombran explícitamente.

- **En contexto del modelo de objetos.** El caso de uso no es una narración libre, sino una especificación del comportamiento en tiempo de ejecución expresada con el vocabulario del dominio y de la interfaz. Esa condición es la que permite diseñar a partir del texto.
- **Requisitos separados y plantillas livianas.** Las declaraciones del tipo "el sistema deberá" quedan en la lista de requisitos, y se evitan plantillas extensas que consumen esfuerzo sin aportar precisión.

En EnsayAR, la ficha del primer ciclo para el registro de venta decía que el sistema "comprueba el disponible". En estilo ICONIX el paso se reescribe así: el Cajero agrega una Variante en la pantalla Punto de Venta; el sistema consulta el StockAlmacén de esa Variante en la Sucursal del Turno abierto y muestra la cantidad disponible. El requisito no cambia, pero quedan visibles una frontera y cuatro entidades que la robustez deberá mostrar.

9.0.5 El análisis de robustez

El análisis de robustez es la técnica que distingue a ICONIX. Proviene del trabajo de Jacobson de comienzos de los noventa, pero ICONIX lo volvió obligatorio para cada caso de uso. El diagrama no es un tipo estándar de UML: se construye con clases estereotipadas y los autores lo describen como una imagen de objetos del caso de uso.

Los tres tipos de objeto.

- **Frontera (boundary):** punto de contacto entre actor y sistema, como una pantalla o la interfaz con un sistema externo.
- **Entidad (entity):** objeto del dominio, normalmente persistente, tomado del modelo de dominio.

- **Control (controller):** función lógica que conecta fronteras con entidades, como validar, calcular o registrar. Solo a veces termina como clase de control; en general representa un verbo que será una operación.

Fronteras y entidades son sustantivos; los controladores son verbos. De esa lectura surgen las reglas de conexión, en las que la dirección de las flechas no importa.

Origen / Destino	Actor	Frontera	Control	Entidad
Actor	No aplica	Permitida	Prohibida	Prohibida
Frontera	Permitida	Prohibida	Permitida	Prohibida
Control	Prohibida	Permitida	Permitida	Permitida
Entidad	Prohibida	Prohibida	Permitida	Prohibida

En síntesis, los sustantivos no hablan con sustantivos y los verbos pueden hablar con ambos. El actor solo usa fronteras, porque nunca invoca funciones ni toca datos directamente. Una frontera no accede a una entidad sin un controlador, porque eso pondría lógica de negocio en la pantalla. Dos entidades no se comunican, porque su relación estructural ya está en el dominio y cualquier comportamiento que las involucre es, por definición, un controlador.

Qué problemas detecta. Al dibujar la robustez aparecen pasos ambiguos o en voz pasiva, pasos del usuario sin respuesta, objetos que faltan en el dominio o tienen otro nombre, pantallas implícitas, cursos alternos ausentes, casos demasiado extensos (con exceso de controladores) y detalle técnico prematuro, como nombres de tablas.

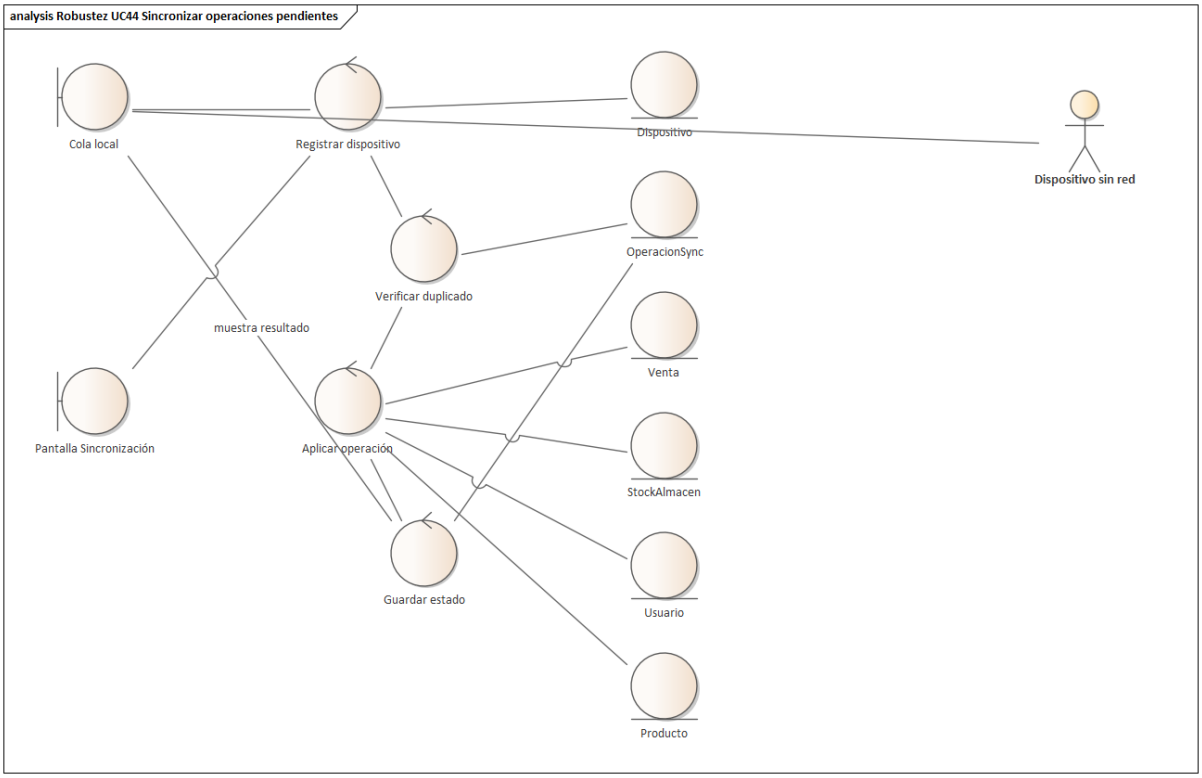
Puente entre el qué y el cómo. El caso de uso expresa qué debe hacer el sistema; la secuencia, cómo lo hacen las clases. Saltar directamente de uno a otro obliga a interpretar el requisito y diseñar en un solo paso, y ahí fallan muchos proyectos. La robustez divide el salto: primero identifica objetos y funciones lógicas sin decidir qué clase hace qué, y recién después se asignan responsabilidades. Cumple así dos funciones, eliminar ambigüedad y descubrir objetos faltantes. Además, sus tres estereotipos se corresponden con el patrón modelo, vista y controlador, lo que permite reconocer en ella la arquitectura técnica.

De controladores a operaciones o servicios. En la secuencia, fronteras y entidades pasan a ser líneas de vida y los controladores pasan a ser mensajes. Un controlador simple suele terminar como método de una entidad; uno que coordina varias entidades en una transacción, como operación de un servicio de aplicación; solo una lógica extensa y reutilizable justifica una clase propia.

En EnsayAR conviene una aclaración. NestJS llama controller a las clases que reciben peticiones HTTP, pero en términos de ICONIX esas clases son fronteras del lado del servidor: traducen la petición y delegan. Los controladores de ICONIX corresponden a métodos de los servicios. Así, "cobrar venta" se realiza en la operación cobrar de VentasService, "aplicar lote de operaciones pendientes" en la operación lote de SyncService, e "interpretar pedido en lenguaje natural" en la operación interpretar de ReportesInteligentesService.

Figura 9.0.2

Robustez de UC44 Sincronizar operaciones pendientes



Nota. El dispositivo sin red entrega la cola local; los controladores registran el dispositivo, descartan duplicados, aplican cada operación y guardan su estado en OperacionSync.

9.0.6 Diagramas de secuencia y asignación de responsabilidades

ICONIX exige exactamente un diagrama de secuencia por caso de uso, con básico y alternos en el mismo diagrama, porque fragmentar un caso dificulta comprobar que todo quedó cubierto. El diagrama parte de lo que dejó la robustez: actor, fronteras y entidades como líneas de vida, y el texto del caso al margen para alinear cada frase con sus mensajes. Luego se decide qué objeto ejecuta cada controlador; ese es el momento de la asignación de responsabilidades.

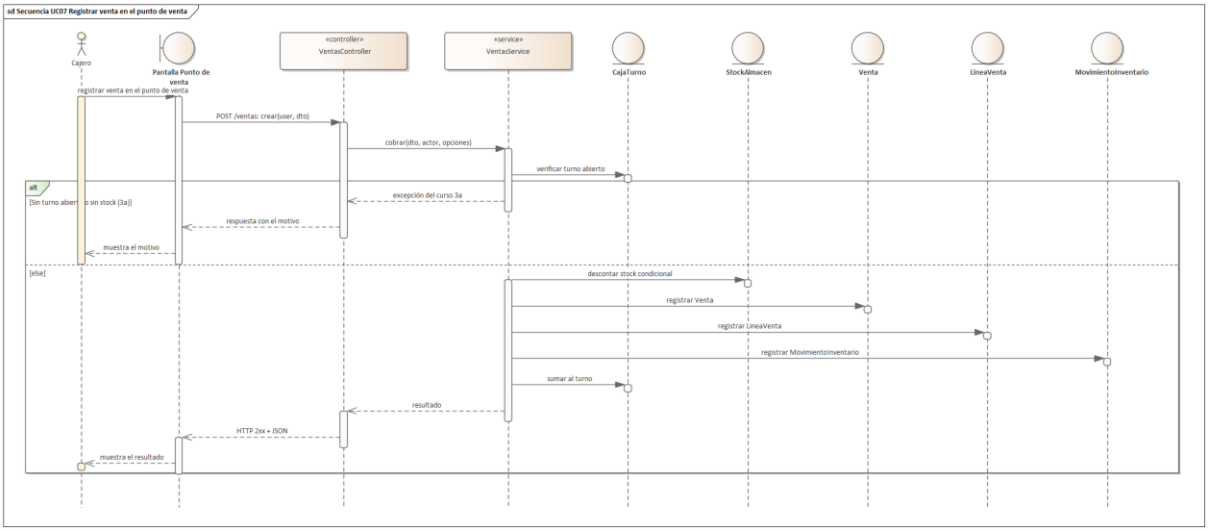
Los criterios son los clásicos de calidad de clases: bajo acoplamiento, alta cohesión, y clases suficientes, completas y primitivas. En la práctica, la operación se asigna a la clase que

posee la información necesaria, y solo se recurre a un servicio coordinador cuando intervienen varias entidades de igual peso. Los autores llaman prefactorización a este trabajo: mejorar la estructura antes de programar, porque mover una operación en un diagrama cuesta minutos y hacerlo en código probado obliga a reescribir pruebas. Las herramientas lo facilitan, ya que al dibujar un mensaje la operación se agrega a la clase destino.

En esta etapa entra la arquitectura técnica que la robustez omitía. En EnsayAR eso incluye el guard de autenticación, el contexto de tienda que aplica el aislamiento multi-tienda, el registro de claves de idempotencia y el acceso a datos con Prisma. Incluirlos permite verificar en la revisión crítica, por ejemplo, que ninguna operación consulta datos sin la tienda de la sesión fijada.

Figura 9.0.3

Secuencia de UC07 Registrar venta en el punto de venta



Nota. Cada controlador de la robustez reaparece como mensaje de VentasService hacia la entidad que modifica.

9.0.7 Trazabilidad

Si cada etapa deriva de la anterior, un requisito puede seguirse hasta la prueba que lo verifica y, en sentido inverso, cada clase y cada prueba se justifican por un requisito. ICONIX recomienda registrar esa relación en una matriz. Algunas filas de la matriz de EnsayAR ilustran su forma.

Caso de uso	Controladores de robustez	Clase y operación	Prueba
UC07 Registrar venta en el punto de venta	verificar turno abierto, descontar stock condicional, registrar venta, sumar al turno	VentasService.cobrar	Venta, uuid duplicado y conflicto en verificar:offline
UC38 Suscribirse y crear tienda	verificar slug, cobrar suscripción, aprovisionar tienda	PlataformaService.suscribir	Pago rechazado, aprobado e idempotente en verificar:saas
UC40 Cambiar de tienda	listar tiendas de la cuenta, emitir token de tienda	AuthService.cambiarTienda	Misma cuenta en dos tiendas sin nuevo inicio de sesión en verificar:saas
UC44 Sincronizar operaciones pendientes	registrar dispositivo, verificar duplicado, aplicar operación, guardar estado	SyncService.lote	Reenvío de lote y última unidad en verificar:offline
UC48 Generar reporte por texto o voz	interpretar pedido, validar especificación, compilar SQL, ejecutar aislado por tienda	ReportesInteligentesService.generar	Especificaciones, inyección y aislamiento en verificar:reportes
UC51 Registrar abono de crédito	descontar saldo condicional, registrar abono, sumar a caja	CreditosController.abonar	Abono repetido y abonos simultáneos en verificar:reportes

La matriz completa, de UC01 a UC54, se presenta en el apartado de implementación y pruebas. En la revisión crítica permite detectar casos sin secuencia; en la implementación, operaciones que nadie pidió; y en la defensa, mostrar de inmediato cómo se realiza y se verifica cualquier funcionalidad.

9.0.8 ICONIX frente a otros procesos

Frente al Proceso Unificado. Ambos son dirigidos por casos de uso, iterativos e incrementales, usan UML y heredan de Jacobson los estereotipos de frontera, control y entidad. Difieren en alcance, estructura y riesgo prioritario.

Aspecto	Proceso Unificado	ICONIX
Naturaleza	Marco amplio y adaptable	Proceso mínimo, del caso de uso al

Aspecto	Proceso Unificado	ICONIX
		código
Estructura	Fases (inicio, elaboración, construcción, transición) cruzadas con disciplinas	Cuatro etapas recorridas por cada lote de casos de uso
Hitos	Objetivos, arquitectura, capacidad operativa inicial, entrega del producto	Revisión de requisitos, PDR, CDR, entrega
Centro	Casos de uso y arquitectura	Casos de uso y trazabilidad
Riesgo que ataca primero	Técnico, con arquitectura ejecutable temprana	Ambigüedad de requisitos y parálisis por análisis
Paso de análisis a diseño	Modelo de análisis, a menudo omitido en la práctica	Robustez obligatoria por caso de uso
Casos de uso	Plantillas detalladas con flujos y subflujos	Dos párrafos, voz activa, dominio y pantallas
Diseño detallado	Varios modelos posibles	Una secuencia por caso de uso y el modelo estático
Pruebas	Disciplina con flujo propio	Derivadas de controladores y casos de uso
Gestión	Incluye planificación y configuración	Casi no la aborda

La diferencia práctica decisiva es la robustez: el Proceso Unificado prevé un modelo de análisis con los mismos estereotipos, pero como artefacto opcional; ICONIX lo vuelve obligatorio y lo somete a una revisión formal.

Frente a Scrum y XP. Scrum es un marco de gestión: organiza el trabajo en iteraciones con roles y eventos, pero no prescribe cómo analizar ni diseñar. ICONIX es un proceso de ingeniería que dice qué modelos construir y en qué orden, pero casi nada sobre planificación. Por eso pueden combinarse: un lote de ICONIX puede ocupar uno o varios sprints. Con XP la relación es más tensa. XP parte de historias de usuario, confía en un cliente presente, deja que el diseño emerja guiado por las pruebas y lo mejora con refactorización continua. Rosenberg y Stephens publicaron en 2003 una crítica de ese enfoque, y ICONIX propone lo contrario: diseñar antes de programar, prefactorizar en lugar de refactorizar y derivar las pruebas del diseño, idea que luego desarrollaron como pruebas dirigidas por el diseño.

Aspecto	Scrum	XP	ICONIX
Tipo	Marco de gestión	Prácticas de programación	Proceso de análisis y diseño
Requisito	Elemento del backlog	Historia de usuario	Caso de uso con básico y alternos
Diseño previo	No lo prescribe	Mínimo, emergente	Explícito y revisado

Aspecto	Scrum	XP	ICONIX
Origen de las pruebas	No lo prescribe	Escritas antes del código	Derivadas del modelo

Versión ágil de ICONIX. En el libro de 2005 los autores definen un subconjunto de doce prácticas. Cuatro son propias del proceso: análisis de requisitos con eliminación de ambigüedades, análisis del dominio como lenguaje común con el cliente, diseño de la interacción para reducir cambios de requisitos y prefactorización guiada por el modelo. Ocho son ágiles: pruebas intensivas, entregas pequeñas y frecuentes, integración frecuente, sincronización de modelo y código después de cada entrega, planificación ágil, responsabilidad colectiva, comunicación del equipo e iteraciones cortas. Los valores que las sostienen son comunicación, retroalimentación temprana, simplicidad, y juicio y equilibrio. Para EnsayAR resulta clave la sincronización de modelo y código: se acepta que diverjan durante una iteración, pero al final de cada entrega se realinean, actualizando solo lo necesario y corrigiendo el análisis si la implementación mostró que no se correspondía con la realidad.

9.0.9 Herramientas: Enterprise Architect

Enterprise Architect, de Sparx Systems, incluye en su instalación una tecnología de modelado para ICONIX que agrega páginas de herramientas específicas para los diagramas de análisis, casos de uso, clases, secuencia y actividades. La página de análisis ofrece los tres estereotipos de robustez como elementos propios, y el vinculador rápido sugiere solo conexiones válidas entre ellos. La herramienta permite además escribir escenarios estructurados en un caso de uso y generar desde ellos una robustez preliminar, y crear la operación en la clase destino al dibujar un mensaje de secuencia, lo que mantiene sincronizados los modelos dinámico y estático. Sparx e ICONIX publicaron también un complemento para pruebas dirigidas por el diseño que genera la estructura de las secuencias a partir de la robustez, transforma controladores en casos de prueba e incorpora reglas de validación de robustez.

En EnsayAR, el modelo del segundo ciclo es el archivo `docs/segundo-parcial/diagramas/EnsayAR-ICONIX.eapx`. Conserva intacto el modelo del primer ciclo y le suma el paquete "ICONIX segundo parcial", ordenado según las etapas del proceso: 1 Requisitos (31 requisitos y su asignación a casos de uso), 2 Modelo de dominio (41 clases en cinco vistas y una vista completa), 3 Casos de uso (un paquete por lote, con 20 diagramas), 4 Robustez (un paquete por caso con sus controladores; fronteras y entidades compartidas), 5 Secuencias (una por caso), 6 Diseño detallado (clases de diseño por módulo, con sus operaciones y dependencias) y 7 Arquitectura (componentes y despliegue). Las entidades de robustez están vinculadas por una dependencia «trace» a su clase del dominio.

El modelo no se dibuja a mano. El archivo `docs/segundo-parcial/iconix/casos.py` describe cada caso de uso con sus actores, fronteras, curso básico, cursos alternos, controladores con las entidades que tocan, la ruta HTTP y la clase y operación que lo realizan. El script `generar_ea.py` recorre esa descripción y construye el modelo con la interfaz de automatización de Enterprise Architect: crea los elementos, deriva los mensajes de cada secuencia a partir de los controladores de la robustez, agrega las operaciones a las clases de diseño, toma atributos y asociaciones del dominio directamente de `schema.prisma`, lee las firmas de los controladores y servicios del backend para escribir parámetros y tipos de retorno, y exporta los 160 diagramas como imágenes. Los diagramas de despliegue y componentes salen de `arquitectura.py` y las máquinas de estado de `estados.py`, ambos invocados por el mismo script. Así se cumple de forma mecánica la regla de ICONIX de que cada diagrama derive del anterior: si cambia el texto de un caso, se regeneran su robustez, su secuencia y las clases afectadas.

9.0.10 Ventajas y críticas

Ventajas. ICONIX ofrece un camino corto y verificable del requisito al código. Al usar pocos diagramas se aprende rápido y el modelado no se vuelve un fin en sí mismo. La robustez detecta defectos de requisitos cuando corregirlos es barato, y las tres revisiones distribuyen el control de calidad a lo largo del proceso. La trazabilidad facilita estimar, porque cada caso tiene un tamaño acotado, y probar, porque los casos de prueba salen de los cursos del caso de uso. Además se adapta a equipos pequeños y se combina bien con prácticas ágiles.

Críticas.

- Dice poco sobre arquitectura: no ofrece un mecanismo para decisiones transversales como el aislamiento multi-tienda o el despliegue.
- Casi no aborda la gestión del proyecto, por lo que debe complementarse.
- Los requisitos que no se expresan como diálogo, como rendimiento, seguridad o procesos sin actor humano, encajan mal en los casos de uso.
- La robustez no es UML estándar y depende del soporte de la herramienta.
- Exigir robustez y secuencia para cada caso puede ser costoso en casos sencillos de alta, baja y modificación.
- Sin sincronización disciplinada entre modelo y código, los diagramas envejecen y la trazabilidad se vuelve formal.

En EnsayAR estas críticas se atienden así: las decisiones de arquitectura provienen del primer ciclo y de los capítulos de fundamentación, y entran al diseño detallado como restricciones; los requisitos no funcionales, como la ventana máxima sin conexión o el aislamiento entre tiendas, se registran en la lista de requisitos y se trazan a los casos afectados; y

en los casos heredados sencillos la robustez y la secuencia se dibujan con el detalle mínimo que permite verificar las reglas.

9.0.11 Por qué ICONIX en el segundo ciclo de EnsayAR

La elección responde a la exigencia de la cátedra de aplicar un proceso distinto del Proceso Unificado y de Scrum, pero también es coherente con el momento del proyecto. El primer ciclo enfrentaba incertidumbre técnica: había que demostrar que la venta concurrente no dejaba stock negativo, que el probador funcionaba en el dispositivo y que los reportes en lenguaje natural se limitaban a consultas seguras. Para eso convenía la orientación del Proceso Unificado hacia una arquitectura ejecutable temprana.

En el segundo ciclo la arquitectura ya existe: el monolito modular en NestJS, los clientes React y React Native, PostgreSQL y el despliegue en la nube no se reemplazan. El riesgo ahora es integrar diecinueve casos de uso nuevos (UC36 a UC54), varios con reglas delicadas como la idempotencia, los conflictos de stock entre dispositivos sin conexión o el aislamiento entre tiendas, sin que ninguno quede a medio implementar o sin verificar. Ese es el problema para el que ICONIX está pensado: llevar cada caso de uso al código con trazabilidad completa y revisada. Además, como la cátedra anunció preguntas sobre cualquier parte del sistema sin mirar la aplicación, contar con robustez, secuencia, operaciones y pruebas por caso de uso permite responder con precisión dónde y cómo se resuelve cada requisito.

9.0.12 Organización iterativa por lotes

El ciclo se divide en seis lotes. Cada uno recorre las cuatro etapas por separado, sobre un dominio y un modelo de clases compartidos.

Lote	Contenido	Casos de uso	Enfoque
Lote 0	Casos heredados	UC01 a UC35	Revisión con robustez y secuencia por el contexto multi-tienda y la operación

Lote	Contenido	Casos de uso	Enfoque
			sin conexión
Lote 1	SaaS multi-tienda	UC36 a UC41	Registro, verificación de correo, suscripción con creación de tienda, renovación, cambio de tienda y métricas
Lote 2	Operación sin conexión	UC42 a UC47	Venta y altas sin red en web y móvil, sincronización, conflicto de stock, ventana configurable y notificaciones push
Lote 3	Reportes y crédito	UC48 a UC51	Reportes por voz o texto en pantalla, PDF o Excel, ventas al crédito y abonos
Lote 4	Analítica	UC52 y UC53	Modelos supervisados y no supervisados con datos sintéticos y tableros
Lote 5	Pagos	UC54	Medios de pago para países de habla hispana con pasarelas en modo de pruebas

Los lotes 4 y 5 están a cargo de otro integrante, con las mismas reglas: casos escritos según ICONIX, diagramas en el mismo modelo y pruebas derivadas de los casos. Sus casos de uso figuran ya en el modelo con el alcance acordado y se ajustarán a los nombres definitivos de su implementación. El lote 0 se justifica porque el segundo ciclo modifica los casos heredados de forma transversal: todos se ejecutan dentro de una tienda, y varios, como la venta o el alta de clientas, deben funcionar sin conexión. La robustez revela en qué pasos interviene el contexto de tienda y dónde aparece un alterno nuevo por falta de red.

9.0.13 Reutilización de los artefactos del primer ciclo

ICONIX no obliga a descartar lo producido con el Proceso Unificado; varios artefactos entran directamente en sus etapas.

Artefacto del primer ciclo	Etapa de ICONIX	Adaptación
Fichas de UC01 a UC35	Requisitos	Reescritura en voz activa, con alternos, dominio y pantallas
Modelo de dominio de la elaboración	Requisitos y análisis	Se agregan Cuenta, Tienda, Plan, Suscripción, OperacionSync y CuentaPorCobrar, entre otras
Pantallas implementadas en web y móvil	Requisitos	Funcionan como prototipos de GUI ya validados
Diagramas de comunicación y	Análisis y diseño preliminar	Base de la robustez, con los mismos

Artefacto del primer ciclo	Etapa de ICONIX	Adaptación
clases de análisis		estereotipos
Secuencias de inicio de sesión, venta y cola sin conexión	Diseño detallado	Se actualizan con contexto de tienda e idempotencia
Diseño físico de datos	Diseño detallado	Modelo estático de persistencia, con tienda en los modelos de negocio
Plan y casos de prueba	Implementación	Se complementan con los scripts de verificación

Esta reutilización refleja la idea de incremento planteada por la cátedra: el segundo ciclo extiende la versión anterior en lugar de reconstruirla.

9.0.14 Hitos y calendario del ciclo

Cada lote atraviesa las tres revisiones. Dado el ciclo corto y el equipo pequeño, se realizan como sesiones breves con listas de verificación basadas en las recomendaciones de los autores, y su resultado se registra en un acta corta. Las fechas son aproximadas.

Hito	Lotes	Fecha aproximada (noviembre de 2026)
Inicio del ciclo y análisis de la consigna	Todos	Primeros días del mes
Revisión de requisitos	1 a 5, y relectura del 0	3 y 4
Revisión de diseño preliminar	1 a 3	5 y 6
Revisión crítica de diseño	1 a 3	6 y 7
Implementación y verificación	1 a 3	Hasta el 8
PDR y CDR	0, 4 y 5	9 y 10
Revisión de código y actualización del modelo	Todos	10
Entrega del documento en PDF	Todos	11, hasta las 8:00
Presentación y defensa	Todos	11

Los lotes 1 a 3 se implementaron primero porque concentran lo que la cátedra anunció que se pondría a prueba en la defensa, como crear varias tiendas o vender la última unidad en dos dispositivos sin conexión. Para ellos, parte de los diagramas se trabajó como bocetos antes de programar y se consolida en Enterprise Architect durante la revisión de código y actualización del modelo, que es la práctica de sincronización propuesta por la versión ágil del proceso.

9.0.15 Los scripts de verificación como pruebas de integración

ICONIX distingue pruebas unitarias, que verifican controladores, y pruebas de integración y de escenario, que verifican casos de uso completos contra el sistema en ejecución. En EnsayAR, este segundo nivel lo cubren cuatro scripts de la carpeta de scripts del backend, que se ejecutan contra la API levantada y una base de datos real.

Script	Lote	Comprobaciones	Escenarios principales
verificar:tenancy	0 y 1	Aislamiento de datos	Lecturas filtradas por tienda, fallo sin tienda en contexto, registros ajenos invisibles e inmodificables
verificar:saas	1	15	Verificación de correo, pago rechazado y aprobado, reintento sin doble cobro, catálogos aislados, límites del plan, cambio de tienda, suscripción vencida y métricas
verificar:offline	2	14	Identificadores del dispositivo, última unidad en dos cajas, reversión de venta, reenvío sin duplicados, envíos simultáneos, pago QR sin red rechazado y ventana de siete días
verificar:reportes	3	24	Pedidos dictados complejos, agrupación y orden, parámetros separados del SQL, inyección, aislamiento, Excel y PDF, crédito y abonos repetidos o simultáneos

La correspondencia con ICONIX es directa: cada chequeo nombra un escenario de un caso de uso y verifica su poscondición. La última unidad vendida en dos cajas sin conexión es el curso alterno de conflicto de stock del caso de sincronización; el abono repetido con la misma clave es el alterno de reintento del caso de abono al crédito. Por eso cada script ocupa la columna de pruebas de la matriz de trazabilidad, y las pruebas unitarias del backend, como la de concurrencia de la venta heredada, completan el nivel de los controladores. El enfoque coincide con las pruebas dirigidas por el diseño de Stephens y Rosenberg: las pruebas no definen el

diseño, sino que comprueban que el diseño aprobado en la revisión crítica quedó bien implementado.

9.0.16 Organización del resto del capítulo

Los apartados siguientes siguen el orden de las etapas de ICONIX para todos los lotes.

- **9.1 Requisitos.** Requisitos del segundo ciclo, modelo de dominio actualizado, prototipos de GUI, diagramas de casos de uso por paquete, texto de UC01 a UC54 con las convenciones de ICONIX y acta de la revisión de requisitos.
- **9.2 Análisis y diseño preliminar.** Diagramas de robustez por caso de uso y lote, controladores identificados, cambios al dominio y al texto, y acta de la revisión de diseño preliminar.
- **9.3 Diseño detallado.** Diagramas de secuencia, modelo estático con operaciones, decisiones de arquitectura multi-tienda y de sincronización incorporadas, y acta de la revisión crítica.
- **9.4 Implementación y pruebas.** Correspondencia entre diseño y módulos, pruebas unitarias, scripts de verificación, matriz de trazabilidad completa y resultado de la revisión de código y actualización del modelo.

9.1 Requisitos

La etapa de requisitos de ICONIX produce cuatro artefactos que se revisan juntos: la lista de requisitos, el modelo de dominio, los prototipos de interfaz y los casos de uso. Para el segundo ciclo se partió de la consigna de la cátedra, de las fichas UC01 a UC35 del primer parcial y de las pantallas ya construidas en la web y en la app móvil.

9.1.1 Requisitos del segundo ciclo

Los requisitos se mantienen como una lista separada del texto de los casos de uso, como pide ICONIX. Cada código se asigna a uno o más casos de uso en el modelo de Enterprise Architect mediante una relación de realización, y esa asignación es la que permite trazar el requisito hasta la prueba.

Código	Área	Requisito	Casos de uso que lo realizan
B1	Base	Todo lo del primer parcial sigue funcionando: catálogo, punto de venta, probador, reportes, bot y pagos.	UC01 a UC35
S1	SaaS	Muchas tiendas operan en una sola aplicación (multi-tenant).	UC10, UC15, UC19, UC48
S2	SaaS	Página comercial pública con módulos, planes y precios.	UC36
S3	SaaS	Planes con duraciones de 1, 3 y 12 meses y niveles por cantidad de usuarios y sucursales.	UC23, UC24, UC36, UC38, UC39
S4	SaaS	Autoservicio: registro, suscripción y pago sin intervención humana.	UC36, UC38, UC39
S5	SaaS	Al suscribirse se crea automáticamente el espacio de la tienda.	UC38
S6	SaaS	Se verifica que el correo del suscriptor exista.	UC37
S7	SaaS	Una persona con varias tiendas usa un solo login y cambia de tienda.	UC14, UC40
S8	SaaS	Estrategia de aislamiento de datos justificada: base compartida con identificador de tienda.	Decisión de arquitectura (capítulo 4); se verifica en todos los casos
S9	SaaS	Reportes de suscripciones y proyecciones para el dueño de la plataforma.	UC41
O1	Offline	La web funciona como PWA y sigue vendiendo y registrando clientas y productos sin internet.	UC08, UC09, UC35, UC42, UC43
O2	Offline	La app móvil guarda en una base local (SQLite) y tiene la misma capacidad.	UC42, UC43
O3	Offline	Sincronización completa y predecible al volver la red.	UC44
O4	Offline	Identificadores generados en el cliente (UUID) sin choque entre dispositivos.	UC42, UC43, UC44
O5	Offline	El conflicto de la última unidad se resuelve con una actualización condicional en	UC05, UC07, UC45

Código	Área	Requisito	Casos de uso que lo realizan
		la base.	
O6	Offline	Idempotencia: guardar dos o tres veces con red inestable no duplica registros.	UC05, UC06, UC07, UC25, UC38, UC44, UC51
O7	Offline	Transacciones con rollback; con microservicios se explica el patrón saga.	UC07, UC31, UC44
O8	Offline	Se decide y justifica qué funciona sin red y qué no (un pago en línea no).	UC27, UC42, UC46, UC50
O9	Offline	Ventana offline configurable por tienda (1 a 30 días, 7 por defecto).	UC42, UC46
O10	Offline	La PWA es instalable y envía notificaciones push.	UC35, UC47
R1	Reportes	Reporte pedido en lenguaje natural por texto o por voz.	UC48
R2	Reportes	Pedidos complejos: varios filtros, cálculos, agrupación, orden y varias tablas.	UC33, UC48, UC49, UC50, UC51
R3	Reportes	Salida en pantalla, PDF o Excel según lo pedido.	UC49
A1	Analítica	Analítica con modelos supervisados y no supervisados.	UC52, UC53
A2	Analítica	Entrenamiento con datos sintéticos generados a medida.	UC52, UC53
A3	Analítica	Resultados presentados en dashboards.	UC52, UC53
P1	Pagos	Pagos que funcionen en todos los países de habla hispana.	UC54
P2	Pagos	Estudio de costos para elegir la opción que menos cobra al cliente.	Estudio del capítulo 5
P3	Pagos	Simulación con una pasarela real en modo sandbox.	UC54
P4	Pagos	Límites locales explicados: QR en Bolivia, PayPal fácil para pagar y difícil para cobrar.	Estudio del capítulo 5

Además de los requisitos funcionales rigen restricciones que no son diálogos con un actor y que ICONIX trata como requisitos transversales: toda consulta se ejecuta con la tienda de la sesión fijada (S8), toda operación que cambia dinero o stock acepta una clave de idempotencia (O6), ninguna operación deja stock negativo (O5) y el reporte generativo solo ejecuta SQL compilado desde una especificación validada (R2).

Figura 9.1.1

Asignación de los requisitos de SaaS a casos de uso

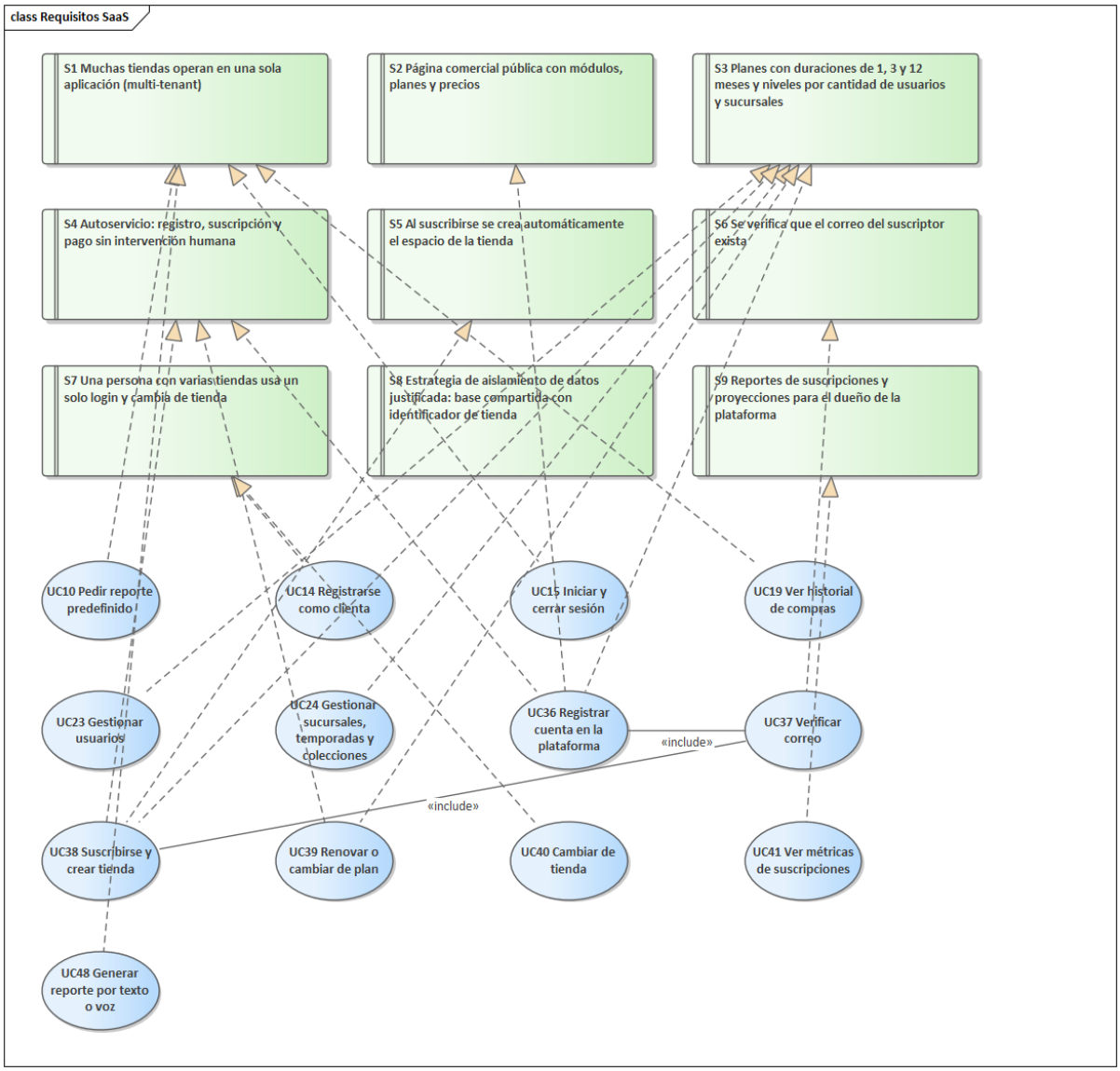


Figura 9.1.2

Asignación de los requisitos de Offline a casos de uso

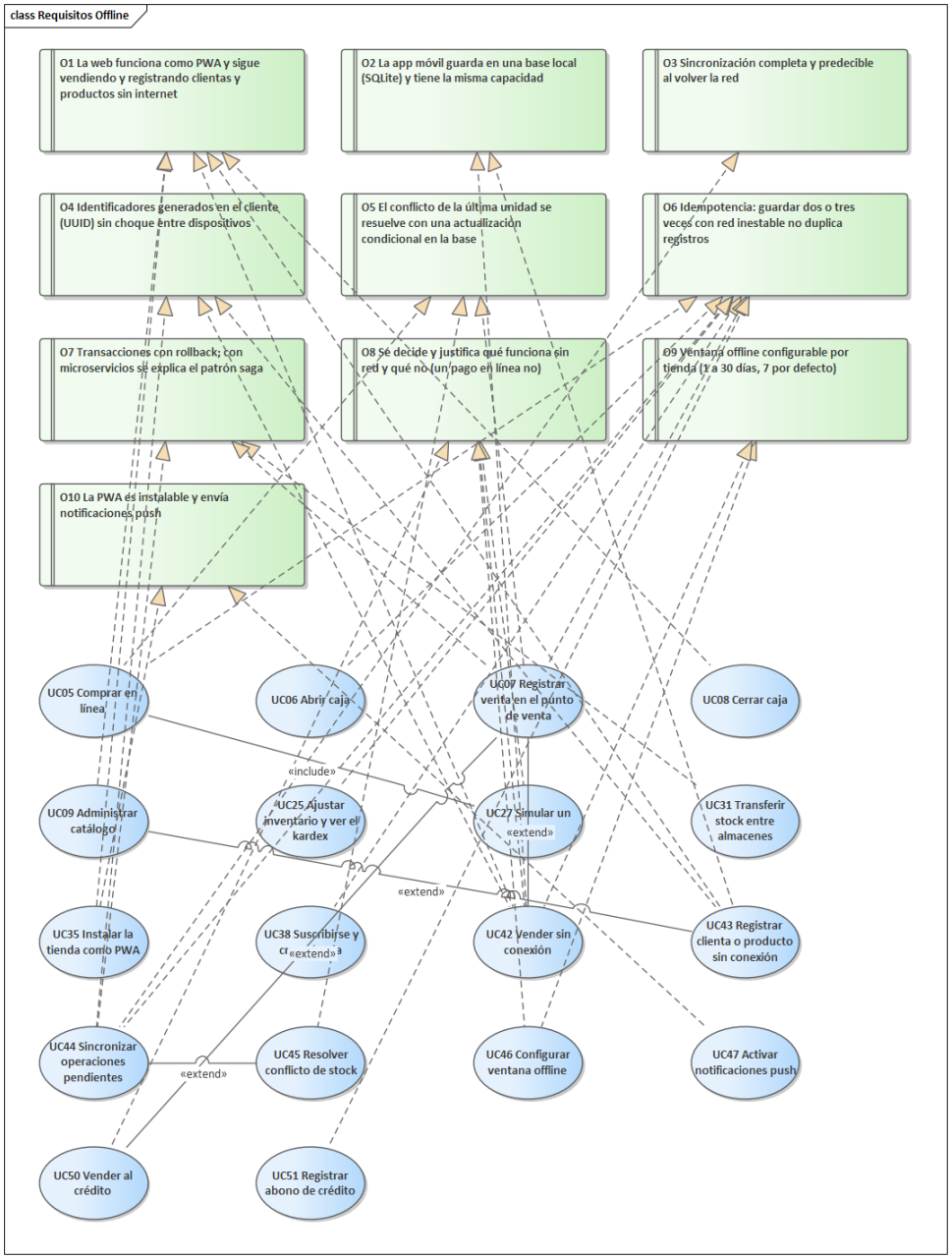


Figura 9.1.3

Asignación de los requisitos de Reportes a casos de uso

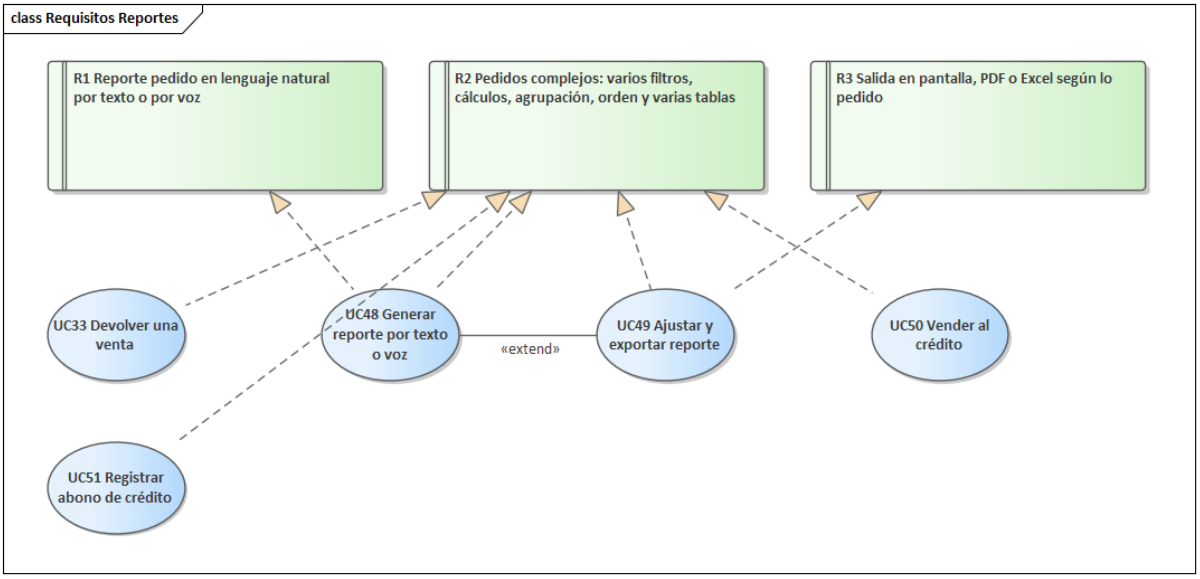


Figura 9.1.4

Asignación de los requisitos de Analítica a casos de uso

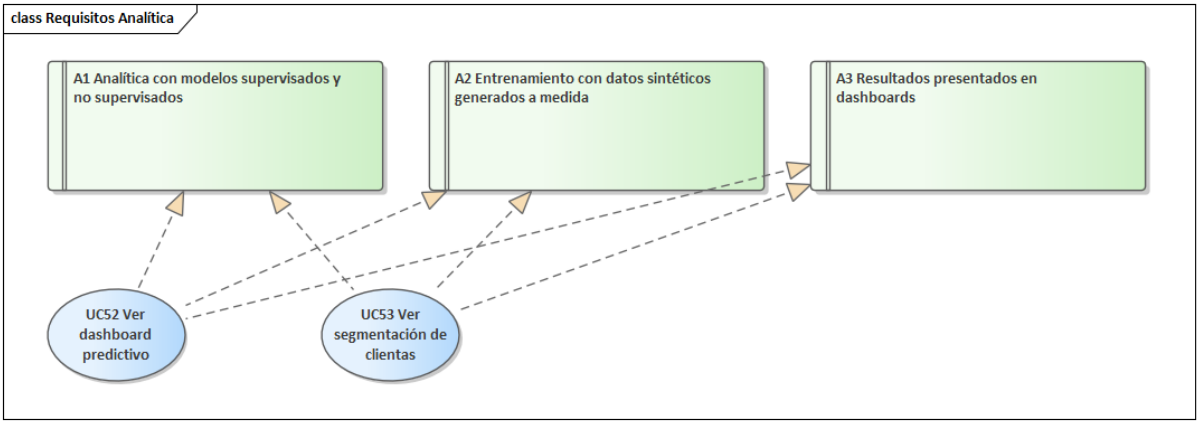
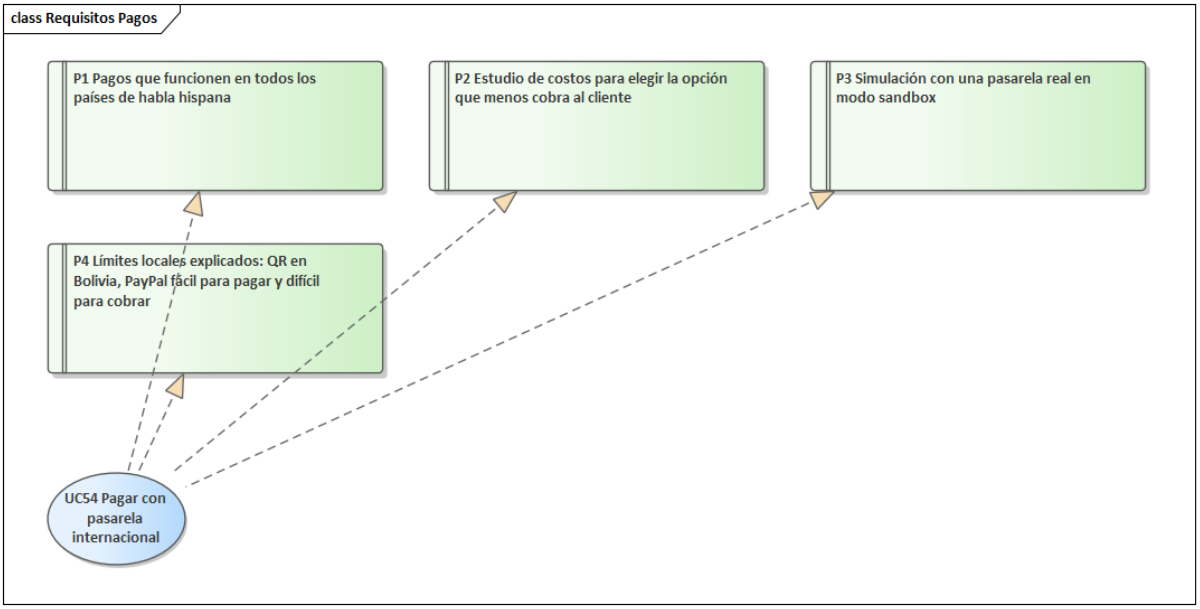


Figura 9.1.5

Asignación de los requisitos de Pagos a casos de uso



Requisitos no funcionales

Los requisitos no funcionales no son diálogos con un actor, pero condicionan a todos los casos. Cada uno se expresa como una propiedad verificable, junto con el mecanismo que la cumple en EnsayAR y la evidencia que la comprueba.

Código	Requisito	Cómo se cumple en EnsayAR	Evidencia
RNF01	Seguridad: solo usuarios autenticados y con el rol adecuado operan, y las credenciales nunca se guardan en claro	Contraseñas con bcrypt; token JWT con expiración y revocación al cerrar sesión (TokenRevocado); guards de rol en cada ruta; validación estricta de la entrada (los campos no declarados se rechazan); los secretos solo en variables de entorno	Pruebas de roles en `verificar:saas`; auditoría de login, logout y cambio de clave
RNF02	Aislamiento: ninguna tienda lee ni modifica datos de otra	Base compartida con `tiendaId`; la extensión de Prisma filtra y completa la tienda desde el contexto de la petición y falla cerrada si no la hay; el SQL de reportes fija `app.tienda_id` con `set_config` dentro de la transacción	`verificar:tenancy`; capítulo 4
RNF03	Persistencia offline: lo registrado sin red sobrevive	Cola en IndexedDB (web) y SQLite (móvil); una	Recorrido manual con la red cortada y recarga del

Código	Requisito	Cómo se cumple en EnsayAR	Evidencia
	al cierre de la aplicación y al reinicio del equipo	operación solo se borra cuando el servidor responde un estado definitivo	navegador; máquina de estados de 9.3
RNF04	Integridad transaccional: una venta se guarda completa o no se guarda	Venta, líneas, movimiento de inventario y suma al turno en una sola transacción; descuento condicional de stock (nunca negativo)	`ventas.spike-b.spec.ts`; reversión en `verificar:offline`
RNF05	Idempotencia: repetir un envío no duplica ventas, cobros ni abonos	Clave `Idempotency-Key` en 11 rutas que cambian dinero o stock; en la sincronización el id de la operación es la clave (OperacionSync)	Reenvío sin duplicados en `verificar:offline`; doble pago en `verificar:saas`; abonos repetidos en `verificar:reportes`
RNF06	Recuperación ante fallos: un corte a mitad del envío no pierde ni duplica operaciones	El cliente reintenta con la misma clave; las respuestas `en_curso` y `error` dejan la operación pendiente; una operación en curso abandonada por más de un minuto se retoma	Envíos simultáneos y reintentos en `verificar:offline`
RNF07	Mantenibilidad: cada área se puede cambiar sin tocar las demás	Monolito modular NestJS con un módulo por área y dependencias explícitas por interfaz; TypeScript en los tres proyectos	Diagrama de componentes (9.3.2)
RNF08	Compatibilidad: funciona en los navegadores y teléfonos que usan las tiendas	PWA instalable en Chrome, Edge y Safari; app Android con Expo; interfaz adaptable a celular, tablet y PC	Diagrama de despliegue (9.3.2); guías del capítulo 10
RNF09	Rendimiento: el punto de venta responde aunque la red sea lenta	Catálogo, stock y política offline en caché local; la sincronización envía lotes de hasta 50 operaciones en segundo plano	Recorridos del capítulo 10
RNF10	Observabilidad: se puede saber qué pasó con cada operación	Registros de la API en la instancia EC2; tabla Auditoria para acciones sensibles; estado de cada operación offline visible en la Pantalla Sincronización	Pantalla Sincronización (9.1.3); diagrama de despliegue
RNF11	Calidad: el comportamiento crítico se verifica de forma automática	Pruebas unitarias y cuatro guiones de verificación de integración contra la API y una base real	9.4
RNF12	Coherencia documental: los diagramas representan la solución construida	El modelo de Enterprise Architect se regenera desde `casos.py`, `schema.prisma` y las firmas del código TypeScript	9.4.6; anexo 14.6

Reglas de negocio

Las reglas de negocio son políticas de la tienda que el sistema hace cumplir siempre, con o sin red. Se aplican en el backend, de modo que un cliente modificado no puede saltarlas. Las

que dependen del dispositivo se validan dos veces: en el cliente para avisar a tiempo y en el servidor al sincronizar.

Código	Regla	Dónde se aplica	Casos de uso
RN01	La tienda y la sucursal de una operación salen de la sesión, nunca de los datos que envía el cliente	Contexto de tienda y extensión de Prisma	Todos
RN02	No se cobra sin un turno de caja abierto en la sucursal	VentasService, también al sincronizar	UC06, UC07, UC42
RN03	El stock nunca queda negativo: si dos cajas venden la última unidad, solo una venta se aplica	Actualización condicional de StockAlmacen	UC07, UC44, UC45
RN04	Sin red solo se cobra en efectivo y sin cupón; QR, tarjeta y crédito requieren conexión	Pantalla Punto de venta y validación del lote	UC42
RN05	Una operación offline solo se acepta si se creó dentro de la ventana de la tienda (1 a 30 días, 7 por defecto)	SyncService; política en la Pantalla Sincronización	UC44, UC46
RN06	Cada operación tiene un id único generado en el dispositivo y se aplica una sola vez	uuid() en el cliente; OperacionSync en el servidor	UC42, UC43, UC44
RN07	Una operación no desaparece del dispositivo por haber intentado enviarla: solo sale de la cola con un estado definitivo	Motor de sincronización del cliente	UC44
RN08	Un conflicto lo resuelve una persona, vendiendo lo disponible o descartando; descartar en el servidor exige rol Encargado o Admin	Pantalla Sincronización y SyncService	UC45
RN09	No se cierra la caja con operaciones offline pendientes o en conflicto	Pantalla Punto de venta	UC08
RN10	Cada plan limita usuarios, sucursales y productos (Inicial: 3, 1 y 200)	PlanLimitesService	UC09, UC23, UC24
RN11	La tienda se crea solo con el pago de la suscripción aprobado, y en una sola transacción	PlataformaService	UC38
RN12	El correo del suscriptor se verifica antes de activar la cuenta	AuthService y CorreoService	UC36, UC37
RN13	Un abono nunca deja el saldo de la cuenta por cobrar en negativo	Descuento condicional de CuentaCobrar	UC51
RN14	No se devuelven más unidades que las vendidas; devolver una venta al crédito reduce la deuda y no saca efectivo	Devoluciones	UC33
RN15	El modelo de lenguaje no accede a la base: solo propone una especificación	ReportesService	UC48, UC49

Código	Regla	Dónde se aplica	Casos de uso
	que se valida contra la capa semántica y se compila a SQL parametrizado con la tienda fijada		

9.1.2 Modelo de dominio

El modelo de dominio fija el vocabulario: cada sustantivo que aparece en un caso de uso con mayúscula inicial (Venta, StockAlmacen, CuentaCobrar) es una clase de este modelo, escrita igual. Se construyó en dos pasos. Primero, por inspección gramatical de la consigna y de las fichas del primer ciclo; después, durante la robustez, se agregaron las clases que los casos nuevos necesitaban (Cuenta, Plan, Suscripcion, PagoSuscripcion, Dispositivo, OperacionSync, PushSuscripcion, CuentaCobrar, AbonoCredito). Los atributos y las multiplicidades se toman del esquema de Prisma, de modo que el dominio y la base no pueden divergir.

Toda clase de negocio pertenece a una Tienda. En el esquema eso es un campo tiendaId que la base completa desde la sesión, sin relación declarada; en el dominio se muestra como la asociación «pertenece a» para Usuario, Sucursal, Producto y Venta, que son las raíces de cada agregado.

Figura 9.1.6

Dominio, plataforma SaaS

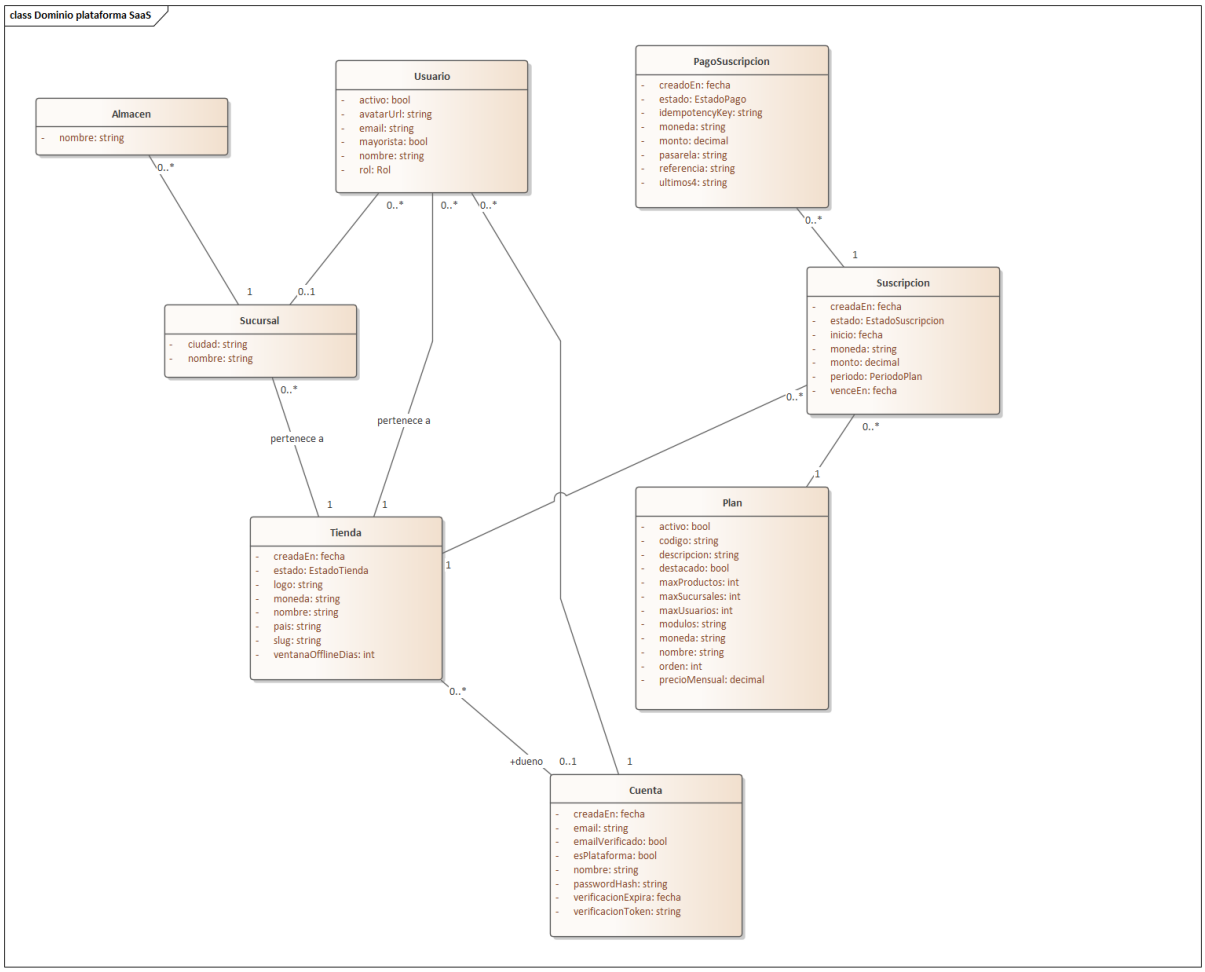


Figura 9.1.7

Dominio, catálogo e inventario

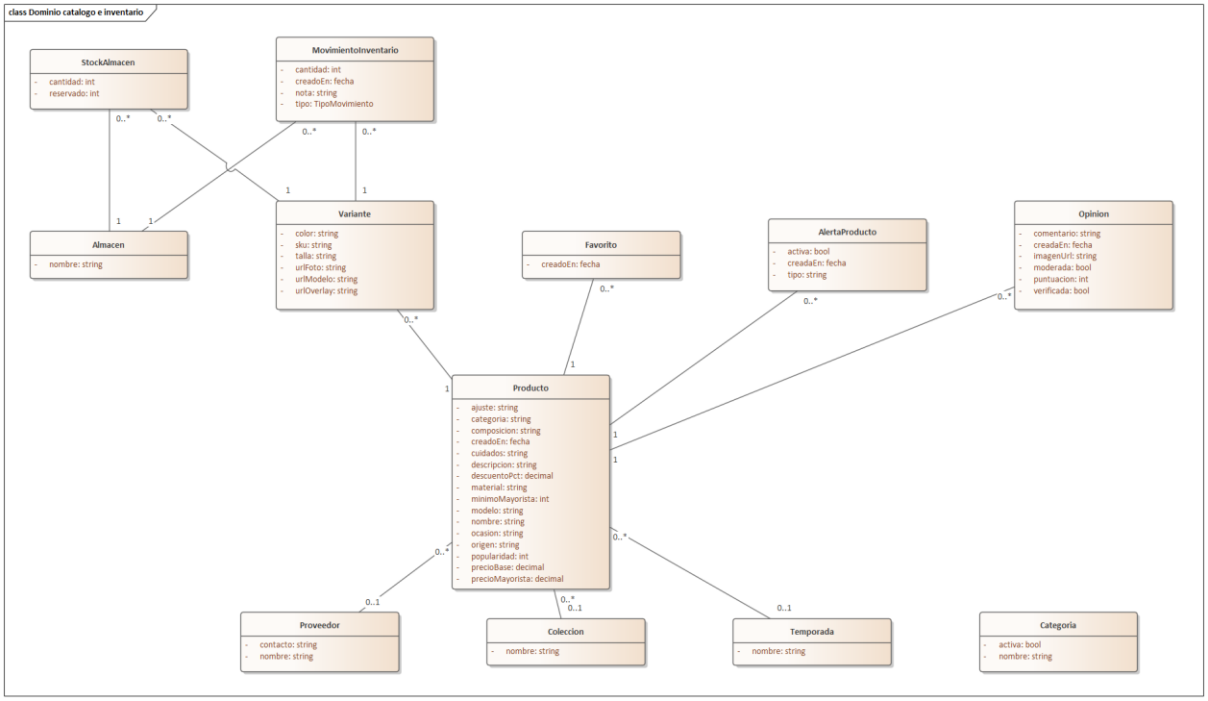


Figura 9.1.8

Dominio, ventas, caja y crédito

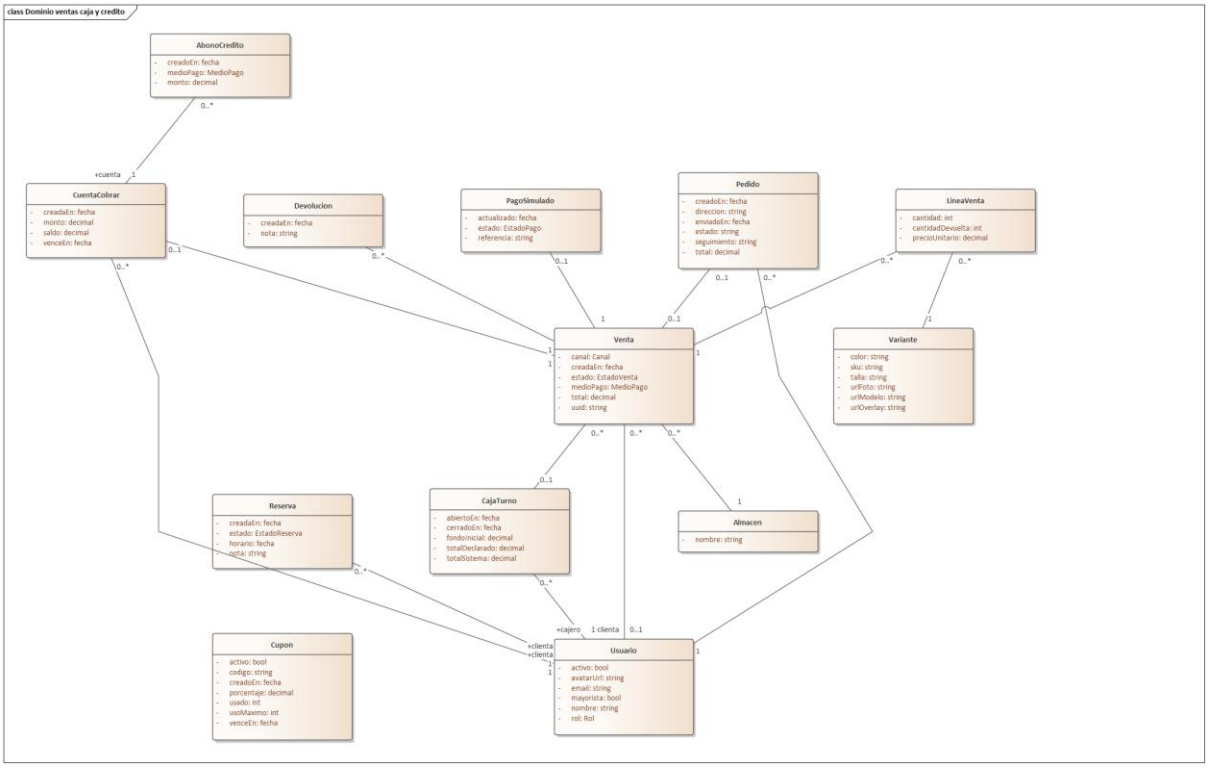


Figura 9.1.9

Dominio, sincronización, reportes y avisos

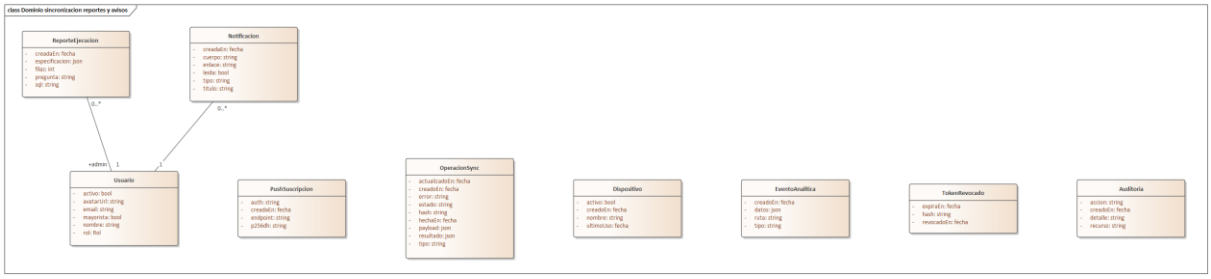


Figura 9.1.10

Dominio, analítica y pagos (por confirmar)

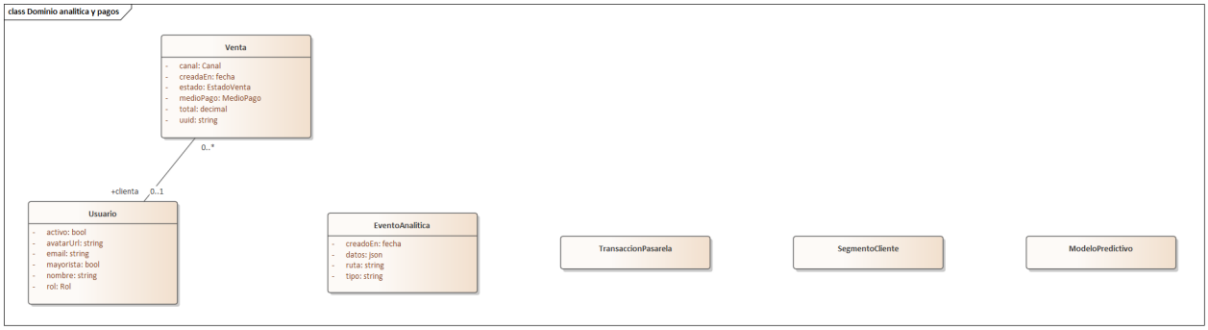
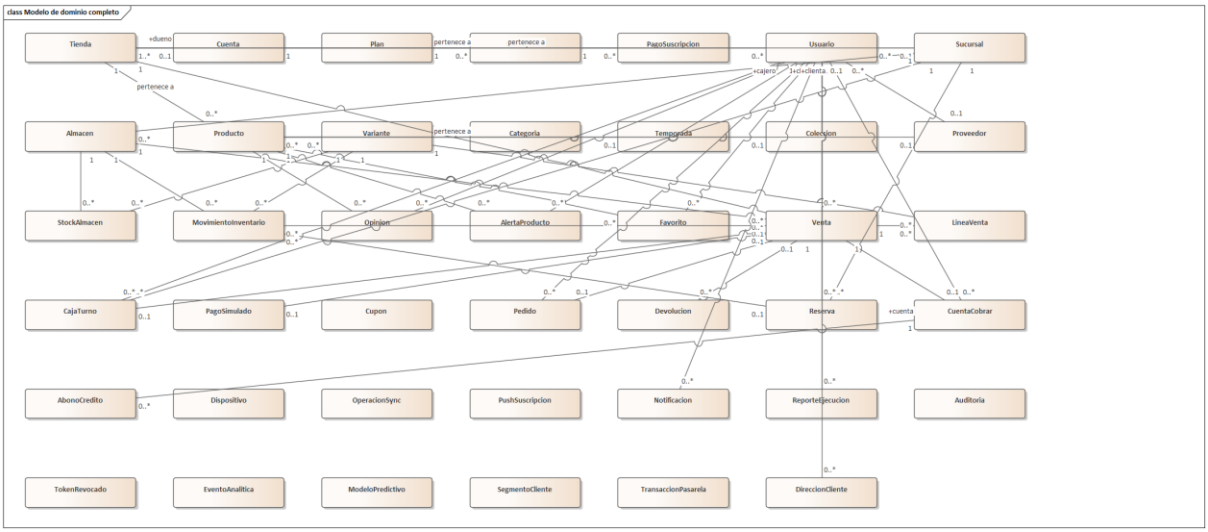


Figura 9.1.11

Dominio, modelo de dominio completo



Clase	Significado
Tienda	Inquilino de la plataforma: slug, nombre, estado y ventana offline.
Cuenta	Identidad global de una persona; una cuenta puede pertenecer a varias tiendas.
Plan	Oferta comercial con precio por duración y límites de usuarios y sucursales.
Suscripcion	Contrato vigente de una tienda con un plan y su fecha de vencimiento.
PagoSuscripcion	Cobro de un período de suscripción con su clave de idempotencia.
Usuario	Membresía de una cuenta en una tienda, con rol y sucursal.
Sucursal	Punto de venta físico de la tienda.
Almacen	Depósito de una sucursal donde vive el stock.
Proveedor	Empresa que abastece productos a la tienda.
Producto	Prenda del catálogo con precio, categorías, temporada

Clase	Significado
	y colección.
Variante	Combinación de talla y color de un producto con su SKU.
Categoria	Clasificación del catálogo (vestidos, blusas, accesorios...).
Temporada	Agrupación comercial por temporada.
Coleccion	Agrupación comercial por colección.
StockAlmacen	Cantidad física y reservada de una variante en un almacén.
MovimientoInventario	Kardex: ingreso, venta, ajuste, transferencia o devolución.
Reserva	Apartado de prendas para retirar en sucursal.
CajaTurno	Turno de caja con fondo inicial, total del sistema y arqueo.
Venta	Venta de mostrador o en línea con su uuid generado en el cliente.
LineaVenta	Prenda vendida con cantidad, precio y devoluciones.
PagoSimulado	Autorización de pago en modo prueba asociada a una venta.
Cupon	Código de descuento con vigencia y usos.
Pedido	Envío de una venta en línea con su estado de despacho.
Devolucion	Devolución total o parcial de una venta.
DireccionCliente	Dirección de entrega de una clienta.
Favorito	Producto marcado por una clienta.
Opinion	Valoración y comentario de un producto, sujeta a moderación.
AlertaProducto	Pedido de aviso por disponibilidad o precio.
Auditoria	Bitácora de accesos y cambios.
Notificacion	Aviso dentro de la aplicación.
TokenRevocado	Token JWT invalidado al cerrar sesión.
ReporteEjecucion	Registro de cada reporte ejecutado con su especificación.
CuentaCobrar	Deuda generada por una venta al crédito, con saldo y vencimiento.
AbonoCredito	Pago parcial o total de una cuenta por cobrar.
Dispositivo	Navegador o teléfono que sincroniza operaciones.
OperacionSync	Operación offline recibida con su estado (aplicada, duplicada, conflicto...).
PushSuscripcion	Suscripción Web Push (endpoint y claves) de un usuario.
EventoAnalitica	Evento de comportamiento usado para analítica.
ModeloPredictivo	Modelo supervisado de demanda (regresión lineal) con sus métricas de evaluación sobre datos sintéticos.
SegmentoCliente	Grupo de clientas obtenido con k-medias a partir de recencia, frecuencia y monto.
TransaccionPasarela	Cobro con un método habilitado para el país de la tienda, procesado por la pasarela simulada con clave de idempotencia.

9.1.3 Prototipos de interfaz

En ICONIX los casos de uso describen el diálogo con pantallas concretas, por lo que cada caso nombra su frontera. En el segundo ciclo los prototipos son las pantallas ya implementadas: se diseñaron como bocetos antes de programar y hoy funcionan como prototipos validados. Las fronteras usadas en los casos de uso son:

Frontera	Casos de uso
Archivo PDF o Excel	UC49
Cola local	UC42, UC43, UC44
Cámara del dispositivo	UC03
Dashboard de analítica	UC52, UC53
Interfaz Azure OpenAI	UC04, UC29, UC48
Interfaz pasarela	UC38, UC39, UC54
Micrófono	UC48
Navegador	UC35, UC47
Panel Mis tiendas	UC39
Panel de plataforma	UC41
Pantalla Administración de catálogo	UC09, UC30, UC43
Pantalla Bitácora	UC26
Pantalla Bot	UC29
Pantalla Carrito	UC05, UC22, UC30
Pantalla Catálogo	UC01
Pantalla Comprobante	UC05, UC34
Pantalla Configuración	UC24
Pantalla Cuenta	UC16, UC18
Pantalla Cuentas por cobrar	UC51
Pantalla Despacho	UC32
Pantalla Empezar	UC36
Pantalla Encargado	UC12
Pantalla Favoritos	UC17
Pantalla Ficha de producto	UC02, UC04, UC11, UC20, UC21
Pantalla Historial	UC19, UC33
Pantalla Inventario	UC25, UC31
Pantalla Login	UC15
Pantalla Moderación	UC26
Pantalla Nueva tienda	UC38
Pantalla Pago	UC27, UC54
Pantalla Planes	UC36
Pantalla Probador	UC03, UC28
Pantalla Proveedor	UC13
Pantalla Punto de venta	UC06, UC07, UC08, UC22, UC42, UC43, UC50
Pantalla Registro	UC14
Pantalla Reporte inteligente	UC48, UC49
Pantalla Reportes	UC10
Pantalla Reservas	UC11
Pantalla Sincronización	UC44, UC45, UC46
Pantalla Usuarios	UC23
Pantalla Verificar	UC37
Selector de tienda	UC40
Service Worker	UC35, UC47

Las pantallas principales del incremento, tomadas del sistema en funcionamiento con los datos de demostración, sirven como prototipo de GUI: cada texto de caso de uso nombra estos elementos tal como aparecen.

Figura 9.1.12

Pantalla Planes: módulos, planes y precios por período (UC36)

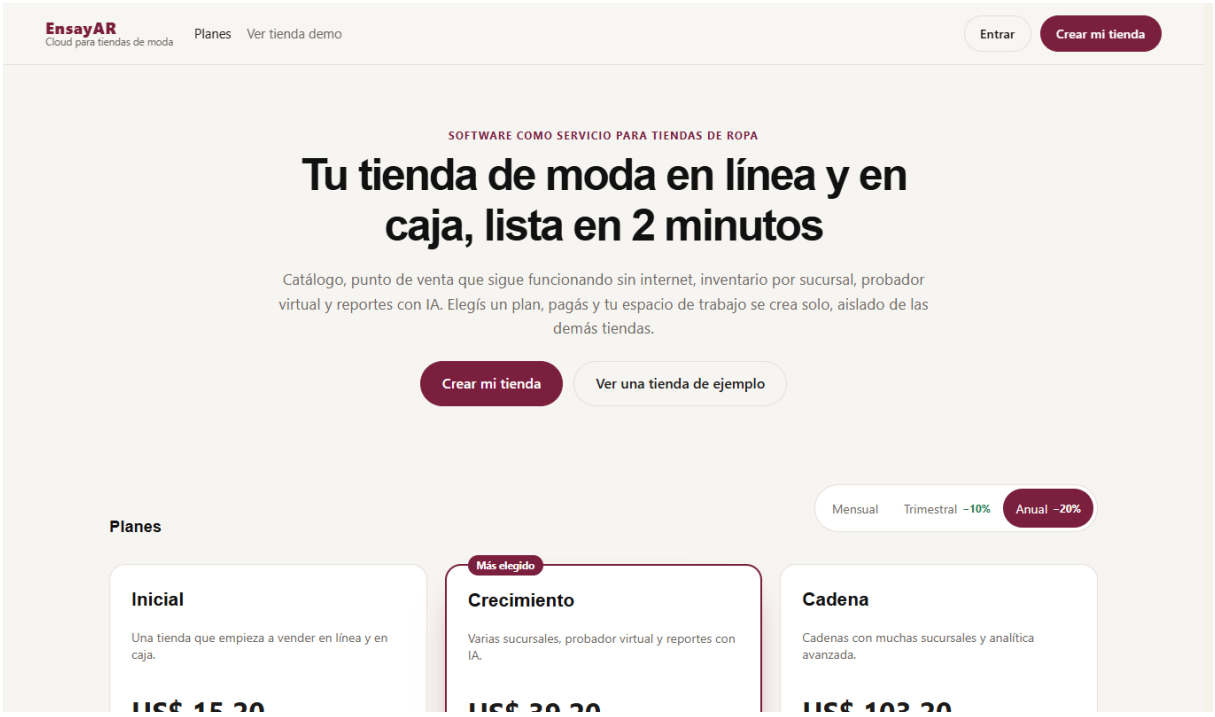


Figura 9.1.13

Pantalla Empezar: registro de la cuenta, paso 1 de 3 (UC36)

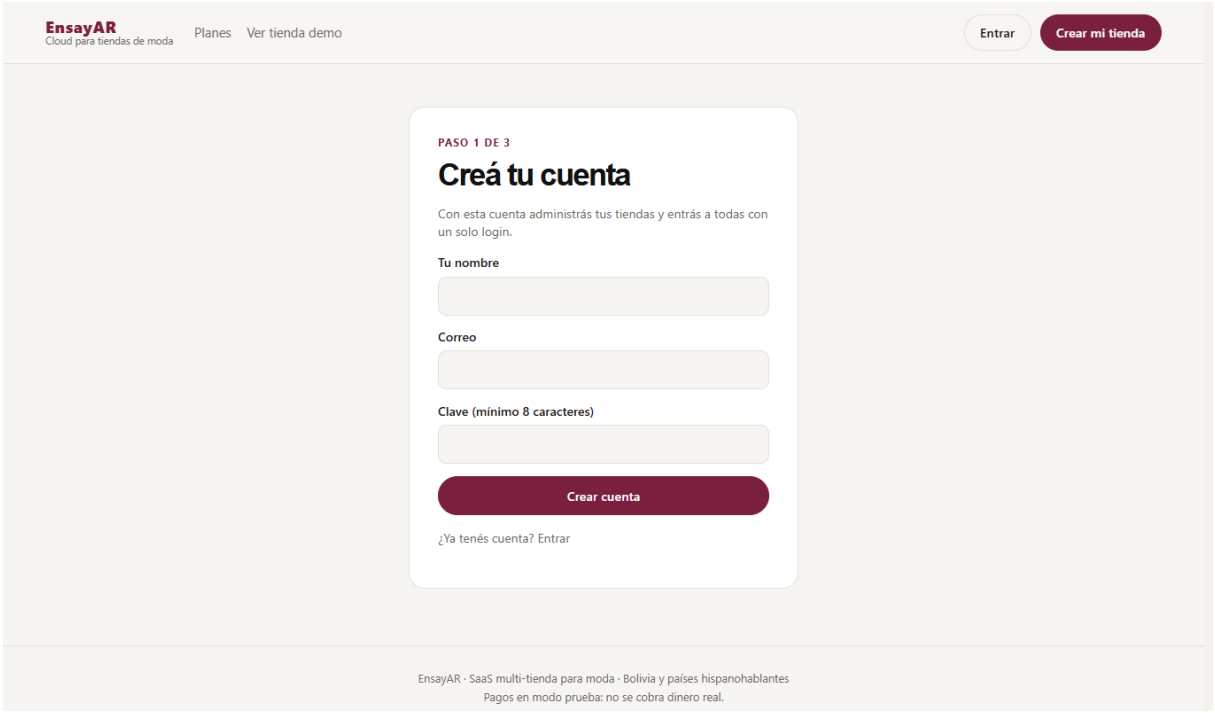


Figura 9.1.14

Panel de plataforma: MRR, ARR, ARPU, churn y proyección (UC41)

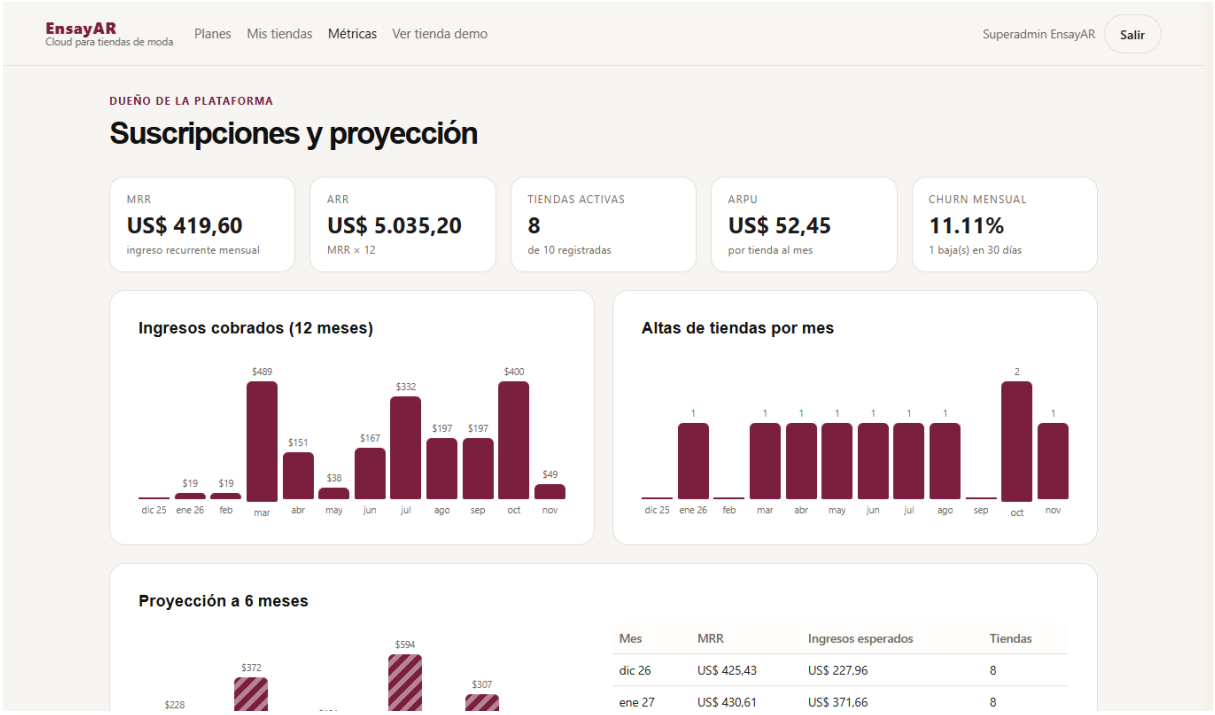


Figura 9.1.15

Pantalla Punto de venta sin conexión (UC42)

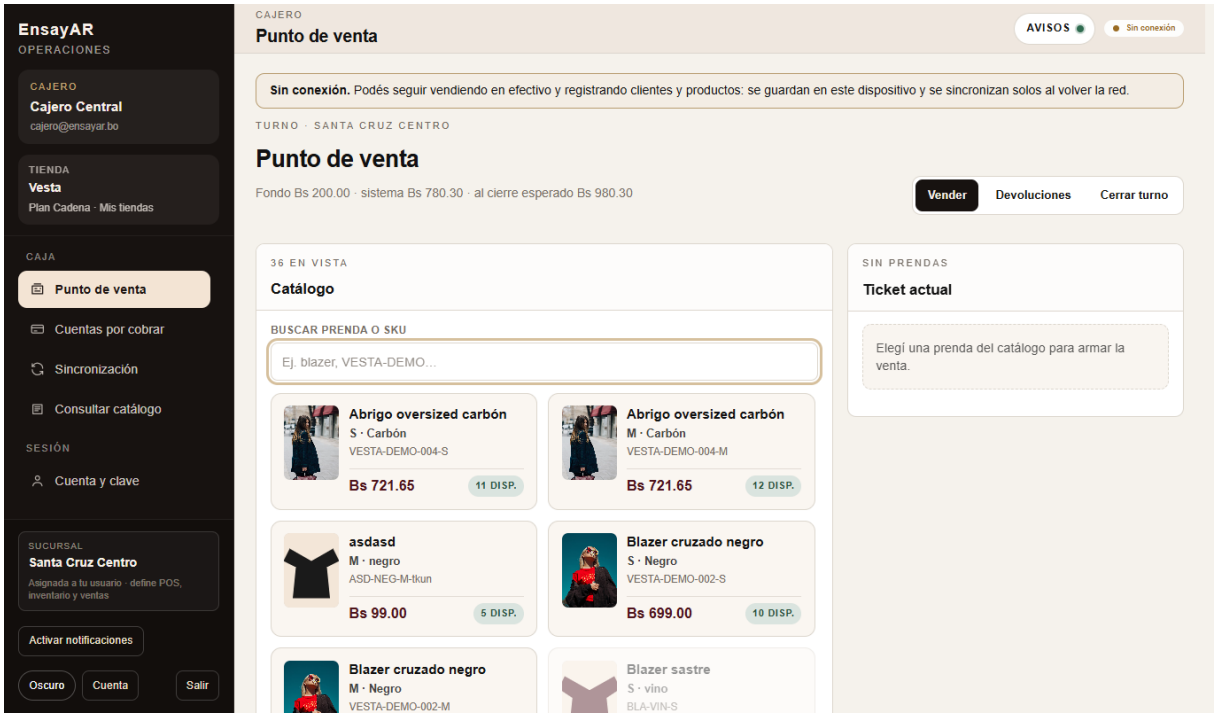


Figura 9.1.16

Pantalla Sincronización: estado del dispositivo y política offline (UC44 a UC46)

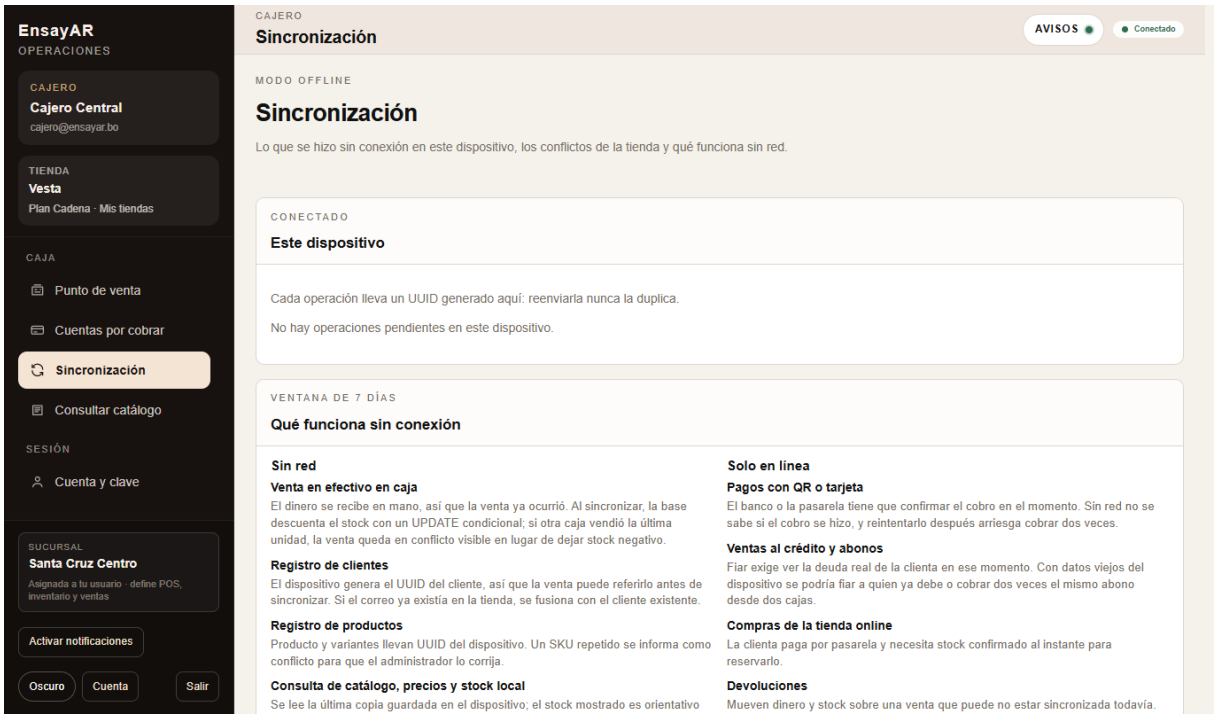


Figura 9.1.17

Pantalla Reportes: reporte inteligente desde lenguaje natural (UC48)



9.1.4 Actores

Actor	Descripción
Clienta	Compra en línea, prueba prendas, reserva y consulta su historial desde la web, la PWA o la app móvil.
Cajero	Opera el punto de venta de una sucursal: abre y cierra caja, cobra, fía y recibe abonos.
Encargado	Atiende reservas, despacho, inventario y reportes de su sucursal.
Administrador	Dueña o administradora de una tienda: catálogo, usuarios, reportes, configuración y analítica.
Proveedor	Registra el ingreso de su mercadería a la tienda.
Suscriptor	Persona o empresa que se registra en la plataforma, contrata un plan y crea su tienda.
Dueño de plataforma	Superadministrador de EnsayAR como SaaS: ve suscripciones, ingresos y proyecciones.
Servicio de IA	Azure OpenAI: interpreta preguntas y responde al bot; puede no estar disponible.
Pasarela de pago	Sistema externo que autoriza cobros con tarjeta, QR o billetera en modo sandbox.
Dispositivo sin red	Navegador (PWA) o teléfono que guarda operaciones mientras no hay conexión.

Servicio de IA, Pasarela de pago y Dispositivo sin red son actores secundarios: no inician casos de uso, pero el sistema dialoga con ellos a través de una frontera propia (Interfaz Azure OpenAI, Interfaz pasarela y Cola local).

9.1.5 Diagramas de casos de uso

Los casos se agrupan por lote. El lote 0 reúne los 35 casos heredados, divididos por paquete funcional; los lotes 1 a 5 son los nuevos del segundo ciclo. Cada diagrama dibuja el sujeto UML 2.5 (el rectángulo "EnsayAR: …"), que marca el límite del sistema: los casos de uso quedan dentro y los actores fuera. Los actores que inician casos van a la izquierda y los secundarios (Servicio de IA, Pasarela de pago y Dispositivo sin red) a la derecha. Las relaciones «include» y «extend» son las siguientes:

Caso	Relación	Caso base
UC03 Usar el probador virtual	«extend»	UC02 Ver ficha y variantes
UC04 Consultar la ficha con el asistente	«extend»	UC02 Ver ficha y variantes
UC11 Reservar una prenda	«extend»	UC02 Ver ficha y variantes
UC20 Opinar un producto	«extend»	UC02 Ver ficha y variantes
UC21 Pedir alerta de producto	«extend»	UC02 Ver ficha y variantes
UC28 Usar el probador 3D	«extend»	UC03 Usar el probador virtual
UC22 Aplicar cupón	«extend»	UC05 Comprar en línea
UC22 Aplicar cupón	«extend»	UC07 Registrar venta en el punto de venta
UC05 Comprar en línea	«include»	UC27 Simular un pago
UC05 Comprar en línea	«include»	UC34 Emitir comprobante simulado
UC07 Registrar venta en el punto de venta	«include»	UC34 Emitir comprobante simulado
UC42 Vender sin conexión	«extend»	UC07 Registrar venta en el punto de venta
UC50 Vender al crédito	«extend»	UC07 Registrar venta en el punto de venta
UC45 Resolver conflicto de stock	«extend»	UC44 Sincronizar operaciones pendientes
UC49 Ajustar y exportar reporte	«extend»	UC48 Generar reporte por texto o voz
UC36 Registrar cuenta en la plataforma	«include»	UC37 Verificar correo
UC38 Suscribirse y crear tienda	«include»	UC37 Verificar correo
UC54 Pagar con pasarela internacional	«extend»	UC27 Simular un pago
UC43 Registrar clienta o producto sin conexión	«extend»	UC09 Administrar catálogo

Figura 9.1.18

Casos de uso del lote 0, paquete Catálogo

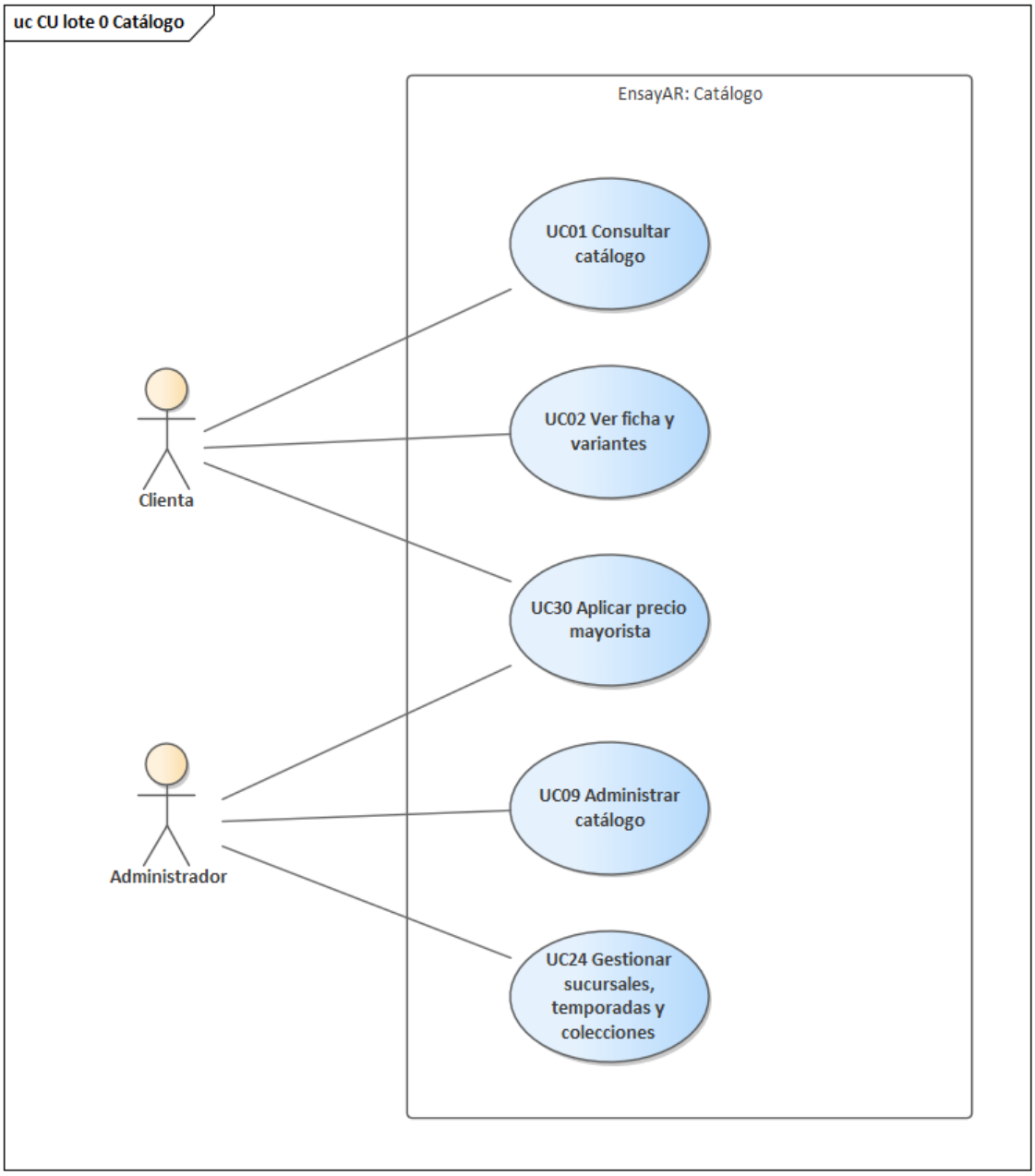


Figura 9.1.19

Casos de uso del lote 0, paquete Probador

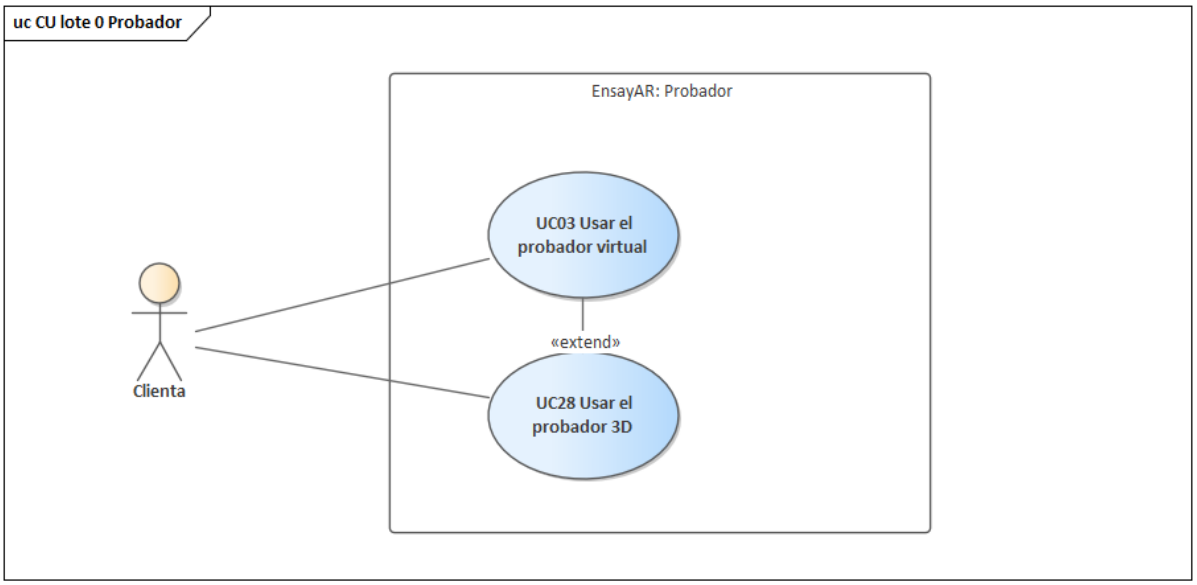


Figura 9.1.20

Casos de uso del lote 0, paquete Bot

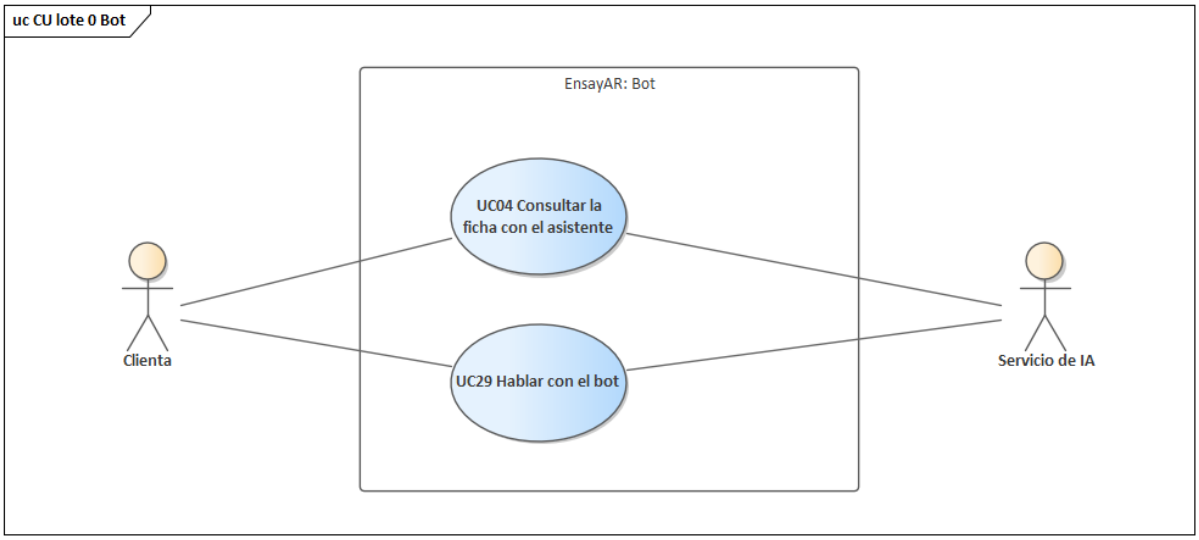


Figura 9.1.21

Casos de uso del lote 0, paquete Ventas

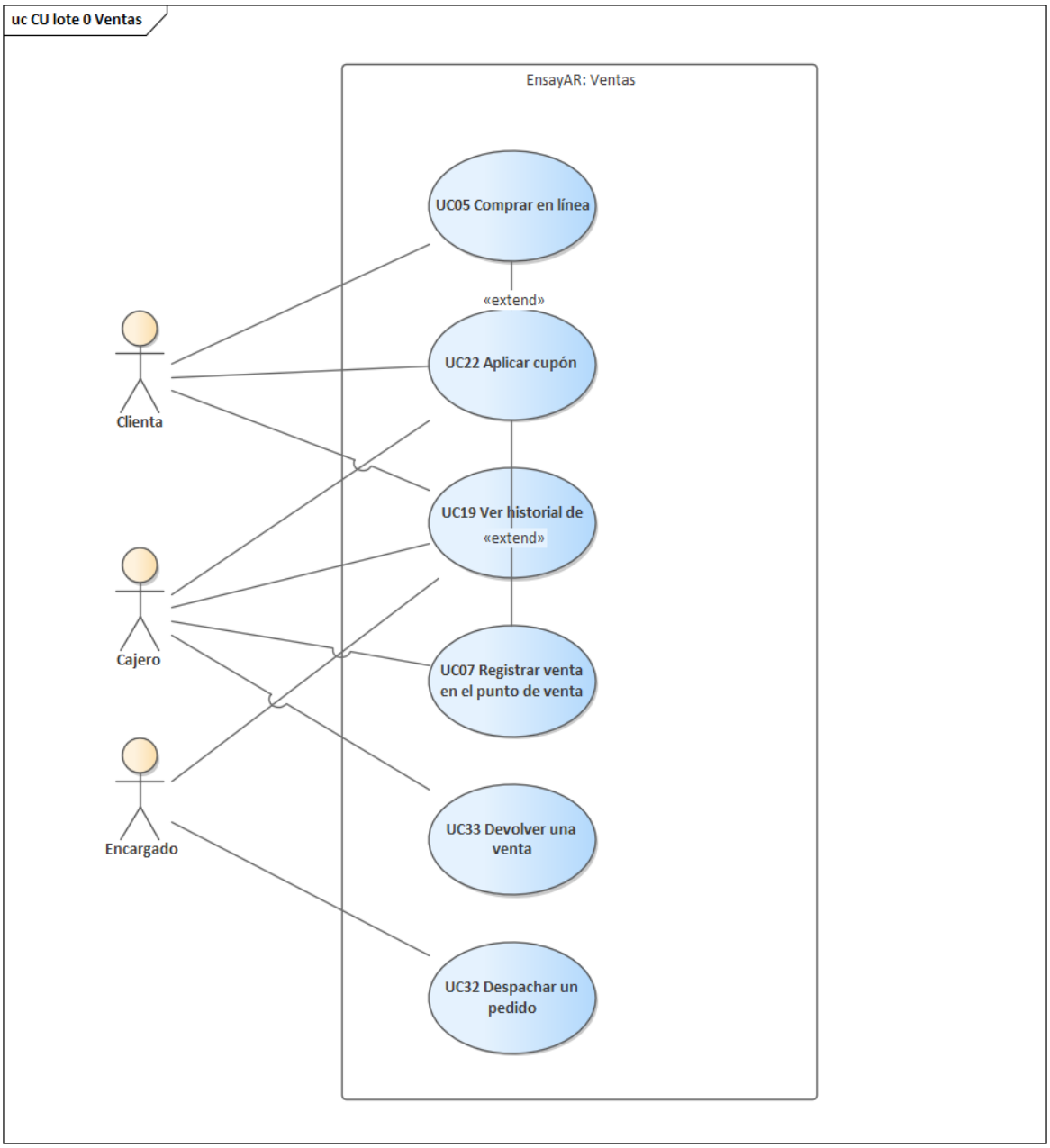


Figura 9.1.22

Casos de uso del lote 0, paquete Caja

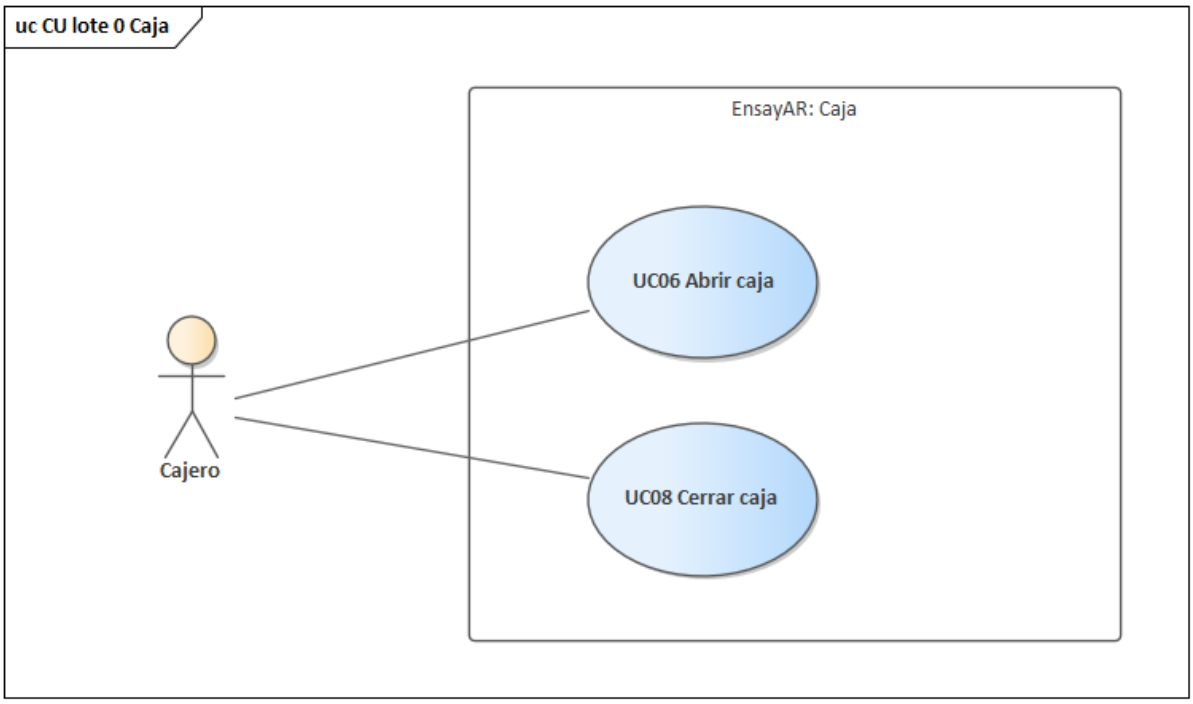


Figura 9.1.23

Casos de uso del lote 0, paquete Reportes

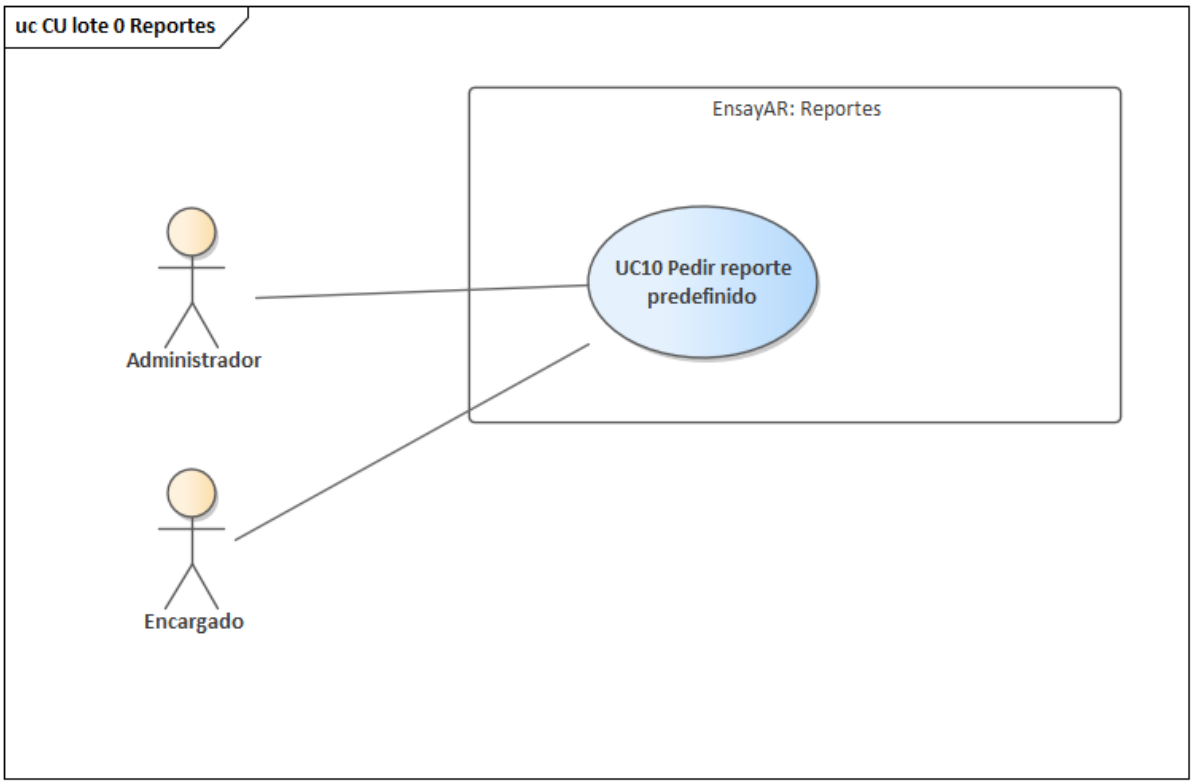


Figura 9.1.24

Casos de uso del lote 0, paquete Reservas

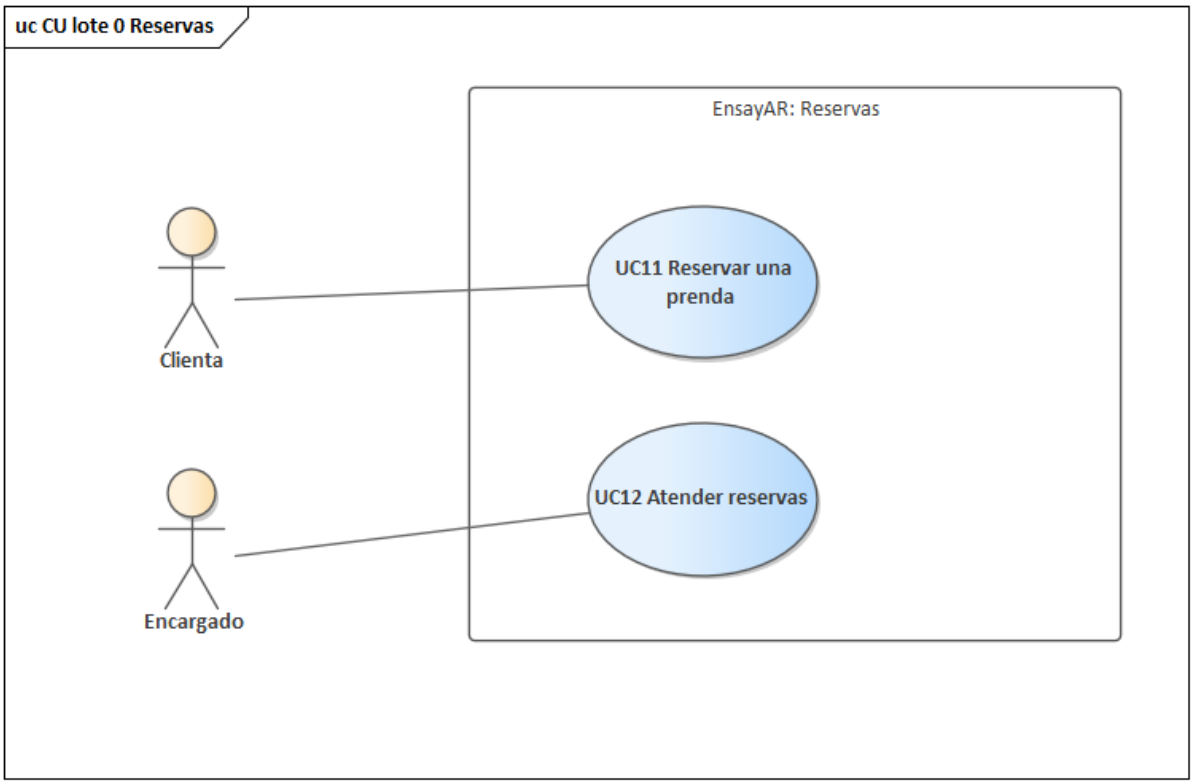


Figura 9.1.25

Casos de uso del lote 0, paquete Proveedores

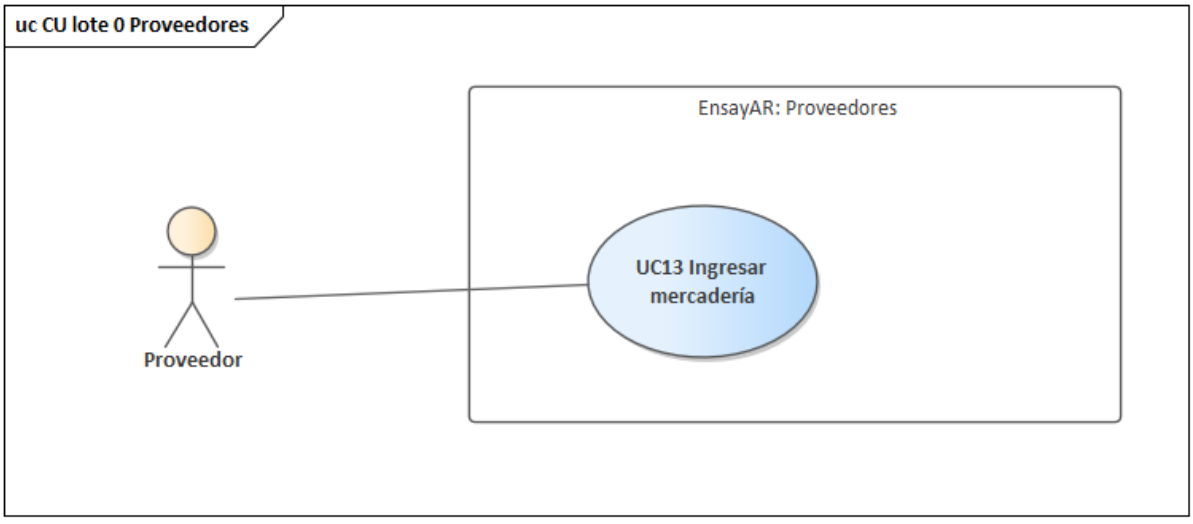


Figura 9.1.26

Casos de uso del lote 0, paquete Auth

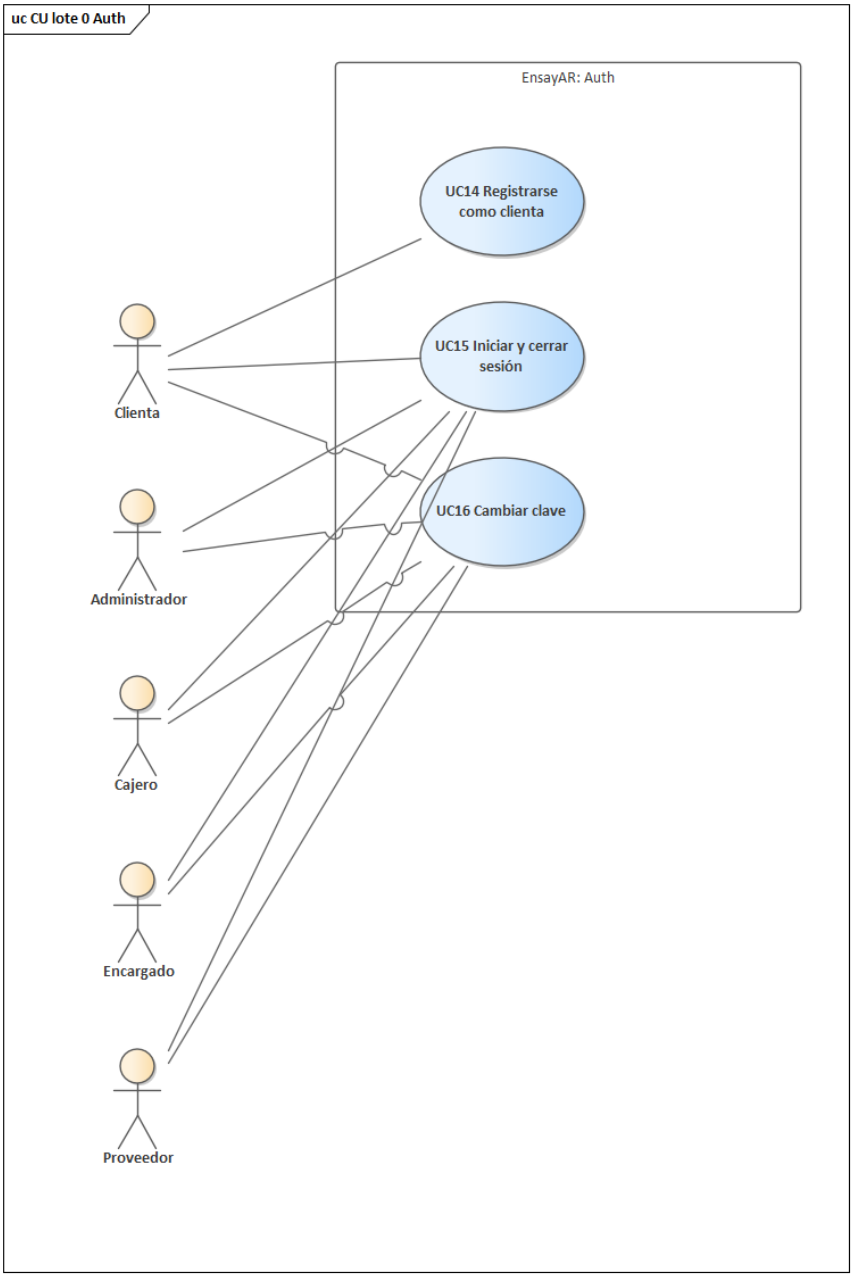


Figura 9.1.27

Casos de uso del lote 0, paquete Cliente

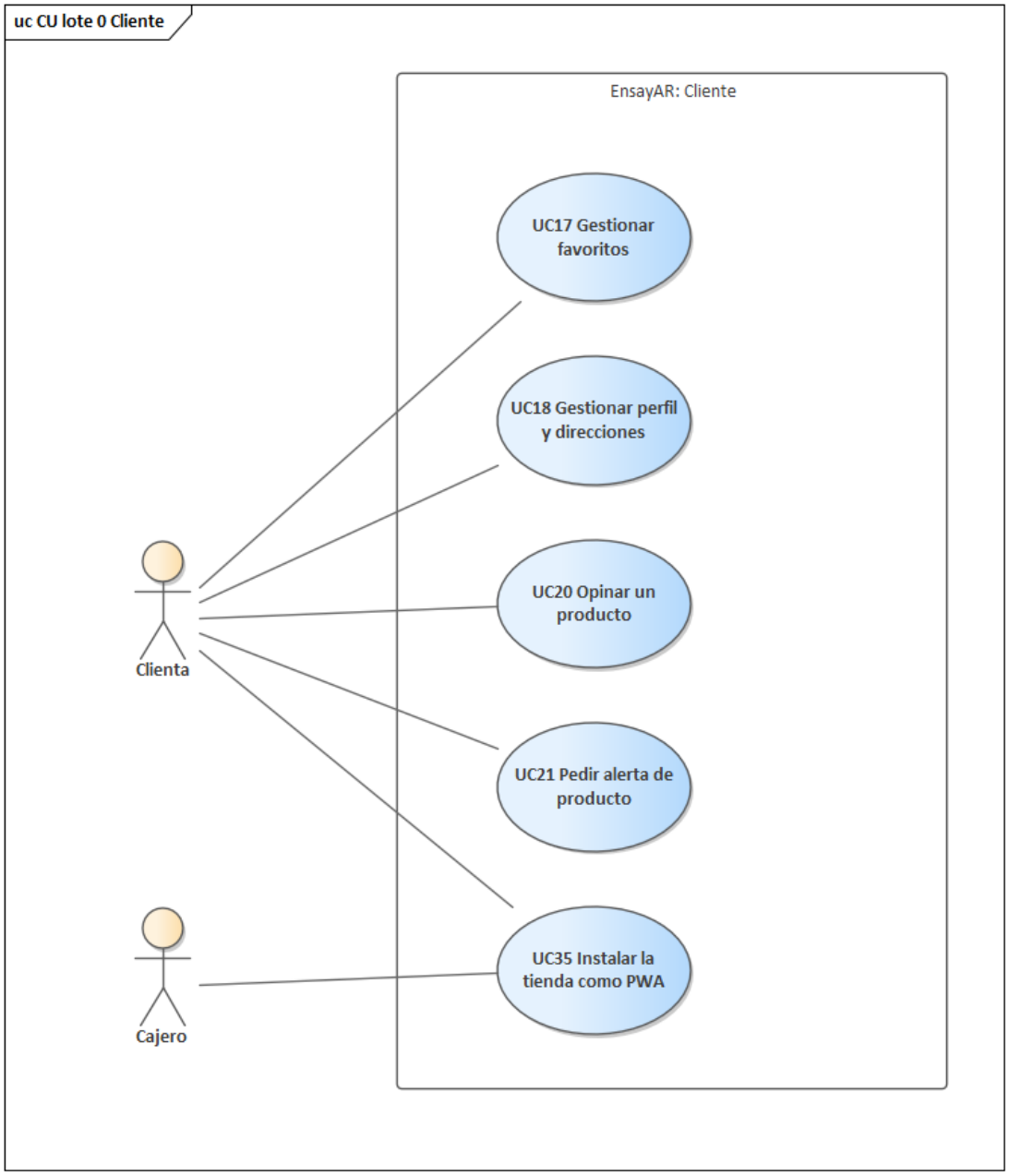


Figura 9.1.28

Casos de uso del lote 0, paquete Administración

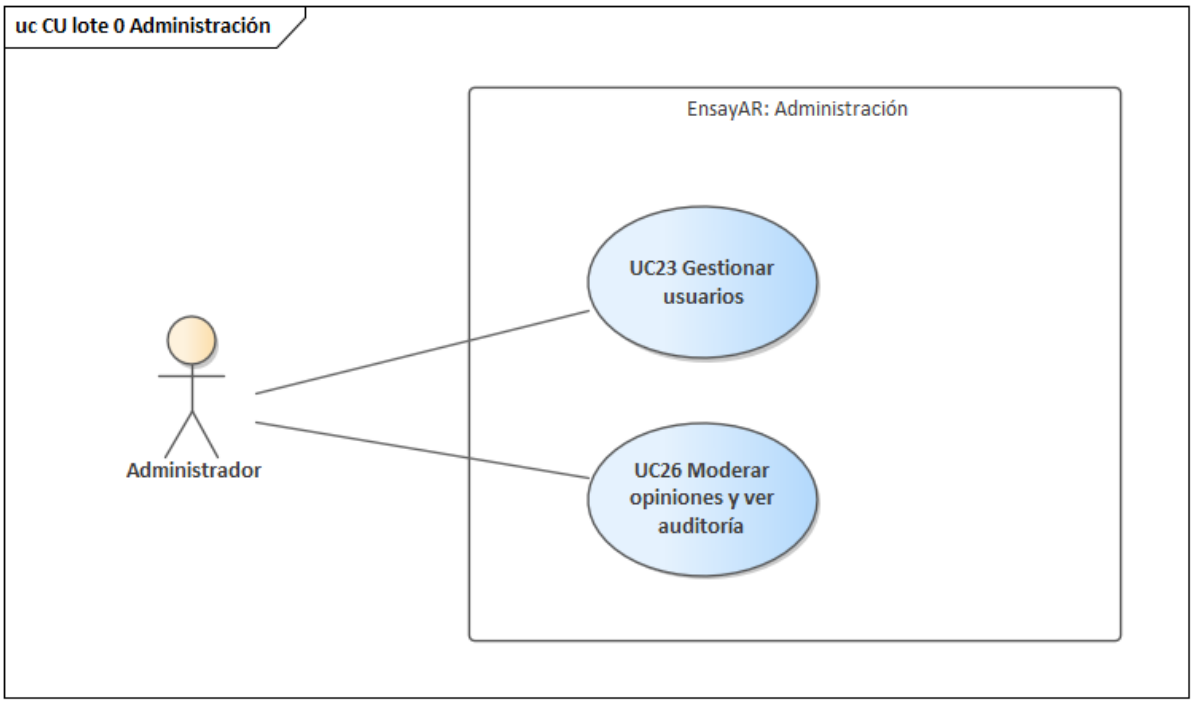


Figura 9.1.29

Casos de uso del lote 0, paquete Inventario

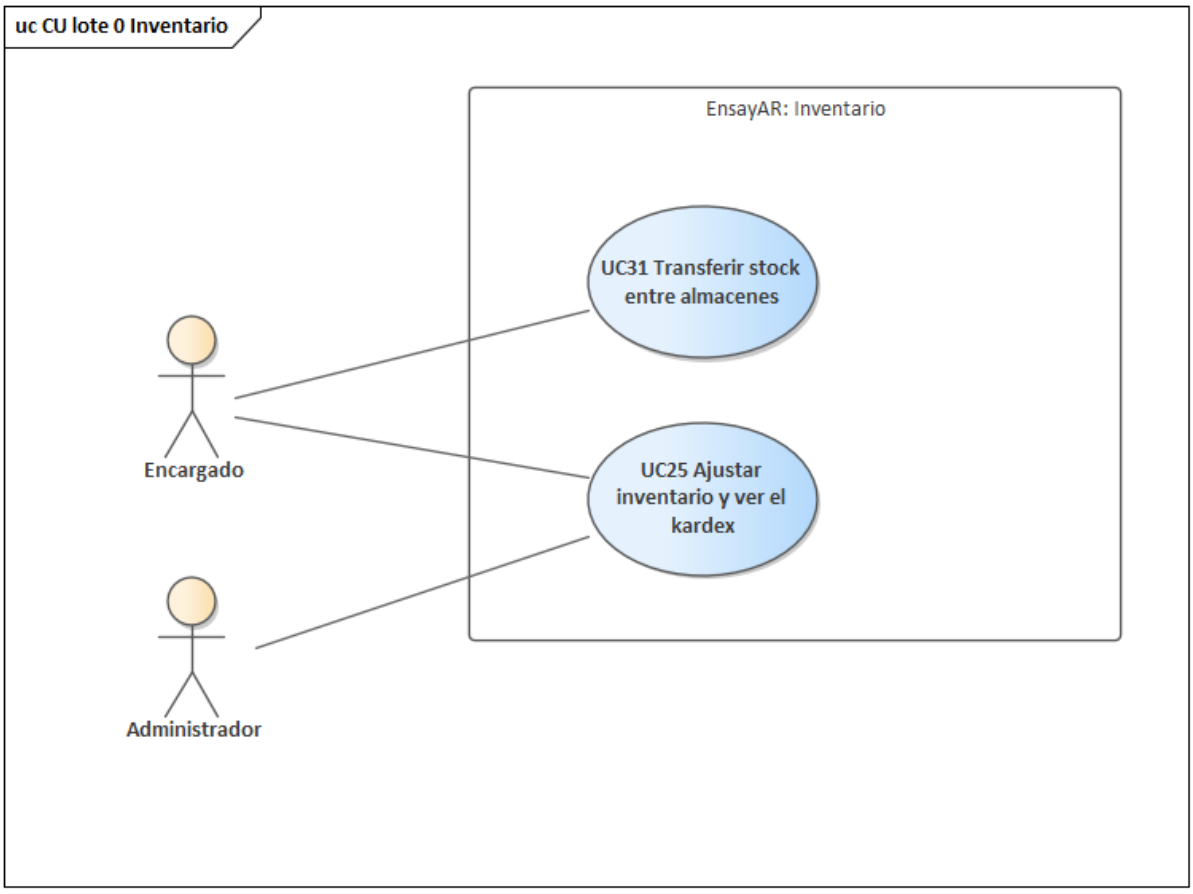


Figura 9.1.30

Casos de uso del lote 0, paquete Pagos

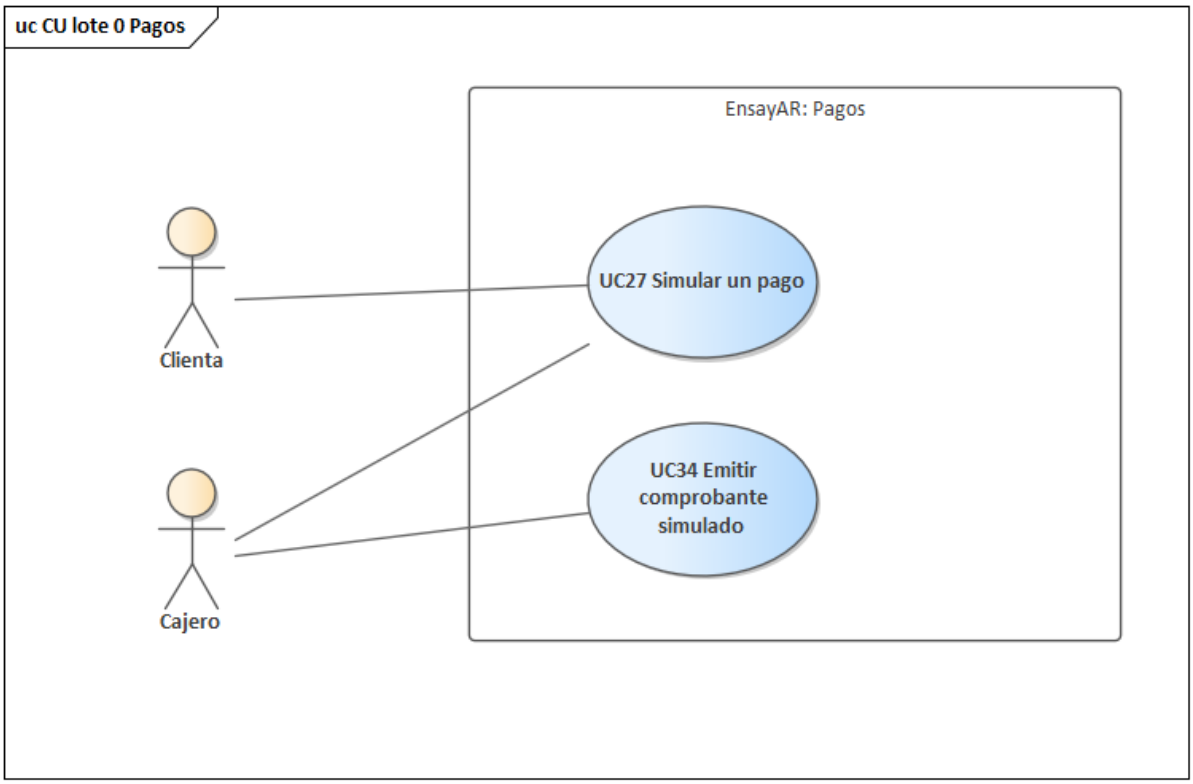


Figura 9.1.31

Casos de uso del lote 1, SaaS multi-tienda

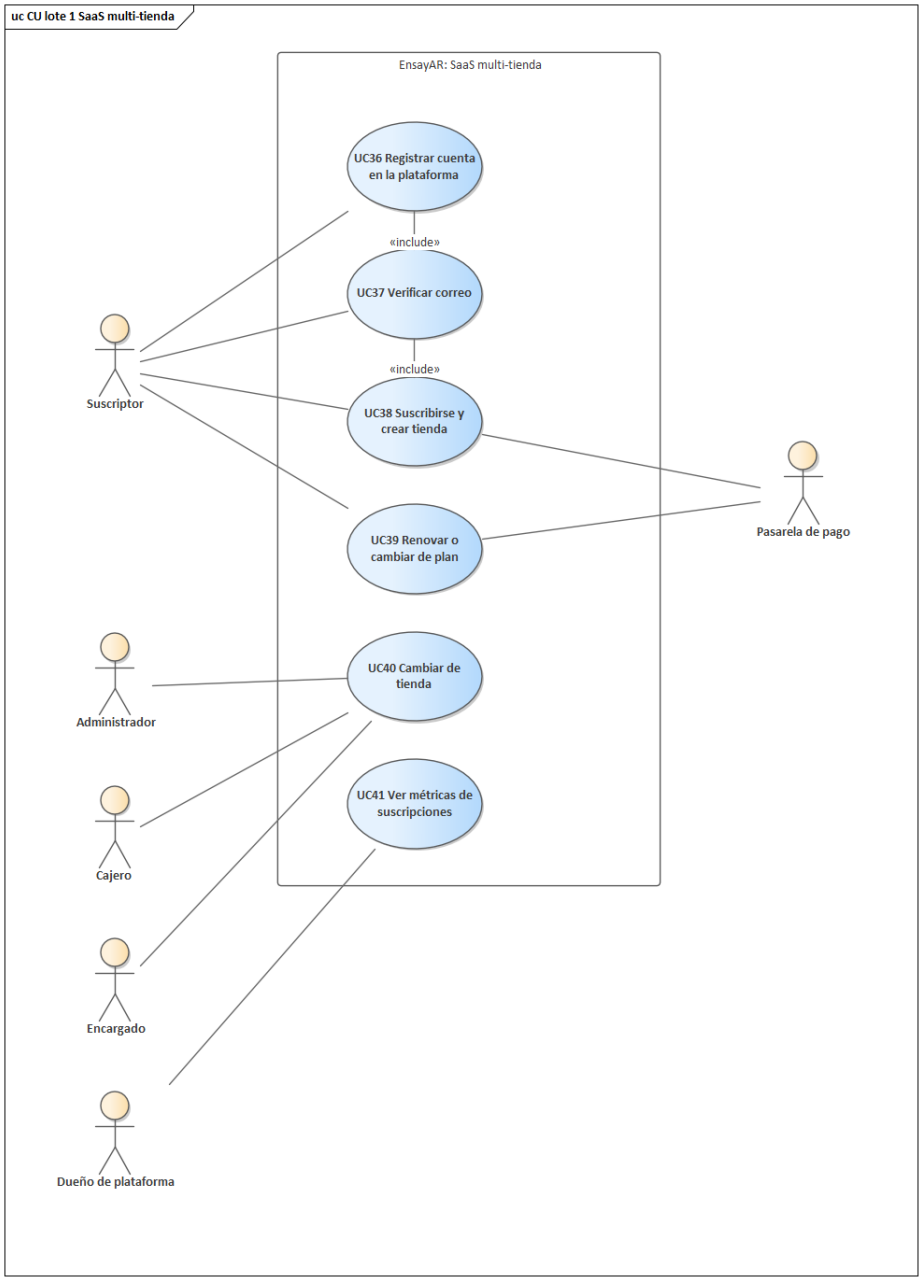


Figura 9.1.32

Casos de uso del lote 2, Operación sin conexión y sincronización

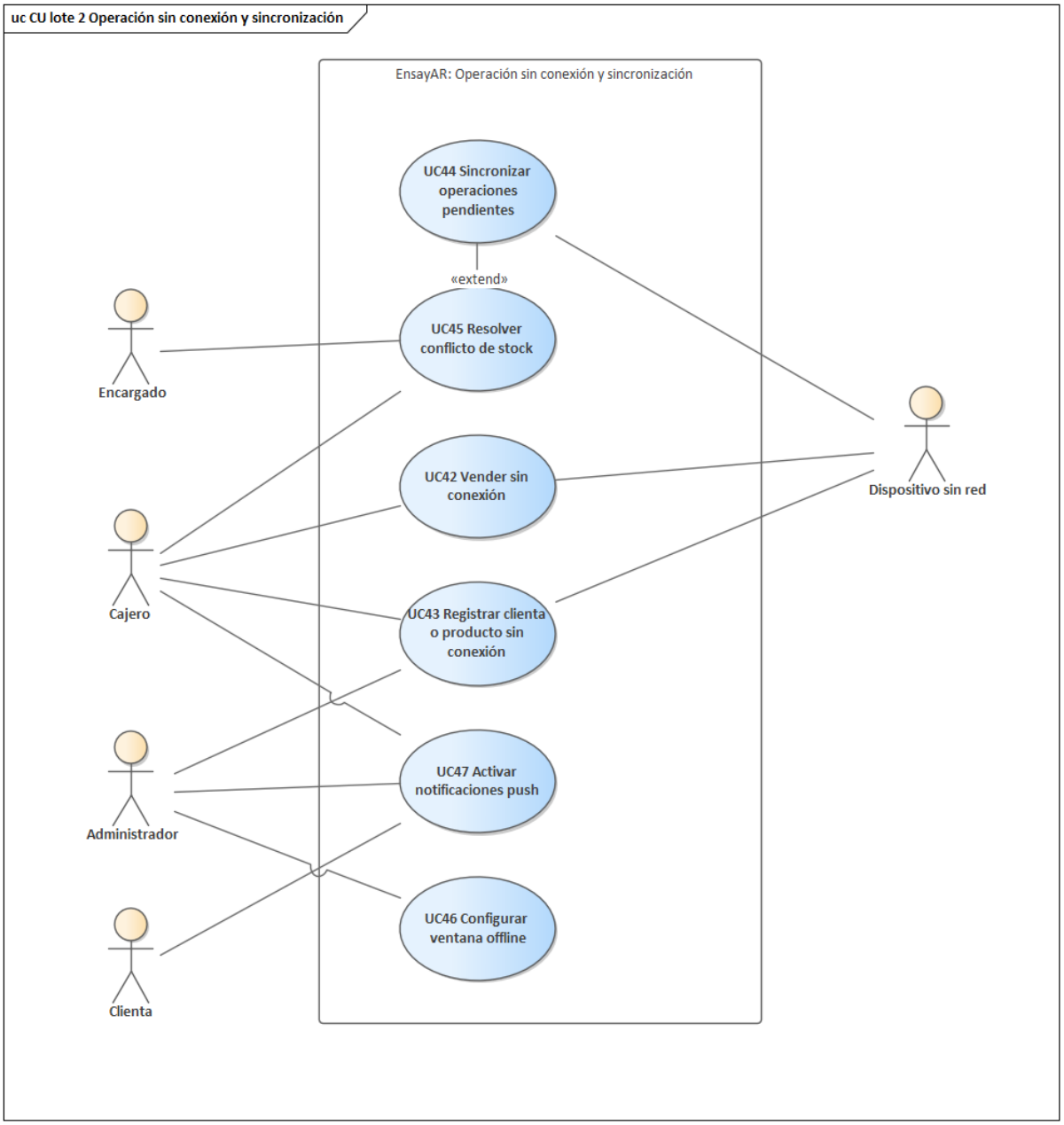


Figura 9.1.33

Casos de uso del lote 3, Reportes generativos y ventas al crédito

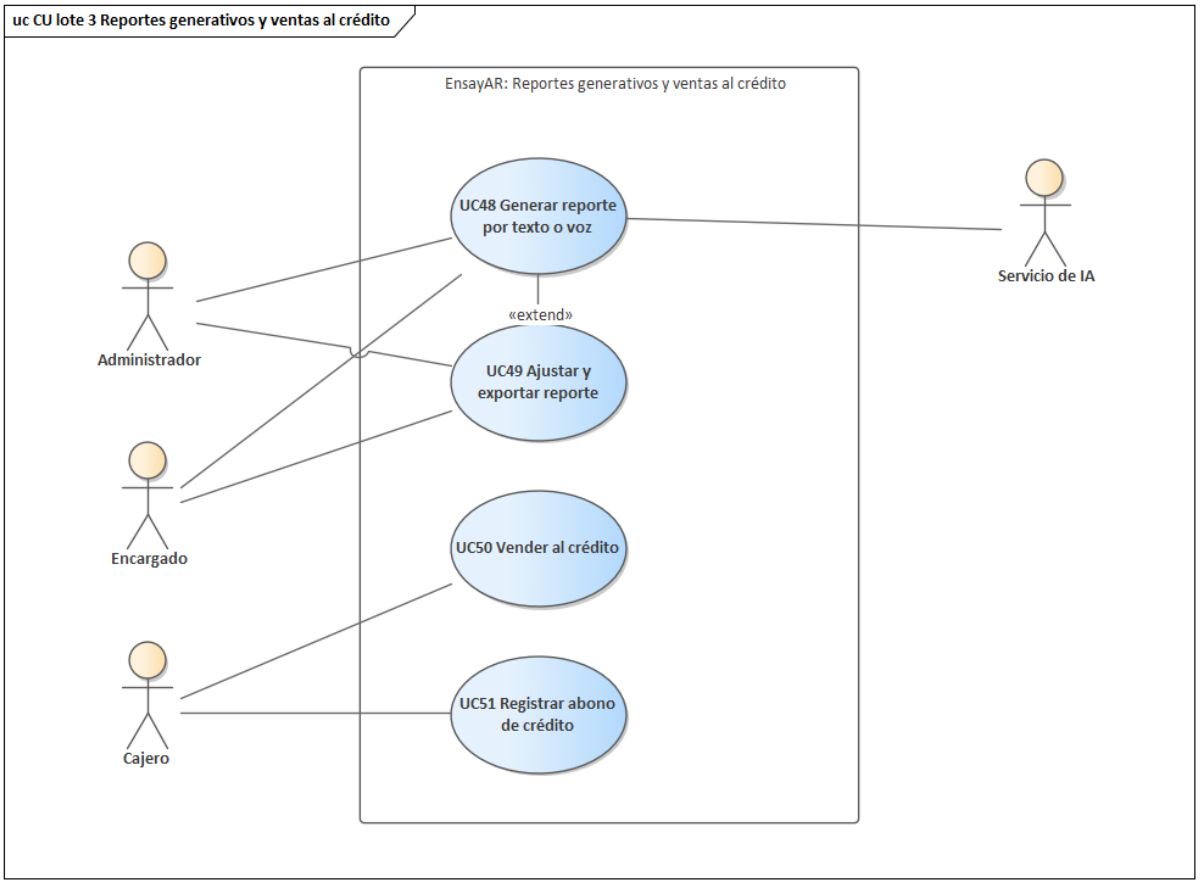


Figura 9.1.34

Casos de uso del lote 4, Analítica con aprendizaje automático

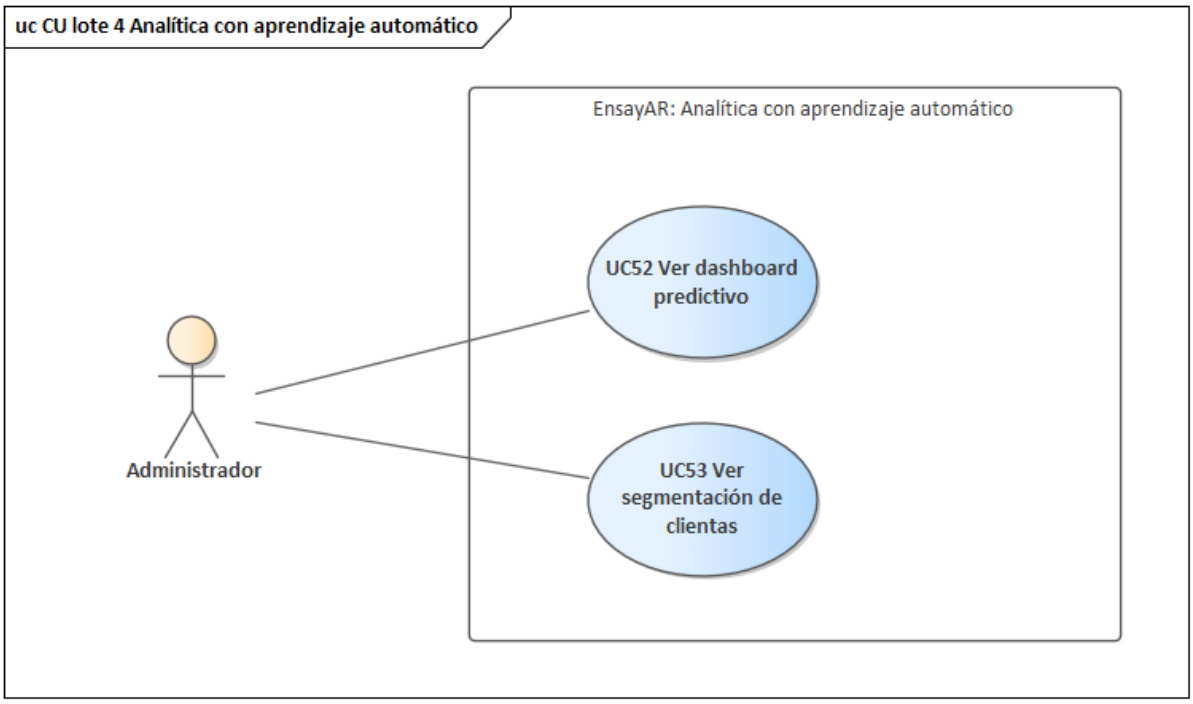
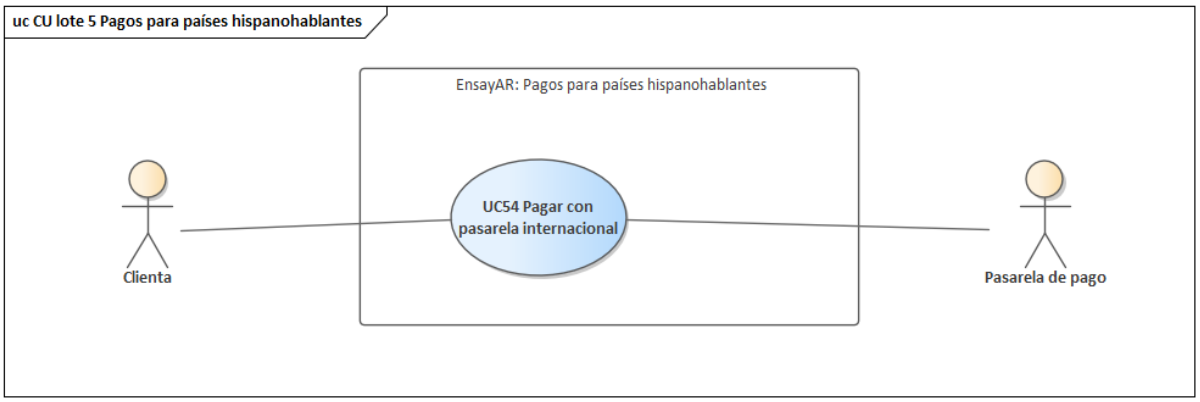


Figura 9.1.35

Casos de uso del lote 5, Pagos para países hispanohablantes



9.1.6 Texto de los casos de uso

Cada caso sigue la regla de los dos párrafos de ICONIX (Rosenberg y Stephens, 2007): un curso básico, que narra el camino en que todo sale bien como una secuencia de evento y respuesta entre el actor y el sistema, y los cursos alternos, cada uno encabezado por la condición

que lo dispara y con la respuesta del sistema. El texto va en voz activa, nombra los objetos del dominio tal como figuran en el modelo de dominio (en singular y sin tilde cuando son nombres de clase, como Categoria o Suscripcion) y nombra la pantalla donde ocurre cada acción. El código entre paréntesis de cada alterno indica la oración del curso básico donde se desvía (3a es la primera desviación de la tercera oración) y es el mismo que aparece en las guardas de los diagramas de secuencia. Los casos UC01 a UC35 son los del primer ciclo reescritos con estas reglas.

Lote 0: Casos heredados del primer ciclo, revisados con robustez y secuencia

UC01 Consultar catálogo

Actores: Clienta. **Paquete:** Catálogo. **Requisitos:** B1. **Pantallas:** Pantalla Catálogo.

Curso básico: La Clienta abre la Pantalla Catálogo de la tienda. El sistema resuelve la Tienda activa y lista los Productos publicados con precio y disponibilidad por Sucursal. La Clienta filtra por Categoria, ocasión, talla, color, precio o sucursal, o escribe una búsqueda. El sistema vuelve a listar solo los productos que cumplen los filtros. La Clienta elige un producto y pasa a la ficha (UC02).

Cursos alternos:

Sin conexión (1a): la PWA muestra el catálogo guardado en caché con un aviso de datos no actualizados.

Ningún producto coincide (4a): la Pantalla Catálogo muestra la lista vacía y ofrece limpiar filtros.

UC02 Ver ficha y variantes

Actores: Clienta. **Paquete:** Catálogo. **Requisitos:** B1. **Pantallas:** Pantalla Ficha de producto.

Curso básico: La Clienta abre la Pantalla Ficha de producto desde el catálogo. El sistema carga el Producto con sus Variantes, fotos, composición y cuidados. La Clienta elige talla y color. El sistema muestra el precio y el stock de la variante en cada Sucursal. Desde la ficha la Clienta puede probar (UC03), reservar (UC11), opinar (UC20) o agregar al carrito (UC05).

Cursos alternos:

El producto no existe o es de otra tienda (2a): el sistema responde 404 y vuelve al catálogo.

La variante está agotada (4a): la ficha la muestra deshabilitada y ofrece pedir alerta (UC21).

UC03 Usar el probador virtual

Actores: Clienta. **Paquete:** Probador. **Requisitos:** B1. **Pantallas:** Pantalla Probador y Cámara del dispositivo.

Curso básico: La Clienta pulsa Probar en la Pantalla Ficha de producto y el sistema abre la Pantalla Probador. La Clienta autoriza la Cámara del dispositivo o carga una foto; el video no sale del dispositivo. El sistema pide el overlay de la Variante y estima los puntos del cuerpo en el dispositivo. El sistema coloca la prenda anclada a los hombros; si la variante tiene modelo 3D usa UC28. La Clienta cambia de color y vuelve a la ficha.

Cursos alternos:

Sin permiso de cámara (2a): la Clienta marca los hombros a mano sobre una foto o una silueta.

La variante no tiene overlay (4a): el sistema avisa y ofrece otra variante.

UC04 Consultar la ficha con el asistente

Actores: Clienta y Servicio de IA. **Paquete:** Bot. **Requisitos:** B1. **Pantallas:** Pantalla Ficha de producto y Interfaz Azure OpenAI.

Curso básico: La Clienta escribe una pregunta sobre precio, talla, color, ocasión o cuidados en la Pantalla Ficha de producto. El sistema arma el contexto con los datos guardados del Producto y sus Variantes. El sistema consulta al Servicio de IA por la Interfaz Azure OpenAI con ese contexto. El sistema muestra la respuesta en la ficha.

Cursos alternos:

El Servicio de IA no responde (3a): el sistema contesta con reglas sobre los mismos datos de la ficha.

El dato no está en la ficha (4a): la respuesta dice que no lo tiene y no inventa stock ni precios.

UC05 Comprar en línea

Actores: Clienta. **Paquete:** Ventas. **Requisitos:** B1, O5, O6. **Pantallas:** Pantalla Carrito y Pantalla Comprobante.

Curso básico: La Clienta revisa la Pantalla Carrito con las variantes elegidas. La Clienta puede aplicar un Cupon (UC22) y elige el medio de pago (UC27). La Clienta confirma la compra; el carrito envía la Venta con un uuid generado en el cliente. El sistema recalcula precios en el servidor, descuenta el StockAlmacen con una actualización condicional y registra la Venta, cada LineaVenta y el MovimientoInventario en una transacción. El sistema crea el Pedido para despacho y muestra la Pantalla Comprobante (UC34).

Cursos alternos:

La Clienta pulsa confirmar dos veces (3a): el uuid repetido devuelve la misma venta sin cobrar de nuevo.

El disponible cambió (4a): el sistema rechaza la línea con conflicto de stock y no deja saldo negativo.

El pago simulado es rechazado (4b): no se registra la venta ni baja el stock.

UC06 Abrir caja

Actores: Cajero. **Paquete:** Caja. **Requisitos:** B1, O6. **Pantallas:** Pantalla Punto de venta.

Curso básico: El Cajero indica la Sucursal y el fondo inicial en la Pantalla Punto de venta. El sistema verifica que el Cajero no tenga otro CajaTurno abierto. El sistema abre el CajaTurno con el fondo inicial y lo muestra en pantalla.

Cursos alternos:

Fondo negativo o sucursal ajena (1a): el sistema rechaza la apertura.

Ya hay un turno abierto (2a): el sistema muestra ese turno en lugar de abrir otro.

UC07 Registrar venta en el punto de venta

Actores: Cajero. **Paquete:** Ventas. **Requisitos:** B1, O5, O6, O7. **Pantallas:** Pantalla Punto de venta.

Curso básico: El Cajero arma el ticket en la Pantalla Punto de venta con variantes y cantidades. El Cajero puede asociar una clienta y un Cupon, y elige el medio de pago. El sistema verifica que haya un CajaTurno abierto y comprueba el disponible en StockAlmacen. El sistema descuenta el stock con una actualización condicional y registra la Venta, cada LineaVenta y el MovimientoInventario en una transacción. El sistema suma el total al CajaTurno y emite el ticket y el comprobante (UC34).

Cursos alternos:

Sin red (1a): la venta en efectivo se guarda en la cola local y se sincroniza después (UC42).

Sin turno abierto o sin stock (3a): el sistema no cobra y explica el motivo.

El mismo uuid llega dos veces (4a): el sistema devuelve la venta ya cobrada sin descontar de nuevo.

UC08 Cerrar caja

Actores: Cajero. **Paquete:** Caja. **Requisitos:** B1, O1. **Pantallas:** Pantalla Punto de venta.

Curso básico: El Cajero declara el monto contado en la Pantalla Punto de venta. El sistema calcula el total del CajaTurno y la diferencia con lo contado. El sistema cierra el CajaTurno y muestra el arqueo.

Cursos alternos:

Hay operaciones offline pendientes o en conflicto (1a): el sistema bloquea el cierre hasta sincronizarlas.

El turno es de otra sucursal (2a): el sistema no lo cierra.

UC09 Administrar catálogo

Actores: Administrador. **Paquete:** Catálogo. **Requisitos:** B1, O1. **Pantallas:** Pantalla Administración de catálogo.

Curso básico: El Administrador crea o edita un Producto en la Pantalla Administración de catálogo. El Administrador agrega Variante con talla, color, fotos, overlay o modelo 3D y precio. El Administrador asigna Categoria, Temporada y Coleccion (UC24). El sistema valida que el plan de la Tienda permita el alta y guarda el producto. El Administrador fija el stock inicial (UC25).

Cursos alternos:

SKU repetido o talla y color ya existentes (4a): el sistema rechaza el alta.

Sin red (4b): el alta del producto queda en la cola local con uuid y se sincroniza después (UC43).

UC10 Pedir reporte predefinido

Actores: Administrador y Encargado. **Paquete:** Reportes. **Requisitos:** B1, S1. **Pantallas:** Pantalla Reportes.

Curso básico: El Administrador elige en la Pantalla Reportes un ranking rápido, el reporte del día o el de inventario. El sistema fija la tienda de la sesión en la consulta y agrega las ventas o el stock del período. El sistema registra la ReporteEjecucion y muestra la tabla con su narración.

Cursos alternos:

El Encargado solo ve su sucursal (1a): el sistema fuerza ese filtro.

No hay datos en el período (2a): el sistema muestra el reporte vacío.

UC11 Reservar una prenda

Actores: Clienta. **Paquete:** Reservas. **Requisitos:** B1. **Pantallas:** Pantalla Ficha de producto y Pantalla Reservas.

Curso básico: La Clienta elige variante y Sucursal en la Pantalla Ficha de producto y pide reservar. El sistema comprueba el disponible y aumenta la cantidad reservada en StockAlmacen; la física no baja. El sistema crea la Reserva con su horario y la muestra en la Pantalla Reservas.

Cursos alternos:

Sin disponible en esa sucursal (2a): el sistema no crea la reserva y sugiere otra sucursal.

UC12 Atender reservas

Actores: Encargado. **Paquete:** Reservas. **Requisitos:** B1. **Pantallas:** Pantalla Encargado.

Curso básico: El Encargado ve las Reservas pendientes de su Sucursal en la Pantalla Encargado. El Encargado completa o cancela una reserva. Al completar, el sistema genera la Venta, descuenta la cantidad física y libera el reservado. Al cancelar, el sistema solo libera el reservado.

Cursos alternos:

La reserva es de otra sucursal (2a): el sistema no la modifica.

UC13 Ingresar mercadería

Actores: Proveedor. **Paquete:** Proveedores. **Requisitos:** B1. **Pantallas:** Pantalla Proveedor.

Curso básico: El Proveedor elige producto, variante, Almacen y cantidad en la Pantalla Proveedor. El sistema verifica que el Producto sea de ese Proveedor. El sistema suma el StockAlmacen y escribe el MovimientoInventario de ingreso.

Cursos alternos:

El producto es de otro proveedor (2a): el sistema rechaza el ingreso.

UC14 Registrarse como clienta

Actores: Clienta. **Paquete:** Auth. **Requisitos:** B1, S7. **Pantallas:** Pantalla Registro.

Curso básico: La Clienta indica nombre, correo y clave en la Pantalla Registro de la tienda. El sistema busca o crea la Cuenta global con la clave cifrada. El sistema crea el Usuario con rol clienta en la Tienda e inicia la sesión.

Cursos alternos:

Clave débil o correo inválido (1a): el sistema no crea la cuenta.

El correo ya tiene cuenta en la plataforma (2a): el sistema exige su clave y la une a esta tienda.

UC15 Iniciar y cerrar sesión

Actores: Clienta, Cajero, Encargado, Administrador y Proveedor. **Paquete:** Auth. **Requisitos:** B1, S1. **Pantallas:** Pantalla Login.

Curso básico: El usuario ingresa correo y clave en la Pantalla Login de la tienda. El sistema valida la clave contra la Cuenta y busca su Usuario en la Tienda. El sistema entrega un token JWT con rol, sucursal y tienda. Al salir, el sistema guarda el token en TokenRevocado.

Cursos alternos:

Clave incorrecta, usuario inactivo o tienda suspendida (2a): el sistema no entrega token.

La cuenta no pertenece a esa tienda (2b): el sistema responde que no tiene acceso.

UC16 Cambiar clave

Actores: Clienta, Cajero, Encargado, Administrador y Proveedor. **Paquete:** Auth. **Requisitos:** B1. **Pantallas:** Pantalla Cuenta.

Curso básico: El usuario escribe la clave actual y la nueva en la Pantalla Cuenta. El sistema valida la clave actual contra la Cuenta. El sistema reemplaza el hash de la clave.

Cursos alternos:

La clave actual no coincide (2a): el sistema no cambia la clave.

UC17 Gestionar favoritos

Actores: Clienta. **Paquete:** Cliente. **Requisitos:** B1. **Pantallas:** Pantalla Favoritos.

Curso básico: La Clienta marca un Producto como favorito desde la ficha. El sistema guarda el Favorito sin duplicarlo. La Clienta ve sus favoritos en la Pantalla Favoritos y puede quitar uno.

Cursos alternos:

El producto ya era favorito (2a): el sistema no crea un registro repetido.

UC18 Gestionar perfil y direcciones

Actores: Clienta. **Paquete:** Cliente. **Requisitos:** B1. **Pantallas:** Pantalla Cuenta.

Curso básico: La Clienta abre la Pantalla Cuenta y consulta su nombre y correo. La Clienta agrega o edita una DireccionCliente de entrega. El sistema valida y guarda la dirección.

Cursos alternos:

La dirección está incompleta (3a): el sistema no la guarda.

UC19 Ver historial de compras

Actores: Clienta, Cajero y Encargado. **Paquete:** Ventas. **Requisitos:** B1, S1. **Pantallas:** Pantalla Historial.

Curso básico: El usuario abre la Pantalla Historial. El sistema lista las Ventas propias de la Clienta, o las de la Sucursal para cajero y encargado. El usuario abre el detalle con LineaVenta y medio de pago.

Cursos alternos:

Venta ajena (2a): si la Venta pertenece a otra persona, otra Sucursal u otra Tienda, el sistema la excluye de la lista.

UC20 Opinar un producto

Actores: Clienta. **Paquete:** Cliente. **Requisitos:** B1. **Pantallas:** Pantalla Ficha de producto.

Curso básico: La Clienta escribe valoración y comentario en la Pantalla Ficha de producto. El sistema guarda la Opinion como pendiente de moderación (UC26).

Cursos alternos:

Ya opinó ese producto (2a): el sistema rechaza la segunda opinión.

UC21 Pedir alerta de producto

Actores: Clienta. **Paquete:** Cliente. **Requisitos:** B1. **Pantallas:** Pantalla Ficha de producto.

Curso básico: En la Pantalla Ficha de producto la Clienta pide aviso de disponibilidad o de precio. El sistema guarda la AlertaProducto y luego avisa con una Notificacion.

Cursos alternos:

La misma alerta ya existe (2a): el sistema no la duplica.

UC22 Aplicar cupón

Actores: Clienta y Cajero. **Paquete:** Ventas. **Requisitos:** B1. **Pantallas:** Pantalla Carrito y Pantalla Punto de venta.

Curso básico: La Clienta o el Cajero ingresa el código de Cupon durante la compra. El sistema valida vigencia y usos, y recalcula el total con el porcentaje. Al cobrar, el sistema suma un uso al cupón y la Venta guarda el descuento.

Cursos alternos:

Código inválido, vencido o agotado (2a): el total no cambia y se informa el motivo.

UC23 Gestionar usuarios

Actores: Administrador. **Paquete:** Administración. **Requisitos:** B1, S3. **Pantallas:** Pantalla Usuarios.

Curso básico: El Administrador lista los Usuarios de su Tienda en la Pantalla Usuarios. El Administrador crea un usuario con rol y sucursal, o edita nombre, rol y estado. El sistema verifica el límite de usuarios del Plan y guarda el cambio con su Auditoria.

Cursos alternos:

El plan llegó a su límite de usuarios (3a): el sistema rechaza el alta y sugiere cambiar de plan (UC39).

El correo ya es usuario de la tienda (3b): el sistema no lo duplica.

UC24 Gestionar sucursales, temporadas y colecciones

Actores: Administrador. **Paquete:** Catálogo. **Requisitos:** B1, S3. **Pantallas:** Pantalla Configuración.

Curso básico: El Administrador crea o edita una Sucursal en la Pantalla Configuración. El sistema verifica el límite de sucursales del plan y crea su Almacen. El Administrador crea Temporada y Coleccion para asignarlas en UC09.

Cursos alternos:

Nombre vacío (1a): el sistema no guarda.

El plan no admite más sucursales (2a): el sistema rechaza el alta.

UC25 Ajustar inventario y ver el kardex

Actores: Encargado y Administrador. **Paquete:** Inventario. **Requisitos:** B1, O6. **Pantallas:** Pantalla Inventario.

Curso básico: El Encargado consulta los registros de MovimientoInventario de su Sucursal en la Pantalla Inventario. El Encargado registra un ajuste con la cantidad resultante. El sistema actualiza el StockAlmacen y guarda el movimiento de ajuste.

Cursos alternos:

La cantidad nueva es menor que lo reservado (3a): el sistema rechaza el ajuste.

UC26 Moderar opiniones y ver auditoría

Actores: Administrador. **Paquete:** Administración. **Requisitos:** B1. **Pantallas:** Pantalla Moderación y Pantalla Bitácora.

Curso básico: El Administrador ve las Opiniones pendientes en la Pantalla Moderación y las publica o rechaza. El Administrador consulta la Auditoria en la Pantalla Bitácora.

Cursos alternos:

La opinión ya fue moderada (1a): un doble clic no la vuelve a pendiente.

UC27 Simular un pago

Actores: Clienta y Cajero. **Paquete:** Pagos. **Requisitos:** B1, O8. **Pantallas:** Pantalla Pago.

Curso básico: La Clienta o el Cajero elige efectivo, QR, tarjeta o transferencia en la Pantalla Pago. Con QR o tarjeta, el sistema crea el PagoSimulado y espera su autorización. Solo si el pago es aprobado, la Venta continúa el cobro de UC05 o UC07.

Cursos alternos:

Sin red (1a): QR y tarjeta no están disponibles porque necesitan confirmación en línea.

El pago es rechazado (3a): no hay venta cobrada y el stock no baja.

UC28 Usar el probador 3D

Actores: Clienta. **Paquete:** Probador. **Requisitos:** B1. **Pantallas:** Pantalla Probador.

Curso básico: Dentro de UC03, el sistema detecta que la Variante tiene modelo 3D. El sistema ajusta la prenda al torso estimado y muestra volumen y caída simplificada. La Clienta gira la vista o cambia de color.

Cursos alternos:

El dispositivo no soporta el modelo (1a): el sistema vuelve al overlay 2D sin perder la ficha.

UC29 Hablar con el bot

Actores: Clienta y Servicio de IA. **Paquete:** Bot. **Requisitos:** B1. **Pantallas:** Pantalla Bot y Interfaz Azure OpenAI.

Curso básico: La Clienta abre la Pantalla Bot y pregunta por talla, combinación, disponibilidad o su pedido. El sistema arma el contexto con Producto, StockAlmacen y las Ventas propias de la Clienta. El sistema consulta al Servicio de IA por la Interfaz Azure OpenAI y muestra la respuesta.

Cursos alternos:

Pide un reporte de la empresa (2a): el bot responde que corresponde a administración.

El Servicio de IA no responde (3a): el bot contesta con reglas sobre el catálogo.

UC30 Aplicar precio mayorista

Actores: Administrador y Clienta. **Paquete:** Catálogo. **Requisitos:** B1. **Pantallas:** Pantalla Administración de catálogo y Pantalla Carrito.

Curso básico: El Administrador carga precio mayorista y cantidad mínima en el Producto. La Clienta mayorista arma un carrito que alcanza el mínimo. Al cobrar, el sistema aplica el precio mayorista y la LineaVenta guarda el precio usado.

Cursos alternos:

No alcanza el mínimo (3a): el sistema aplica el precio de detalle.

UC31 Transferir stock entre almacenes

Actores: Encargado. **Paquete:** Inventario. **Requisitos:** B1, O7. **Pantallas:** Pantalla Inventario.

Curso básico: El Encargado elige variante, Almacen de origen, destino y cantidad en la Pantalla Inventario. El sistema resta en origen con una actualización condicional y suma en destino. El sistema escribe dos registros de MovimientoInventario de transferencia en la misma transacción.

Cursos alternos:

La cantidad supera el disponible u origen y destino son iguales (2a): el sistema no transfiere.

UC32 Despachar un pedido

Actores: Encargado. **Paquete:** Ventas. **Requisitos:** B1. **Pantallas:** Pantalla Despacho.

Curso básico: El Encargado ve los Pedidos pendientes en la Pantalla Despacho. El Encargado confirma el despacho. El sistema marca el pedido como enviado con su referencia y avisa a la Clienta con una Notificacion.

Cursos alternos:

La venta es de caja o el pedido ya fue enviado (2a): el sistema no lo despacha.

UC33 Devolver una venta

Actores: Cajero. **Paquete:** Ventas. **Requisitos:** B1, R2. **Pantallas:** Pantalla Historial.

Curso básico: El Cajero elige la Venta y las líneas a devolver en la Pantalla Historial. El sistema repone el StockAlmacen y registra el MovimientoInventario de devolución. El sistema crea la Devolucion y ajusta la caja; si la venta fue al crédito, reduce el saldo de la CuentaCobrar.

Cursos alternos:

Devolver más unidades que las vendidas (2a): el sistema rechaza la devolución.

Venta al crédito sin abonos (3a): la deuda se cancela y no sale efectivo de la caja.

UC34 Emitir comprobante simulado

Actores: Cajero. **Paquete:** Pagos. **Requisitos:** B1. **Pantallas:** Pantalla Comprobante.

Curso básico: Al terminar UC05 o UC07, el sistema arma el comprobante con Sucursal, LineaVenta, totales y un NIT de prueba. El sistema lo muestra en la Pantalla Comprobante y permite imprimirlo o enviarlo por correo.

Cursos alternos:

Venta no cobrada (1a): el sistema no genera el comprobante y la pantalla muestra el motivo.

UC35 Instalar la tienda como PWA

Actores: Clienta y Cajero. **Paquete:** Cliente. **Requisitos:** O1, O10. **Pantallas:** Navegador y Service Worker.

Curso básico: El Navegador ofrece instalar la aplicación a partir del manifest. El Service Worker guarda la estructura de la aplicación y los datos recientes en caché. La Clienta o el Cajero abre la tienda instalada desde el escritorio o el teléfono.

Cursos alternos:

El navegador no soporta instalación (1a): la tienda funciona igual como sitio web.

Sin red (2a): la aplicación abre desde la caché y la caja trabaja en cola (UC42).

Lote 1: SaaS multi-tienda

UC36 Registrar cuenta en la plataforma

Actores: Suscriptor. **Paquete:** Plataforma. **Requisitos:** S2, S3, S4. **Pantallas:** Pantalla Planes y Pantalla Empezar.

Curso básico: El Suscriptor revisa módulos, planes y precios en la Pantalla Planes. El Suscriptor elige un plan y abre la Pantalla Empezar. El Suscriptor indica nombre, correo y clave. El sistema crea la Cuenta sin verificar y envía el enlace de verificación (UC37).

Cursos alternos:

El correo ya tiene cuenta (4a): el sistema pide iniciar sesión en lugar de crear otra.

No hay servidor de correo configurado (4b): en demo, el enlace se muestra en pantalla.

UC37 Verificar correo

Actores: Suscriptor. **Paquete:** Plataforma. **Requisitos:** S6. **Pantallas:** Pantalla Verificar.

Curso básico: El Suscriptor abre el enlace recibido y llega a la Pantalla Verificar. El sistema valida el token y marca la Cuenta como verificada. El sistema inicia la sesión de plataforma y lleva al Suscriptor a elegir plan.

Cursos alternos:

El token venció o no existe (2a): el sistema ofrece reenviar el correo.

Correo sin verificar (2b): el sistema impide que la Cuenta se suscriba o entre a una Tienda.

UC38 Suscribirse y crear tienda

Actores: Suscriptor y Pasarela de pago. **Paquete:** Plataforma. **Requisitos:** S3, S4, S5, O6. **Pantallas:** Pantalla Nueva tienda y Interfaz pasarela.

Curso básico: El Suscriptor elige plan y duración (1, 3 o 12 meses) e indica nombre y dirección (slug) de la tienda en la Pantalla Nueva tienda. El sistema verifica que el slug esté libre. El Suscriptor ingresa la tarjeta de prueba; el sistema cobra por la Interfaz pasarela con una clave de idempotencia. Con el pago aprobado, el sistema crea en una transacción la Tienda, la Suscripcion, el PagoSuscripcion, la Sucursal, el Almacen y el Usuario administrador. El sistema entra a la tienda nueva sin pedir otro login.

Cursos alternos:

El slug está ocupado (2a): el sistema sugiere otro.

El pago es rechazado (3a): no se crea la tienda y el Suscriptor puede reintentar.

El Suscriptor pulsa pagar dos veces (3b): la misma clave de idempotencia devuelve el primer resultado.

UC39 Renovar o cambiar de plan

Actores: Suscriptor y Pasarela de pago. **Paquete:** Plataforma. **Requisitos:** S3, S4. **Pantallas:** Panel Mis tiendas y Interfaz pasarela.

Curso básico: El Suscriptor abre el detalle de su Tienda en el Panel Mis tiendas y ve uso y vencimiento. El Suscriptor elige renovar o cambiar a otro Plan y duración. El sistema cobra por la Interfaz pasarela y extiende la Suscripcion desde su vencimiento.

Cursos alternos:

El plan nuevo tiene menos usuarios que los activos (2a): el sistema avisa antes de cobrar.

El pago es rechazado (3a): la suscripción no cambia.

UC40 Cambiar de tienda

Actores: Administrador, Cajero y Encargado. **Paquete:** Auth. **Requisitos:** S7. **Pantallas:** Selector de tienda.

Curso básico: El usuario abre el Selector de tienda en la barra lateral. El sistema lista las Tiendas donde su Cuenta tiene un Usuario activo. El usuario elige otra tienda. El sistema emite un token nuevo para esa tienda y recarga los módulos sin pedir la clave.

Cursos alternos:

La cuenta tiene una sola tienda (2a): el selector no se muestra.

La tienda está suspendida o vencida (3a): el sistema no permite entrar y muestra el motivo.

UC41 Ver métricas de suscripciones

Actores: Dueño de plataforma. **Paquete:** Plataforma. **Requisitos:** S9. **Pantallas:** Panel de plataforma.

Curso básico: El Dueño de plataforma abre el Panel de plataforma. El sistema calcula MRR, ARR, ARPU, tiendas activas y churn a partir de Suscripcion y PagoSuscripcion. El sistema arma las series de 12 meses y la proyección a 6 meses y las muestra.

Cursos alternos:

Usuario sin rol de superadministrador (1a): el sistema rechaza el acceso al panel.

Lote 2: Operación sin conexión y sincronización

UC42 Vender sin conexión

Actores: Cajero y Dispositivo sin red. **Paquete:** Sincronización. **Requisitos:** O1, O2, O4, O8, O9. **Pantallas:** Pantalla Punto de venta y Cola local.

Curso básico: Sin red, el Cajero arma el ticket en la Pantalla Punto de venta y cobra en efectivo. El sistema genera el uuid de la Venta en el dispositivo y guarda la operación en la Cola local (IndexedDB en web, SQLite en móvil). El sistema descuenta el stock en la copia local y muestra el ticket marcado como pendiente. Al volver la red, la operación se envía en el siguiente lote de sincronización (UC44).

Cursos alternos:

El Cajero elige QR, tarjeta o crédito (1a): el sistema no lo permite sin red y lo explica.

La operación supera la ventana offline de la tienda (4a): llegará como vencida y se informa al Cajero.

UC43 Registrar clienta o producto sin conexión

Actores: Cajero, Administrador y Dispositivo sin red. **Paquete:** Sincronización. **Requisitos:** O1, O2, O4. **Pantallas:** Pantalla Punto de venta, Pantalla Administración de catálogo y Cola local.

Curso básico: Sin red, el Cajero registra una clienta nueva, o el Administrador da de alta un producto. El sistema asigna el uuid en el dispositivo y guarda la operación en la Cola local. Las ventas posteriores pueden referir a esa clienta por su uuid antes de sincronizar.

Cursos alternos:

Al sincronizar, el correo de la clienta ya existía (3a): el servidor fusiona con el Usuario existente.

UC44 Sincronizar operaciones pendientes

Actores: Dispositivo sin red. **Paquete:** Sincronización. **Requisitos:** O3, O4, O6, O7. **Pantallas:** Cola local y Pantalla Sincronización.

Curso básico: Al volver la red, al abrir la app o cada 30 segundos, el sistema toma las operaciones de la Cola local. El sistema envía un lote con la cabecera de idempotencia y el identificador del Dispositivo. El servidor procesa cada operación en su propia transacción y guarda su OperacionSync. El servidor responde un estado por operación: aplicada, duplicada, conflicto, vencida o rechazada. El sistema actualiza la Cola local y muestra el resultado en la Pantalla Sincronización.

Cursos alternos:

Se corta la red a mitad del envío (2a): el lote se reintenta con la misma clave y no se duplica nada.

Una operación ya había llegado (3a): el servidor la marca duplicada y devuelve el resultado original.

Falta stock (3b): la operación queda en conflicto (UC45) y las demás siguen.

UC45 Resolver conflicto de stock

Actores: Cajero y Encargado. **Paquete:** Sincronización. **Requisitos:** O5. **Pantallas:** Pantalla Sincronización.

Curso básico: Dos cajas vendieron sin red la última unidad; al sincronizar, la segunda operación queda en conflicto con el disponible real. El Cajero ve el conflicto en la Pantalla Sincronización con lo pedido y lo disponible. El Cajero elige vender lo disponible o descartar la operación. El sistema reintenta con la cantidad disponible, o marca la OperacionSync como descartada.

Cursos alternos:

Resolución por el Encargado (2a): el Encargado resuelve desde el panel de la Tienda los conflictos de cualquier caja.

Ya no queda nada disponible (3a): solo se puede descartar.

UC46 Configurar ventana offline

Actores: Administrador. **Paquete:** Sincronización. **Requisitos:** O8, O9. **Pantallas:** Pantalla Sincronización.

Curso básico: El Administrador abre la política offline en la Pantalla Sincronización. El sistema muestra qué funciona sin red y qué no, con el motivo de cada caso. El Administrador fija la ventana offline entre 1 y 30 días. El sistema guarda la ventana en la Tienda; las operaciones más viejas llegarán como vencidas.

Cursos alternos:

Valor fuera de rango (3a): el sistema no lo guarda.

UC47 Activar notificaciones push

Actores: Clienta, Cajero y Administrador. **Paquete:** Notificaciones. **Requisitos:** O10. **Pantallas:** Navegador y Service Worker.

Curso básico: El usuario pulsa Activar notificaciones y el Navegador pide permiso. El sistema obtiene la clave pública VAPID y crea la suscripción en el Service Worker. El sistema guarda la PushSuscripcion del usuario. Ante un evento (venta, conflicto, despacho), el sistema envía el aviso y el Service Worker lo muestra aunque la app esté cerrada.

Cursos alternos:

El usuario niega el permiso (1a): la app sigue funcionando sin push.

El endpoint expiró (4a): el sistema borra la suscripción.

Lote 3: Reportes generativos y ventas al crédito

UC48 Generar reporte por texto o voz

Actores: Administrador, Encargado y Servicio de IA. **Paquete:** Reportes. **Requisitos:** R1, R2, S1. **Pantallas:** Pantalla Reporte inteligente, Micrófono y Interfaz Azure OpenAI.

Curso básico: El Administrador escribe o dicta por el Micrófono el reporte que necesita en la Pantalla Reporte inteligente. El sistema normaliza el texto y pide al Servicio de IA una especificación JSON usando el catálogo semántico y los valores reales de la tienda. El sistema valida la especificación contra el catálogo y la compila a SQL con parámetros. El sistema ejecuta la consulta fijando la tienda de la sesión y registra la ReporteEjecucion. El sistema muestra criterios, tabla, totales, barras y la narración del resultado.

Cursos alternos:

El pedido dice en PDF o en Excel (1a): el sistema genera el archivo al terminar (UC49).

El Servicio de IA no responde o devuelve un JSON inválido (2a): el intérprete por reglas arma la especificación.

Se pide un campo inexistente (teléfono, edad) (3a): el sistema avisa y no lo inventa.

UC49 Ajustar y exportar reporte

Actores: Administrador y Encargado. **Paquete:** Reportes. **Requisitos:** R2, R3. **Pantallas:** Pantalla Reporte inteligente y Archivo PDF o Excel.

Curso básico: El Administrador quita un criterio, quita una agrupación o invierte el orden en la Pantalla Reporte inteligente. El sistema vuelve a ejecutar la especificación editada sin pasar por la IA. El Administrador pide Excel o PDF. El sistema genera el Archivo PDF o Excel con encabezado, criterios, totales y la hoja de cómo se generó, y lo descarga.

Cursos alternos:

La edición deja una especificación inválida (2a): el sistema responde 400 y conserva el reporte anterior.

Exportación desde el móvil (4a): el sistema entrega el archivo al menú de compartir del sistema operativo.

UC50 Vender al crédito

Actores: Cajero. **Paquete:** Ventas. **Requisitos:** R2, O8. **Pantallas:** Pantalla Punto de venta.

Curso básico: El Cajero arma el ticket, asocia una clienta y elige Crédito (30 días) en la Pantalla Punto de venta. El sistema descuenta el stock y registra la Venta como en UC07, pero no suma dinero al CajaTurno. El sistema crea la CuentaCobrar con monto, saldo y vencimiento a 30 días.

Cursos alternos:

No hay clienta asociada (1a): el sistema rechaza el crédito.

Sin red (1b): el crédito no está disponible porque exige ver la deuda real.

UC51 Registrar abono de crédito

Actores: Cajero. **Paquete:** Créditos. **Requisitos:** R2, O6. **Pantallas:** Pantalla Cuentas por cobrar.

Curso básico: El Cajero busca la CuentaCobrar de la clienta en la Pantalla Cuentas por cobrar. El Cajero ingresa monto y medio del abono. El sistema descuenta el saldo con una actualización condicional que nunca lo deja negativo. El sistema registra el AbonoCredito y suma el monto al CajaTurno abierto.

Cursos alternos:

El Cajero pulsa dos veces o se corta la red (2a): la misma clave de idempotencia no cobra dos veces.

El abono supera el saldo (3a): el sistema lo rechaza e informa el saldo.

Dos cajas abonan a la vez por el saldo total (3b): una pasa y la otra recibe el rechazo.

Lote 4: Analítica con aprendizaje automático

UC52 Ver dashboard predictivo

Actores: Administrador. **Paquete:** Analítica. **Requisitos:** A1, A2, A3. **Pantallas:** Dashboard de analítica.

Curso básico: El Administrador abre el Dashboard de analítica. El sistema aplica el ModeloPredictivo, entrenado con datos sintéticos, a las ventas de 7 y 28 días de cada variante. El sistema muestra la demanda estimada de los próximos 7 días junto al stock disponible y el error del modelo.

Cursos alternos:

La tienda no tiene ventas en el período (2a): el sistema muestra un estado vacío explicativo.

UC53 Ver segmentación de clientas

Actores: Administrador. **Paquete:** Analítica. **Requisitos:** A1, A2, A3. **Pantallas:** Dashboard de analítica.

Curso básico: El Administrador abre la segmentación en el Dashboard de analítica. El sistema asigna cada clienta con compras en los últimos 90 días a un grupo de k-medias según recencia, frecuencia y monto. El sistema muestra cada SegmentoCliente con su tamaño y su centro, sin exponer identidades.

Cursos alternos:

No hay clientas con compras identificadas (2a): el sistema muestra los grupos vacíos.

Lote 5: Pagos para países hispanohablantes

UC54 Pagar con pasarela internacional

Actores: Clienta y Pasarela de pago. **Paquete:** Pagos. **Requisitos:** P1, P2, P3, P4. **Pantallas:** Pantalla Pago y Interfaz pasarela.

Curso básico: La Clienta elige en la Pantalla Pago un método habilitado para el país de la tienda. El sistema valida el método contra el catálogo del país y registra la TransaccionPasarela pendiente con una clave de idempotencia. La Pasarela de pago, simulada, aprueba o rechaza el cobro. Con el pago aprobado, la Venta continúa su cobro (UC05).

Cursos alternos:

La pasarela rechaza el cobro (3a): la venta no se confirma y se liberan las unidades reservadas.

La confirmación llega dos veces (3b): el sistema la procesa una sola vez.

9.1.7 Revisión de requisitos

La revisión se hizo por lotes, justo antes de pasar cada uno a robustez, con la lista de verificación de ICONIX. El acta resume lo encontrado y lo corregido.

Verificación	Resultado
Cada requisito S, O, R, A y P está asignado a al menos un caso o a una decisión documentada	Sí. S8, P2 y P4 se cubren con capítulos, no con diálogos
Cada caso tiene curso básico y al menos un curso alterno	Sí, en los 54 casos
Los sustantivos de los casos existen en el dominio con el mismo nombre	Sí, tras agregar Cuenta, Plan, Suscripcion, PagoSuscripcion, Dispositivo, OperacionSync, PushSuscripcion, CuentaCobrar y AbonoCredito
Cada caso nombra su pantalla	Sí; se nombraron fronteras que estaban implícitas, como Cola local y Selector de tienda
Los casos heredados contemplan la tienda y la falta de red	Se agregaron alternos sin red a UC01, UC07, UC08, UC09 y UC27, y el control de límites del plan a UC09,

Verificación	Resultado
No hay requisitos del tipo "el sistema deberá" dentro del texto	UC23 y UC24 Se movieron a la lista de requisitos

9.2 Análisis y diseño preliminar

Para cada caso de uso se dibujó su diagrama de robustez con los tres estereotipos de ICONIX: fronteras (pantallas e interfaces externas), controles (funciones lógicas, que son verbos) y entidades (clases del dominio). Las conexiones respetan las reglas del proceso: el actor solo habla con fronteras, las fronteras solo con controles y las entidades solo con controles. Cada entidad de robustez está vinculada por «trace» a su clase del dominio.

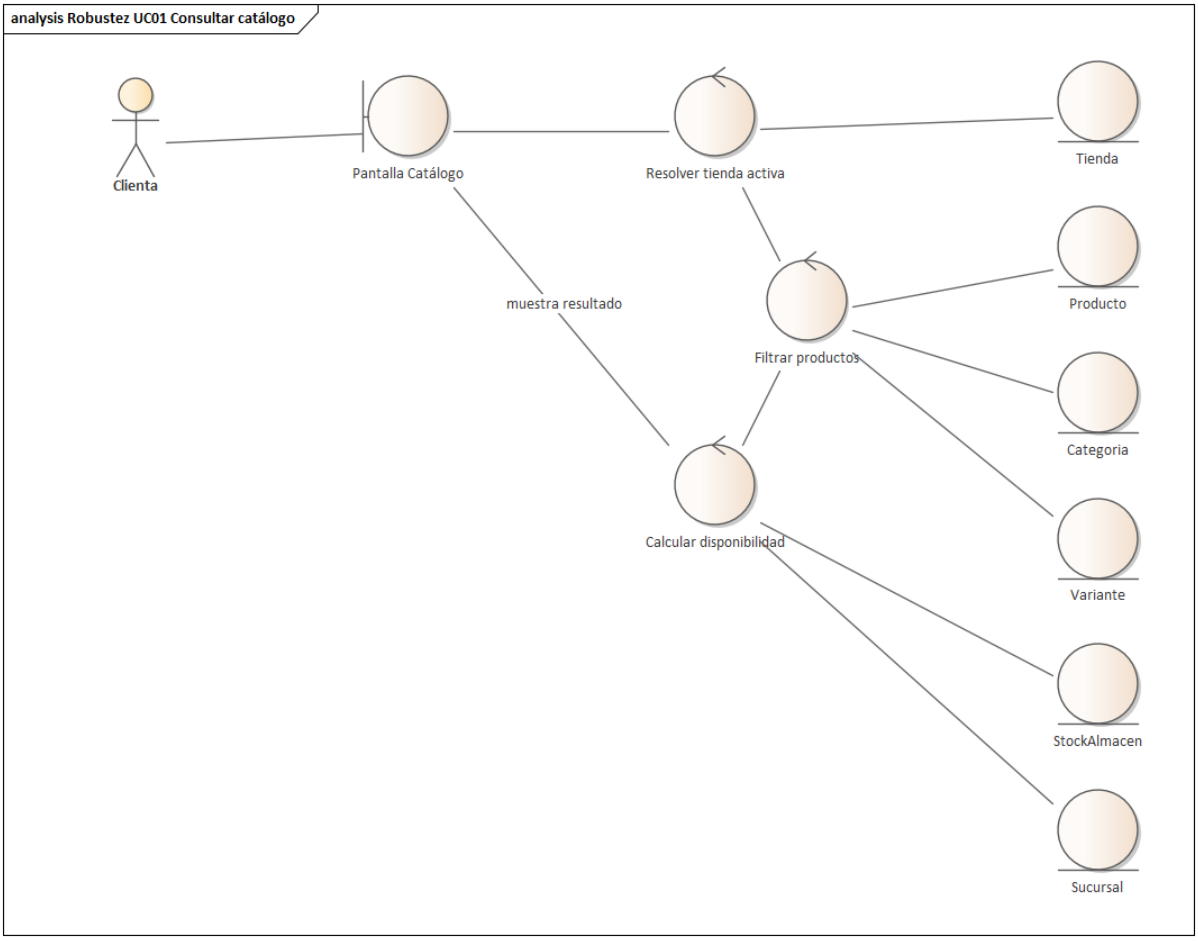
Cada diagrama se lee de izquierda a derecha siguiendo el curso básico: actor, pantalla, cadena de controles en el orden del texto y, a la derecha, las entidades que cada control lee o modifica. Cuando un control habla con un sistema externo, la frontera de ese sistema y su actor secundario aparecen en el extremo derecho.

Lote 0: Casos heredados del primer ciclo, revisados con robustez y secuencia

Robustez UC01 Consultar catálogo

Figura 9.2.1

Robustez de UC01 Consultar catálogo

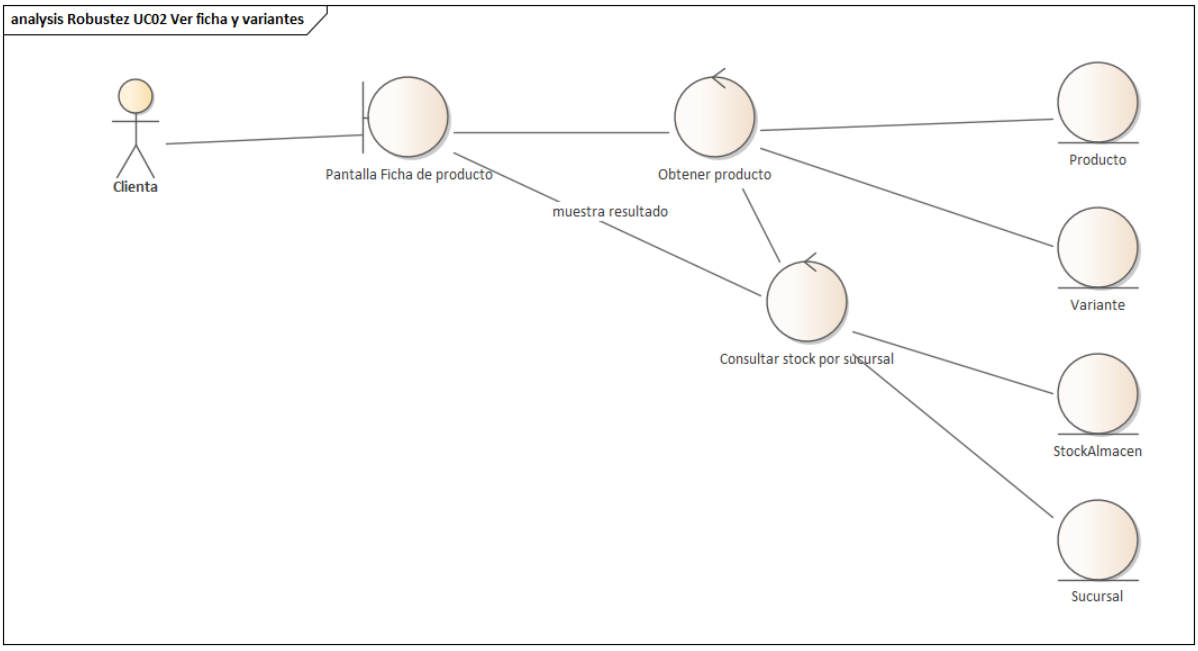


Control	Entidades o fronteras que usa
Resolver tienda activa	Tienda
Filtrar productos	Producto, Categoria y Variante
Calcular disponibilidad	StockAlmacen y Sucursal

Robustez UC02 Ver ficha y variantes

Figura 9.2.2

Robustez de UC02 Ver ficha y variantes

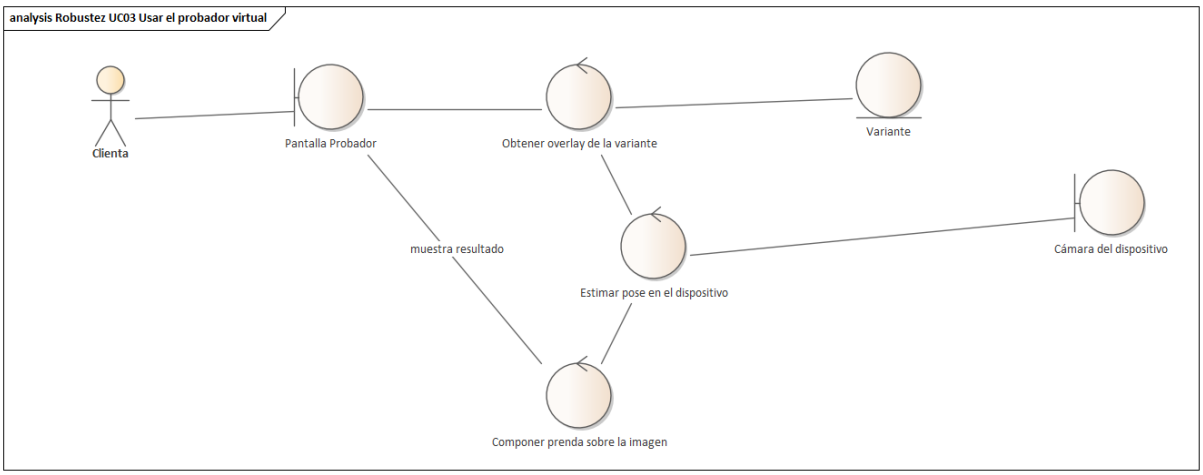


Control	Entidades o fronteras que usa
Obtener producto	Producto y Variante
Consultar stock por sucursal	StockAlmacen y Sucursal

Robustez UC03 Usar el probador virtual

Figura 9.2.3

Robustez de UC03 Usar el probador virtual

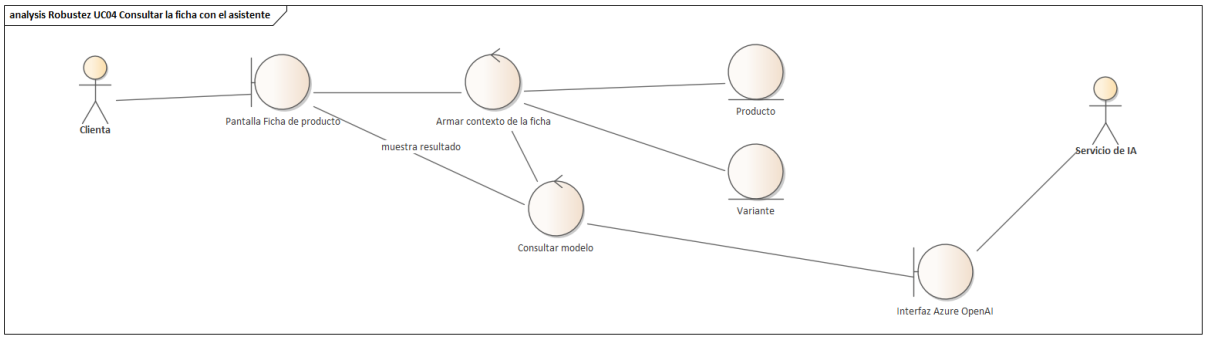


Control	Entidades o fronteras que usa
Obtener overlay de la variante	Variante
Estimar pose en el dispositivo	Cámara del dispositivo
Componer prenda sobre la imagen	Cálculo interno

Robustez UC04 Consultar la ficha con el asistente

Figura 9.2.4

Robustez de UC04 Consultar la ficha con el asistente

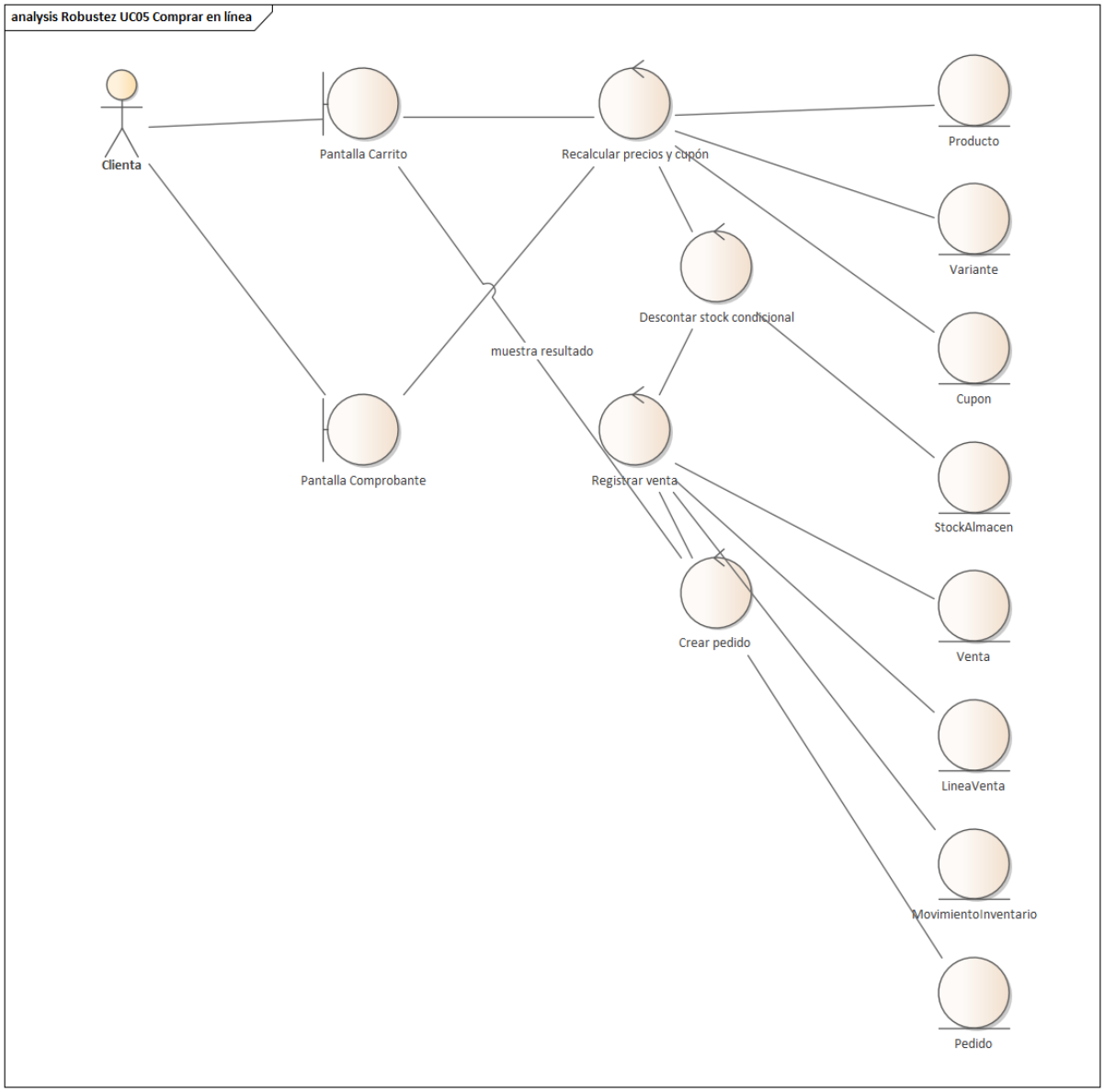


Control	Entidades o fronteras que usa
Armar contexto de la ficha	Producto y Variante
Consultar modelo	Interfaz Azure OpenAI

Robustez UC05 Comprar en línea

Figura 9.2.5

Robustez de UC05 Comprar en línea

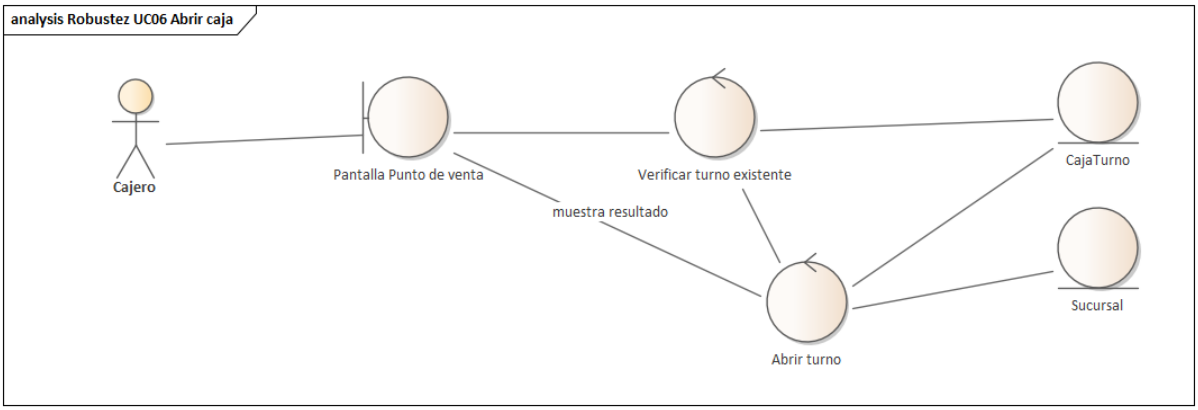


Control	Entidades o fronteras que usa
Recalcular precios y cupón	Producto, Variante y Cupon
Descontar stock condicional	StockAlmacen
Registrar venta	Venta, LineaVenta y MovimientoInventario
Crear pedido	Pedido

Robustez UC06 Abrir caja

Figura 9.2.6

Robustez de UC06 Abrir caja

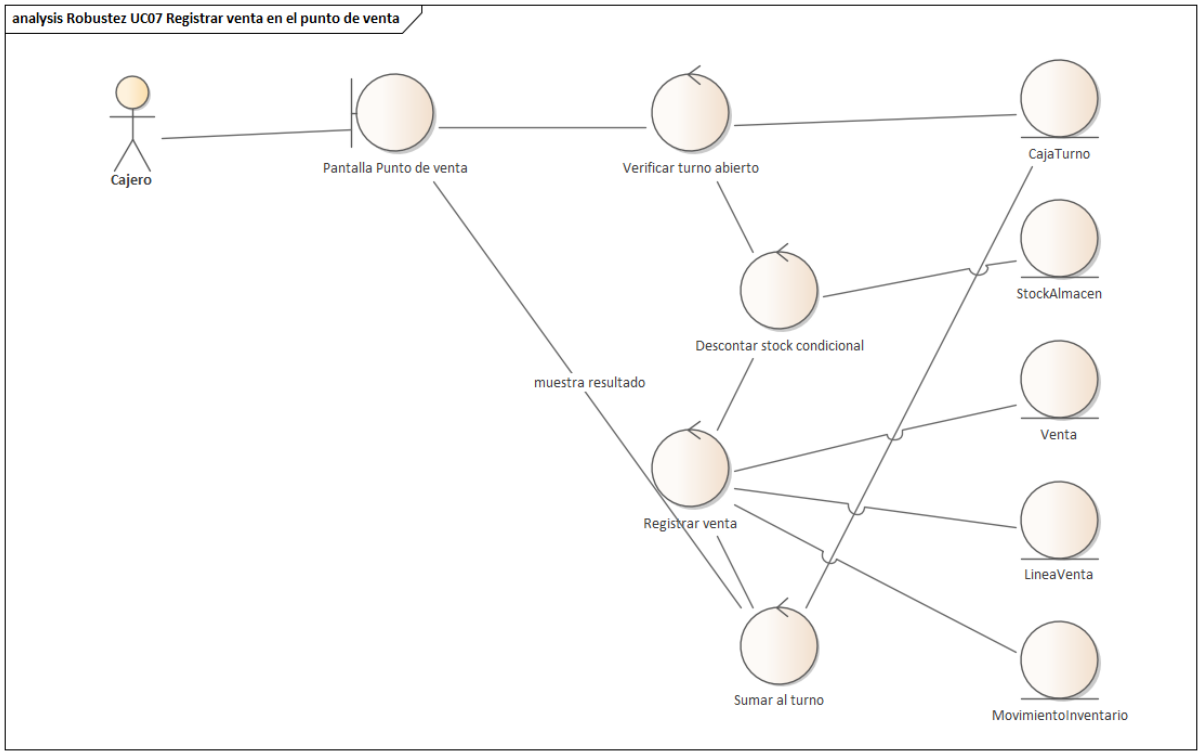


Control	Entidades o fronteras que usa
Verificar turno existente	CajaTurno
Abrir turno	CajaTurno y Sucursal

Robustez UC07 Registrar venta en el punto de venta

Figura 9.2.7

Robustez de UC07 Registrar venta en el punto de venta



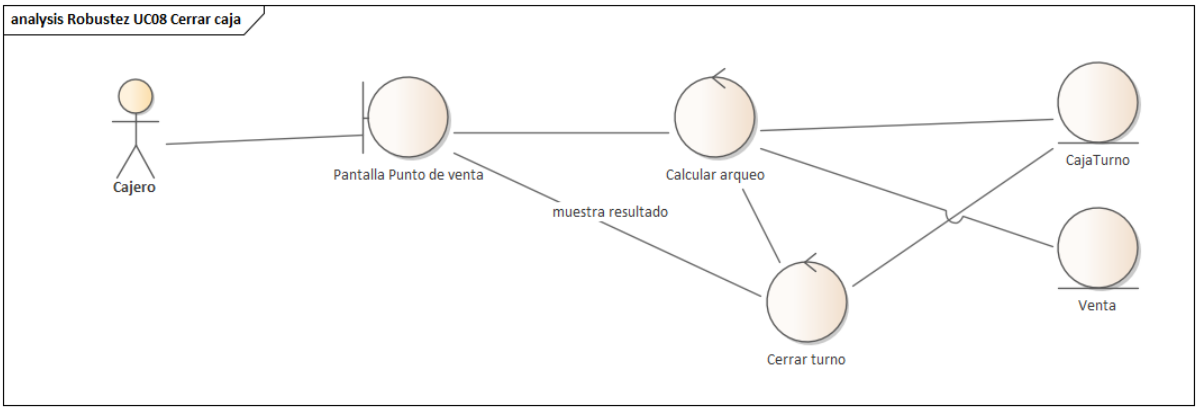
Control	Entidades o fronteras que usa
Verificar turno abierto	CajaTurno
Descontar stock condicional	StockAlmacen
Registrar venta	Venta, LineaVenta y MovimientoInventario
Sumar al turno	CajaTurno

Hallazgo de la robustez. La robustez mostró que el cobro toca cuatro entidades y que el turno de caja debe verificarse antes del stock; el orden de los controladores quedó fijado así en el texto.

Robustez UC08 Cerrar caja

Figura 9.2.8

Robustez de UC08 Cerrar caja



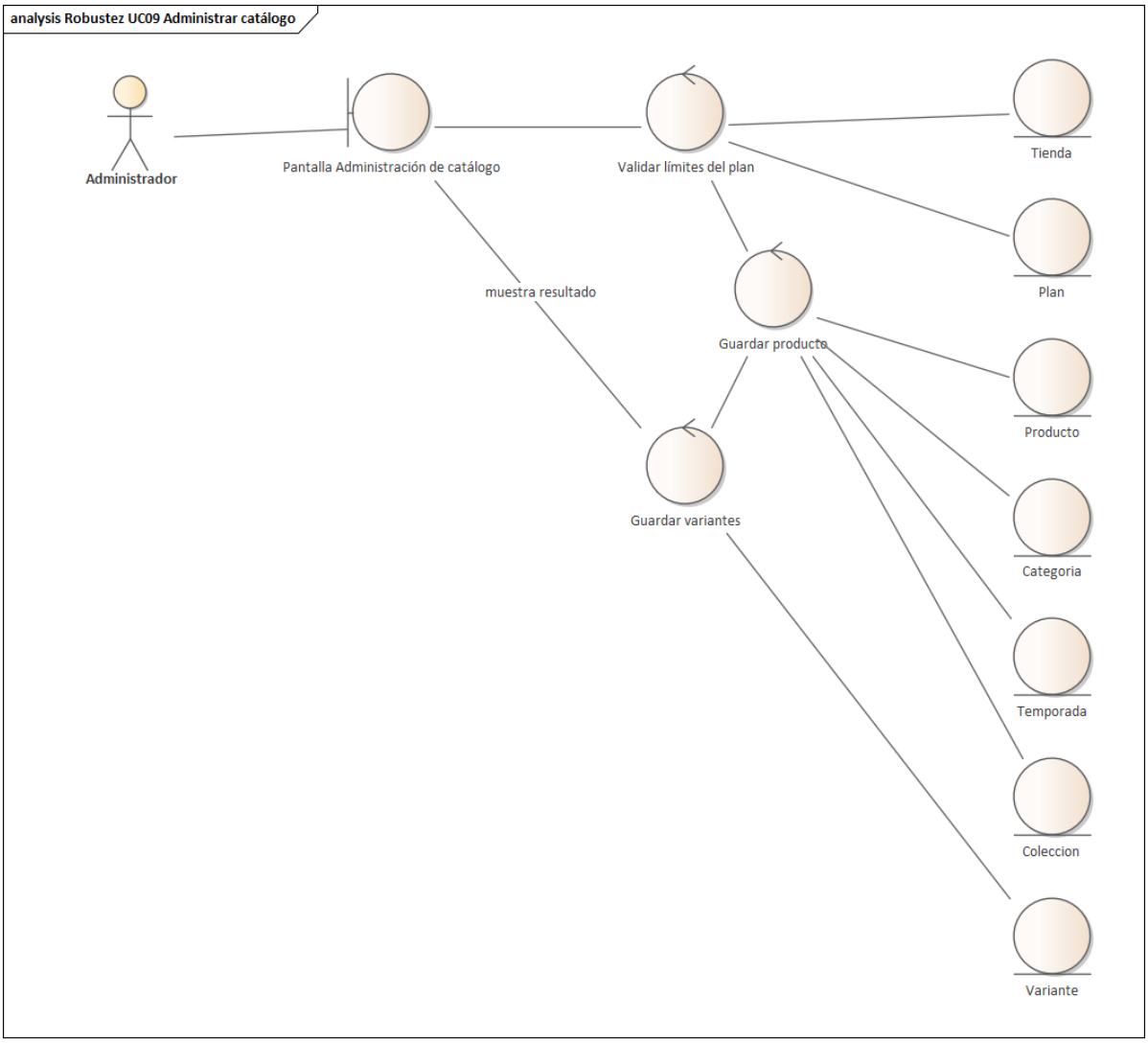
Control	Entidades o fronteras que usa
Calcular arqueo	CajaTurno y Venta
Cerrar turno	CajaTurno

Hallazgo de la robustez. Al dibujar el cierre apareció la dependencia con la cola offline: un turno no debe cerrarse con operaciones pendientes. Se agregó el alterno 1a.

Robustez UC09 Administrar catálogo

Figura 9.2.9

Robustez de UC09 Administrar catálogo

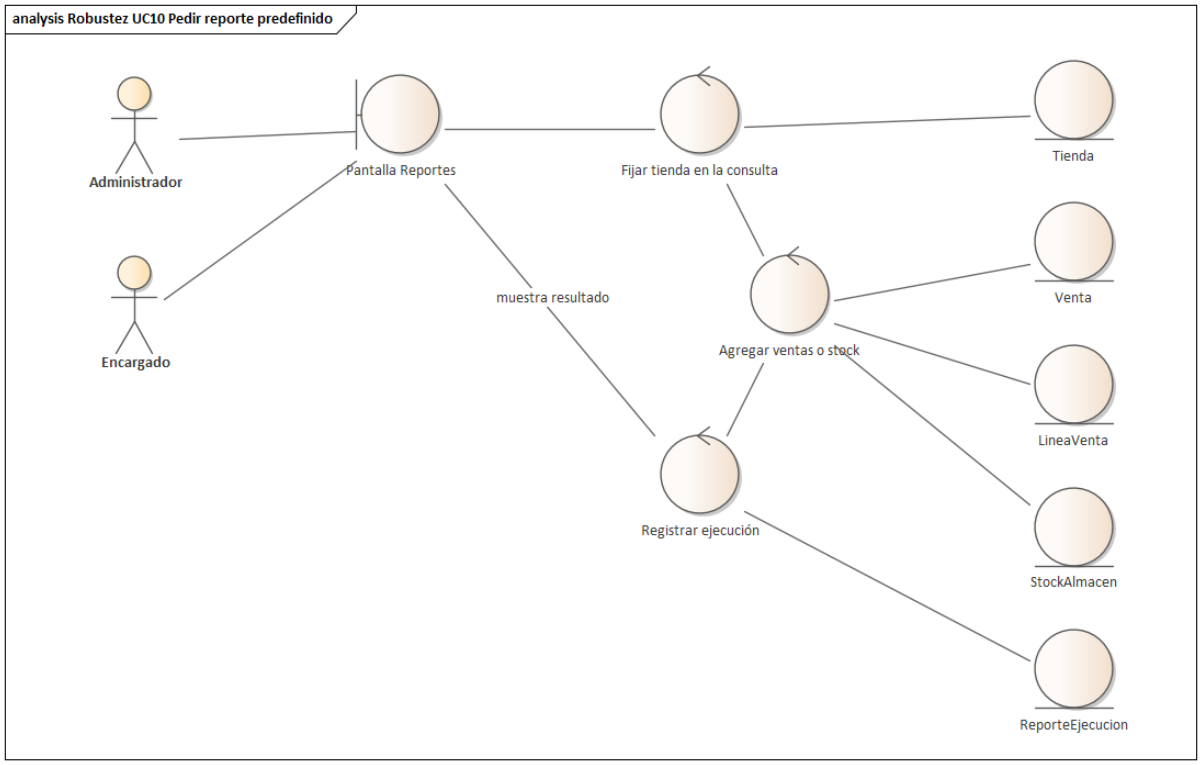


Control	Entidades o fronteras que usa
Validar límites del plan	Tienda y Plan
Guardar producto	Producto, Categoria, Temporada y Coleccion
Guardar variantes	Variante

Robustez UC10 Pedir reporte predefinido

Figura 9.2.10

Robustez de UC10 Pedir reporte predefinido

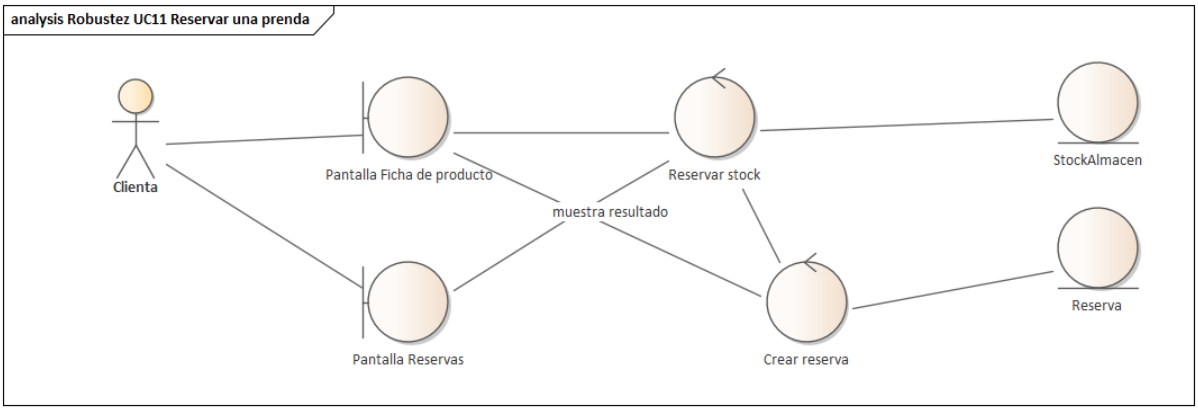


Control	Entidades o fronteras que usa
Fijar tienda en la consulta	Tienda
Agregar ventas o stock	Venta, LineaVenta y StockAlmacen
Registrar ejecución	ReporteEjecucion

Robustez UC11 Reservar una prenda

Figura 9.2.11

Robustez de UC11 Reservar una prenda

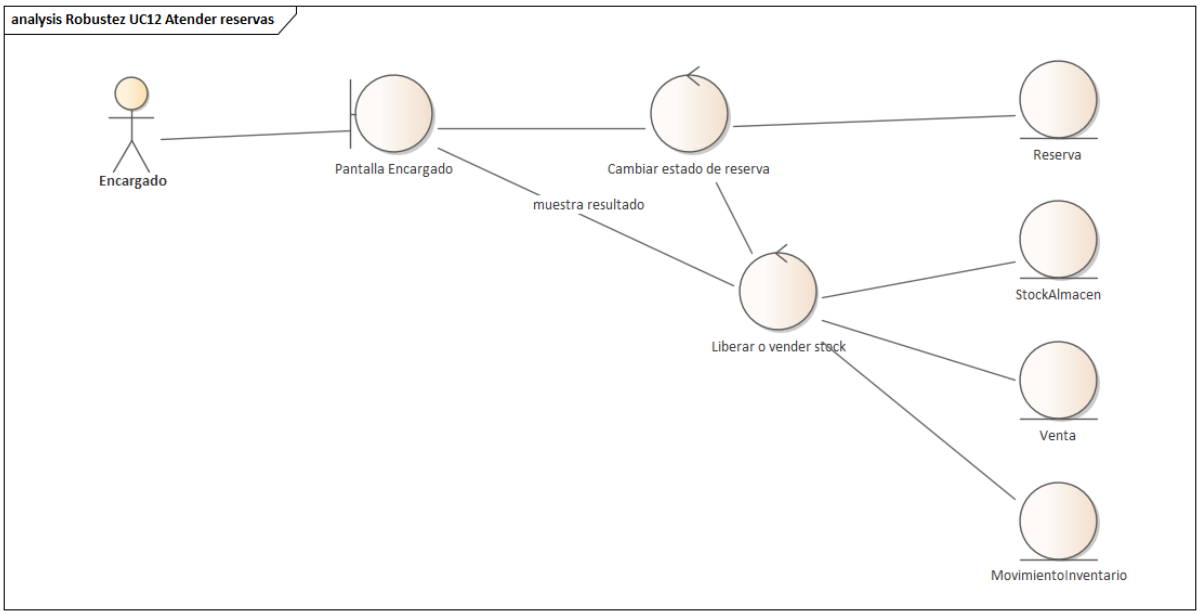


Control	Entidades o fronteras que usa
Reservar stock	StockAlmacen
Crear reserva	Reserva

Robustez UC12 Atender reservas

Figura 9.2.12

Robustez de UC12 Atender reservas



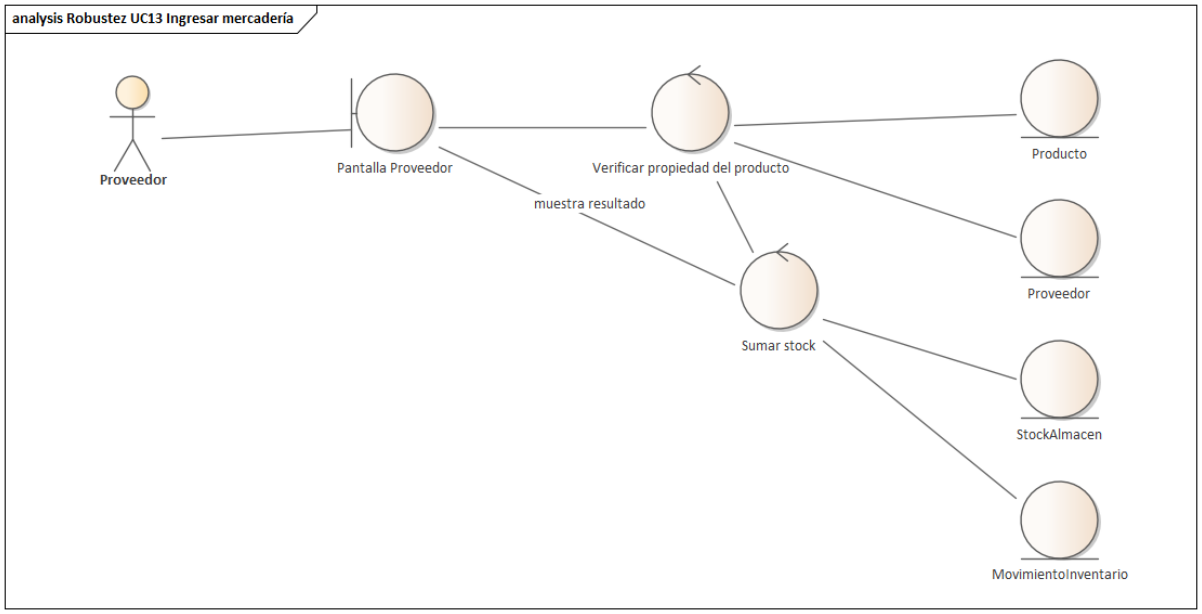
Control	Entidades o fronteras que usa

Control	Entidades o fronteras que usa
Cambiar estado de reserva	Reserva
Liberar o vender stock	StockAlmacen, Venta y MovimientoInventario

Robustez UC13 Ingresar mercadería

Figura 9.2.13

Robustez de UC13 Ingresar mercadería

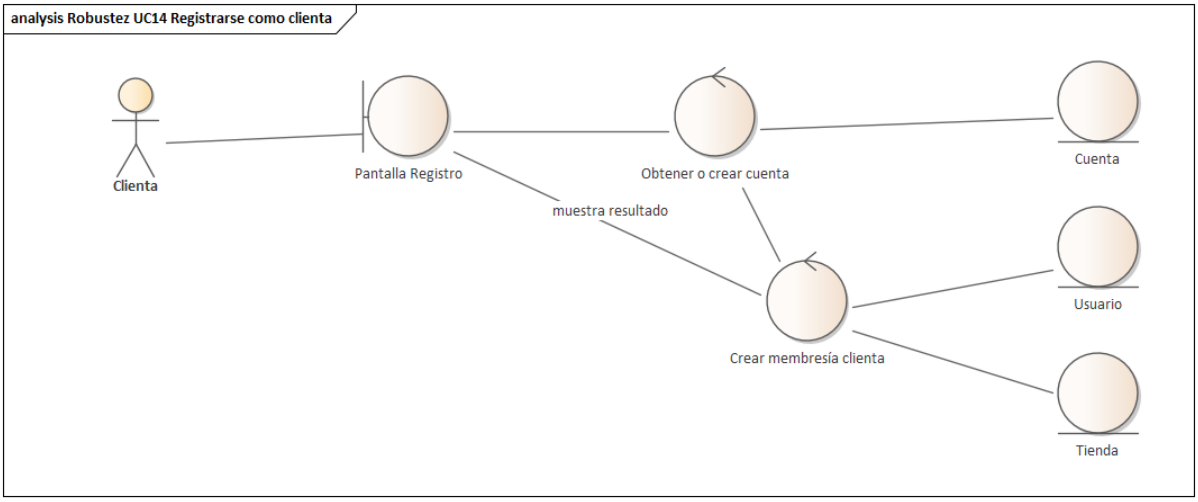


Control	Entidades o fronteras que usa
Verificar propiedad del producto	Producto y Proveedor
Sumar stock	StockAlmacen y MovimientoInventario

Robustez UC14 Registrarse como clienta

Figura 9.2.14

Robustez de UC14 Registrarse como clienta

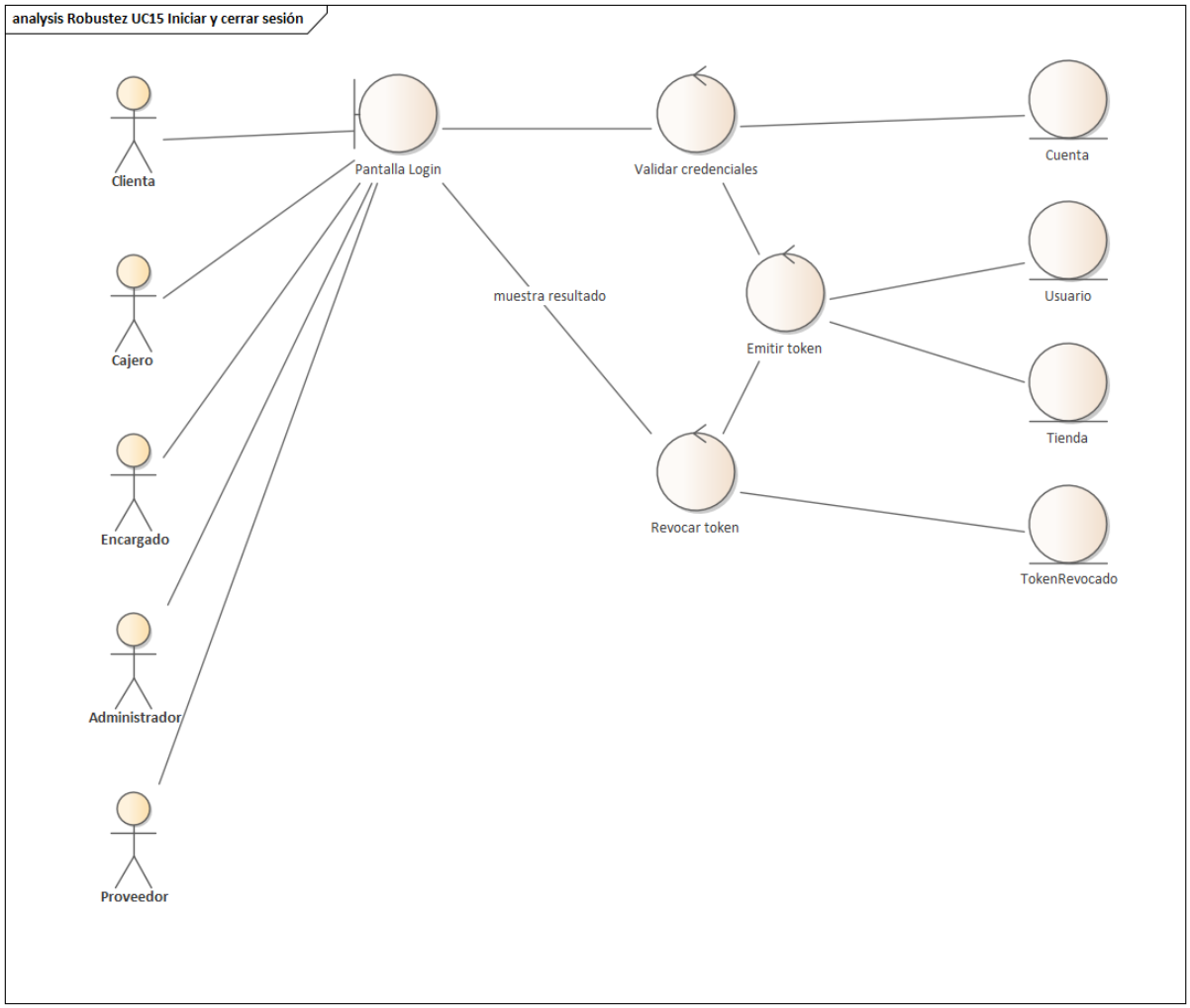


Control	Entidades o fronteras que usa
Obtener o crear cuenta	Cuenta
Crear membresía clienta	Usuario y Tienda

Robustez UC15 Iniciar y cerrar sesión

Figura 9.2.15

Robustez de UC15 Iniciar y cerrar sesión



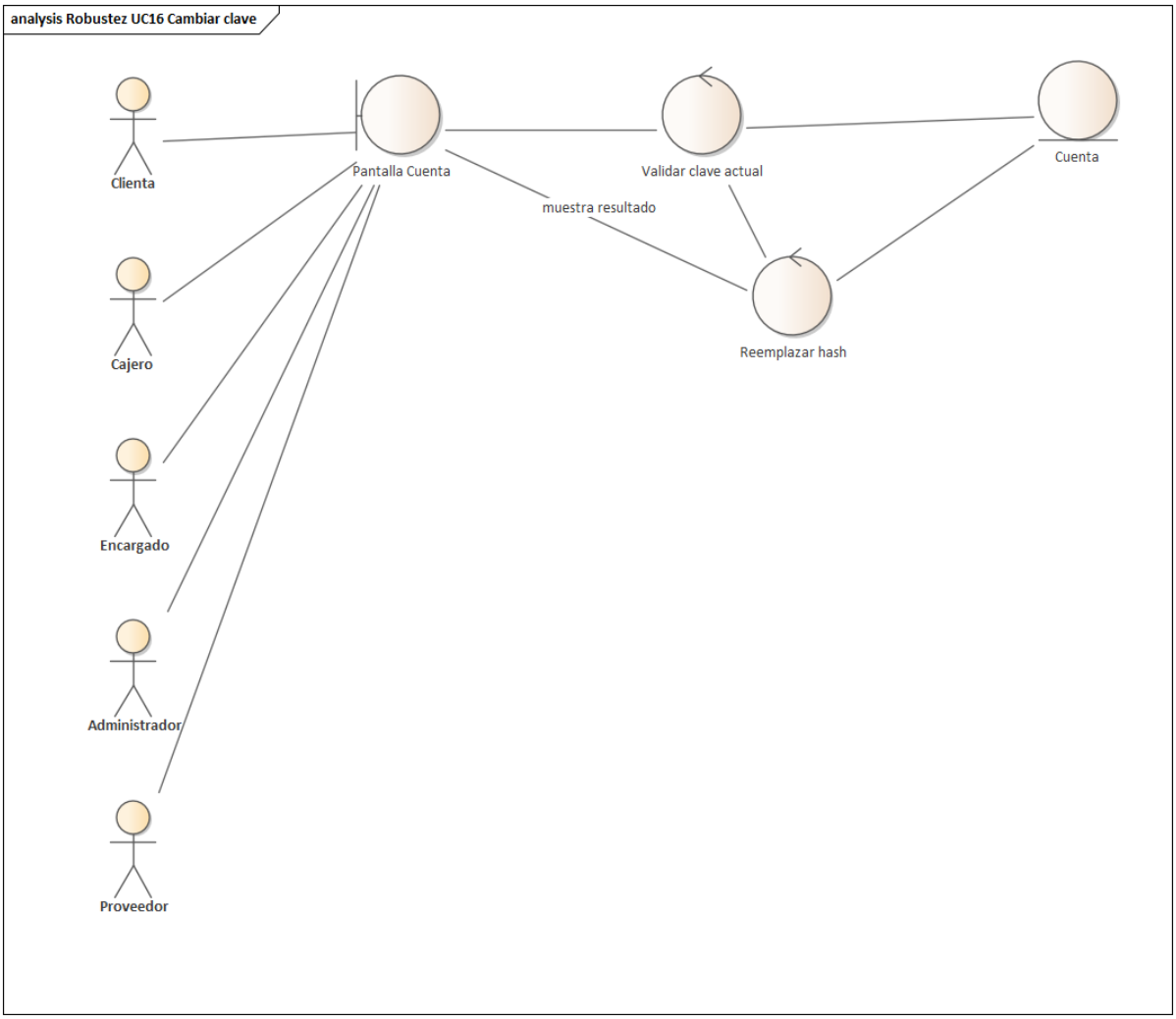
Control	Entidades o fronteras que usa
Validar credenciales	Cuenta
Emitir token	Usuario y Tienda
Revocar token	TokenRevocado

Hallazgo de la robustez. El inicio de sesión necesitaba dos entidades distintas: Cuenta (identidad global) y Usuario (membresía en una tienda). El dominio se separó en consecuencia.

Robustez UC16 Cambiar clave

Figura 9.2.16

Robustez de UC16 Cambiar clave

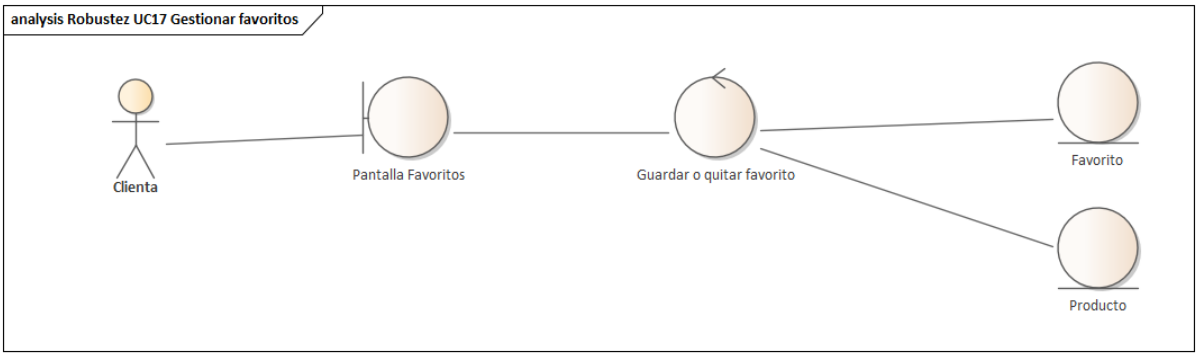


Control	Entidades o fronteras que usa
Validar clave actual	Cuenta
Reemplazar hash	Cuenta

Robustez UC17 Gestionar favoritos

Figura 9.2.17

Robustez de UC17 Gestionar favoritos

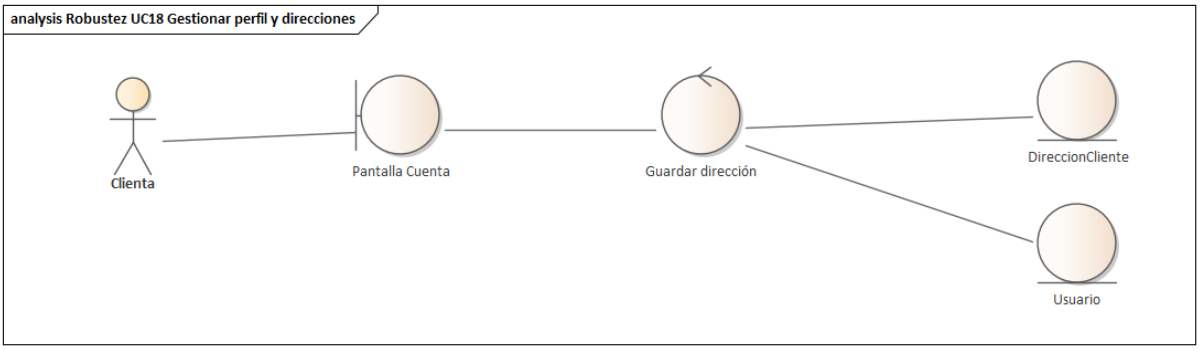


Control	Entidades o fronteras que usa
Guardar o quitar favorito	Favorito y Producto

Robustez UC18 Gestionar perfil y direcciones

Figura 9.2.18

Robustez de UC18 Gestionar perfil y direcciones

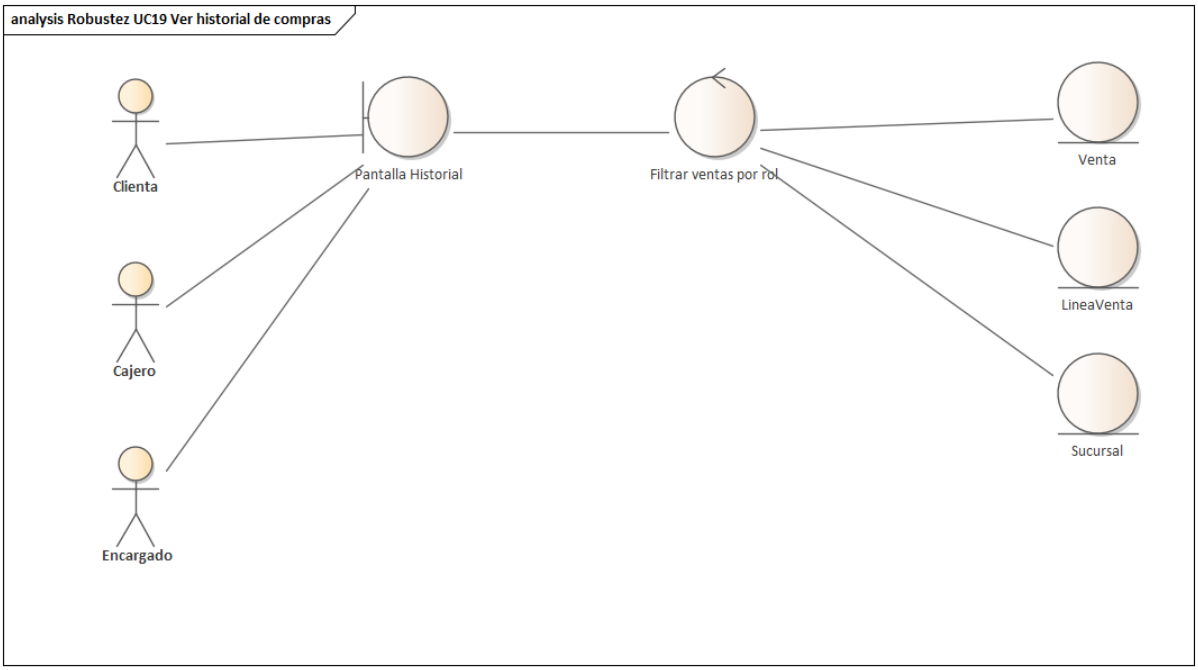


Control	Entidades o fronteras que usa
Guardar dirección	DireccionCliente y Usuario

Robustez UC19 Ver historial de compras

Figura 9.2.19

Robustez de UC19 Ver historial de compras

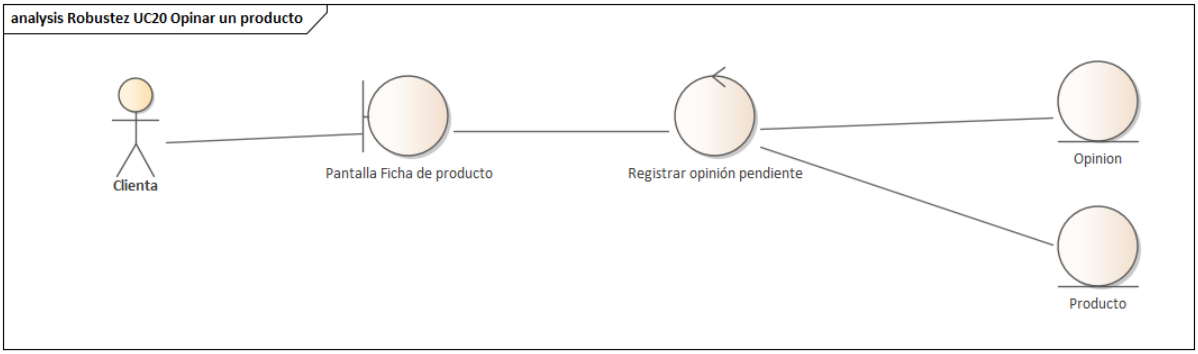


Control	Entidades o fronteras que usa
Filtrar ventas por rol	Venta, LineaVenta y Sucursal

Robustez UC20 Opinar un producto

Figura 9.2.20

Robustez de UC20 Opinar un producto

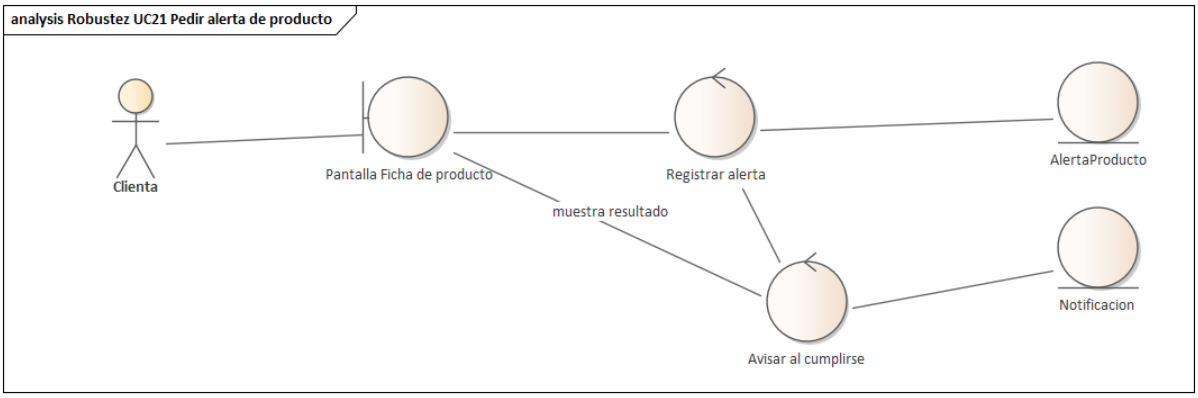


Control	Entidades o fronteras que usa
Registrar opinión pendiente	Opinion y Producto

Robustez UC21 Pedir alerta de producto

Figura 9.2.21

Robustez de UC21 Pedir alerta de producto

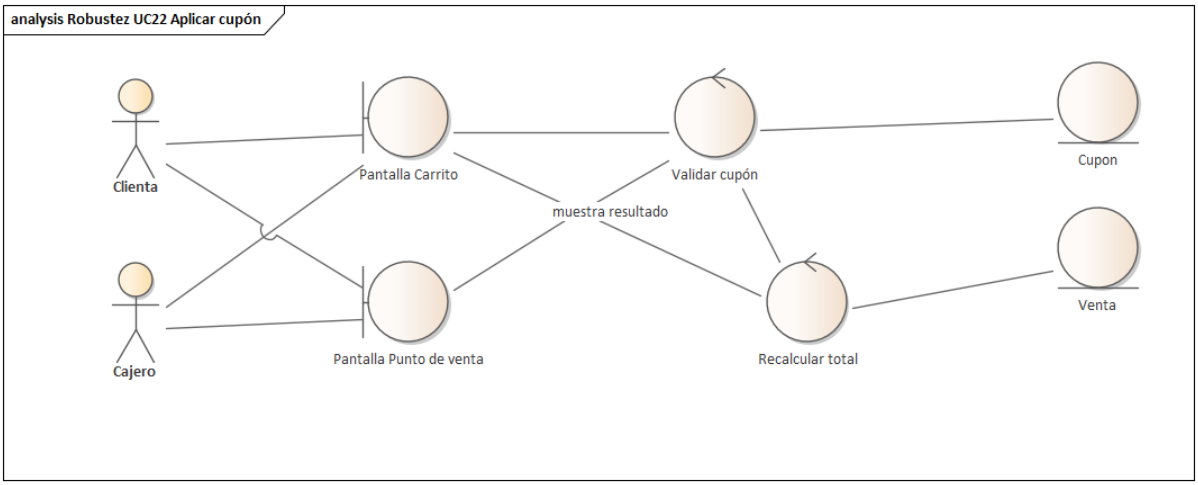


Control	Entidades o fronteras que usa
Registrar alerta	AlertaProducto
Avisar al cumplirse	Notificacion

Robustez UC22 Aplicar cupón

Figura 9.2.22

Robustez de UC22 Aplicar cupón

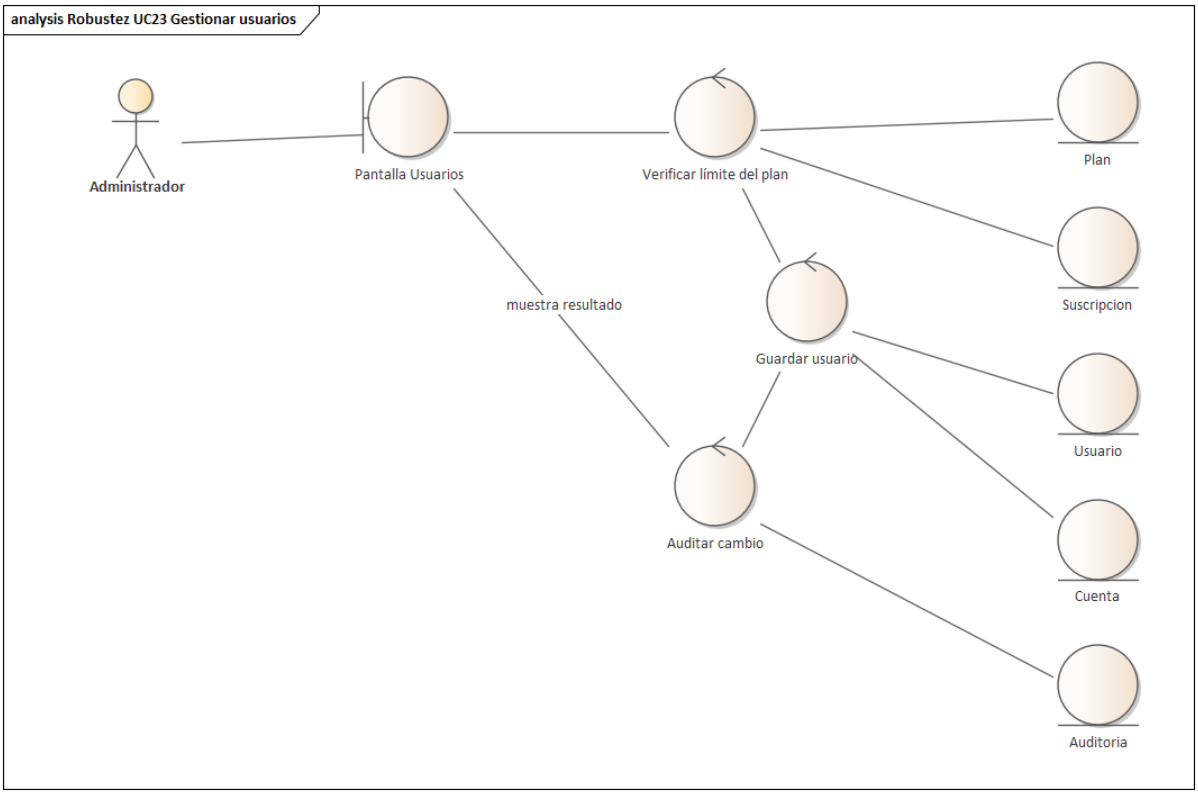


Control	Entidades o fronteras que usa
Validar cupón	Cupon
Recalcular total	Venta

Robustez UC23 Gestionar usuarios

Figura 9.2.23

Robustez de UC23 Gestionar usuarios



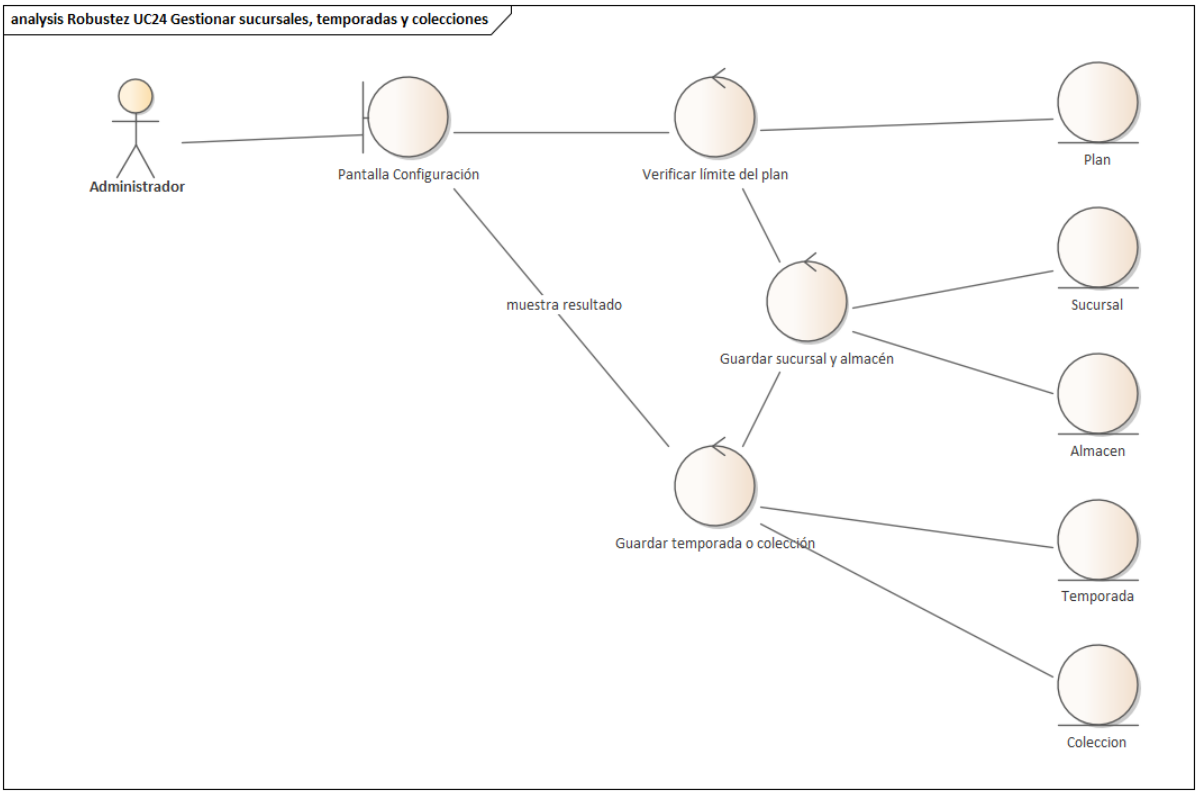
Control	Entidades o fronteras que usa
Verificar límite del plan	Plan y Suscripcion
Guardar usuario	Usuario y Cuenta
Auditar cambio	Auditoria

Hallazgo de la robustez. El alta de usuarios no tenía control de límites. La robustez agregó el control Verificar límite del plan y las entidades Plan y Suscripcion.

Robustez UC24 Gestionar sucursales, temporadas y colecciones

Figura 9.2.24

Robustez de UC24 Gestionar sucursales, temporadas y colecciones

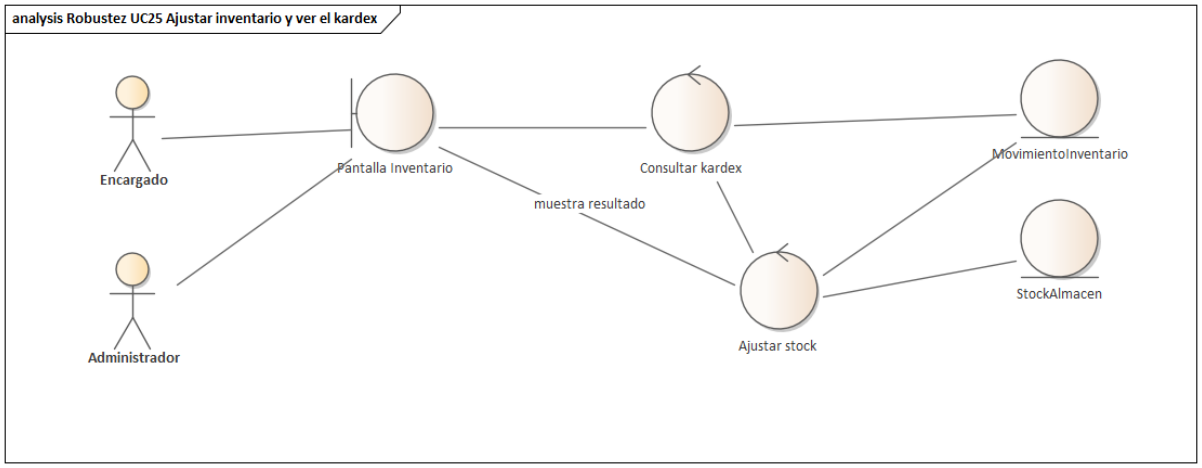


Control	Entidades o fronteras que usa
Verificar límite del plan	Plan
Guardar sucursal y almacén	Sucursal y Almacen
Guardar temporada o colección	Temporada y Coleccion

Robustez UC25 Ajustar inventario y ver el kardex

Figura 9.2.25

Robustez de UC25 Ajustar inventario y ver el kardex

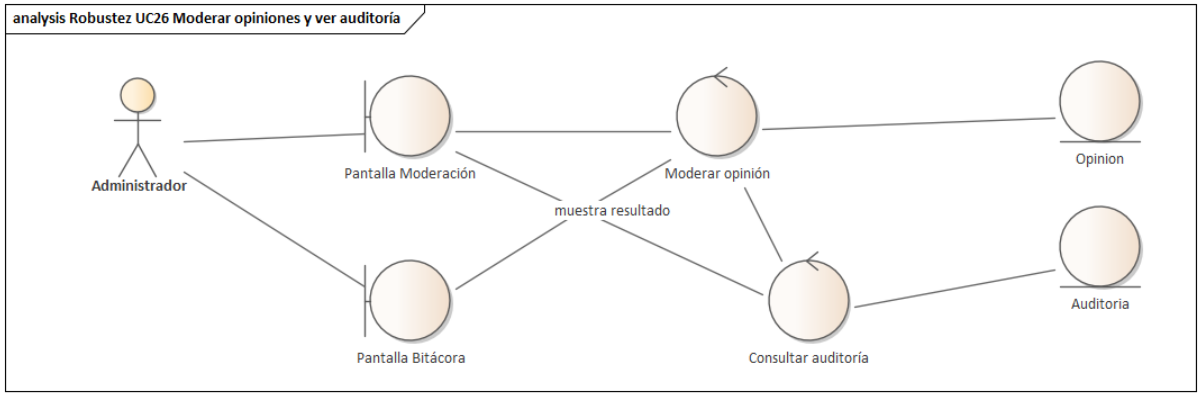


Control	Entidades o fronteras que usa
Consultar kardex	MovimientoInventario
Ajustar stock	StockAlmacen y MovimientoInventario

Robustez UC26 Moderar opiniones y ver auditoría

Figura 9.2.26

Robustez de UC26 Moderar opiniones y ver auditoría

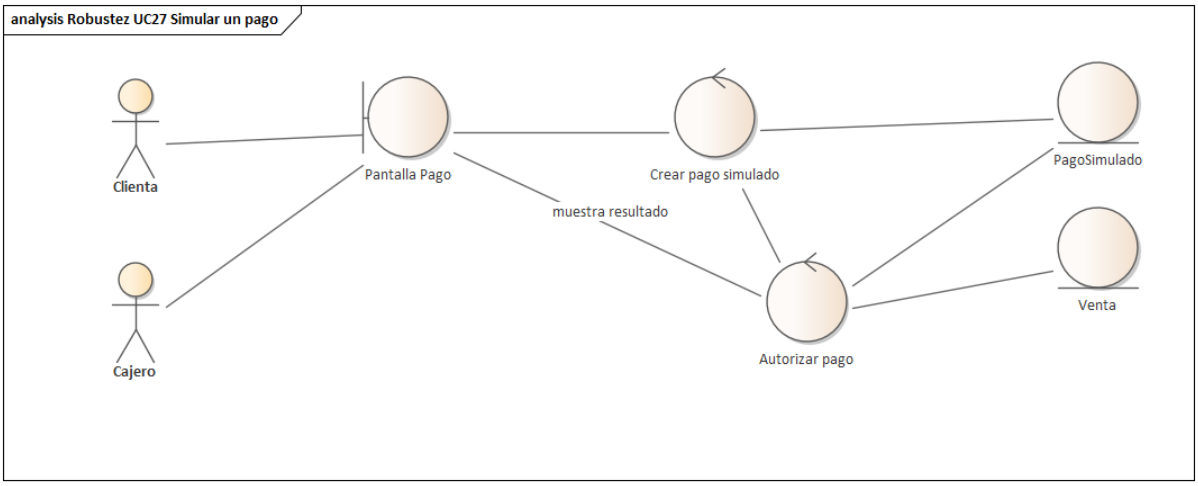


Control	Entidades o fronteras que usa
Moderar opinión	Opinion
Consultar auditoría	Auditoria

Robustez UC27 Simular un pago

Figura 9.2.27

Robustez de UC27 Simular un pago

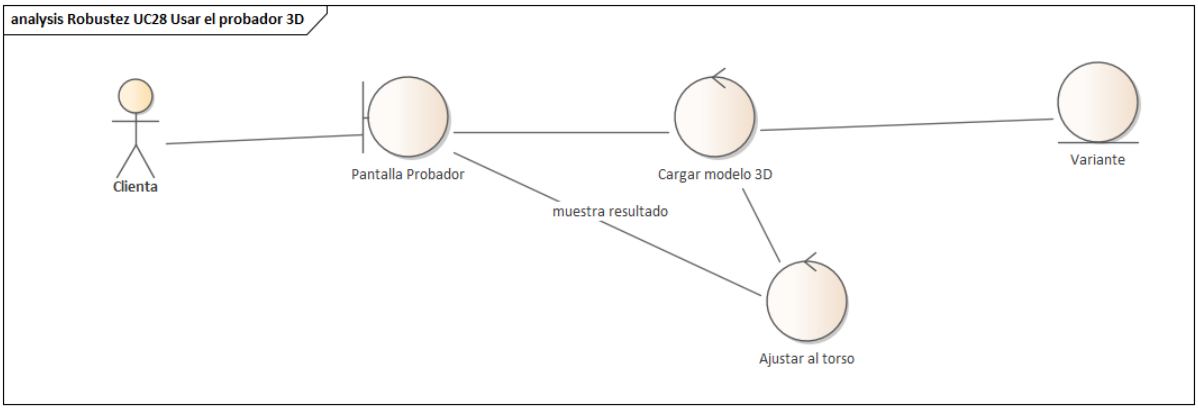


Control	Entidades o fronteras que usa
Crear pago simulado	PagoSimulado
Autorizar pago	PagoSimulado y Venta

Robustez UC28 Usar el probador 3D

Figura 9.2.28

Robustez de UC28 Usar el probador 3D

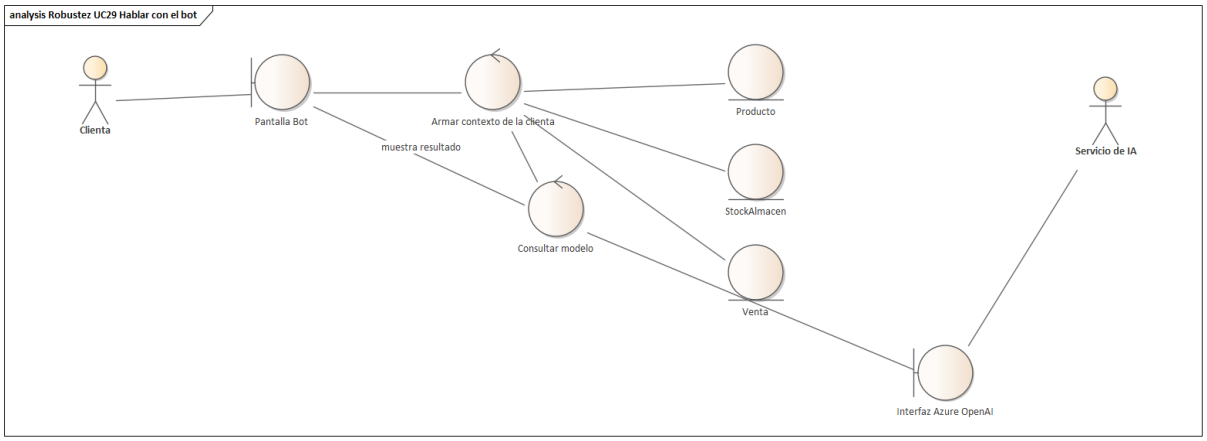


Control	Entidades o fronteras que usa
Cargar modelo 3D	Variante
Ajustar al torso	Cálculo interno

Robustez UC29 Hablar con el bot

Figura 9.2.29

Robustez de UC29 Hablar con el bot

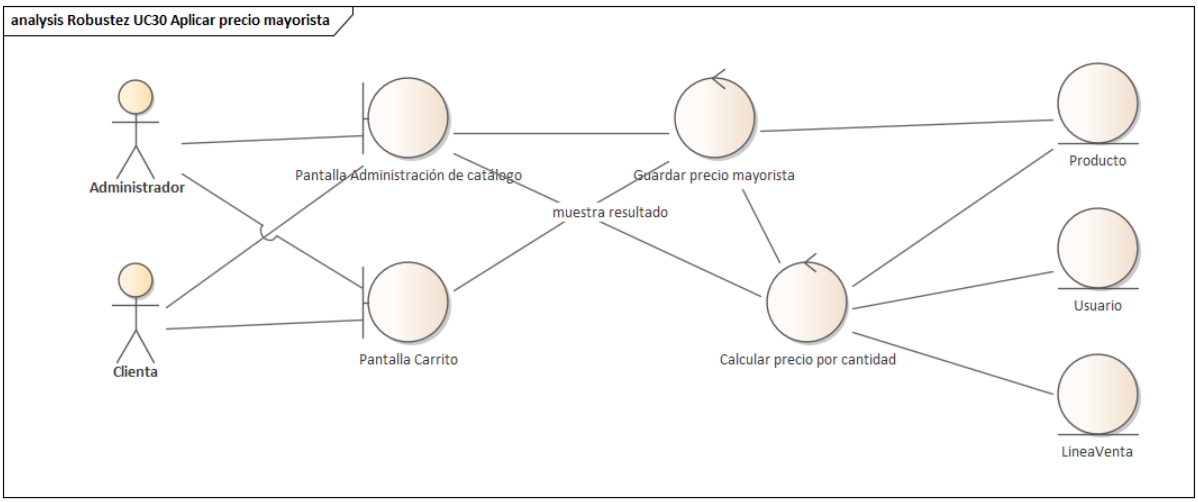


Control	Entidades o fronteras que usa
Armar contexto de la clienta	Producto, StockAlmacen y Venta
Consultar modelo	Interfaz Azure OpenAI

Robustez UC30 Aplicar precio mayorista

Figura 9.2.30

Robustez de UC30 Aplicar precio mayorista

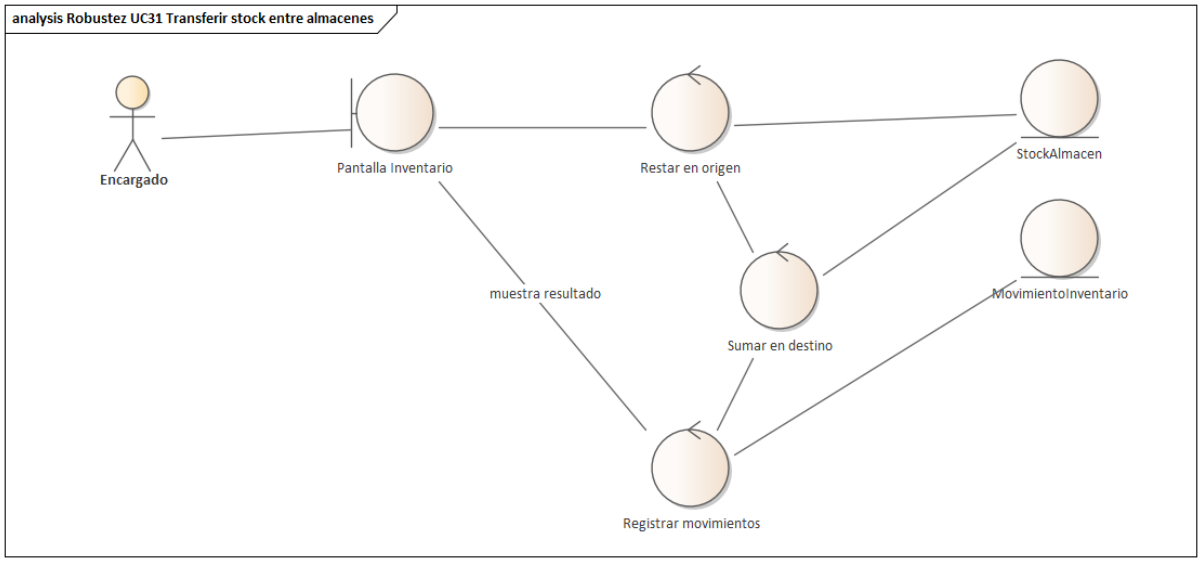


Control	Entidades o fronteras que usa
Guardar precio mayorista	Producto
Calcular precio por cantidad	Producto, Usuario y LineaVenta

Robustez UC31 Transferir stock entre almacenes

Figura 9.2.31

Robustez de UC31 Transferir stock entre almacenes

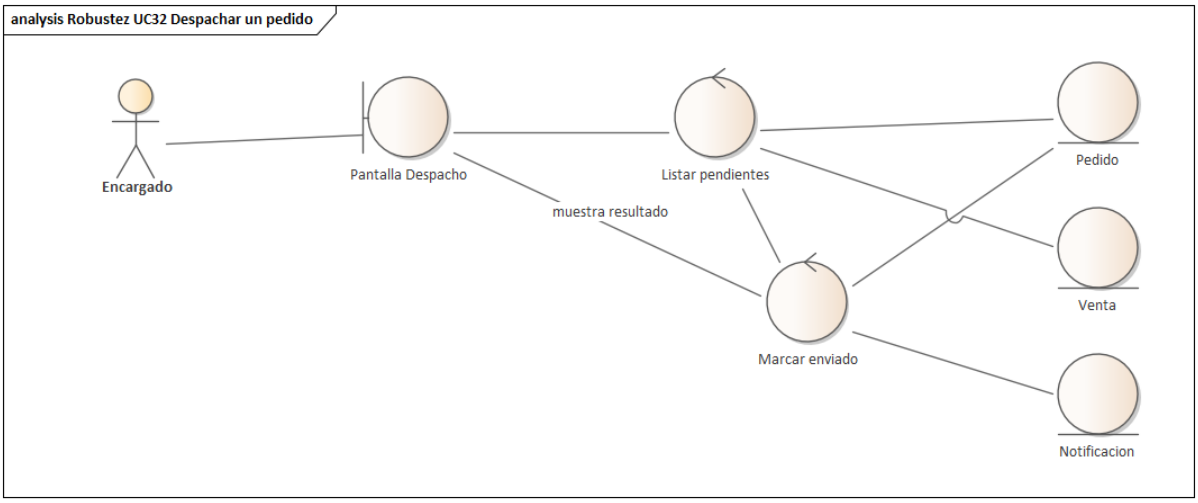


Control	Entidades o fronteras que usa
Restar en origen	StockAlmacen
Sumar en destino	StockAlmacen
Registrar movimientos	MovimientoInventario

Robustez UC32 Despachar un pedido

Figura 9.2.32

Robustez de UC32 Despachar un pedido

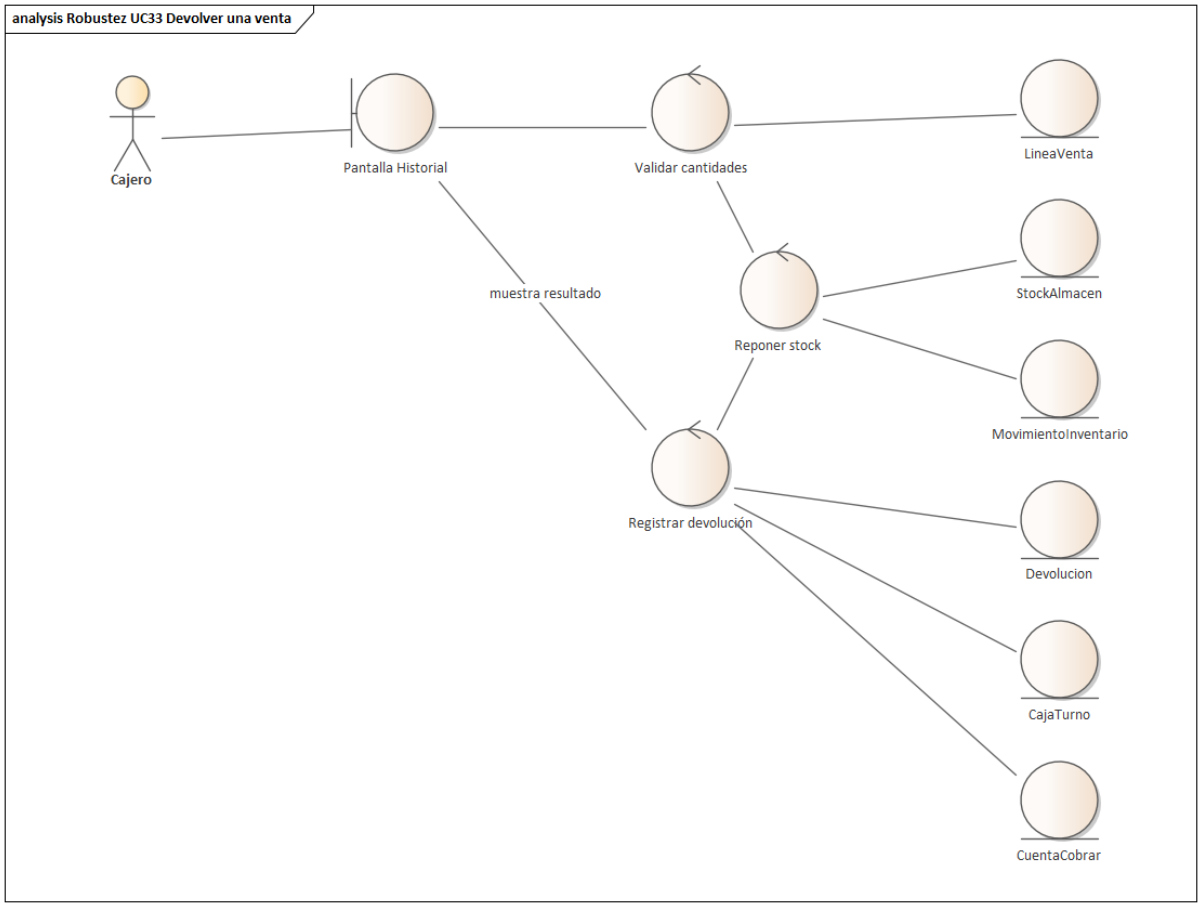


Control	Entidades o fronteras que usa
Listar pendientes	Pedido y Venta
Marcar enviado	Pedido y Notificacion

Robustez UC33 Devolver una venta

Figura 9.2.33

Robustez de UC33 Devolver una venta



Control	Entidades o fronteras que usa
Validar cantidades	LineaVenta
Reponer stock	StockAlmacen y MovimientoInventario
Registrar devolución	Devolucion, CajaTurno y CuentaCobrar

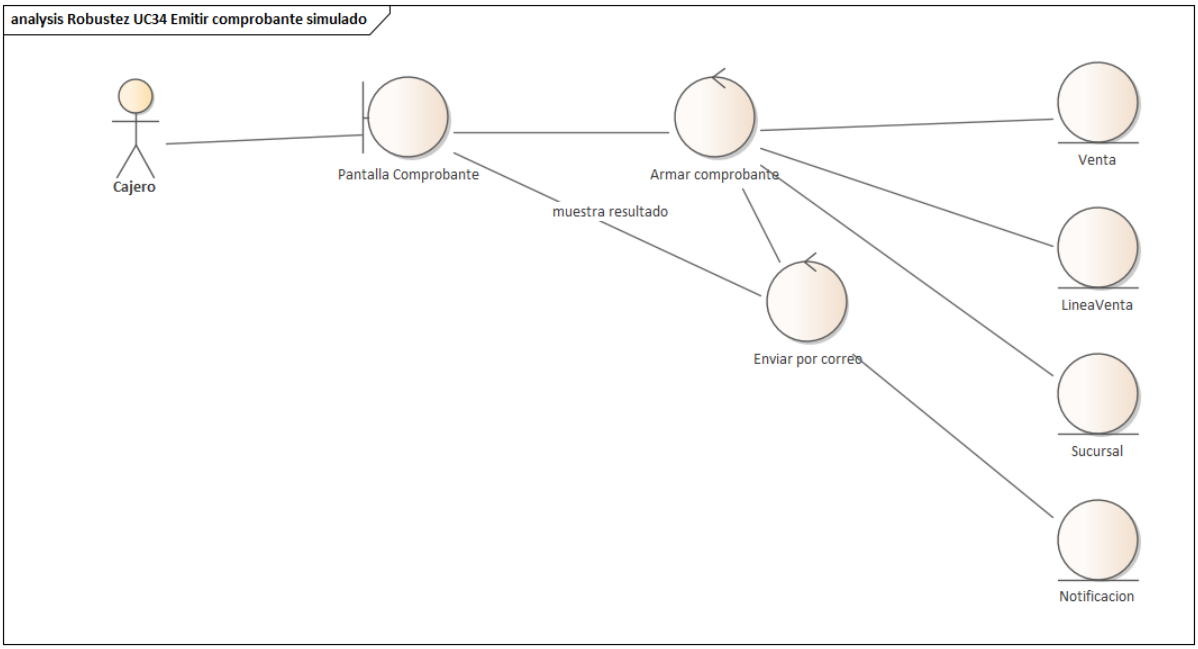
Hallazgo de la robustez. Al conectar la devolución con las entidades apareció

CuentaCobrar: devolver una venta al crédito debe reducir la deuda y no sacar efectivo de la caja.

Robustez UC34 Emitir comprobante simulado

Figura 9.2.34

Robustez de UC34 Emitir comprobante simulado

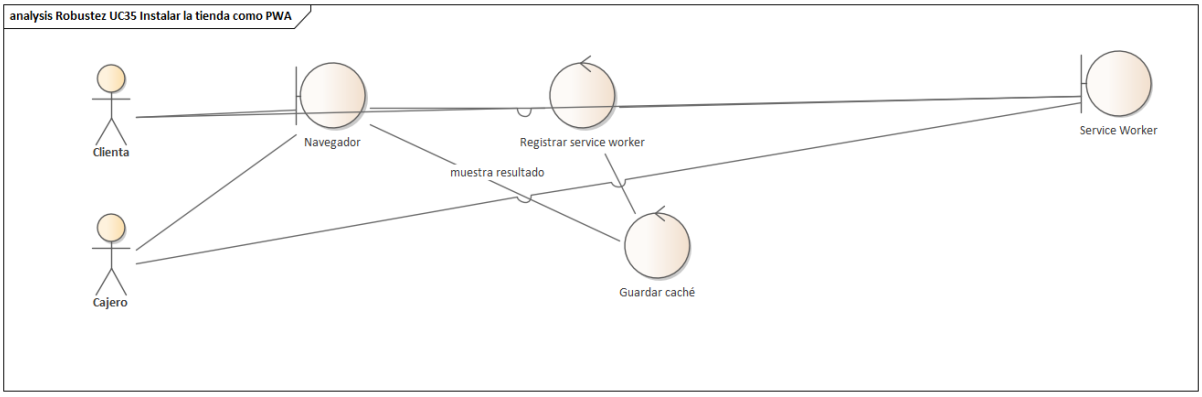


Control	Entidades o fronteras que usa
Armar comprobante	Venta, LineaVenta y Sucursal
Enviar por correo	Notificacion

Robustez UC35 Instalar la tienda como PWA

Figura 9.2.35

Robustez de UC35 Instalar la tienda como PWA



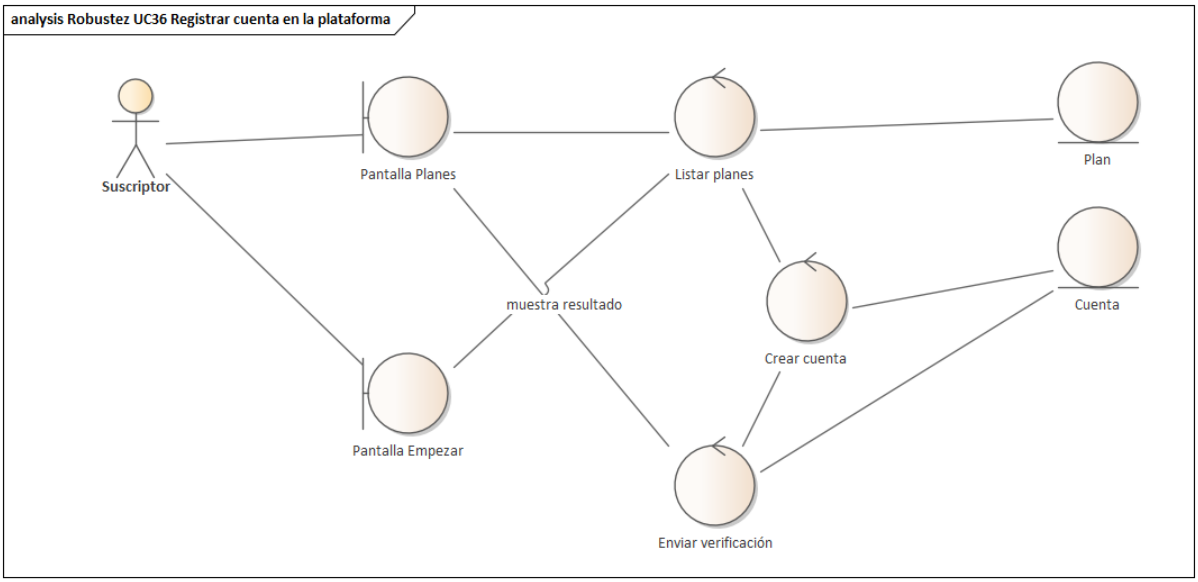
Control	Entidades o fronteras que usa
Registrar service worker	Service Worker
Guardar caché	Cálculo interno

Lote 1: SaaS multi-tienda

Robustez UC36 Registrar cuenta en la plataforma

Figura 9.2.36

Robustez de UC36 Registrar cuenta en la plataforma

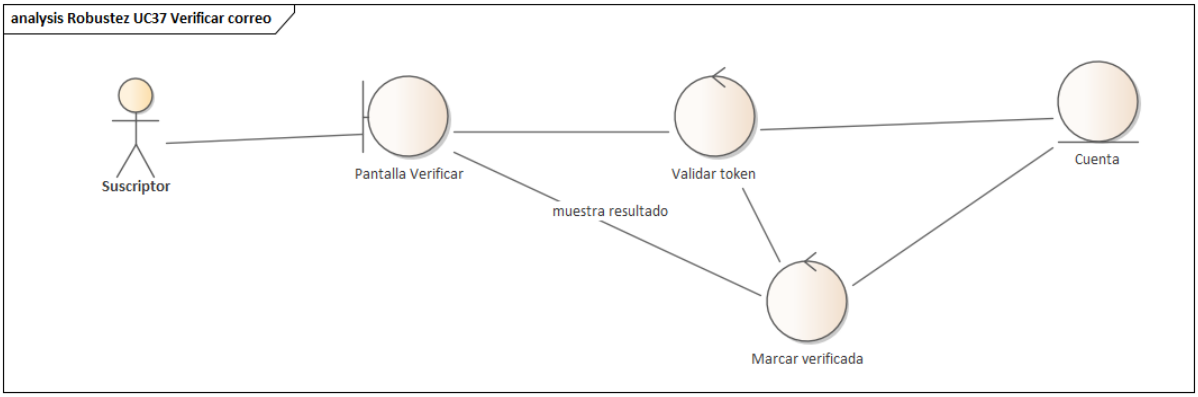


Control	Entidades o fronteras que usa
Listar planes	Plan
Crear cuenta	Cuenta
Enviar verificación	Cuenta

Robustez UC37 Verificar correo

Figura 9.2.37

Robustez de UC37 Verificar correo

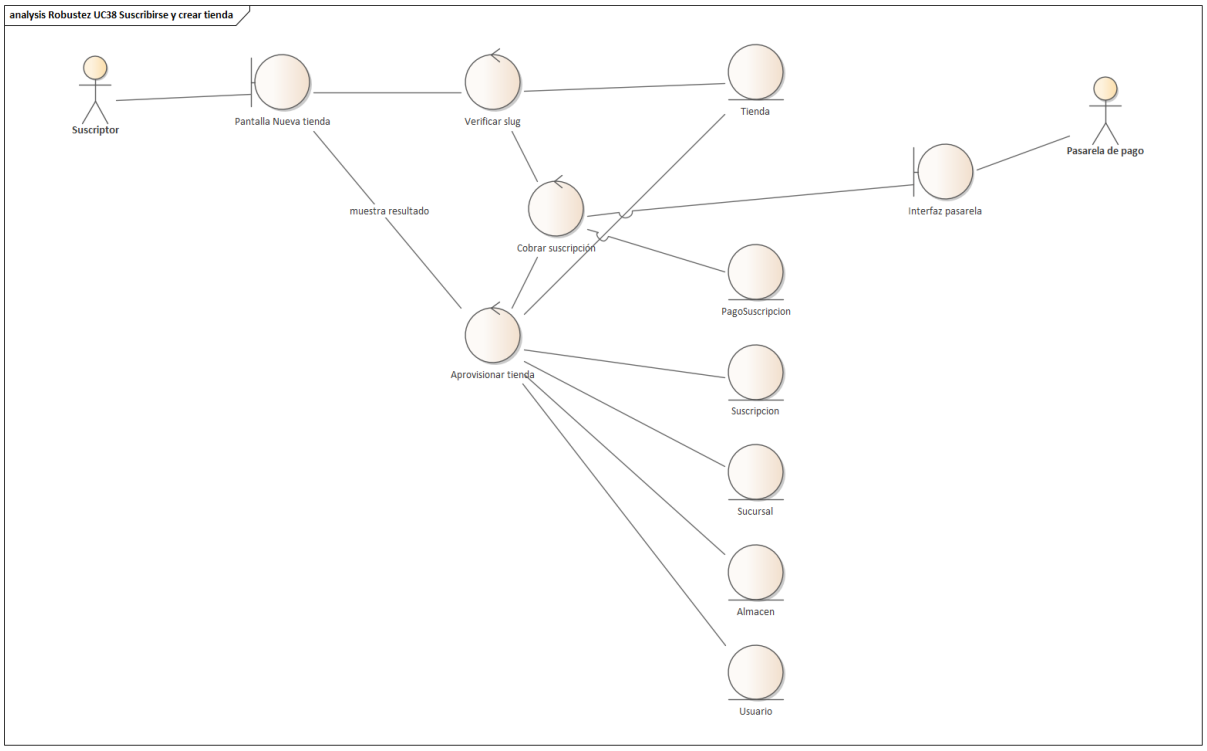


Control	Entidades o fronteras que usa
Validar token	Cuenta
Marcar verificada	Cuenta

Robustez UC38 Suscribirse y crear tienda

Figura 9.2.38

Robustez de UC38 Suscribirse y crear tienda



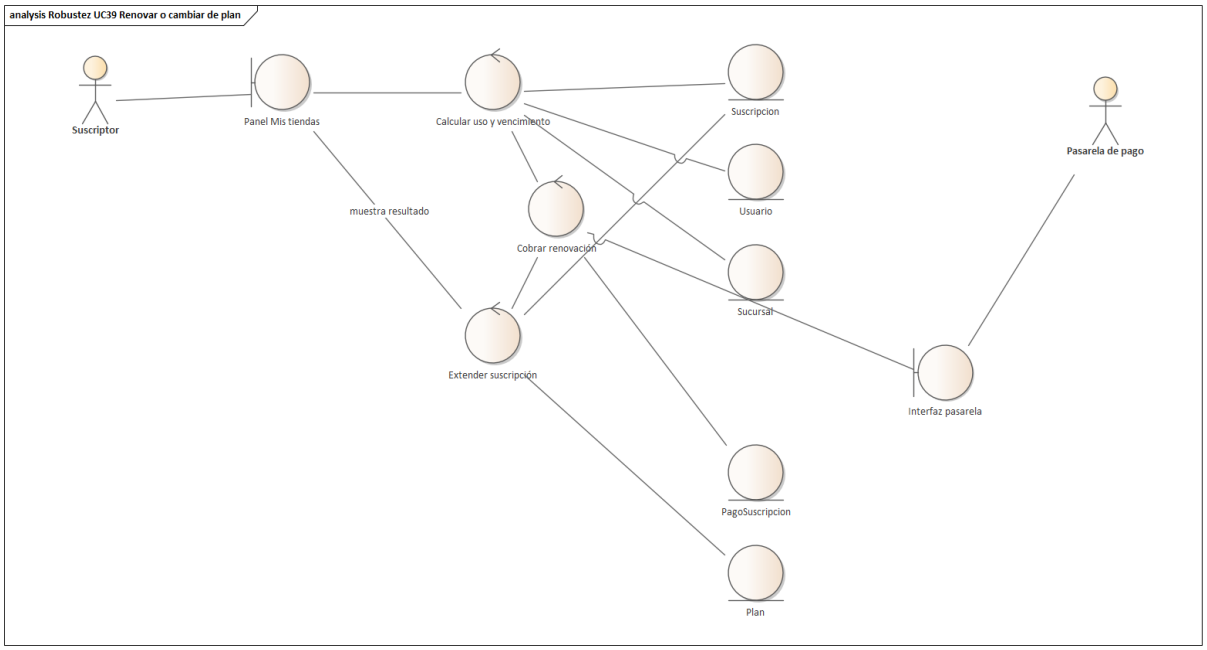
Control	Entidades o fronteras que usa
Verificar slug	Tienda
Cobrar suscripción	Interfaz pasarela y PagoSuscripcion
Aprovisionar tienda	Tienda, Suscripcion, Sucursal, Almacen y Usuario

Hallazgo de la robustez. El pago y la creación de la tienda debían ser una sola unidad. La robustez separó Cobrar suscripción de Aprovisionar tienda para que el segundo solo ocurra con el pago aprobado.

Robustez UC39 Renovar o cambiar de plan

Figura 9.2.39

Robustez de UC39 Renovar o cambiar de plan

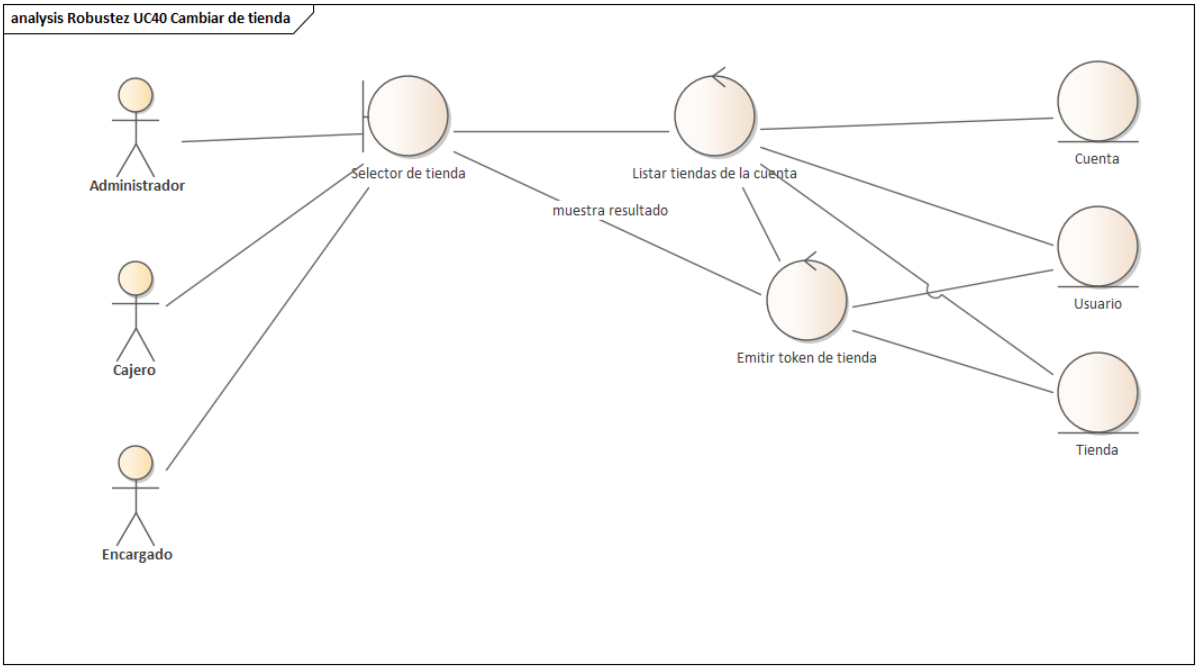


Control	Entidades o fronteras que usa
Calcular uso y vencimiento	Suscripcion, Usuario y Sucursal
Cobrar renovación	Interfaz pasarela y PagoSuscripcion
Extender suscripción	Suscripcion y Plan

Robustez UC40 Cambiar de tienda

Figura 9.2.40

Robustez de UC40 Cambiar de tienda

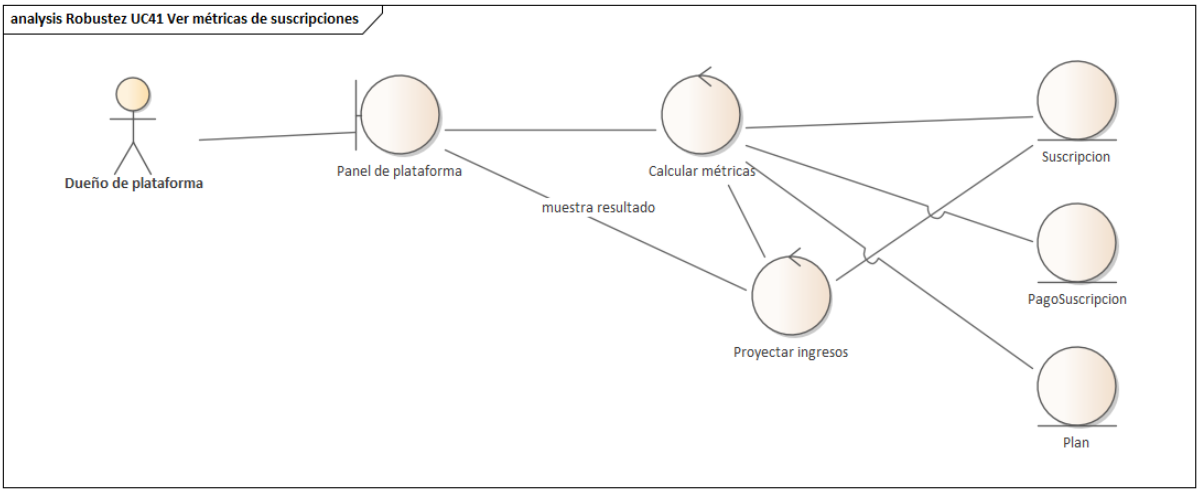


Control	Entidades o fronteras que usa
Listar tiendas de la cuenta	Cuenta, Usuario y Tienda
Emitir token de tienda	Usuario y Tienda

Robustez UC41 Ver métricas de suscripciones

Figura 9.2.41

Robustez de UC41 Ver métricas de suscripciones



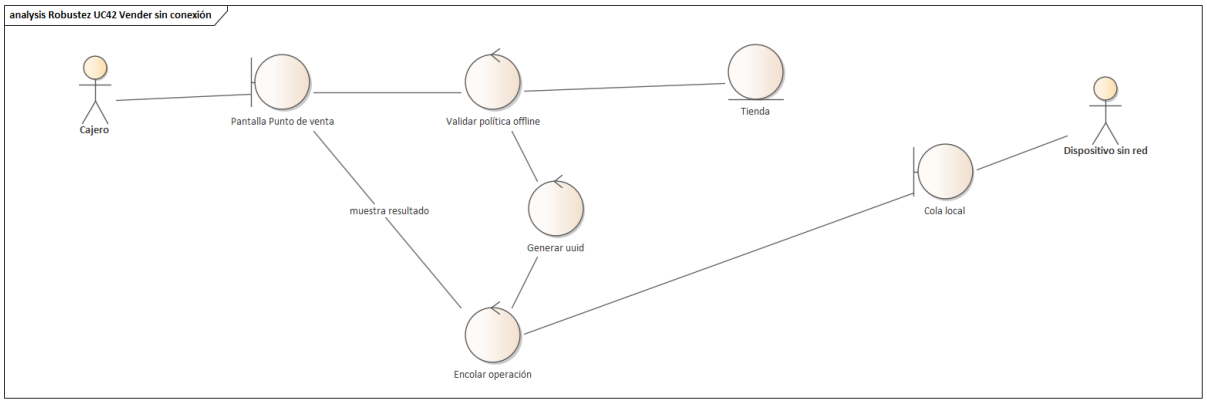
Control	Entidades o fronteras que usa
Calcular métricas	Suscripcion, PagoSuscripcion y Plan
Proyectar ingresos	Suscripcion

Lote 2: Operación sin conexión y sincronización

Robustez UC42 Vender sin conexión

Figura 9.2.42

Robustez de UC42 Vender sin conexión



Control	Entidades o fronteras que usa
Validar política offline	Tienda

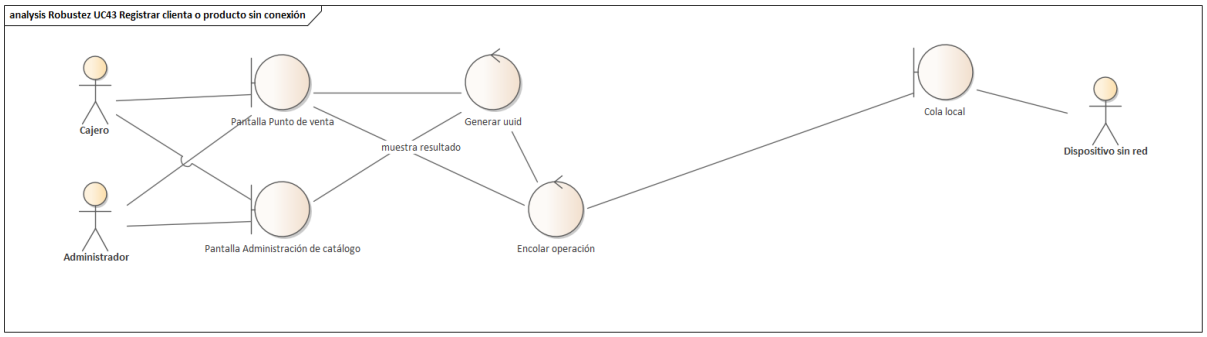
Control	Entidades o fronteras que usa
Generar uuid	Cálculo interno
Encolar operación	Cola local

Hallazgo de la robustez. La venta sin red no llega al servidor: sus controladores viven en el cliente (los módulos red, cola y caché de la PWA) y la frontera es la Cola local, no la API.

Robustez UC43 Registrar clienta o producto sin conexión

Figura 9.2.43

Robustez de UC43 Registrar clienta o producto sin conexión

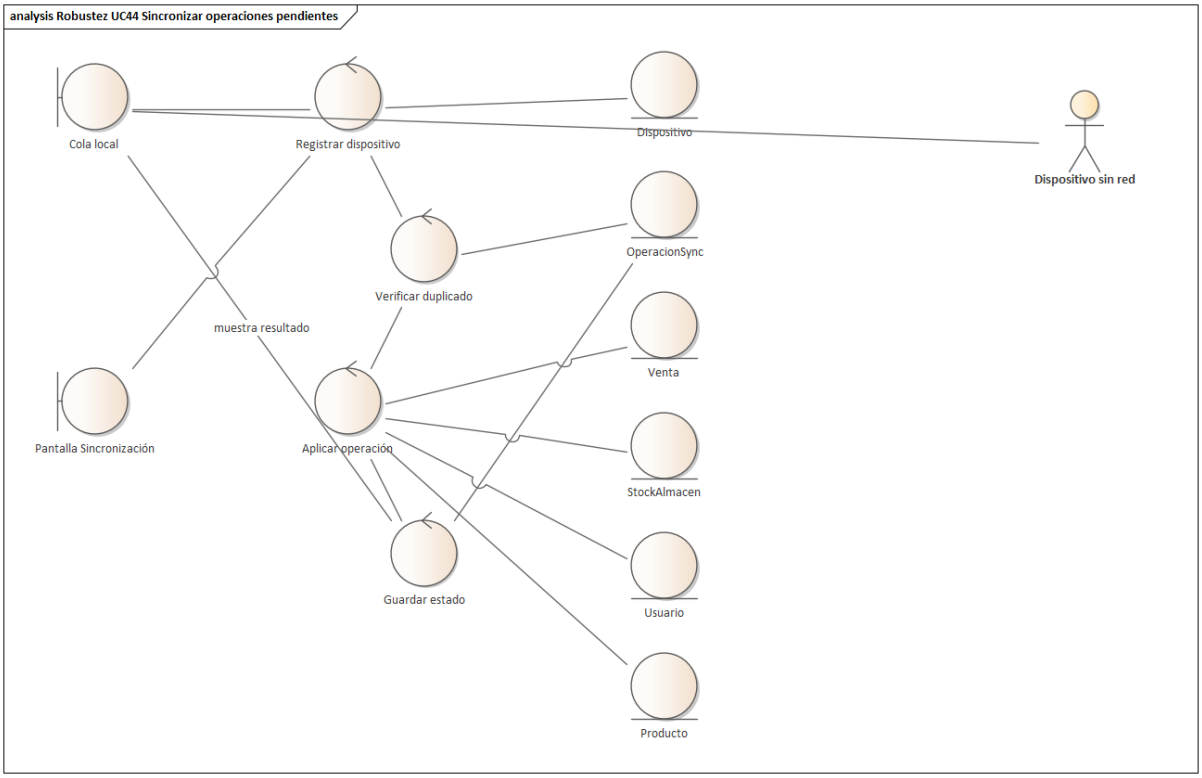


Control	Entidades o fronteras que usa
Generar uuid	Cálculo interno
Encolar operación	Cola local

Robustez UC44 Sincronizar operaciones pendientes

Figura 9.2.44

Robustez de UC44 Sincronizar operaciones pendientes



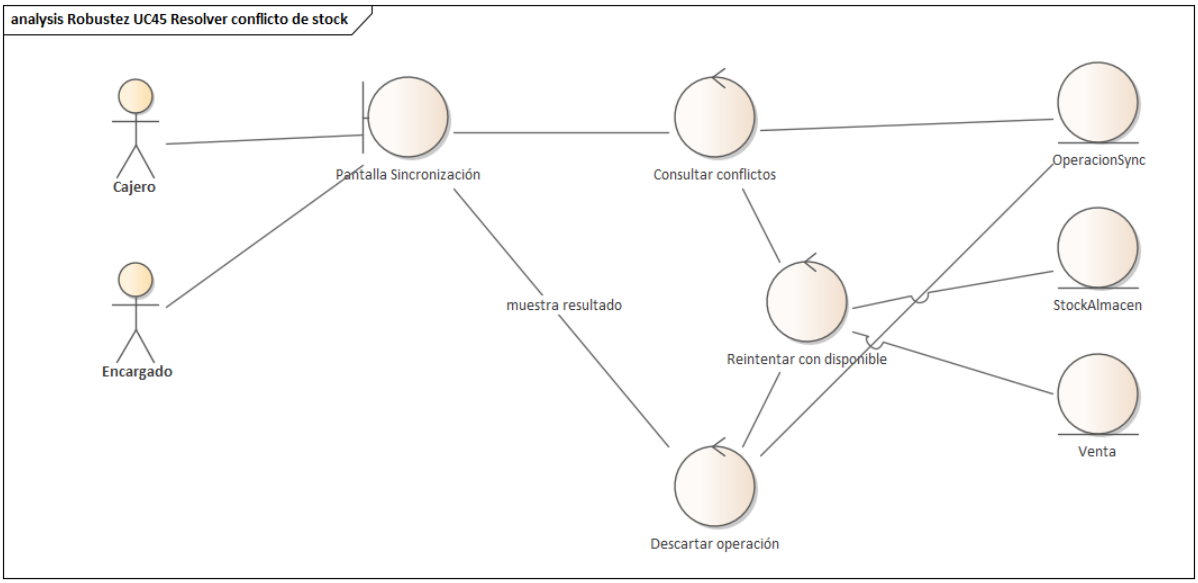
Control	Entidades o fronteras que usa
Registrar dispositivo	Dispositivo
Verificar duplicado	OperacionSync
Aplicar operación	Venta, StockAlmacen, Usuario y Producto
Guardar estado	OperacionSync

Hallazgo de la robustez. El control Verificar duplicado necesitaba una entidad propia para recordar qué uuid ya se aplicó; así nació OperacionSync.

Robustez UC45 Resolver conflicto de stock

Figura 9.2.45

Robustez de UC45 Resolver conflicto de stock

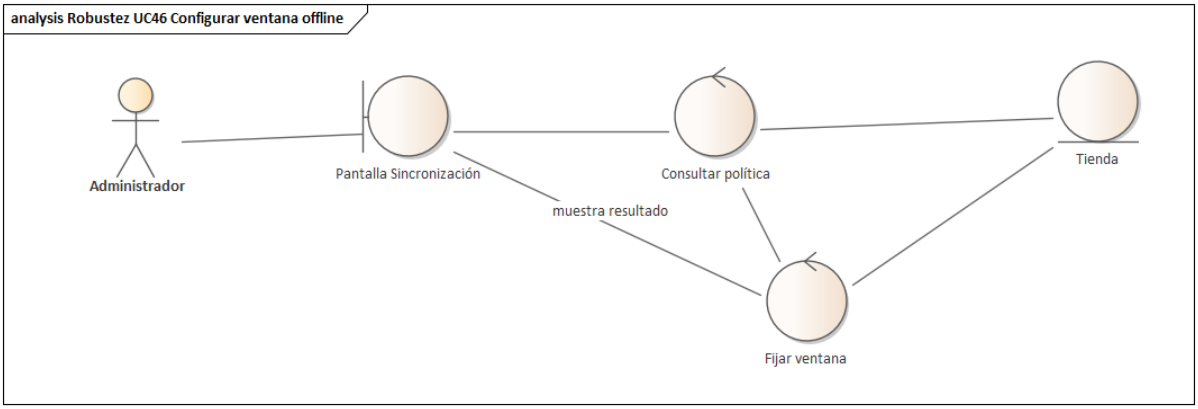


Control	Entidades o fronteras que usa
Consultar conflictos	OperacionSync
Reintentar con disponible	StockAlmacen y Venta
Descartar operación	OperacionSync

Robustez UC46 Configurar ventana offline

Figura 9.2.46

Robustez de UC46 Configurar ventana offline



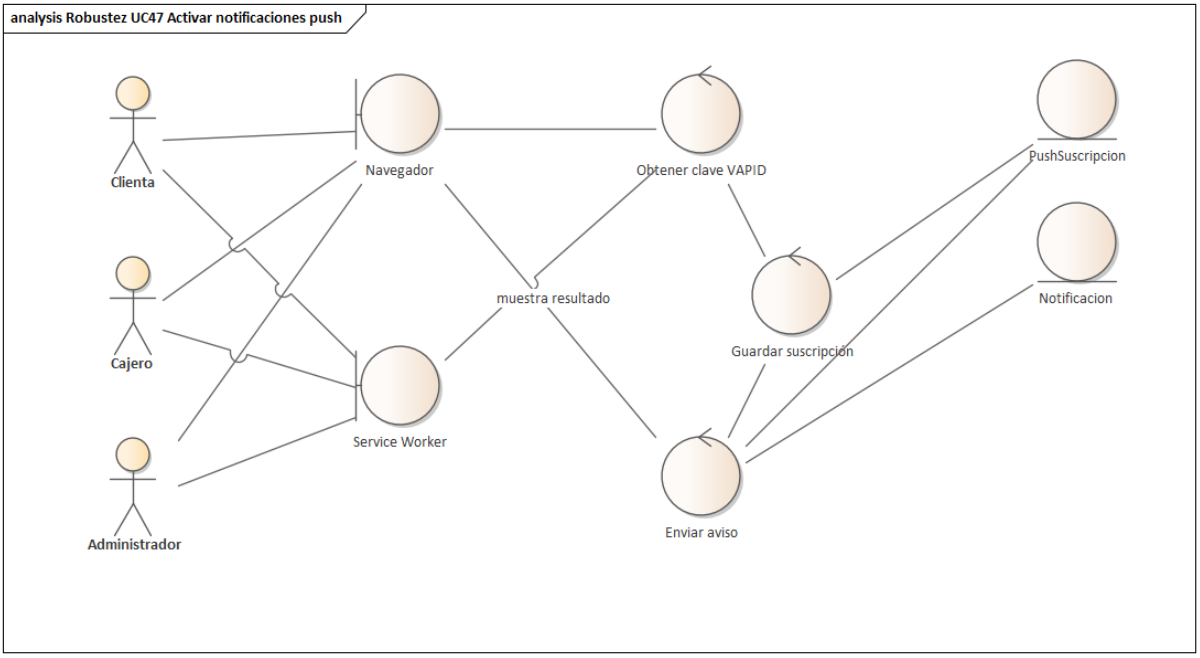
Control	Entidades o fronteras que usa

Control	Entidades o fronteras que usa
Consultar política	Tienda
Fijar ventana	Tienda

Robustez UC47 Activar notificaciones push

Figura 9.2.47

Robustez de UC47 Activar notificaciones push



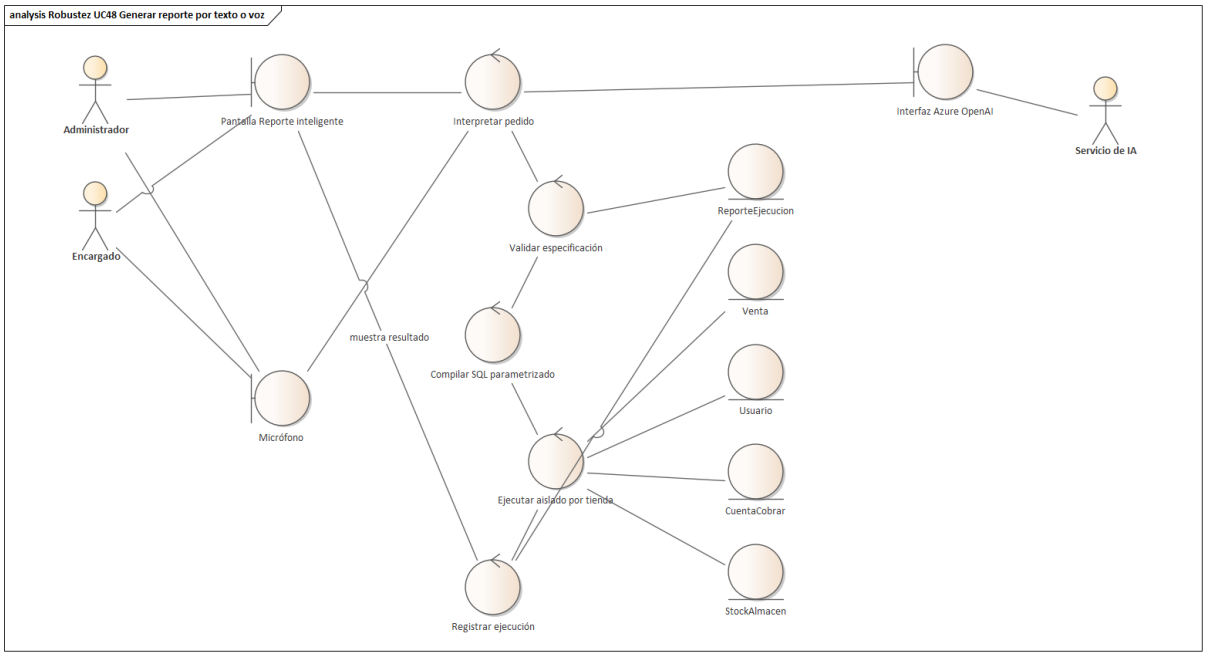
Control	Entidades o fronteras que usa
Obtener clave VAPID	Cálculo interno
Guardar suscripción	PushSuscripcion
Enviar aviso	PushSuscripcion y Notificacion

Lote 3: Reportes generativos y ventas al crédito

Robustez UC48 Generar reporte por texto o voz

Figura 9.2.48

Robustez de UC48 Generar reporte por texto o voz



Control	Entidades o fronteras que usa
Interpretar pedido	Interfaz Azure OpenAI
Validar especificación	ReporteEjecucion
Compilar SQL parametrizado	Cálculo interno
Ejecutar aislado por tienda	Venta, Usuario, CuentaCobrar y StockAlmacen
Registrar ejecución	ReporteEjecucion

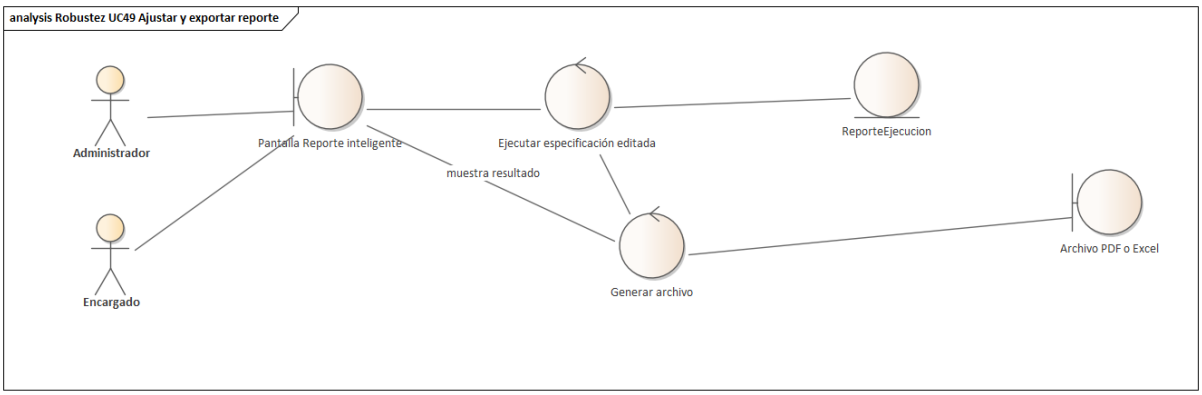
Hallazgo de la robustez. La robustez separó interpretar (IA o reglas), validar y compilar.

Solo el compilador produce SQL, y lo hace desde una especificación validada.

Robustez UC49 Ajustar y exportar reporte

Figura 9.2.49

Robustez de UC49 Ajustar y exportar reporte

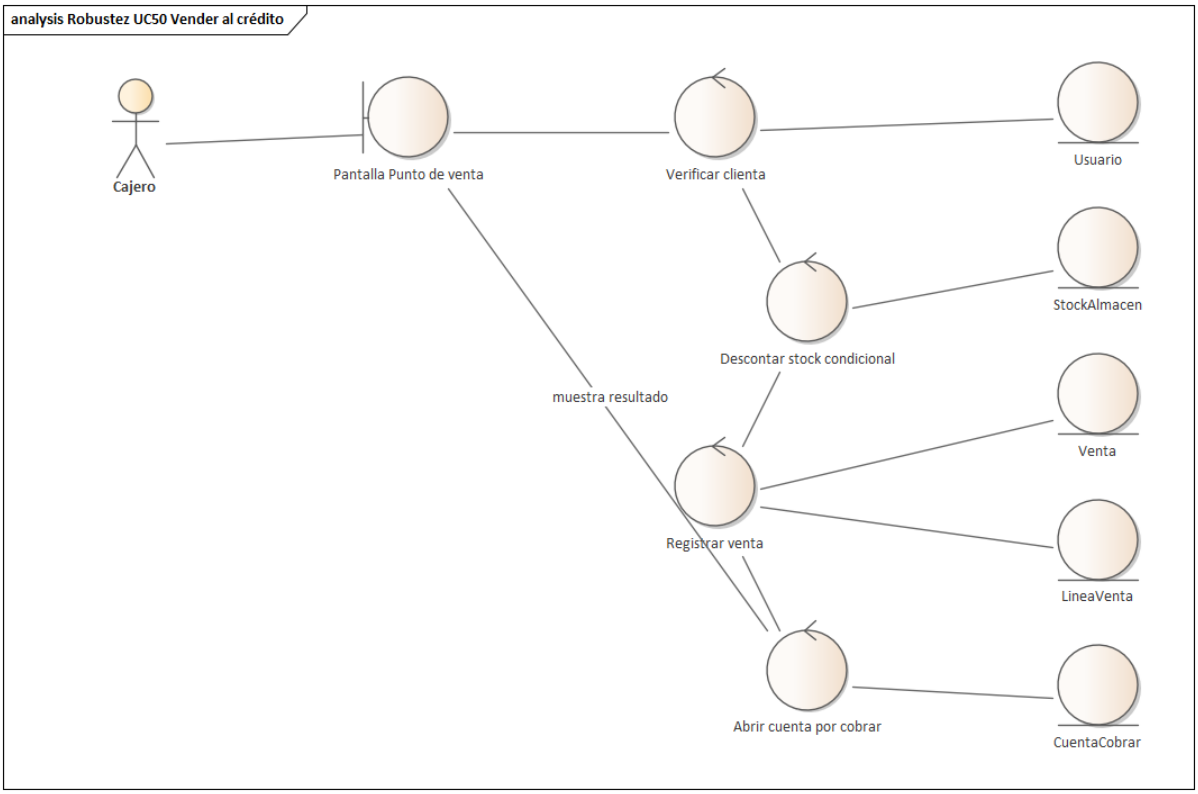


Control	Entidades o fronteras que usa
Ejecutar especificación editada	ReporteEjecucion
Generar archivo	Archivo PDF o Excel

Robustez UC50 Vender al crédito

Figura 9.2.50

Robustez de UC50 Vender al crédito

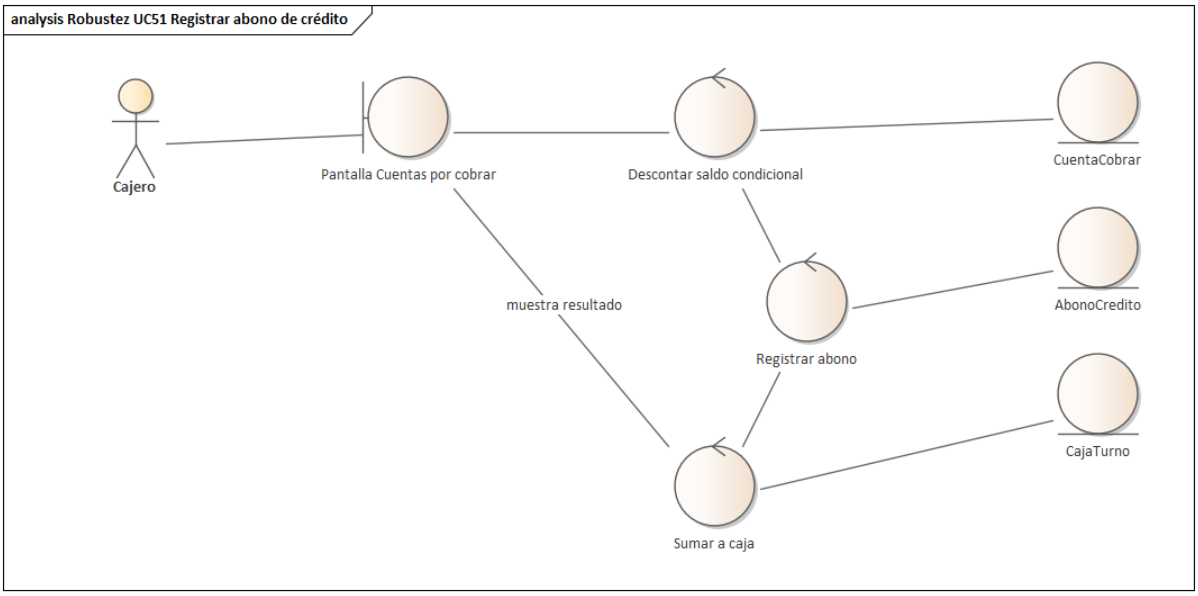


Control	Entidades o fronteras que usa
Verificar clienta	Usuario
Descontar stock condicional	StockAlmacen
Registrar venta	Venta y LineaVenta
Abrir cuenta por cobrar	CuentaCobrar

Robustez UC51 Registrar abono de crédito

Figura 9.2.51

Robustez de UC51 Registrar abono de crédito



Control	Entidades o fronteras que usa
Descontar saldo condicional	CuentaCobrar
Registrar abono	AbonoCredito
Sumar a caja	CajaTurno

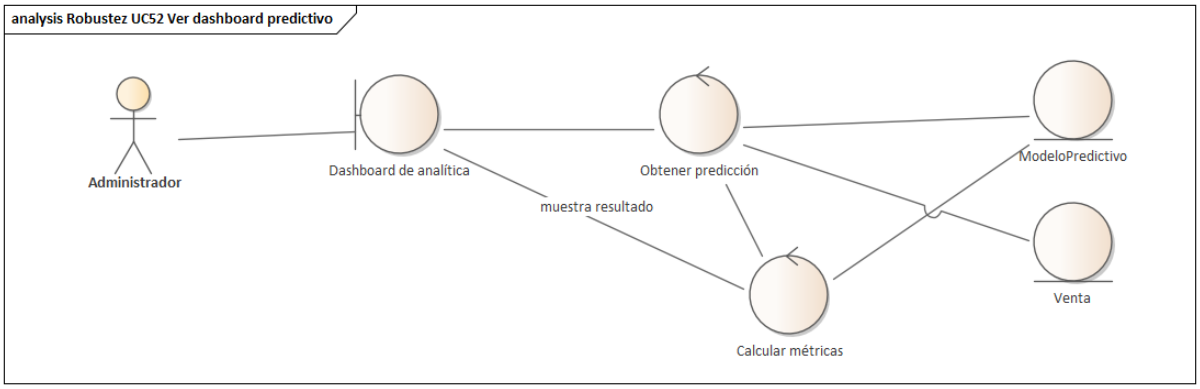
Hallazgo de la robustez. Descontar el saldo debía ser condicional para que dos abonos simultáneos no lo dejen negativo; el control lleva ese nombre a propósito.

Lote 4: Analítica con aprendizaje automático

Robustez UC52 Ver dashboard predictivo

Figura 9.2.52

Robustez de UC52 Ver dashboard predictivo

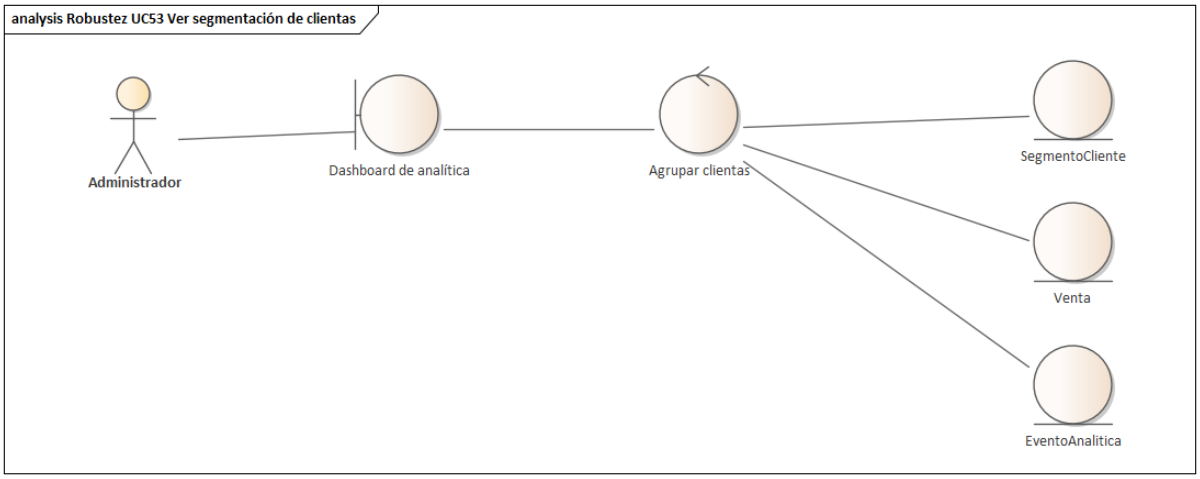


Control	Entidades o fronteras que usa
Obtener predicción	ModeloPredictivo y Venta
Calcular métricas	ModeloPredictivo

Robustez UC53 Ver segmentación de clientas

Figura 9.2.53

Robustez de UC53 Ver segmentación de clientas



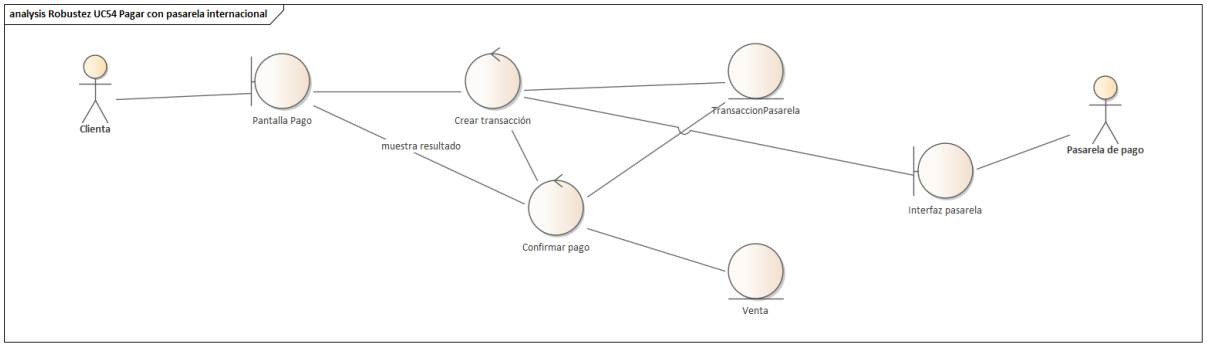
Control	Entidades o fronteras que usa
Agrupar clientas	SegmentoCliente, Venta y EventoAnalitica

Lote 5: Pagos para países hispanohablantes

Robustez UC54 Pagar con pasarela internacional

Figura 9.2.54

Robustez de UC54 Pagar con pasarela internacional



Control	Entidades o fronteras que usa
Crear transacción	TransaccionPasarela y Interfaz pasarela
Confirmar pago	TransaccionPasarela y Venta

9.2.1 Revisión de diseño preliminar (PDR)

La PDR aplicó la prueba del resaltador: se recorrió cada frase del curso básico y de los alternos y se marcó el elemento del diagrama que la realiza, hasta que no quedó texto sin objeto ni objeto sin texto.

Verificación	Resultado
Cada caso tiene diagrama de robustez	Sí, 54 diagramas
Número de controles por caso dentro de lo recomendado (2 a 5)	Sí; el máximo es 5 (UC48)
Ninguna frontera habla con una entidad	Sí; se verificó al generar el modelo
Todas las entidades existen en el dominio	Sí; las de los lotes 4 y 5 están marcadas como previstas
Los hallazgos se corrigieron en el texto y en el dominio	Sí; ver los hallazgos de cada caso
No hay detalle de diseño prematuro (tablas, SQL, frameworks)	Sí; los nombres de tablas y métodos aparecen recién en la secuencia

9.3 Diseño detallado

El diseño detallado asigna comportamiento a las clases. Hay un diagrama de secuencia por caso de uso, y sus líneas de vida son las de la robustez: el actor, la pantalla, las entidades y las interfaces externas. A ellas se suman las clases del backend que realizan el caso. Cada control de la robustez reaparece como un mensaje del servicio hacia la entidad que toca.

En NestJS, la clase que recibe la petición HTTP se llama controller, pero en términos de ICONIX es una frontera del lado del servidor: valida la entrada, toma el usuario y la tienda del token, y delega. Los controles de ICONIX quedan como operaciones del servicio. En los módulos sin servicio propio (Créditos, Pagos, Devoluciones, Pedidos, Usuarios, Cliente y Administración), el controller ejecuta los controles directamente sobre Prisma; la CDR lo registra como deuda técnica.

Los diagramas siguen la notación de secuencia de UML 2.5:

- **Mensajes síncronos con argumentos reales.** El mensaje a la API lleva la ruta y la operación del controller con sus parámetros (por ejemplo `POST /ventas: crear(user, dto)`), y el mensaje al servicio nombra la operación y sus argumentos tal como están en el código (`cobrar(dto, actor, opciones)`).
- **Mensajes de respuesta.** Se dibujan con línea punteada y flecha abierta.
- **Fragmento combinado alt.** ICONIX pide que la secuencia cubra el curso básico y los alternos. El fragmento se abre en el mensaje donde el sistema toma la decisión: después de validar o consultar, o después del cobro cuando el alterno es un pago rechazado (UC27, UC38, UC39, UC54). El primer operando tiene como guarda la condición del curso alterno (por ejemplo `[Sin turno abierto o sin stock (3a)]`) y devuelve el motivo hasta la pantalla. El segundo, `[else]`, sigue el curso básico. Cuando el alterno no corta el caso sino

que lo resuelve de otra forma, como el respaldo por reglas si el Servicio de IA no responde (UC04, UC29, UC48), los dos operandos continúan hacia el mismo cierre. Los alternos que ocurren solo en el dispositivo (permiso de cámara o de notificaciones) se cubren en la robustez y no en la secuencia del servidor.

- **Fragmento combinado loop.** Se usa cuando el caso repite pasos por elemento. En UC44 el servidor procesa cada operación del lote en su propia transacción: `loop [por cada operación del lote]`.

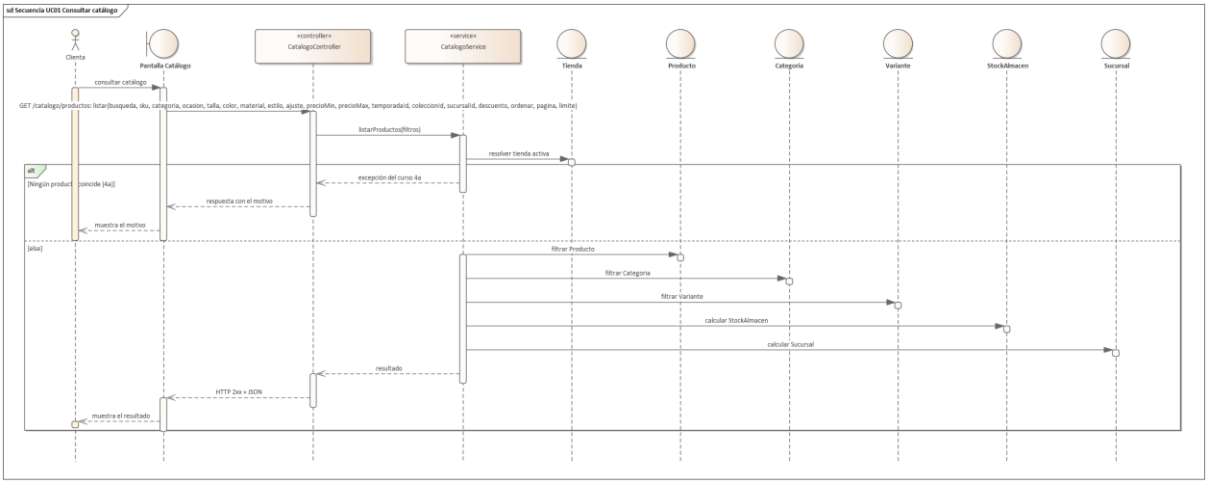
La arquitectura técnica que la robustez omitía (el guard JWT, el contexto de tienda y la clave de idempotencia) no se repite en cada secuencia: se aplica a todas las rutas por el mismo mecanismo y se muestra una sola vez en la arquitectura (9.3.2).

Lote 0: Casos heredados del primer ciclo, revisados con robustez y secuencia

Secuencia UC01 Consultar catálogo

Figura 9.3.1

Secuencia de UC01 Consultar catálogo

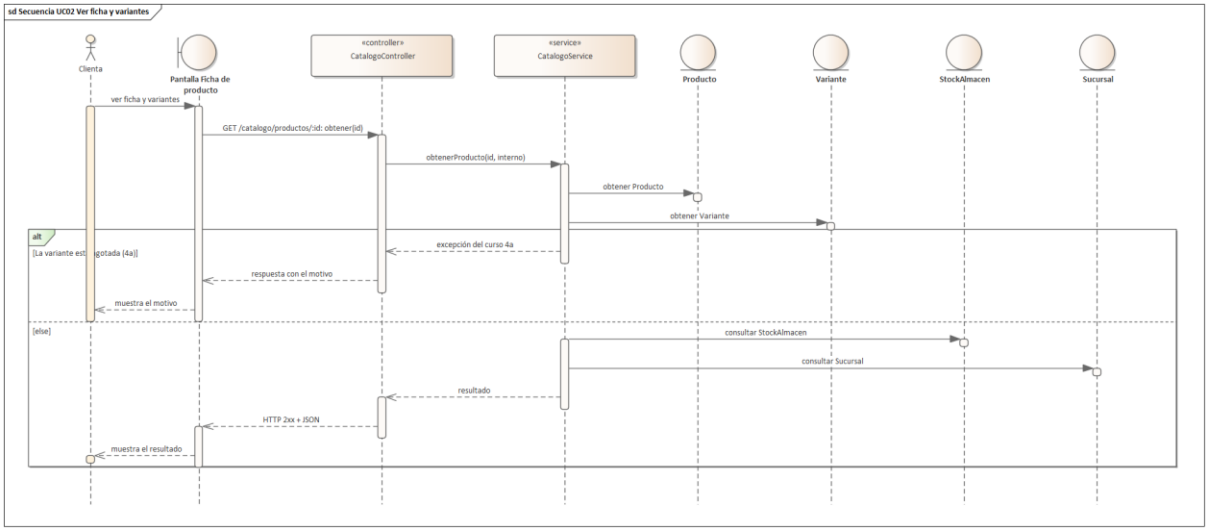


Ruta `GET /catalogo/productos`, realizada por `CatalogoService.listarProductos`.

Secuencia UC02 Ver ficha y variantes

Figura 9.3.2

Secuencia de UC02 Ver ficha y variantes

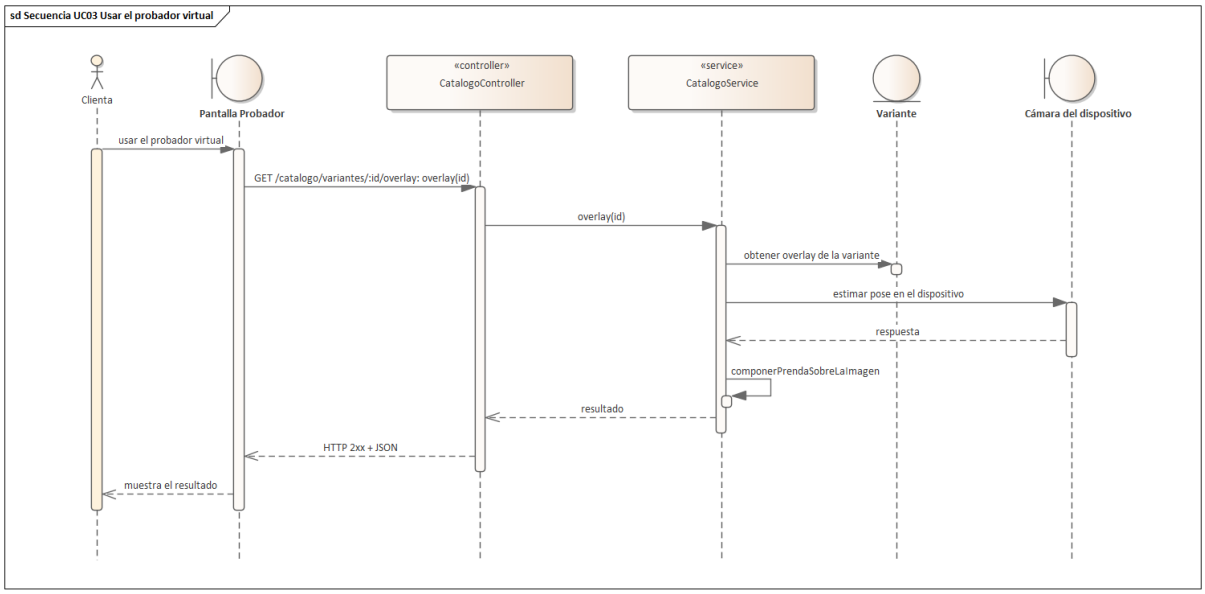


Ruta `GET /catalogo/productos/:id`, realizada por `CatalogoService.obtenerProducto`.

Secuencia UC03 Usar el probador virtual

Figura 9.3.3

Secuencia de UC03 Usar el probador virtual

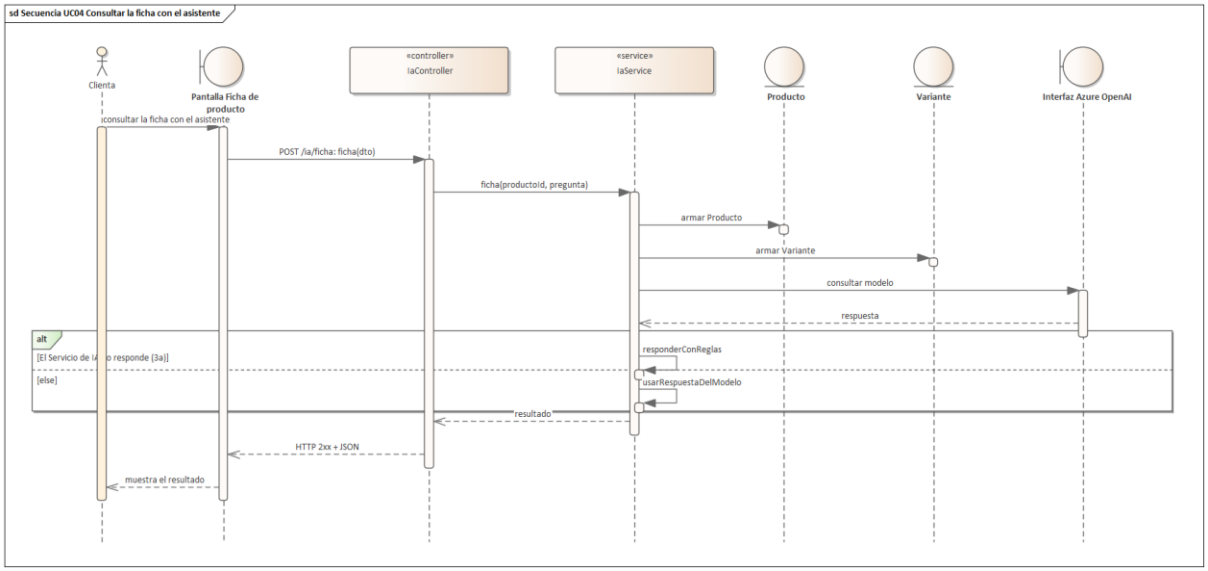


Ruta `GET /catalogo/variantes/:id/overlay`, realizada por `CatalogoService.overlay`.

Secuencia UC04 Consultar la ficha con el asistente

Figura 9.3.4

Secuencia de UC04 Consultar la ficha con el asistente

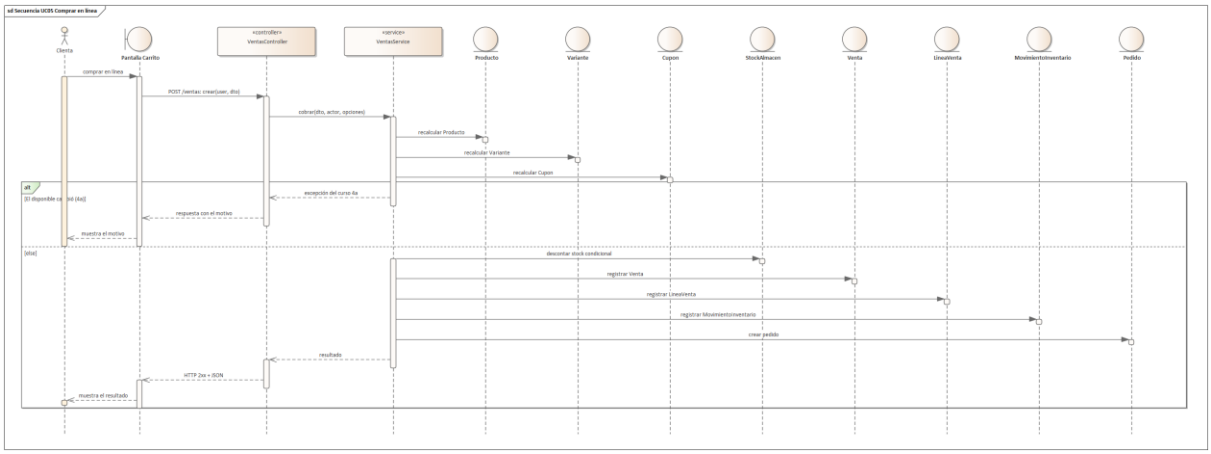


Ruta `POST /ia/ficha`, realizada por `IaService.ficha`.

Secuencia UC05 Comprar en línea

Figura 9.3.5

Secuencia de UC05 Comprar en línea

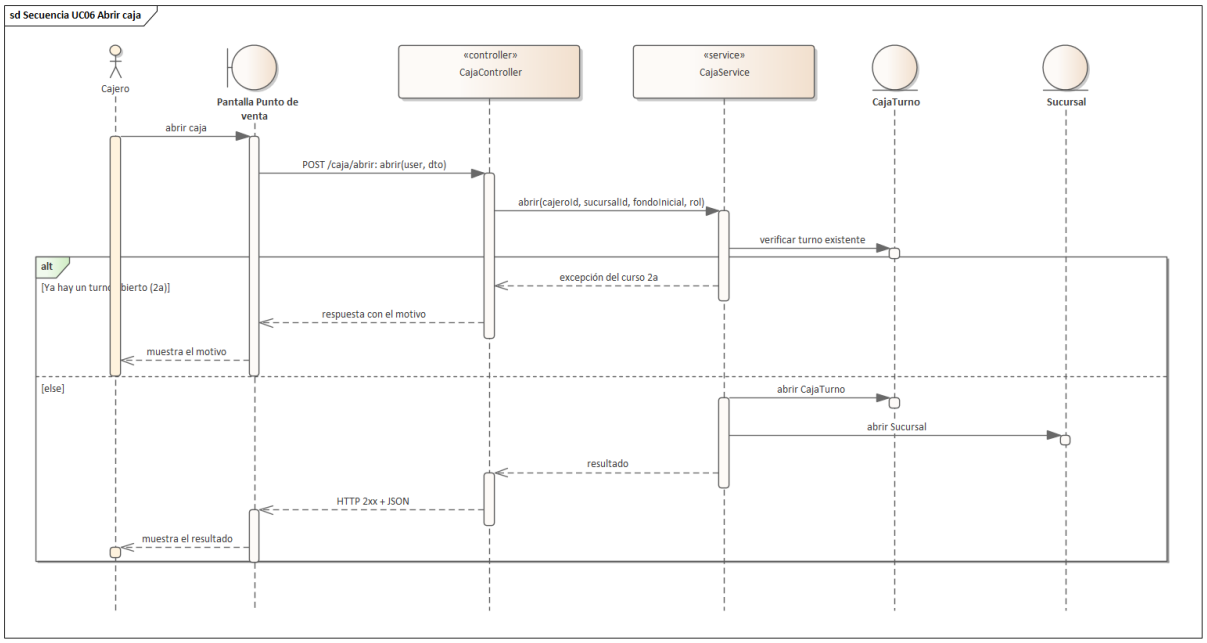


Ruta `POST /ventas`, realizada por `VentasService.cobrar`.

Secuencia UC06 Abrir caja

Figura 9.3.6

Secuencia de UC06 Abrir caja

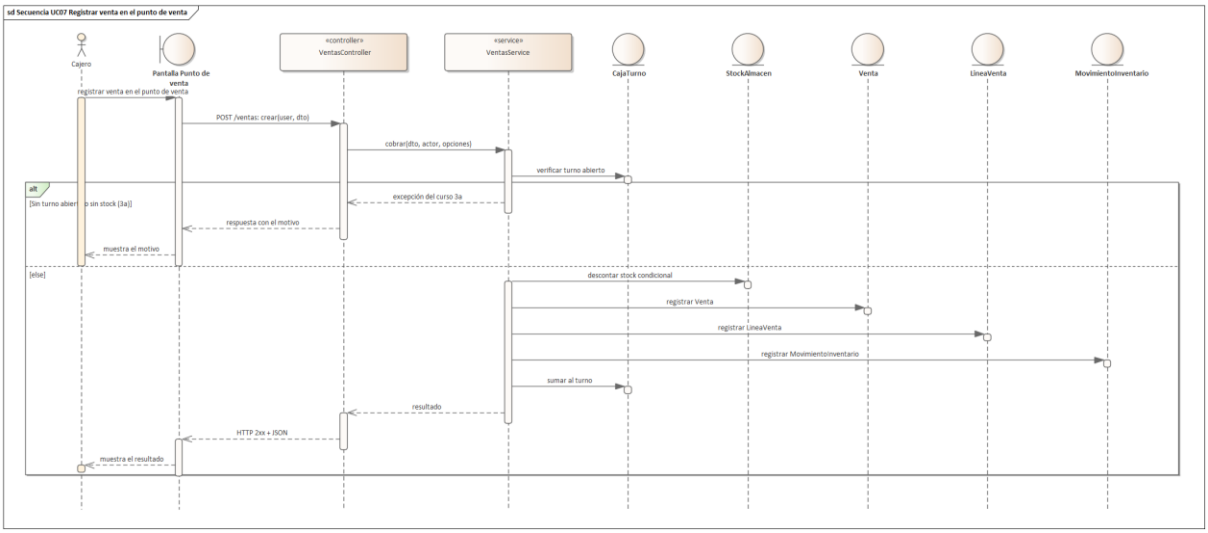


Ruta `POST /caja/abrir`, realizada por `CajaService.abrir`.

Secuencia UC07 Registrar venta en el punto de venta

Figura 9.3.7

Secuencia de UC07 Registrar venta en el punto de venta

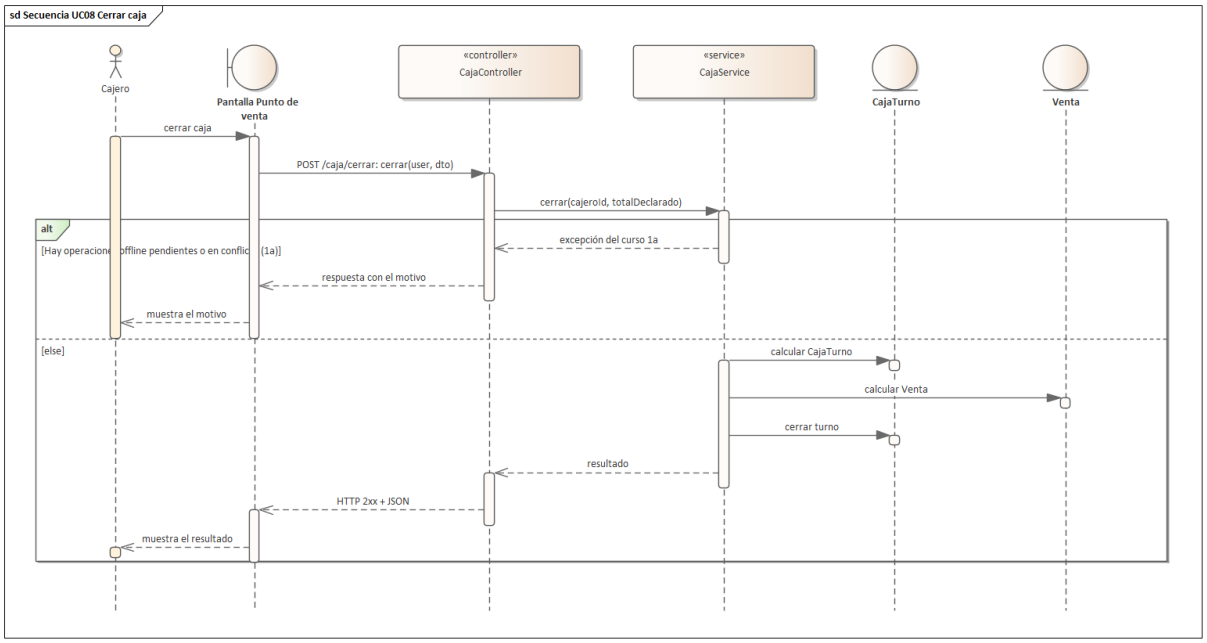


Ruta `POST /ventas`, realizada por `VentasService.cobrar`.

Secuencia UC08 Cerrar caja

Figura 9.3.8

Secuencia de UC08 Cerrar caja

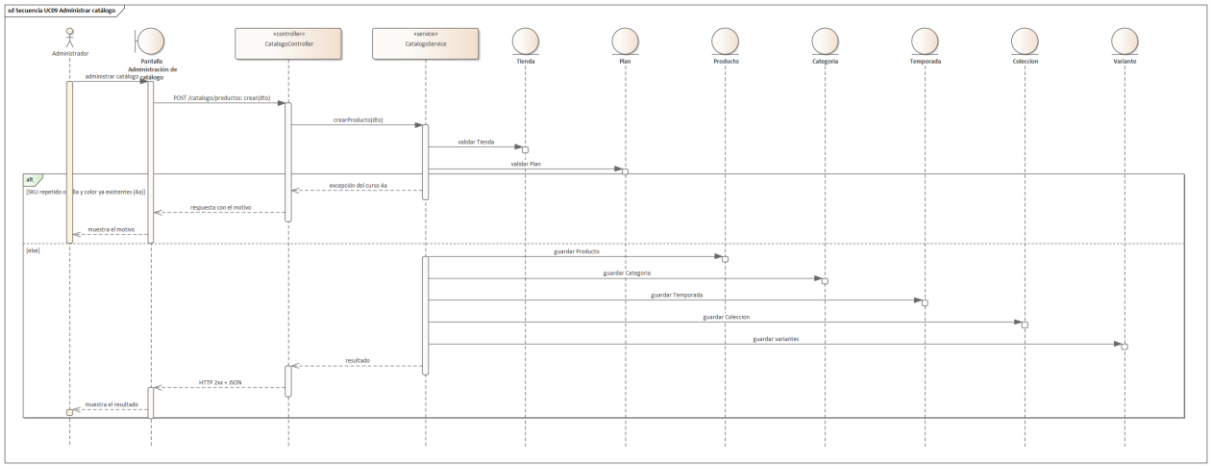


Ruta `POST /caja/cerrar`, realizada por `CajaService.cerrar`.

Secuencia UC09 Administrar catálogo

Figura 9.3.9

Secuencia de UC09 Administrar catálogo

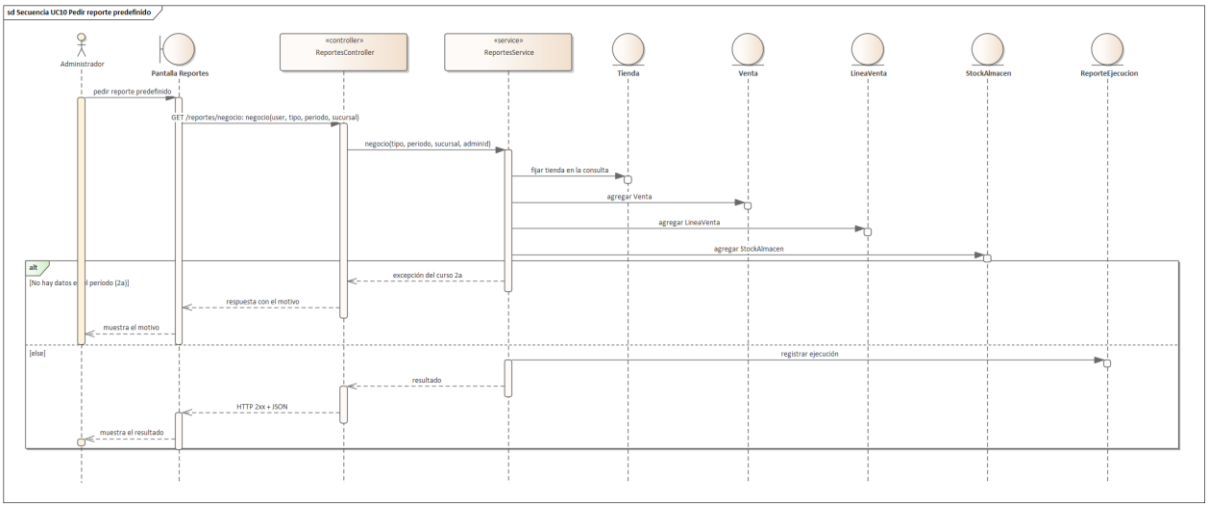


Ruta `POST /catalogo/productos`, realizada por `CatalogoService.crearProducto`.

Secuencia UC10 Pedir reporte predefinido

Figura 9.3.10

Secuencia de UC10 Pedir reporte predefinido

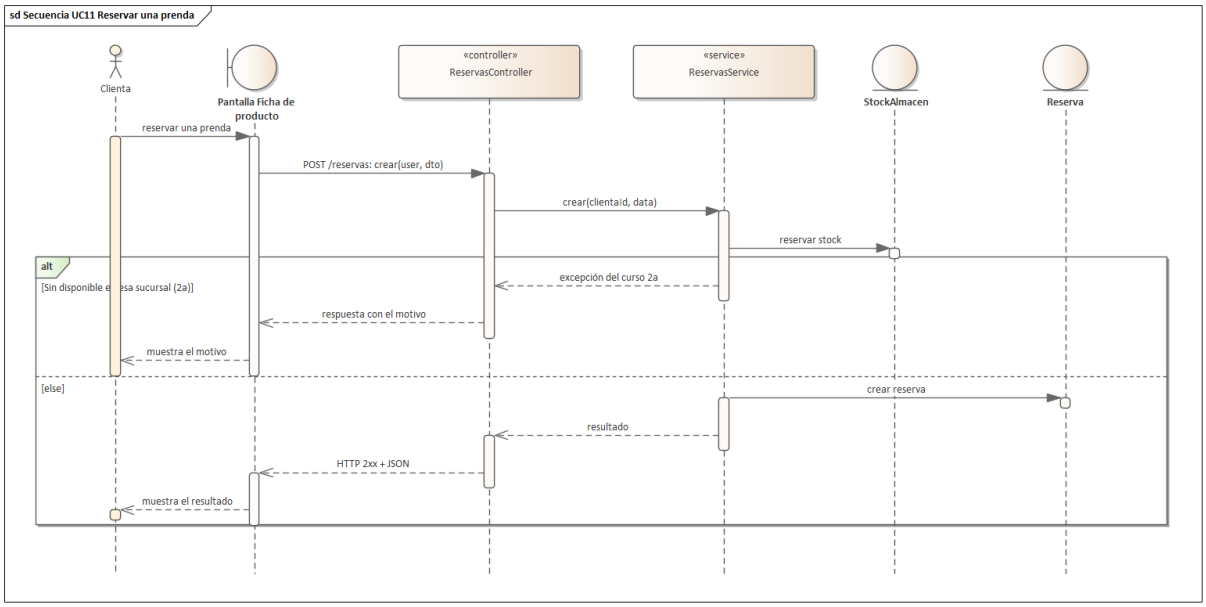


Ruta `GET /reportes/negocio`, realizada por `ReportesService.negocio`.

Secuencia UC11 Reservar una prenda

Figura 9.3.11

Secuencia de UC11 Reservar una prenda

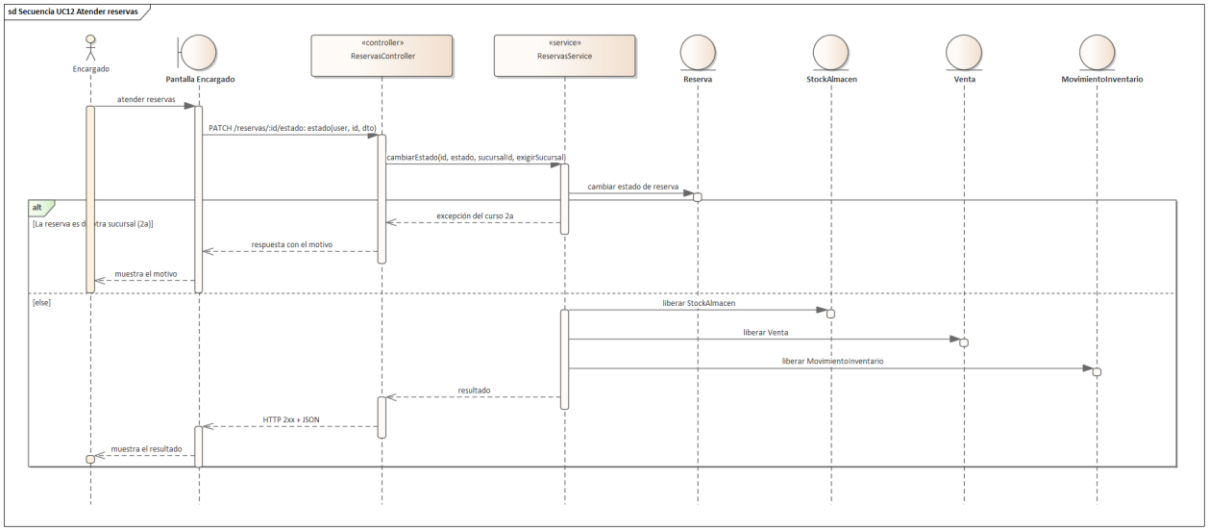


Ruta `POST /reservas`, realizada por `ReservasService.crear`.

Secuencia UC12 Atender reservas

Figura 9.3.12

Secuencia de UC12 Atender reservas

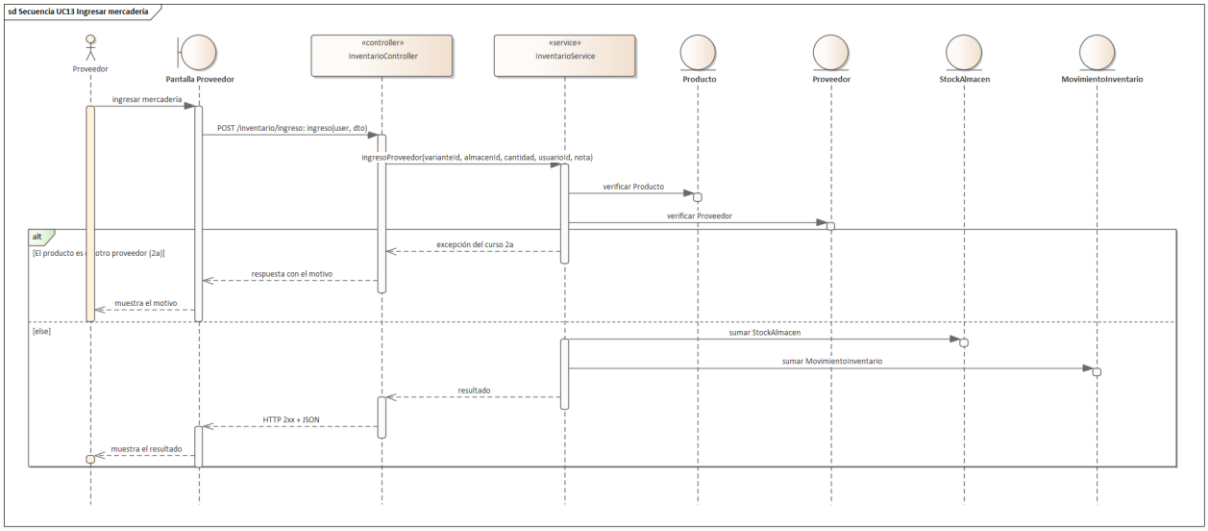


Ruta `PATCH /reservas/:id/estado`, realizada por `ReservasService.cambiarEstado`.

Secuencia UC13 Ingresar mercadería

Figura 9.3.13

Secuencia de UC13 Ingresar mercadería

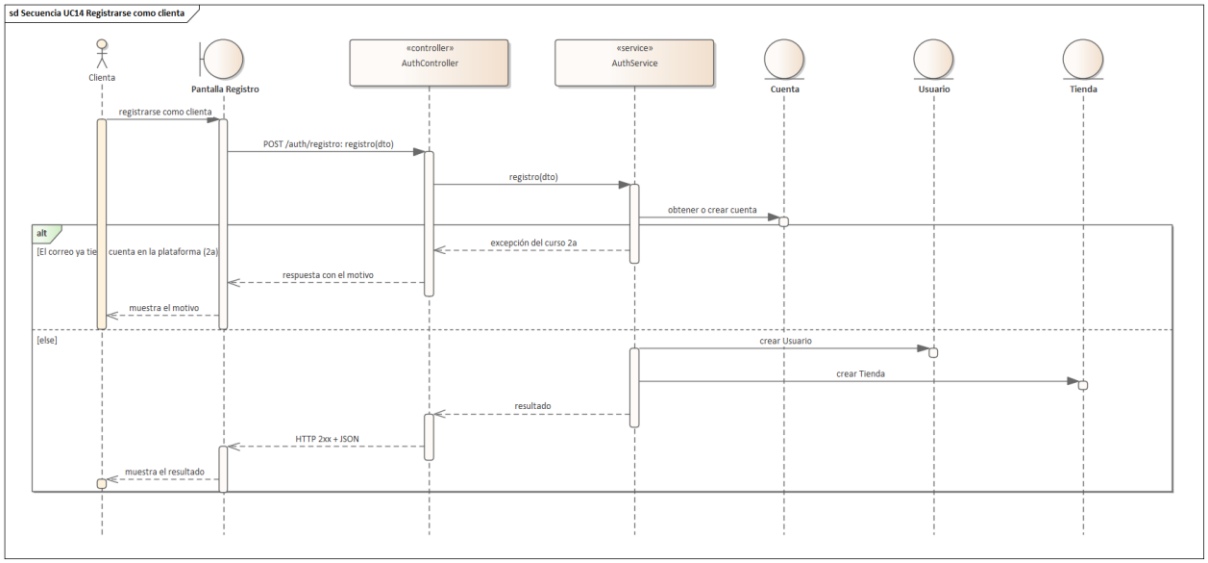


Ruta `POST /inventario/ingreso`, realizada por `InventarioService.ingresoProveedor`.

Secuencia UC14 Registrarse como clienta

Figura 9.3.14

Secuencia de UC14 Registrarse como clienta

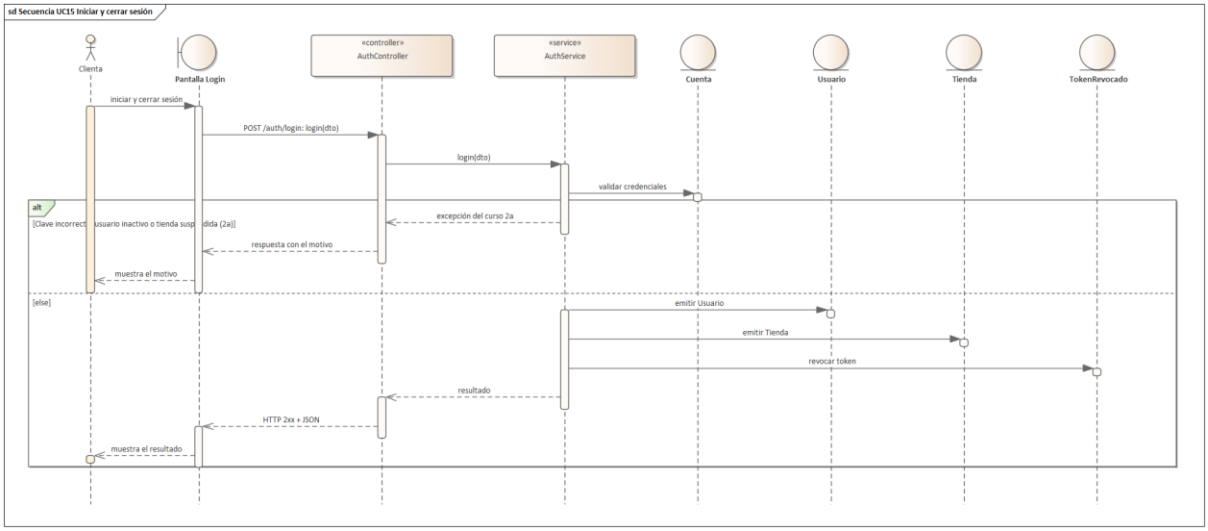


Ruta `POST /auth/registro`, realizada por `AuthService.registro`.

Secuencia UC15 Iniciar y cerrar sesión

Figura 9.3.15

Secuencia de UC15 Iniciar y cerrar sesión

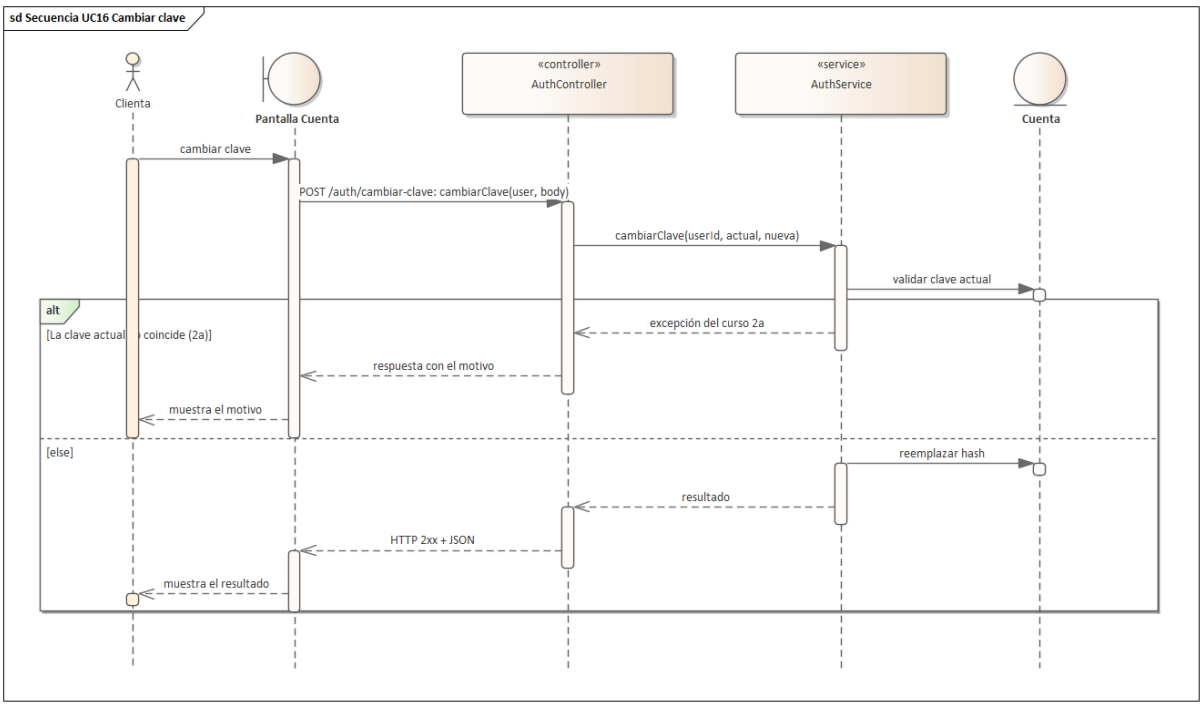


Ruta `POST /auth/login`, realizada por `AuthService.login`.

Secuencia UC16 Cambiar clave

Figura 9.3.16

Secuencia de UC16 Cambiar clave

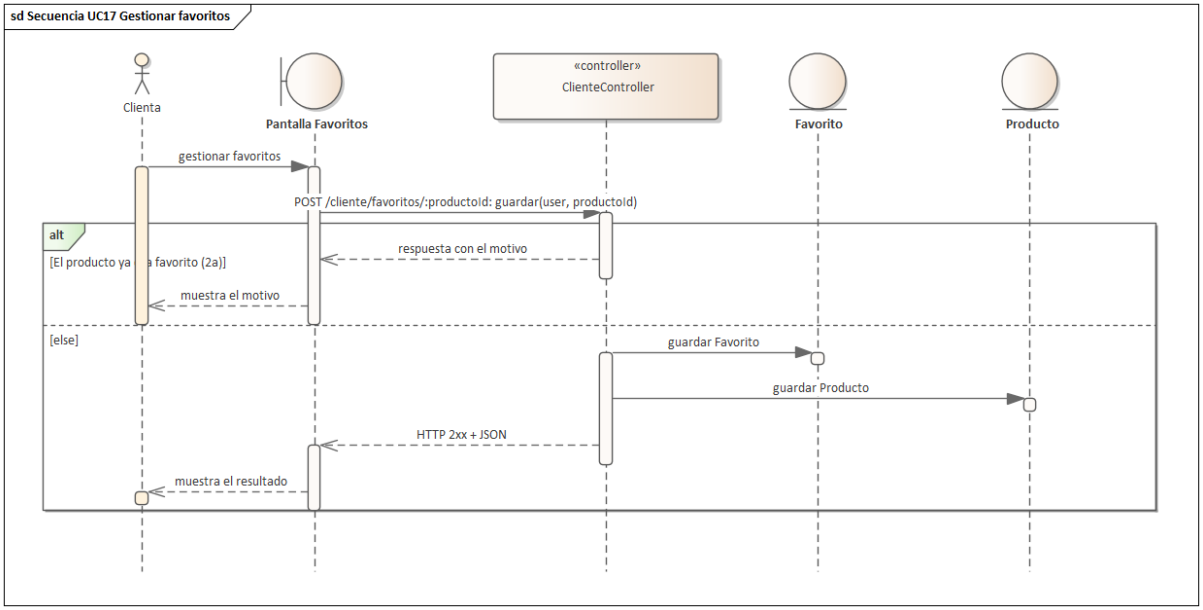


Ruta `POST /auth/cambiar-clave`, realizada por `AuthService.cambiarClave`.

Secuencia UC17 Gestionar favoritos

Figura 9.3.17

Secuencia de UC17 Gestionar favoritos

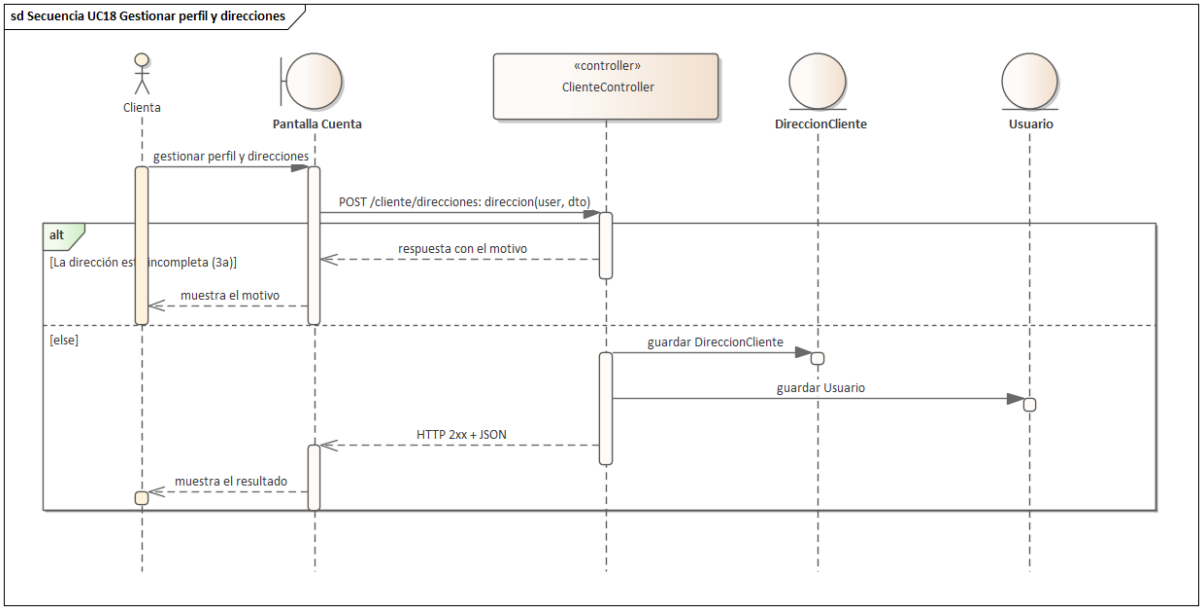


Ruta `POST /cliente/favoritos/:productoId`, realizada por `ClienteController.guardar`.

Secuencia UC18 Gestionar perfil y direcciones

Figura 9.3.18

Secuencia de UC18 Gestionar perfil y direcciones

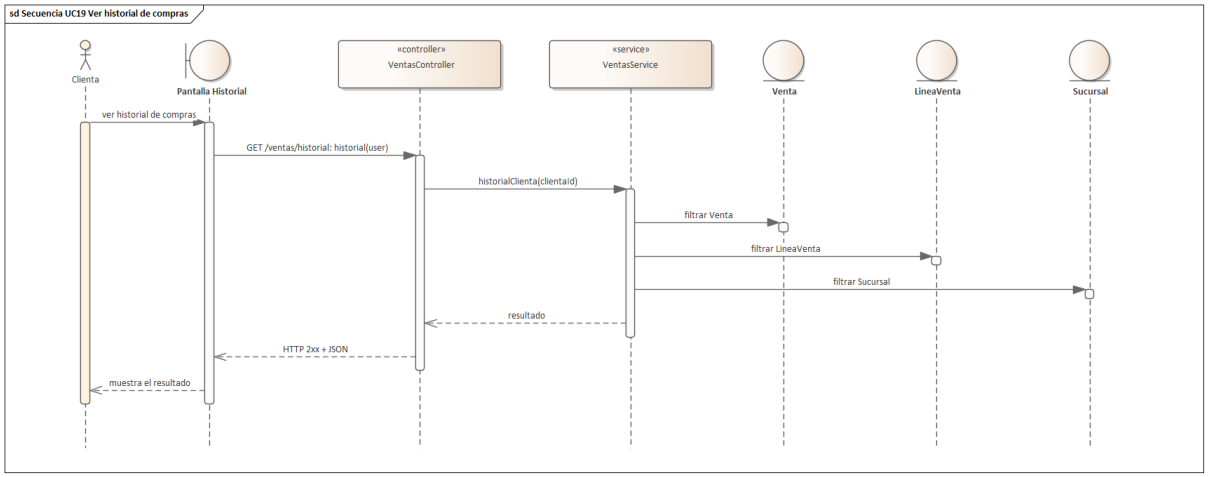


Ruta `POST /cliente/direcciones`, realizada por `ClienteController.direccion`.

Secuencia UC19 Ver historial de compras

Figura 9.3.19

Secuencia de UC19 Ver historial de compras

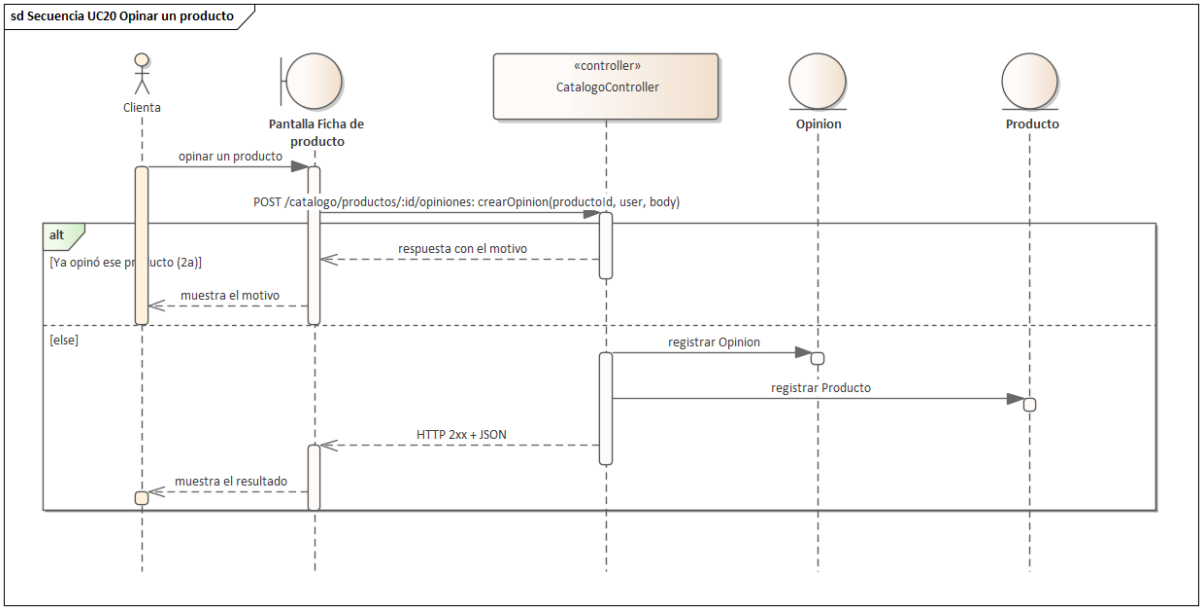


Ruta `GET /ventas/historial`, realizada por `VentasService.historialClienta`.

Secuencia UC20 Opinar un producto

Figura 9.3.20

Secuencia de UC20 Opinar un producto

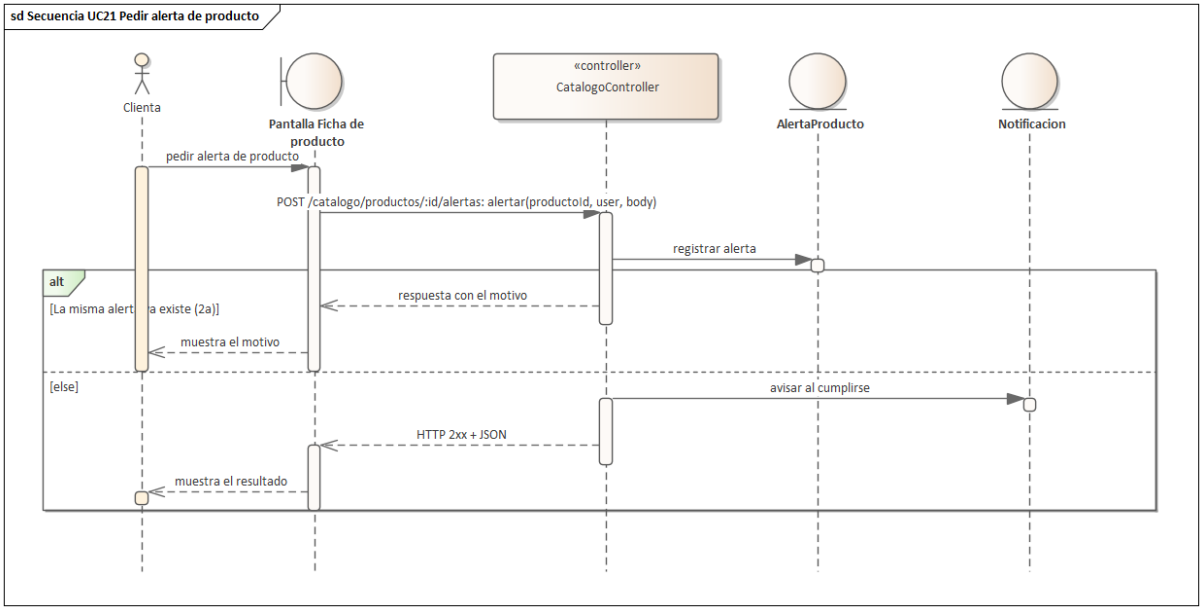


Ruta `POST /catalogo/productos/:id/opiniones`, realizada por `CatalogoController.crearOpinion`.

Secuencia UC21 Pedir alerta de producto

Figura 9.3.21

Secuencia de UC21 Pedir alerta de producto

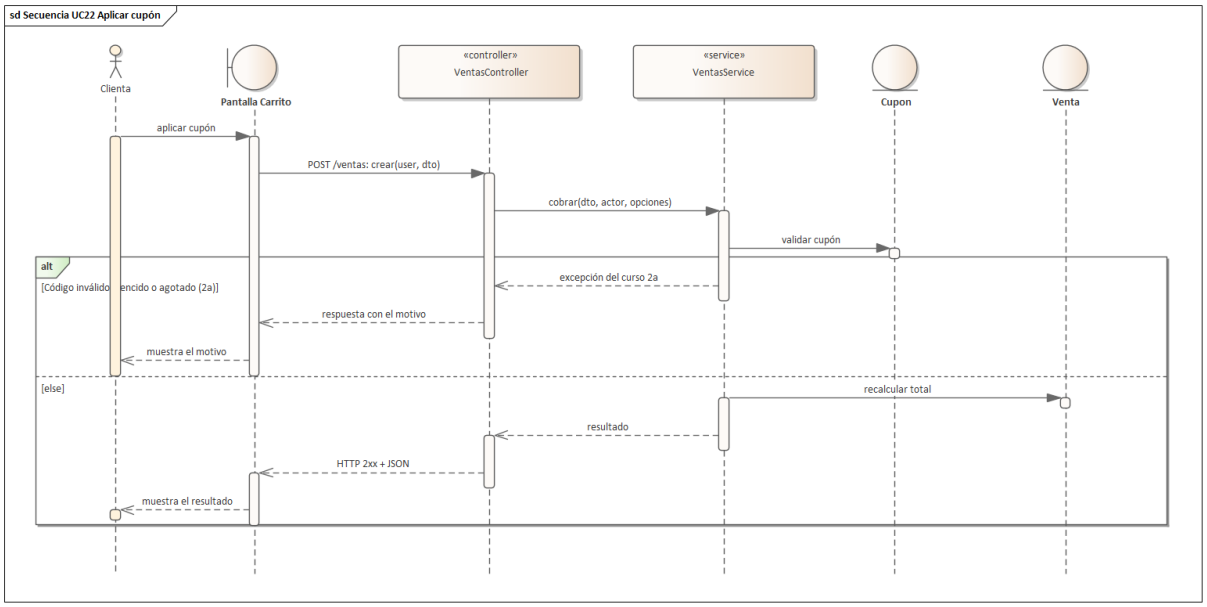


Ruta `POST /catalogo/productos/:id/alertas`, realizada por `CatalogoController.alertar`.

Secuencia UC22 Aplicar cupón

Figura 9.3.22

Secuencia de UC22 Aplicar cupón

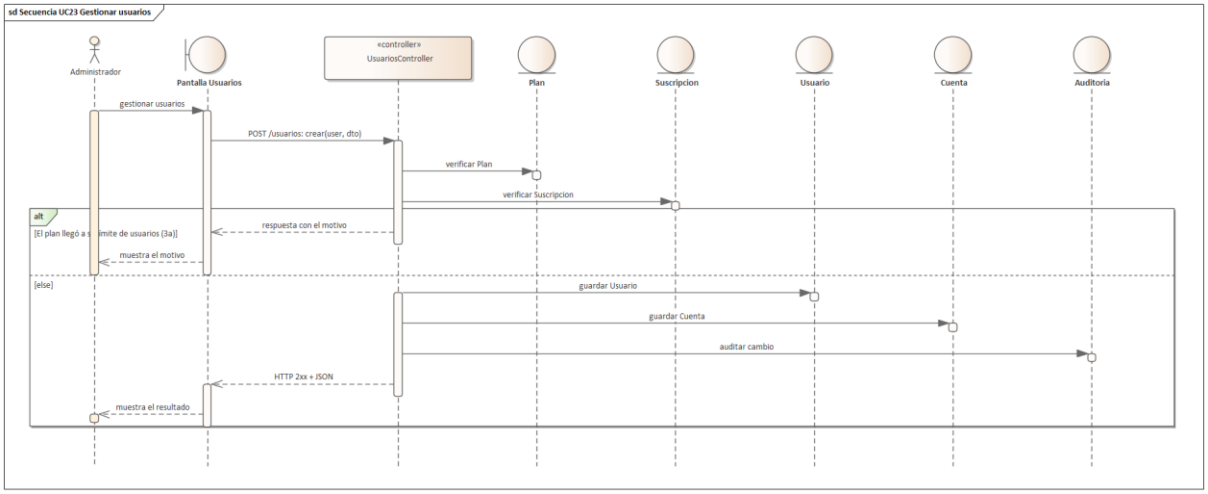


Ruta `POST /ventas`, realizada por `VentasService.cobrar`.

Secuencia UC23 Gestionar usuarios

Figura 9.3.23

Secuencia de UC23 Gestionar usuarios

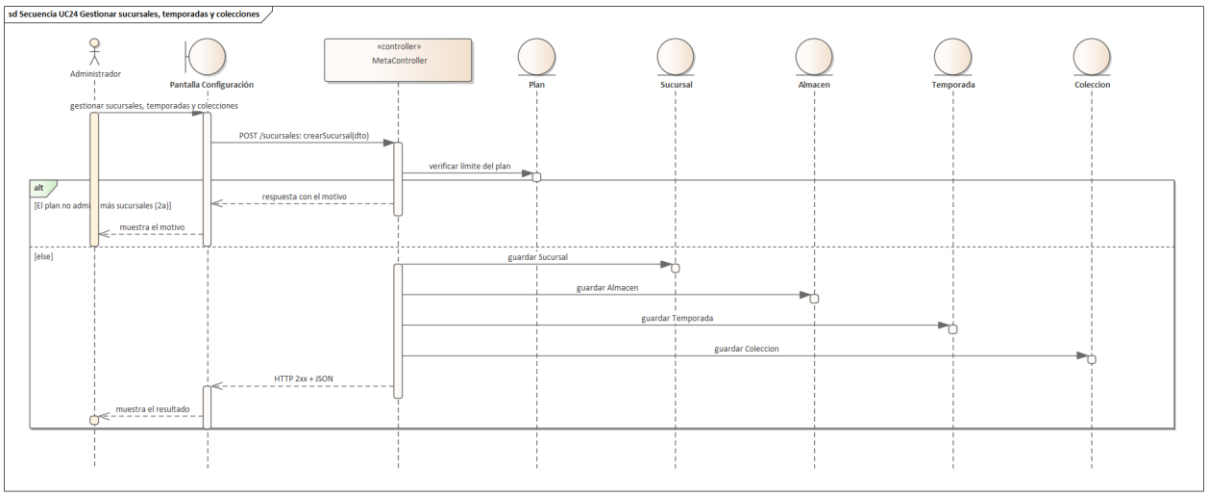


Ruta `POST /usuarios`, realizada por `UsuariosController.crear`.

Secuencia UC24 Gestionar sucursales, temporadas y colecciones

Figura 9.3.24

Secuencia de UC24 Gestionar sucursales, temporadas y colecciones

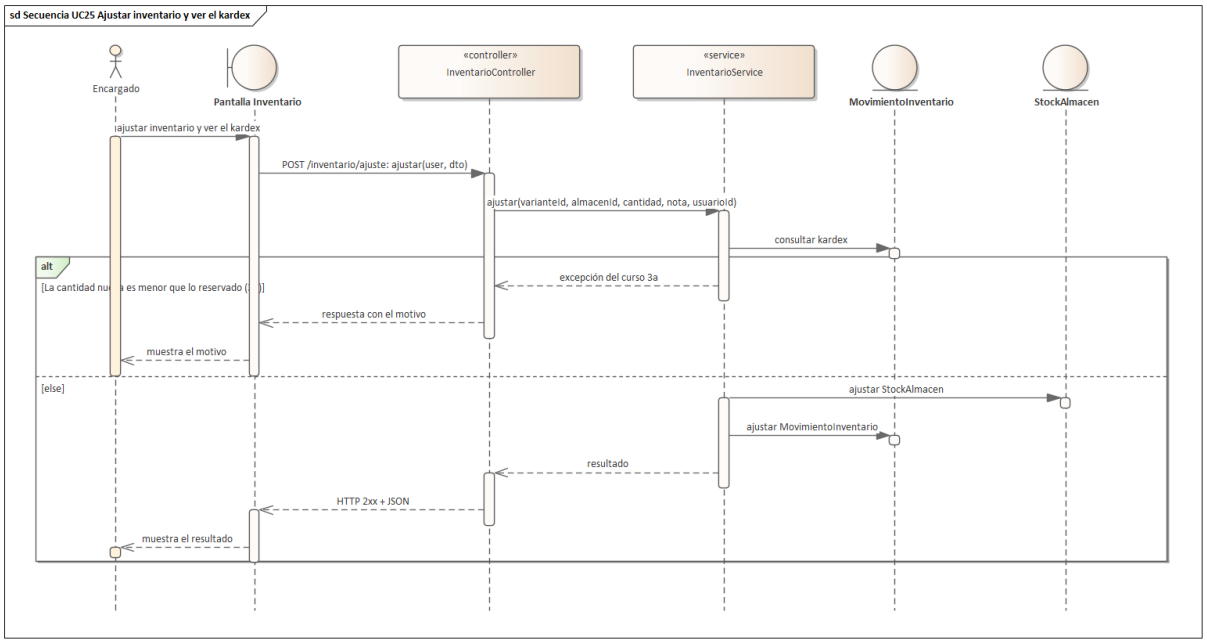


Ruta `POST /sucursales`, realizada por `MetaController.crearSucursal`.

Secuencia UC25 Ajustar inventario y ver el kardex

Figura 9.3.25

Secuencia de UC25 Ajustar inventario y ver el kardex

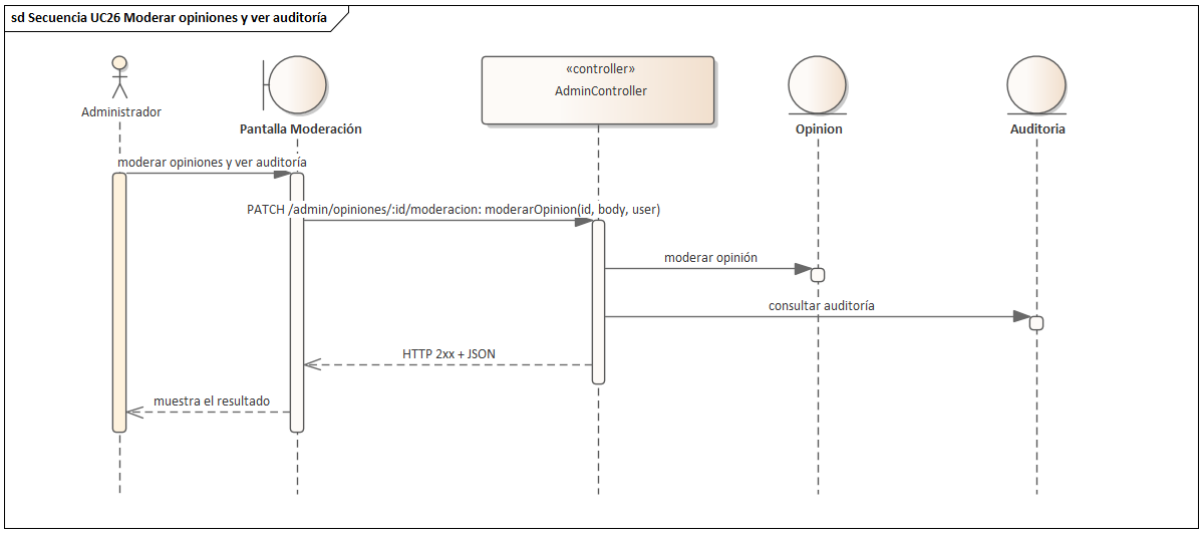


Ruta `POST /inventario/ajuste`, realizada por `InventarioService.ajustar`.

Secuencia UC26 Moderar opiniones y ver auditoría

Figura 9.3.26

Secuencia de UC26 Moderar opiniones y ver auditoría

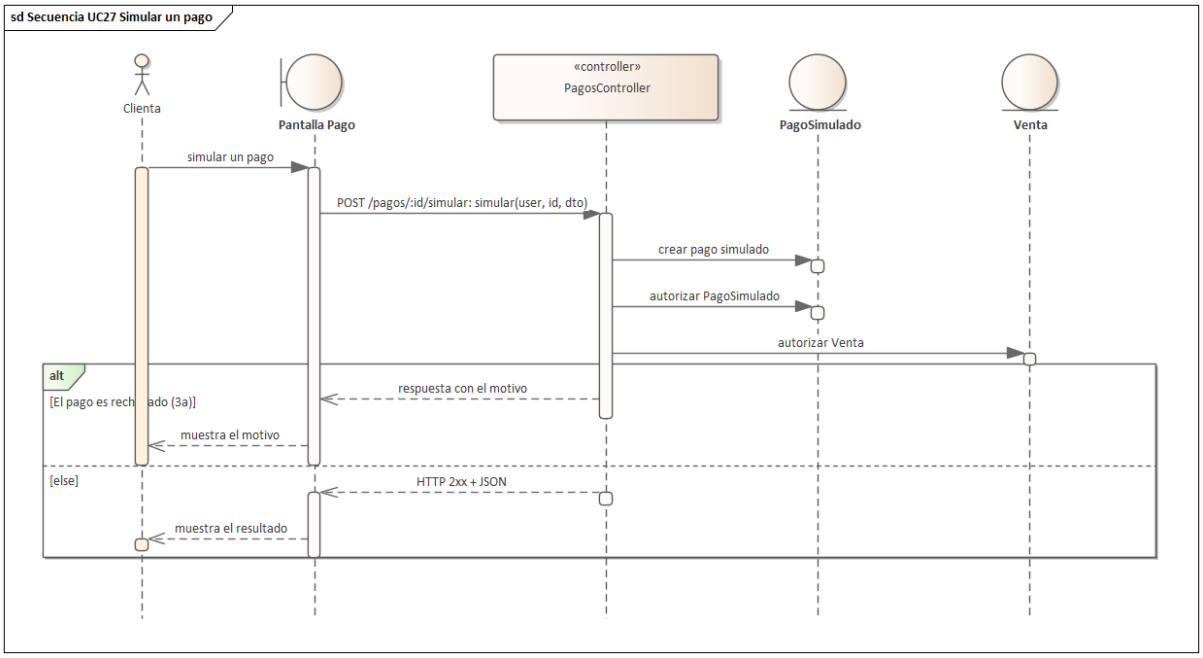


Ruta `PATCH /admin/opiniones/:id/moderacion`, realizada por `AdminController.moderarOpinion`.

Secuencia UC27 Simular un pago

Figura 9.3.27

Secuencia de UC27 Simular un pago

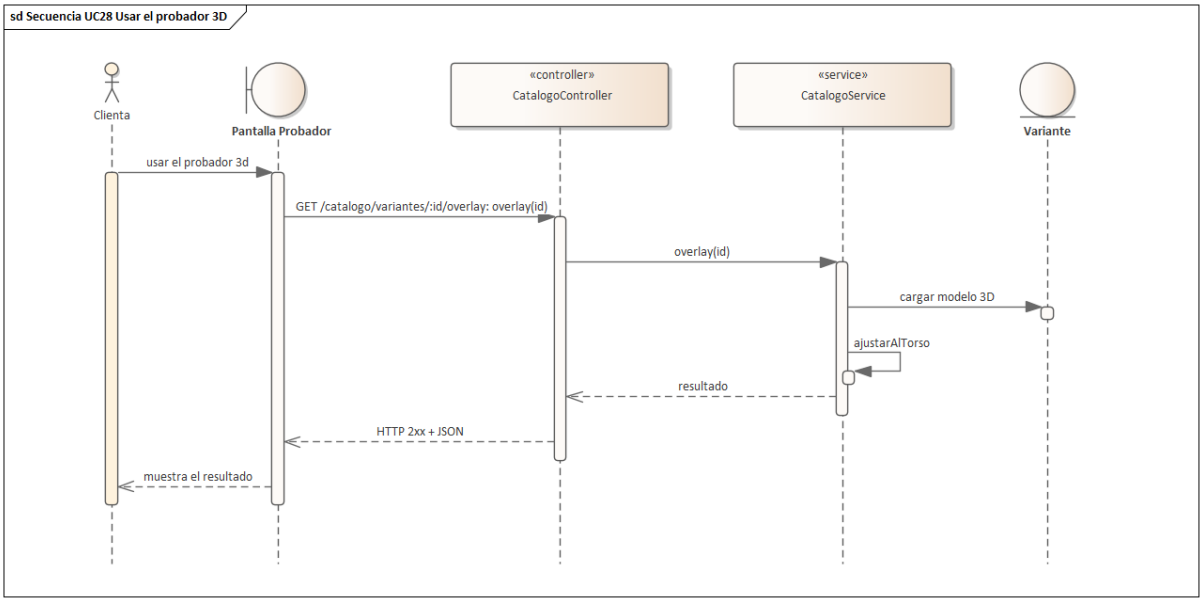


Ruta `POST /pagos/:id/simular`, realizada por `PagosController.simular`.

Secuencia UC28 Usar el probador 3D

Figura 9.3.28

Secuencia de UC28 Usar el probador 3D

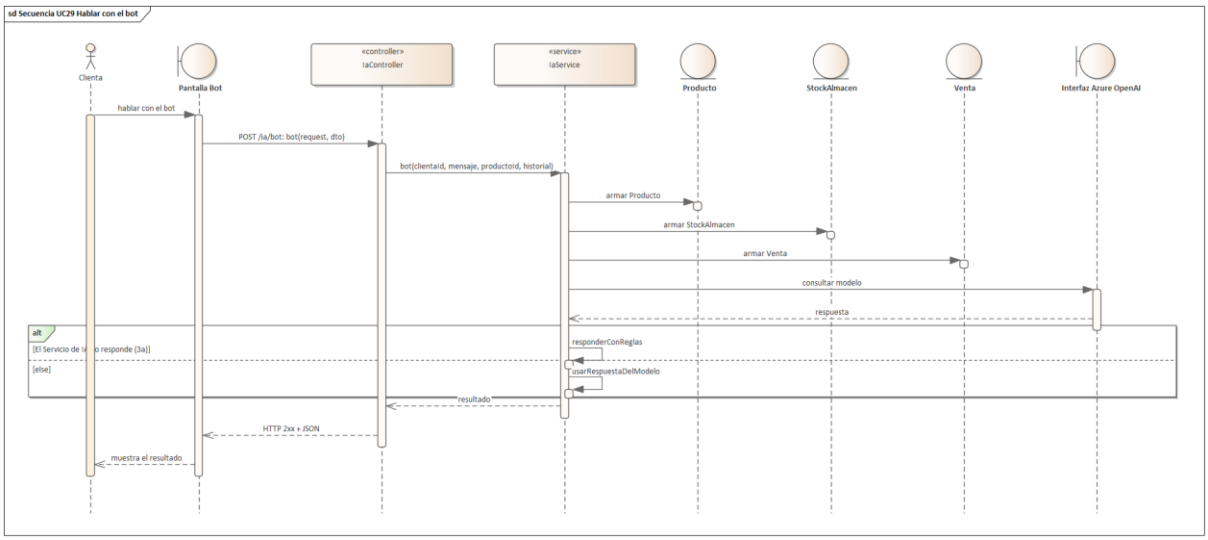


Ruta `GET /catalogo/variantes/:id/overlay`, realizada por `CatalogoService.overlay`.

Secuencia UC29 Hablar con el bot

Figura 9.3.29

Secuencia de UC29 Hablar con el bot

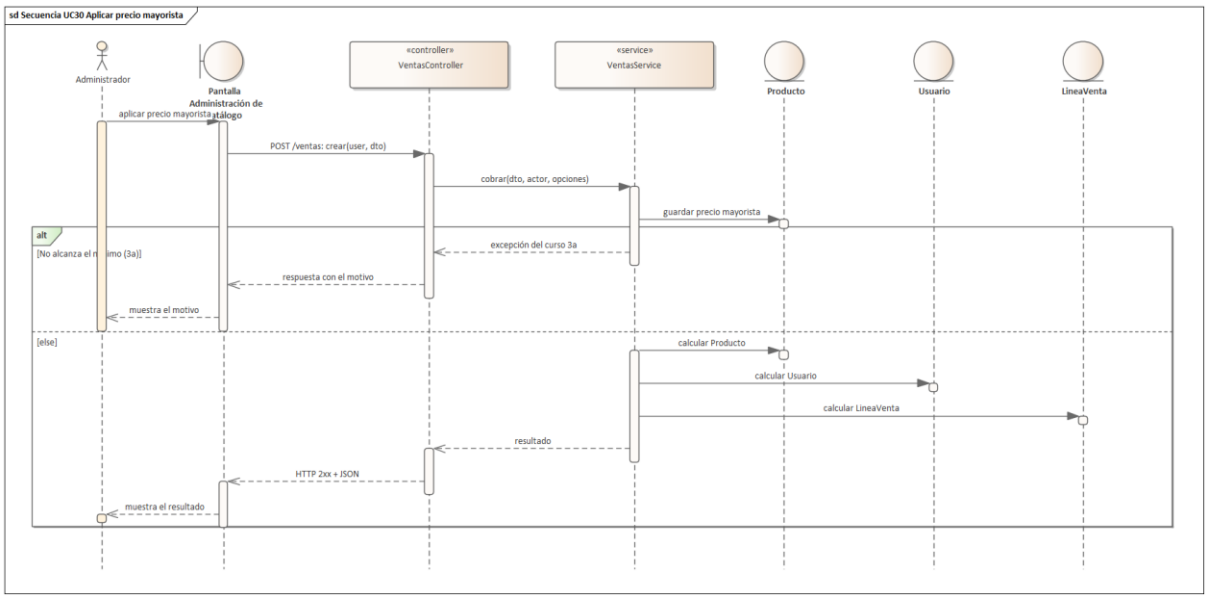


Ruta `POST /ia/bot`, realizada por `IaService.bot`.

Secuencia UC30 Aplicar precio mayorista

Figura 9.3.30

Secuencia de UC30 Aplicar precio mayorista

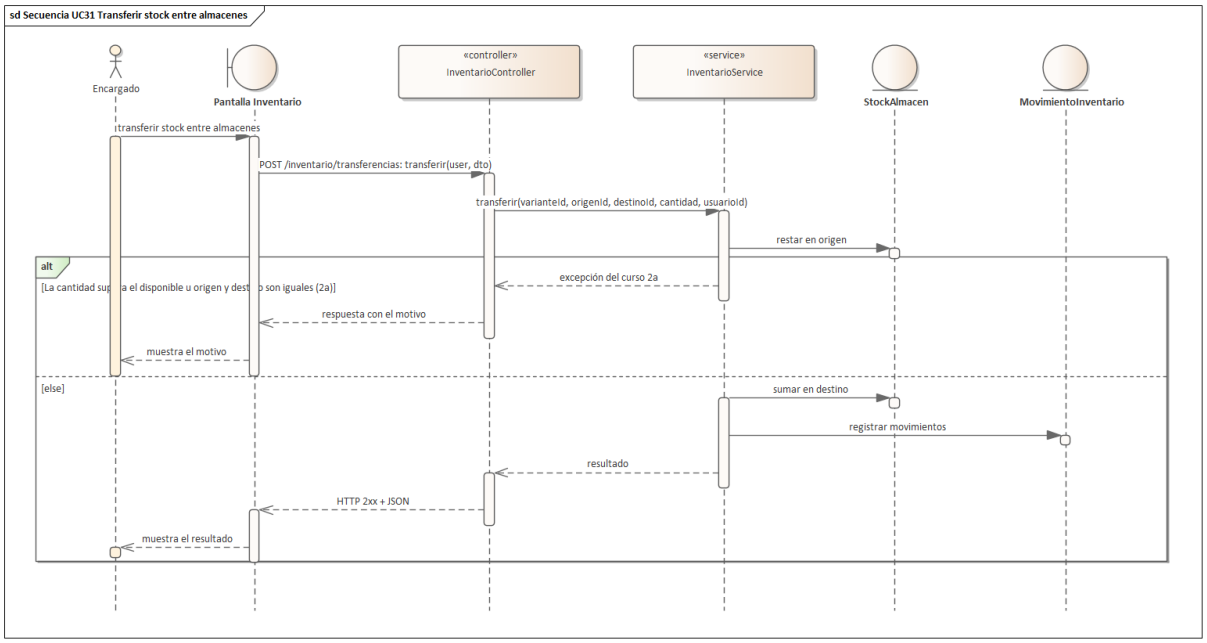


Ruta `POST /ventas`, realizada por `VentasService.cobrar`.

Secuencia UC31 Transferir stock entre almacenes

Figura 9.3.31

Secuencia de UC31 Transferir stock entre almacenes

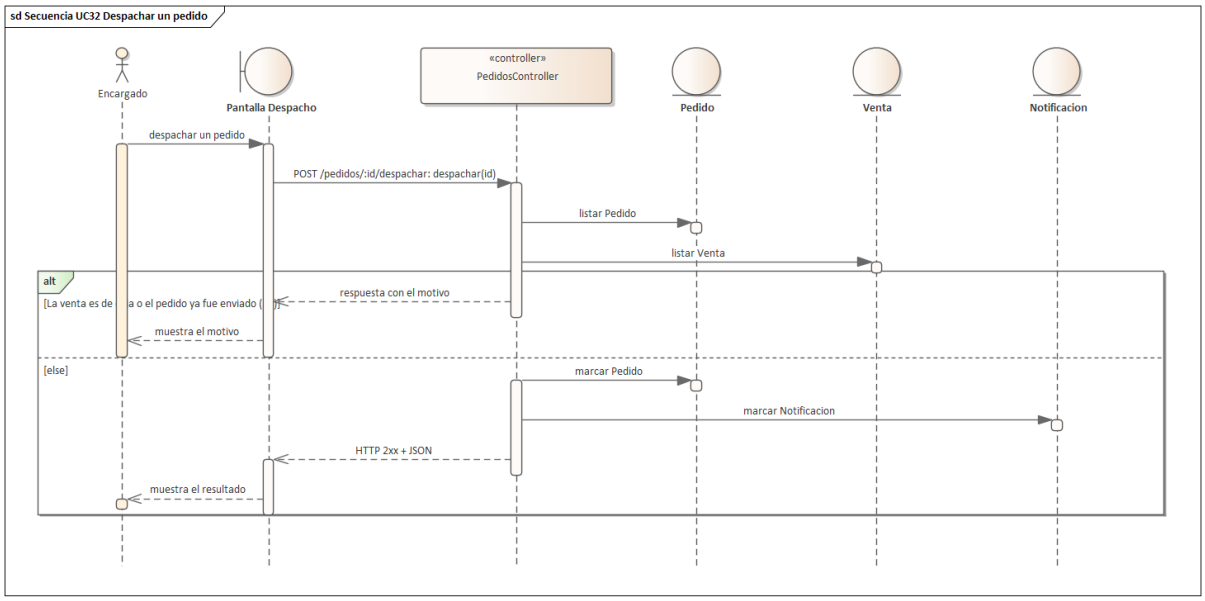


Ruta `POST /inventario/transferencias`, realizada por `InventarioService.transferir`.

Secuencia UC32 Despachar un pedido

Figura 9.3.32

Secuencia de UC32 Despachar un pedido

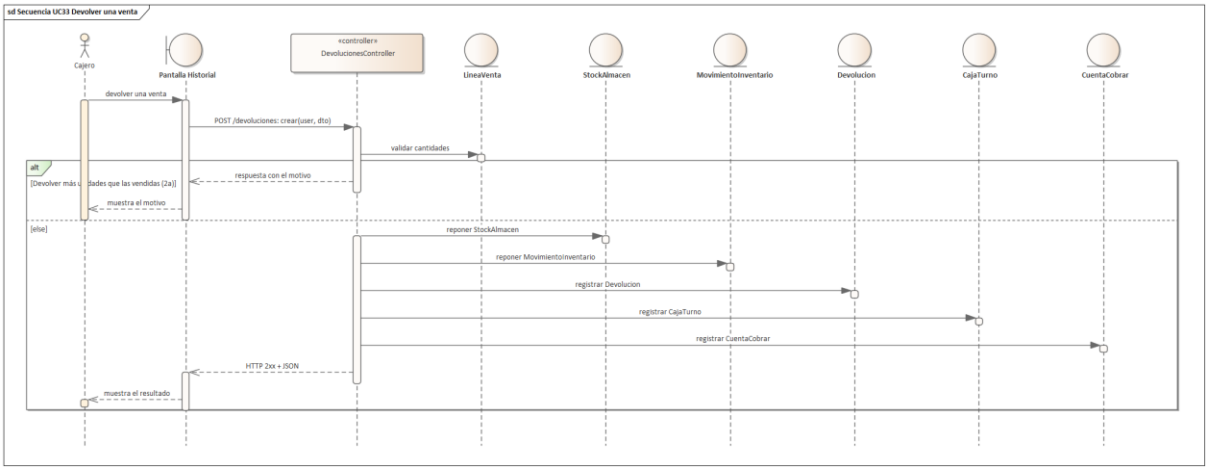


Ruta `POST /pedidos/:id/despachar`, realizada por `PedidosController.despachar`.

Secuencia UC33 Devolver una venta

Figura 9.3.33

Secuencia de UC33 Devolver una venta

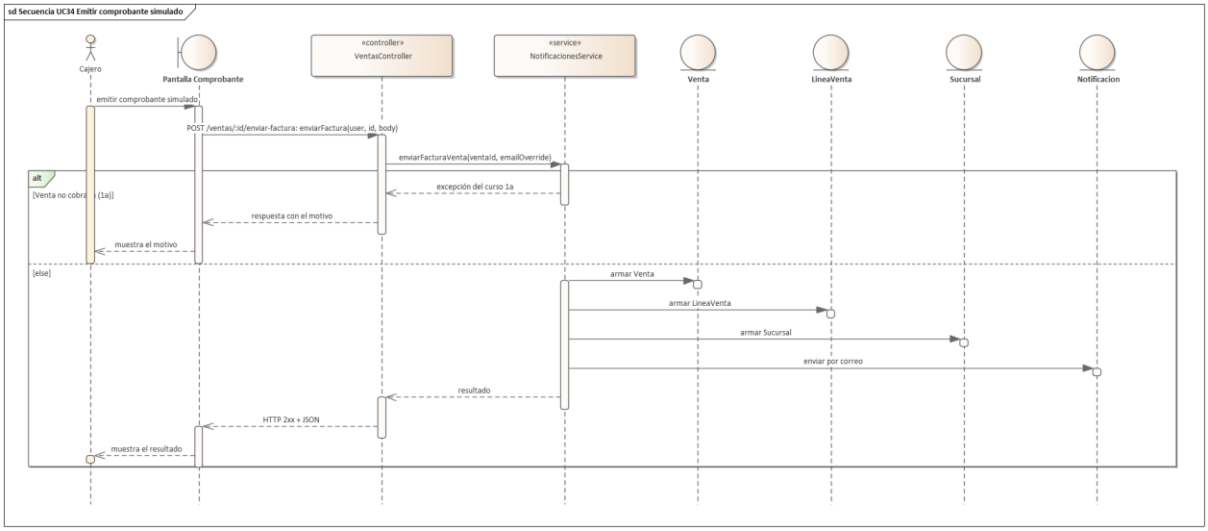


Ruta `POST /devoluciones`, realizada por `DevolucionesController.crear`.

Secuencia UC34 Emitir comprobante simulado

Figura 9.3.34

Secuencia de UC34 Emitir comprobante simulado

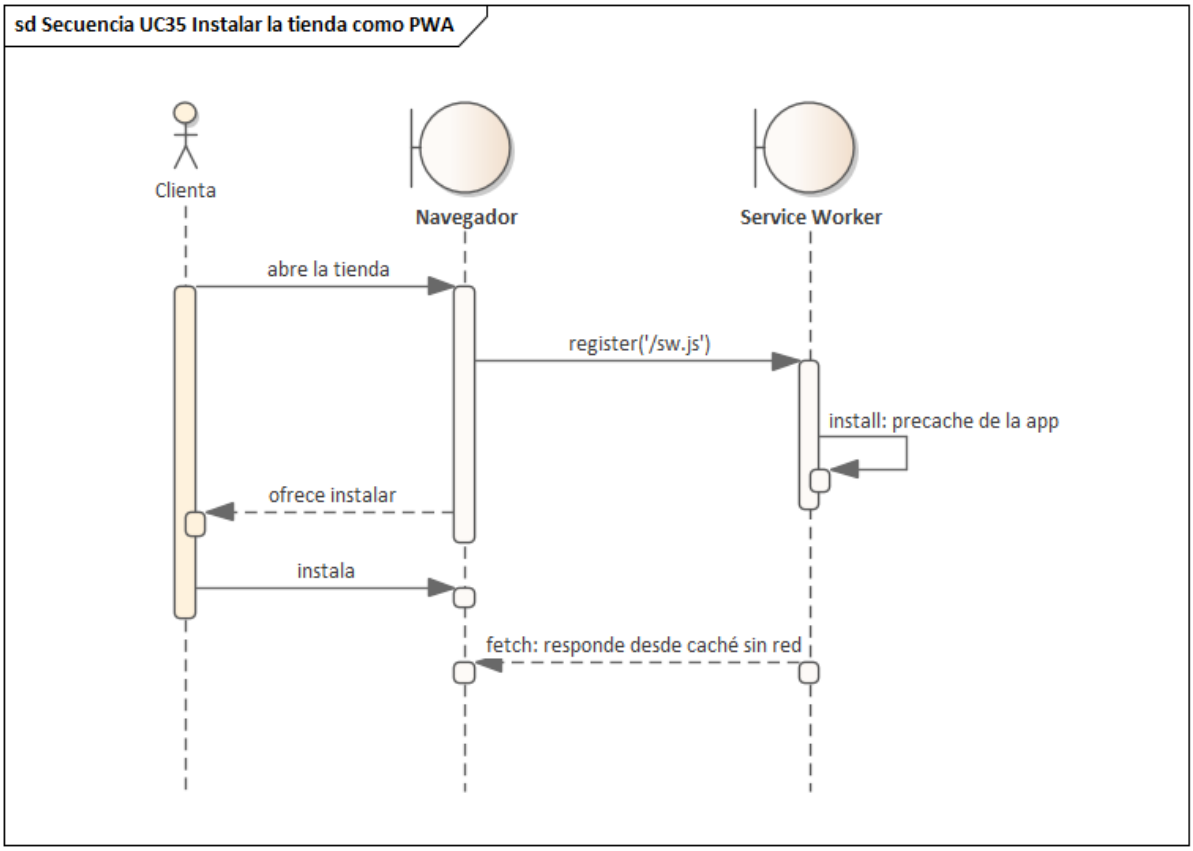


Ruta `POST /ventas/:id/enviar-factura`, realizada por `NotificacionesService.enviarFacturaVenta`.

Secuencia UC35 Instalar la tienda como PWA

Figura 9.3.35

Secuencia de UC35 Instalar la tienda como PWA

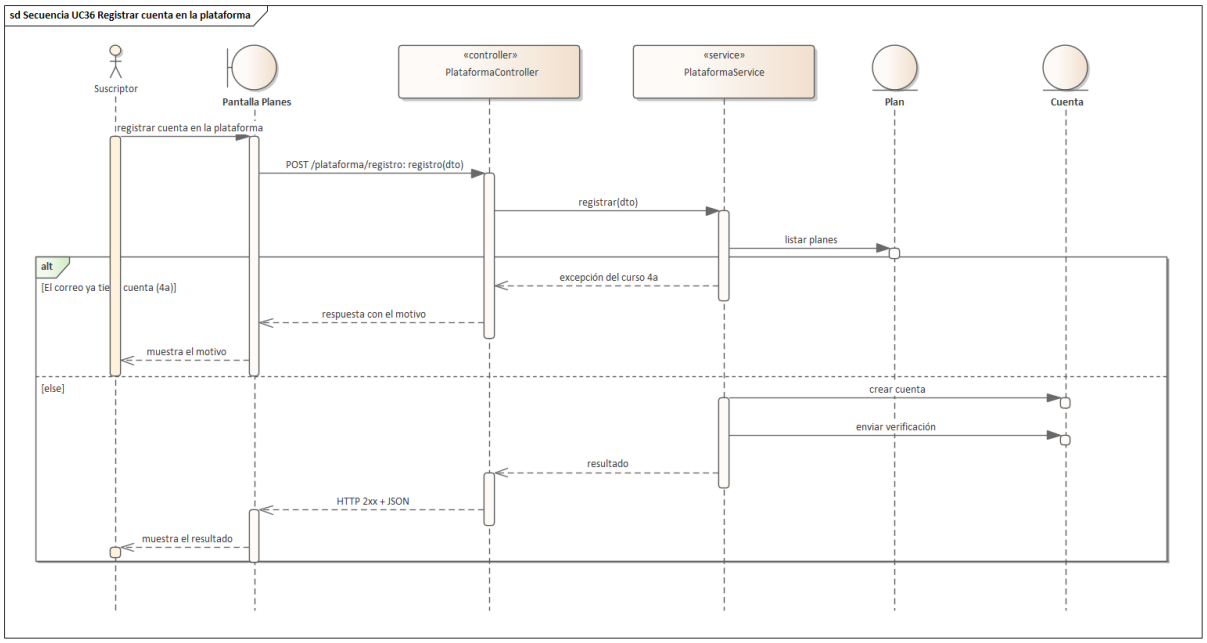


El caso se realiza en el cliente, sin pasar por la API hasta la sincronización.

Secuencia UC36 Registrar cuenta en la plataforma

Figura 9.3.36

Secuencia de UC36 Registrar cuenta en la plataforma

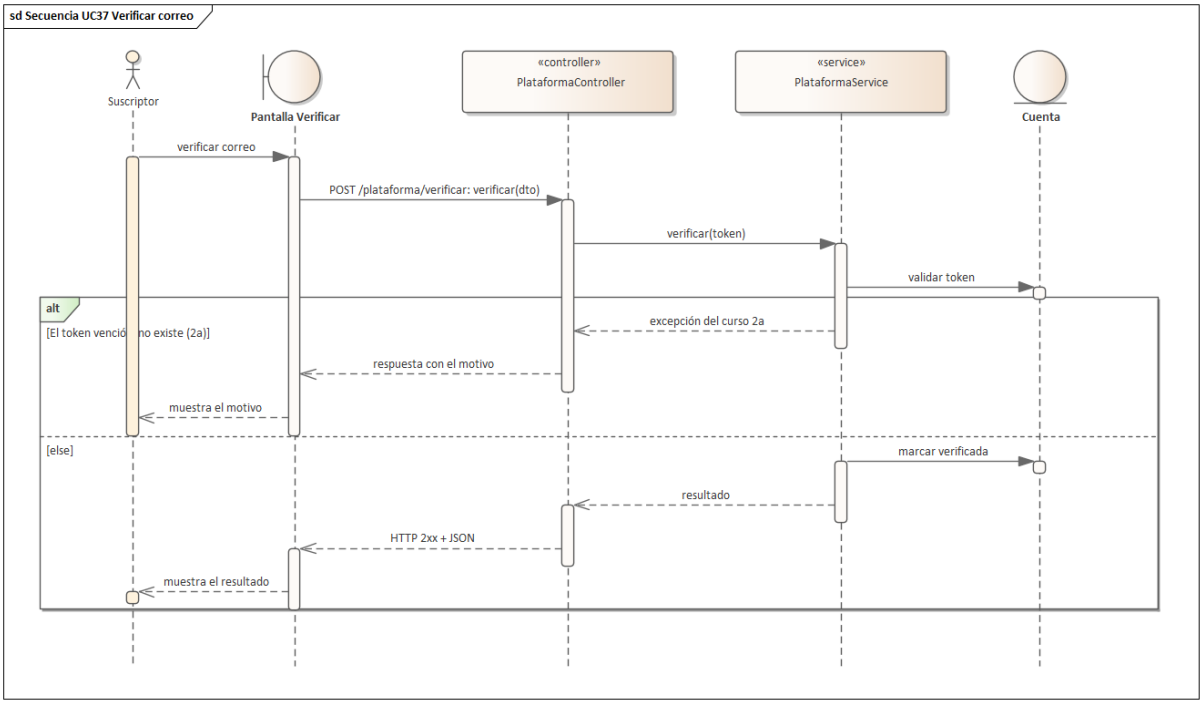


Ruta `POST /plataforma/registro`, realizada por `PlataformaService.registrar`.

Secuencia UC37 Verificar correo

Figura 9.3.37

Secuencia de UC37 Verificar correo

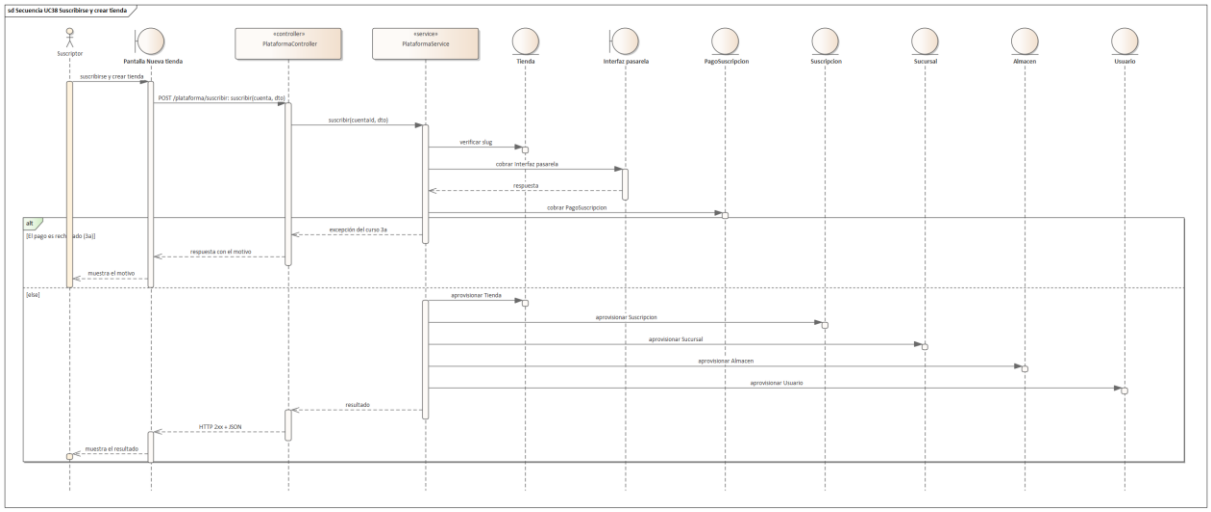


Ruta `POST /plataforma/verificar`, realizada por `PlataformaService.verificar`.

Secuencia UC38 Suscribirse y crear tienda

Figura 9.3.38

Secuencia de UC38 Suscribirse y crear tienda

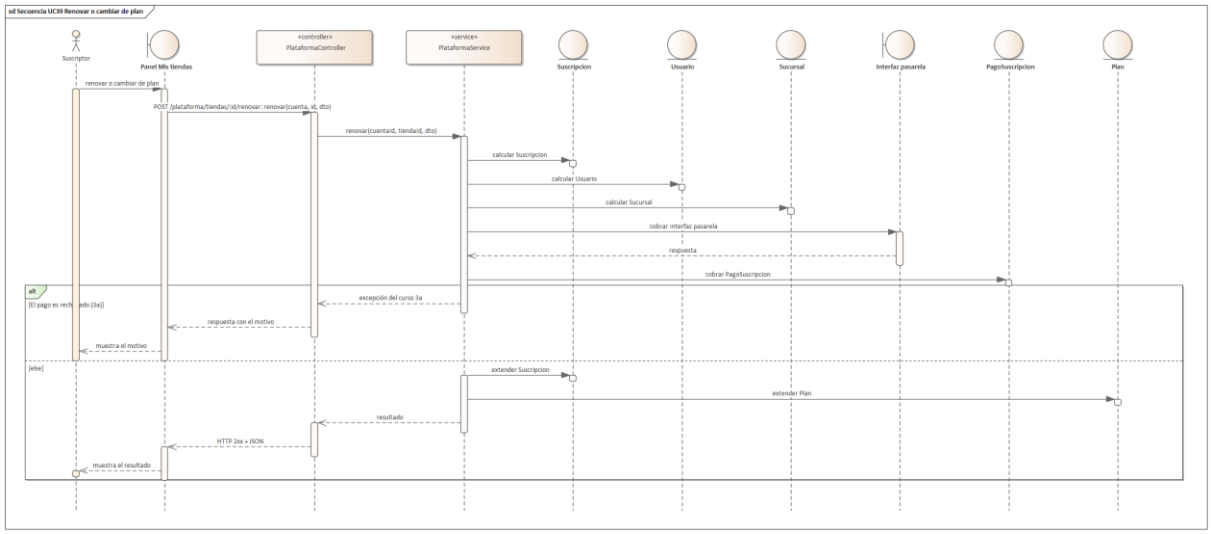


Ruta `POST /plataforma/suscribir`, realizada por `PlataformaService.suscribir`.

Secuencia UC39 Renovar o cambiar de plan

Figura 9.3.39

Secuencia de UC39 Renovar o cambiar de plan

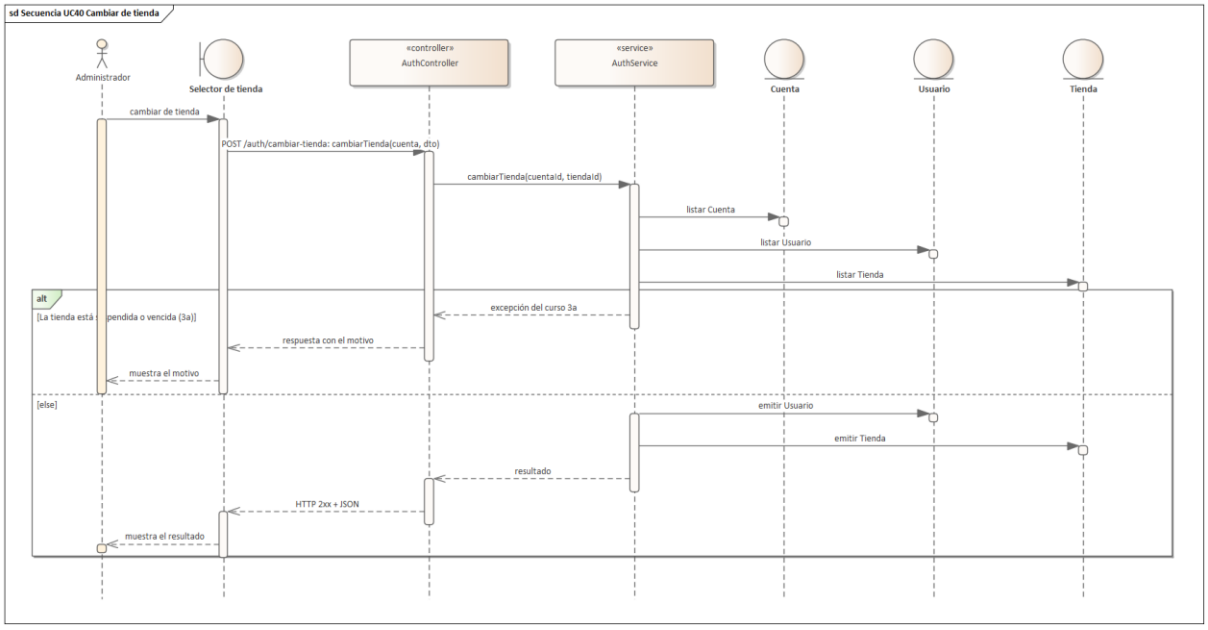


Ruta `POST /plataforma/tiendas/:id/renovar`, realizada por `PlataformaService.renovar`.

Secuencia UC40 Cambiar de tienda

Figura 9.3.40

Secuencia de UC40 Cambiar de tienda

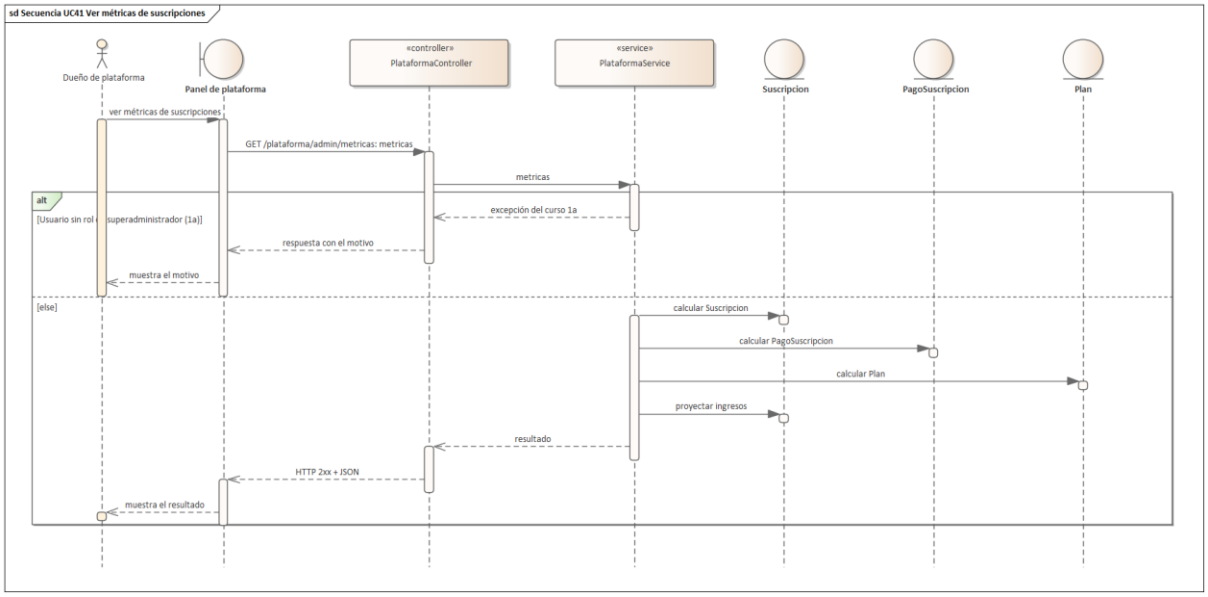


Ruta `POST /auth/cambiar-tienda`, realizada por `AuthService.cambiarTienda`.

Secuencia UC41 Ver métricas de suscripciones

Figura 9.3.41

Secuencia de UC41 Ver métricas de suscripciones



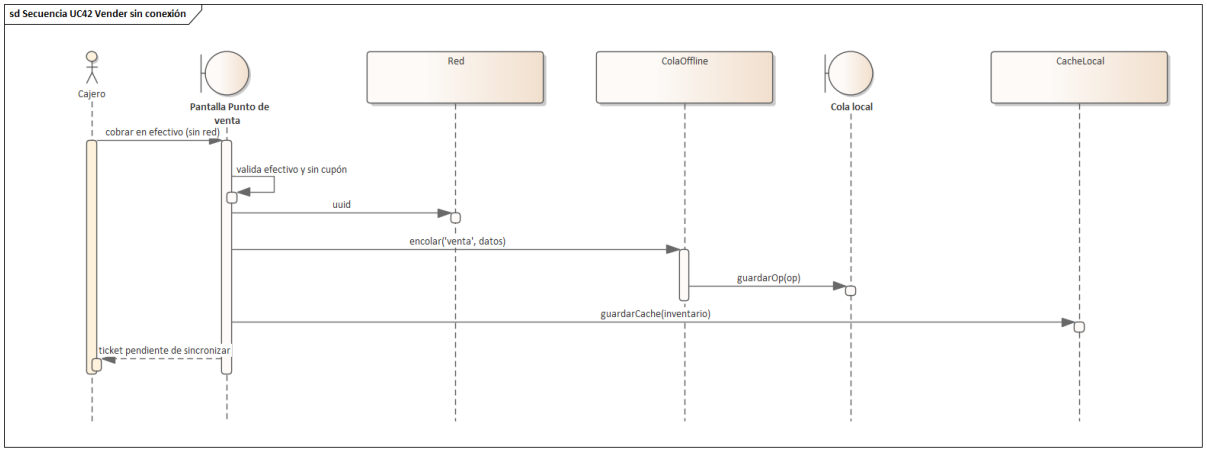
Ruta `GET /plataforma/admin/metricas`, realizada por `PlataformaService.metricas`.

Lote 2: Operación sin conexión y sincronización

Secuencia UC42 Vender sin conexión

Figura 9.3.42

Secuencia de UC42 Vender sin conexión

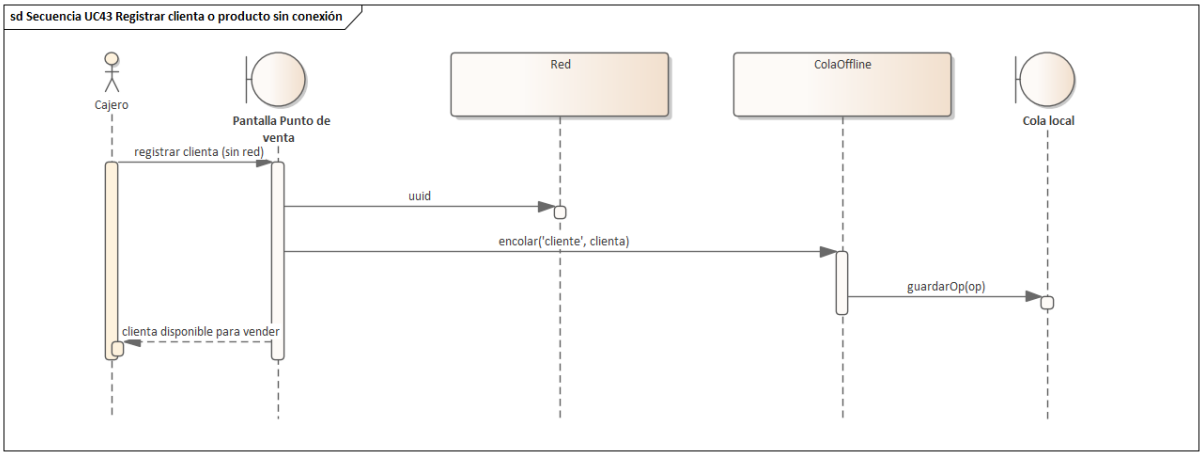


El caso se realiza en el cliente, sin pasar por la API hasta la sincronización.

Secuencia UC43 Registrar clienta o producto sin conexión

Figura 9.3.43

Secuencia de UC43 Registrar clienta o producto sin conexión

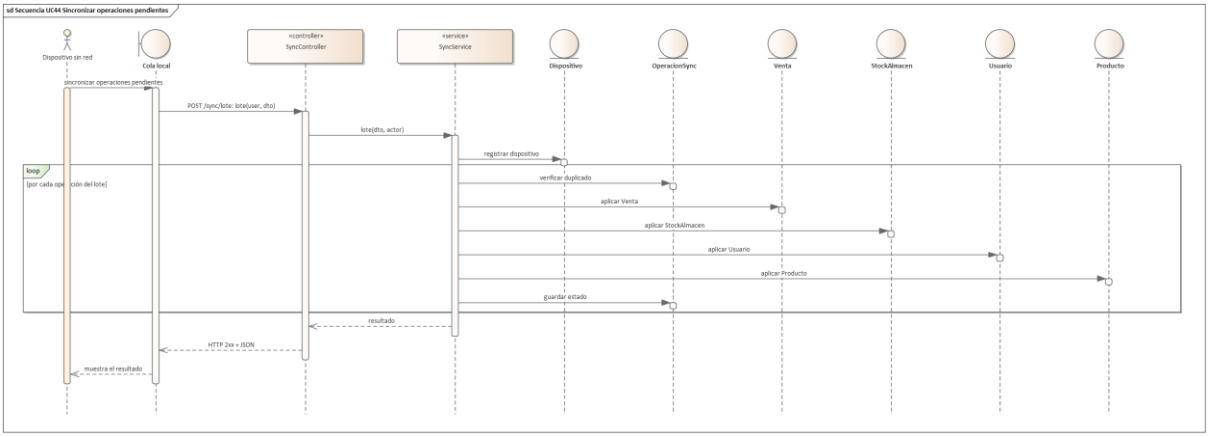


El caso se realiza en el cliente, sin pasar por la API hasta la sincronización.

Secuencia UC44 Sincronizar operaciones pendientes

Figura 9.3.44

Secuencia de UC44 Sincronizar operaciones pendientes

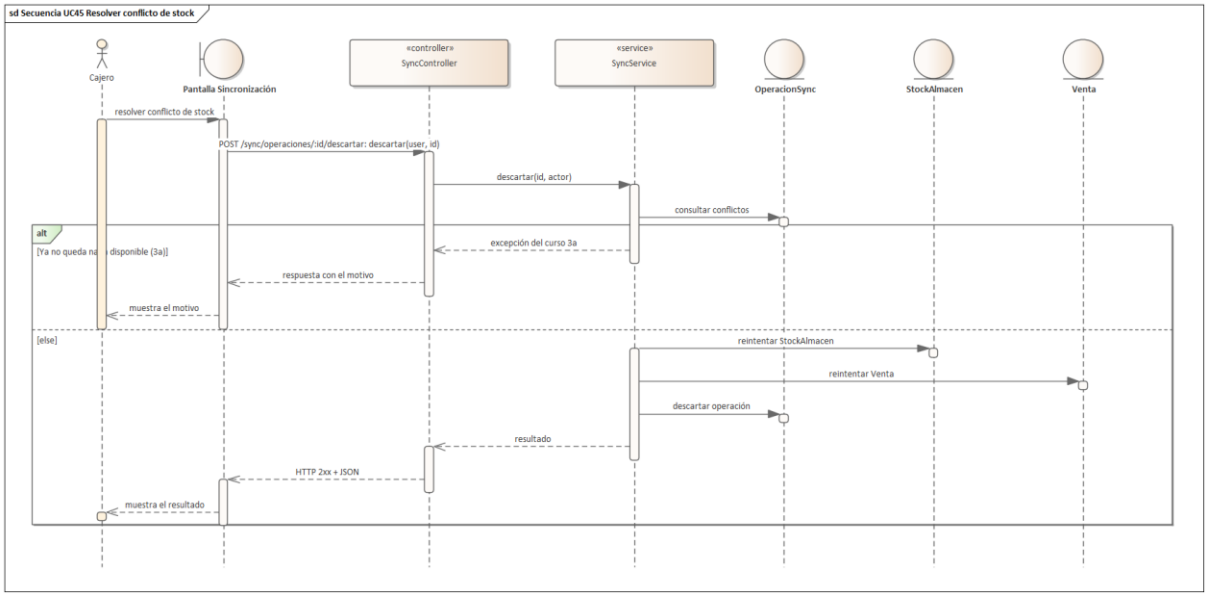


Ruta `POST /sync/lote`, realizada por `SyncService.lote`.

Secuencia UC45 Resolver conflicto de stock

Figura 9.3.45

Secuencia de UC45 Resolver conflicto de stock

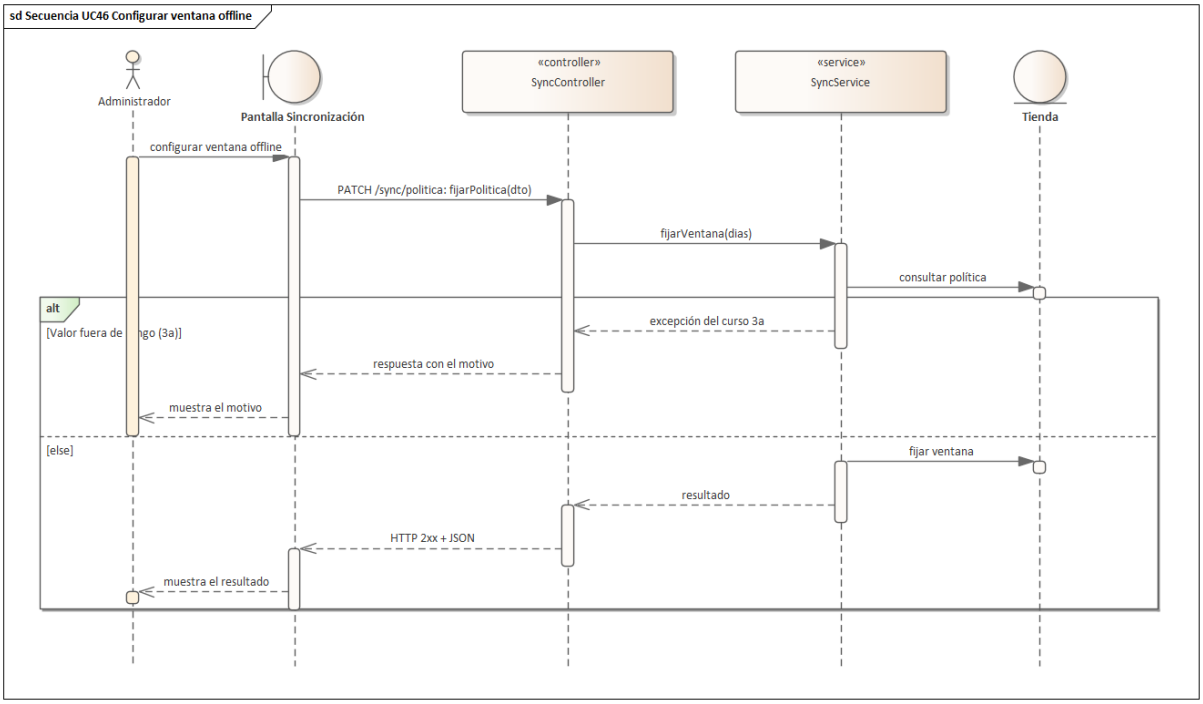


Ruta `POST /sync/operaciones/:id/descartar`, realizada por `SyncService.descartar`.

Secuencia UC46 Configurar ventana offline

Figura 9.3.46

Secuencia de UC46 Configurar ventana offline

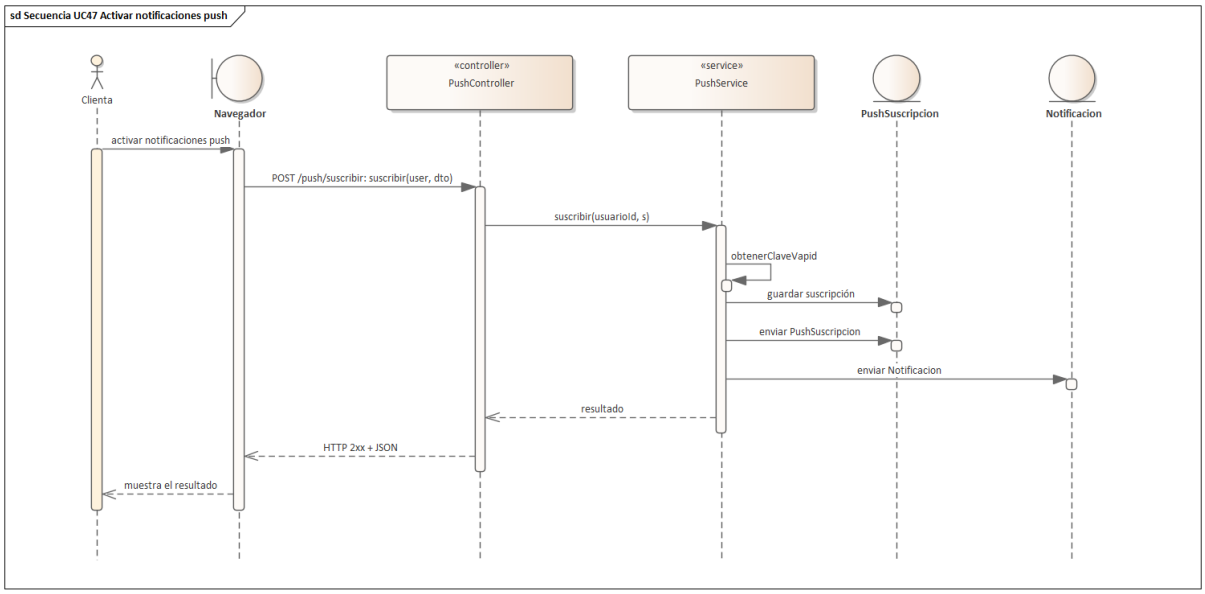


Ruta `PATCH /sync/politica`, realizada por `SyncService.fijarVentana`.

Secuencia UC47 Activar notificaciones push

Figura 9.3.47

Secuencia de UC47 Activar notificaciones push



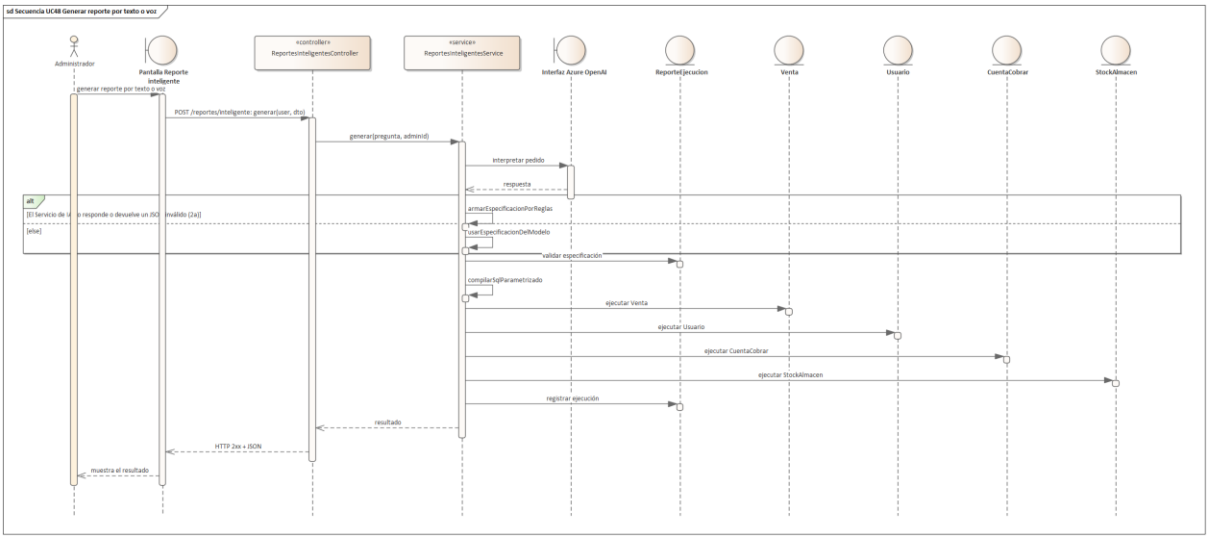
Ruta `POST /push/suscribir`, realizada por `PushService.suscribir`.

Lote 3: Reportes generativos y ventas al crédito

Secuencia UC48 Generar reporte por texto o voz

Figura 9.3.48

Secuencia de UC48 Generar reporte por texto o voz

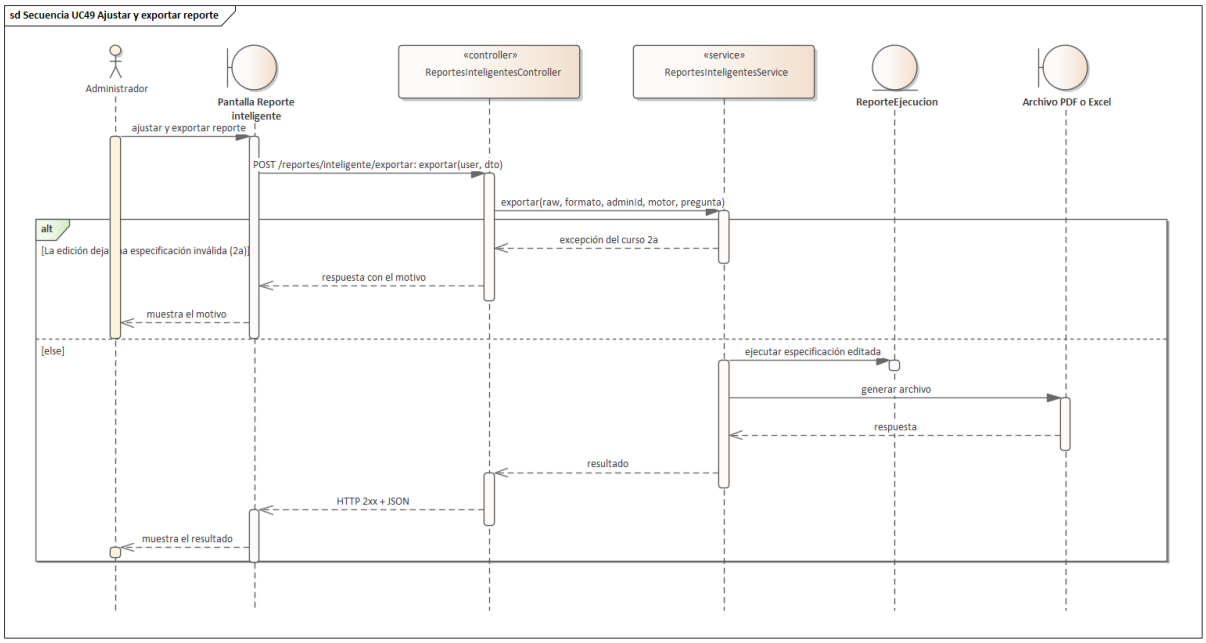


Ruta `POST /reportes/inteligente`, realizada por `ReportesInteligentesService.generar`.

Secuencia UC49 Ajustar y exportar reporte

Figura 9.3.49

Secuencia de UC49 Ajustar y exportar reporte

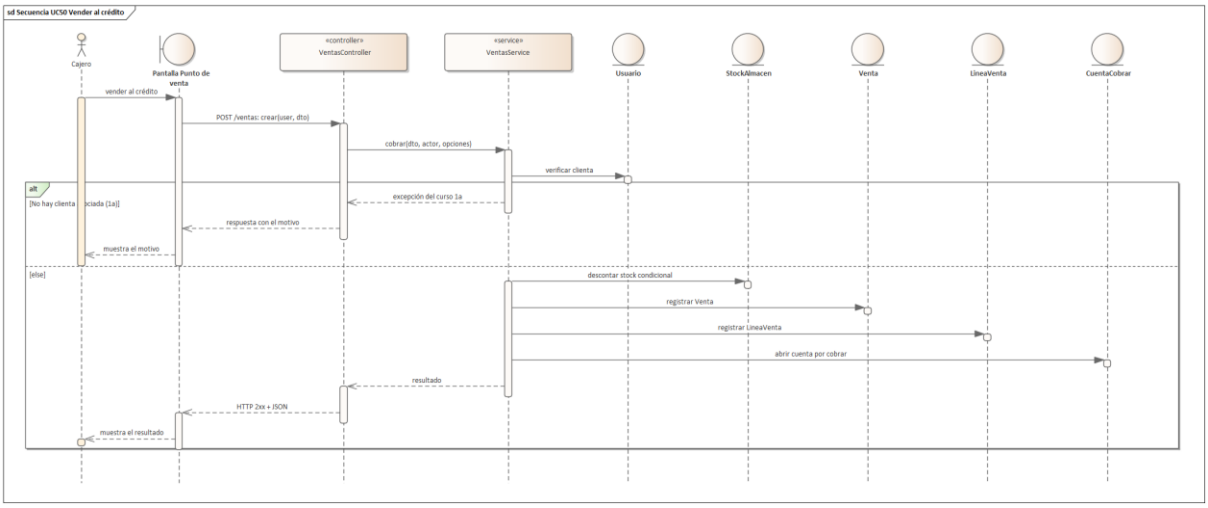


Ruta `POST /reportes/inteligente/exportar`, realizada por `ReportesInteligentesService.exportar`.

Secuencia UC50 Vender al crédito

Figura 9.3.50

Secuencia de UC50 Vender al crédito

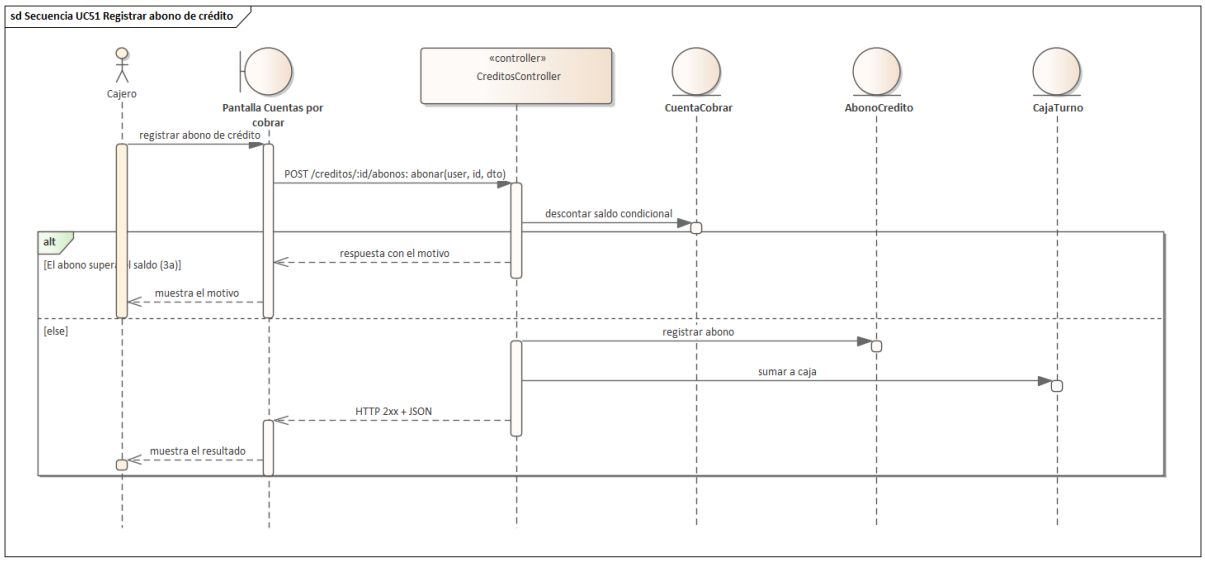


Ruta `POST /ventas`, realizada por `VentasService.cobrar`.

Secuencia UC51 Registrar abono de crédito

Figura 9.3.51

Secuencia de UC51 Registrar abono de crédito



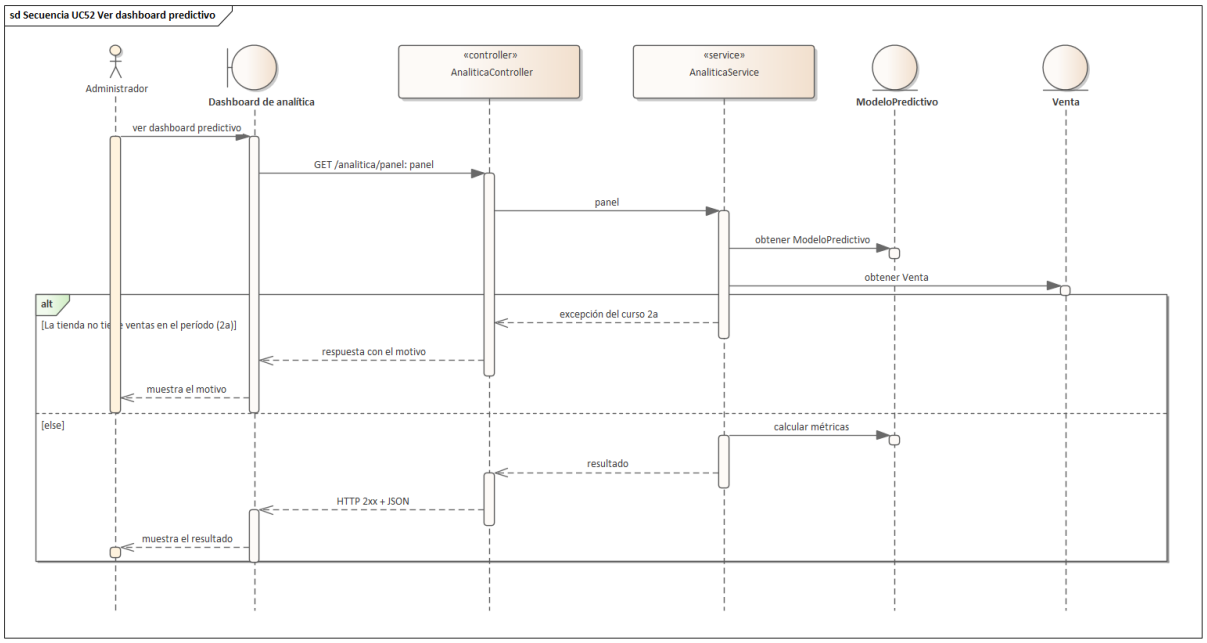
Ruta `POST /creditos/:id/abonos`, realizada por `CreditosController.abonar`.

Lote 4: Analítica con aprendizaje automático

Secuencia UC52 Ver dashboard predictivo

Figura 9.3.52

Secuencia de UC52 Ver dashboard predictivo

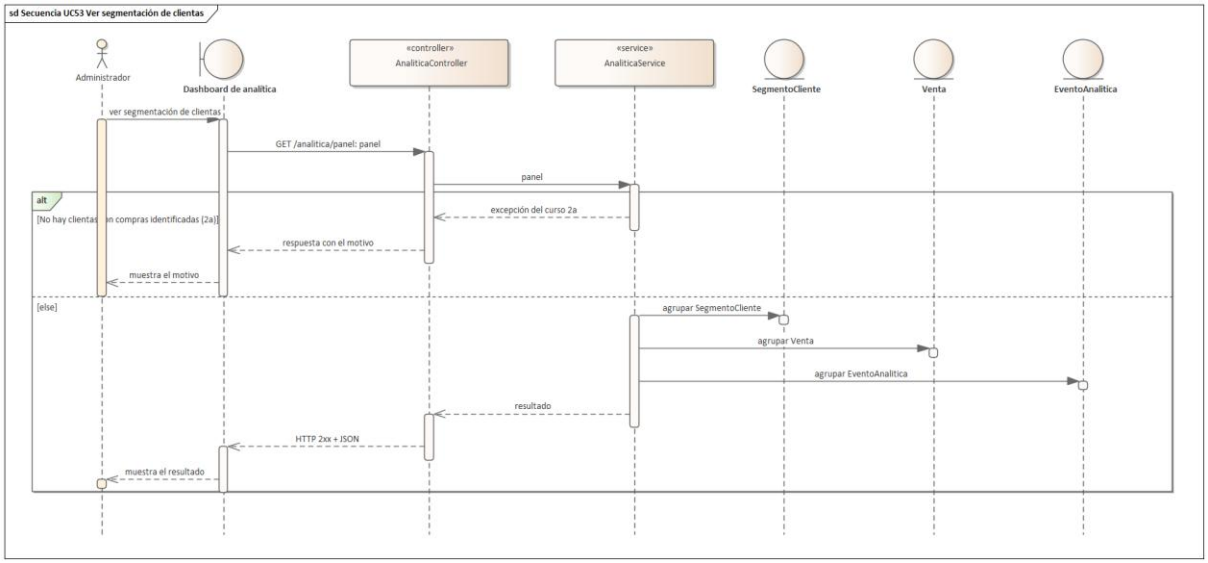


Ruta `GET /analitica/panel`, realizada por `AnaliticaService.panel`.

Secuencia UC53 Ver segmentación de clientas

Figura 9.3.53

Secuencia de UC53 Ver segmentación de clientas



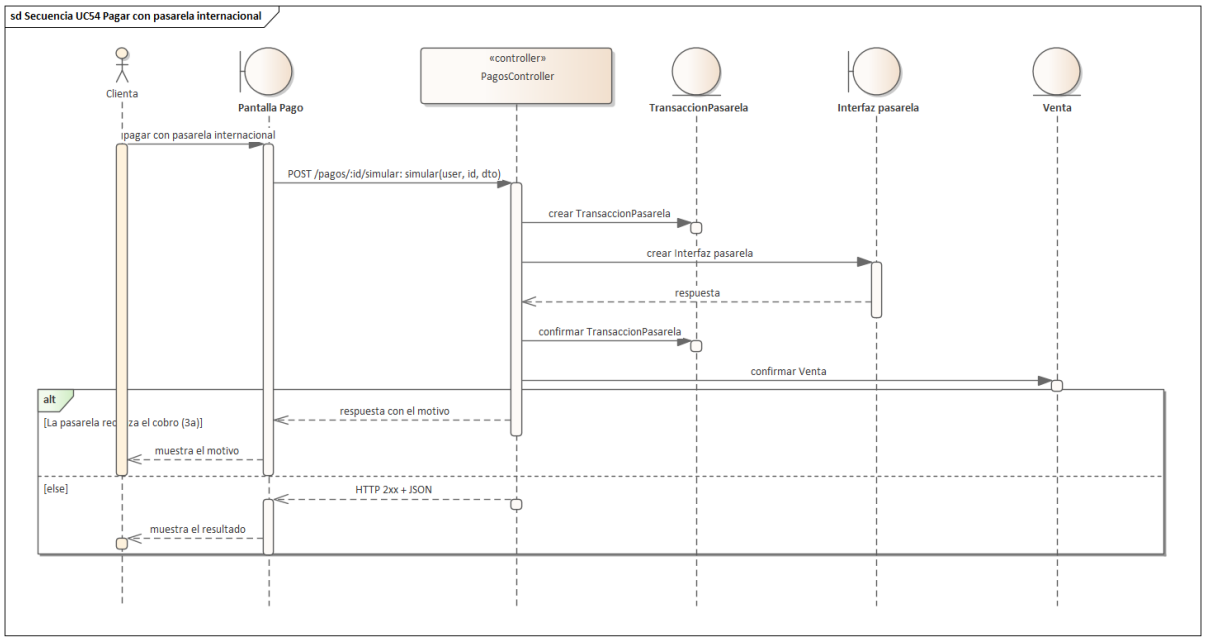
Ruta `GET /analitica/panel`, realizada por `AnaliticaService.panel`.

Lote 5: Pagos para países hispanohablantes

Secuencia UC54 Pagar con pasarela internacional

Figura 9.3.54

Secuencia de UC54 Pagar con pasarela internacional



Ruta `POST /pagos/:id/simular`, realizada por `PagosController.simular`.

9.3.1 Modelo estático de diseño

Al dibujar cada mensaje, la operación se agregó a la clase que lo recibe. El resultado es el modelo de clases de diseño por módulo: arriba, los controllers y servicios con sus operaciones; abajo, las clases del dominio de las que dependen. Las firmas no se escribieron a mano: el generador del modelo lee el código TypeScript del backend, así que cada clase muestra lo que realmente existe.

Elemento del diagrama	Notación UML 2.5	Ejemplo
Clase con estereotipo	«controller» o «service» sobre el nombre	«service» SyncService
Atributo	Visibilidad, nombre, tipo y propiedad	`- sync: SyncService {readOnly}` (dependencia inyectada por el constructor)

Elemento del diagrama	Notación UML 2.5	Ejemplo
Operación	Visibilidad, nombre, parámetros con tipo	`+ lote(user: JwtPayload, dto: LoteSyncDto)`
Asociación navegable	Línea con flecha, nombre de rol y multiplicidad en el extremo	SyncController → SyncService, rol `-sync`, multiplicidad 1
Dependencia de uso	Línea punteada con flecha y «use»	SyncService ··> OperacionSync
Asociación del dominio	Línea con multiplicidades en ambos extremos	Venta 0..* — 0..1 Usuario (rol clienta)

Figura 9.3.55

Clases de diseño del módulo Catálogo

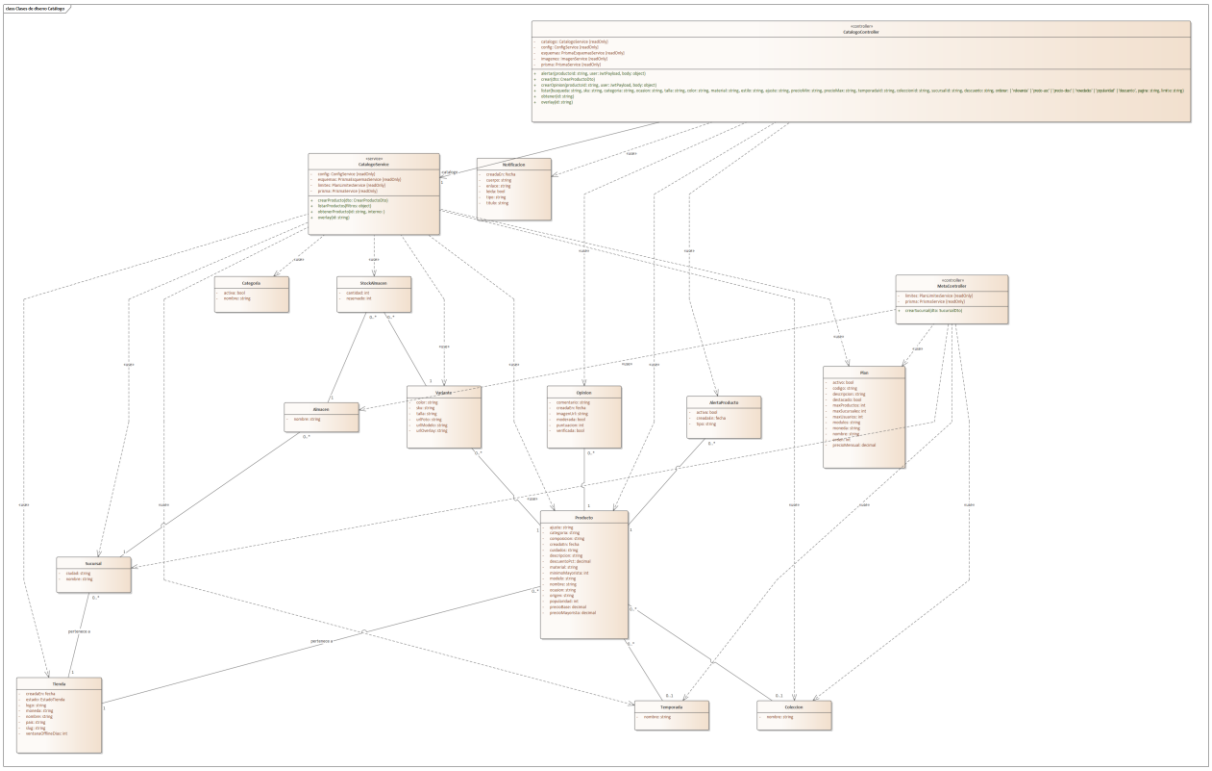


Figura 9.3.56

Clases de diseño del módulo Bot

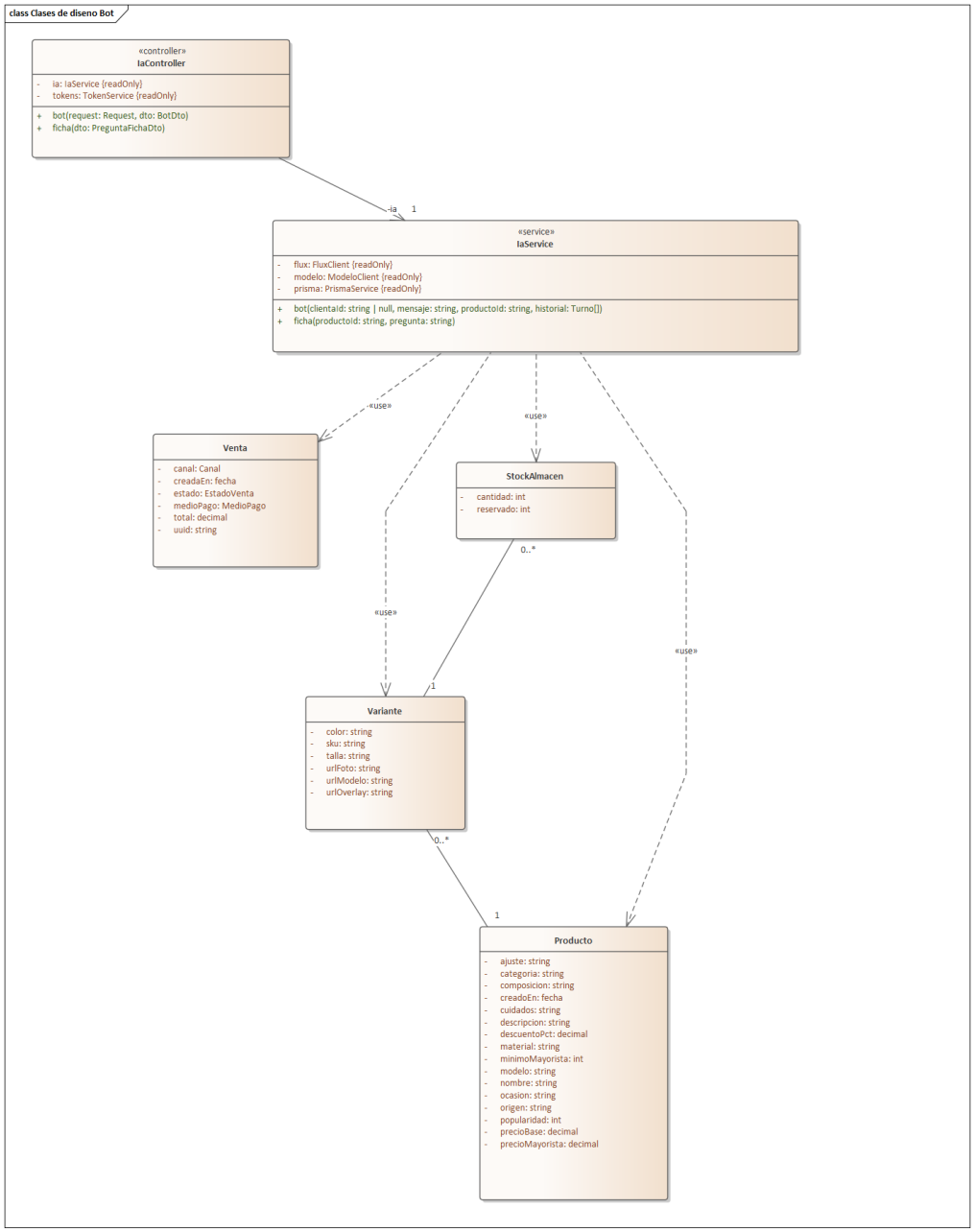


Figura 9.3.57

Clases de diseño del módulo Ventas

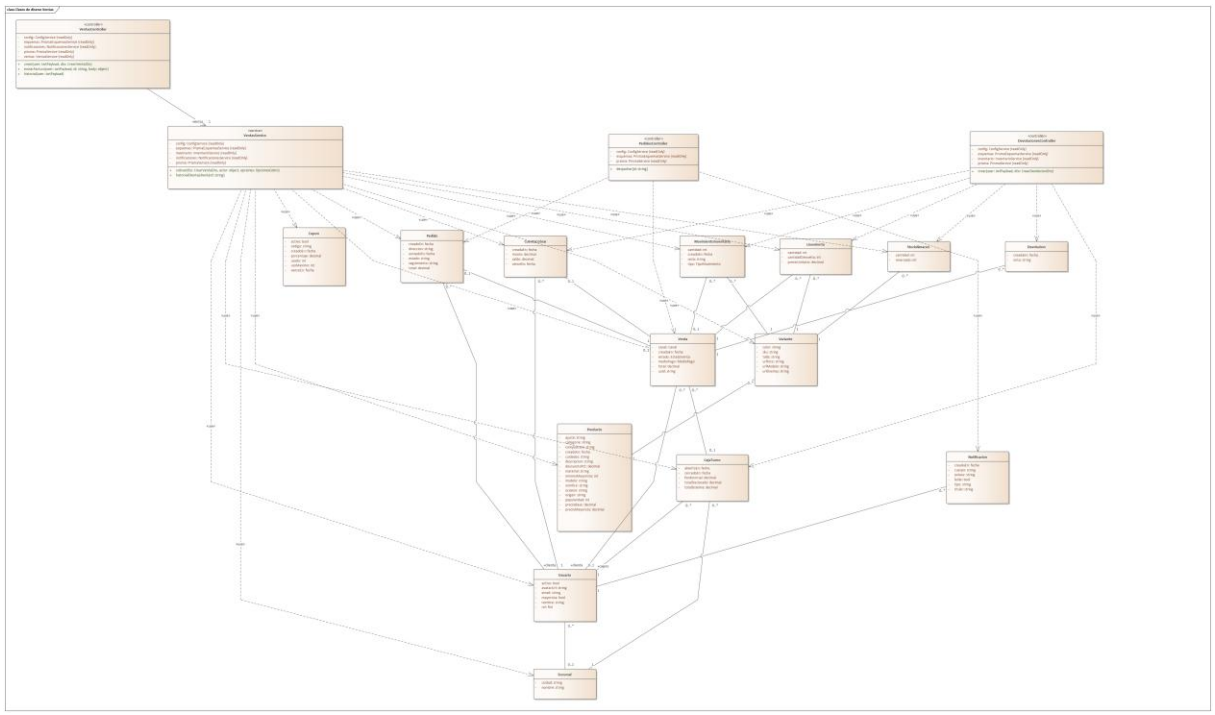


Figura 9.3.58

Clases de diseño del módulo Caja

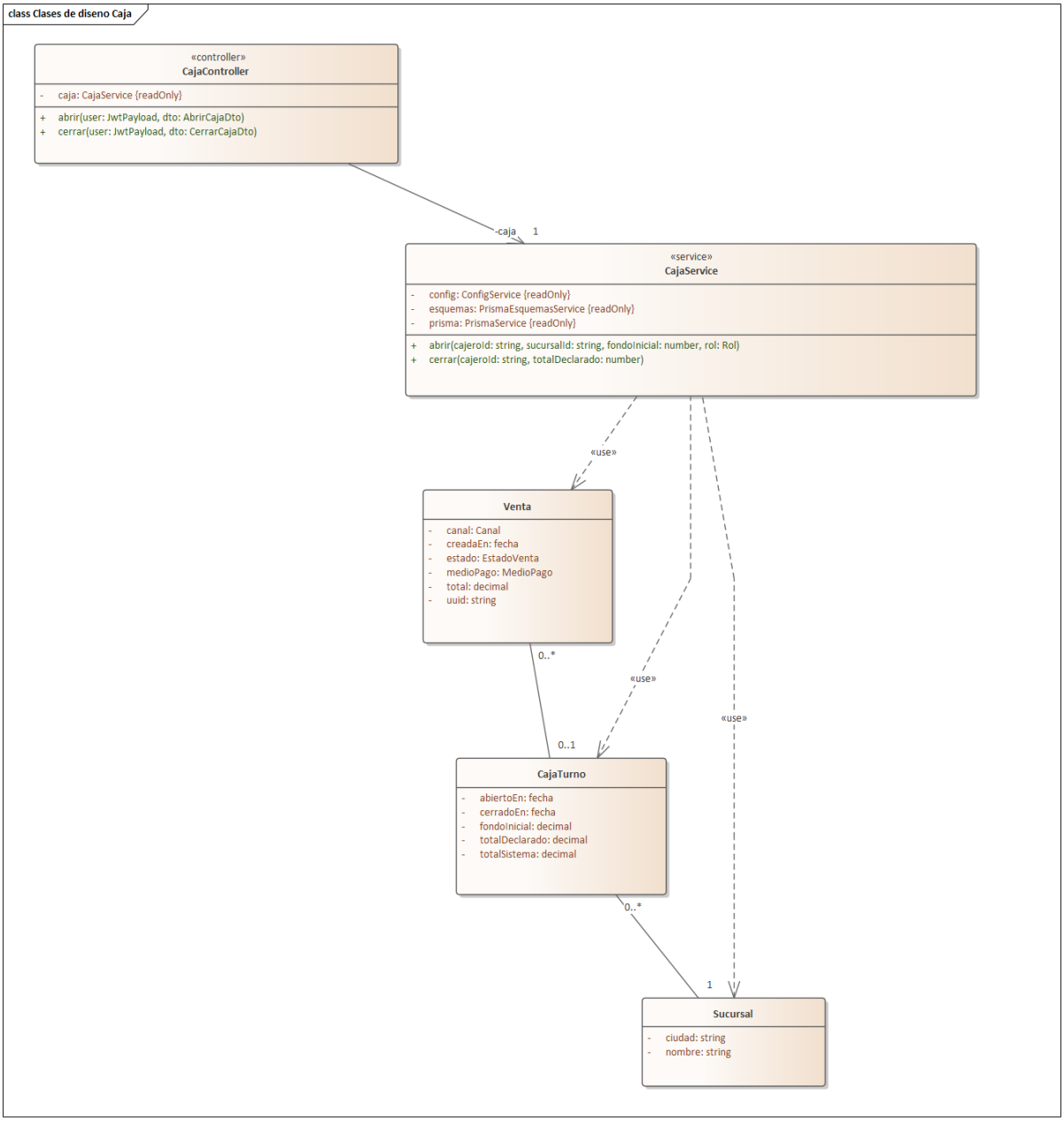


Figura 9.3.59

Clases de diseño del módulo Reportes

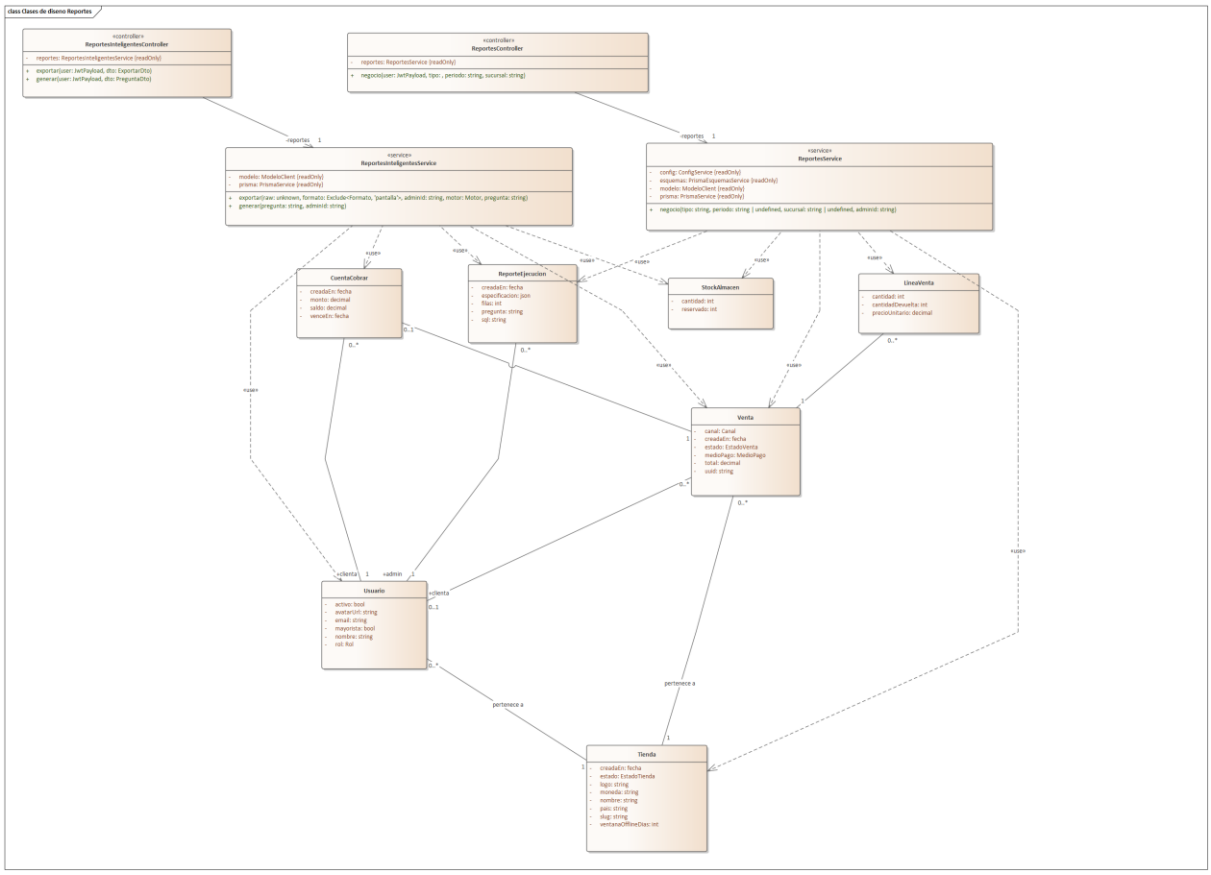


Figura 9.3.60

Clases de diseño del módulo Reservas

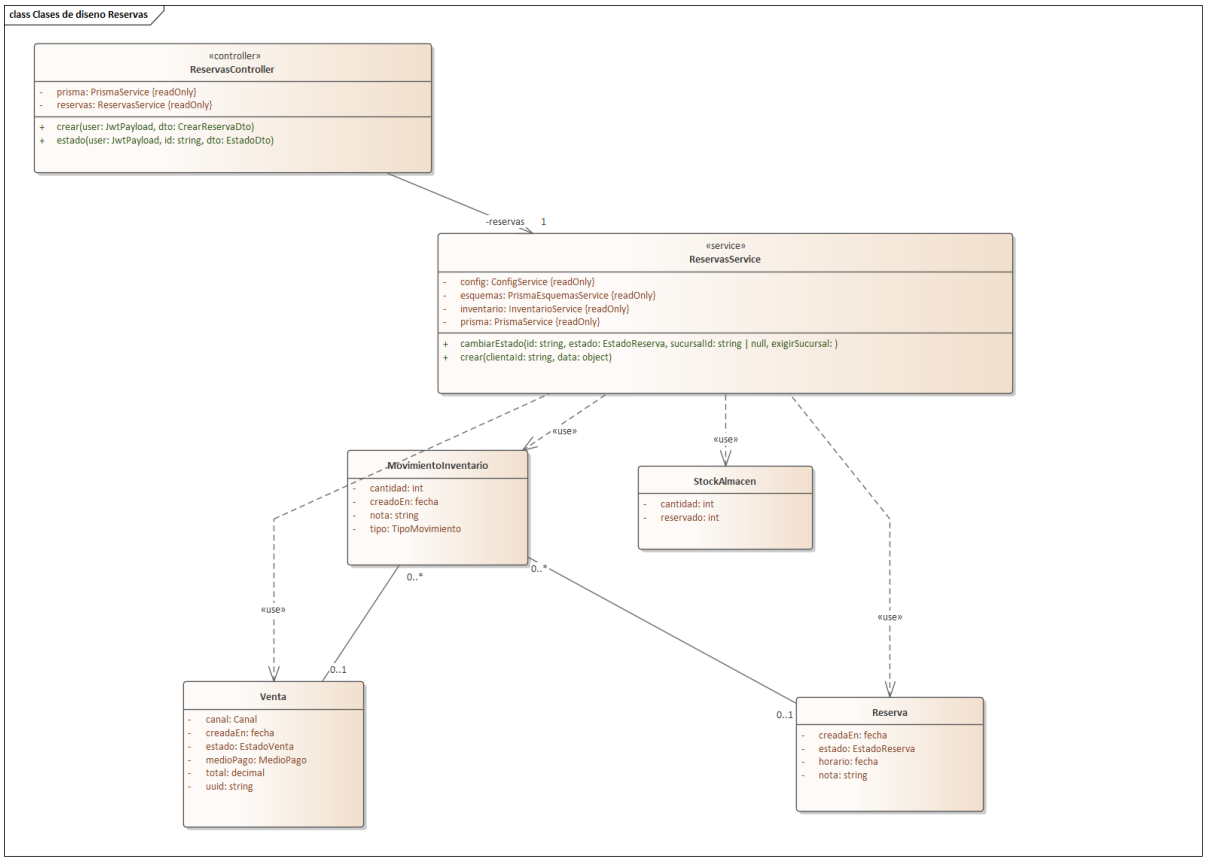


Figura 9.3.61

Clases de diseño del módulo Proveedores

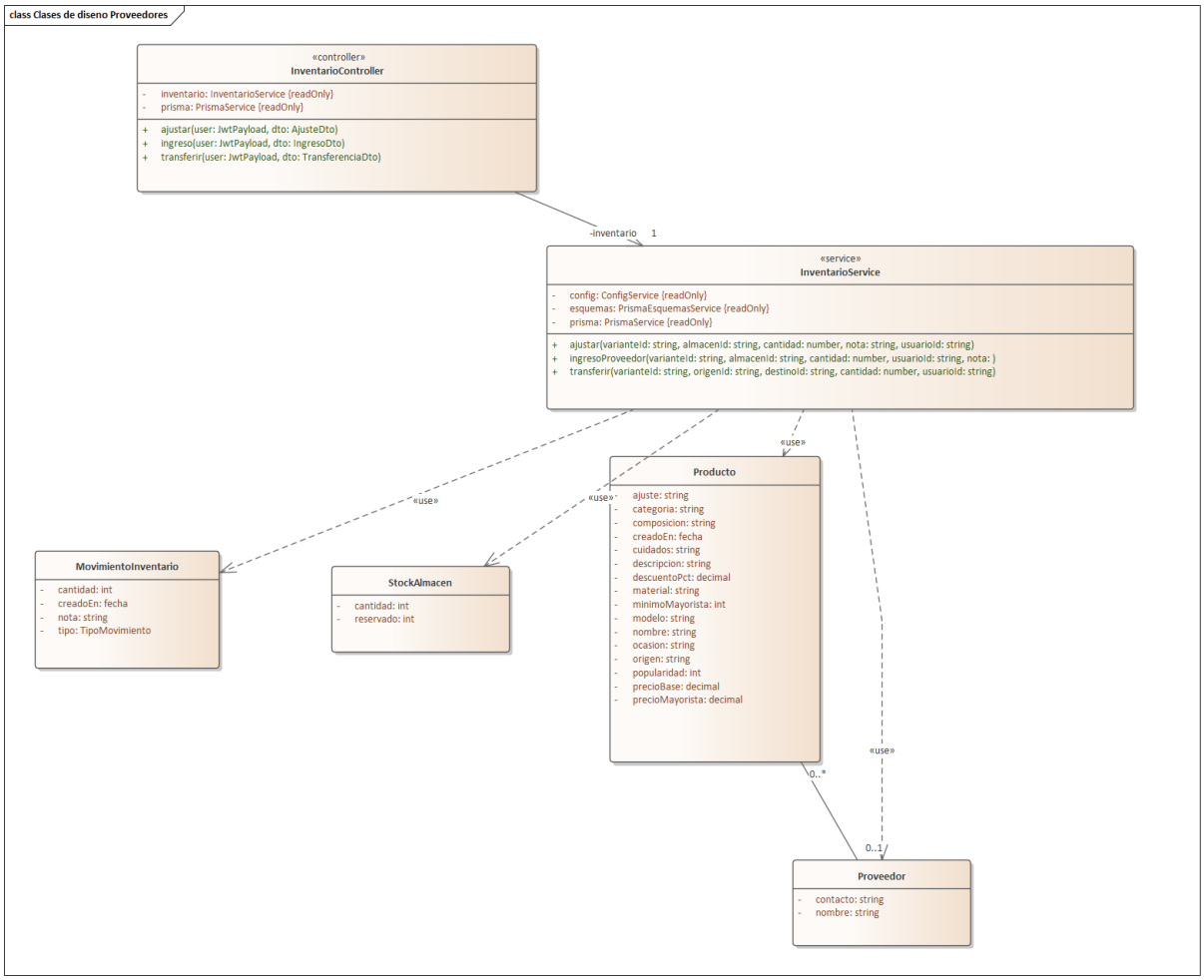


Figura 9.3.62

Clases de diseño del módulo Auth

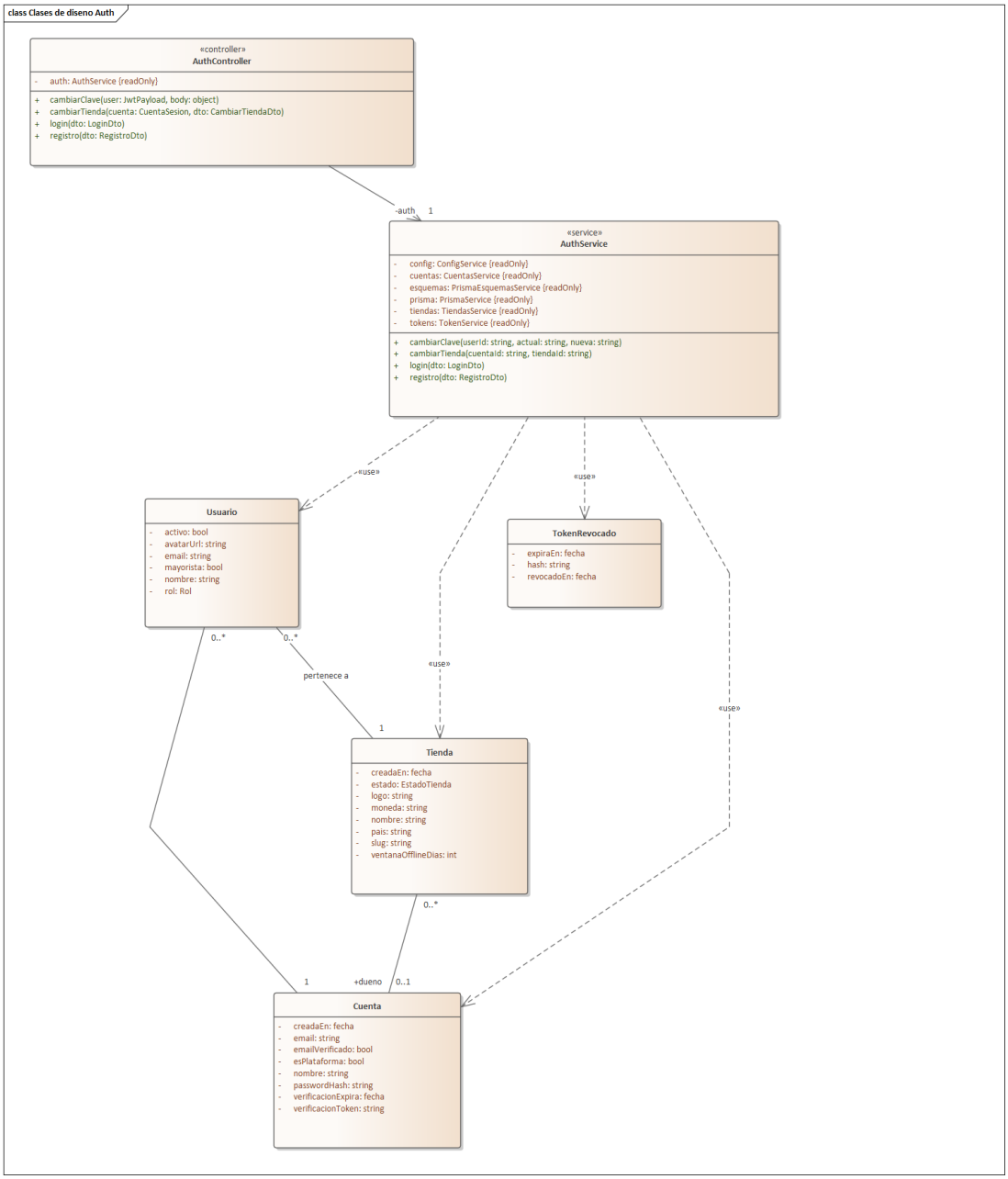


Figura 9.3.63

Clases de diseño del módulo Cliente

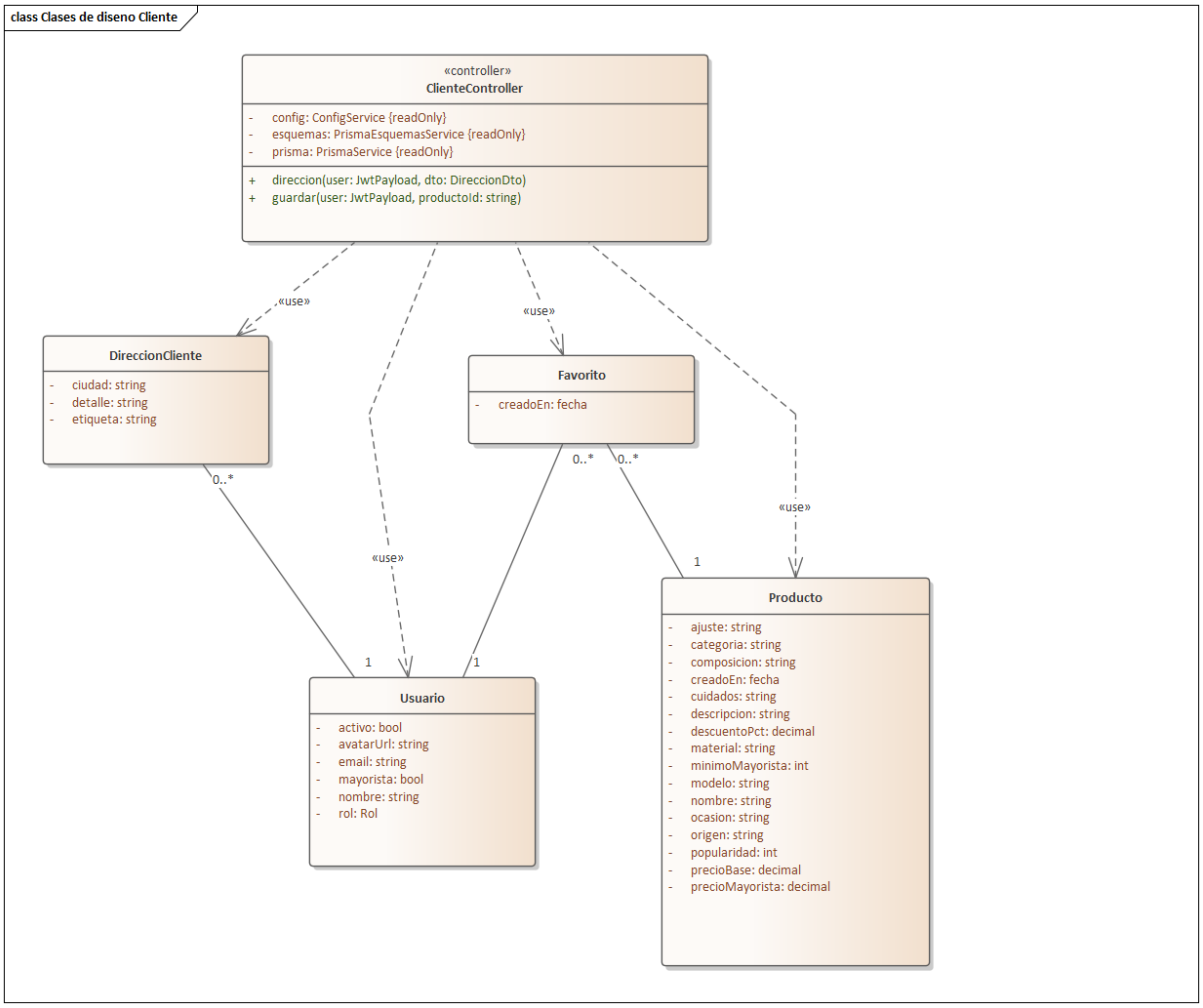


Figura 9.3.64

Clases de diseño del módulo Administración

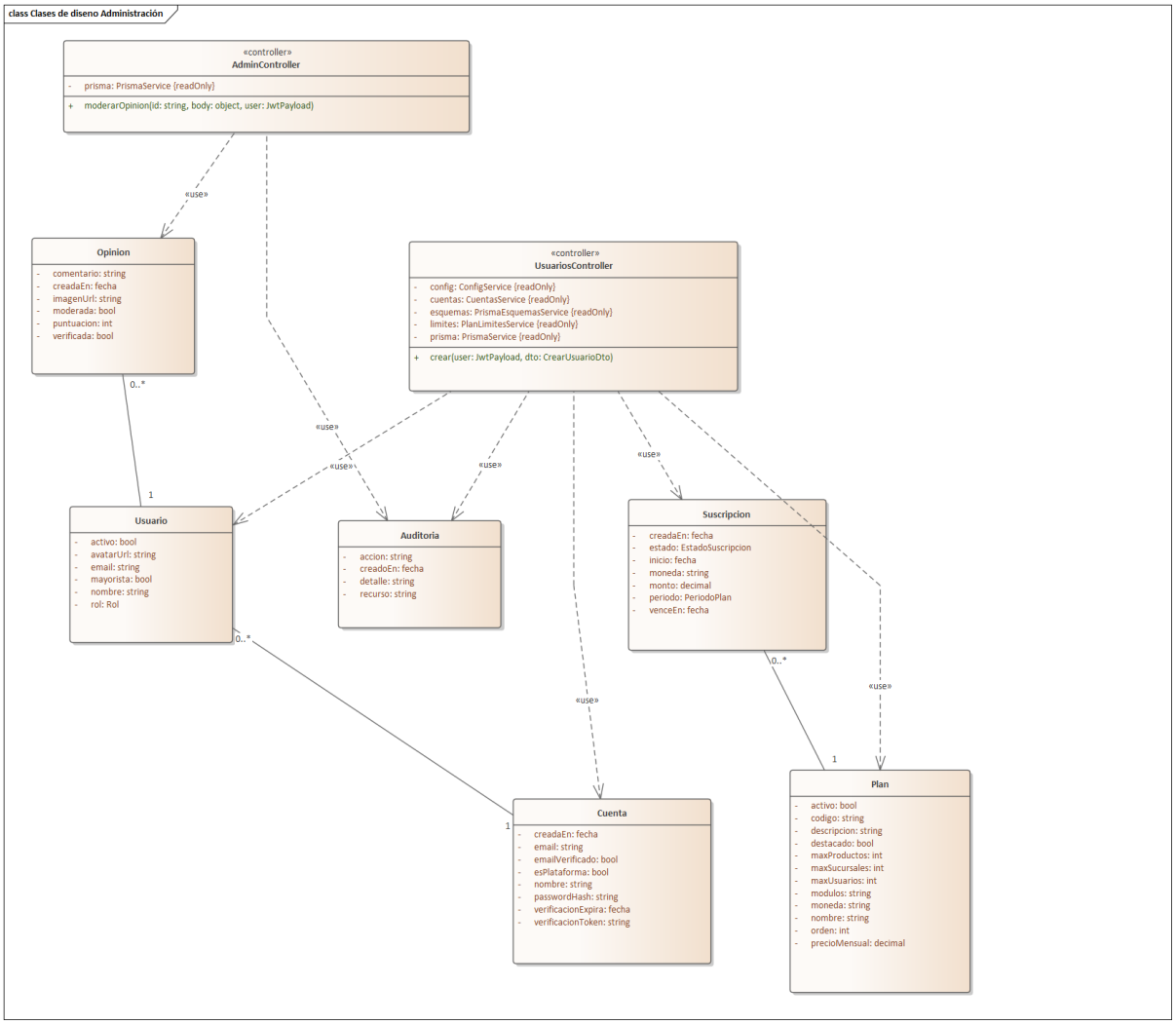


Figura 9.3.65

Clases de diseño del módulo Pagos

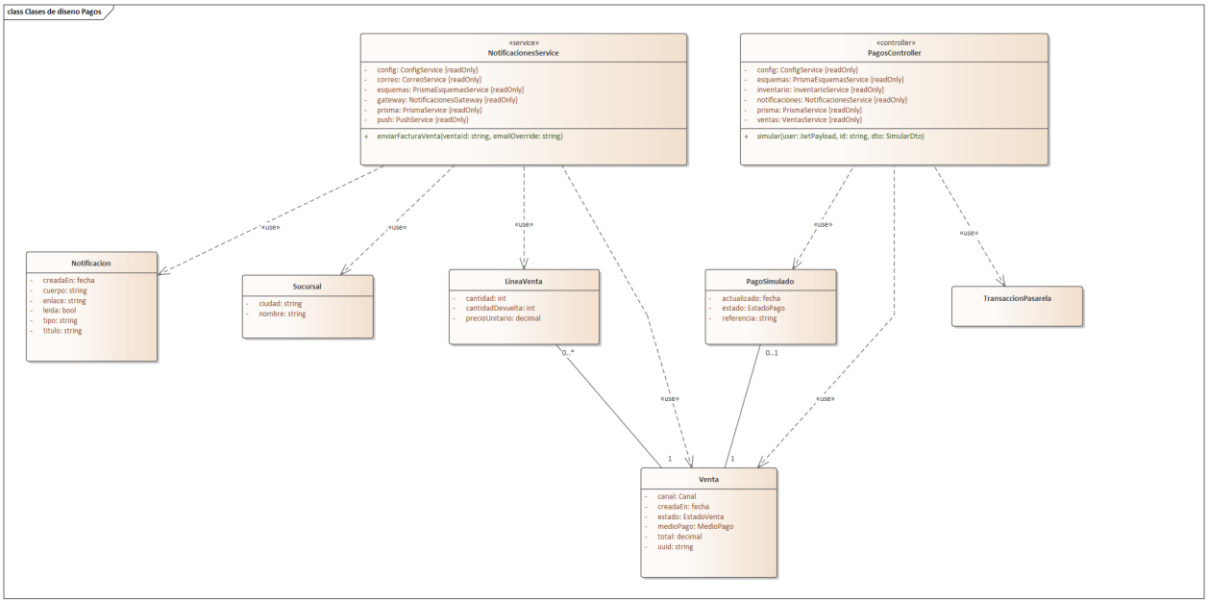


Figura 9.3.66

Clases de diseño del módulo Cliente offline

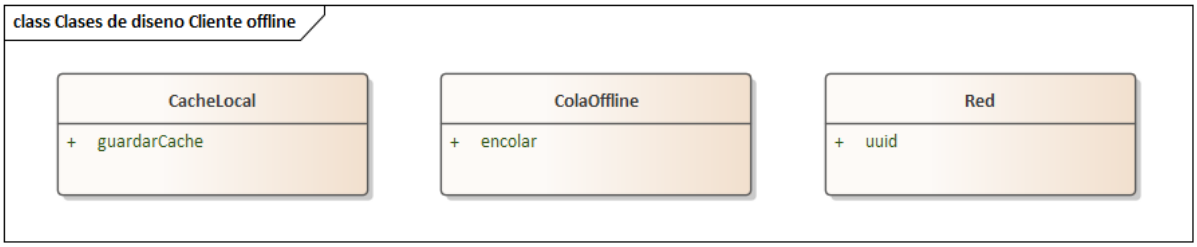


Figura 9.3.67

Clases de diseño del módulo Plataforma

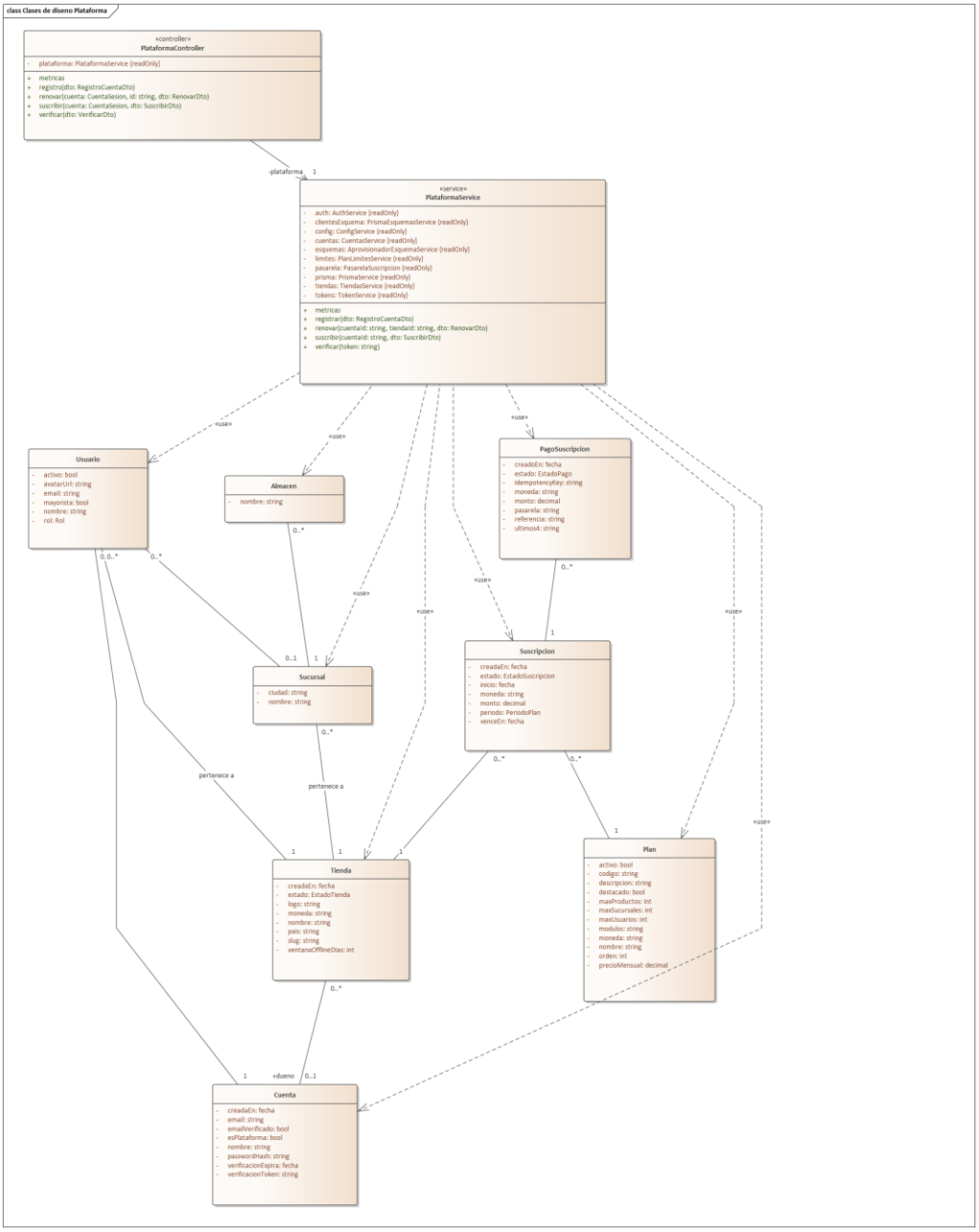


Figura 9.3.68

Clases de diseño del módulo Sincronización

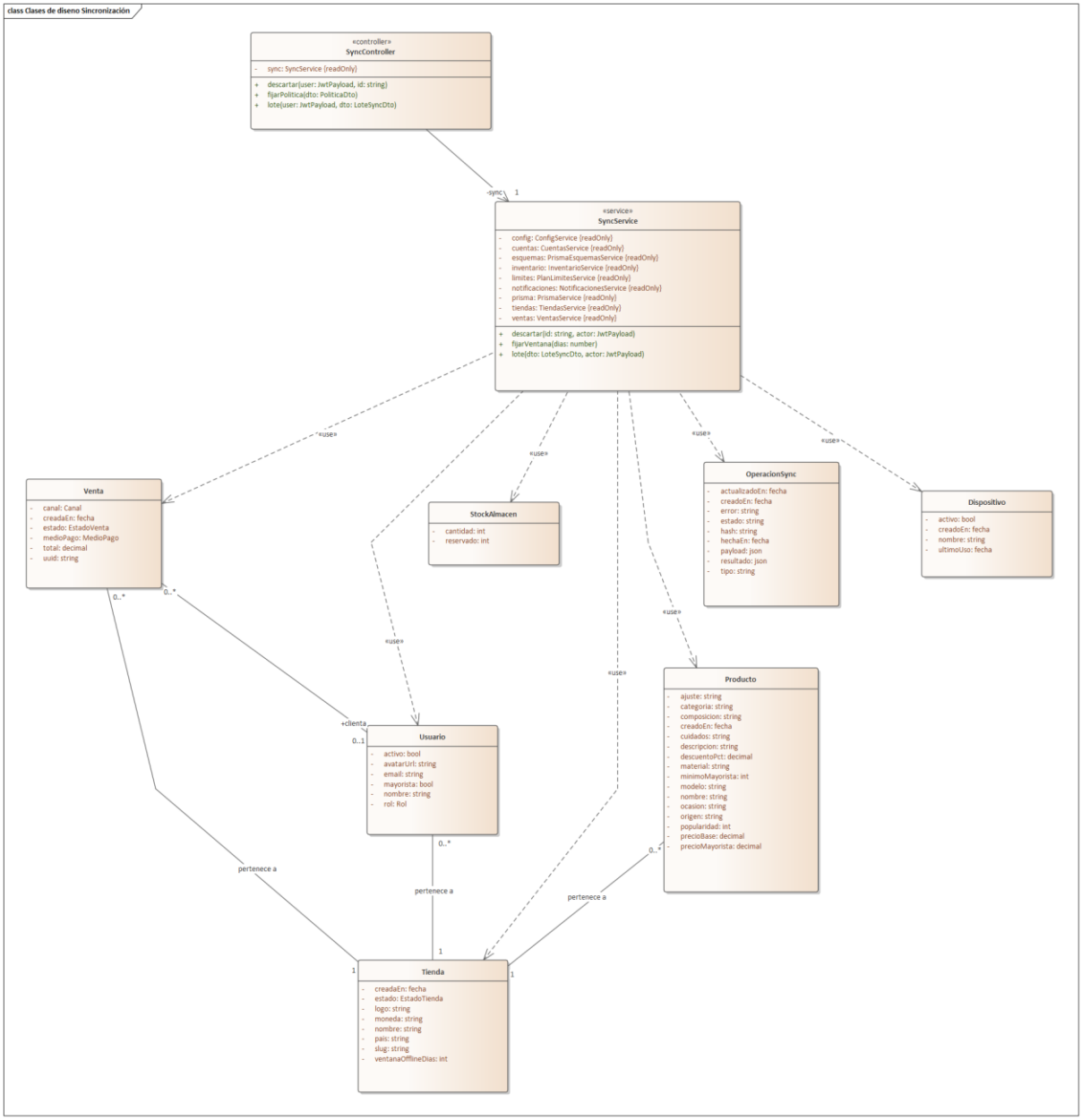


Figura 9.3.69

Clases de diseño del módulo Notificaciones

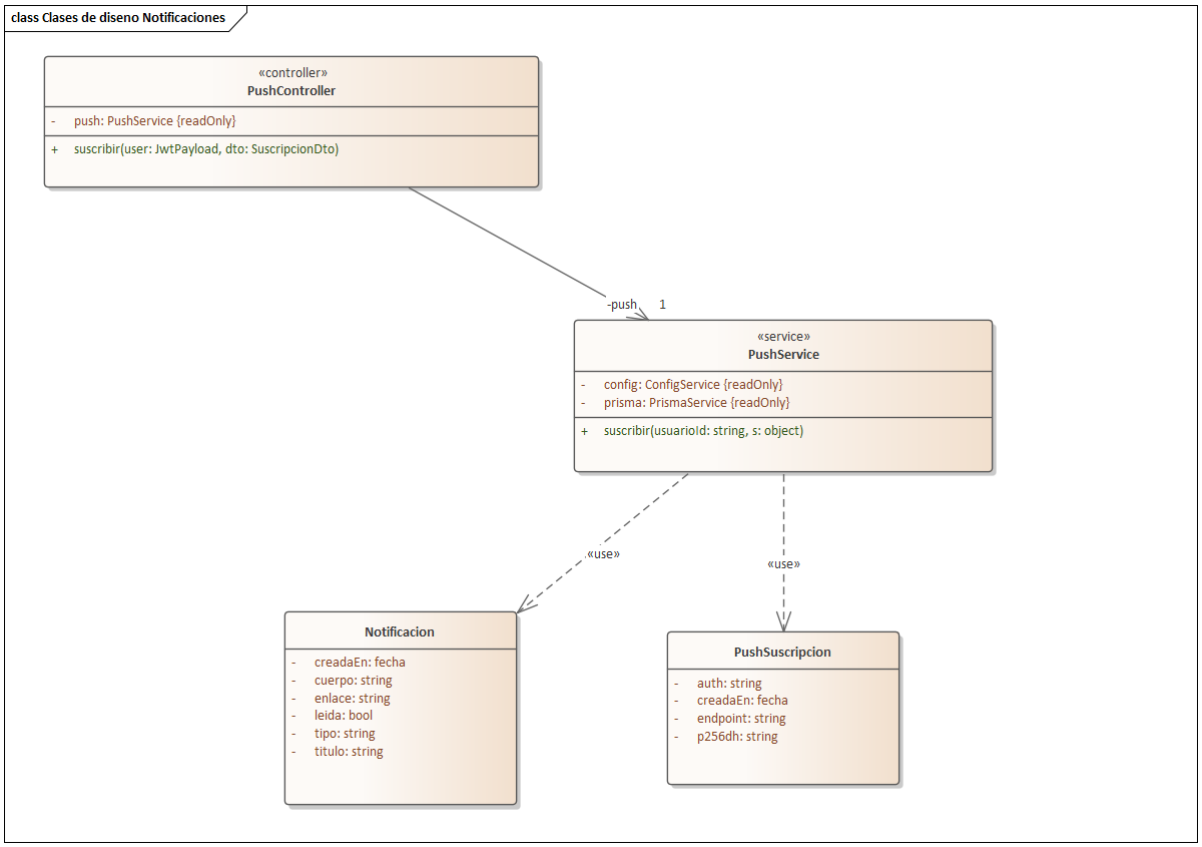


Figura 9.3.70

Clases de diseño del módulo Créditos

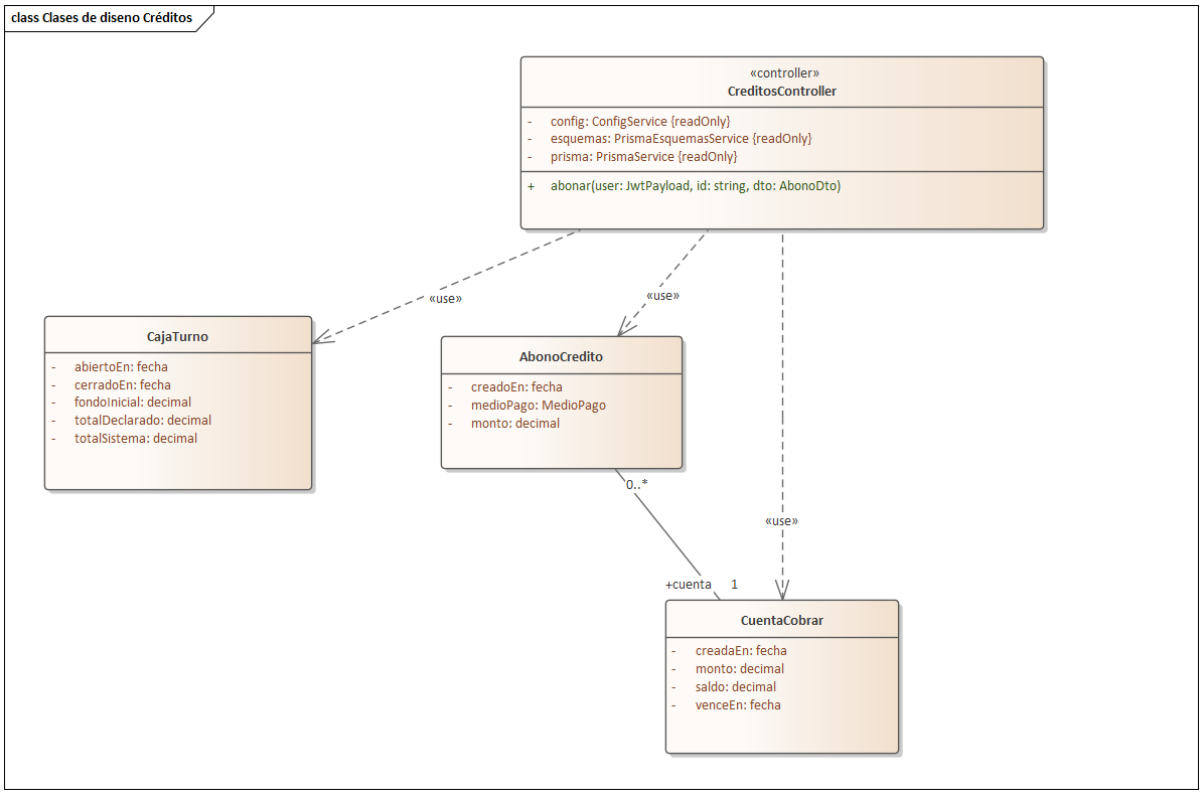
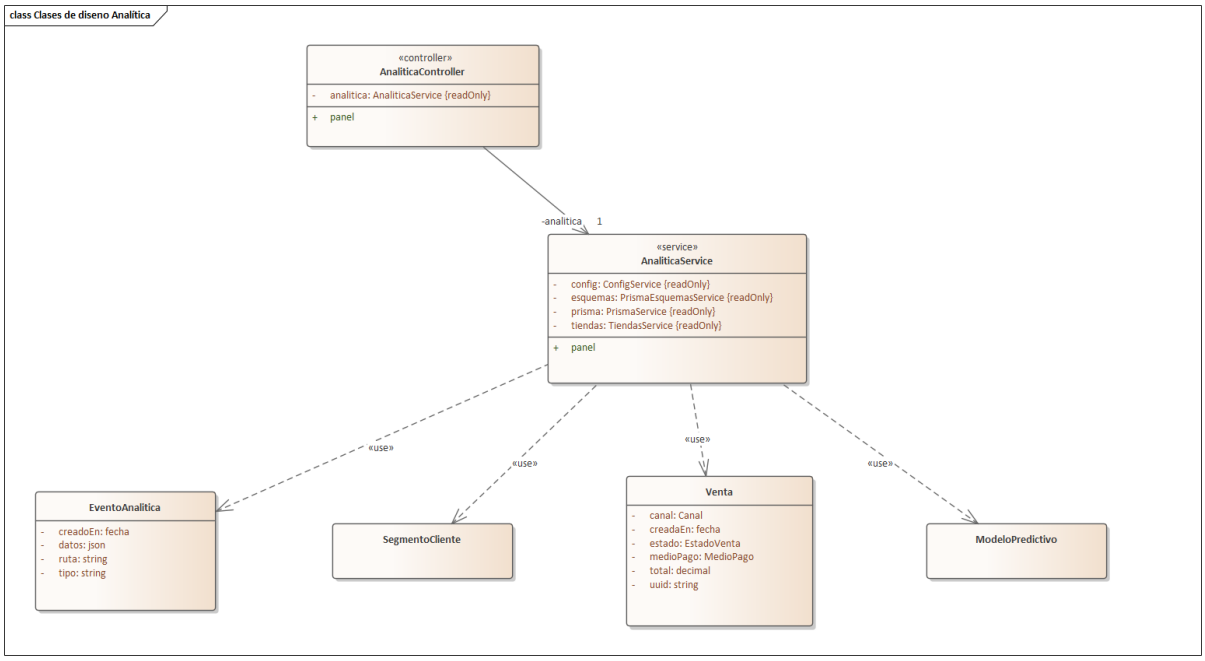


Figura 9.3.71

Clases de diseño del módulo Analítica



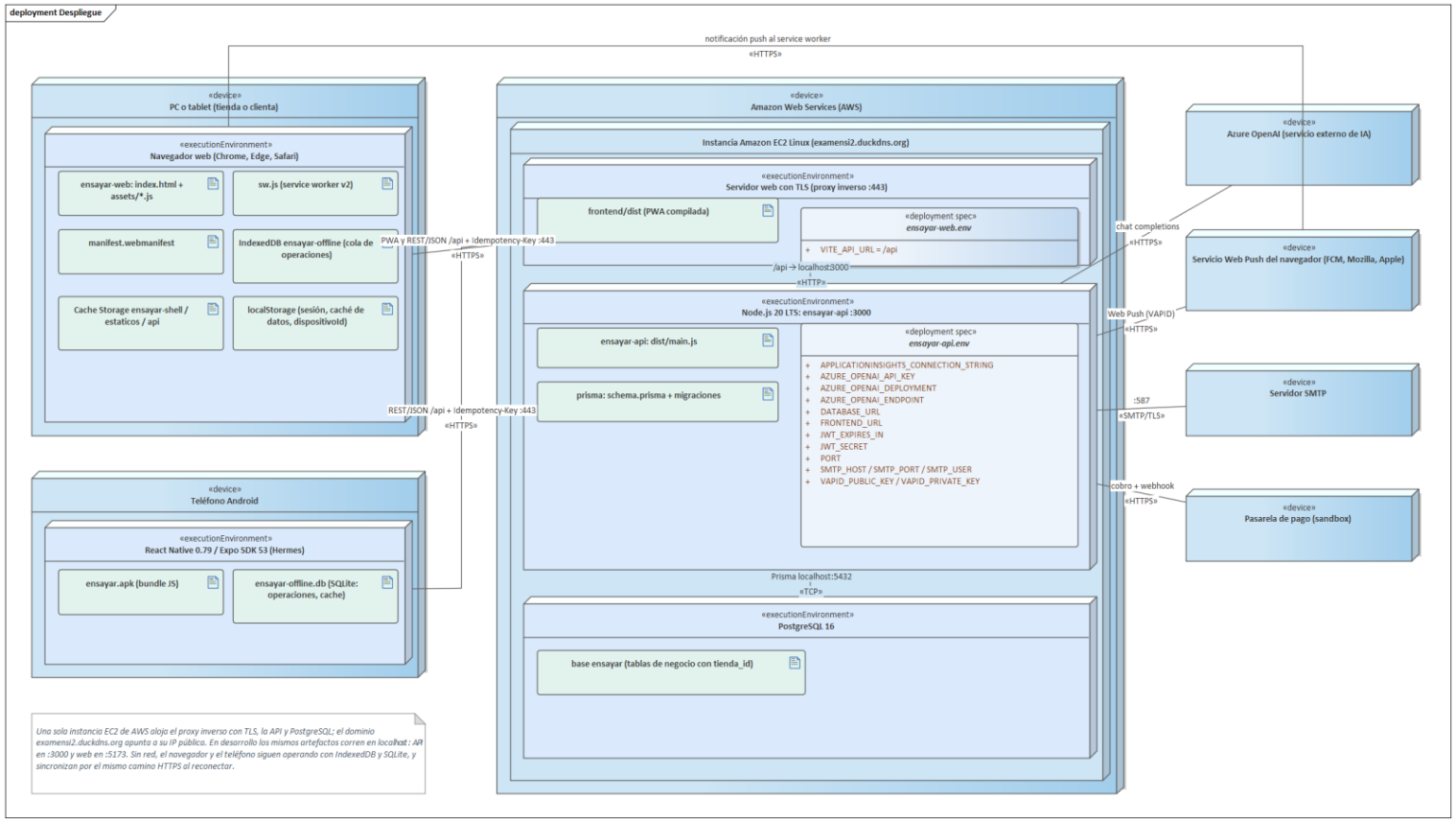
9.3.2 Arquitectura

Las decisiones de arquitectura vienen del primer ciclo y de los capítulos de fundamentación, y entran al diseño detallado como restricciones. La aplicación es un monolito modular: un proceso NestJS con un módulo por área, una sola base PostgreSQL compartida por todas las tiendas y dos clientes (web PWA y app Expo) que guardan una cola local para operar sin red. La arquitectura se documenta con cuatro diagramas UML 2.5: despliegue, componentes del backend, componentes de los clientes y manifestación de artefactos.

Diagrama de despliegue

Figura 9.3.72

Despliegue de EnsayAR en AWS y en los dispositivos



El diagrama muestra dónde se ejecuta cada pieza en producción. La versión de la entrega corre en una sola instancia Amazon EC2 con Linux, a la que apunta el dominio `examensi2.duckdns.org`; la instancia aloja el proxy inverso con TLS, la API y PostgreSQL.

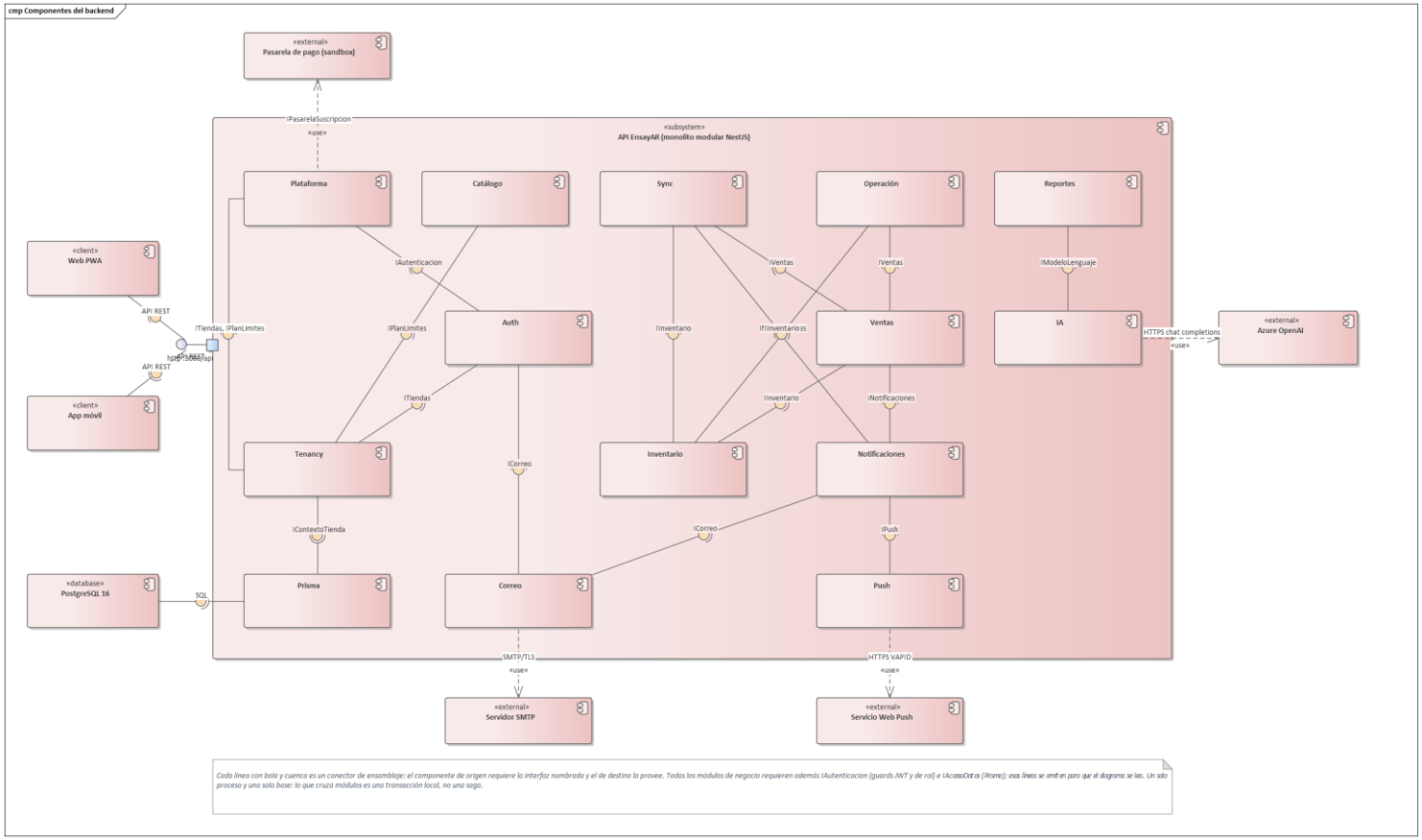
Elemento UML	Qué representa en EnsayAR
Nodo «device»	Hardware físico o un recurso que se contrata como unidad: la PC o tablet de la tienda o de la clienta, el teléfono Android, la cuenta de AWS y los servicios externos (Azure OpenAI, Web Push, SMTP y pasarela de pago)
Nodo anidado	La instancia Amazon EC2 con Linux dentro de la cuenta de AWS
«executionEnvironment»	Software que hospeda a los artefactos: el navegador web, el runtime React Native con Hermes, el servidor web con TLS que actúa como proxy inverso y sirve la PWA, Node.js 20 LTS con la API y PostgreSQL 16
Artefacto	Archivo concreto que se despliega o se crea en el dispositivo: `dist/main.js`, el esquema y las migraciones de Prisma, `frontend/dist`, `sw.js`, `manifest.webmanifest`, la base IndexedDB `ensayar-offline`, Cache Storage, `localStorage`, el `.apk` y la base SQLite `ensayar-offline.db`
«deployment spec»	Configuración con la que se despliega un artefacto: `ensayar-api.env`, con las 11 variables que lee la API (sin valores, porque son secretos), y `ensayar-web.env`, con `VITE_API_URL` igual a la ruta relativa `/api`
Camino de comunicación	Enlace entre nodos con su protocolo como estereotipo y su puerto o uso como nombre: «HTTPS» REST/JSON con `Idempotency-Key` en el puerto 443, «HTTP» del proxy a la API en `localhost:3000`, «TCP» Prisma en `localhost:5432`, «SMTP/TLS» en el 587 y «HTTPS» Web Push firmado con VAPID

Se leen tres recorridos. El primero es la descarga de la PWA desde el proxy inverso de la instancia y su instalación en el navegador. El segundo son las llamadas REST de ambos clientes a la API, que con red operan en línea y sin red guardan en IndexedDB o SQLite y luego reenvían por el mismo camino. El tercero es el aviso push: la API lo firma con VAPID, el servicio push del navegador lo entrega y el service worker lo muestra aunque la aplicación esté cerrada.

Diagrama de componentes del backend

Figura 9.3.73

Componentes del backend (monolito modular NestJS)

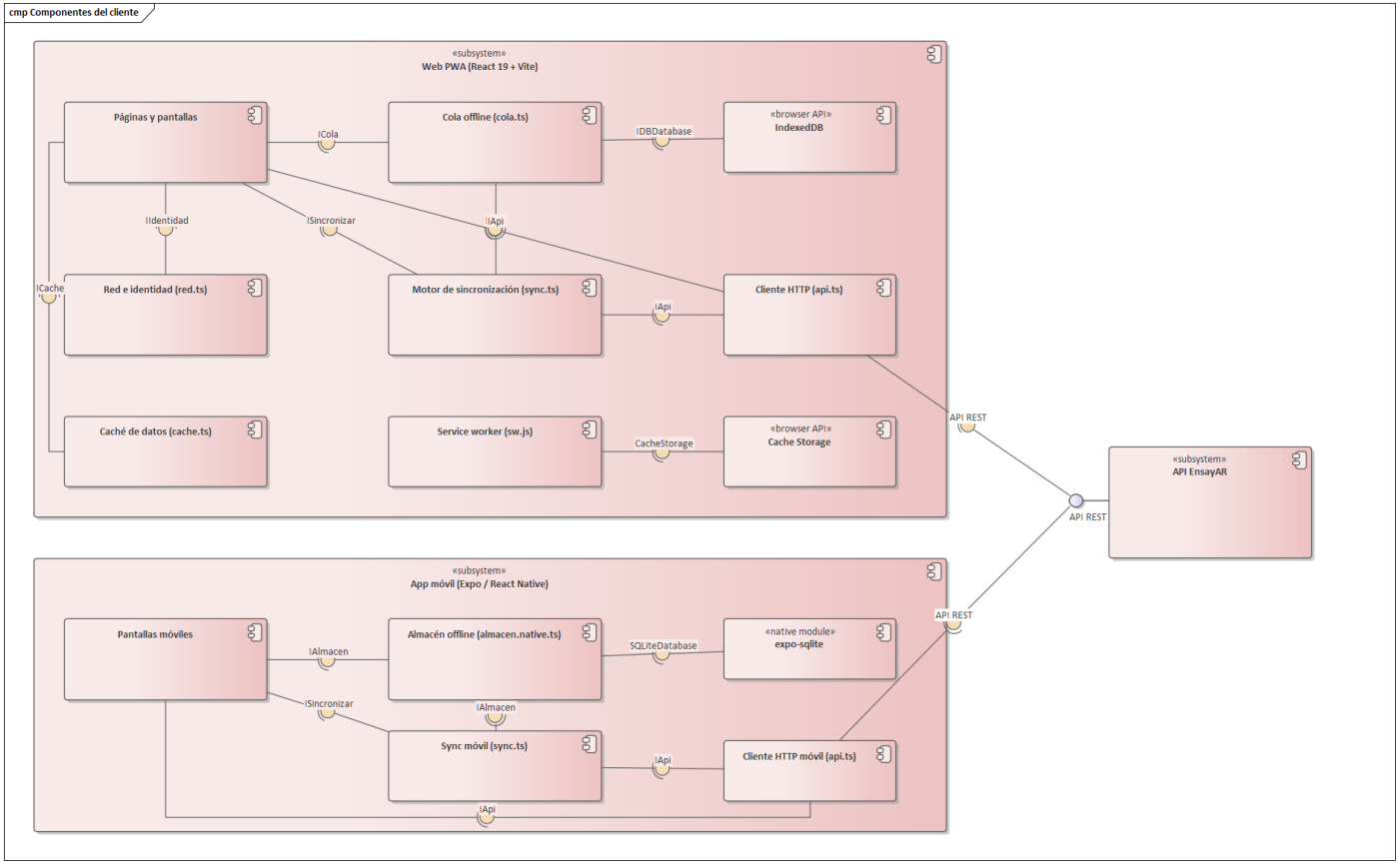


El subsistema «subsystem» API EnsayAR expone un puerto `http :3000/api` que provee la interfaz API REST. Los dos clientes la requieren y se conectan con un conector de ensamblaje. Dentro del subsistema, cada módulo NestJS es un componente. Cada línea con el símbolo de bola y cuenca es un conector de ensamblaje: el componente de origen requiere la interfaz nombrada y el de destino la provee. Por ejemplo, Sync requiere IVentas, IInventario e INotificaciones, y Prisma requiere IContextoTienda de Tenancy para filtrar por tienda. Las dependencias con sistemas externos (Azure OpenAI, SMTP, Web Push y la pasarela) se dibujan como «use», y Prisma requiere la interfaz SQL de PostgreSQL. Para que el diagrama se lea se omiten dos dependencias que tienen todos los módulos: IAutenticacion (guards) e IAccesoDatos (Prisma).

Diagrama de componentes de los clientes

Figura 9.3.74

Componentes de la PWA y de la app móvil

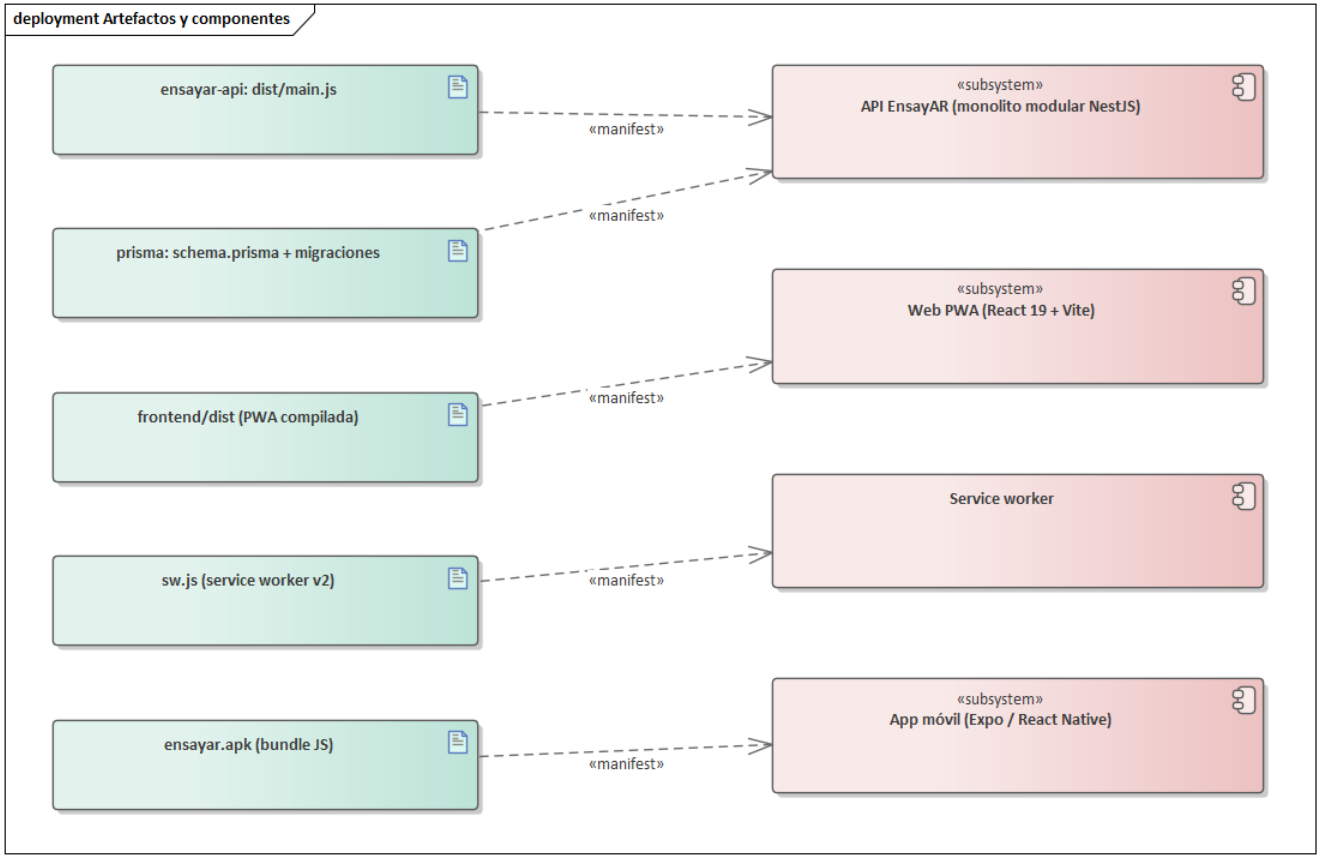


La web y la app repiten la misma estructura. Las pantallas usan la cola offline, el motor de sincronización y el cliente HTTP. La cola persiste en IndexedDB (web) o en SQLite (móvil). El motor de sincronización vacía la cola contra la API REST, la misma interfaz del diagrama anterior. En la web, el service worker guarda el app shell y los datos en Cache Storage, y es quien recibe los avisos push.

Manifestación de artefactos

Figura 9.3.75

Artefactos que manifiestan a cada componente



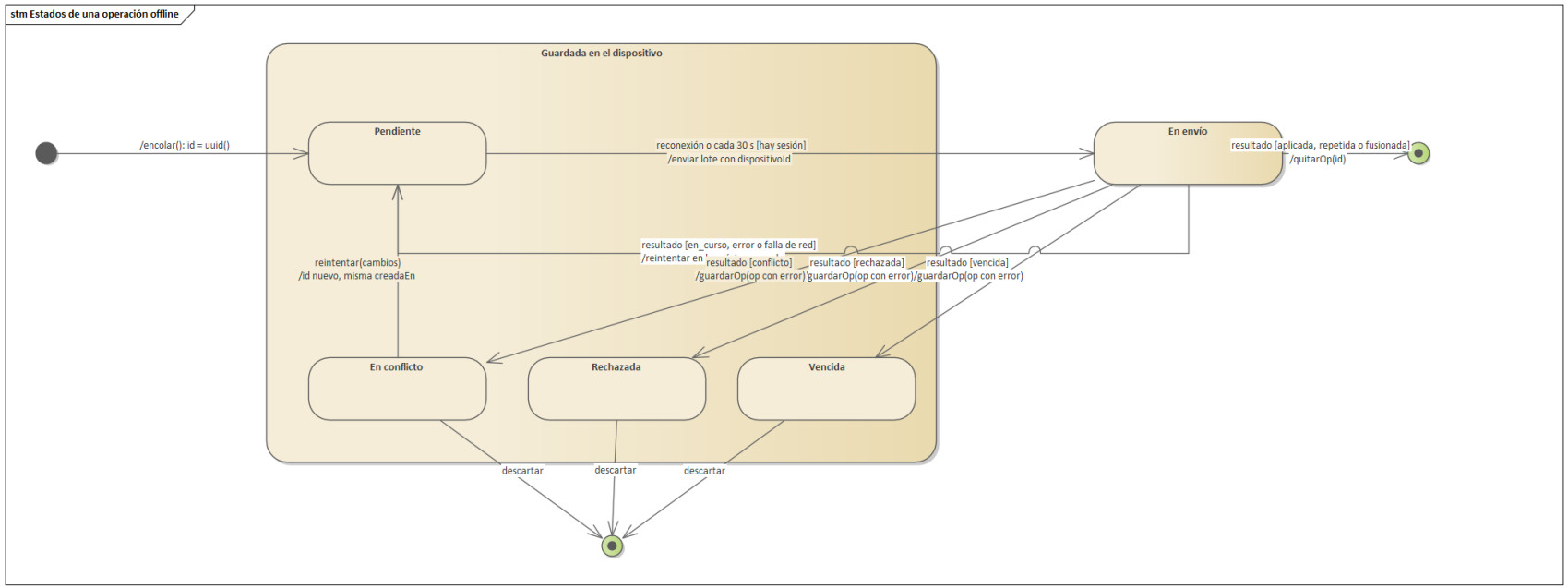
La relación «manifest» une el diseño lógico con el físico: dice qué archivo desplegado implementa cada componente. `dist/main.js` y el esquema de Prisma manifiestan la API, `frontend/dist` la PWA, `sw.js` el service worker y el `.apk` la app móvil. Con esto, cada componente del diagrama de componentes se puede seguir hasta un nodo del diagrama de despliegue.

9.3.3 Máquinas de estado de la sincronización

La operación offline es el único objeto del sistema cuyo comportamiento depende de su historia: la misma venta puede estar pendiente en el teléfono, aplicada en el servidor o detenida en un conflicto. Por eso, además de las secuencias, se modela con dos máquinas de estado UML 2.5. Cada transición se escribe como `evento [guarda] / efecto`.

Figura 9.3.76

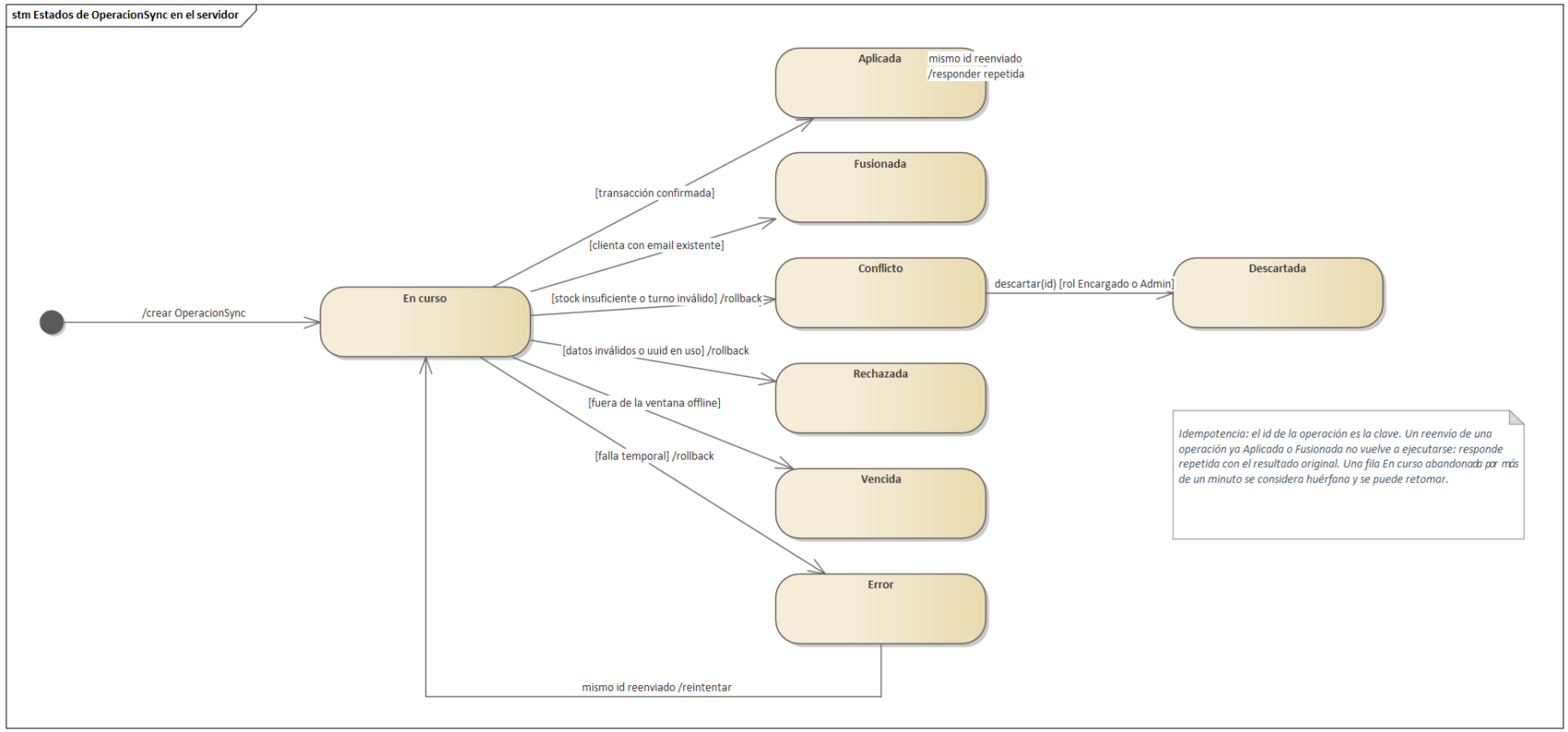
Estados de una operación offline en el dispositivo



En el dispositivo, la operación nace Pendiente dentro del estado compuesto *Guardada en el dispositivo*, que agrupa todo lo que está persistido en IndexedDB o SQLite. Al reconectar, o cada 30 segundos, pasa a En envío. Solo un estado definitivo del servidor (aplicada, repetida o fusionada) la saca de la cola. Si llega en_curso, error o se corta la red, vuelve a Pendiente. Un conflicto, un rechazo o un vencimiento la dejan guardada con el motivo hasta que la persona la reintenta o la descarta. Así se cumple que una operación nunca desaparece por el solo hecho de haber intentado enviarla (RN07).

Figura 9.3.77

Estados de OperacionSync en el servidor



En el servidor, cada operación recibida crea una fila OperacionSync En curso y abre su propia transacción. Si se confirma, termina Aplicada (o Fusionada, si la clienta ya existía). Si falta stock o el turno ya no es válido, termina en Conflicto con rollback. Un reenvío del mismo id sobre una operación Aplicada no vuelve a ejecutar nada: responde *repetida* (transición sobre sí misma). Los estados se corresponden con los que el usuario ve en la Pantalla Sincronización.

Estado en el dispositivo	Significado para la persona	Estado en el servidor
Pendiente	Registrada localmente, todavía no enviada	(sin fila)
En envío	Enviada en un lote, esperando respuesta	En curso
(sale de la cola)	Confirmada	Aplicada o Fusionada
En conflicto	Requiere decisión: vender lo disponible o descartar	Conflicto, y luego Descartada si se descarta
Rechazada	Datos inválidos o sin permiso	Rechazada
Vencida	Fuera de la ventana offline de la tienda	Vencida

9.3.4 Revisión crítica de diseño (CDR)

Verificación	Resultado
Cada caso tiene secuencia y coincide con su texto	Sí, 54 secuencias
La secuencia cubre el curso básico y los alternos	Sí; el alterno que decide en el servidor queda en un fragmento `alt` ubicado en el paso donde ocurre, y el lote de sincronización en un `loop`
Las firmas del diseño coinciden con el código	Sí; se leen de los archivos TypeScript al regenerar el modelo
Cada control de robustez aparece como mensaje	Sí; los mensajes se derivan de los controles
Las operaciones están en la clase que tiene la información	Sí en los servicios. Créditos, Pagos y Devoluciones concentran lógica en el controller: deuda técnica aceptada para este ciclo
Las operaciones que cambian dinero o stock son idempotentes	Sí; la clave de idempotencia está en 11 rutas
Ninguna operación consulta datos sin la tienda de la sesión	Sí; la extensión de Prisma falla cerrada sin tienda y el SQL de reportes usa set_config
Existe un plan de pruebas por caso	Sí; ver 9.4

9.4 Implementación y pruebas

El código se escribió a partir del diseño de cada lote. ICONIX organiza las pruebas en dos niveles. Las unitarias verifican controles aislados, como el compilador de SQL seguro o los

filtros del catálogo. Las de integración recorren un caso de uso completo contra el sistema en ejecución, con su curso básico y sus alternos.

9.4.1 Plan de pruebas

Aspecto	Definición
Objetivo	Demostrar que cada caso de uso cumple su curso básico y sus alternos, y que se cumplen los requisitos no funcionales RNF02 a RNF06 (aislamiento, persistencia offline, integridad, idempotencia y recuperación)
Niveles	Unitarias sobre controles aislados; integración por caso de uso contra la API y una base PostgreSQL real; sistema, con recorridos manuales en navegador y en Expo con la red cortada
Datos	Semilla de demostración con al menos tres tiendas (Vesta, Aurora Boutique y Andes Moda) bajo una misma cuenta, más tiendas `demo-*` creadas por autoservicio; tarjetas de prueba de la pasarela sandbox
Entorno	Backend en el puerto 3000 y web en el 5173, con la base local migrada (`prisma migrate deploy`) y la semilla cargada
Escenarios críticos	Dos tiendas con datos parecidos (aislamiento); dos cajas que venden la última unidad sin red; el mismo lote enviado dos veces; un corte de red a mitad del envío; una operación fuera de la ventana offline; pedidos de reporte ambiguos o maliciosos
Criterio de aceptación	Todos los chequeos de los cuatro guiones en verde, ninguna lectura con datos de otra tienda, ningún stock o saldo negativo y ninguna operación duplicada
Registro	Cada guion imprime el escenario, el resultado esperado y el obtenido; los fallos detienen la ejecución con código distinto de cero

9.4.2 Casos de prueba

Cada caso de prueba nombra el caso de uso y el curso que verifica, la entrada, el resultado esperado y el guion que lo automatiza. Se eligieron los que cubren los requisitos de mayor riesgo.

Id	Caso de uso (curso)	Entrada	Resultado esperado	Verificación
CP01	UC01 (básico)	Usuario de Vesta lista productos	Solo aparecen productos de Vesta	`verificar:tenancy`
CP02	Transversal (RNF02)	Usuario de una tienda intenta leer o modificar un registro de otra por su id	No lo encuentra y no lo cambia	`verificar:tenancy`
CP03	Transversal (RNF02)	Consulta ejecutada sin	Error; la extensión	`verificar:tenancy`

Id	Caso de uso (curso)	Entrada	Resultado esperado	Verificación
		tienda en el contexto	falla cerrada y no devuelve datos	
CP04	UC37 (2b)	Cuenta nueva sin verificar intenta suscribirse	Se rechaza hasta abrir el enlace de verificación	`verificar:saas`
CP05	UC38 (3a)	Pago con la tarjeta de prueba de rechazo	No se crea la tienda; se puede reintentar	`verificar:saas`
CP06	UC38 (3b)	Dos pedidos de pago con la misma clave de idempotencia	Un solo cobro y una sola tienda	`verificar:saas`
CP07	UC23 (3a)	Alta de un usuario por encima del límite del plan	Rechazo con el límite del plan	`verificar:saas`
CP08	UC40 (básico)	La misma cuenta cambia de Vesta a Aurora Boutique	Token nuevo; solo ve datos de la tienda elegida	`verificar:saas`
CP09	UC42 (básico)	Venta en efectivo con la red cortada y recarga del navegador	La venta sigue en la cola como pendiente	Recorrido manual; `verificar:offline`
CP10	UC42 (1a)	Venta con QR sin red	No se permite y se explica el motivo	`verificar:offline`
CP11	UC44 (2a)	El mismo lote enviado dos veces	La segunda respuesta es repetida; no hay ventas duplicadas	`verificar:offline`
CP12	UC44 (3b) y UC45	Dos cajas venden sin red la última unidad y sincronizan	Una venta aplicada y otra en conflicto; el stock queda en cero, no negativo	`verificar:offline`
CP13	UC42 (4a)	Operación creada antes de la ventana offline de siete días	Llega como vencida	`verificar:offline`
CP14	UC44 (transversal RNF04)	Falla dentro de la transacción de una operación	Rollback completo; nada parcial en la base	`verificar:offline`
CP15	UC48 (básico)	Pedido dictado con filtros, agrupación y orden	Reporte correcto con SQL parametrizado	`verificar:reportes`
CP16	UC48 (alterno)	Pedido con texto que intenta inyectar SQL	Se rechaza o se trata como texto; no se ejecuta nada extra	`verificar:reportes`
CP17	UC49 (básico)	Exportar el reporte a Excel y a PDF	Se descargan los dos archivos con los mismos datos	`verificar:reportes`
CP18	UC51 (3b)	Dos abonos simultáneos por el saldo total	Uno pasa y el otro se rechaza; el saldo no queda negativo	`verificar:reportes`
CP19	UC07 (básico, concurrencia)	Ventas concurrentes sobre el mismo stock	Nunca se vende más de lo disponible	`ventas.spike-b.spec.ts`

9.4.3 Pruebas de integración

Los scripts de verificación del backend se ejecutan contra la API levantada y una base real. Cada chequeo nombra un escenario de un caso de uso y verifica su poscondición.

Script	Casos de uso que verifica	Escenarios
`npm run verificar:tenancy`	UC01, UC10, UC15, UC19	Lecturas filtradas por tienda, fallo

Script	Casos de uso que verifica	Escenarios
		sin tienda en contexto, registros ajenos invisibles e inmodificables
`npm run verificar:saas`	UC14, UC23, UC24, UC36 a UC41	15 chequeos: verificación de correo, pago rechazado y aprobado, reintento sin doble cobro, catálogos aislados, límites del plan, cambio de tienda, suscripción vencida y métricas
`npm run verificar:offline`	UC05 a UC07, UC09, UC35, UC42 a UC46	14 chequeos: uuid del dispositivo, última unidad en dos cajas, reversión, reenvío sin duplicados, envíos simultáneos, pago QR sin red rechazado y ventana de siete días
`npm run verificar:reportes`	UC33, UC48 a UC51	24 chequeos: pedidos dictados complejos, agrupación y orden, parámetros separados del SQL, inyección, aislamiento, Excel y PDF, crédito y abonos repetidos o simultáneos

9.4.4 Pruebas unitarias

Prueba	Control que verifica
`catalogo.service.spec.ts`	Filtrar productos y obtener producto (UC01, UC02, UC09)
`ventas.spike-b.spec.ts`	Descontar stock condicional con ventas concurrentes (UC05, UC07)
`sql-seguro.spec.ts`	Validar consulta y fijar tienda (UC10)
`ficha.spec.ts`	Armar contexto de la ficha y respuesta por reglas (UC04)

9.4.5 Matriz de trazabilidad

La matriz une cada caso con sus requisitos, la ruta que lo atiende, la clase y la operación que lo realizan y la prueba que lo verifica. Se lee en ambos sentidos: desde un requisito hasta su prueba, o desde una clase hasta el caso que la justifica.

Caso	Requisitos	Ruta	Realizado por	Prueba
UC01 Consultar catálogo	B1	`GET /catalogo/productos`	CatalogoService.listarProductos	catalogo.service.spec.ts (filtros) y verificar:tenancy (catálogo aislado por tienda).
UC02 Ver ficha y variantes	B1	`GET /catalogo/productos/:id`	CatalogoService.obtenerProducto	catalogo.service.spec.ts.
UC03 Usar	B1	`GET`	CatalogoService.overlay	Prueba manual

Caso	Requisitos	Ruta	Realizado por	Prueba
el probador virtual		`/catalogo/variantes/:id/overlay`		guiada: con cámara, sin permiso de cámara y con foto.
UC04 Consultar la ficha con el asistente	B1	`POST /ia/ficha`	IaService.ficha	ficha.spec.ts y prueba manual con y sin clave de Azure OpenAI.
UC05 Comprar en línea	B1, O5, O6	`POST /ventas`	VentasService.cobrar	verificar:offline (uuid duplicado y conflicto de última unidad).
UC06 Abrir caja	B1, O6	`POST /caja/abrir`	CajaService.abrir	verificar:offline (apertura idempotente).
UC07 Registrar venta en el punto de venta	B1, O5, O6, O7	`POST /ventas`	VentasService.cobrar	ventas.spike-b.spec.ts y verificar:offline (venta, duplicado y conflicto).
UC08 Cerrar caja	B1, O1	`POST /caja/cerrar`	CajaService.cerrar	Prueba manual de arqueo; bloqueo con cola pendiente en la web.
UC09 Administrar catálogo	B1, O1	`POST /catalogo/productos`	CatalogoService.crearProducto	catalogo.service.spec.ts y verificar:offline (alta de producto offline).
UC10 Pedir reporte predefinido	B1, S1	`GET /reportes/negocio`	ReportesService.negocio	sql-seguro.spec.ts y verificar:tenancy.
UC11 Reservar una prenda	B1	`POST /reservas`	ReservasService.crear	Prueba manual: reservar con y sin disponible.
UC12 Atender reservas	B1	`PATCH /reservas/:id/estado`	ReservasService.cambiarEstado	Prueba manual de completar y cancelar.
UC13 Ingresar mercadería	B1	`POST /inventario/ingreso`	InventarioService.ingresoProveedor	Prueba manual con producto propio y ajeno.
UC14 Registrarse como clienta	B1, S7	`POST /auth/registro`	AuthService.registro	verificar:saas (cuenta global y membresías).
UC15 Iniciar y cerrar sesión	B1, S1	`POST /auth/login`	AuthService.login	verificar:tenancy (token de una tienda no sirve en otra).
UC16 Cambiar clave	B1	`POST /auth/cambiar-clave`	AuthService.cambiarClave	Prueba manual con clave correcta e incorrecta.
UC17 Gestionar favoritos	B1	`POST /cliente/favoritos/:productoId`	ClienteController.guardar	Prueba manual.
UC18 Gestionar perfil y direcciones	B1	`POST /cliente/direcciones`	ClienteController.direccion	Prueba manual.
UC19 Ver historial de compras	B1, S1	`GET /ventas/historial`	VentasService.historialClienta	verificar:tenancy.
UC20 Opinar un	B1	`POST /catalogo/productos/:id/opinio`	CatalogoController.crearOpinion	Prueba manual.

Caso	Requisitos	Ruta	Realizado por	Prueba
producto		nes		
UC21 Pedir alerta de producto	B1	POST /catalogo/productos/:id/alertas	CatalogoController.alertar	Prueba manual.
UC22 Aplicar cupón	B1	POST /ventas	VentasService.cobrar	Prueba manual con cupón válido, vencido y agotado.
UC23 Gestionar usuarios	B1, S3	POST /usuarios	UsuariosController.crear	verificar:saas (límites del plan).
UC24 Gestionar sucursales, temporadas y colecciones	B1, S3	POST /sucursales	MetaController.crearSucursal	verificar:saas.
UC25 Ajustar inventario y ver el kardex	B1, O6	POST /inventario/ajuste	InventarioService.ajustar	Prueba manual de ajuste y rechazo.
UC26 Moderar opiniones y ver auditoría	B1	PATCH /admin/opiniones/:id/moderacion	AdminController.moderarOpinion	Prueba manual.
UC27 Simular un pago	B1, O8	POST /pagos/:id/simular	PagosController.simular	Prueba manual de aprobación y rechazo.
UC28 Usar el probador 3D	B1	GET /catalogo/variantes/:id/overlay	CatalogoService.overlay	Prueba manual en web y móvil.
UC29 Hablar con el bot	B1	POST /ia/bot	IaService.bot	Prueba manual con y sin modelo.
UC30 Aplicar precio mayorista	B1	POST /ventas	VentasService.cobrar	Prueba manual por encima y por debajo del mínimo.
UC31 Transferir stock entre almacenes	B1, O7	POST /inventario/transferencias	InventarioService.transferir	Prueba manual; la suma total no cambia.
UC32 Despachar un pedido	B1	POST /pedidos/:id/despachar	PedidosController.despachar	Prueba manual.
UC33 Devolver una venta	B1, R2	POST /devoluciones	DevolucionesController.crear	verificar:reportes (devolución de venta al crédito).
UC34 Emitir comprobante simulado	B1	POST /ventas/:id/enviar-factura	NotificacionesService.enviarFacturaVenta	Prueba manual.
UC35 Instalar la tienda como PWA	O1, O10	Cliente	Lógica del cliente (web y móvil)	verificar:offline y prueba en el navegador sin red.
UC36 Registrar cuenta en la plataforma	S2, S3, S4	POST /plataforma/registro	PlataformaService.registrar	verificar:saas (registro).

Caso	Requisitos	Ruta	Realizado por	Prueba
UC37 Verificar correo	S6	`POST /plataforma/verificar`	PlataformaService.verificar	verificar:saas (sin verificar no suscribe).
UC38 Suscribirse y crear tienda	S3, S4, S5, O6	`POST /plataforma/suscribir`	PlataformaService.suscribir	verificar:saas (pago rechazado, aprobado e idempotente).
UC39 Renovar o cambiar de plan	S3, S4	`POST /plataforma/tiendas/:id/renovar`	PlataformaService.renovar	verificar:saas (renovación).
UC40 Cambiar de tienda	S7	`POST /auth/cambiar-tienda`	AuthService.cambiarTienda	verificar:saas (una cuenta, varias tiendas).
UC41 Ver métricas de suscripciones	S9	`GET /plataforma/admin/metricas`	PlataformaService.metricas	verificar:saas (métricas).
UC42 Vender sin conexión	O1, O2, O4, O8, O9	Cliente	Lógica del cliente (web y móvil)	verificar:offline y recorrido en navegador y Expo sin red.
UC43 Registrar clienta o producto sin conexión	O1, O2, O4	Cliente	Lógica del cliente (web y móvil)	verificar:offline (clienta y producto offline).
UC44 Sincronizar operaciones pendientes	O3, O4, O6, O7	`POST /sync/lote`	SyncService.lote	verificar:offline (lote, duplicado, vencida y rollback).
UC45 Resolver conflicto de stock	O5	`POST /sync/operaciones/:id/descartar`	SyncService.descartar	verificar:offline (conflicto de la última unidad).
UC46 Configurar ventana offline	O8, O9	`PATCH /sync/politica`	SyncService.fijarVentana	verificar:offline (operación vencida).
UC47 Activar notificaciones push	O10	`POST /push/suscribir`	PushService.suscribir	Prueba en el navegador con /push/probar.
UC48 Generar reporte por texto o voz	R1, R2, S1	`POST /reportes/inteligente`	ReportesInteligentesService.generar	verificar:reportes (intérprete, compilador, aislamiento e inyección).
UC49 Ajustar y exportar reporte	R2, R3	`POST /reportes/inteligente/exportar`	ReportesInteligentesService.exportar	verificar:reportes (Excel válido y PDF paginado).
UC50 Vender al crédito	R2, O8	`POST /ventas`	VentasService.cobrar	verificar:reportes (venta al crédito y caja sin cambios).
UC51 Registrar abono de crédito	R2, O6	`POST /creditos/:id/abonos`	CreditosController.abonar	verificar:reportes (abonos idempotentes y concurrentes).
UC52 Ver	A1, A2,	`GET /analitica/panel`	AnaliticaService.panel	verificar-analitica-

Caso	Requisitos	Ruta	Realizado por	Prueba
dashboard predictivo	A3			http.ts: 401 sin sesión, 403 por rol o plan, 200 con plan Cadena y aislamiento por tienda.
UC53 Ver segmentación de clientas	A1, A2, A3	`GET /analitica/panel`	AnaliticaService.panel	analitica.service.spec.ts y verificar-analitica-http.ts: grupos, silueta sintética y tienda sin ventas.
UC54 Pagar con pasarela internacional	P1, P2, P3, P4	`POST /pagos/:id/simular`	PagosController.simular	pagos-pais.spec.ts: métodos por país; reintento con la misma clave devuelve el mismo pago.

9.4.6 Revisión de código y sincronización del modelo

Al cerrar cada lote, el modelo se regeneró desde `casos.py` y se comparó con el código: rutas, nombres de métodos y modelos de Prisma. Las diferencias encontradas se corrigieron en el modelo (por ejemplo, la operación del catálogo es `listarProductos` y el ingreso de mercadería es `ingresoProveedor`), y los módulos sin servicio quedaron registrados como deuda técnica. Como el dominio se lee de `schema.prisma` cada vez, una migración nueva aparece en el modelo sin trabajo manual.

10. Adopción y capacitación

En el primer ciclo la capacitación era para una sola cadena: un entorno sembrado, un recorrido de quince minutos y una guía por rol. En el segundo ciclo la persona que adopta EnsayAR ya no es una cajera de Vesta sino la dueña de cualquier tienda, que llega sola a la página de planes y debe poder operar sin que nadie la llame. Por eso la adopción se diseñó en tres capas:

- **Autoservicio.** El propio flujo de alta enseña el producto: elegir plan, verificar el correo, pagar y entrar a una tienda ya creada, con sucursal, almacén y su usuario administrador.
- **Recorridos guiados por rol.** Cada rol ve solo sus pantallas y tiene una guía corta, la misma del primer ciclo más las pantallas nuevas.
- **Simulacros.** El corte de red y el reporte por voz se practican en el entorno de demostración antes de usarlos con clientas reales.

10.1 Entorno de demostración

Las cuentas siguientes sirven solo en el laboratorio. La clave de todas es `diego`. Las de la tienda Vesta entran con la tienda Vesta seleccionada; las dueñas de las tiendas de demostración entran desde la página de la plataforma.

Rol	Correo	Qué muestra
Dueño de la plataforma	superadmin@ensayar.bo	Métricas de suscripciones de todas las tiendas
Administración (varias tiendas)	admin@ensayar.bo	Vesta, Aurora Boutique y Andes Moda con un solo login
Caja	cajero@ensayar.bo	Punto de venta, créditos, sincronización
Encargado de sucursal	encargado@ensayar.bo	Inventario, reservas y revisión de conflictos
Proveedor	proveedor@ensayar.bo	Ingresos de mercadería
Clienta	clienta@ensayar.bo	Catálogo, probador, carrito, reservas
Dueña de una tienda demo	duena@demo-nueva.demo (y las demás `demo-*`)	Su propia tienda y su suscripción

La API escucha en el puerto 3000 y la web en el 5173. La semilla crea la tienda Vesta con sus usuarios; el script `demo:saas` crea nueve tiendas de distintos países con historia de suscripciones, y `datos:reportes` agrega las clientas y ventas al crédito que usan los reportes. Los comandos están en los anexos.

10.2 Alta de una tienda nueva (autoservicio)

1. Entrar a `/planes`, comparar los tres planes (Inicial, Crecimiento, Cadena) y elegir el período: mensual, trimestral o anual.
2. Registrar la cuenta con nombre, correo y clave. El sistema envía un enlace de verificación; en el laboratorio, sin servidor de correo, el enlace aparece en pantalla y en el registro de la API.
3. Abrir el enlace. La cuenta queda verificada; sin verificar no se puede pagar.
4. Completar los datos de la tienda (nombre, país, moneda) y pagar con la tarjeta de prueba `4242 4242 4242 4242`. Las tarjetas `4000 0000 0000 0002`, `…9995` y `…0069` simulan rechazo, fondos insuficientes y tarjeta vencida, y sirven para mostrar que un pago rechazado no crea la tienda.
5. Al aprobarse el pago, la plataforma crea la tienda y deja a la dueña dentro, como administradora. Si pulsa pagar dos veces, el segundo intento devuelve el mismo resultado sin cobrar de nuevo.

Lo que se enseña en este paso es que la tienda es suya: no ve productos, clientas ni ventas de ninguna otra tienda, aunque comparta la aplicación.

10.3 Una cuenta, varias tiendas

La misma persona puede administrar varias tiendas. Con `admin@ensayar.bo` se entra a Vesta; en la barra lateral aparece el selector de tiendas con Aurora Boutique y Andes Moda. Al

elegir otra, el sistema emite un token nuevo para esa tienda y la pantalla muestra sus datos. Desde `/plataforma` la dueña ve todas sus tiendas y desde `/plataforma/nueva` agrega otra con el mismo flujo de pago. Cuando una suscripción vence, la tienda deja de atender y muestra el aviso de suscripción vencida hasta que se renueve.

10.4 Guía del dueño de la plataforma

Entra en `/plataforma/login` con `superadmin@ensayar.bo` y abre `/plataforma/admin`. Ve las tiendas registradas y las vigentes, las bajas de los últimos treinta días, el ingreso recurrente mensual (MRR) y anual (ARR), el ingreso promedio por tienda (ARPU), la tasa de abandono mensual (churn), la distribución por plan y por período, la serie mensual de ingresos y altas, las renovaciones próximas y la proyección de seis meses. Debe saber explicar que el MRR normaliza los planes trimestrales y anuales a su equivalente mensual, y que la proyección aplica una regla visible en pantalla: el MRR del mes siguiente es el MRR actual por uno menos el churn, más las altas promedio de los últimos tres meses por el ARPU. Es una tendencia sobre la historia, no una promesa.

10.5 Guía de caja (con y sin conexión)

Lo del primer ciclo sigue igual: sin turno abierto no se cobra; con turno abierto se arma el ticket, se elige el medio de pago y se confirma. Lo nuevo es la operación sin red:

- Cuando cae la conexión aparece el aviso "sin conexión". La venta en **efectivo**, el registro de una clienta y el alta de un producto siguen funcionando; se guardan en el dispositivo con un identificador propio.
- El **QR**, la **tarjeta** y la **venta al crédito** se bloquean sin red, porque necesitan una confirmación externa o el saldo actualizado de la clienta.

- Al volver la red, la cola se envía sola. En `/sincronizacion` se ven las operaciones pendientes, aplicadas y en conflicto.
- Si una venta quedó **en conflicto** porque otra caja vendió la última unidad, el cajero elige vender lo que queda o descartar la línea. Nunca se vuelve a cobrar a mano la misma operación: el sistema reconoce los reintentos por su clave.

Los **abonos** de ventas al crédito se registran en `/creditos`, con el saldo de cada clienta.

10.6 Guía del encargado

Ve el inventario, las ventas y las reservas de su sucursal. En `/sincronizacion` revisa los conflictos de todas las cajas de la tienda y puede resolverlos. Puede completar o cancelar reservas como en el primer ciclo.

10.7 Guía de administración de la tienda

Mantiene catálogo, usuarios y sucursales dentro de los límites del plan: el plan Inicial permite tres usuarios, una sucursal y doscientos productos, así que la segunda sucursal se rechaza con un aviso para mejorar el plan. En `/sincronizacion` fija la **ventana offline** (de 1 a 30 días): cuánto tiempo puede operar un dispositivo sin conectarse antes de exigir sincronizar.

En `/reportes` pide el reporte en lenguaje natural, por texto o por voz con el micrófono. Ejemplos para practicar:

- "Ventas por mes de los últimos 6 meses."
- "Top 10 productos más vendidos por cantidad."
- "Clientes con nombre, dirección y email que me deben de 500 a 3000 bolivianos por ventas al crédito."

La pantalla muestra cómo entendió el pedido (fuente, columnas, filtros, orden), la tabla y los totales, y permite descargar PDF o Excel. Si el pedido no se entiende o pide algo prohibido, el sistema lo dice y no ejecuta nada. Se enseña que la IA solo propone la estructura del reporte: el servidor la valida y arma la consulta.

10.8 Guías de clienta y proveedor

No cambian respecto del primer ciclo: la clienta recorre el catálogo, usa el probador 2D o 3D, guarda favoritos, reserva y compra; el proveedor registra ingresos de mercadería. La única diferencia es que ahora lo hacen dentro de una tienda concreta.

10.9 Simulacros

Simulacro de corte de red (diez minutos). Dos personas abren el punto de venta en dos navegadores. En las herramientas del navegador se marca "sin conexión" en ambos. Cada una vende en efectivo la última unidad de la misma variante. Se reconecta primero una y luego la otra. Se observa que la primera venta se aplica, la segunda queda en conflicto y el stock no baja de cero. Se resuelve el conflicto desde la pantalla de sincronización.

Simulacro de reporte (cinco minutos). La administradora dicta un pedido de reporte, revisa la interpretación, corrige una palabra si el dictado se equivocó y descarga el Excel. Se abre la hoja "Cómo se generó" para ver los criterios aplicados.

10.10 Guion de capacitación

Bloque	Minutos	Contenido
Alta de la tienda	10	Planes, registro, verificación, pago de prueba y entrada a la tienda nueva
Roles	5	Qué ve cada rol; una cuenta, varias tiendas
Caja	10	Venta en línea y simulacro de corte de red
Reportes	10	Tres pedidos de ejemplo, voz, PDF

Bloque	Minutos	Contenido
Plataforma	5	y Excel Métricas de suscripción, para el dueño

10.11 Mensajes clave y alcance comunicado a los usuarios

La capacitación transmite a los usuarios cinco garantías del sistema: cada tienda accede únicamente a sus datos, la caja continúa vendiendo en efectivo sin conexión, la sincronización no duplica ventas, los reportes generados por lenguaje natural nunca ejecutan texto escrito por el modelo de IA y los cobros de suscripción y de ventas operan en modo de prueba.

Con el mismo cuidado se comunica lo que el sistema no hace: no realiza cobros bancarios reales, no acepta QR ni tarjeta sin conexión porque ambos requieren confirmación en línea, y los módulos de analítica predictiva y de pasarela regional se presentan según su estado de implementación descrito en los capítulos 5 y 8.

11. Conclusiones

Sobre el modelo SaaS. EnsayAR pasó de ser el sistema de una cadena a una plataforma donde cualquier tienda se suscribe y opera sin intervención humana. El costo de esa escala no estuvo en las pantallas nuevas sino en una decisión transversal: cada fila de negocio pertenece a una tienda. Con 36 modelos marcados con `tienda_id`, una extensión de Prisma que filtra e inyecta la tienda y que se niega a operar sin contexto, y SQL crudo dentro de transacciones que fijan la tienda en la sesión, la verificación automática confirmó que ninguna tienda lee ni escribe datos de otra. Separar la identidad global (Cuenta) de la membresía por tienda (Usuario) permitió que una persona administre varias tiendas con un solo login.

Sobre la operación sin conexión. La regla que sostiene todo el diseño es que el servidor sigue siendo la fuente de verdad del stock, aunque el dispositivo venda sin red. Los identificadores generados en el cliente, la clave de idempotencia por operación, una transacción por operación y la actualización condicional del disponible hicieron que reintentar sea seguro y que la última unidad no se venda dos veces: el conflicto se detecta al sincronizar, queda visible y lo resuelve una persona. Decidir qué operaciones se permiten sin red (efectivo sí; QR, tarjeta y crédito no) fue tan importante como la mecánica de la cola.

Sobre los reportes con IA. El modelo de lenguaje no escribe SQL. Traduce el pedido a una especificación JSON contra un catálogo semántico cerrado; el servidor la valida, la compila a una única sentencia SELECT parametrizada y la ejecuta en una transacción que fija la tienda en la sesión de PostgreSQL. Así el sistema responde preguntas que nadie previó, que cruzan varias tablas con filtros, agrupaciones y orden, sin abrir una vía de inyección ni de fuga entre tiendas. El intérprete por reglas, que atiende cuando la IA no responde, demostró que la arquitectura no depende de un proveedor.

Sobre la metodología. ICONIX resultó adecuada para un incremento sobre un sistema existente: el modelo de dominio salió del esquema real, el análisis de robustez obligó a revisar cada caso de uso contra el código y encontró hallazgos concretos antes de diseñar, y la matriz de trazabilidad llega del requisito al método y a la prueba. Generar el modelo de Enterprise Architect desde una sola fuente (`casos.py`) mantuvo consistentes los 54 casos de uso con sus diagramas de robustez y de secuencia, 160 diagramas en total, entre ellos el despliegue, los componentes y las máquinas de estado de la sincronización en notación UML 2.5.

Sobre el incremento. Lo construido en el primer ciclo se conservó y ahora opera dentro de cada tienda. Los puntos de analítica con aprendizaje automático y de pagos regionales, a cargo del segundo integrante, se apoyan en piezas ya preparadas: la clave de idempotencia y el puerto de pasarela de la suscripción, y el aislamiento por tienda que también deberán respetar los modelos y tableros.

12. Recomendaciones

Aislamiento en la base de datos. El aislamiento actual vive en la aplicación. Se recomienda agregar políticas de seguridad por fila (RLS) de PostgreSQL sobre `current_setting('app.tienda_id')` como segunda barrera, para que un error de código no pueda filtrar datos aunque omita la extensión de Prisma.

Pasarela real. Reemplazar la pasarela simulada de la suscripción por la pasarela elegida en el capítulo 5, en modo de pruebas primero, verificando la firma de los webhooks y conservando la clave de idempotencia ya existente.

Correo transaccional. En producción, configurar un servicio de correo para la verificación de cuentas y los avisos de vencimiento; hoy el envío es simulado cuando no hay servidor de correo.

Despliegue. Llevar PostgreSQL de la instancia EC2 a un servicio administrado como Amazon RDS, con respaldos automáticos y sin competir por memoria con la API; registrar un dominio propio en lugar del subdominio de DuckDNS; y renovar la clave de Azure OpenAI del entorno de demostración.

Móvil. Probar la cola SQLite y la sincronización en dispositivos Android físicos con cortes de red reales, no solo en Expo web, y medir el tamaño de la base local con la ventana offline máxima de 30 días.

Reportes. Registrar los pedidos que el intérprete no entiende para ampliar sinónimos y fuentes, y agregar gráficos al resultado y al PDF. Limitar el uso del intérprete con IA por plan, porque cada pedido tiene costo.

Sincronización. Si el volumen crece, separar la cola de sincronización y los reportes pesados en trabajadores en segundo plano, manteniendo el monolito modular como base.

Analítica. Reentrenar los modelos con datos reales de cada tienda cuando haya historia suficiente, y mostrar siempre la precisión del modelo junto a la predicción.

Pruebas. Convertir los scripts `verificar:*` en pruebas de integración ejecutadas en cada cambio, y agregar pruebas de extremo a extremo del alta de tienda y del simulacro de corte de red.

13. Referencias

Abadi, D. J. (2012). Consistency tradeoffs in modern distributed database system design: CAP is only part of the story. *Computer, 45*(2), 37–42. https://doi.org/10.1109/MC.2012.33

Amazon Web Services. (s.f.-a). *The bridge model*. En *SaaS Lens: AWS Well-Architected Framework*. https://docs.aws.amazon.com/wellarchitected/latest/saas-lens/bridge-model.html

Amazon Web Services. (s.f.-b). *Control plane vs. application plane*. En *SaaS architecture fundamentals*. https://docs.aws.amazon.com/whitepapers/latest/saas-architecture-fundamentals/control-plane-vs.-application-plane.html

Amazon Web Services. (s.f.-c). *Hiding the details of pooled isolation*. En *SaaS tenant isolation strategies*. https://docs.aws.amazon.com/whitepapers/latest/saas-tenant-isolation-strategies/hiding-the-details-of-pooled-isolation.html

Amazon Web Services. (s.f.-d). *Pool isolation*. En *SaaS Lens: AWS Well-Architected Framework*. https://docs.aws.amazon.com/wellarchitected/latest/saas-lens/pool-isolation.html

Amazon Web Services. (s.f.-e). *Silo isolation*. En *SaaS Lens: AWS Well-Architected Framework*. https://docs.aws.amazon.com/wellarchitected/latest/saas-lens/silo-isolation.html

Amazon Web Services. (s.f.-f). *Software and architecture patterns*. En *SaaS Lens: AWS Well-Architected Framework*. https://docs.aws.amazon.com/wellarchitected/latest/saas-lens/software-and-architecture-patterns.html

Amazon Web Services. (s.f.-g). *Tenant onboarding best practices in SaaS with the AWS Well-Architected SaaS Lens*. AWS Partner Network Blog.

https://aws.amazon.com/blogs/apn/tenant-onboarding-best-practices-in-saas-with-the-aws-well-architected-saas-lens/

Androutsopoulos, I., Ritchie, G. D., y Thanisch, P. (1995). Natural language interfaces to databases: An introduction. *Natural Language Engineering, 1*(1), 29–81. https://arxiv.org/abs/cmp-lg/9503016

Archibald, J. (s.f.-a). *The offline cookbook*. web.dev. https://web.dev/articles/offline-cookbook

Archibald, J. (s.f.-b). *The service worker lifecycle*. web.dev. https://web.dev/articles/service-worker-lifecycle

Banco Central do Brasil. (s.f.). *Pix*. https://www.bcb.gov.br/estabilidadefinanceira/pix

Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5–32. https://doi.org/10.1023/A:1010933404324

Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., . . . Amodei, D. (2020). *Language models are few-shot learners*. arXiv. https://doi.org/10.48550/arXiv.2005.14165

Chong, F., y Carraro, G. (2006). *Architecture strategies for catching the long tail*. Microsoft Corporation.

Cube Dev. (s.f.). *Data modeling: Getting started*. https://cube.dev/docs/product/data-modeling/overview

Davis, K., Peabody, B., y Leach, P. (2024). *Universally unique identifiers (UUIDs)* (RFC 9562). Internet Engineering Task Force. https://doi.org/10.17487/RFC9562

dbt Labs. (s.f.). *dbt Semantic Layer*. https://docs.getdbt.com/docs/use-dbt-semantic-layer/dbt-sl

dLocal. (s.f.). *Payment methods by country*. https://docs.dlocal.com/

Eidson, B., y Simmons, J. (2023, 16 de febrero). Web Push for web apps on iOS and iPadOS. *WebKit Blog*. https://webkit.org/blog/13878/web-push-for-web-apps-on-ios-and-ipados/

Fader, P. S., Hardie, B. G. S., y Lee, K. L. (2005). RFM and CLV: Using iso-value curves for customer base analysis. *Journal of Marketing Research, 42*(4), 415–430. https://doi.org/10.1509/jmkr.2005.42.4.415

Few, S. (2006). *Information dashboard design: The effective visual communication of data*. O'Reilly Media.

Google Cloud. (s.f.). *Introduction to LookML*. https://cloud.google.com/looker/docs/what-is-lookml

Hastie, T., Tibshirani, R., y Friedman, J. (2009). *The elements of statistical learning: Data mining, inference, and prediction* (2.ª ed.). Springer. https://doi.org/10.1007/978-0-387-84858-7

Hong, Z., Yuan, Z., Zhang, Q., Chen, H., Dong, J., Huang, F., y Huang, X. (2024). *Next-generation database interfaces: A survey of LLM-based text-to-SQL*. arXiv. https://doi.org/10.48550/arXiv.2406.08426

Hyndman, R. J., y Athanasopoulos, G. (2021). *Forecasting: Principles and practice* (3.ª ed.). OTexts. https://otexts.com/fpp3/

Jacobson, I., Booch, G., y Rumbaugh, J. (1999). *The unified software development process*. Addison-Wesley.

Jacobson, I., Christerson, M., Jonsson, P., y Övergaard, G. (1992). *Object-oriented software engineering: A use case driven approach*. Addison-Wesley.

Jena, J., Dalal, S., y Tishkin, D. (2026). *The Idempotency-Key HTTP header field* (Internet-Draft draft-ietf-httpapi-idempotency-key-header-08). Internet Engineering Task Force. https://datatracker.ietf.org/doc/draft-ietf-httpapi-idempotency-key-header/

Jordon, J., Szpruch, L., Houssiau, F., Bottarelli, M., Cherubin, G., Maple, C., Cohen, S. N., y Weller, A. (2022). *Synthetic data: What, why and how?* arXiv. https://doi.org/10.48550/arXiv.2205.03257

Kleppmann, M. (2017). *Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems*. O'Reilly Media.

Li, J., Hui, B., Qu, G., Yang, J., Li, B., Li, B., Wang, B., Qin, B., Geng, R., Huo, N., Zhou, X., Ma, C., Li, G., Chang, K. C. C., Huang, F., Cheng, R., y Li, Y. (2023). *Can LLM already serve as a database interface? A big bench for large-scale database grounded text-to-SQLs*. arXiv. https://doi.org/10.48550/arXiv.2305.03111

Libélula. (s.f.). *Suite de pagos y facturación electrónica*. https://libelula.bo/

Lloyd, S. P. (1982). Least squares quantization in PCM. *IEEE Transactions on Information Theory, 28*(2), 129–137. https://doi.org/10.1109/TIT.1982.1056489

MDN Web Docs. (s.f.-a). *Push API*. Mozilla. https://developer.mozilla.org/en-US/docs/Web/API/Push_API

MDN Web Docs. (s.f.-b). *Storage quotas and eviction criteria*. Mozilla. https://developer.mozilla.org/en-US/docs/Web/API/Storage_API/Storage_quotas_and_eviction_criteria

MDN Web Docs. (s.f.-c). *Web Speech API*. Mozilla. https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API

Mell, P., y Grance, T. (2011). *The NIST definition of cloud computing* (Special Publication 800-145). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-145

Mercado Pago. (2026). *Cómo integrar Mercado Pago para el Hot Sale 2026*. https://www.mercadopago.com.mx/blog/hot-sale-integra-mercado-pago

Mercado Pago. (s.f.-a). *Cobrá con el checkout de Mercado Pago en tu sitio web*. https://www.mercadopago.com.ar/herramientas-para-vender/check-out

Mercado Pago. (s.f.-b). *Cobra con la pasarela de pago de Mercado Pago*. https://www.mercadopago.cl/herramientas-para-vender/check-out

Microsoft. (s.f.-a). *Architect multitenant solutions on Azure*. Azure Architecture Center. https://learn.microsoft.com/azure/architecture/guide/multitenant/overview

Microsoft. (s.f.-b). *Architectural approaches for identity in multitenant solutions*. Azure Architecture Center. https://learn.microsoft.com/en-us/azure/architecture/guide/multitenant/approaches/identity

Microsoft. (s.f.-c). *Architectural approaches for storage and data in multitenant solutions*. Azure Architecture Center. https://learn.microsoft.com/en-us/azure/architecture/guide/multitenant/approaches/storage-data

Microsoft. (s.f.-d). *How to use JSON mode with Azure OpenAI in Microsoft Foundry Models*. Microsoft Learn. https://learn.microsoft.com/en-us/azure/ai-foundry/openai/how-to/json-mode

Microsoft. (s.f.-e). *How to use structured outputs with Azure OpenAI*. Microsoft Learn. https://learn.microsoft.com/en-us/azure/ai-foundry/openai/how-to/structured-outputs

Microsoft. (s.f.-f). *Map requests to tenants in a multitenant solution*. Azure Architecture Center. https://learn.microsoft.com/en-us/azure/architecture/guide/multitenant/considerations/map-requests

Microsoft. (s.f.-g). *Pricing models for a multitenant solution*. Azure Architecture Center. https://learn.microsoft.com/en-us/azure/architecture/guide/multitenant/considerations/pricing-models

Microsoft. (s.f.-h). *Tenancy models for a multitenant solution*. Azure Architecture Center. https://learn.microsoft.com/en-us/azure/architecture/guide/multitenant/considerations/tenancy-models

Microsoft. (s.f.-i). *What is speech to text?* Microsoft Learn. https://learn.microsoft.com/en-us/azure/ai-services/speech-service/speech-to-text

NestJS. (s.f.). *Middleware*. https://docs.nestjs.com/middleware

OpenJS Foundation. (s.f.). *Asynchronous context tracking*. Node.js Documentation. https://nodejs.org/api/async_context.html

OWASP Foundation. (s.f.). *SQL injection prevention cheat sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

PayPal. (2026, 28 de octubre). *Comisiones de PayPal para vendedores*. https://www.paypal.com/bo/business/paypal-business-fees

PCI Security Standards Council. (2022). *Payment Card Industry Data Security Standard: Requirements and testing procedures* (versión 4.0). https://www.pcisecuritystandards.org/

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D.,

Brucher, M., Perrot, M., y Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

PostgreSQL Global Development Group. (s.f.-a). *Row security policies*. PostgreSQL Documentation. https://www.postgresql.org/docs/current/ddl-rowsecurity.html

PostgreSQL Global Development Group. (s.f.-b). *System administration functions*. PostgreSQL Documentation. https://www.postgresql.org/docs/current/functions-admin.html

PostgreSQL Global Development Group. (s.f.-c). *Transaction isolation*. PostgreSQL Documentation. https://www.postgresql.org/docs/current/transaction-iso.html

Prisma. (s.f.-a). *Prisma Client extension: Row level security* [Código fuente]. GitHub. https://github.com/prisma/prisma-client-extensions/tree/main/row-level-security

Prisma. (s.f.-b). *Prisma Client extensions: Query component*. Prisma Documentation. https://www.prisma.io/docs/orm/prisma-client/client-extensions/query

Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., y Sutskever, I. (2022). *Robust speech recognition via large-scale weak supervision*. arXiv. https://doi.org/10.48550/arXiv.2212.04356

Richardson, C. (s.f.-a). *Pattern: Saga*. Microservice Architecture. https://microservices.io/patterns/data/saga.html

Richardson, C. (s.f.-b). *Pattern: Transactional outbox*. Microservice Architecture. https://microservices.io/patterns/data/transactional-outbox.html

Rosenberg, D., y Scott, K. (1999). *Use case driven object modeling with UML: A practical approach*. Addison-Wesley.

Rosenberg, D., y Scott, K. (2001). *Applying use case driven object modeling with UML: An annotated e-commerce example*. Addison-Wesley.

Rosenberg, D., y Stephens, M. (2007). *Use case driven object modeling with UML: Theory and practice*. Apress. https://doi.org/10.1007/978-1-4302-0369-8

Rosenberg, D., Stephens, M., y Collins-Cope, M. (2005). *Agile development with ICONIX process: People, process, and pragmatism*. Apress. https://doi.org/10.1007/978-1-4302-0009-3

Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics, 20*, 53–65. https://doi.org/10.1016/0377-0427(87)90125-7

Russell, A. (2015, 15 de junio). Progressive web apps: Escaping tabs without losing our soul. *Infrequently Noted*. https://infrequently.org/2015/06/progressive-apps-escaping-tabs-without-losing-our-soul/

Saavedra Alberto, D. J., y Sánchez Sandy, A. É. (2026). *EnsayAR: Sistema de información web y móvil omnicanal para tienda virtual de moda femenina. Informe del primer parcial* [Informe académico no publicado]. Universidad Autónoma Gabriel René Moreno.

Skok, D. (s.f.-a). *SaaS metrics 2.0: A guide to measuring and improving what matters*. For Entrepreneurs. https://www.forentrepreneurs.com/saas-metrics-2/

Skok, D. (s.f.-b). *SaaS metrics 2.0: Detailed definitions*. For Entrepreneurs. https://www.forentrepreneurs.com/saas-metrics-2-definitions/

Sparx Systems. (s.f.-a). *ICONIX*. En *Enterprise Architect user guide* (versión 17.2). https://www.sparxsystems.com/enterprise_architect_user_guide/17.2/modeling_domains/iconix_process.html

Sparx Systems. (s.f.-b). *Model Driven Generation (MDG) technologies*. https://sparxsystems.com/resources/mdg_tech/

Stephens, M., y Rosenberg, D. (2003). *Extreme programming refactored: The case against XP*. Apress.

Stephens, M., y Rosenberg, D. (2010). *Design driven testing: Test smarter, not harder*. Apress. https://doi.org/10.1007/978-1-4302-2944-5

Stripe. (s.f.-a). *Idempotent requests*. Stripe API Reference. https://docs.stripe.com/api/idempotent_requests

Stripe. (s.f.-b). *Local payment methods pricing* [México]. https://stripe.com/en-mx/pricing/local-payment-methods

Stripe. (s.f.-c). *Pricing & fees* [Brasil]. https://stripe.com/en-br/pricing

Stripe. (s.f.-d). *Receive Stripe events in your webhook endpoint*. Stripe Documentation. https://docs.stripe.com/webhooks

Stripe. (s.f.-e). *Tarifas y comisiones* [España]. https://stripe.com/es/pricing

Thomson, M., y Beverloo, P. (2017). *Voluntary application server identification (VAPID) for Web Push* (RFC 8292). Internet Engineering Task Force. https://doi.org/10.17487/RFC8292

Yu, T., Zhang, R., Yang, K., Yasunaga, M., Wang, D., Li, Z., Ma, J., Li, I., Yao, Q., Roman, S., Zhang, Z., y Radev, D. (2018). Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. En *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing* (pp. 3911–3921). Association for Computational Linguistics. https://doi.org/10.48550/arXiv.1809.08887

14. Anexos

14.1 Repositorio

El código está en https://github.com/DSaav22/Examen-Si2-2026-2 (backend, frontend, móvil, infraestructura y documentación). Este informe está en `docs/segundo-parcial`; el del primer parcial, en `docs/parcial`.

14.2 URL de despliegue y código QR

La versión de la entrega está publicada en el dominio `examensi2.duckdns.org`. La aplicación web y la API comparten el dominio: la PWA llama a la API por la ruta relativa `/api`, de modo que no hay peticiones entre orígenes distintos. El servidor es una instancia Amazon EC2 con Linux en AWS (sección 9.3, diagrama de despliegue).

Componente	URL
Aplicación web (PWA)	https://examensi2.duckdns.org
API REST	https://examensi2.duckdns.org/api

Figura 14.1

Código QR de acceso a la aplicación web



14.3 Puesta en marcha local

Requisitos: Node.js 20 o superior, PostgreSQL 16 y las variables de entorno de `backend/.env.example` copiadas a `backend/.env`.

```
cd backend
npm install
npx prisma migrate deploy
npx prisma db seed
npm run demo:saas
npm run datos:reportes
npm run start:dev

cd frontend
npm install
npm run dev

cd movil
npm install
npx expo start
```

La API queda en el puerto 3000 y la web en el 5173. En un teléfono, la app móvil debe apuntar a la IP de la máquina donde corre la API, no a localhost.

14.4 Verificaciones automáticas

Con la API en marcha, desde la carpeta `backend`:

Comando	Qué comprueba
`npm run verificar:tenancy`	Que ninguna tienda lee ni modifica datos de otra, y que sin contexto de tienda el acceso falla
`npm run verificar:saas`	Registro, verificación de correo, pago idempotente, alta de tienda, cambio de tienda, límites del plan y métricas
`npm run verificar:offline`	Lotes de sincronización, reintentos idempotentes, conflicto de la última unidad y ventana offline
`npm run verificar:reportes`	Interpretación, validación, compilación, aislamiento por tienda, PDF y Excel
`npm test`	Pruebas unitarias (catálogo, venta concurrente, SQL seguro, ficha)

14.5 Cuentas de demostración

Clave de todas las cuentas: `diego`. Solo para el laboratorio.

Rol	Correo
Dueño de la plataforma	superadmin@ensayar.bo
Administración (Vesta, Aurora, Andes)	admin@ensayar.bo
Caja	cajero@ensayar.bo
Encargado de sucursal	encargado@ensayar.bo
Proveedor	proveedor@ensayar.bo
Clienta	clienta@ensayar.bo
Dueñas de las tiendas demo	duena@aurora.demo, duena@andes.demo, duena@nueva.demo y las demás `duena@<tienda>.demo`

Tarjeta de prueba aprobada: `4242 4242 4242 4242`. Rechazos simulados: `4000 0000 0000 0002` (rechazada), `4000 0000 0000 9995` (fondos insuficientes), `4000 0000 0000 0069` (vencida).

14.6 Regenerar el modelo de Enterprise Architect

El modelo ICONIX se genera por automatización COM desde `docs/segundo-parcial/iconix/casos.py`, que es la única fuente de requisitos, casos de uso, robustez y secuencias.

1. Instalar Python 3 y el paquete `pywin32`.
2. Habilitar el motor JET 4 de Enterprise Architect: en el registro, `HKCU\Software\Sparx Systems\EA400\EA\OPTIONS`, valor DWORD `JET4` igual a 1. Sin ese valor EA abre un aviso modal y la automatización queda detenida.
3. Cerrar Enterprise Architect si tiene abierto el archivo del modelo.
4. Desde `docs/segundo-parcial/iconix` ejecutar `python generar_ea.py`. Reconstruye el paquete "ICONIX segundo parcial" dentro de `diagramas/EnsayAR-ICONIX.eapx`, sin tocar el modelo del primer parcial, y exporta las imágenes a `imagenes/ea`. Con `--solo UC07,UC44` regenera solo esos casos; con `--sin-exportar` omite las imágenes.
5. Ejecutar `python generar_md.py` para regenerar los capítulos 9.1 a 9.4 con las figuras numeradas.

14.7 Generar este informe

Desde `docs/segundo-parcial` ejecutar `python _build_docx.py`. Requiere `python-docx`. Produce `EnsayAR-Informe-Segundo-Parcial.docx` y, si Microsoft Word está instalado, también el PDF.